

Annex de codi de BEMS-RL Studio

Arquitectura, diagrames UML i codi funcional complet

Guillem Torm

© 2025, Guillem Torm

Índex

1	Objectiu i abast	5
1.1	Criteri d'inclusió	5
1.2	Inventari resumit	5
2	Guia ràpida de fitxers	6
2.1	Entrypoints i navegació	6
2.2	Pàgines Streamlit	6
2.3	Components reutilitzables	7
2.4	Backend funcional	7
2.5	Grafics i mètriques	9
2.6	Eines de manteniment	10
3	Guia de classes	11
4	Visió general del codi	12
4.1	Arquitectura d'execució	12
4.2	Topologia Docker i documentació	12
4.3	Propietat de dades i artefactes	12
4.4	Límit d'importació i autodoc	13
5	Fluxos principals	13
5.1	Creació d'entorns	13
5.2	Cicle d'entrenament	13
5.3	Resultats i selecció de gràfics	14
5.4	Control en viu	14
6	Dependències entre fitxers	14
6.1	Creació d'entorns	14
6.2	Entrenament d'agents	15
6.3	Resultats, dashboard i informes	15
6.4	Simulació, avaluació i control en viu	15
6.5	Visors i gestió de fitxers	15
7	Classes i dataclasses principals	16

A	Annex de codi funcional	16
B	Codi: Entrypoints i navegacio	17
B.1	<code>__init__.py</code>	17
B.2	<code>Inici.py</code>	17
B.3	<code>sidebar_nav.py</code>	20
C	Codi: Pagine Streamlit	23
C.1	<code>pages/__init__.py</code>	24
C.2	<code>pages/Afegir_Entorn.py</code>	24
C.3	<code>pages/Ajuda.py</code>	33
C.4	<code>pages/Avaluar_Agent.py</code>	36
C.5	<code>pages/Entrenar_Agent.py</code>	43
C.6	<code>pages/Gestionar_Arxius.py</code>	47
C.7	<code>pages/Interaccionar_amb_l'Agent.py</code>	51
C.8	<code>pages/Mostrar_Entorn.py</code>	59
C.9	<code>pages/Resultats.py</code>	65
C.10	<code>pages/Simular_Entorn.py</code>	67
C.11	<code>pages/Visor_Climes_EPW.py</code>	72
D	Codi: Components reutilitzables	80
D.1	<code>page_components/__init__.py</code>	80
D.2	<code>page_components/resultats_dashboard.py</code>	80
D.3	<code>page_components/training_agent.py</code>	89
D.4	<code>page_components/training_page.py</code>	91
D.5	<code>page_components/training_rewards.py</code>	91
D.6	<code>page_components/training_shared.py</code>	96
D.7	<code>page_components/training_wrappers.py</code>	97
E	Codi: Backend funcional	103
E.1	<code>backend/__init__.py</code>	103
E.2	<code>backend/afegir_entorn_analysis.py</code>	103
E.3	<code>backend/afegir_entorn_backend.py</code>	111
E.4	<code>backend/afegir_entorn_config.py</code>	113
E.5	<code>backend/afegir_entorn_constants.py</code>	118
E.6	<code>backend/afegir_entorn_conversion.py</code>	119
E.7	<code>backend/afegir_entorn_types.py</code>	121
E.8	<code>backend/afegir_entorn_utils.py</code>	122
E.9	<code>backend/avaluar_agent_backend.py</code>	123
E.10	<code>backend/entrenar_agent_artifacts.py</code>	128
E.11	<code>backend/entrenar_agent_backend.py</code>	131
E.12	<code>backend/entrenar_agent_charts.py</code>	132
E.13	<code>backend/entrenar_agent_constants.py</code>	134
E.14	<code>backend/entrenar_agent_env.py</code>	136

E.15 backend/entrenar_agent_rewards.py	139
E.16 backend/entrenar_agent_runtime.py	146
E.17 backend/entrenar_agent_session.py	148
E.18 backend/entrenar_agent_utils.py	150
E.19 backend/entrenar_agent_wrappers.py	150
E.20 backend/epw_figures.py	154
E.21 backend/gestionar_arxius_backend.py	160
E.22 backend/interaccionar_agent_backend.py	162
E.23 backend/mapa_backend.py	169
E.24 backend/mostrar_entorn_backend.py	170
E.25 backend/paths.py	173
E.26 backend/resultats_backend.py	174
E.27 backend/resultats_report_backend.py	184
E.28 backend/sb3_utils.py	191
E.29 backend/simular_entorn_backend.py	195
E.30 backend/training_scripts.py	199
E.31 backend/viewer_3d_backend.py	202
E.32 backend/visor_epw_backend.py	207
E.33 backend/weather_profiles.py	215
F Codi: Grafics i metriques	219
F.1 backend/grafics/__init__.py	219
F.2 backend/grafics/battery.py	219
F.3 backend/grafics/column_utils.py	225
F.4 backend/grafics/comfort.py	227
F.5 backend/grafics/comfort_scope.py	232
F.6 backend/grafics/control.py	232
F.7 backend/grafics/data_loader.py	236
F.8 backend/grafics/energy_price.py	238
F.9 backend/grafics/energy_units.py	240
F.10 backend/grafics/episode.py	241
F.11 backend/grafics/figures_common.py	242
F.12 backend/grafics/figures_zones.py	243
F.13 backend/grafics/heatmaps.py	245
F.14 backend/grafics/hvac.py	248
F.15 backend/grafics/kpis.py	253
F.16 backend/grafics/metrics.py	255
F.17 backend/grafics/observation_columns.py	257
F.18 backend/grafics/style.py	260
F.19 backend/grafics/thermal.py	262
F.20 backend/grafics/time_utils.py	266
G Codi: Eines de manteniment	269

G.1	<code>test_eppy.py</code>	269
G.2	<code>test_sql.py</code>	270

1 Objectiu i abast

Aquest annex acompanya la documentació professional de BEMS-RL Studio. El seu objectiu és donar una visió tècnica suficient per entendre com està estructurat el projecte i, després, conservar una còpia compilable de tot el codi Python funcional de l'aplicació.

L'annex inclou 77 fitxers Python i 22,763 línies de codi. Abans de les llistes completes s'hi incorporen els diagrames principals de la documentació Sphinx, adaptats a LaTeX/TikZ, i un UML conceptual dels tipus de dades que vertebren els workflows.

El codi reproduït en aquest annex correspon exclusivament a BEMS-RL Studio. No s'hi copia ni s'hi reproduïx codi font del paquet Sinergym: les aparicions del nom Sinergym dins del document indiquen integracions, importacions, artefactes, rutes o dependències utilitzades per l'aplicació.

1.1 Criteri d'inclusió

- S'inclou el codi funcional de pàgines, components, backend, generació de gràfics, informes, eines de manteniment i punts d'entrada.
- No s'inclouen els mòduls d'estil CSS pur: `page_styles/*.py` i `ui_theme.py`. El seu contingut és visual i ja queda cobert per la documentació d'interfície.
- El codi queda incrustat en el fitxer LaTeX generat. Les icones Unicode del codi es mostren com a escapes per evitar glifs buits en el PDF, però els fitxers originals no es modifiquen.

1.2 Inventari resumit

Grup	Fitxers	Línies
Entrypoints i navegació	3	604
Pàgines Streamlit	11	5,064
Components reutilitzables	7	2,031
Backend funcional	33	10,372
Gràfics i mètriques	20	4,406
Eines de manteniment	3	286

2 Guia ràpida de fitxers

Aquesta taula resumeix cada fitxer inclòs a l'annex: què fa, quina part del projecte afecta i com treballa per dins. Les descripcions surten del docstring del mòdul i d'una lectura estàtica de funcions, classes i imports.

2.1 Entrypoints i navegació

Fitxer	Què fa	A què afecta	Com funciona
<code>__init__.py</code>	Metadades de l'aplicació per a l'espai de treball Streamlit de BEMS-RL Studio.	Entrypoints i navegació.	agrupa constants o inicialització de mòdul.
<code>Inici.py</code>	Pàgina inicial de BEMS-RL STUDIO.	Entrypoints i navegació.	organitza funcions com <code>inject_home_styles</code> , <code>get_energyplus_logo_data_uri</code> , <code>detect_energyplus_status...</code> ; connecta amb <code>sidebar_nav</code> .
<code>sidebar_nav.py</code>	Shared sidebar navigation for BEMS-RL STUDIO.	Entrypoints i navegació.	organitza funcions com <code>configure_studio_page</code> , <code>render_studio_sidebar</code> .

2.2 Pàgines Streamlit

Fitxer	Què fa	A què afecta	Com funciona
<code>pages/__init__.py</code>	Streamlit Marcador de paquet de pàgina per a BEMS-RL Studio.	Pàgines Streamlit.	agrupa constants o inicialització de mòdul.
<code>pages/Afegir_Entorn.py</code>	Sinergym Studio: vista per afegir entorns.	Creació i registre d'entorns.	organitza funcions com <code>render_add_environment_hero</code> , <code>render_add_environment_section</code> , <code>render_add_environment_subsection...</code> ; connecta amb <code>backend</code> , <code>sidebar_nav</code> .
<code>pages/Ajuda.py</code>	Mòdul funcional <code>pages/Ajuda.py</code> .	Pàgines Streamlit.	organitza funcions com <code>inject_help_styles</code> , <code>render_hero</code> , <code>render_resources_card...</code> ; connecta amb <code>sidebar_nav</code> .
<code>pages/Avaluar_Agent.py</code>	Pàgina per avaluar un agent SB3 entrenat en un entorn Sinergym.	Avaluació de models SB3.	organitza funcions com <code>render_evaluation_hero</code> , <code>render_evaluation_panel</code> , <code>render_evaluation_title...</code> ; connecta amb <code>avaluar_agent_backend</code> , <code>sidebar_nav</code> .
<code>pages/Entrenar_Agent.py</code>	Pàgina d'agent de tren per a l'aplicació BEMS-RL-STUDIO.	Flux d'entrenament i estat de Streamlit.	organitza funcions com <code>train_tab</code> ; connecta amb <code>entrenar_agent_backend</code> , <code>entrenar_agent_charts</code> , <code>entrenar_agent_session...</code> .
<code>pages/Gestionar_Arxiu.py</code>	Mòdul funcional <code>pages/Gestionar_Arxiu.py</code> .	Gestió de fitxers del projecte.	organitza funcions com <code>render_hero</code> , <code>render_weather_view_selector</code> , <code>build_selection_archive...</code> ; connecta amb <code>gestionar_arxiu_backend</code> , <code>mapa_backend</code> , <code>sidebar_nav</code> .
<code>pages/Interaccionar_amb_l'Agent.py</code>	Pàgina de control en viu d'un entorn Sinergym.	Control en viu i prova manual de polítiques.	organitza funcions com <code>render_interaction_hero</code> , <code>render_interaction_section</code> , <code>init_session...</code> ; connecta amb <code>interaccionar_agent_backend</code> , <code>sidebar_nav</code> .
<code>pages/Mostrar_Entorn.py</code>	Pàgina de visualització d'entorns Sinergym.	Visualització d'entorns i geometria 3D.	organitza funcions com <code>render_environment_hero</code> , <code>render_environment_section</code> , <code>describe_environment_tags...</code> ; connecta amb <code>mapa_backend</code> , <code>mostrar_entorn_backend</code> , <code>paths...</code> .
<code>pages/Resultats.py</code>	Streamlit frontend per a la pàgina de resultats de BEMS-RL STUDIO.	Selecció de runs, dashboard i descàrregues.	organitza funcions com <code>build_download_filename</code> , <code>render_hero</code> , <code>render_results_page</code> ; connecta amb <code>resultats_backend</code> , <code>resultats_dashboard</code> , <code>sidebar_nav</code> .

Fitxer	Què fa	A què afecta	Com funciona
pages/Simular_Entorn.py	Mòdul funcional pages/Simular_Entorn.py.	Baseline sense agent i comparació posterior.	organitza funcions com render_baseline_hero, render_section_header, empty_baseline_runtime...; connecta amb simular_entorn_backend, sidebar_nav.
pages/Visor_Climes_EPW.py	Mòdul funcional pages/Visor_Climes_EPW.py.	Inspecció de fitxers EPW i clima.	Defineix EpwDashboardState; organitza funcions com ui_text, render_hero, format_number...; connecta amb backend, epw_figures, mapa_backend...

2.3 Components reutilitzables

Fitxer	Què fa	A què afecta	Com funciona
page_components/__init__.py page_components/resultats_dashboard.py	Components Streamlit reutilitzables per a pàgines BEMS-RL Studio. Component del panell en línia per a la pàgina de resultats.	Components reutilitzables. Dashboard incrustat de resultats.	agrupa constants o inicialització de mòdul. organitza funcions com render_inline_dashboard; connecta amb resultats_backend, resultats_report_backend.
page_components/training_agent.py	Controls d'hiperparàmetres de l'agent per a la pàgina d'entrenament.	Formularis compartits d'entrenament.	organitza funcions com render_agent_params_section; connecta amb entrenar_agent_session.
page_components/training_page.py	Façana component públic per a la pàgina de entrenament.	Formularis compartits d'entrenament.	connecta amb training_agent, training_rewards, training_shared...
page_components/training_rewards.py	Controls de configuració de recompenses per a la pàgina de entrenament.	Formularis compartits d'entrenament.	organitza funcions com render_reward_kwargs_section; connecta amb entrenar_agent_backend, entrenar_agent_session.
page_components/training_shared.py	Constants de pàgina d'entrenament compartides i components de capçalera reutilitzables.	Formularis compartits d'entrenament.	organitza funcions com inject_training_styles, render_training_hero, render_training_section...; connecta amb entrenar_agent_backend, entrenar_agent_session.
page_components/training_wrappers.py	Controls de wrappers i registre per a la pàgina d'entrenament.	Formularis compartits d'entrenament.	organitza funcions com render_observation_wrappers, render_action_wrappers, render_energy_logging_wrappers...; connecta amb entrenar_agent_backend, entrenar_agent_session.

2.4 Backend funcional

Fitxer	Què fa	A què afecta	Com funciona
backend/__init__.py backend/afegir_entorn_analysis.py	Lògica d'aplicació de BEMS-RL Studio segura per importar. Lectura i anàlisi d'edificis per preparar la configuració dels entorns.	Backend funcional. Anàlisi, conversió i YAML d'entorns.	agrupa constants o inicialització de mòdul. Defineix ProbeReward; organitza funcions com load_building_json, extract_schedule_type_index, resolve_schedule_name...; connecta amb afegir_entorn_constants, afegir_entorn_types, afegir_entorn_utils.
backend/afegir_entorn_backend.py	Façana de compatibilitat per al backend afegir entorns.	Anàlisi, conversió i YAML d'entorns.	connecta amb afegir_entorn_analysis, afegir_entorn_config, afegir_entorn_constants...
backend/afegir_entorn_config.py	Crea i valideu les configuracions de Sinergym des de l'entrada del formulari Studio.	Anàlisi, conversió i YAML d'entorns.	organitza funcions com build_variable_output_names, add_variable_spec, build_training_observation_config...; connecta amb afegir_entorn_analysis, afegir_entorn_constants, afegir_entorn_types...
backend/afegir_entorn_constants.py	Constants i valors per defecte del flux per afegir entorns.	Anàlisi, conversió i YAML d'entorns.	agrupa constants o inicialització de mòdul.

Fitxer	Què fa	A què afecta	Com funciona
backend/afegir_entorn_conversion.py	Conversió i actualització d'IDF/epJSON dins del flux per afegir entorns.	Anàlisi, conversió i YAML d'entorns.	organitza funcions com detect_idf_version, needs_idf_upgrade, get_transition_updater_dir...; connecta amb afegir_entorn_constants, afegir_entorn_types. Defineix UpgradeResult, EnvironmentValidationResult, ActuatorOption....
backend/afegir_entorn_types.py	Tipus de dades compartits pel flux per afegir entorns.	Anàlisi, conversió i YAML d'entorns.	organitza funcions com save_uploaded_bytes, sanitize_identifier, normalize_token.... Defineix EvaluationResult; organitza funcions com push_evaluation_progress, build_evaluation_env, empty_evaluation_runtime...; connecta amb paths, sb3_utils.
backend/afegir_entorn_utils.py	Funcions comunes per desar fitxers i normalitzar noms dins del flux per afegir entorns.	Anàlisi, conversió i YAML d'entorns.	organitza funcions com sanitize_training_name_component, get_training_artifacts_root, build_training_artifact_paths...; connecta amb entrenar_agent_constants.
backend/avaluar_agent_backend.py	Temps d'execució d'avaluació asíncrona per a agents Studio entrenats.	Execució de models entrenats sobre entorns.	connecta amb entrenar_agent_artifacts, entrenar_agent_constants, entrenar_agent_env.... organitza funcions com render_live_training_charts; connecta amb entrenar_agent_session, grafics.data_loader, grafics.figures_common.
backend/entrenar_agent_artifacts.py	Rutes d'artefacte d'entrenament, metadades i postprocessament CSV.	Preparació, execució i persistència d'entrenaments.	connecta amb paths.
backend/entrenar_agent_backend.py	Façana pública per al backend de entrenament Studio.	Preparació, execució i persistència d'entrenaments.	organitza funcions com list_registered_envs, list_files, with_detailed_hvac_meters...; connecta amb entrenar_agent_constants.
backend/entrenar_agent_charts.py	Renderització de gràfics d'entrenament en directe per a la pàgina entrenar l'agent.	Preparació, execució i persistència d'entrenaments.	organitza funcions com reward_summary_frames, build_multizone_temp_mapping, build_multizone_occupancy_mapping...; connecta amb entrenar_agent_artifacts, entrenar_agent_constants, entrenar_agent_env....
backend/entrenar_agent_constants.py	Constants i registres per al backend de l'agent de entrenament.	Preparació, execució i persistència d'entrenaments.	organitza funcions com clear_training_runtime, prepare_incremental_learn, save_training_artifacts...; connecta amb entrenar_agent_artifacts, entrenar_agent_constants, entrenar_agent_env....
backend/entrenar_agent_env.py	Descobrimet d'entorns i metadades per configurar entrenaments.	Preparació, execució i persistència d'entrenaments.	organitza funcions com training_form_key, reset_widget_state_if_disabled, ensure_select_state...; connecta amb entrenar_agent_backend.
backend/entrenar_agent_rewards.py	Recompenses, algorismes i creadors de càrrega útil de entrenament completa.	Preparació, execució i persistència d'entrenaments.	organitza funcions com filter_supported_kwargs, format_ui_value. Defineix EnergyCostFileLogger; organitza funcions com wrapper_details, build_wrapper_summary_rows, apply_training_wrappers...; connecta amb entrenar_agent_constants, entrenar_agent_utils.
backend/entrenar_agent_runtime.py	Cicle de vida d'execució per a sessions d'entrenament en directe.	Preparació, execució i persistència d'entrenaments.	organitza funcions com build_active_month_axis, apply_figure_style...; connecta amb backend.
backend/entrenar_agent_session.py	Gestió de l'estat de sessió per a la pàgina d'entrenament de l'agent.	Preparació, execució i persistència d'entrenaments.	Defineix ExplorerItem; organitza funcions com get_size_format, parse_epw_overview, load_weather_map_rows....
backend/entrenar_agent_utils.py	Funcions comunes per preparar arguments i valors que es mostren durant l'entrenament.	Preparació, execució i persistència d'entrenaments.	organitza funcions com mode_requires_model, load_model_training_metadata, build_env_factory...; connecta amb sb3_utils.
backend/entrenar_agent_wrappers.py	Configuració i aplicació de wrapper per a entorns d'entrenament.	Preparació, execució i persistència d'entrenaments.	
backend/epw_figures.py	Plotly creadors de figures per al visor de clima EPW.	Lectura i gràfics de clima EPW.	
backend/gestionar_arxius_backend.py	Gestió de fitxers per a actius, models i dades meteorològiques de Studio.	Exploració, pujada, descàrrega i eliminació de fitxers.	
backend/interaccionar_agent_backend.py	Execució en directe per provar agents entrenats dins d'un entorn Sinergym.	Execució de models entrenats sobre entorns.	

Fitxer	Què fa	A què afecta	Com funciona
backend/mapa_backend.py	Utilitats de representació de mapes compartides construïdes a pydeck i Streamlit.	Backend funcional.	organitza funcions com render_location_map, render_multipoint_map.
backend/mostrar_entorn_backend.py	Backend de metadades de l'entorn: carrega actius, PV, emmagatzematge i dades per pintar la UI.	Lectura d'actius i representació 3D.	organitza funcions com load_epjson, parse_pv_and_storage, load_environment_assets...; connecta amb viewer_3d_backend.
backend/paths.py	Camins del sistema de fitxers canònics utilitzats pels mòduls de fons BEMS-RL Studio.	Backend funcional.	agrupa constants o inicialització de mòdul.
backend/resultats_backend.py	Càrrega de dades, preparació de KPI i descobriment d'execució per a la pàgina de resultats.	Càrrega de runs, mètriques i exportació d'informes.	Defineix RunOption, ResultsPageState, RunArtifacts...; organitza funcions com build_results_page_state, get_run_artifacts, load_action_data...; connecta amb grafics.time_utils.
backend/resultats_report_backend.py	Generació d'informes HTML i PDF per a les execucions de Studio completades.	Càrrega de runs, mètriques i exportació d'informes.	Defineix ReportFigure; organitza funcions com generate_report_bytes; connecta amb grafics.comfort_scope, grafics.figures_common, grafics.figures_zones... organitza funcions com scan_model_zips, candidate_vecnorm, load_sb3_model_bytes... Defineix EnvironmentSummary, BaselineSimulationResult, ScheduleBaselineReward; organitza funcions com list_environment_ids, describe_environment, run_baseline_simulation.
backend/sb3_utils.py	Descoberta, càrrega i format d'accions per a models Stable-Baselines3.	Backend funcional.	organitza funcions com build_training_eval_script, build_training_repro_script.
backend/simular_entorn_backend.py	Simulacions de referència per a execucions no controlades o basades en regles.	Simulació baseline i artefactes comparables.	organitza funcions com extract_vertices_general, build_surface_records, plot_3d_model.
backend/training_scripts.py	Genereu scripts d'ajuda d'entrenament reproduïbles per a les execucions BEMS-RL desades.	Backend funcional.	organitza funcions com variable_label, list_epw_catalog, load_epw_bundle... Defineix WeatherProfileSuggestion; organitza funcions com summarize_weather_file, load_epw_dry_bulb_series, build_weather_temperature_preview...
backend/viewer_3d_backend.py	Backend del visor d'edificis 3D: processament de geometria i generació de figures Plotly.	Lectura d'actius i representació 3D.	
backend/visor_epw_backend.py	Utilitats de preparació de dades per al visor climàtic EPW.	Lectura i gràfics de clima EPW.	
backend/weather_profiles.py	Anàlisi de perfils meteorològics i vista prèvia per crear entorns.	Backend funcional.	

2.5 Gràfics i mètriques

Fitxer	Què fa	A què afecta	Com funciona
backend/grafics/__init__.py	Gràfics Plotly i mètriques compartides per a resultats i informes.	KPIs, normalització temporal i figures Plotly.	agrupa constants o inicialització de mòdul.
backend/grafics/battery.py	Dades del quadre de comandament de la bateria, la xarxa i les tarifes.	KPIs, normalització temporal i figures Plotly.	organitza funcions com make_battery_power_plot, make_battery_soc_plot, make_battery_vs_grid_plot...; connecta amb grafics.column_utils, grafics.energy_units, grafics.hvac... agrupa constants o inicialització de mòdul.
backend/grafics/column_utils.py	Detecció de columnes disponibles per construir les figures del panell.	KPIs, normalització temporal i figures Plotly.	organitza funcions com make_comfort_compliance, make_violationBars, make_violation_single_line; connecta amb grafics.column_utils, grafics.comfort_scope, grafics.style....
backend/grafics/comfort.py	Compliment de la comoditat i xifres d'incompliment de la temperatura.	KPIs, normalització temporal i figures Plotly.	organitza funcions com derive_occupied_mask, has_occupied_data, filter_by_comfort_scope...
backend/grafics/comfort_scope.py	Filtres d'abast de confort compartits per KPI, gràfics i informes.	KPIs, normalització temporal i figures Plotly.	

Fitxer	Què fa	A què afecta	Com funciona
backend/grafics/control.py	Figures de control i acciÃ³ per a HVAC i pistes de terra radiant.	KPIs, normalització temporal i figures Plotly.	organitza funcions com make_radiant_control_plot, make_agent_actions_plot; connecta amb grafics.style, grafics.time_utils.
backend/grafics/data_loader.py	Carrega i normalitza els fitxers CSV de resultats per a panells i informes.	KPIs, normalització temporal i figures Plotly.	organitza funcions com load_data; connecta amb grafics.observation_columns, grafics.time_utils.
backend/grafics/energy_price.py	Dades del quadre de comandament del preu de l'energia.	KPIs, normalització temporal i figures Plotly.	organitza funcions com make_energy_price_plot; connecta amb grafics.column_utils, grafics.energy_units, grafics.style....
backend/grafics/energy_units.py	Conversions d'energia, preu i unitats de bateria per als gràfics.	KPIs, normalització temporal i figures Plotly.	connecta amb grafics.time_utils.
backend/grafics/episode.py	Xifres del quadre de comandament a nivell d'episodi.	KPIs, normalització temporal i figures Plotly.	organitza funcions com make_episode_metrics; connecta amb grafics.style.
backend/grafics/figures_common.py	Façana de compatibilitat per a tots els creadors de figures del panell de resultats.	KPIs, normalització temporal i figures Plotly.	connecta amb grafics.battery, grafics.column_utils, grafics.comfort....
backend/grafics/figures_zones.py	Detecció i filtratge de zones per als panells de resultats.	KPIs, normalització temporal i figures Plotly.	organitza funcions com detect_zones, filter_obs_by_zone, get_zone_options.
backend/grafics/heatmaps.py	Xifres de mapes de calor per als resums del panell global i de zona.	KPIs, normalització temporal i figures Plotly.	organitza funcions com make_heatmap, make_violation_heatmap_percent, compute_zone_temperature_pivots...;
backend/grafics/hvac.py	HVAC xifres de consum i avaria del comptador.	KPIs, normalització temporal i figures Plotly.	connecta amb grafics.column_utils, grafics.comfort, grafics.time_utils.
backend/grafics/kpis.py	KPI càlculs per a BEMS-RL Studio resums de resultats.	KPIs, normalització temporal i figures Plotly.	organitza funcions com make_hvac_consumption_plot, make_hvac_meter_breakdown_plot; connecta amb grafics.style, grafics.time_utils.
backend/grafics/metrics.py	Agrega les dades d'observació i progrés en mètriques preparades per a gràfics.	KPIs, normalització temporal i figures Plotly.	organitza funcions com compute_kpis; connecta amb grafics.comfort_scope, grafics.observation_columns.
backend/grafics/observation_columns.py	Utilitats de normalització de columnes per a fitxers d'observació Sinergym.	KPIs, normalització temporal i figures Plotly.	organitza funcions com compute_metrics; connecta amb grafics.observation_columns, grafics.time_utils.
backend/grafics/style.py	Estil Plotly compartit per a les figures del panell BEMS-RL Studio.	KPIs, normalització temporal i figures Plotly.	organitza funcions com find_first_available_column, normalize_observation_columns, find_hvac_consumption_source....
backend/grafics/thermal.py	Dades d'entrada de confort tèrmic: temperatura, humitat i consignes.	KPIs, normalització temporal i figures Plotly.	organitza funcions com pick_semantic_trace_color, style_figure_semantics.
backend/grafics/time_utils.py	Filtrat temporal i ajuda d'eix per a figures del panell.	KPIs, normalització temporal i figures Plotly.	organitza funcions com make_indoor_temperature_plot, make_indoor_humidity_plot, make_setpoints_plot...;
			connecta amb grafics.column_utils, grafics.style, grafics.time_utils.
			organitza funcions com sanitize_observation_time_columns, filter_by_season; connecta amb grafics.style.

2.6 Eines de manteniment

Fitxer	Què fa	A què afecta	Com funciona
test_eppy.py	Sonda manual Eppy per inspeccionar les variables de sortida EnergyPlus.	Eines de manteniment.	organitza funcions com main.
test_sql.py	Sonda manual EnergyPlus SQLite.	Eines de manteniment.	organitza funcions com main.

3 Guia de classes

Aquesta secció explica totes les classes i dataclasses del codi funcional inclòs a l'annex. Serveix com a mapa ràpid abans d'entrar al codi complet.

Classe	Tipus	Què fa	Fitxer	Com funciona
ProbeReward	reward Sinergym	Reward nul usat només per inspeccionar handlers disponibles.	backend/afegir_entorn_analysis.py	Hereta de BaseReward.
UpgradeResult	dataclass	Contenidor de dades per a UpgradeResult.	backend/afegir_entorn_types.py	guarda dades tipades sense lògica complexa.
EnvironmentValidationResult	dataclass	Contenidor de dades per a EnvironmentValidationResult.	backend/afegir_entorn_types.py	guarda dades tipades sense lògica complexa.
ActuatorOption	dataclass	Contenidor de dades per a ActuatorOption.	backend/afegir_entorn_types.py	guarda dades tipades sense lògica complexa.
BuildingTrainingAnalysis	dataclass	Contenidor de dades per a BuildingTrainingAnalysis.	backend/afegir_entorn_types.py	guarda dades tipades sense lògica complexa.
EvaluationResult	dataclass	Resum de resultats d'una execució d'un treballador d'avaluació.	backend/avaluar_agent_backend.py	guarda dades tipades sense lògica complexa.
EnergyCostFileLogger	wrapper Gym	Logger addicional (opcional) que escriu cost per pas en un CSV independent.	backend/entrenar_agent_wrappers.py	Hereta de Wrapper; mètodes clau: step, reset, close.
ExplorerItem	dataclass	Fila del navegador de fitxers serialitzable que es mostra a la pàgina de gestió d'actius.	backend/gestionar_arxius_backend.py	guarda dades tipades sense lògica complexa.
RunOption	dataclass	Directorio d'execució seleccionable que es mostra a la pàgina de resultats.	backend/resultats_backend.py	guarda dades tipades sense lògica complexa.
ResultsPageState	dataclass	S'han resolt les entrades necessàries abans que es pugui representar la pàgina de resultats.	backend/resultats_backend.py	guarda dades tipades sense lògica complexa.
RunArtifacts	dataclass	Fitxers d'artefactes derivats d'una execució seleccionada.	backend/resultats_backend.py	guarda dades tipades sense lògica complexa.
DashboardData	dataclass	Es requereix un paquet de dades carregat per representar el panell de resultats en línia.	backend/resultats_backend.py	guarda dades tipades sense lògica complexa.
ReportFigure	dataclass	Plotly gràfic més la secció d'informe on hauria d'aparèixer.	backend/resultats_report_backend.py	guarda dades tipades sense lògica complexa.
EnvironmentSummary	dataclass	Descripció compacta d'un entorn Sinergym registrat.	backend/simular_entorn_backend.py	guarda dades tipades sense lògica complexa.
BaselineSimulationResult	dataclass	Sortides persistents i metadades d'estat per a una simulació de referència.	backend/simular_entorn_backend.py	guarda dades tipades sense lògica complexa.
ScheduleBaselineReward	reward Sinergym	Reward zero que conserva les mètriques físiques per comparar contra agents entrenats.	backend/simular_entorn_backend.py	Hereta de BaseReward; mètodes clau: _compute_power_demand, _compute_temperature_violation.
WeatherProfileSuggestion	dataclass	Descriu un suggeriment de fitxer meteorològic per a la creació de l'entorn UI.	backend/weather_profiles.py	guarda dades tipades sense lògica complexa.
EpwDashboardState	dataclass	Dades preparades necessàries per representar les pestanyes del panell EPW.	pages/Visor_Climes_EPW.py	guarda dades tipades sense lògica complexa.

4 Visió general del codi

BEMS-RL Studio està organitzat com una capa d'aplicació sobre Sinergym. Les pàgines Streamlit gestionen la interacció amb l'usuari, els components reutilitzables concentren formularis i panells compartits, i el backend manté la lògica de fitxers, validació, entrenament, simulació, avaluació, gràfics i informes. Aquesta separació és la que permet documentar el codi amb autodoc i, alhora, mantenir les pàgines com a punts d'entrada lleugers.

4.1 Arquitectura d'execució

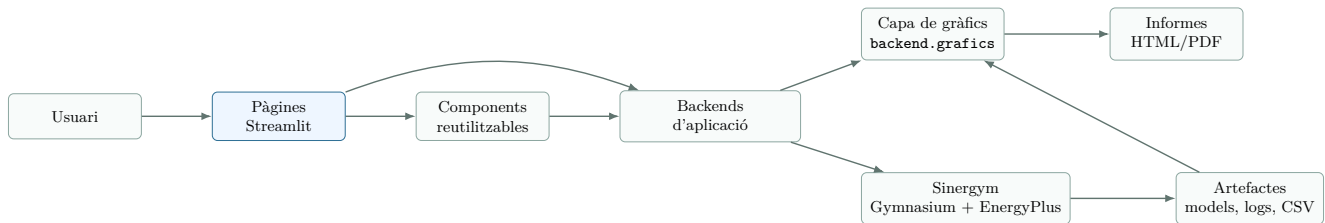


Figura 1: Arquitectura lògica de BEMS-RL Studio i relació amb Sinergym.

Lectura del diagrama. El diagrama separa l'aplicació en capes. L'usuari només toca les pàgines Streamlit; aquestes pàgines deleguen en components i backends. El backend és qui parla amb Sinergym, llegeix o escriu artefactes i passa dades a la capa de gràfics i informes.

4.2 Topologia Docker i documentació

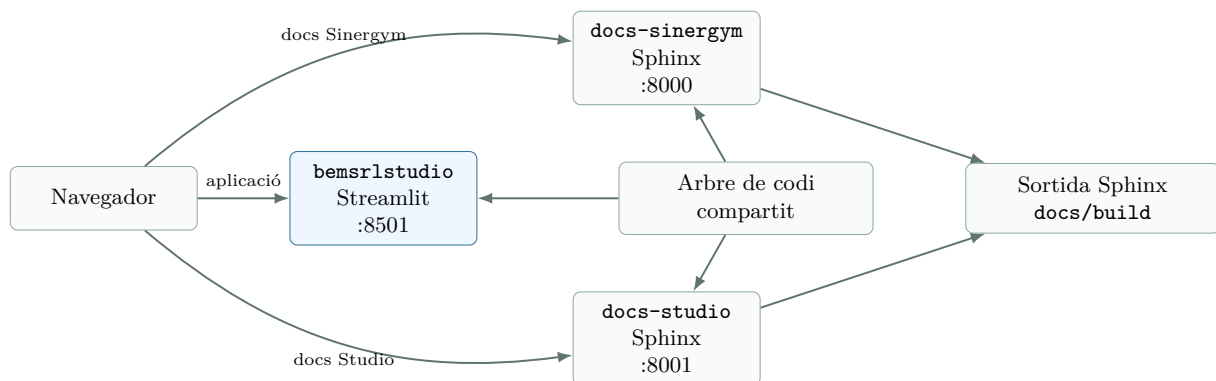


Figura 2: Serveis locals: aplicació i documentació separada en dos ports.

Lectura del diagrama. Aquest dibuix mostra la topologia d'execució local. Streamlit serveix la interfície principal, mentre que la documentació queda separada en serveis propis. Tots tres serveis llegeixen el mateix arbre de codi, però generen sortides diferents.

4.3 Propietat de dades i artefactes

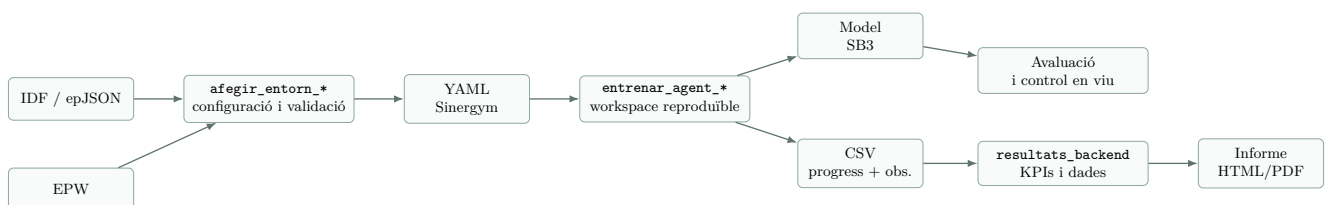


Figura 3: Mòduls responsables de cada família d'artefactes.

Lectura del diagrama. Aquí es veu el recorregut dels fitxers físics. Els IDF/epJSON i EPW entren pel flux de creació d'entorns, es converteixen en YAML de Sinergym i després alimenten entrenaments. Dels entrenaments surten models i CSV, que són la base de resultats i informes.

4.4 Límit d'importació i autodoc

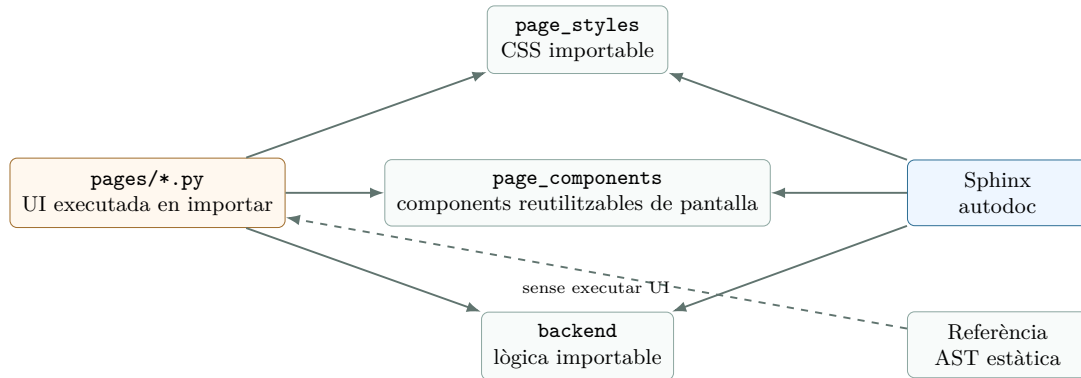


Figura 4: Frontera entre lògica importable i punts d'entrada de Streamlit.

Lectura del diagrama. La frontera important és que les pàgines Streamlit s'executen en importar-les, mentre que backend i components han de ser importables sense efectes laterals pesats. Per això la documentació automàtica tracta les pàgines amb més prudència.

5 Fluxos principals

5.1 Creació d'entorns

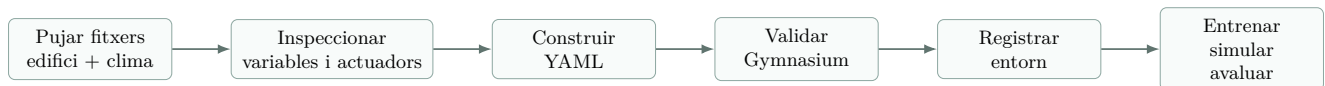


Figura 5: Flux d'autoria d'entorns de la documentació de Studio.

Lectura del diagrama. El flux va d'esquerra a dreta: primer es carreguen fitxers, després s'inspeccionen variables i actuadors, es construeix el YAML, es valida amb Gymnasium i finalment l'entorn queda llest per entrenar, simular o avaluar.

5.2 Cicle d'entrenament

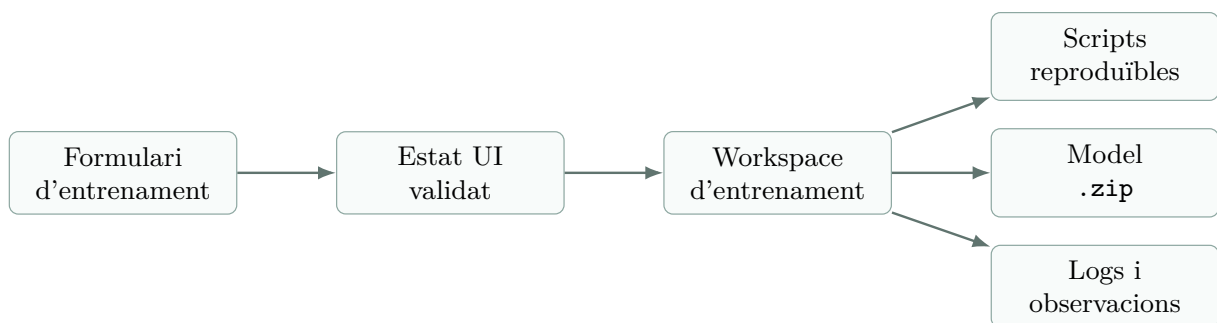


Figura 6: Cicle de vida dels artefactes d'entrenament.

Lectura del diagrama. El formulari no entrena directament. Primer es converteix en estat validat, després es crea un workspace reproduïble i, a partir d'aquí, es guarden model, logs, observacions i scripts que permeten repetir l'experiment.

5.3 Resultats i selecció de gràfics

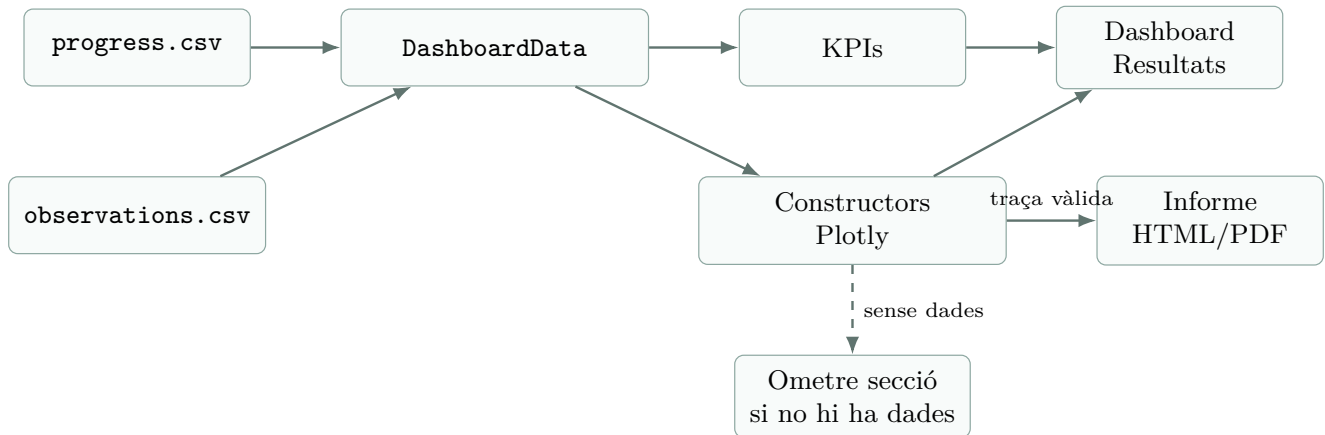


Figura 7: Ruta de dades de resultats i criteri d'inclusió de figures.

Lectura del diagrama. Els resultats sempre parteixen de `progress.csv` i `observations.csv`. El carregador els converteix en `DashboardData`; les figures només passen al dashboard o a l'informe quan tenen dades reals, així s'eviten gràfics buits.

5.4 Control en viu

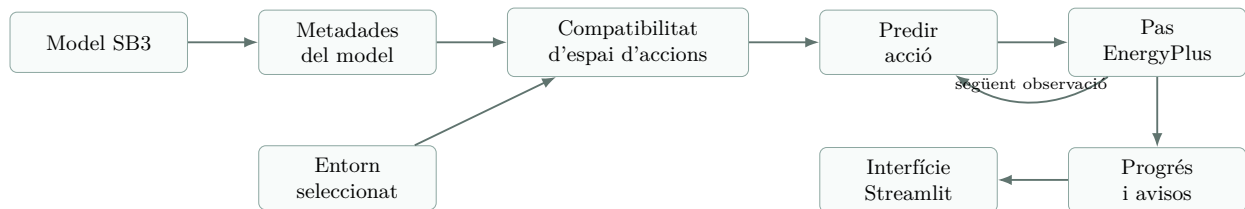


Figura 8: Bucle d'execució de control en viu.

Lectura del diagrama. El bucle de control comprova metadades i compatibilitat abans de predir. Després aplica l'acció a EnergyPlus, rep la següent observació i actualitza la UI. La fletxa de retorn indica que el procés es repeteix a cada pas.

6 Dependències entre fitxers

Els diagrames següents mostren les dependències principals entre fitxers. No inclouen cada import petit de suport, sinó les relacions que cal entendre per defensar el projecte: pàgina, components, backend, artefactes i classes de dades.

6.1 Creació d'entorns



Figura 9: Dependències dels fitxers que creen i registren entorns Sinergym.

Lectura del diagrama. Aquest flux comença a la pàgina d'alta d'entorns i passa pel backend principal, que reparteix la feina entre lectura del model, conversió de fitxers, construcció del YAML i validació. Els fitxers de tipus i utilitats eviten repetir estructures; `weather_profiles` i `paths` situen l'entorn dins del projecte.

6.2 Entrenament d'agents

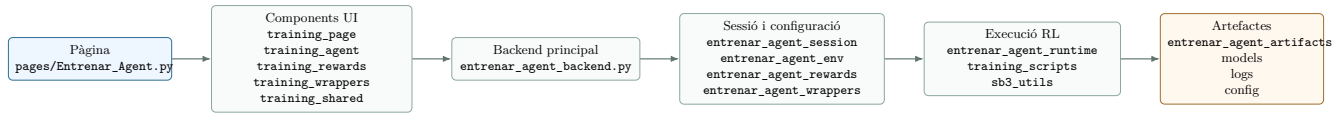


Figura 10: Dependències principals del flux d'entrenament.

Lectura del diagrama. La pàgina només recull opcions i mostra estat. Els components separen pestanyes i formularis, el backend principal valida la petició i els mòduls de sessió, entorn, recompenses i wrappers preparen una configuració coherent. L'execució real queda a `entrenar_agent_runtime`, `training_scripts` i `sb3_utils`; al final es deixen models, logs i configuració reproducible.

6.3 Resultats, dashboard i informes

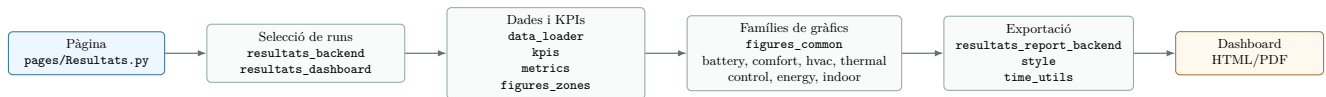


Figura 11: Dependències entre fitxers del flux de resultats i exportació.

Lectura del diagrama. El flux de resultats primer localitza la run i els fitxers necessaris. Després carrega CSV, calcula mètriques i prepara dades de zona. Les famílies de gràfics comparteixen format i criteris visuals; l'exportador reutilitza aquestes figures per generar l'informe HTML/PDF sense duplicar càlculs.

6.4 Simulació, avaluació i control en viu

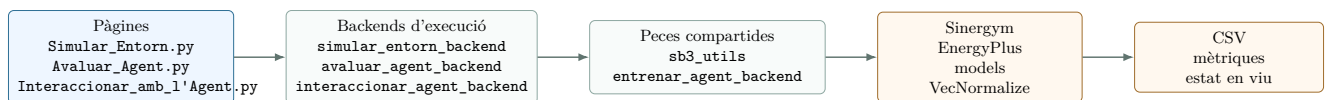


Figura 12: Dependències dels fluxos d'execució sense editar entorns.

Lectura del diagrama. Aquests tres fluxos comparteixen la idea d'executar un entorn ja existent. La simulació genera una línia base, l'avaluació carrega un agent entrenat i el control en viu repeteix prediccions pas a pas. `sb3_utils` centralitza càrrega i formes d'acció perquè els tres camins no tinguin regles diferents.

6.5 Visors i gestió de fitxers

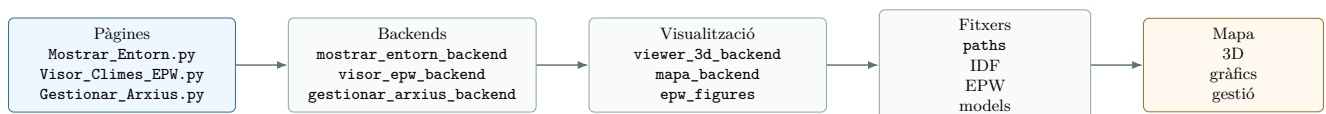


Figura 13: Dependències dels visors d'entorn/clima i del gestor d'arxius.

Lectura del diagrama. Aquest bloc agrupa les pantalles que no entrenen agents. Els backends preparen metadades i fitxers; els mòduls de visualització converteixen aquesta informació en mapa, visor 3D o gràfics EPW. `paths` manté rutes consistents perquè el gestor d'arxius i els visors mirin el mateix lloc.

7 Classes i dataclasses principals

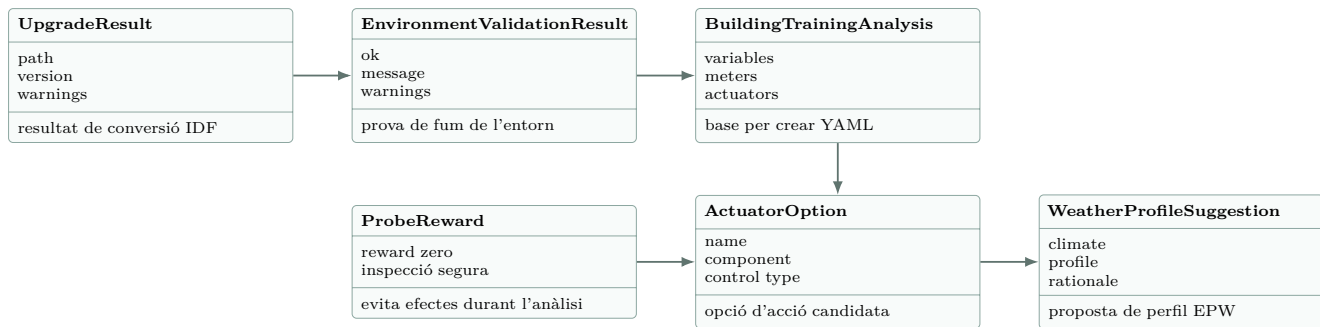


Figura 14: Classes i dataclasses del flux de creació d'entorns.

Lectura del diagrama. UpgradeResult i EnvironmentValidationResult descriuen resultats de processos que poden fallar o generar avisos. BuildingTrainingAnalysis concentra allò que s'ha llegit de l'edifici i es connecta amb actuadors, recompenses de prova i suggeriments de clima per acabar generant una configuració entrenable.

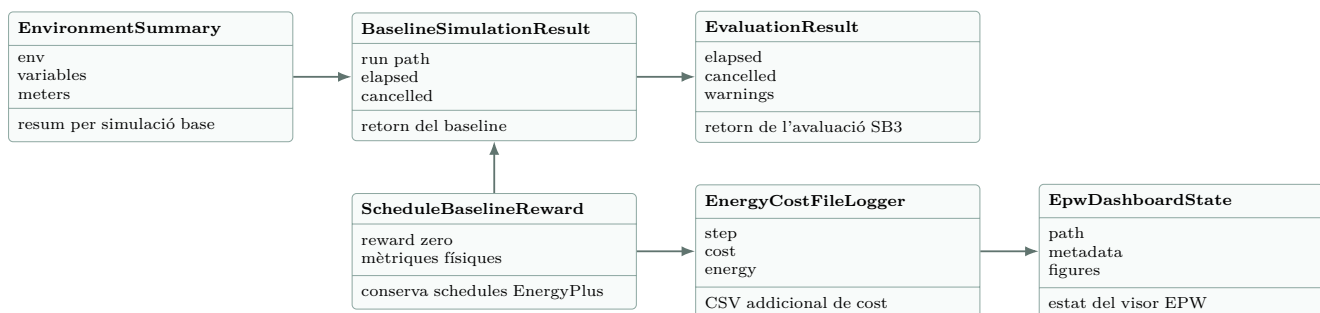


Figura 15: Classes del flux d'execució: baseline, entrenament, avaluació i clima.

Lectura del diagrama. EnvironmentSummary, BaselineSimulationResult i EvaluationResult són els retorns grans que les pàgines poden mostrar sense conèixer els detalls d'EnergyPlus. A sota hi ha classes de suport: una recompensa neutra per executar baselines, un logger de cost i l'estat del visor EPW.

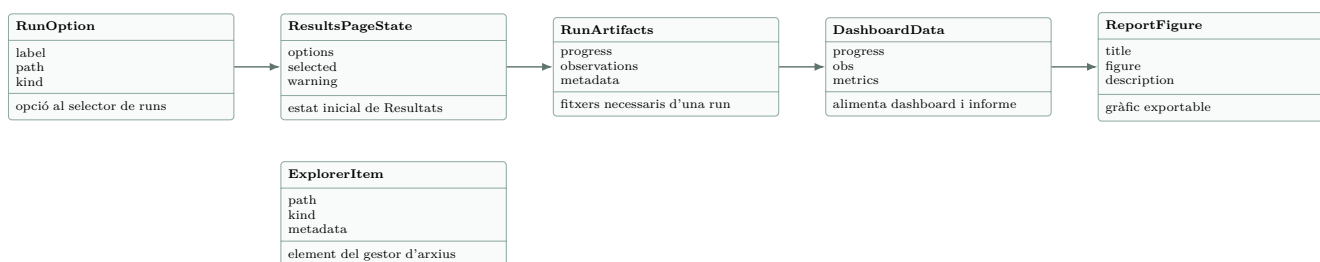


Figura 16: Classes i dataclasses de resultats, informes i gestió de fitxers.

Lectura del diagrama. El primer tram explica el camí normal de Resultats: opció seleccionable, estat de pàgina, fitxers trobats, dades preparades i figures exportables. ExplorerItem queda separat perquè representa elements del gestor d'arxius, no del càlcul principal de KPIs.

A Annex de codi funcional

Les llistes següents inclouen el codi Python funcional de BEMS-RL Studio dins d'aquest mateix document LaTeX. S'exclouen només els mòduls d'estil CSS pur (page_styles/*.py i ui_theme.py) perquè no aporten lògica d'aplicació, dades, entrenament, simulació o generació d'informes. Les icones Unicode es representen com a escapes \u... o \U... per evitar glifs buits en el PDF.

B Codi: Entrypoints i navegacio

B.1 __init__.py

Listing 1: __init__.py

```
1 """Metadades de l'aplicació per a l'espai de treball Streamlit de BEMS-RL Studio.
2
3 El punt d'entrada executable és :mod:`Inici`; les pàgines individuals viuen sota
4 ``pages`` i Streamlit les carrega automàticament. Aquest mòdul es manté sense
5 efectes secundaris perquè la documentació i les eines de diagnòstic puguin
6 importar l'arrel de Studio sense renderitzar cap pàgina.
7 """
8
9 from __future__ import annotations
10
11 APP_NAME = "BEMS-RL Studio"
12 APP_ENTRYPOINT = "Inici.py"
13
14 __all__ = [APP_NAME, "APP_ENTRYPOINT"]
```

B.2 Inici.py

Listing 2: Inici.py

```
1 """Pàgina inicial de BEMS-RL STUDIO."""
2
3 from __future__ import annotations
4
5 import base64
6 from html import escape
7 from pathlib import Path
8 from subprocess import STDOUT, CalledProcessError, check_output
9
10 import streamlit as st
11
12 from sidebar_nav import configure_studio_page
13 from ui_theme import inject_studio_theme
14
15
16 PAGE_TITLE = "BEMS-RL STUDIO"
17 PAGE_LAYOUT = "wide"
18 INTRODUCTION_TEXT = (
19     "Benvingut a BEMS-RL STUDIO, una interfície gràfica per treballar "
20     "amb Reinforcement Learning en simulacions energètiques d'edificis."
21 )
22 CAPABILITIES = (
23     "Afegir nous entorns amb edificis i meteorologia.",
24     "Visualitzar i inspeccionar els entorns disponibles abans de treballar-hi.",
25     "Entrenar agents d'Aprenentatge per Reforç en un entorn Eplus personalitzat.",
26     "Simular entorns energètics per validar comportaments i condicions de partida.",
27     "Avaluar el comportament del teu agent.",
28     "Interaccionar amb agents entrenats mitjançant el control en viu.",
29     "Explorar resultats amb dashboards i gràfics.",
30     "Gestionar fitxers, execucions, models, climes i entorns del projecte.",
31     "Analitzar fitxers climàtics EPW amb resum, patrons anuals i roses dels vents.",
32     "Consultar el centre d'ajuda per entendre el flux de treball de l'aplicació.",
33 )
34 NAVIGATION_HINT = "Utilitza el menú lateral per accedir a cada secció."
35 RESOURCES_DIR = Path(__file__).resolve().parent / "resources"
36 ENERGYPLUS_LOGO_PATH = RESOURCES_DIR / "energyplus-logo.png"
37
38
39 def inject_home_styles() -> None:
40     """Aplica l'estil visual específic de la pàgina inicial."""
41
42     inject_studio_theme(max_width=1180)
43     # La home té peces visuals pròpies; les deixem aquí per no carregar aquest CSS a totes
44     # les pàgines de l'aplicació.
45     st.markdown(
46         """
47         <style>
48         .home-top-panel {
49             min-height: 15rem;
50         }
51
52         .status-shell {
53             position: relative;
54             padding-right: 7rem;
55         }
56
57         .status-logo {
58             position: absolute;
59             bottom: 1.25rem;
60             right: 1.4rem;
```

```

61         width: 4.9rem;
62         height: auto;
63         object-fit: contain;
64         opacity: 0.97;
65     }
66
67     .status-copy {
68         margin-top: 0.85rem;
69     }
70
71     .feature-card {
72         display: flex;
73         flex-direction: column;
74         justify-content: flex-start;
75         height: 100%;
76         min-height: 12.5rem;
77         padding: 1.15rem 1.2rem 1.2rem;
78         margin-bottom: 1rem;
79     }
80
81     .feature-copy {
82         margin: 0;
83         font-size: 1rem;
84         line-height: 1.72;
85     }
86
87     .footer-card {
88         margin-top: 2.25rem;
89         padding: 1.5rem 1.6rem;
90     }
91
92     .home-features-gap {
93         height: 0.95rem;
94     }
95
96     @media (max-width: 900px) {
97         .status-shell {
98             padding-right: 5.4rem;
99         }
100
101         .status-logo {
102             bottom: 1rem;
103             right: 1.05rem;
104             width: 4rem;
105         }
106
107         .feature-card,
108         .footer-card {
109             padding: 1.15rem 1.1rem;
110         }
111
112         .feature-card {
113             min-height: auto;
114         }
115
116         .footer-card {
117             margin-top: 1.55rem;
118         }
119     }
120 </style>
121 """
122     unsafe_allow_html=True,
123 )
124
125
126 @st.cache_data(show_spinner=False)
127 def get_energyplus_logo_data_uri() -> str:
128     """Carrega el logo d'EnergyPlus com a data URI per incrustar-lo a la UI."""
129
130     if not ENERGYPLUS_LOGO_PATH.exists():
131         return ""
132
133     encoded_logo = base64.b64encode(ENERGYPLUS_LOGO_PATH.read_bytes()).decode("ascii")
134     return f"data:image/png;base64,{encoded_logo}"
135
136
137 @st.cache_data(show_spinner=False)
138 def detect_energyplus_status() -> tuple[bool, str]:
139     """Retorna si EnergyPlus està disponible i el missatge per mostrar a la UI."""
140
141     try:
142         version = check_output(["energyplus", "--version"], stderr=STDOUT).decode("utf-8").strip()
143     except (CalledProcessError, FileNotFoundError, OSError):
144         return (
145             False,
146             "No s'ha detectat una instal·lació d'EnergyPlus al sistema operatiu o no està disponible al PATH.",
147         )
148
149     if not version:
150         return False, "EnergyPlus respon sense versió. Revisa la instal·lació del motor."
151
152     return True, f"Motor actiu detectat: {version}"

```

```

153
154
155 def render_hero() -> None:
156     """Renderitza el bloc principal de presentació."""
157
158     st.markdown(
159         f"""
160         <section class="home-hero">
161             <div class="hero-kicker">Plataforma de simulació energètica</div>
162             <h1 class="hero-title">{escape(PAGE_TITLE)}</h1>
163             <div class="hero-copy">{escape(INTRODUCTION_TEXT)}</div>
164         </section>
165         """,
166         unsafe_allow_html=True,
167     )
168
169
170 def render_status_panel(energyplus_available: bool, energyplus_message: str) -> None:
171     """Renderitza l'estat d'EnergyPlus amb una targeta visual."""
172
173     status_title = "EnergyPlus detectat" if energyplus_available else "EnergyPlus no disponible"
174     status_class = "status-ok" if energyplus_available else "status-warning"
175     logo_data_uri = get_energyplus_logo_data_uri()
176     logo_html = ""
177     if logo_data_uri:
178         logo_html = f''
179
180     st.markdown(
181         f"""
182         <section class="home-panel home-top-panel status-shell {status_class}">
183             {logo_html}
184             <div class="panel-kicker">Entorn de simulació</div>
185             <div class="panel-title">{escape(status_title)}</div>
186             <div class="panel-copy status-copy">{escape(energyplus_message)}</div>
187         </section>
188         """,
189         unsafe_allow_html=True,
190     )
191
192
193 def render_navigation_panel() -> None:
194     """Renderitza la pista de navegació lateral."""
195
196     st.markdown(
197         f"""
198         <section class="home-panel home-top-panel">
199             <div class="panel-kicker">Navegació</div>
200             <div class="panel-title">Començar a treballar</div>
201             <div class="panel-copy">{escape(NAVIGATION_HINT)}</div>
202         </section>
203         """,
204         unsafe_allow_html=True,
205     )
206
207
208 def render_capability_card(index: int, capability: str) -> None:
209     """Renderitza una targeta de funcionalitat."""
210
211     st.markdown(
212         f"""
213         <section class="feature-card">
214             <div class="feature-kicker">Funció {index:02d}</div>
215             <p class="feature-copy">{escape(capability)}</p>
216         </section>
217         """,
218         unsafe_allow_html=True,
219     )
220
221
222 def render_capability_grid() -> None:
223     """Renderitza les funcionalitats en files de tres targetes alineades."""
224
225     row_size = 3
226     for row_start in range(0, len(CAPABILITIES), row_size):
227         row_items = CAPABILITIES[row_start : row_start + row_size]
228         row_columns = st.columns(row_size, gap="large")
229         for column, capability_index, capability in zip(
230             row_columns,
231             range(row_start + 1, row_start + len(row_items) + 1),
232             row_items,
233         ):
234             with column:
235                 render_capability_card(capability_index, capability)
236
237
238 def render_organization_footer() -> None:
239     """Renderitza el bloc informatiu final."""
240
241     st.markdown(
242         f"""
243         <section class="footer-card">
244             <div class="panel-kicker">Projecte</div>

```

```

245         <div class="footer-title">Desenvolupat per SUNO</div>
246         <div class="footer-copy">
247             Aquest projecte ha estat desenvolupat per
248             <a href="https://www.suno.cat/" target="_blank">SUNO</a>,
249             una empresa pionera en simulació energètica d'edificis,
250             optimització de sistemes i integració de solucions
251             d'intel·ligència artificial.
252         </div>
253     </section>
254     """
255     unsafe_allow_html=True,
256 )
257
258
259 def render_home_page() -> None:
260     """Renderitza la pàgina inicial."""
261
262     configure_studio_page(PAGE_TITLE, "Inici.py", layout=PAGE_LAYOUT)
263     inject_home_styles()
264     energyplus_available, energyplus_message = detect_energyplus_status()
265
266     render_hero()
267
268     status_col, navigation_col = st.columns(2, gap="large")
269     with status_col:
270         render_status_panel(energyplus_available, energyplus_message)
271     with navigation_col:
272         render_navigation_panel()
273
274     st.markdown("<div style='height: 1.15rem;'></div>", unsafe_allow_html=True)
275     st.markdown('<h2 class="section-title">Què pots fer?</h2>', unsafe_allow_html=True)
276     render_capability_grid()
277     st.markdown('<div class="home-features-gap"></div>', unsafe_allow_html=True)
278     render_organization_footer()
279
280
281 render_home_page()

```

B.3 sidebar_nav.py

Listing 3: sidebar_nav.py

```

1  """Shared sidebar navigation for BEMS-RL STUDIO."""
2
3  from __future__ import annotations
4
5  from textwrap import dedent
6
7  import streamlit as st
8
9
10 APP_NAME = "BEMS-RL STUDIO"
11
12 SIDEBAR_SECTIONS: tuple[tuple[str, str, tuple[tuple[str, str, str], ...], ...], ...] = (
13     (
14         "ENTORNS I AGENTS",
15         "Configura, entrena i analitza",
16         (
17             ("Inici.py", "Inici", ":material/space_dashboard:"),
18             ("pages/Afegir_Entorn.py", "Crear Entorn", ":material/home_work:"),
19             ("pages/Mostrar_Entorn.py", "Mostrar Entorn", ":material/apartment:"),
20             ("pages/Entrenar_Agent.py", "Entrenar Agent", ":material/model_training:"),
21             ("pages/Resultats.py", "Resultats", ":material/insights:"),
22             ("pages/Simular_Entorn.py", "Simular Entorn", ":material/play_circle:"),
23             ("pages/Avaluar_Agent.py", "Avaluar Agent", ":material/fact_check:"),
24             ("pages/Interaccionar_amb_l'Agent.py", "Control en Viu", ":material/joystick:"),
25         ),
26     ),
27     (
28         "GESTIÓ",
29         "Organitza i inspecciona els fitxers del projecte",
30         (
31             ("pages/Gestionar_Arxiu.py", "Arxiu", ":material/folder_open:"),
32             ("pages/Visor_Climes_EPW.py", "Visor EPW", ":material/cloud:"),
33         ),
34     ),
35     (
36         "SUPORT",
37         "Guies i ajuda",
38         (
39             ("pages/Ajuda.py", "Centre d'Ajuda", ":material/support_agent:"),
40         ),
41     ),
42 )
43
44
45 def configure_studio_page(page_title: str, current_page: str, *, layout: str = "wide") -> None:
46     """Configura una pàgina i renderitza la barra lateral compartida."""

```

```

47 browser_title = page_title if page_title.upper().startswith(APP_NAME) else f"{APP_NAME} - {page_title}"
48 st.set_page_config(
49     page_title=browser_title,
50     page_icon=":material/neurology:",
51     layout=layout,
52     initial_sidebar_state="expanded",
53 )
54 render_studio_sidebar(current_page)
55
56
57
58 def render_studio_sidebar(current_page: str) -> None:
59     """Mostra la navegació personalitzada de la barra lateral Streamlit a la UI de Streamlit."""
60
61     _inject_sidebar_shell_styles()
62
63     with st.sidebar:
64         st.markdown(
65             """
66             <div class="studio-sidebar-brand">
67                 <div class="studio-sidebar-brand-title">BEMS-RL</div>
68                 <div class="studio-sidebar-brand-subtitle">STUDIO</div>
69             </div>
70             <div class="studio-sidebar-divider"></div>
71             """
72             ,
73             unsafe_allow_html=True,
74         )
75
76         if hasattr(st, "page_link"):
77             for section_title, section_copy, items in SIDEBAR_SECTIONS:
78                 _render_section_header(section_title, section_copy)
79                 for page_path, label, icon in items:
80                     st.page_link(page_path, label=label, icon=icon)
81         else:
82             st.warning("La navegació lateral personalitzada necessita una versió de Streamlit amb `st.page_link`.")
83             for section_title, section_copy, items in SIDEBAR_SECTIONS:
84                 _render_section_header(section_title, section_copy)
85                 for page_path, label, _ in items:
86                     prefix = " " if page_path == current_page else ". "
87                     st.write(f"{prefix}{label}")
88
89 def _render_section_header(title: str, copy_text: str) -> None:
90     """Renderitza la capçalera de la secció."""
91     st.markdown(
92         f"""
93         <div class="studio-sidebar-section">{title}</div>
94         <div class="studio-sidebar-section-copy">{copy_text}</div>
95         """
96         ,
97         unsafe_allow_html=True,
98     )
99
100 def _inject_sidebar_shell_styles() -> None:
101     """Injecta el CSS de la barra lateral aviat per reduir parpellejos en rerender."""
102
103     # Streamlit pinta la sidebar abans que alguns components; injectar aquest bloc al principi
104     # evita veure botons nadius i estats intermedis durant mig segon.
105     st.markdown(
106         dedent(
107             """
108             <style>
109                 [data-testid="collapsedControl"],
110                 [data-testid="stSidebarCollapseButton"],
111                 button[kind="header"][aria-label*="sidebar" i],
112                 button[kind="header"][aria-label*="barra lateral" i] {
113                     display: none !important;
114                     visibility: hidden !important;
115                     pointer-events: none !important;
116                 }
117
118                 [data-testid="stSidebar"] {
119                     background: linear-gradient(180deg, #12192f 0%, #10162b 100%);
120                     border-right: 1px solid rgba(255, 255, 255, 0.08);
121                     box-shadow: inset -1px 0 0 rgba(255, 255, 255, 0.02);
122                 }
123
124                 [data-testid="stSidebar"],
125                 [data-testid="stSidebar"] > div:first-child,
126                 [data-testid="stSidebarContent"],
127                 [data-testid="stSidebar"] [data-testid="stPageLinkContainer"] a,
128                 [data-testid="stSidebar"] [data-testid="stPageLink-NavLink"] {
129                     transition: none !important;
130                     animation: none !important;
131                 }
132
133                 [data-testid="stSidebar"][aria-expanded="true"],
134                 [data-testid="stSidebar"][aria-expanded="false"] {
135                     min-width: 18rem !important;
136                     max-width: 18rem !important;
137                     width: 18rem !important;
138                     transform: translateX(0) !important;

```

```

139     }
140
141     [data-testid="stSidebar"][aria-expanded="true"] > div:first-child,
142     [data-testid="stSidebar"][aria-expanded="false"] > div:first-child {
143         width: 18rem !important;
144         min-width: 18rem !important;
145         max-width: 18rem !important;
146         background: transparent;
147         transform: translateX(0) !important;
148     }
149
150     [data-testid="stSidebar"][aria-expanded="false"] {
151         margin-left: 0 !important;
152     }
153
154     [data-testid="stSidebar"][aria-expanded="false"] + div,
155     [data-testid="stSidebar"][aria-expanded="false"] ~ section {
156         margin-left: 18rem !important;
157     }
158
159     [data-testid="stSidebarContent"] {
160         padding: 1rem 1rem 1.35rem;
161         background: transparent;
162     }
163
164     [data-testid="stSidebarUserContent"] {
165         padding-top: 0;
166     }
167
168     [data-testid="stSidebar"] .studio-sidebar-brand {
169         padding: 0.4rem 0.1rem 0.18rem;
170         background: transparent;
171         border: none;
172         border-radius: 0;
173         box-shadow: none;
174     }
175
176     [data-testid="stSidebar"] .studio-sidebar-brand-title,
177     [data-testid="stSidebar"] .studio-sidebar-brand-subtitle {
178         color: #eef3ff;
179         font-family: "Arial Narrow", "Aptos Narrow", "Bahnschrift SemiCondensed", sans-serif;
180         font-weight: 800;
181         letter-spacing: -0.045em;
182         text-transform: uppercase;
183     }
184
185     [data-testid="stSidebar"] .studio-sidebar-brand-title {
186         font-size: 2.22rem;
187         line-height: 0.88;
188     }
189
190     [data-testid="stSidebar"] .studio-sidebar-brand-subtitle {
191         margin-top: 0.06rem;
192         font-size: 1.4rem;
193         line-height: 0.92;
194     }
195
196     [data-testid="stSidebar"] .studio-sidebar-divider {
197         height: 1px;
198         margin: 1rem -1rem 1.08rem;
199         background: rgba(255, 255, 255, 0.08);
200     }
201
202     [data-testid="stSidebar"] .studio-sidebar-section {
203         margin-top: 1.15rem;
204         margin-bottom: 0.15rem;
205         color: rgba(123, 113, 255, 0.96);
206         font-family: "Consolas", "Bahnschrift", monospace;
207         font-size: 0.72rem;
208         font-weight: 700;
209         letter-spacing: 0.14em;
210         text-transform: uppercase;
211     }
212
213     [data-testid="stSidebar"] .studio-sidebar-section-copy {
214         margin-bottom: 0.58rem;
215         color: rgba(238, 243, 255, 0.60);
216         font-size: 0.79rem;
217         line-height: 1.34;
218     }
219
220     [data-testid="stSidebar"] [data-testid="stPageLinkContainer"] {
221         margin-bottom: 0.16rem;
222     }
223
224     [data-testid="stSidebar"] [data-testid="stPageLinkContainer"] a,
225     [data-testid="stSidebar"] [data-testid="stPageLink-NavLink"] {
226         display: flex;
227         align-items: center;
228         min-height: 2.85rem;
229         padding: 0.28rem 0.58rem;
230         border-radius: 16px;

```

```

231         border: 1px solid transparent;
232         background: transparent;
233         text-decoration: none !important;
234     }
235
236     [data-testid="stSidebar"] [data-testid="stPageLinkContainer"] a:hover,
237     [data-testid="stSidebar"] [data-testid="stPageLink-NavLink"]:hover {
238         background: rgba(255, 255, 255, 0.045);
239         border-color: rgba(255, 255, 255, 0.05);
240     }
241
242     [data-testid="stSidebar"] [data-testid="stPageLinkContainer"] a[aria-current="page"],
243     [data-testid="stSidebar"] [data-testid="stPageLink-NavLink"][aria-current="page"] {
244         background: linear-gradient(
245             180deg,
246             rgba(255, 255, 255, 0.085) 0%,
247             rgba(255, 255, 255, 0.06) 100%
248         );
249         border-color: rgba(255, 255, 255, 0.08);
250         box-shadow: inset 0 1px 0 rgba(255, 255, 255, 0.04);
251     }
252
253     [data-testid="stSidebar"] [data-testid="stPageLinkContainer"] p,
254     [data-testid="stSidebar"] [data-testid="stPageLink-NavLink"] p {
255         color: #eef3ff !important;
256         font-size: 0.98rem;
257         font-weight: 700;
258         letter-spacing: 0.005em;
259     }
260
261     [data-testid="stSidebar"] [data-testid="stPageLinkContainer"] svg,
262     [data-testid="stSidebar"] [data-testid="stPageLink-NavLink"] svg {
263         width: 1.1rem;
264         height: 1.1rem;
265         color: rgba(255, 255, 255, 0.68);
266     }
267
268     [data-testid="stSidebar"] [data-testid="stPageLinkContainer"] a[aria-current="page"] svg,
269     [data-testid="stSidebar"] [data-testid="stPageLink-NavLink"][aria-current="page"] svg {
270         color: #eef3ff;
271     }
272
273     @media (max-width: 900px) {
274         [data-testid="stSidebar"][aria-expanded="true"],
275         [data-testid="stSidebar"][aria-expanded="false"] {
276             min-width: 15.5rem !important;
277             max-width: 15.5rem !important;
278             width: 15.5rem !important;
279         }
280
281         [data-testid="stSidebar"][aria-expanded="true"] > div:first-child,
282         [data-testid="stSidebar"][aria-expanded="false"] > div:first-child {
283             width: 15.5rem !important;
284             min-width: 15.5rem !important;
285             max-width: 15.5rem !important;
286         }
287
288         [data-testid="stSidebar"][aria-expanded="false"] + div,
289         [data-testid="stSidebar"][aria-expanded="false"] ~ section {
290             margin-left: 15.5rem !important;
291         }
292
293         [data-testid="stSidebarContent"] {
294             padding: 0.9rem 0.8rem 1.1rem;
295         }
296
297         [data-testid="stSidebar"] .studio-sidebar-brand-title {
298             font-size: 1.9rem;
299         }
300
301         [data-testid="stSidebar"] .studio-sidebar-brand-subtitle {
302             font-size: 1.2rem;
303         }
304     }
305 </style>
306 """
307 ),
308 unsafe_allow_html=True,
309 )

```

C Codi: Paines Streamlit

C.1 pages/___init___.py

Listing 4: pages/___init___.py

```
1 """Streamlit Marcadador de paquet de pàgina per a BEMS-RL Studio."""
```

C.2 pages/Afegir_Entorn.py

Listing 5: pages/Afegir_Entorn.py

```
1 """Sinergym Studio: vista per afegir entorns.
2
3 Aquesta vista gestiona la UI de Streamlit per carregar models d'edifici
4 EnergyPlus (.idf/.epJSON) i fitxers meteorològics (.epw), configurar els
5 actuadors HVAC i la variabilitat climàtica, i registrar un entorn Gym nou a
6 Sinergym mitjançant un fitxer YAML generat.
7 """
8
9 import importlib
10 import subprocess
11 from html import escape
12 from pathlib import Path
13 from typing import Any, Dict, List, Optional
14
15 import pandas as pd
16 import streamlit as st
17 import yaml
18
19 from backend import afegir_entorn_backend as add_env_backend
20 from page_styles.afegir_entorn import inject_add_environment_styles
21 from sidebar_nav import configure_studio_page
22
23 importlib.reload(add_env_backend)
24
25 BUILDINGS_DIR = add_env_backend.BUILDINGS_DIR
26 CFG_DIR = add_env_backend.CFG_DIR
27 WEATHERS_DIR = add_env_backend.WEATHERS_DIR
28 DEFAULT_WEATHER_VARIABILITY = add_env_backend.DEFAULT_WEATHER_VARIABILITY
29
30 build_environment_config = add_env_backend.build_environment_config
31 build_registered_env_id = add_env_backend.build_registered_env_id
32 build_training_analysis = add_env_backend.build_training_analysis
33 build_weather_temperature_preview_figure = add_env_backend.build_weather_temperature_preview_figure
34 build_weather_temperature_preview = add_env_backend.build_weather_temperature_preview
35 build_weather_profile_suggestions = add_env_backend.build_weather_profile_suggestions
36 cleanup_failed_environment = add_env_backend.cleanup_failed_environment
37 convert_idf_to_epjson = add_env_backend.convert_idf_to_epjson
38 detect_idf_version = add_env_backend.detect_idf_version
39 download_transition_tools = add_env_backend.download_transition_tools
40 get_transition_updater_dir = add_env_backend.get_transition_updater_dir
41 needs_idf_upgrade = add_env_backend.needs_idf_upgrade
42 register_environment_from_yaml = add_env_backend.register_environment_from_yaml
43 sanitize_identifier = add_env_backend.sanitize_identifier
44 save_uploaded_bytes = add_env_backend.save_uploaded_bytes
45 upgrade_idf_version = add_env_backend.upgrade_idf_version
46 validate_registered_environment = add_env_backend.validate_registered_environment
47 write_yaml_config = add_env_backend.write_yaml_config
48
49
50 PAGE_TITLE = "Crear Nou Entorn"
51 PAGE_LAYOUT = "wide"
52 INTRODUCTION_TEXT = (
53     "Carrega un edifici i un únic clima, detecta controladors HVAC i bateria "
54     "usables del model i configura l'espai d'accions abans de registrar el nou entorn."
55 )
56
57
58
59 def render_add_environment_hero() -> None:
60     """Renderitza la capçalera principal amb el mateix llenguatge visual de l'app."""
61
62     st.markdown(
63         f"""
64         <section class="add-environment-hero">
65             <div class="hero-kicker">CREACIÓ D'ENTORNS</div>
66             <h1 class="hero-title">{escape(PAGE_TITLE)}</h1>
67             <div class="hero-copy">{escape(INTRODUCTION_TEXT)}</div>
68         </section>
69         """,
70         unsafe_allow_html=True,
71     )
72
73
74 def render_add_environment_section(title: str, kicker: str, description: str = "") -> None:
75     """Renderitza un disseny de capçalera de secció al UI.
76
77     Paràmetres:
```



```

78     title (str): títol de la secció principal.
79     kicker (str): text kicker (etiqueta breu superior) que indica el tema de la secció.
80     description (str, optional): text adicional que explica la secció.
81     """
82
83     st.markdown(f'<h2 class="page-section-title">{escape(title)}</h2>', unsafe_allow_html=True)
84
85
86 def render_add_environment_subsection(title: str, description: str = "") -> None:
87     """Renderitza una capçalera de subsecció a UI.
88
89     Paràmetres:
90         title (str): títol de la subsecció.
91         description (str, optional): context de descripció adicional.
92     """
93
94     st.markdown(f'<h3 class="section-title">{escape(title)}</h3>', unsafe_allow_html=True)
95
96 def render_upload_card_header(target, kicker: str, title: str, description: str) -> None:
97     """Renderitza el bloc de capçalera dins d'un contenidor de targetes de càrrega.
98
99     Paràmetres:
100         target: un generador/contenidor delta Streamlit per escriure.
101         kicker (str): etiqueta petita en majúscula.
102         title (str): títol principal de la targeta.
103         description (str): text d'ajuda que descriu l'acció de càrrega.
104     """
105
106     target.markdown(
107         f"""
108         <div class="upload-card-shell-anchor"></div>
109         <div class="upload-card-kicker">{escape(kicker)}</div>
110         <div class="upload-card-title">{escape(title)}</div>
111         <div class="upload-card-copy">{escape(description)}</div>
112         """,
113         unsafe_allow_html=True,
114     )
115
116
117 def render_weather_source_selector(target=st) -> str:
118     """Renderitza un control segmentat o un grup de ràdio per seleccionar la font meteorològica.
119
120     Paràmetres:
121         target: un generador/contenidor delta Streamlit per vincular el giny. Per defecte a st.
122
123     Retorna:
124         str: El mode seleccionat ("Escollir existents" o "Pujar nous fitxers").
125     """
126
127     options = ["Escollir existents", "Pujar nous fitxers"]
128     current_value = st.session_state.get("add_env_weather_mode", options[0])
129     target.markdown('<div class="studio-segmented-pill-anchor"></div>', unsafe_allow_html=True)
130
131     if hasattr(target, "segmented_control"):
132         selected_value = target.segmented_control(
133             "Origen del clima",
134             options,
135             default=current_value,
136             key="add_env_weather_mode",
137         )
138         return selected_value or current_value
139
140     return target.radio(
141         "Origen del clima",
142         options,
143         horizontal=True,
144         key="add_env_weather_mode",
145     )
146
147
148 def format_current_schedule_range(option) -> str:
149     """Formata l'interval numèric vàlid d'un actuator per mostrar-lo en funció de la seva planificació inicial.
150
151     Paràmetres:
152         option (ActuatorOption): l'opció de l'actuator detectada des del model.
153
154     Retorna:
155         str: una cadena preformatada que presenta l'interval numèric detectat.
156     """
157     if option.current_low is None or option.current_high is None:
158         return "Sense resum automàtic"
159     if option.category == "availability":
160         if abs(option.current_low - option.current_high) < 1e-6:
161             return f"{option.current_low:.0f}"
162         return f"{option.current_low:.0f} - {option.current_high:.0f}"
163     if abs(option.current_low - option.current_high) < 1e-6:
164         return f"{option.current_low:.2f}"
165     return f"{option.current_low:.2f} - {option.current_high:.2f}"
166
167
168 def render_controller_selection_table(
169     title: str,

```

```

170 options,
171 *,
172 default_selected: bool,
173 key: str,
174 empty_message: str,
175 ):
176     """Renderitza una taula de DataFrame interactiva per incloure/excloure controladors HVAC.
177
178     Paràmetres:
179         title (str): l'encapçalament que es mostra a sobre de la taula.
180         options (Sequence[ActuatorOption]): les opcions de l'actuador disponibles.
181         default_selected (bool): Si True, les caselles de selecció estan marcades inicialment.
182         key (str): clau única d'estat de sessió Streamlit per a l'editor de dades.
183         empty_message (str): Text alternativa quan no hi ha opcions per mostrar.
184
185     Retorna:
186         List[str]: una llista de les cadenes `option_id` seleccionades.
187     """
188     if not options:
189         st.caption(empty_message)
190         return []
191
192     st.markdown(f"***{title}***")
193     selection_df = pd.DataFrame(
194         [
195             {
196                 "Afegir": default_selected,
197                 "Control": option.label,
198                 "Afecta": option.reference,
199                 "Programa actual": format_current_schedule_range(option),
200             }
201             for option in options
202         ],
203         index=[option.option_id for option in options],
204     )
205     edited_selection_df = st.data_editor(
206         selection_df,
207         hide_index=True,
208         use_container_width=True,
209         key=key,
210         disabled=["Control", "Afecta", "Programa actual"],
211         column_config={
212             "Afegir": st.column_config.CheckboxColumn("Incloure", width="small"),
213             "Control": st.column_config.TextColumn("Controlador"),
214             "Afecta": st.column_config.TextColumn("Afecta"),
215             "Programa actual": st.column_config.TextColumn("Programa actual"),
216         },
217     )
218     return [
219         option_id
220         for option_id, row in edited_selection_df.iterrows()
221         if bool(row["Afegir"])
222     ]
223
224
225 def render_building_upload_card() -> Any:
226     """Renderitza la targeta de càrrega del fitxer d'edifici i retorna l'objecte del fitxer penjat.
227
228     Retorna:
229         Any: l'objecte del fitxer carregat Streamlit o None si no s'ha carregat cap fitxer.
230     """
231     building_card = st.container(border=True)
232     render_upload_card_header(
233         building_card,
234         "Edifici",
235         "Fitxer d'edifici",
236         "Puja un fitxer IDF o epJSON per preparar el model base.",
237     )
238     return building_card.file_uploader(
239         "Fitxer d'edifici",
240         type=["idf", "epjson", "epJSON"],
241         label_visibility="collapsed",
242     )
243
244
245 def render_weather_upload_card() -> tuple[Path, Any]:
246     """Mostra la targeta del fitxer meteorològic i torna el camí resolt al fitxer EPW seleccionat.
247
248     Gestiona tant el selector de fitxers existents com els modes de càrrega de fitxers nous.
249     Crida st.stop() si no hi ha cap fitxer meteorològic vàlid disponible o seleccionat.
250
251     Retorna:
252         tuple[Path, Any]: el camí del fitxer EPW seleccionat i el seu contenidor de la targeta Streamlit.
253     """
254     climate_card = st.container(border=True)
255     render_upload_card_header(
256         climate_card,
257         "Clima",
258         "Fitxer climàtic",
259         "Escull un .epw existent o puja'n un de nou per registrar l'entorn.",
260     )
261     mode_weather = render_weather_source_selector(climate_card)

```

```

262
263 selected_weather_path = None
264
265 if mode_weather == "Escollir existents":
266     weather_files = sorted(weather_file.name for weather_file in WEATHERS_DIR.glob("*.epw"))
267     if not weather_files:
268         st.error("No hi ha fitxers .epw disponibles al catàleg actual.")
269         st.stop()
270     with climate_card:
271         selected_weather_name = st.selectbox(
272             "Selecciona el fitxer de clima",
273             weather_files,
274         )
275     selected_weather_path = WEATHERS_DIR / selected_weather_name
276 else:
277     with climate_card:
278         uploaded_weather_file = st.file_uploader(
279             "Puja un fitxer .epw",
280             type=["epw"],
281             key="weather_upload",
282         )
283     if uploaded_weather_file:
284         weather_upload_token = (uploaded_weather_file.name, uploaded_weather_file.size)
285         cached_weather_path = st.session_state.get("add_env_weather_path")
286         cached_weather_file = Path(cached_weather_path) if cached_weather_path else None
287         if (
288             st.session_state.get("add_env_weather_token") != weather_upload_token
289             or cached_weather_file is None
290             or not cached_weather_file.exists()
291         ):
292             cached_weather_file = WEATHERS_DIR / uploaded_weather_file.name
293             save_uploaded_bytes(cached_weather_file, uploaded_weather_file.read())
294             st.session_state.add_env_weather_token = weather_upload_token
295             st.session_state.add_env_weather_path = str(cached_weather_file)
296             selected_weather_path = cached_weather_file
297         with climate_card:
298             st.success(f"Fitxer de clima carregat: {uploaded_weather_file.name}")
299
300 if selected_weather_path is None:
301     st.info("Selecciona un fitxer climàtic per definir l'entorn.")
302     st.stop()
303
304 return selected_weather_path, climate_card
305
306
307 def render_weather_variability_section() -> tuple[Optional[Dict], bool]:
308     """Renderitza la secció de configuració de la variabilitat meteorològica estocàstica opcional.
309
310     Retorna:
311         tuple[Optional[Dict], bool]: configuració de la variabilitat del clima i si
312         les variants estocàstiques estan habilitades.
313     """
314     render_add_environment_subsection("Variabilitat climàtica opcional")
315     variability_card = st.container(border=True)
316     variability_card.markdown('<div class="weather-variability-card-anchor"></div>', unsafe_allow_html=True)
317     enable_stochastic = variability_card.checkbox(
318         "Generar també variants estocàstiques",
319         value=True,
320         help="Quan s'activa, es registren també entorns amb el sufix -continuous-stochastic-v1.",
321     )
322
323     if not enable_stochastic:
324         return None, False
325
326     default_sigma, default_mu, default_tau = DEFAULT_WEATHER_VARIABILITY["Dry Bulb Temperature"]
327     sigma_col, mu_col, tau_col = variability_card.columns(3)
328     with sigma_col:
329         sigma = st.number_input("Sigma", min_value=0.0, value=float(default_sigma), step=0.1)
330     with mu_col:
331         mu = st.number_input("Mu", value=float(default_mu), step=0.1)
332     with tau_col:
333         tau = st.number_input("Tau (hores)", min_value=0.1, value=float(default_tau), step=1.0)
334
335     return {"Dry Bulb Temperature": [float(sigma), float(mu), float(tau)]}, True
336
337
338 @st.cache_data(show_spinner=False)
339 def load_weather_temperature_preview(
340     weather_path: str,
341     weather_mtime_ns: int,
342     sigma: float,
343     mu: float,
344     tau: float,
345 ) -> List[Dict[str, float]]:
346     """Carrega les dades de previsualització meteorològica anual a la memòria cau per al fitxer EPW seleccionat.
347
348     Paràmetres:
349         weather_path (str): camí cap al fitxer meteorològic EPW.
350         weather_mtime_ns (int): marca de temps de modificació de fitxers utilitzada com a clau d'invalidació de la memòria cau.
351         sigma (float): paràmetre de desviació estàndard per al procés d'OU.
352         mu (float): paràmetre mitjà per al procés d'OU.
353         tau (float): paràmetre constant de temps per al procés d'OU.

```

```

354
355     Retorna:
356         List[Dict[str, float]]: registres de previsualització diaris preparats per a la traça.
357     """
358
359     del weather_mtime_ns
360     return build_weather_temperature_preview(Path(weather_path), sigma, mu, tau)
361
362
363 def render_weather_temperature_preview(
364     selected_weather_path: Path,
365     weather_variability: Optional[Dict],
366 ) -> None:
367     """Mostra la previsualització anual de la temperatura EPW per als canvis meteorològics estocàstics.
368
369     Paràmetres:
370         selected_weather_path (Path): fitxer EPW seleccionat o carregat actualment.
371         weather_variability (Optional[Dict]): configuració activa de la variabilitat del clima.
372     """
373
374     if not weather_variability:
375         return
376
377     sigma, mu, tau = weather_variability["Dry Bulb Temperature"]
378     try:
379         weather_mtime_ns = selected_weather_path.stat().st_mtime_ns
380     except OSError:
381         weather_mtime_ns = 0
382
383     preview_records = load_weather_temperature_preview(
384         str(selected_weather_path),
385         weather_mtime_ns,
386         float(sigma),
387         float(mu),
388         float(tau),
389     )
390     if not preview_records:
391         st.info("No s'ha pogut llegir la temperatura seca del fitxer EPW seleccionat.")
392         return
393
394     preview_card = st.container(border=True)
395     preview_card.markdown(
396         """
397         <div class="weather-preview-card-anchor"></div>
398         <div class="upload-card-kicker">Clima modificat</div>
399         <div class="upload-card-title">Previsualització anual de temperatura</div>
400         """,
401         unsafe_allow_html=True,
402     )
403     preview_card.plotly_chart(
404         build_weather_temperature_preview_figure(preview_records),
405         use_container_width=True,
406         config={"displayModeBar": False},
407     )
408
409
410 def prepare_building_file(uploaded_file: Any) -> tuple[Path, Path]:
411     """Desa, actualitza si cal i converteix el fitxer de construcció penjat a epJSON.
412
413     Utilitza l'estat de la sessió per emmagatzemar el resultat a la memòria cau i evitar el processament redundant en les
414     reexecucions.
415     Crida st.stop() per errors irreversibles.
416
417     Paràmetres:
418         uploaded_file: l'objecte de fitxer carregat Streamlit per al model d'edifici.
419
420     Retorna:
421         Tuple[Path, Path]: el camí desat en brut (tmp_path) i el camí final epJSON (building_path).
422     """
423     uploaded_token = (uploaded_file.name, uploaded_file.size)
424     cached_tmp_path = st.session_state.get("add_env_tmp_path")
425     cached_building_path = st.session_state.get("add_env_building_path")
426
427     tmp_path = Path(cached_tmp_path) if cached_tmp_path else None
428     building_path = Path(cached_building_path) if cached_building_path else None
429
430     # Streamlit reexecuta la pàgina cada vegada que es toca un widget; aquest token evita
431     # reconvertir el mateix fitxer una vegada i una altra.
432     needs_prepare = (
433         st.session_state.get("add_env_current_building_upload_token") != uploaded_token
434         or tmp_path is None
435         or building_path is None
436         or not tmp_path.exists()
437         or not building_path.exists()
438     )
439
440     if not needs_prepare:
441         return Path(st.session_state["add_env_tmp_path"]), Path(st.session_state["add_env_building_path"])
442
443     st.session_state.add_env_current_building_upload_token = uploaded_token
444     tmp_path = BUILDINGS_DIR / uploaded_file.name
445     save_uploaded_bytes(tmp_path, uploaded_file.read())

```

```

445
446 if tmp_path.suffix.lower() == ".idf":
447     with st.spinner("Escanejant i actualitzant l'arxiu IDF, si cal..."):
448         detected_version = detect_idf_version(tmp_path)
449         if detected_version and needs_idf_upgrade(detected_version):
450             # EnergyPlus només converteix bé IDF de versions properes. Si el fitxer és
451             # antic, el passem primer pels Transition tools oficials.
452             updater_dir = get_transition_updater_dir()
453             if updater_dir is None:
454                 with st.spinner("Descarregant les eines de transició d'EnergyPlus..."):
455                     updater_dir = download_transition_tools()
456
457             if updater_dir is not None:
458                 st.info(
459                     "S'ha detectat un fitxer IDF antic "
460                     f"({detected_version[0]}.{detected_version[1]}). "
461                     "S'actualitzarà automàticament abans de convertir-lo."
462                 )
463                 try:
464                     upgrade_result = upgrade_idf_version(tmp_path, updater_dir)
465                 except subprocess.TimeoutExpired:
466                     st.error(
467                         "L'actualització automàtica de l'IDF ha superat el temps límit. "
468                         "Revisa el fitxer o converteix-lo prèviament abans de continuar."
469                     )
470                     st.stop()
471                 if upgrade_result.failed_transition_version is not None:
472                     failed_major, failed_minor = upgrade_result.failed_transition_version
473                     st.warning(
474                         "La transició automàtica ha fallat a la versió "
475                         f"{failed_major}.{failed_minor}. Revisa l'IDF abans de continuar."
476                     )
477                 elif upgrade_result.upgraded and upgrade_result.final_version is not None:
478                     st.success(
479                         "L'arxiu s'ha actualitzat correctament a la versió "
480                         f"{upgrade_result.final_version[0]}.{upgrade_result.final_version[1]}."
481                     )
482
483 with st.spinner("Convertint i preparant el model de l'edifici..."):
484     try:
485         # A partir d'aquí treballem sempre amb epJSON; simplifica l'anàlisi posterior
486         # i evita duplicar parser per IDF i epJSON.
487         building_path = (
488             convert_idf_to_epjson(tmp_path) if tmp_path.suffix.lower() == ".idf" else tmp_path
489         )
490     except FileNotFoundError:
491         st.error("No s'ha trobat `energyplus`. Assegura't que està instal·lat i disponible al PATH.")
492         st.stop()
493     except subprocess.TimeoutExpired:
494         st.error(
495             "La conversió o preparació del model ha superat el temps límit. "
496             "Revisa el fitxer d'edifici o prova una conversió prèvia a epJSON."
497         )
498         st.stop()
499     except Exception as error:
500         st.error(f"Error en la conversió amb EnergyPlus: {error}")
501         st.stop()
502
503 st.session_state.add_env_tmp_path = str(tmp_path)
504 st.session_state.add_env_building_path = str(building_path)
505 st.success(f"Fitxer carregat i preparat correctament: {building_path.name}")
506 return tmp_path, building_path
507
508
509 def load_or_run_training_analysis(building_path: Path, weather_path: Path):
510     """Retorna la memòria cau BuildingTrainingAnalysis o l'executa nova si les entrades han canviat.
511
512     Paràmetres:
513         building_path (Path): camí cap al model d'edifici epJSON.
514         weather_path (Path): camí cap al fitxer meteorològic EPW.
515
516     Retorna:
517         BuildingTrainingAnalysis: Detectats termòstats, actuadors, variables i comptadors.
518     """
519     analysis_cache_key = (str(building_path.resolve()), str(weather_path.resolve()))
520     if st.session_state.get("add_env_training_analysis_key") != analysis_cache_key:
521         with st.spinner("Detectant termòstats i actuadors del model..."):
522             st.session_state.add_env_training_analysis = build_training_analysis(
523                 building_path,
524                 weather_path,
525                 probe_handlers=False,
526             )
527         st.session_state.add_env_training_analysis_key = analysis_cache_key
528     return st.session_state.add_env_training_analysis
529
530
531 def render_environment_id_section(
532     building_path: Path,
533     weather_profiles: List[Dict],
534     enable_stochastic: bool,
535 ) -> str:
536     """Renderitza l'entrada d'ID de base de l'entorn i previsualitza tots els ID que es registraran.

```

```

537
538 Paràmetres:
539     building_path (Path): S'utilitza per obtenir un nom base predeterminat de la base del fitxer.
540     weather_profiles (List[Dict]): perfils meteorològics per calcular tots els ID generats.
541     enable_stochastic (bool): si s'han d'incloure els ID de variants estocàstiques.
542
543 Retorna:
544     str: l'identificador de l'entorn base validat i netejat.
545     """
546     render_add_environment_section(
547         "Configuració del nou entorn",
548         "Nom i variants",
549     )
550
551     with st.container(border=True):
552         default_id_base = sanitize_identifier(building_path.stem, "nou_entorn")
553         id_base_input = st.text_input("***ID base de l'entorn***", value=default_id_base)
554         id_base = sanitize_identifier(id_base_input, default_id_base)
555         if id_base != id_base_input:
556             st.caption(f"L'identificador base es registrarà com a `{id_base}`.")
557
558         generated_env_ids = [build_registered_env_id(id_base, profile["key"]) for profile in weather_profiles]
559         if enable_stochastic:
560             generated_env_ids.extend(
561                 build_registered_env_id(id_base, profile["key"], stochastic=True)
562                 for profile in weather_profiles
563             )
564         st.markdown(
565             "<p style='font-size: 0.875rem; font-weight: 600; margin-bottom: 0.2rem; color: var(--text-color);'>IDs que es
566             registraràn:</p>",
567             unsafe_allow_html=True,
568         )
569         st.code("\n".join(generated_env_ids), language="text")
570
571     return id_base
572
573 def render_controller_selection_section(analysis) -> tuple[dict, list]:
574     """Renderitza la secció de detecció del controlador i retorna els ID de l'actuador seleccionats.
575
576     Crida st.stop() si no hi ha actuadors disponibles o no n'hi ha cap seleccionat.
577
578     Paràmetres:
579         analysis (BuildingTrainingAnalysis): resultat de l'anàlisi de l'edifici detectat.
580
581     Retorna:
582         Tuple[dict, list]: les opcions completes de l'actuador en forma de diccionari i la llista d'ID d'opcions seleccionades.
583     """
584     render_add_environment_section(
585         "Controladors detectats",
586         "Selecció",
587     )
588
589     detected_actuator_options = [
590         *list(analysis.thermostat_actuators),
591         *list(analysis.hvac_actuators),
592     ]
593     actuator_options = {option.option_id: option for option in detected_actuator_options}
594
595     if not actuator_options:
596         st.error(
597             "No s'ha detectat cap controlador usable al model. "
598             "Cal preparar prèviament l'edifici amb termòstats, setpoints, bateria o altres controls editables abans de registrar l'
599             entorn."
600         )
601         st.stop()
602
603     thermostat_options = list(analysis.thermostat_actuators)
604     scheduled_setpoint_options = [
605         option for option in analysis.hvac_actuators if option.category == "scheduled_temperature_setpoint"
606     ]
607     availability_options = [
608         option for option in analysis.hvac_actuators if option.category == "availability"
609     ]
610     battery_options = [
611         option for option in analysis.hvac_actuators if option.category in {"battery_charge", "battery_discharge"}
612     ]
613
614     # Separem els controladors per família perquè l'usuari pugui deixar fora actuadors
615     # massa experimentals sense perdre els termòstats principals.
616     st.caption(
617         f"S'han detectat {len(detected_actuator_options)} controladors usables. "
618         "Selecciona només els que vulguis incloure."
619     )
620
621     selected_actuator_ids = []
622     selected_actuator_ids.extend(
623         render_controller_selection_table(
624             "Termòstats de zona",
625             thermostat_options,
626             default_selected=True,
627             key="add_env_select_thermostats",
628         )

```

```

627         empty_message="No s'han detectat termòstats de zona seleccionables.",
628     )
629 )
630 selected_actuator_ids.extend(
631     render_controller_selection_table(
632         "Setpoints de sistema HVAC",
633         scheduled_setpoint_options,
634         default_selected=False,
635         key="add_env_select_hvac_setpoints",
636         empty_message="No s'han detectat setpoints HVAC programats addicionals.",
637     )
638 )
639 selected_actuator_ids.extend(
640     render_controller_selection_table(
641         "Disponibilitat HVAC",
642         availability_options,
643         default_selected=False,
644         key="add_env_select_hvac_availability",
645         empty_message="No s'han detectat controls de disponibilitat HVAC.",
646     )
647 )
648 selected_actuator_ids.extend(
649     render_controller_selection_table(
650         "Bateria",
651         battery_options,
652         default_selected=True,
653         key="add_env_select_battery",
654         empty_message="No s'han detectat controls de bateria.",
655     )
656 )
657
658 selected_actuator_ids = list(dict.fromkeys(selected_actuator_ids))
659 if not selected_actuator_ids:
660     st.info("Selecciona com a mínim un controlador per poder definir l'espai d'accions.")
661     st.stop()
662
663 return actuator_options, selected_actuator_ids
664
665
666 def render_action_space_section(actuator_options: dict, selected_actuator_ids: List[str]) -> dict:
667     """Renderitza l'editor de límits de l'espai d'acció i retorna els límits de l'actuador validats.
668
669     Crida a st.stop() si algun actuador té un límit no vàlid (mínim >= màxim).
670
671     Paràmetres:
672         actuator_options (dict): Dict complet dels objectes ActuatorOption disponibles per ID.
673         selected_actuator_ids (List[str]): ID dels actuadors seleccionats per l'usuari.
674
675     Retorna:
676         dict: Mapeig de option_id a tuples flotants (baix, alt).
677     """
678     render_add_environment_section(
679         "Espai d'accions",
680         "Rangs",
681     )
682
683     selected_actuator_options = [actuator_options[option_id] for option_id in selected_actuator_ids]
684     controls_df = pd.DataFrame(
685         [
686             {
687                 "Control": option.label,
688                 "Afecta": option.reference,
689                 "Programa actual": format_current_schedule_range(option),
690                 "Mínim": float(option.default_low),
691                 "Màxim": float(option.default_high),
692             }
693             for option in selected_actuator_options
694         ],
695         index=[option.option_id for option in selected_actuator_options],
696     )
697     edited_controls_df = st.data_editor(
698         controls_df,
699         hide_index=True,
700         use_container_width=True,
701         key="add_env_actuator_bounds_editor",
702         disabled=["Control", "Afecta", "Programa actual"],
703         column_config={
704             "Control": st.column_config.TextColumn("Controlador"),
705             "Afecta": st.column_config.TextColumn("Afecta"),
706             "Programa actual": st.column_config.TextColumn("Programa actual"),
707             "Mínim": st.column_config.NumberColumn("Mínim", step=0.5, format="%.2f"),
708             "Màxim": st.column_config.NumberColumn("Màxim", step=0.5, format="%.2f"),
709         },
710     )
711
712     actuator_bounds = {}
713     bounds_are_valid = True
714     for option_id, row in edited_controls_df.iterrows():
715         low_value = float(row["Mínim"])
716         high_value = float(row["Màxim"])
717         if low_value >= high_value:
718             bounds_are_valid = False

```



```

719     actuator_bounds[option_id] = (low_value, high_value)
720
721     if not bounds_are_valid:
722         st.error("Cada actuador ha de tenir un límit mínim estrictament inferior al màxim.")
723         st.stop()
724
725     return actuator_bounds
726
727
728 def render_registration_section(
729     id_base: str,
730     building_path: Path,
731     tmp_path: Path,
732     weather_profiles: List[Dict],
733     analysis,
734     actuator_options: dict,
735     selected_actuator_ids: List[str],
736     actuator_bounds: dict,
737     weather_variability: Optional[Dict],
738 ) -> None:
739     """Renderitza el botó de registre i gestiona la creació i validació de l'entorn.
740
741     En cas d'èxit, mostra el YAML generat i executa una simulació de validació ràpida.
742     En cas d'error, reverteix els fitxers escrits parcialment i mostra els detalls de l'error.
743
744     Paràmetres:
745     id_base (str): identificador de base netejat per al entorn.
746     building_path (Path): Camí al fitxer de construcció epJSON preparat.
747     tmp_path (Path): camí cap al fitxer de construcció original penjat.
748     weather_profiles (List[Dict]): llista de perfils meteorològics utilitzat per a la generació de la configuració.
749     analysis (BuildingTrainingAnalysis): resultat de l'anàlisi de l'edifici detectat.
750     actuator_options (dict): totes les opcions de l'actuador detectades teclejades per option_id.
751     selected_actuator_ids (List[str]): ID dels actuadors seleccionats per l'usuari.
752     actuator_bounds (dict): Mapeig de option_id a tuples de lligat (baix, alt).
753     weather_variability (Optional[Dict]): configuració de la variabilitat del clima estocàstic, o None.
754     """
755     render_add_environment_section(
756         "Creació i registre",
757         "Generació",
758     )
759
760     if not st.button("Crear entorn automàticament"):
761         return
762
763     cfg_path = CFG_DIR / f"{id_base}.yaml"
764     selected_actuator_objects = [actuator_options[option_id] for option_id in selected_actuator_ids]
765
766     try:
767         # Primer escrivim i registrem l'entorn. Si qualsevol pas falla, netegem el YAML
768         # i el fitxer temporal per no deixar configuracions mig creades.
769         env_cfg = build_environment_config(
770             id_base=id_base,
771             building_file_name=building_path.name,
772             weather_profiles=weather_profiles,
773             analysis=analysis,
774             selected_actuators=selected_actuator_objects,
775             actuator_bounds=actuator_bounds,
776             weather_variability=weather_variability,
777         )
778         write_yaml_config(cfg_path, env_cfg)
779         register_environment_from_yaml(cfg_path)
780     except Exception as error:
781         st.error(f"No s'ha pogut construir o registrar l'entorn: {error}")
782         cleanup_messages = cleanup_failed_environment(cfg_path, tmp_path)
783         for level, message in cleanup_messages:
784             if level == "warning":
785                 st.warning(message)
786             else:
787                 st.error(message)
788         st.stop()
789
790     st.success(f"Entorn creat i registrat correctament: {id_base}")
791     with st.expander("Veure el YAML generat"):
792         st.code(yaml.dump(env_cfg, sort_keys=False, allow_unicode=True), language="yaml")
793
794     validation_env_id = build_registered_env_id(id_base, weather_profiles[0]["key"])
795     try:
796         # La validació curta és una prova de fum: si falla aquí, fallaria després en training.
797         with st.spinner(f"Mutant l'entorn {validation_env_id} per validar la configuració..."):
798             validate_registered_environment(validation_env_id, cfg_path)
799     except Exception as error:
800         st.error(f"No s'ha pogut validar l'entorn registrat. Es revertirà la configuració: {error}")
801         cleanup_messages = cleanup_failed_environment(cfg_path, tmp_path)
802         for level, message in cleanup_messages:
803             if level == "warning":
804                 st.warning(message)
805             else:
806                 st.error(message)
807         st.stop()
808
809     st.success(
810         "S'ha realitzat una validació ràpida de simulació amb l'entorn registrat procedimentalment de Sinergym. "

```



```

811     "Tots els espais (accions, observacions) responen de manera fluida i sense errors!"
812 )
813
814
815 def main() -> None:
816     """Punt d'entrada a la pàgina Afegir Entorn Streamlit.
817
818     Organitza el flux complet de la pàgina: estils, heroi, càrregues de fitxers, processament d'edificis,
819     detecció d'actuadors, configuració de l'espai d'acció i registre de l'entorn.
820     """
821     configure_studio_page(PAGE_TITLE, "pages/Afegir_Entorn.py", layout=PAGE_LAYOUT)
822     inject_add_environment_styles()
823     render_add_environment_hero()
824
825     render_add_environment_section("Pujar fitxers", "Preparació")
826     building_column, climate_column = st.columns([1, 1.35], gap="large")
827     with building_column:
828         uploaded_file = render_building_upload_card()
829     with climate_column:
830         selected_weather_path, climate_card = render_weather_upload_card()
831
832     selected_weather_paths = [selected_weather_path]
833     weather_profile_suggestions = build_weather_profile_suggestions(selected_weather_paths)
834     with climate_card:
835         st.caption(weather_profile_suggestions[0].climate_label)
836     weather_profiles = [
837         {"key": suggestion.suggested_key, "weather_file": suggestion.file_name}
838         for suggestion in weather_profile_suggestions
839     ]
840
841     weather_variability, enable_stochastic = render_weather_variability_section()
842     render_weather_temperature_preview(selected_weather_path, weather_variability)
843
844     if not uploaded_file:
845         st.info("Puja un fitxer d'edifici per continuar amb la detecció de termòstats i actuadors.")
846         st.stop()
847
848     tmp_path, building_path = prepare_building_file(uploaded_file)
849     analysis = load_or_run_training_analysis(building_path, selected_weather_paths[0])
850
851     if not analysis.thermostat_zones:
852         st.error(
853             "No s'han detectat zones amb termòstat. Aquesta alta automàtica està pensada per edificis amb heating/cooling setpoints configurables."
854         )
855         st.stop()
856
857     id_base = render_environment_id_section(building_path, weather_profiles, enable_stochastic)
858     actuator_options, selected_actuator_ids = render_controller_selection_section(analysis)
859     actuator_bounds = render_action_space_section(actuator_options, selected_actuator_ids)
860
861     render_registration_section(
862         id_base=id_base,
863         building_path=building_path,
864         tmp_path=tmp_path,
865         weather_profiles=weather_profiles,
866         analysis=analysis,
867         actuator_options=actuator_options,
868         selected_actuator_ids=selected_actuator_ids,
869         actuator_bounds=actuator_bounds,
870         weather_variability=weather_variability,
871     )
872
873
874 main()

```

C.3 pages/Ajuda.py

Listing 6: pages/Ajuda.py

```

1  from html import escape
2
3  import streamlit as st
4  from sidebar_nav import configure_studio_page
5  from ui_theme import inject_studio_theme
6
7
8  PAGE_TITLE = "Ajuda"
9  INTRODUCTION_TEXT = (
10     "BEMS-RL STUDIO permet controlar el cicle complet de l'entrenament "
11     "d'un agent RL aplicat a simulacions d'EnergyPlus."
12 )
13  INTRODUCTION_HINT = (
14     "A continuació trobaràs una explicació detallada de cada funcionalitat "
15     "disponible al menú de l'esquerra."
16 )
17  RESOURCES = (
18

```

```

19     "Documentació de Sinergym",
20     "https://ugr-sail.github.io/sinergym/compilation/main/index.html",
21 ),
22 ("Stable Baselines 3", "https://stable-baselines3.readthedocs.io/"),
23 ("EnergyPlus", "https://energyplus.net/"),
24 ("Gymnasium", "https://gymnasium.farama.org/"),
25 )
26 SUPPORT_CONTACT = {
27     "name": "Guillem Torm",
28     "email": "guillem@suno.cat",
29 }
30 HELP_SECTIONS = (
31     (
32         "1. Inici",
33         """
34         - Objectiu: Oferir una entrada ràpida al flux complet de treball: crear o revisar entorns, entrenar agents, simular
35         baselines, avaluar models i analitzar resultats.
36         - Quan usar-ho: Quan vulguis situar-te abans de començar una sessió o saltar cap a una tasca concreta des de la barra
37         lateral.
38         - Consell: Si no tens clar per on començar, segueix l'ordre natural: Crear Entorn o Mostrar Entorn, després 
39         Simular Entorn, Entrenar Agent, Avaluar Agent i finalment Resultats.
40         """,
41     ),
42     (
43         "2. Crear Entorn",
44         """
45         - Objectiu: Registrar nous entorns Sinergym a partir d'un edifici, un fitxer climàtic i la configuració d'accions,
46         observacions i actius energètics.
47         - Què pots fer: Pujar o seleccionar un clima EPW, detectar controladors HVAC i bateria disponibles, ajustar l'espai d'
48         accions i revisar el YAML generat abans de crear l'entorn.
49         - Consell: Revisa el YAML final i comprova que les variables d'observació i actuació existeixen al model abans de llançar
50         entrenaments llargs.
51         """,
52     ),
53     (
54         "3. Mostrar Entorn",
55         """
56         - Objectiu: Inspeccionar els entorns registrats i entendre què veu i què pot controlar l'agent.
57         - Què pots consultar: Observacions, accions, configuració de reward, clima associat, geometria, zones, actius energètics
58         i configuració crua.
59         - Consell: Usa aquesta pàgina per validar compatibilitats abans d'entrenar: un entorn discret encaixa millor amb
60         algorismes discrets, mentre que un entorn continu requereix polítiques preparades per accions contínues.
61         """,
62     ),
63     (
64         "4. Entrenar Agent",
65         """
66         - Objectiu: Configurar i llançar entrenaments amb Stable Baselines3 sobre entorns Sinergym.
67         - Què pots configurar: Entorn, algorisme, política, reward, wrappers, paràmetres SB3 avançats, timesteps i configuració
68         climàtica o de cost energètic quan la reward ho necessita.
69         - Consell: Dona noms curts i descriptius a les execucions, sense espais ni caràcters especials, per facilitar la lectura
70         dels resultats i evitar problemes amb rutes de fitxer.
71         """,
72     ),
73     (
74         "5. Resultats",
75         """
76         - Objectiu: Analitzar execucions guardades i convertir els CSV d'entrenament, simulació o avaluació en gràfics i KPIs.
77         - Què pots fer: Seleccionar una execució, obrir el dashboard integrat, comparar agregacions horàries, diàries, mensuals o
78         sèries reals, revisar confort per zones i descarregar dades.
79         - Consell: No eliminis els fitxers de monitoratge d'una execució si encara vols visualitzar-ne el dashboard; la pàgina
80         necessita els CSV generats durant el procés.
81         """,
82     ),
83     (
84         "6. Simular Entorn",
85         """
86         - Objectiu: Executar un baseline d'EnergyPlus sense control RL per tenir una referència física i energètica de l'entorn.
87         - Què pots fer: Triar l'entorn, ajustar la configuració del baseline, iniciar o cancel·lar la simulació i comparar-ne els
88         resultats amb execucions guardades.
89         - Consell: Genera sempre un baseline abans de valorar si un agent RL millora el comportament; així pots comparar consum,
90         confort i desviacions amb una referència clara.
91         """,
92     ),
93     (
94         "7. Avaluar Agent",
95         """
96         - Objectiu: Carregar un model entrenat i executar una avaluació controlada sobre un entorn Sinergym.
97         - Què pots fer: Seleccionar models `.zip`, revisar metadades detectades, ajustar wrappers manuals si el model no inclou
98         tota la configuració i iniciar, cancel·lar o reiniciar l'avaluació.
99         - Consell: Mantén l'entorn, la normalització i els wrappers coherents amb l'entrenament original; canvis petits poden
100        alterar molt les accions que produeix la política.
101        """,
102     ),
103     (
104         "8. Control en Viu",
105         """
106         - Objectiu: Interaccionar pas a pas amb el simulador, ja sigui deixant actuar l'agent o provant accions manuals.
107         - Què pots fer: Inicialitzar un entorn, carregar un model opcional, generar observacions aleatòries, avançar passos amb l'
108         agent o aplicar accions manuals i veure l'impacte en els KPIs.
109         - Consell: Fes servir aquest mode per diagnosticar decisions concretes, no per substituir una avaluació completa. Quan
110        acabis, atura o tanca la sessió abans de canviar de pàgina.

```

```

93     """
94 ),
95 (
96     "9. Arxius",
97     """
98     - **Objectiu:** Gestionar fitxers del projecte des de la interfície sense obrir el terminal.
99     - **Què pots organitzar:** Entrenaments, scripts, models, fitxers climàtics i configuracions d'entorns. També pots pujar nous
100     fitxers o eliminar artefactes que ja no necessitis.
101     - **Consell:** Abans d'esborrar una carpeta d'entrenament, comprova si encara la necessites a **Resultats** o com a
102     referència d'avaluació.
103     """
104 ),
105 (
106     "10. Visor EPW",
107     """
108     - **Objectiu:** Explorar fitxers climàtics EPW abans d'usar-los en entorns o simulacions.
109     - **Què pots veure:** Resum climàtic, metadades, sèries temporals, patrons mensuals, distribucions i dades tabulars del
110     fitxer.
111     - **Consell:** Consulta el visor abans de crear un entorn nou; ajuda a detectar climes incoherents, períodes estranys o
112     diferències fortes entre ciutats.
113     """
114 ),
115 (
116     "Consells generals",
117     """
118     - **Ordre recomanat:** Primer valida l'entorn, després genera un baseline, entrena l'agent, avalua'l i compara els resultats
119     amb el dashboard.
120     - **Compatibilitat:** Comprova sempre que l'algorisme, l'espai d'accions i els wrappers coincideixen amb l'entorn i amb el
121     model que vols carregar.
122     - **Execucions llargues:** Evita tancar el navegador o canviar de pàgina mentre una simulació, entrenament o avaluació està
123     en curs.
124     - **Noms d'execució:** Usa noms llegibles i estables, com `PPO_Radiant_Hivern` o `SAC_OfficeGrid_Test01`.
125     - **Comparació justa:** Quan comparis models, intenta mantenir el mateix clima, horitzó temporal, reward i configuració de
126     confort.
127     - **Neteja:** Usa el gestor d'arxius amb prudència: eliminar models o monitoratges pot deixar sense dades les pàgines d'
128     avaluació i resultats.
129     """
130 ),
131 ),
132 )
133
134 def inject_help_styles() -> None:
135     """Aplica el tema global d'estudi a la pàgina d'ajuda."""
136     inject_studio_theme(max_width=1180)
137
138
139 def render_hero() -> None:
140     """Renderitza el bloc principal de la pàgina d'ajuda."""
141
142     st.markdown(
143         f"""
144         <section class="help-hero">
145             <div class="hero-kicker">Guia d'ús</div>
146             <h1 class="hero-title">{escape(PAGE_TITLE)}</h1>
147             <div class="hero-copy">
148                 {escape(INTRODUCTION_TEXT)}<br><br>{escape(INTRODUCTION_HINT)}
149             </div>
150         </section>
151         """,
152         unsafe_allow_html=True,
153     )
154
155
156 def render_resources_card() -> None:
157     """Renderitza els recursos addicionals."""
158
159     resource_items = "\n".join(
160         f'<li><a href="{escape(url)}" target="_blank">{escape(label)}</a></li>'
161         for label, url in RESOURCES
162     )
163
164     st.markdown(
165         f"""
166         <section class="resources-card">
167             <div class="panel-kicker">Recursos</div>
168             <div class="panel-title">Enllaços útils</div>
169             <div class="panel-copy">
170                 Referències externes per aprofundir en les tecnologies que fa servir la plataforma.
171             </div>
172             <ul>
173                 {resource_items}
174             </ul>
175         </section>
176         """,
177         unsafe_allow_html=True,
178     )
179
180
181 def render_support_card() -> None:
182     """Renderitza les dades de contacte del centre d'ajuda."""

```

```

176 support_name = SUPPORT_CONTACT["name"]
177 support_email = SUPPORT_CONTACT["email"]
178 st.markdown(
179     f"""
180     <section class="section-card">
181         <div class="panel-kicker">Contacte</div>
182         <div class="panel-title">Suport del centre d'ajuda</div>
183         <div class="panel-copy">
184             Per dubtes o incidències, pots contactar amb
185             <strong>{escape(support_name)}</strong> a
186             <a href="mailto:{escape(support_email)}">{escape(support_email)}</a>.
187         </div>
188     </section>
189     """,
190     unsafe_allow_html=True,
191 )
192
193
194 def render_help_page() -> None:
195     """Renderitza la pàgina d'ajuda."""
196
197     configure_studio_page(PAGE_TITLE, "pages/Ajuda.py", layout="wide")
198     inject_help_styles()
199
200     render_hero()
201
202     st.markdown("<div style='height: 1.15rem;'></div>", unsafe_allow_html=True)
203     st.markdown("<h2 class='section-title'>Funcionalitats i consells</h2>", unsafe_allow_html=True)
204
205     for title, content in HELP_SECTIONS:
206         with st.expander(title):
207             st.markdown(content)
208
209     render_support_card()
210     st.markdown("<div style='height: 1rem;'></div>", unsafe_allow_html=True)
211     render_resources_card()
212
213
214 render_help_page()

```

C.4 pages/Avaluar_Agent.py

Listing 7: pages/Avaluar_Agent.py

```

1  """Pàgina per avaluar un agent SB3 entrenat en un entorn Sinergym.
2
3  Escaneja models, detecta l'entorn més probable i executa l'avaluació en segon pla
4  perquè la interfície continuï responnent mentre EnergyPlus treballa.
5  """
6
7  from __future__ import annotations
8
9  from functools import partial
10 from html import escape
11 from pathlib import Path
12 import queue
13 import threading
14 import time
15 import traceback
16
17 import streamlit as st
18 from page_styles.avaluar_agent import inject_evaluation_styles
19 from sidebar_nav import configure_studio_page
20
21 from backend.avaluar_agent_backend import (
22     ONE_YEAR_STEPS,
23     candidate_vecnorm,
24     env_id_from_meta_or_name,
25     load_model_metadata,
26     load_sb3_model_bytes,
27     run_agent_evaluation,
28     scan_model_zips,
29 )
30 PAGE_TITLE = "Avaluar Agent"
31 PAGE_LAYOUT = "wide"
32 INTRODUCTION_TEXT = (
33     "Carrega un model entrenat, revisa la configuració detectada i executa una "
34     "avaluació controlada sobre un entorn Sinergym."
35 )
36
37
38 def _key(s: str) -> str:
39     """Key."""
40     return f"eval_{s}"
41
42
43 def render_evaluation_hero() -> None:
44     """Renderitza el bloc principal de la pàgina."""

```

```

45
46
47 st.markdown(
48     f"""
49     <section class="eval-hero">
50     <div class="hero-kicker">Validació del model</div>
51     <h1 class="hero-title">{escape(PAGE_TITLE)}</h1>
52     <div class="hero-copy">{escape(INTRODUCTION_TEXT)}</div>
53     </section>
54     """,
55     unsafe_allow_html=True,
56 )
57
58
59 def render_evaluation_panel(title: str, kicker: str, copy_text: str) -> None:
60     """Renderitza un panell lateral informatiu."""
61
62
63     st.markdown(
64         f"""
65         <section class="eval-panel">
66         <div class="panel-kicker">{escape(kicker)}</div>
67         <div class="panel-title">{escape(title)}</div>
68         <div class="panel-copy">{escape(copy_text)}</div>
69         </section>
70         """,
71         unsafe_allow_html=True,
72     )
73
74
75 def render_evaluation_title(title: str) -> None:
76     """Renderitza només el títol d'una secció, sense targeta descriptiva."""
77
78     st.markdown(f'<h2 class="page-section-title">{escape(title)}</h2>', unsafe_allow_html=True)
79
80
81 def render_metadata_grid(cards: list[tuple[str, list[tuple[str, str]]]]) -> None:
82     """Renderitza un resum en graella amb targetes d'alçada uniforme."""
83
84     cards_markup = ""
85     for kicker, items in cards:
86         items_markup = "".join(
87             (
88                 '<div class="eval-meta-item">'
89                 f'<span class="eval-meta-label">{escape(label)}</span>'
90                 f'<span class="eval-meta-value">{escape(value)}</span>'
91                 "</div>"
92             )
93             for label, value in items
94         )
95         cards_markup += (
96             '<section class="eval-meta-card">'
97             f'<div class="eval-meta-kicker">{escape(kicker)}</div>'
98             f'{items_markup}'
99             "</section>"
100         )
101
102     st.markdown(
103         f'<div class="eval-meta-grid">{cards_markup}</div>',
104         unsafe_allow_html=True,
105     )
106
107
108 def empty_evaluation_runtime() -> dict[str, object]:
109     """Temps d'execució d'avaluació buit."""
110     return {
111         "running": False,
112         "completed": False,
113         "cancel_requested": False,
114         "needs_full_rerun": False,
115         "thread": None,
116         "queue": None,
117         "stop_event": None,
118         "result": None,
119         "error": None,
120         "traceback": None,
121         "model_path": None,
122         "env_id": None,
123         "steps_target": 1,
124         "latest_step": 0,
125         "status": "",
126         "started_at": None,
127     }
128
129
130 def ensure_evaluation_runtime() -> dict[str, object]:
131     """Assegura el temps d'execució de l'avaluació."""
132     runtime_key = _key("runtime")
133     if runtime_key not in st.session_state:
134         st.session_state[runtime_key] = empty_evaluation_runtime()
135     return st.session_state[runtime_key]
136

```

```

137
138 def reset_evaluation_runtime() -> dict[str, object]:
139     """Restableix el temps d'execució de l'avaluació."""
140     runtime = ensure_evaluation_runtime()
141     previous_thread = runtime.get("thread")
142     if previous_thread is not None and getattr(previous_thread, "is_alive", lambda: False)():
143         return runtime
144
145     st.session_state[_key("runtime")] = empty_evaluation_runtime()
146     return st.session_state[_key("runtime")]
147
148
149 def drain_evaluation_runtime(runtime: dict[str, object]) -> None:
150     """Temps d'execució de l'avaluació del buidatge."""
151     event_queue = runtime.get("queue")
152     if event_queue is None:
153         return
154
155     # Mateix patró que el baseline: el worker envia progrés/resultat/error i la pàgina
156     # ho incorpora sense bloquejar el fil de Streamlit.
157     while True:
158         try:
159             event = event_queue.get_nowait()
160         except queue.Empty:
161             break
162
163         event_type = str(event.get("type") or "")
164         if event_type == "progress":
165             runtime["latest_step"] = max(
166                 int(runtime.get("latest_step") or 0),
167                 int(event.get("step_number") or 0),
168             )
169             runtime["steps_target"] = max(
170                 int(runtime.get("steps_target") or 1),
171                 int(event.get("total_steps") or 1),
172             )
173             runtime["status"] = str(event.get("status") or runtime.get("status") or "")
174         elif event_type == "result":
175             runtime["result"] = event.get("result")
176             runtime["completed"] = True
177             runtime["running"] = False
178             runtime["needs_full_rerun"] = True
179         elif event_type == "error":
180             runtime["error"] = str(event.get("error") or "Error desconegut durant l'avaluació.")
181             runtime["traceback"] = event.get("traceback")
182             runtime["completed"] = True
183             runtime["running"] = False
184             runtime["needs_full_rerun"] = True
185         elif event_type == "finished":
186             runtime["completed"] = True
187             runtime["running"] = False
188             runtime["thread"] = None
189             runtime["needs_full_rerun"] = True
190
191         thread = runtime.get("thread")
192         if thread is not None and not thread.is_alive() and runtime.get("running"):
193             runtime["running"] = False
194             runtime["completed"] = True
195             runtime["thread"] = None
196             runtime["needs_full_rerun"] = True
197
198
199 def sync_evaluation_runtime() -> dict[str, object]:
200     """Sincronitza el temps d'execució de l'avaluació."""
201     runtime = ensure_evaluation_runtime()
202     drain_evaluation_runtime(runtime)
203     return runtime
204
205
206 def consume_evaluation_runtime_rerun_flag(runtime: dict[str, object] | None = None) -> bool:
207     """Indicador de reexecució del clima d'execució de l'avaluació."""
208     active_runtime = runtime if runtime is not None else ensure_evaluation_runtime()
209     if active_runtime.get("needs_full_rerun") and not active_runtime.get("running"):
210         active_runtime["needs_full_rerun"] = False
211         return True
212     return False
213
214
215 def request_evaluation_stop(runtime: dict[str, object] | None = None) -> bool:
216     """Sol·licitar l'aturada de l'avaluació."""
217     active_runtime = runtime if runtime is not None else ensure_evaluation_runtime()
218     if not active_runtime.get("running") or active_runtime.get("stop_event") is None:
219         return False
220
221     active_runtime["cancel_requested"] = True
222     active_runtime["status"] = "Aturant avaluació..."
223     active_runtime["stop_event"].set()
224     return True
225
226
227 def push_evaluation_progress_event(
228     event_queue: "queue.Queue[dict[str, object]]",

```

```

229     job_config: dict[str, object],
230     step_number: int,
231     total_steps: int,
232 ) -> None:
233     """Envieu un esdeveniment de progrés d'avaluació a la cua d'execució de la pàgina."""
234     event_queue.put(
235         {
236             "type": "progress",
237             "step_number": int(step_number or 0),
238             "total_steps": int(total_steps or job_config["steps_target"]),
239             "status": f"Passos: {int(step_number or 0)}/{int(total_steps or job_config['steps_target'])}",
240         }
241     )
242
243
244 def evaluation_worker(
245     job_config: dict[str, object],
246     event_queue: "queue.Queue[dict[str, object]]",
247     stop_event: threading.Event,
248 ) -> None:
249     """Executa l'avaluació en segon pla."""
250     try:
251         result = run_agent_evaluation(
252             model_path=Path(job_config["model_path"]),
253             env_id=str(job_config["env_id"]),
254             use_vecnorm=bool(job_config["use_vecnorm"]),
255             vecnorm_path=Path(job_config["vecnorm_path"]) if job_config.get("vecnorm_path") else None,
256             steps_target=int(job_config["steps_target"]),
257             deterministic=bool(job_config["deterministic"]),
258             should_stop=stop_event.is_set,
259             on_progress=partial(push_evaluation_progress_event, event_queue, job_config),
260             override_wrapper_configs=job_config.get("wrapper_configs") or None,
261         )
262         event_queue.put({"type": "result", "result": result})
263     except Exception as exc:
264         event_queue.put(
265             {
266                 "type": "error",
267                 "error": str(exc),
268                 "traceback": traceback.format_exc(),
269             }
270         )
271     finally:
272         event_queue.put({"type": "finished"})
273
274
275 def start_evaluation_run(request: dict[str, object]) -> dict[str, object]:
276     """Iniciar l'execució d'avaluació."""
277     runtime = reset_evaluation_runtime()
278     response: dict[str, object] = {
279         "started": False,
280         "errors": [],
281         "runtime": runtime,
282     }
283
284     if runtime.get("running"):
285         response["errors"].append("Ja hi ha una avaluació en curs.")
286         return response
287
288     model_path = Path(request.get("model_path") or "")
289     env_id = str(request.get("env_id") or "").strip()
290     vecnorm_path = request.get("vecnorm_path")
291
292     if not model_path.exists():
293         response["errors"].append("No s'ha trobat el model seleccionat.")
294         return response
295     if not env_id:
296         response["errors"].append("No s'ha pogut determinar un env_id vàlid.")
297         return response
298
299     event_queue: "queue.Queue[dict[str, object]]" = queue.Queue()
300     stop_event = threading.Event()
301     job_config = {
302         "model_path": str(model_path),
303         "env_id": env_id,
304         "use_vecnorm": bool(request.get("use_vecnorm")),
305         "vecnorm_path": str(vecnorm_path) if vecnorm_path else None,
306         "steps_target": int(request.get("steps_target") or ONE_YEAR_STEPS),
307         "deterministic": bool(request.get("deterministic", True)),
308         "wrapper_configs": list(request.get("wrapper_configs") or []),
309     }
310     worker_thread = threading.Thread(
311         target=evaluation_worker,
312         args=(job_config, event_queue, stop_event),
313         daemon=True,
314     )
315
316     runtime.update(
317         {
318             "running": True,
319             "completed": False,
320             "cancel_requested": False,

```

```

321         "needs_full_rerun": False,
322         "thread": worker_thread,
323         "queue": event_queue,
324         "stop_event": stop_event,
325         "result": None,
326         "error": None,
327         "traceback": None,
328         "model_path": str(model_path),
329         "env_id": env_id,
330         "steps_target": int(job_config["steps_target"]),
331         "latest_step": 0,
332         "status": "Inicialitzant avaluació...",
333         "started_at": time.time(),
334     }
335 )
336 worker_thread.start()
337
338 response["started"] = True
339 response["runtime"] = runtime
340 return response
341
342
343 def render_evaluation_runtime(runtime: dict[str, object]) -> None:
344     """Mostra el progrés i el resultat de l'avaluació activa."""
345
346     render_evaluation_title("Estat de l'avaluació")
347
348     # El runtime pot estar viu, acabat o cancel·lat. Cal calcular les mètriques defensivament
349     # perquè Streamlit pot repintar la pantalla entre dos esdeveniments de la cua.
350     steps_target = max(int(runtime.get("steps_target") or 1), 1)
351     latest_step = min(int(runtime.get("latest_step") or 0), steps_target)
352     progress_value = min(latest_step / steps_target, 1.0)
353     elapsed_seconds = 0
354     started_at = runtime.get("started_at")
355     if started_at:
356         elapsed_seconds = max(0, int(time.time() - float(started_at)))
357
358     st.progress(progress_value)
359
360     metric_cols = st.columns(3)
361     with metric_cols[0]:
362         st.metric("Passos", f"{latest_step}/{steps_target}")
363     with metric_cols[1]:
364         st.metric("Progrés", f"{progress_value * 100:.0f}%")
365     with metric_cols[2]:
366         st.metric("Temps", f"{elapsed_seconds // 60}m {elapsed_seconds % 60:02d}s")
367
368     status_text = str(runtime.get("status") or "").strip()
369     if status_text:
370         st.caption(status_text)
371
372     if runtime.get("cancel_requested") and runtime.get("running"):
373         st.warning("S'ha sol·licitat l'aturada. L'avaluació es tancarà al proper punt segur.")
374
375     if runtime.get("error"):
376         st.error(str(runtime.get("error")))
377         if runtime.get("traceback"):
378             with st.expander("Traceback intern", expanded=False):
379                 st.code(str(runtime.get("traceback")), language="text")
380         return
381
382     result = runtime.get("result")
383     if result is None or runtime.get("running"):
384         return
385
386     for warn_msg in getattr(result, "warnings", ()):
387         st.warning(warn_msg)
388
389     if bool(getattr(result, "cancelled", False)):
390         st.warning(
391             f"Avaluació cancel·lada per l'usuari després de {float(getattr(result, 'elapsed_seconds', 0.0)):.2f} s."
392         )
393     else:
394         st.success(
395             f"Avaluació finalitzada en {float(getattr(result, 'elapsed_seconds', 0.0)):.2f} s."
396         )
397
398
399 def render_evaluation_runtime_frame() -> None:
400     """Manté sincronitzat el bloc d'estat amb el runtime en segon pla."""
401
402     runtime = sync_evaluation_runtime()
403     if runtime.get("running") or runtime.get("completed") or runtime.get("error") or runtime.get("result"):
404         st.divider()
405         render_evaluation_runtime(runtime)
406
407     if consume_evaluation_runtime_rerun_flag(runtime):
408         st.rerun()
409
410
411 if hasattr(st, "fragment"):
412     render_evaluation_runtime_fragment = st.fragment(run_every="1s")(render_evaluation_runtime_frame)

```



```

413 else:
414     def render_evaluation_runtime_fragment() -> None:
415         """Renderitza la secció del fragment de temps d'execució de l'avaluació a la UI de Streamlit."""
416         render_evaluation_runtime_frame()
417         runtime = sync_evaluation_runtime()
418         if runtime.get("running"):
419             time.sleep(1.0)
420             st.rerun()
421
422
423 configure_studio_page(PAGE_TITLE, "pages/Avaluar_Agent.py", layout=PAGE_LAYOUT)
424 inject_evaluation_styles()
425
426 st.session_state.setdefault(_key("dirs"), ".join([str(Path.cwd() / "models"), str(Path.cwd())]))
427 st.session_state.setdefault(_key("selected_idx"), 0)
428 st.session_state.setdefault(_key("deterministic"), True)
429 st.session_state.setdefault(_key("use_vecnorm"), True)
430 st.session_state.setdefault(_key("steps"), ONE_YEAR_STEPS)
431
432 render_evaluation_hero()
433
434 summary_col, scale_col = st.columns(2, gap="large")
435 with summary_col:
436     render_evaluation_panel(
437         "Avaluació controlada",
438         "Objectiu",
439         "Carrega un model SB3, detecta l'entorn probable i executa una passada completa amb seguiment de progrés.",
440     )
441 with scale_col:
442     render_evaluation_panel(
443         f"{ONE_YEAR_STEPS} passos per defecte",
444         "Escala",
445         "La configuració inicial parteix d'una avaluació equivalent a un any aproximat de simulació.",
446     )
447
448 st.markdown("<div style='height: 1.15rem;'></div>", unsafe_allow_html=True)
449 render_evaluation_title("Localització de models")
450 dirs_text = st.session_state[_key("dirs")]
451 df_models = scan_model_zips(dirs_text)
452 runtime = sync_evaluation_runtime()
453 controls_disabled = bool(runtime.get("running"))
454
455 if df_models.empty:
456     st.info("No s'han trobat models .zip als directoris indicats.")
457     st.stop()
458
459 options = list(range(len(df_models)))
460 idx = st.selectbox(
461     "Selecciona el model entrenat (.zip)",
462     options=options,
463     format_func=lambda i: str(df_models.iloc[i]["stem"]),
464     index=min(st.session_state[_key("selected_idx")], len(df_models) - 1),
465     disabled=controls_disabled,
466 )
467 st.session_state[_key("selected_idx")] = idx
468
469 row = df_models.iloc[idx]
470 model_path = Path(row["path"])
471 zip_stem = row["stem"]
472
473 with open(model_path, "rb") as fh:
474     model_tmp, meta = load_sb3_model_bytes(fh.read(), device="cpu")
475
476 algo_name = type(model_tmp).__name__
477 policy_name = type(getattr(model_tmp, "policy", object())).__name__
478
479 model_metadata = load_model_metadata(model_path)
480 env_id_guess = model_metadata.get("env_id") or env_id_from_meta_or_name(meta, zip_stem)
481 vecnorm_path = candidate_vecnorm(model_path)
482 wrapper_configs = model_metadata.get("wrapper_configs", [])
483 wrapper_names = [
484     str(wrapper.get("name"))
485     for wrapper in wrapper_configs
486     if isinstance(wrapper, dict) and wrapper.get("name")
487 ]
488 wrappers_label = ", ".join(wrapper_names) if wrapper_names else "Cap (metadades no trobades)"
489
490 render_evaluation_title("Configuració detectada")
491 render_metadata_grid(
492     [
493         (
494             "Model",
495             [
496                 ("Agent", zip_stem),
497                 ("Algorisme", algo_name),
498             ],
499         ),
500         (
501             "Configuració",
502             [
503                 ("Política", policy_name),
504                 ("VecNormalize", vecnorm_path.name if vecnorm_path else "No"),

```

```

505         ],
506     ),
507     (
508         "Context",
509         [
510             ("Entorn detectat", env_id_guess or "No detectat"),
511             ("Wrappers d'entrenament", wrappers_label),
512             ("Observacions esperades", str(getattr(model_tmp.observation_space, "shape", ("?",))[0])),
513         ],
514     ),
515 ]
516 )
517
518 st.divider()
519
520 render_evaluation_title("Paràmetres d'avaluació")
521 env_id = env_id_guess or ""
522 col_b, col_c = st.columns(2)
523 with col_b:
524     st.checkbox(
525         "Deterministic",
526         key=_key("deterministic"),
527         value=st.session_state[_key("deterministic")],
528         disabled=controls_disabled,
529     )
530 with col_c:
531     st.checkbox(
532         "Usar VecNormalize si hi és",
533         key=_key("use_vecnorm"),
534         value=bool(vecnorm_path),
535         disabled=controls_disabled,
536     )
537
538 steps_limit = st.number_input(
539     "Limit de passos (per defecte 35040)",
540     min_value=1,
541     max_value=5_000_000,
542     value=st.session_state[_key("steps")],
543     step=35040,
544     disabled=controls_disabled,
545 )
546
547 if not wrapper_configs:
548     st.info(
549         "No s'han trobat metadades de wrappers per aquest model. "
550         "L'avaluació inferirà automàticament l'espai d'accions a partir del model i de l'entorn seleccionat."
551     )
552
553 st.divider()
554
555 render_evaluation_title("Control")
556 col_run, col_cancel, col_reset = st.columns(3, gap="medium")
557 with col_run:
558     start_requested = st.button(
559         "Iniciar avaluació",
560         type="primary",
561         use_container_width=True,
562         disabled=controls_disabled or not env_id,
563     )
564 with col_cancel:
565     cancel_requested = st.button(
566         "Cancel·lar",
567         use_container_width=True,
568         disabled=not bool(runtime.get("running")),
569     )
570 with col_reset:
571     reset_requested = st.button(
572         "Netejar estat",
573         use_container_width=True,
574         disabled=not bool(runtime.get("completed") or runtime.get("error")),
575     )
576
577 if cancel_requested and request_evaluation_stop(runtime):
578     st.rerun()
579
580 if reset_requested:
581     reset_evaluation_runtime()
582     st.rerun()
583
584 if start_requested:
585     start_result = start_evaluation_run(
586         {
587             "model_path": model_path,
588             "env_id": env_id,
589             "use_vecnorm": bool(st.session_state[_key("use_vecnorm")]),
590             "vecnorm_path": vecnorm_path,
591             "steps_target": int(steps_limit),
592             "deterministic": bool(st.session_state[_key("deterministic")]),
593             "wrapper_configs": wrapper_configs,
594         }
595     )
596     for error in start_result.get("errors") or []:

```

```

597         st.error(str(error))
598         if start_result.get("started"):
599             st.rerun()
600
601 render_evaluation_runtime_fragment()

```

C.5 pages/Entrenar_Agent.py

Listing 8: pages/Entrenar_Agent.py

```

1  """Pàgina d'agent de tren per a l'aplicació BEMS-RL-STUDIO.
2
3  Orquestra el flux complet de la entrenament: selecció de l'entorn, recompensa
4  configuració, configuració de l'embolcall, hiperparàmetres de l'agent i l'incremental
5  bucle d'entrenament amb gràfics en directe per episodi.
6  """
7
8  from __future__ import annotations
9
10 import streamlit as st
11
12 from sidebar_nav import configure_studio_page
13
14 from backend.entrenar_agent_backend import (
15     ALGOS,
16     POLICIES,
17     REWARD_CLASSES,
18     TRAINING_RESULT_KEY,
19     TRAINING_RUNTIME_KEY,
20     assemble_training_payload,
21     clear_training_runtime,
22     create_training_runtime,
23     get_env_metadata,
24     list_registered_envs,
25     prepare_incremental_learn,
26     save_training_artifacts,
27 )
28 from backend.entrenar_agent_session import (
29     TRAINING_WORKSPACE_KEY,
30     ensure_select_state,
31     normalize_training_range_state,
32     training_form_key,
33 )
34 from backend.entrenar_agent_charts import (
35     _get_workspace_from_runtime,
36     render_live_training_charts,
37 )
38 from page_components.training_page import (
39     PAGE_LAYOUT,
40     PAGE_TITLE,
41     inject_training_styles,
42     render_agent_params_section,
43     render_reward_kwargs_section,
44     render_saved_training_library,
45     render_training_hero,
46     render_training_section,
47     render_wrappers_section,
48 )
49
50
51 def train_tab() -> None:
52     """Punt d'entrada a la pàgina de entrenament.
53
54     Orquestra totes les seccions UI en ordre:
55     1. Inicialització de l'estat de la sessió.
56     2. Entorn, algorisme i selecció de polítiques.
57     3. Recompensa kwargs (pesos, rangs estacionals, comoditat).
58     4. Selecció i configuració del wrapper (observació, acció, registres).
59     5. Paràmetres de l'agent (taxa d'aprenentatge, n_steps, total_timesteps).
60     6. Controls d'execució (inici/aturada).
61     7. Cicle d'entrenament incremental amb gràfics en directe.
62     """
63     configure_studio_page(PAGE_TITLE, "pages/Entrenar_Agent.py", layout=PAGE_LAYOUT)
64     inject_training_styles()
65
66     # Inicialització de l'estat de la sessió
67     if "is_training" not in st.session_state:
68         st.session_state.is_training = False
69     if "stop_training" not in st.session_state:
70         st.session_state.stop_training = False
71
72     envs = list_registered_envs()
73     normalize_training_range_state()
74     render_training_hero()
75     render_saved_training_library(envs)
76     st.divider()
77
78     # Secció 1: configuració de l'entorn

```

```

79 render_training_section(
80     "Configuracio d'entorn",
81     "Seleccio base",
82     "Defineix l'entorn, l'algorisme i la politica abans de construir la recompensa i els wrappers.",
83 )
84 ensure_select_state("env_id", envs)
85 ensure_select_state("algo_name", list(ALGOS.keys()))
86 env_col1, env_col2 = st.columns(2)
87 with env_col1:
88     env_id = st.selectbox(
89         "Entorn",
90         envs,
91         index=envs.index(st.session_state[training_form_key("env_id")]),
92         key=training_form_key("env_id"),
93     )
94     algo_options = list(ALGOS.keys())
95     algo_name = st.selectbox(
96         "Algorisme SB3",
97         algo_options,
98         index=algo_options.index(st.session_state[training_form_key("algo_name")]),
99         key=training_form_key("algo_name"),
100     )
101 with env_col2:
102     ensure_select_state("policy_name", POLICIES[algo_name])
103     policy_name = st.selectbox(
104         "Politica SB3",
105         POLICIES[algo_name],
106         index=POLICIES[algo_name].index(st.session_state[training_form_key("policy_name")]),
107         key=training_form_key("policy_name"),
108     )
109     reward_options = list(REWARD_CLASSES.keys())
110     ensure_select_state("reward_name", reward_options)
111     reward_name = st.selectbox(
112         "Recompensa",
113         reward_options,
114         index=reward_options.index(st.session_state[training_form_key("reward_name")]),
115         key=training_form_key("reward_name"),
116     )
117
118 env_meta = get_env_metadata(env_id)
119 spec = env_meta["spec"]
120 variables = env_meta["variables"]
121 observation_variables = env_meta["observation_variables"]
122 action_variables = env_meta["action_variables"]
123 action_space = env_meta["action_space"]
124 candidate_grid_energy_vars = env_meta["candidate_grid_energy_vars"]
125 candidate_battery_charge_vars = env_meta["candidate_battery_charge_vars"]
126 candidate_battery_discharge_vars = env_meta["candidate_battery_discharge_vars"]
127 candidate_battery_loss_vars = env_meta["candidate_battery_loss_vars"]
128
129 # Secció 2: recompensa kwargs
130 st.divider()
131 render_training_section(
132     "Reward kwargs",
133     "Confort i energia",
134     "Ajusta pesos, rangs estacionals i criteris de confort per construir la reward final.",
135 )
136 reward_kwargs_values = render_reward_kwargs_section(
137     reward_name=reward_name,
138     observation_variables=observation_variables,
139     candidate_grid_energy_vars=candidate_grid_energy_vars,
140     candidate_battery_charge_vars=candidate_battery_charge_vars,
141     candidate_battery_discharge_vars=candidate_battery_discharge_vars,
142     candidate_battery_loss_vars=candidate_battery_loss_vars,
143 )
144
145 # Secció 3: embolcalls i troncs
146 st.divider()
147 render_training_section(
148     "Wrappers i logs",
149     "Observacio i control",
150     "Activa nomes els wrappers que necessites per enriquir l'observacio o restringir l'espai d'accions.",
151 )
152 wrapper_values = render_wrappers_section(
153     env_meta=env_meta,
154     observation_variables=observation_variables,
155     action_variables=action_variables,
156     action_space=action_space,
157     context_variables=env_meta["context_variables"],
158     candidate_temp_vars=env_meta["candidate_temp_vars"],
159     candidate_setpoint_vars=env_meta["candidate_setpoint_vars"],
160 )
161
162 # Apartat 4: paràmetres de l'agent
163 st.divider()
164 render_training_section(
165     "Parametres de l'agent",
166     "Aprenentatge",
167     "Configura el ritme d'aprenentatge, la mida dels blocs i el nombre total de timesteps.",
168 )
169 agent_params = render_agent_params_section(algo_name=algo_name)
170

```

```

171 # Secció 5: controls d'execució
172 st.divider()
173 render_training_section(
174     "Execucio",
175     "Control",
176     "Llança l'entrenament o atura'l de forma segura mantenint els artefactes guardats.",
177 )
178 controls_col1, controls_col2 = st.columns(2)
179 start_clicked = controls_col1.button(
180     "Entrenar agent",
181     disabled=st.session_state.is_training,
182     use_container_width=True,
183 )
184 stop_clicked = controls_col2.button(
185     "Aturar entrenament",
186     disabled=not st.session_state.is_training,
187     use_container_width=True,
188 )
189
190 if start_clicked:
191     clear_training_runtime()
192     st.session_state.pop(TRAINING_RESULT_KEY, None)
193     st.session_state.pop(TRAINING_WORKSPACE_KEY, None)
194     st.session_state.is_training = True
195     st.session_state.stop_training = False
196
197 if stop_clicked and st.session_state.is_training:
198     st.session_state.stop_training = True
199
200 if not st.session_state.is_training:
201     return
202
203 # A partir d'aquí Streamlit reexecuta la pàgina moltes vegades. Si ja hi ha runtime,
204 # no tornem a muntar payload ni entorn: només continuem el model que és a memòria.
205 runtime = st.session_state.get(TRAINING_RUNTIME_KEY)
206 runtime_ui_state = runtime.get("ui_state", {}) if runtime is not None else {}
207 training_payload = (
208     None
209     if runtime is not None
210     else assemble_training_payload({
211         "spec": spec,
212         "env_id": env_id,
213         "algo_name": algo_name,
214         "policy_name": policy_name,
215         "reward_name": reward_name,
216         "is_continuous": env_meta["is_continuous"],
217         "is_discrete": env_meta["is_discrete"],
218         "action_space_type": type(action_space).__name__,
219         "building_name": env_meta["building_name"],
220         "variables": variables,
221         **reward_kwargs_values,
222         **wrapper_values,
223         **agent_params,
224     })
225 )
226
227 if training_payload is not None:
228     dropped_reward_kwargs = training_payload["dropped_reward_kwargs"]
229     if dropped_reward_kwargs:
230         st.warning(
231             "La classe de reward carregada no admet alguns parametres i s'han omes. "
232             f"Parametres ignorats: {', '.join(dropped_reward_kwargs)}."
233         )
234
235     dropped_algo_kwargs = training_payload.get("dropped_algo_kwargs", [])
236     if dropped_algo_kwargs:
237         st.info(
238             "L'algorisme seleccionat no fa servir alguns parametres SB3 avançats: "
239             f"{', '.join(dropped_algo_kwargs)}."
240         )
241
242     validation_errors = training_payload["validation_errors"]
243     if validation_errors:
244         clear_training_runtime()
245         st.session_state.is_training = False
246         st.session_state.stop_training = False
247         for message in validation_errors:
248             st.error(message)
249         return
250
251     training_config = training_payload["training_config"]
252 else:
253     training_config = runtime["config"]
254
255 if runtime is not None:
256     runtime_config = runtime.get("config", {})
257     linear_cost_rewards = {"LinearReward", "EnergyCostLinearReward"}
258     # Si una run antiga porta EnergyCostWrapper amb una configuració que ja no quadra,
259     # és més segur recrear el runtime que entrenar amb una reward recalculada a mitges.
260     stale_energy_wrapper = any(
261         wrapper.get("name") == "EnergyCostWrapper"
262         and (

```

```

263         "recalculate_reward" not in wrapper.get("params", {})
264     or (
265         runtime_config.get("reward_name") not in linear_cost_rewards
266         and wrapper.get("params", {}).get("recalculate_reward", True)
267     )
268 )
269 for wrapper in runtime_config.get("wrapper_configs", [])
270 )
271 # El runtime creat pel botó queda congelat entre reruns; els widgets poden canviar
272 # visualment, però el model en curs ha de continuar amb la configuració inicial.
273 if stale_energy_wrapper:
274     clear_training_runtime()
275     st.session_state.pop(TRAINING_WORKSPACE_KEY, None)
276     runtime = None
277 if runtime is None:
278     runtime = create_training_runtime(
279         training_config,
280         ui_state=training_payload["ui_state"] if training_payload is not None else runtime_ui_state,
281     )
282     st.session_state[TRAINING_RUNTIME_KEY] = runtime
283
284 if TRAINING_WORKSPACE_KEY not in st.session_state:
285     workspace = _get_workspace_from_runtime(runtime)
286     if workspace:
287         st.session_state[TRAINING_WORKSPACE_KEY] = workspace
288
289 active_config = runtime["config"]
290
291 render_training_section(
292     "Estat de l'entrenament",
293     "Seguiment",
294     "Visualitza el progres actual, l'entorn actiu i l'estat del proces abans de guardar el model.",
295 )
296 st.write(f"Entorn seleccionat: `{active_config['env_id']}`")
297 st.write(f"Algorisme: `{active_config['algo_name']}`")
298 st.write(f"Recompensa: `{active_config['reward_name']}`")
299
300 progress_bar = st.progress(0.0)
301 training_status = st.empty()
302
303 current_timesteps = int(runtime["model"].num_timesteps)
304 total_timesteps = int(active_config["timesteps_per_year"])
305 frac = min(current_timesteps / max(1, total_timesteps), 1.0)
306 progress_bar.progress(frac)
307
308 render_training_section(
309     "Evolucio de l'entrenament",
310     "Temps real",
311     "Metriques per episodi actualitzades automaticament en completar-se cada episodi.",
312 )
313 render_live_training_charts()
314
315 try:
316     if st.session_state.stop_training:
317         training_status.warning("Aturant l'entrenament i guardant l'estat actual...")
318         was_stopped = True
319     elif current_timesteps >= total_timesteps:
320         training_status.success("Entrenament completat. Guardant el model...")
321         was_stopped = False
322     else:
323         training_status.info(
324             f"Entrenament en curs: {current_timesteps}/{total_timesteps} timesteps ({frac * 100:.1f}%)."
325         )
326         remaining_timesteps = max(1, total_timesteps - current_timesteps)
327         next_chunk = min(max(1, int(runtime["chunk_timesteps"])), remaining_timesteps)
328         # Entrenem en blocs petits perquè la UI respongui, actualitzi gràfics i pugui
329         # aturar-se sense esperar tot un any de simulació.
330         prepare_incremental_learn(runtime["model"], runtime["vec"])
331         runtime["model"].learn(total_timesteps=next_chunk, reset_num_timesteps=False)
332         st.session_state[TRAINING_RUNTIME_KEY] = runtime
333         st.rerun()
334
335     st.session_state[TRAINING_RESULT_KEY] = save_training_artifacts(runtime)
336     clear_training_runtime()
337     st.session_state.is_training = False
338     st.session_state.stop_training = False
339     st.rerun()
340 except Exception:
341     clear_training_runtime()
342     st.session_state.is_training = False
343     st.session_state.stop_training = False
344     raise
345
346 # Streamlit executa aquesta funció quan carrega la pàgina.
347
348 train_tab()

```

C.6 pages/Gestionar_Arxius.py

Listing 9: pages/Gestionar_Arxius.py

```
1 from __future__ import annotations
2
3 import time
4 from datetime import datetime
5 from html import escape
6 from io import BytesIO
7 from pathlib import Path
8 from zipfile import ZIP_DEFLATED, ZipFile
9
10 import streamlit as st
11 from page_styles.gestionar_arxius import inject_file_manager_styles
12 from sidebar_nav import configure_studio_page
13
14 from backend.mapa_backend import render_multipoint_map
15 from backend.gestionar_arxius_backend import (
16     DATA_BTC_PATH,
17     DATA_WEA_PATH,
18     ROOT_PATH,
19     delete_item,
20     filter_envs,
21     filter_models,
22     filter_trainings,
23     filter_weather,
24     list_explorer_items,
25     load_weather_map_rows,
26 )
27
28 PAGE_TITLE = "Gestor de Fitxers Avançat"
29 PAGE_LAYOUT = "wide"
30 INTRODUCTION_TEXT = "Gestiona els fitxers del projecte organitzats per categories."
31 EXPLORER_COLUMN_LAYOUT = [0.8, 1.4, 5.9, 1.4, 2.5]
32 TRAINING_ARTIFACTS_PATH = ROOT_PATH / "trainings"
33
34
35 def render_hero() -> None:
36     """Renderitza el bloc principal de la pàgina."""
37
38     st.markdown(
39         f"""
40         <section class="files-hero">
41             <div class="hero-kicker">Exploració i manteniment</div>
42             <h1 class="hero-title">{escape(PAGE_TITLE)}</h1>
43             <div class="hero-copy">{escape(INTRODUCTION_TEXT)}</div>
44         </section>
45         """,
46         unsafe_allow_html=True,
47     )
48
49
50 def render_weather_view_selector() -> str:
51     """Renderitza el selector de vista del bloc de climes sense ràdios."""
52
53     options = ["Explorador", "Mapa"]
54     current_value = st.session_state.get("weather_submenu", "Explorador")
55
56     if hasattr(st, "segmented_control"):
57         try:
58             selected_value = st.segmented_control(
59                 "Vista de climes",
60                 options,
61                 default=current_value,
62                 key="weather_submenu_segmented",
63             )
64         except TypeError:
65             selected_value = st.segmented_control(
66                 "Vista de climes",
67                 options,
68                 key="weather_submenu_segmented",
69             )
70
71     if selected_value:
72         st.session_state["weather_submenu"] = selected_value
73     return st.session_state.get("weather_submenu", current_value)
74
75
76 selected_value = st.selectbox(
77     "Vista de climes",
78     options,
79     index=options.index(current_value) if current_value in options else 0,
80     key="weather_submenu_select",
81 )
82 st.session_state["weather_submenu"] = selected_value
83 return selected_value
84
85
86 def build_selection_archive(selected_paths: list[str]) -> bytes:
87     """Construeix un zip en memòria amb els elements seleccionats."""
```

```

88
89 archive_buffer = BytesIO()
90 normalized_paths = sorted({str(Path(path_str)) for path_str in selected_paths})
91
92 with ZipFile(archive_buffer, "w", ZIP_DEFLATED) as archive:
93     for path_str in normalized_paths:
94         path = Path(path_str)
95         if not path.exists():
96             continue
97
98         if path.is_dir():
99             archive.writestr(f"{path.name}/", "")
100             for nested_path in sorted(path.rglob("*")):
101                 archive_name = nested_path.relative_to(path.parent).as_posix()
102                 if nested_path.is_dir():
103                     try:
104                         next(nested_path.iterdir())
105                     except StopIteration:
106                         archive.writestr(f"{archive_name}/", "")
107                     except OSError:
108                         pass
109                     continue
110
111                 archive.write(nested_path, archive_name)
112             continue
113
114         archive.write(path, path.name)
115
116 archive_buffer.seek(0)
117 return archive_buffer.getvalue()
118
119
120 def build_download_name(key_prefix: str) -> str:
121     """Construeix un nom de fitxer estable per a la descàrrega."""
122
123     timestamp = datetime.now().strftime("%Y%m%d_%H%M")
124     return f"{key_prefix}_seleccio_{timestamp}.zip"
125
126
127 def clear_prepared_archive_if_stale(key_prefix: str, selection_signature: tuple[str, ...]) -> None:
128     """Neteja l'arxiu preparat si la selecció actual ja no coincideix."""
129
130     archive_key = f"{key_prefix}_download_archive"
131     archive_name_key = f"{key_prefix}_download_name"
132     archive_signature_key = f"{key_prefix}_download_signature"
133
134     prepared_signature = st.session_state.get(archive_signature_key)
135     if prepared_signature == selection_signature:
136         return
137
138     st.session_state.pop(archive_key, None)
139     st.session_state.pop(archive_name_key, None)
140     st.session_state.pop(archive_signature_key, None)
141
142
143 def delete_items(items: list[str]) -> None:
144     """Esborra els elements seleccionats mostrant progrés."""
145
146     progress_bar = st.progress(0)
147     status_text = st.empty()
148
149     for index, path_str in enumerate(items):
150         path = Path(path_str)
151         status_text.text(f"Eliminant: {path.name}...")
152         error_message = delete_item(path_str)
153         if error_message:
154             st.error(f"Error eliminant {path.name}: {error_message}")
155         progress_bar.progress((index + 1) / len(items))
156
157     status_text.text("Fet.")
158     time.sleep(1)
159     st.rerun()
160
161
162 def handle_uploaded_files(current_path: Path, uploaded_files) -> None:
163     """Desa els fitxers pujats a la ruta actual de l'explorador."""
164
165     for uploaded_file in uploaded_files:
166         destination = current_path / Path(uploaded_file.name).name
167         try:
168             with open(destination, "wb") as destination_handle:
169                 destination_handle.write(uploaded_file.getbuffer())
170             st.toast(f"Fitxer carregat correctament: {destination.name}", icon="📁")
171         except Exception as exc:
172             st.error(f"Error carregant {uploaded_file.name}: {exc}")
173
174     time.sleep(1)
175     st.rerun()
176
177
178 def navigate_file_explorer_up(path_key: str, current_path: Path, root_path: Path) -> None:
179     """Mou l'explorador de fitxers un directori cap amunt sense sortir de l'arrel."""

```



```

180 parent = current_path.parent
181 if parent == root_path or root_path in parent.parents:
182     st.session_state[path_key] = str(parent)
183 else:
184     st.session_state[path_key] = str(root_path)
185
186
187 def navigate_file_explorer_into(path_key: str, folder_path: str) -> None:
188     """Mou l'explorador de fitxers cap a una carpeta filla."""
189     st.session_state[path_key] = str(Path(folder_path).resolve())
190
191
192 def render_explorer(
193     key_prefix: str,
194     root_path: Path,
195     filter_func=None,
196     allow_nav_up: bool = True,
197     allow_upload: bool = False,
198     upload_label: str | None = None,
199     upload_types: list[str] | None = None,
200     upload_help: str | None = None,
201 ) -> None:
202     """Renderitza l'explorador de fitxers per a cada categoria."""
203
204     root_path = root_path.resolve()
205     path_key = f"{key_prefix}_path"
206     if path_key not in st.session_state:
207         st.session_state[path_key] = str(root_path)
208
209     current_path = Path(st.session_state[path_key]).resolve()
210
211     # El path surt de session_state, així que el validem sempre contra l'arrel de la categoria.
212     # D'aquesta manera un valor vell o manipulat no pot fer navegar fora de la carpeta prevista.
213     try:
214         current_path.relative_to(root_path)
215     except ValueError:
216         st.session_state[path_key] = str(root_path)
217         current_path = root_path
218
219     nav_columns = st.columns([1.1, 5.4], gap="small")
220     with nav_columns[0]:
221         disabled = (current_path == root_path) or not allow_nav_up
222         if st.button("Amunt", key=f"{key_prefix}_up", disabled=disabled, type="primary", use_container_width=True):
223             navigate_file_explorer_up(path_key, current_path, root_path)
224             # Canviar carpeta modifica l'estat; forcem rerun perquè la llista es repinti de seguida.
225             st.rerun()
226
227     try:
228         relative_path = current_path.relative_to(root_path)
229         display_path = f"/{relative_path.as_posix()}" if str(relative_path) != "." else "/"
230     except ValueError:
231         display_path = str(current_path)
232
233     with nav_columns[1]:
234         st.markdown(f"<div class='file-explorer-path'>{escape(display_path)}</div>", unsafe_allow_html=True)
235
236     if allow_upload:
237         uploaded_files = st.file_uploader(
238             upload_label or f"Pujar fitxers a {display_path}",
239             accept_multiple_files=True,
240             type=upload_types,
241             help=upload_help,
242             key=f"{key_prefix}_upload",
243         )
244         if uploaded_files:
245             # Les pujades es desen a la carpeta actual, no a l'arrel global del projecte.
246             handle_uploaded_files(current_path, uploaded_files)
247
248     all_items, error_message = list_explorer_items(current_path, root_path, filter_func)
249     if error_message:
250         st.error(f"Error llegint el directori: {error_message}")
251         all_items = ()
252
253     if not all_items:
254         st.info("Aquesta carpeta és buida.")
255         return
256
257     header_columns = st.columns(EXPLORER_COLUMN_LAYOUT, gap="small")
258     headers = ("Sel.", "Tipus", "Nom", "Mida", "Data")
259     for column, header in zip(header_columns, headers):
260         column.markdown(f"<div class='file-explorer-header'>{header}</div>", unsafe_allow_html=True)
261
262     st.markdown("---")
263
264     selected_items: list[str] = []
265     for index, item in enumerate(all_items):
266         row_columns = st.columns(EXPLORER_COLUMN_LAYOUT, gap="small")
267         checkbox_key = f"{key_prefix}_chk_{item.path}"
268         is_selected = row_columns[0].checkbox(
269             f"Seleccionar {item.name}",
270             key=checkbox_key,
271             label_visibility="collapsed",

```

```

272 )
273 if is_selected:
274     selected_items.append(item.path)
275
276 item_type = "Carpetes" if item.is_dir else "Fitxer"
277 item_size = "-" if item.is_dir else item.size
278 item_icon = "\U0001f4c1" if item.is_dir else "\U0001f4c4"
279
280 row_columns[1].markdown(f"<div class='file-explorer-muted'>{item_type}</div>", unsafe_allow_html=True)
281 name_columns = row_columns[2].columns([0.6, 7.8], gap="small")
282 name_columns[0].markdown(
283     f"<div class='file-explorer-icon-cell'><span class='file-explorer-icon'>{item_icon}</span></div>",
284     unsafe_allow_html=True,
285 )
286
287 if item.is_dir:
288     if name_columns[1].button(
289         item.name,
290         key=f"{key_prefix}_btn_{item.path}",
291         type="secondary",
292         use_container_width=True,
293     ):
294         navigate_file_explorer_into(path_key, item.path)
295         st.rerun()
296 else:
297     safe_name = escape(item.name)
298     name_columns[1].markdown(
299         (
300             "<div class='file-explorer-cell file-explorer-name'>"
301             f"<span class='file-explorer-label'>{safe_name}</span></div>"
302             "</div>"
303         ),
304         unsafe_allow_html=True,
305     )
306
307 row_columns[3].markdown(f"<div class='file-explorer-muted'>{escape(item_size)}</div>", unsafe_allow_html=True)
308 row_columns[4].markdown(f"<div class='file-explorer-muted'>{escape(item.mtime)}</div>", unsafe_allow_html=True)
309 if index < len(all_items) - 1:
310     st.markdown("<div class='file-explorer-row-spacer'></div>", unsafe_allow_html=True)
311
312 st.markdown("----")
313
314 if not selected_items:
315     return
316
317 selection_columns = st.columns([2.4, 1.5, 1.2], gap="small")
318 selection_columns[0].markdown(
319     f"<div class='file-explorer-selection'>{len(selected_items)} elements seleccionats.</div>",
320     unsafe_allow_html=True,
321 )
322 selection_signature = tuple(sorted({str(Path(path_str)) for path_str in selected_items}))
323 # Si canvia la selecció, descartem el zip preparat anterior. Evita descarregar un arxiu
324 # que ja no correspon al que es veu marcat a pantalla.
325 clear_prepared_archive_if_stale(key_prefix, selection_signature)
326
327 archive_key = f"{key_prefix}_download_archive"
328 archive_name_key = f"{key_prefix}_download_name"
329 archive_signature_key = f"{key_prefix}_download_signature"
330 archive_ready = (
331     st.session_state.get(archive_signature_key) == selection_signature
332     and st.session_state.get(archive_key) is not None
333 )
334
335 with selection_columns[1]:
336     download_slot = st.empty()
337     if not archive_ready and download_slot.button(
338         "Descarregar selecció",
339         key=f"{key_prefix}_prepare_download",
340         type="primary",
341         use_container_width=True,
342     ):
343         try:
344             with st.spinner("Preparant la descàrrega..."):
345                 st.session_state[archive_key] = build_selection_archive(selected_items)
346                 st.session_state[archive_name_key] = build_download_name(key_prefix)
347                 st.session_state[archive_signature_key] = selection_signature
348                 archive_ready = True
349         except Exception as exc:
350             st.error(f"No s'ha pogut preparar la descàrrega: {exc}")
351             archive_ready = False
352
353 if archive_ready:
354     download_slot.download_button(
355         "Descarregar selecció",
356         data=st.session_state[archive_key],
357         file_name=st.session_state[archive_name_key],
358         mime="application/zip",
359         key=f"{key_prefix}_download",
360         type="primary",
361         use_container_width=True,
362     )
363

```

```

364     with selection_columns[2]:
365         if st.button("Esborrar", key=f"{key_prefix}_del", type="primary", use_container_width=True):
366             delete_items(selected_items)
367
368
369 configure_studio_page(PAGE_TITLE, "pages/Gestionar_Arxius.py", layout=PAGE_LAYOUT)
370 inject_file_manager_styles()
371
372 render_hero()
373
374 st.markdown("<div style='height: 0.45rem;'></div>", unsafe_allow_html=True)
375 tab_train, tab_scripts, tab_models, tab_weather, tab_envs = st.tabs(
376     ["Entrenaments", "Scripts", "Models", "Climes", "Entorns"]
377 )
378
379 with tab_train:
380     render_explorer("train", ROOT_PATH, filter_trainings)
381
382 with tab_scripts:
383     if TRAINING_ARTIFACTS_PATH.exists():
384         render_explorer("training_scripts", TRAINING_ARTIFACTS_PATH)
385     else:
386         st.info("Encara no hi ha cap carpeta `trainings/`. Es creara quan es guardi el primer entrenament.")
387
388 with tab_models:
389     models_root = TRAINING_ARTIFACTS_PATH if TRAINING_ARTIFACTS_PATH.exists() else ROOT_PATH
390     render_explorer("models", models_root, filter_models)
391
392 with tab_weather:
393     weather_root = DATA_WEA_PATH if DATA_WEA_PATH.exists() else ROOT_PATH
394     weather_view = render_weather_view_selector()
395
396     if weather_view == "Explorador":
397         render_explorer(
398             "weather",
399             weather_root,
400             filter_weather,
401             allow_upload=True,
402             upload_label="Pujar fitxers climàtics (.epw)",
403             upload_types=["epw"],
404             upload_help="Afegeix nous fitxers meteorològics EPW al catàleg de climes.",
405         )
406     else:
407         st.caption("Mapa amb tots els fitxers `.epw` disponibles.")
408         with st.spinner("Generant mapa de climes..."):
409             weather_df = load_weather_map_rows(str(weather_root))
410
411         if weather_df.empty:
412             st.warning("No s'han trobat fitxers `.epw` per mostrar al mapa.")
413         else:
414             render_multipoint_map(
415                 weather_df,
416                 zoom=2,
417                 color=(255, 0, 0, 200),
418                 radius=55000,
419                 tooltip_html=(
420                     "<b>{file_name}</b><br>{city}, {country}"
421                     "<br>Clima: {climate}<br>Avg tmp: {avg_temp} C"
422                 ),
423             )
424             st.dataframe(
425                 weather_df[["file_name", "city", "country", "climate", "avg_temp"]],
426                 hide_index=True,
427                 use_container_width=True,
428             )
429
430 with tab_envs:
431     env_root = DATA_BTC_PATH if DATA_BTC_PATH.exists() else ROOT_PATH
432     render_explorer("envs", env_root, filter_envs)

```

C.7 pages/Interaccionar_amb_l'Agent.py

Listing 10: pages/Interaccionar_amb_l'Agent.py

```

1  """Pàgina de control en viu d'un entorn Sinergym.
2
3  Permet provar un model entrenat o aplicar accions manuals pas a pas.
4  """
5
6  from html import escape
7  from pathlib import Path
8
9  import gymnasium as gym
10 import numpy as np
11 import pandas as pd
12 import streamlit as st
13 from page_styles.interaccionar_agent import inject_interaction_styles
14 from sidebar_nav import configure_studio_page

```

```

15
16 from backend.interaccionar_agent_backend import (
17     env_id_from_meta_or_name,
18     extract_display_observation,
19     extract_policy_test_defaults,
20     find_matching_column,
21     coerce_observation_values,
22     get_model_observation_size,
23     get_runtime_observation_variables,
24     get_wrapper_variables,
25     infer_unit,
26     initialize_runtime,
27     load_model_training_metadata,
28     load_sb3_model_bytes,
29     mode_requires_model,
30     normalize_policy_observation,
31     prepare_action_display,
32     randomize_observation_values,
33     run_environment_steps,
34     scan_model_zips,
35 )
36
37
38 PAGE_TITLE = "Control en Viu"
39 PAGE_LAYOUT = "wide"
40 INTRODUCTION_TEXT = (
41     "Configura un entorn Sinergym, carrega un model si cal i controla "
42     "la simulació pas a pas amb l'agent o amb accions manuals."
43 )
44
45
46 def render_interaction_hero() -> None:
47     """Mostra la introducció de la pàgina principal a la UI de Streamlit."""
48
49
50     st.markdown(
51         f"""
52         <section class="interaction-hero">
53             <div class="hero-kicker">Simulació i control</div>
54             <h1 class="hero-title">{escape(PAGE_TITLE)}</h1>
55             <div class="hero-copy">{escape(INTRODUCTION_TEXT)}</div>
56         </section>
57         """,
58         unsafe_allow_html=True,
59     )
60
61
62 def render_interaction_section(title: str, kicker: str, description: str) -> None:
63     """Renderitza una capçalera visual compacta per a una secció de pàgina."""
64
65
66     st.markdown(f'<h2 class="page-section-title">{escape(title)}</h2>', unsafe_allow_html=True)
67     st.markdown(
68         f"""
69         <section class="interaction-section-card">
70             <div class="section-kicker">{escape(kicker)}</div>
71             <div class="section-copy">{escape(description)}</div>
72         </section>
73         """,
74         unsafe_allow_html=True,
75     )
76
77
78 # La configuració visual es fa al principi perquè qualsevol rerun mantingui el mateix marc.
79 configure_studio_page(PAGE_TITLE, "pages/Interaccionar_amb_l'Agent.py", layout=PAGE_LAYOUT)
80 inject_interaction_styles()
81
82 render_interaction_hero()
83
84 # Inicialitzem les claus que Streamlit reutilitza entre reruns.
85 def init_session():
86     """Inicialitza les claus de sessió utilitzades pel flux d'interacció en directe."""
87     if "ix_running" not in st.session_state:
88         st.session_state.ix_running = False
89     if "ix_app_mode" not in st.session_state:
90         st.session_state.ix_app_mode = None
91     if "ix_env" not in st.session_state:
92         st.session_state.ix_env = None
93     if "ix_model" not in st.session_state:
94         st.session_state.ix_model = None
95     if "ix_obs" not in st.session_state:
96         st.session_state.ix_obs = None
97     if "ix_step" not in st.session_state:
98         st.session_state.ix_step = 0
99     if "ix_history" not in st.session_state:
100         st.session_state.ix_history = []
101     if "ix_reward" not in st.session_state:
102         st.session_state.ix_reward = 0.0
103     if "ix_info" not in st.session_state:
104         st.session_state.ix_info = {}
105
106 init_session()

```

```

107
108
109 def stop_simulation():
110     """Tanqueu l'entorn actiu i restabliu l'estat d'interacció en directe."""
111     if st.session_state.ix_env is not None:
112         try:
113             st.session_state.ix_env.close()
114         except Exception:
115             pass
116     st.session_state.ix_env = None
117     st.session_state.ix_app_mode = None
118     st.session_state.ix_model = None
119     st.session_state.ix_obs = None
120     st.session_state.ix_running = False
121     st.session_state.ix_step = 0
122     st.session_state.ix_history = []
123     st.session_state.ix_reward = 0.0
124     st.session_state.ix_info = {}
125     st.session_state.pop("ix_observation_variables", None)
126     st.session_state.pop("ix_model_observation_size", None)
127     st.session_state.pop("ix_test_obs_vals", None)
128
129
130 def randomize_policy_test_observations(obs_vars: list[str]) -> None:
131     """Substituiu les observacions de la prova de política per valors aleatoris per a les variables actives."""
132
133     random_obs_values = randomize_observation_values(obs_vars)
134     st.session_state.ix_test_obs_vals = coerce_observation_values(random_obs_values, len(obs_vars))
135
136
137 def apply_live_action_step(
138     vec_env: object,
139     core_env: object,
140     action_to_take: object,
141     action_source: str = "",
142     repeat_n: int = 1,
143 ) -> None:
144     """Avançar la simulació en directe i mantenir l'estat d'observació resultant."""
145
146     try:
147         with st.spinner(f"Simulant els propers {repeat_n} pas(sos) EnergyPlus ({action_source})..."):
148             result = run_environment_steps(
149                 vec_env=vec_env,
150                 core_env=core_env,
151                 action_to_take=action_to_take,
152                 current_step=st.session_state.ix_step,
153                 repeat_n=repeat_n,
154                 obs_vars=st.session_state.get("ix_observation_variables"),
155             )
156
157             st.session_state.ix_obs = result["next_obs"]
158             st.session_state.ix_reward = result["reward"]
159             st.session_state.ix_info = result["info_dict"]
160             st.session_state.ix_history.extend(result["history_entries"])
161             st.session_state.ix_observation_variables = get_runtime_observation_variables(
162                 core_env=core_env,
163                 obs=result["next_obs"],
164                 model_obj=st.session_state.ix_model,
165                 vec_env=vec_env,
166             )
167
168             if result["history_entries"]:
169                 st.session_state.ix_step = result["history_entries"][-1]["pas"]
170
171             if result["episode_finished"]:
172                 st.warning("Episodi acabat (Terminated / Truncated). L'entorn s'ha reiniciat.")
173                 st.session_state.ix_obs = result["reset_obs"]
174                 st.session_state.ix_step = 0
175                 st.session_state.ix_history = []
176
177     except Exception as exc:
178         st.error(f"Error aplicant l'acció: {exc}")
179
180
181 # Flux de configuració abans de crear l'entorn.
182 if not st.session_state.ix_running:
183     # Selecció del mode
184     col_mode, col_env = st.columns([1, 1.35], gap="small")
185     col_mode.markdown('<div class="studio-radio-card-anchor"></div>', unsafe_allow_html=True)
186     col_env.markdown('<div class="interaction-env-card-anchor"></div>', unsafe_allow_html=True)
187     use_agent = col_mode.radio(
188         "Mode de Funcionament",
189         [
190             "Simulació en Temps Real interactiva",
191             "Avaluació Manual Estàtica (Sense Simulació)",
192         ],
193         index=0,
194     )
195
196 # Si cal model, primer mirem el metadata del .zip; és més fiable que deduir l'entorn pel nom.
197 selected_env_id = ""
198 model_path = None

```

```

199
200 if mode_requires_model(use_agent):
201     model_scan_dirs = ",".join([str(Path.cwd() / "models"), str(Path.cwd())])
202     df_models = scan_model_zips(model_scan_dirs)
203
204     if not df_models.empty:
205         options = list(range(len(df_models)))
206         idx = st.selectbox(
207             "Selecciona el model entrenat (.zip)",
208             options=options,
209             format_func=lambda i: str(df_models.iloc[i]["stem"])
210         )
211         row = df_models.iloc[idx]
212         model_path = Path(row["path"])
213
214         with open(model_path, "rb") as fh:
215             _, meta = load_sb3_model_bytes(fh.read(), device="cpu")
216
217         training_metadata = load_model_training_metadata(model_path)
218         suggested_env = training_metadata.get("env_id") or env_id_from_meta_or_name(meta, row["stem"])
219         selected_env_id = col_env.text_input("ID Entorn Sinergym", value=suggested_env or "")
220     else:
221         st.warning("No s'han trobat models .zip")
222         selected_env_id = col_env.text_input("ID Entorn Sinergym", value="Eplus-5zone-hot-continuous-v1")
223
224 else:
225     # En mode manual no hi ha model, però igualment cal un env_id vàlid per crear l'entorn.
226     selected_env_id = col_env.text_input("ID Entorn Sinergym", value="Eplus-5zone-hot-continuous-v1")
227
228 st.divider()
229 init_text = "Inicialitzar Entorn i Simulació" if "Simulació" in use_agent else "Carregar Agent (Sense Simular)"
230 if st.button(
231     init_text,
232     key="ix_initialize_button",
233     type="primary",
234     disabled=not selected_env_id,
235     use_container_width=True,
236 ):
237     with st.spinner("Creant l'entorn EnergyPlus i inicialitzant..."):
238         try:
239             venv, core_env, model_obj, obs, init_warnings = initialize_runtime(
240                 selected_env_id=selected_env_id,
241                 mode_label=use_agent,
242                 model_path=model_path,
243             )
244             for warn_msg in init_warnings:
245                 st.warning(warn_msg)
246
247             st.session_state.ix_env = venv
248             st.session_state.ix_core_env = core_env
249             st.session_state.ix_model = model_obj
250             st.session_state.ix_obs = obs
251             st.session_state.ix_observation_variables = get_runtime_observation_variables(
252                 core_env=core_env,
253                 obs=obs,
254                 model_obj=model_obj,
255                 vec_env=venv,
256             )
257             st.session_state.ix_model_observation_size = get_model_observation_size(model_obj)
258             st.session_state.ix_running = True
259             st.session_state.ix_app_mode = use_agent
260             st.session_state.ix_step = 0
261             st.session_state.ix_reward = 0.0
262             st.session_state.ix_history = []
263             st.session_state.ix_info = {}
264             st.rerun()
265
266         except Exception as e:
267             st.error(f"Error inicialitzant: {e}")
268             stop_simulation()
269
270 # Flux d'interacció quan l'entorn ja està creat.
271 else:
272     venv = st.session_state.ix_env
273     core_env = st.session_state.get("ix_core_env", venv)
274     model = st.session_state.ix_model
275     obs = st.session_state.ix_obs
276     app_mode = st.session_state.get("ix_app_mode", "")
277
278     # En aquest mode només consultem la política: no avancem EnergyPlus.
279     if app_mode and "Avaluació" in app_mode:
280         st.info("Avaluació Manual Estàtica. Aquest mode no avança el rellotge del simulador, simplement mostra què faria l'agent donat qualsevol input de l'edifici.")
281         render_interaction_section(
282             "Avaluació estàtica",
283             "Consulta de política",
284             "Construeix una observació numèrica i comprova quina acció retornaria el model sense avançar el simulador.",
285         )
286     cols_hdr = st.columns([4, 1])
287     if cols_hdr[1].button(
288         "Aturar i Sortir",
289         key="btn_stop_test",

```

```

290     type="primary",
291     use_container_width=True,
292 ):
293     stop_simulation()
294     st.rerun()
295
296 st.subheader("Simulació numèrica: observacions")
297 obs_vars = st.session_state.get("ix_observation_variables") or get_runtime_observation_variables(
298     core_env=core_env,
299     obs=obs,
300     model_obj=model,
301     vec_env=venv,
302 )
303 expected_obs_size = st.session_state.get("ix_model_observation_size") or get_model_observation_size(model)
304
305 # Agafem una observació real com a punt de partida perquè la consulta estàtica no
306 # comenci amb zeros que no tindrien sentit físic.
307 if "ix_test_obs_vals" not in st.session_state or len(st.session_state.ix_test_obs_vals) != len(obs_vars):
308     default_obs_values = extract_policy_test_defaults(
309         obs=obs,
310         info_dict=st.session_state.get("ix_info", {}),
311         obs_vars=obs_vars,
312         vec_env=venv,
313         core_env=core_env,
314     )
315     st.session_state.ix_test_obs_vals = coerce_observation_values(default_obs_values, len(obs_vars))
316
317 st.button(
318     "Generar observacions aleatòries",
319     key="ix_randomize_observations",
320     on_click=randomize_policy_test_observations,
321     args=(obs_vars,),
322 )
323
324 custom_obs = []
325
326 # El formulari evita reexecutar la predicció cada vegada que es toca un input numèric.
327 with st.form("form_policy_test"):
328     grid = st.columns(3)
329     for i, var_name in enumerate(obs_vars):
330         val_d = float(st.session_state.ix_test_obs_vals[i]) if i < len(st.session_state.ix_test_obs_vals) else 0.0
331         unit = infer_unit(var_name)
332         label = f"{var_name} ({unit})" if unit else var_name
333
334         with grid[i % 3]:
335             val = st.number_input(label, value=val_d, format="%.3f")
336             custom_obs.append(val)
337
338     submitted = st.form_submit_button("Consultar Decisió de l'Agent", type="primary")
339
340 if submitted:
341     st.divider()
342     if model is None:
343         st.error("No hi ha model carregat!")
344     else:
345         custom_obs = coerce_observation_values(custom_obs, expected_obs_size)
346         norm_obs = normalize_policy_observation(custom_obs, core_env, venv, expected_obs_size)
347
348         # Predim amb l'observació normalitzada igual que durant l'entrenament.
349         action, state = model.predict(norm_obs, deterministic=True)
350
351         # Desnormalitzem l'acció només per ensenyar-la; al simulador li passarem
352         # sempre el format que espera el VecEnv.
353         action_display = prepare_action_display(action, venv, core_env)
354
355         action_vars = get_wrapper_variables(core_env, "action_variables")
356
357         st.subheader("Resposta immediata de la xarxa neuronal")
358
359         acts_c = st.columns(max(len(action_vars), 1))
360         if len(action_vars) > 0 and len(action_vars) == getattr(action_display, 'size', -1):
361             for i, act_n in enumerate(action_vars):
362                 with acts_c[i]:
363                     unit_a = infer_unit(act_n)
364                     val_str = f"{action_display[i]:.2f} {unit_a}" if unit_a else f"{action_display[i]:.2f}"
365                     st.metric(f"Acció '{act_n}'", val_str)
366         else:
367             st.write(action_display)
368
369         st.caption(f"L'acció crua normalitzada retornada per la xarxa era: `{action}`. Si el model conté `NormalizeAction`,
370 es tradueix als llindars de Sinergym a dalt.")
371
372         # Interrompe execució aquí perquè no es pinti el mode dinàmic abaix
373         st.stop()
374
375 st.success("Simulació en curs amb el motor d'EnergyPlus real. Esperant interacció pas a pas.")
376
377 render_interaction_section(
378     "Simulació activa",
379     "Temps real",
380     "Consulta l'estat actual de l'entorn, aplica accions amb l'agent o manualment i segueix els indicadors principals sense
381     alterar el flux existent.",

```

```

380 )
381
382 # Resum ràpid de l'estat actual.
383 col1, col2, col3, col4 = st.columns(4)
384 col1.metric("Pas Actual", st.session_state.ix_step)
385 col2.metric("Última Recompensa", f"{st.session_state.ix_reward:.3f}")
386 if col4.button("Aturar i Tancar", key="ix_stop_live_button", use_container_width=True):
387     stop_simulation()
388     st.rerun()
389
390 st.divider()
391
392 render_interaction_section(
393     "Observació actual",
394     "Estat de l'entorn",
395     "Mostra els valors físics disponibles a la darrera observació per facilitar la lectura abans de decidir la propera acció.",
396 )
397
398 # Bloc d'observacions actuals.
399 st.subheader("Observació actual (valors físics reals)")
400
401 obs_vars = st.session_state.get("ix_observation_variables") or get_runtime_observation_variables(
402     core_env=core_env,
403     obs=obs,
404     model_obj=model,
405     vec_env=venv,
406 )
407
408 _, display_obs = extract_display_observation(
409     obs=obs,
410     info_dict=st.session_state.get("ix_info", {}),
411     obs_vars=obs_vars,
412     vec_env=venv,
413 )
414
415 if len(obs_vars) == len(display_obs):
416     # Crea un DataFrame agradable
417     df_obs = pd.DataFrame({"Variable": obs_vars, "Valor Original": display_obs})
418     # Arrodonim una mica per que no sigui massa brut
419     df_obs["Valor Original"] = df_obs["Valor Original"].apply(lambda x: round(x, 4) if isinstance(x, (int, float)) else x)
420     st.dataframe(df_obs, use_container_width=True, hide_index=True)
421 else:
422     st.write(display_obs)
423
424 st.divider()
425
426 # Bloc d'execució d'accions.
427 render_interaction_section(
428     "Accions",
429     "Agent o manual",
430     "Escull si vols delegar la decisió a la política carregada o fixar manualment els valors aplicats al simulador.",
431 )
432
433 # Selecció de l'acció a aplicar al pas següent.
434 st.subheader("Acció")
435
436 tab_agent, tab_manual = st.tabs(["Agent", "Control Manual"])
437
438 with tab_agent:
439     if model is None:
440         st.info("No hi ha cap model carregat en aquest moment. Fes servir el mode manual o atura i carrega'n un.")
441     else:
442         n_steps = st.number_input("Quants passos vols avançar d'un cop?", min_value=1, max_value=1000, value=1, step=1, key="n_agent")
443         if st.button(
444             f"Avançar {n_steps} pas(sos) amb l'Agent",
445             key="ix_agent_step_button",
446             type="primary",
447             use_container_width=True,
448 ):
449             try:
450                 with st.spinner(f"L'agent està simulant {n_steps} passos..."):
451                     for _ in range(n_steps):
452                         current_obs = st.session_state.ix_obs
453                         act, _ = model.predict(current_obs, deterministic=True)
454                         apply_live_action_step(venv, core_env, act, "Agent", repeat_n=1)
455             except Exception as e: st.error(str(e))
456             st.rerun()
457
458 with tab_manual:
459     st.write("Selecciona manualment els valors. L'espai d'accions es detecta automàticament des de l'entorn actiu.")
460
461     # Recuperem les variables exposades pels wrappers, no només l'action_space cru.
462     action_vars = get_wrapper_variables(core_env, "action_variables")
463     action_space = getattr(venv, "action_space", getattr(core_env, "action_space", None))
464
465     user_action = []
466
467     if isinstance(action_space, gym.spaces.Box):
468         # Accions contínues: un slider per dimensió.
469         action_dim = int(np.prod(action_space.shape))
470         lows = np.asarray(action_space.low, dtype=np.float32).reshape(-1)
471         highs = np.asarray(action_space.high, dtype=np.float32).reshape(-1)

```



```

471     for i in range(action_dim):
472         v_name = action_vars[i] if len(action_vars) > i else f"Acció_{i}"
473         v_low = float( lows[i] )
474         v_high = float( highs[i] )
475
476         # Alguns espais declaren límits infinits; els acotem per poder pintar sliders.
477         if v_low == -np.inf: v_low = -100.0
478         if v_high == np.inf: v_high = 100.0
479
480         val = st.slider(v_name, min_value=v_low, max_value=v_high, value=(v_low+v_high)/2, step=0.1)
481         user_action.append(val)
482
483     elif isinstance(action_space, gym.spaces.Discrete):
484         # Acció discreta simple: un enter entre 0 i n-1.
485         st.write(f"L'espai d'acció és Discret ({action_space.n} possibilitats).")
486         val = st.number_input("Selecciona Acció", min_value=0, max_value=int(action_space.n)-1, step=1)
487         user_action.append(val)
488
489     elif isinstance(action_space, gym.spaces.MultiDiscrete):
490         nvec = np.asarray(action_space.nvec, dtype=np.int64).reshape(-1)
491         st.write(f"L'espai d'acció és MultiDiscrete ({len(nvec)} dimensions).")
492         for i, n_value in enumerate(nvec):
493             v_name = action_vars[i] if len(action_vars) > i else f"Acció_{i}"
494             val = st.number_input(
495                 v_name,
496                 min_value=0,
497                 max_value=max(int(n_value) - 1, 0),
498                 value=0,
499                 step=1,
500                 key=f"manual_multidiscrete_{i}",
501             )
502             user_action.append(val)
503
504     elif isinstance(action_space, gym.spaces.MultiBinary):
505         action_dim = int(np.prod(action_space.shape))
506         st.write(f"L'espai d'acció és MultiBinary ({action_dim} dimensions).")
507         for i in range(action_dim):
508             v_name = action_vars[i] if len(action_vars) > i else f"Acció_{i}"
509             val = st.checkbox(v_name, value=False, key=f"manual_multibinary_{i}")
510             user_action.append(1 if val else 0)
511
512     else:
513         st.warning("Espai d'acció no suportat actualment per la inserció dinàmica manual.")
514
515     n_steps_mod = st.number_input("Quants passos (repetint l'acció) vols avançar d'un cop?", min_value=1, max_value=1000, value=1, step=1, key="n_manual")
516
517     if st.button(
518         f"Avançar {n_steps_mod} pas(sos) (Manual)",
519         key="ix_manual_step_button",
520         use_container_width=True,
521     ):
522         if isinstance(action_space, (gym.spaces.Box, gym.spaces.Discrete, gym.spaces.MultiDiscrete, gym.spaces.MultiBinary)):
523             apply_live_action_step(venv, core_env, user_action, "Manual", repeat_n=n_steps_mod)
524             st.rerun()
525
526 # Resum ràpid de KPIs del pas actual.
527 if len(st.session_state.ix_history) > 0:
528     render_interaction_section(
529         "Dashboard principal",
530         "Indicadors",
531         "Resumeix l'estat recent de la simulació amb setpoints, temperatures, consum i confort a partir de l'històric disponible.",
532     )
533
534 st.divider()
535 st.subheader("Dashboard principal (estat actual)")
536
537 # Agafem només els dos últims passos per poder calcular deltes sense carregar
538 # tot l'històric a cada rerun.
539 history_len = len(st.session_state.ix_history)
540 curr = st.session_state.ix_history[-1]
541 prev = st.session_state.ix_history[-2] if history_len > 1 else None
542
543 heat_col = find_matching_column(curr, ['heating', 'htg_setpoint'])
544 cool_col = find_matching_column(curr, ['cooling', 'clg_setpoint'])
545 in_temp_col = find_matching_column(curr, ['air_temperature', 'indoor_temperature'])
546 out_temp_col = find_matching_column(curr, ['outdoor_temperature', 'site_outdoor_air_drybulb_temperature'])
547 power_col = find_matching_column(curr, ['demand_rate', 'power', 'hvac_elèctricity_demand'])
548
549 # Primera fila: què ha decidit l'actuador i com canvia respecte al pas anterior.
550 st.markdown("#### Accions de l'actuador")
551 cols_act = st.columns(3)
552 with cols_act[0]:
553     if heat_col:
554         val, dval = curr[heat_col], (curr[heat_col] - prev[heat_col] if prev else 0.0)
555         st.metric("Setpt. Calor (°C)", f"{val:.1f}°C", delta=f"{dval:.1f}°C", delta_color="normal")
556     else: st.info("Sense Setpoint de Calor")
557
558 with cols_act[1]:
559     if cool_col:
560         val, dval = curr[cool_col], (curr[cool_col] - prev[cool_col] if prev else 0.0)
561         # Invertim el color del delta del fred perquè baixar-lo acostuma a gastar més

```

```

561         st.metric("Setpt. Fred (°C)", f"{val:.1f}°C", delta=f"{dval:+.1f}°C", delta_color="inverse")
562     else: st.info("Sense Setpoint de Fred")
563
564 with cols_act[2]:
565     month_col = find_matching_column(curr, ['month'])
566     day_col = find_matching_column(curr, ['day_of_month', 'day'])
567
568     if month_col and day_col:
569         m = int(curr[month_col])
570         d = int(curr[day_col])
571
572         mesos = ["Gen", "Feb", "Mar", "Abr", "Mai", "Jun", "Jul", "Ago", "Set", "Oct", "Nov", "Des"]
573         mes_str = mesos[m-1] if 1 <= m <= 12 else str(m)
574
575         # Intenent esbrinar l'estació aproximada per posar-hi una icona
576         estacio = "\u2744 Hivern" if m in [12, 1, 2] else "\U0001f338 Primavera" if m in [3, 4, 5] else "\u2600 Estiu" if m
in [6, 7, 8] else "\U0001f342 Tardor"
577
578         # Intenent trobar la mida del pas de temps (timestep) en minuts
579         timestep_mins = "Desconegut"
580         try:
581             if hasattr(core_env, 'get_wrapper_attr'):
582                 timestep_mins = getattr(core_env.unwrapped, 'timestep', 15)
583         except: pass
584
585         st.metric(f"Calendari (Pas: {timestep_mins} min)", f"{d} {mes_str}", delta=estacio, delta_color="off")
586     else:
587         # Fallback al reward si no hi ha dades de temps
588         val = curr.get("reward", 0.0)
589         dval = val - prev.get("reward", 10.0) if prev else 0.0
590         st.metric("Recompensa Agent", f"{val:.2f}", delta=f"{dval:+.2f}", delta_color="normal")
591
592 st.markdown("<br>", unsafe_allow_html=True)
593
594 # Segona fila: estat sensorial mínim per entendre si l'acció tenia sentit.
595 st.markdown("#### Sensors (observacions)")
596 cols_obs = st.columns(3)
597 with cols_obs[0]:
598     if in_temp_col:
599         val, dval = curr[in_temp_col], (curr[in_temp_col] - prev[in_temp_col] if prev else 0.0)
600         st.metric("Temp. Interior T1", f"{val:.1f}°C", delta=f"{dval:+.2f}°C", delta_color="off")
601     else: st.info("Sense T. Interior")
602
603 with cols_obs[1]:
604     if out_temp_col:
605         val, dval = curr[out_temp_col], (curr[out_temp_col] - prev[out_temp_col] if prev else 0.0)
606         st.metric("Meteo. exterior", f"{val:.1f}°C", delta=f"{dval:+.1f}°C", delta_color="off")
607     else: st.info("Sense T. Exterior")
608
609 with cols_obs[2]:
610     if power_col:
611         val = curr[power_col]
612         dval = val - prev[power_col] if prev else 0.0
613         st.metric("Consum HVAC (Act/Rate)", f"{val/1000:.2f} kW", delta=f"{dval/1000:+.2f} kW", delta_color="inverse")
614     else: st.info("Sense Consum Electric")
615
616 st.markdown("<br>", unsafe_allow_html=True)
617
618 # Tercera fila: confort calculat amb el rang actiu de la reward.
619 st.markdown("#### Estat de Confort")
620
621 comfort_col1, comfort_col2 = st.columns(2)
622
623 with comfort_col1:
624     if in_temp_col:
625         current_temp = curr[in_temp_col]
626         # Busquem els rangs a la reward activa. Si no hi són, fem servir valors típics
627         # per poder donar feedback sense rebentar la simulació.
628         if hasattr(core_env, 'get_wrapper_attr'):
629             # Sinergym acostuma a guardar aquests rangs dins reward_kwargs.
630             is_summer = False
631             try:
632                 # Inferència simple de temporada a partir del mes actual.
633                 month_col = find_matching_column(curr, ['month'])
634                 if month_col and curr[month_col] >= 6 and curr[month_col] <= 9:
635                     is_summer = True
636             except: pass
637
638             # Depèn de la reward concreta, per això el fallback és intencionat.
639             cfg = core_env.get_wrapper_attr('reward_kwargs') if hasattr(core_env, 'get_wrapper_attr') else {}
640             r_comfort_winter = cfg.get('range_comfort_winter', (20.0, 23.5))
641             r_comfort_summer = cfg.get('range_comfort_summer', (23.0, 26.0))
642         except:
643             r_comfort_winter = (20.0, 23.5)
644             r_comfort_summer = (23.0, 26.0)
645
646         active_range = r_comfort_summer if is_summer else r_comfort_winter
647
648         if active_range[0] <= current_temp <= active_range[1]:

```

```

652         st.success(f"\u2705 Dins el Rang de Confort ({active_range[0]}°C a {active_range[1]}°C)")
653     elif current_temp < active_range[0]:
654         st.error(f"\u2744 Massa Fred (Objectiu: {active_range[0]}°C a {active_range[1]}°C)")
655     else:
656         st.error(f"\U0001f525 Massa Calor (Objectiu: {active_range[0]}°C a {active_range[1]}°C)")
657     else:
658         st.info("No es pot calcular el confort (Sense Temperatura Interior)")
659
660     with comfort_col2:
661         pmv_col = find_matching_column(curr, ['pmv'])
662         ppd_col = find_matching_column(curr, ['ppd'])
663
664         if pmv_col and ppd_col:
665             val_pmv = curr[pmv_col]
666             val_ppd = curr[ppd_col]
667
668             # PMV Ideal: 0 (Rang -0.5 a +0.5). PPD Ideal: < 10%
669             color_pmv = "green" if -0.5 <= val_pmv <= 0.5 else "orange" if -1.0 <= val_pmv <= 1.0 else "red"
670             color_ppd = "green" if val_ppd <= 10.0 else "orange" if val_ppd <= 20.0 else "red"
671
672             st.markdown(f"**Vot Mitjà (PMV):** <span style='color:{color_pmv}; font-weight:bold;'>{val_pmv:.2f}</span>",
673 unsafe_allow_html=True)
674             st.markdown(f"**Insatisfacció (PPD):** <span style='color:{color_ppd}; font-weight:bold;'>{val_ppd:.1f}%</span>",
675 unsafe_allow_html=True)
676         else:
677             st.write("Variables PMV/PPD no disponibles en aquest entorn.")

```

C.8 pages/Mostrar_Entorn.py

Listing 11: pages/Mostrar_Entorn.py

```

1  """Pàgina de visualització d'entorns Sinergym.
2
3  Mostra la geometria 3D, el clima, la configuració i els actius energètics
4  associats a l'entorn seleccionat.
5  """
6
7  from html import escape
8  from typing import Any, Dict, List, Tuple
9
10 import gymnasium as gym
11 import numpy as np
12 import pandas as pd
13 import streamlit as st
14 from sidebar_nav import configure_studio_page
15
16 from backend.mapa_backend import render_location_map
17 from backend.mostrar_entorn_backend import (
18     _format_ui_value,
19     _reward_summary_frames,
20     _sequence_to_df,
21     _tuple_mapping_to_df,
22     load_environment_assets,
23     parse_epw_metadata_and_temperature,
24     summarize_pv,
25 )
26 from backend.paths import BUILDINGS_DIR, WEATHERS_DIR
27 from backend.viewer_3d_backend import plot_3d_model
28 from page_styles.mostrar_entorn import inject_environment_styles
29
30
31 PAGE_TITLE = "Mostrar Entorn"
32 PAGE_LAYOUT = "wide"
33 INTRODUCTION_TEXT = (
34     "Consulta els entorns registrats, explora la geometria de l'edifici "
35     "i revisa configuració, clima i actius energètics des d'una sola pàgina."
36 )
37
38
39 def render_environment_hero() -> None:
40     """Renderitza el bloc principal de la pàgina."""
41
42
43     st.markdown(
44         f"""
45         <section class="environment-hero">
46             <div class="hero-kicker">Visualització d'entorns</div>
47             <h1 class="hero-title">{escape(PAGE_TITLE)}</h1>
48             <div class="hero-copy">{escape(INTRODUCTION_TEXT)}</div>
49         </section>
50         """,
51         unsafe_allow_html=True,
52     )
53
54
55 def render_environment_section(title: str, kicker: str, description: str) -> None:
56     """Renderitza la capçalera visual d'una secció."""

```

```

57
58     st.markdown(f'<h2 class="page-section-title">{escape(title)}</h2>', unsafe_allow_html=True)
59
60
61 def describe_environment_tags(env_name: str) -> List[str]:
62     """Resumeix propietats de l'entorn en etiquetes curtes."""
63
64     tags: List[str] = []
65     lower_name = env_name.lower()
66     if "continuous" in lower_name:
67         tags.append("Accions continues")
68     if "discrete" in lower_name:
69         tags.append("Accions discretes")
70     if "stochastic" in lower_name:
71         tags.append("Entorn aleatori")
72     if "mixed" in lower_name:
73         tags.append("Accions mixtes")
74     if "hot" in lower_name:
75         tags.append("Clima calid")
76     if "cool" in lower_name:
77         tags.append("Clima moderat")
78     if "cold" in lower_name:
79         tags.append("Clima fred")
80     return tags
81
82
83 def render_environment_tags(tags: List[str]) -> None:
84     """Mostra propietats detectades en format d'etiqueta."""
85
86     if not tags:
87         return
88     tags_html = "".join(f'<span class="environment-tag">{escape(tag)}</span>' for tag in tags)
89     st.markdown(f'<div class="environment-tags">{tags_html}</div>', unsafe_allow_html=True)
90
91
92 def render_metric_card(label: str, value: str, variant: str = "") -> None:
93     """Renderitza una targeta compacta amb una metrica o text curt."""
94
95     extra_class = f" environment-metric-card--{variant}" if variant else ""
96     st.markdown(
97         (
98             f'<div class="environment-metric-card{extra_class}">'
99             f'<div class="environment-metric-label">{escape(label)}</div>'
100             f'<div class="environment-metric-value">{escape(value)}</div>'
101             "</div>"
102         ),
103         unsafe_allow_html=True,
104     )
105
106
107 def render_metric_cards(items: List[Tuple[str, str]], columns: int = 2, wide_first: bool = False) -> None:
108     """Renderitza targetes compactes en files estables."""
109
110     if not items:
111         return
112
113     remaining_items = items
114     if wide_first:
115         first_label, first_value = remaining_items[0]
116         render_metric_card(first_label, first_value, variant="wide")
117         remaining_items = remaining_items[1:]
118
119     column_count = max(1, columns)
120     for start in range(0, len(remaining_items), column_count):
121         row_items = remaining_items[start : start + column_count]
122         row_columns = st.columns(len(row_items), gap="small")
123         for column, (label, value) in zip(row_columns, row_items):
124             with column:
125                 render_metric_card(label, value)
126
127
128 def render_detail_card(title: str, rows: List[Tuple[str, str]]) -> None:
129     """Renderitza una targeta compacta de detall."""
130
131     rows_html = "".join(
132         (
133             '<div class="environment-detail-row">'
134             f'<div class="environment-detail-label">{escape(str(label))}</div>'
135             f'<div class="environment-detail-value">{escape(str(value))}</div>'
136             '</div>'
137         )
138         for label, value in rows
139     )
140     card_parts = ['<div class="environment-detail-card">']
141     if title:
142         card_parts.append(f'<div class="environment-detail-title">{escape(title)}</div>')
143     card_parts.append(f'<div class="environment-detail-list">{rows_html}</div>')
144     card_parts.append('</div>')
145     st.markdown("".join(card_parts), unsafe_allow_html=True)
146
147
148 def render_battery_cards(storage_list: List[dict]) -> None:

```

```

149 """Renderitza les bateries detectades com a targetes visuals."""
150
151 for start in range(0, len(storage_list), 2):
152     row_items = storage_list[start : start + 2]
153     row_columns = st.columns(len(row_items), gap="small")
154     for item_index, (column, storage) in enumerate(zip(row_columns, row_items), start=start + 1):
155         rows = [
156             ("Capacitat util", f"{_format_ui_value(storage.get('energy_kwh'))} kWh"),
157             ("Carrega maxima", f"{_format_ui_value(storage.get('p_charge_kw'))} kW"),
158             ("Descarrega maxima", f"{_format_ui_value(storage.get('p_discharge_kw'))} kW"),
159             ("Rendiment", _format_ui_value(storage.get('eff_roundtrip'))),
160         ]
161         display_name = "Bateria" if len(storage_list) == 1 else f"Bateria {item_index}"
162         rows_html = "".join(
163             (
164                 '<div class="environment-detail-row">'
165                 f'<div class="environment-detail-label">{escape(label)}</div>'
166                 f'<div class="environment-detail-value">{escape(value)}</div>'
167                 "</div>"
168             )
169             for label, value in rows
170         )
171         with column:
172             st.markdown(
173                 (
174                     '<div class="environment-battery-card">'
175                     f'<div class="environment-battery-name">{escape(display_name)}</div>'
176                     f'<div class="environment-battery-type">{escape(_format_ui_value(storage.get("type")))}</div>'
177                     f'<div class="environment-battery-list">{rows_html}</div>'
178                     "</div>"
179                 ),
180                 unsafe_allow_html=True,
181             )
182
183 def _reward_label(reward_value: Any) -> str:
184     """Retorna una etiqueta llegible per al reward configurat."""
185
186     return getattr(reward_value, '_name_', _format_ui_value(reward_value))
187
188
189 def render_basic_config(spec: gym.envs.registration.EnvSpec, kwargs: Dict[str, Any]) -> None:
190     """Renderitza la secció de configuració bàsica a la UI de Streamlit."""
191     st.markdown("#### Configuració general")
192     render_detail_card(
193         """
194         [
195             ("ID", _format_ui_value(spec.id)),
196             ("Entry point", _format_ui_value(spec.entry_point)),
197             ("Fitxer edifici", _format_ui_value(kwargs.get('building_file'))),
198             ("Reward", _reward_label(kwargs.get('reward'))),
199         ],
200     )
201
202
203 def render_action_space_summary(action_space: Any) -> None:
204     """Renderitza la secció de resum de l'espai d'acció a la UI de Streamlit."""
205     if action_space is None:
206         st.code('-', language=None)
207         return
208
209     if isinstance(action_space, gym.spaces.Box):
210         low = np.asarray(action_space.low, dtype=float).reshape(-1)
211         high = np.asarray(action_space.high, dtype=float).reshape(-1)
212         bounds_df = pd.DataFrame([
213             {
214                 'Dimensió': index + 1,
215                 'Min': round(float(low_value), 3),
216                 'Max': round(float(high_value), 3),
217             }
218             for index, (low_value, high_value) in enumerate(zip(low, high))
219         ])
220         st.dataframe(bounds_df, hide_index=True, use_container_width=True, height=min(220, 35 * len(bounds_df) + 38))
221         return
222
223     if isinstance(action_space, gym.spaces.Discrete):
224         render_detail_card(
225             """
226             [
227                 ("Tipus", "Discret"),
228                 ("Accions possibles", str(int(action_space.n))),
229             ],
230         )
231         return
232
233     if isinstance(action_space, gym.spaces.MultiDiscrete):
234         nvec = np.asarray(action_space.nvec).reshape(-1)
235         dims_df = pd.DataFrame([
236             {
237                 'Dimensió': index + 1,
238                 'Tipus': 'MultiDiscrete',
239                 'Valors possibles': int(values),
240

```

```

241         }
242         for index, values in enumerate(nvec)
243     ])
244     st.dataframe(dims_df, hide_index=True, use_container_width=True, height=min(220, 35 * len(dims_df) + 38))
245     return
246
247 if isinstance(action_space, gym.spaces.MultiBinary):
248     render_detail_card(
249         "",
250         [
251             ("Tipus", "MultiBinary"),
252             ("Bits", str(int(np.prod(action_space.shape)))),
253         ],
254     )
255     return
256
257 st.code(_format_ui_value(action_space), language=None)
258
259
260 def _actuators_mapping_df(actuators: Dict[str, Any]) -> pd.DataFrame:
261     """Construeix un resum llegible del mapatge entre dimensions i actuadors."""
262
263     rows = []
264     for index, (actuator_key, metadata) in enumerate((actuators or {}).items(), start=1):
265         if isinstance(metadata, dict):
266             rows.append(
267                 {
268                     'Dimensió': index,
269                     'Actuador': actuator_key,
270                     'Tipus objecte': _format_ui_value(metadata.get('element_type')),
271                     'Camp de control': _format_ui_value(metadata.get('value_type')),
272                     'Variable RL': _format_ui_value(metadata.get('variable_name')),
273                 }
274             )
275         else:
276             rows.append(
277                 {
278                     'Dimensió': index,
279                     'Actuador': actuator_key,
280                     'Tipus objecte': _format_ui_value(metadata),
281                     'Camp de control': '-',
282                     'Variable RL': '-',
283                 }
284             )
285     return pd.DataFrame(rows)
286
287
288 def render_kwargs_overview(kwargs: Dict[str, Any]) -> None:
289     """Secció de visió general de renderització de kwargs a la UI de Streamlit."""
290     # Els kwargs de Sinergym barregen variables, actuadors, reward i camps interns.
291     # Els separen en pestanyes perquè sigui revisable sense llegir el JSON sencer.
292     tab_obs, tab_control, tab_reward, tab_misc, tab_raw = st.tabs([
293         'Variables', 'Control', 'Reward', 'Extra', 'JSON brut'
294     ])
295
296     with tab_obs:
297         time_df = _sequence_to_df(kwargs.get('time_variables', []), 'Variable temporal')
298         if not time_df.empty:
299             st.markdown('**Variables temporals**')
300             st.dataframe(time_df, hide_index=True, use_container_width=True)
301
302         variables_df = _tuple_mapping_to_df(kwargs.get('variables', {}), ["Variable d'EnergyPlus", "Ambit"])
303         if not variables_df.empty:
304             st.markdown('**Variables observades**')
305             st.dataframe(variables_df, hide_index=True, use_container_width=True, height=320)
306         else:
307             st.info('No hi ha variables definides per aquest entorn.')
308
309         meters_df = pd.DataFrame(
310             [{ 'Alias': alias, 'Meter': meter } for alias, meter in (kwargs.get('meters') or {}).items()]
311         )
312         if not meters_df.empty:
313             st.markdown('**Meters**')
314             st.dataframe(meters_df, hide_index=True, use_container_width=True)
315
316     with tab_control:
317         st.markdown("'''Espai d'acció'''")
318         render_action_space_summary(kwargs.get('action_space'))
319         st.caption("Cada dimensió correspon, en aquest ordre, als actuadors definits a continuació.")
320
321         actuators_df = _actuators_mapping_df(kwargs.get('actuators', {}))
322         if not actuators_df.empty:
323             st.markdown('**Actuadors**')
324             st.dataframe(actuators_df, hide_index=True, use_container_width=True, height=260)
325         else:
326             st.info('No hi ha actuadors configurats.')
327
328     with tab_reward:
329         scalar_reward_df, grouped_reward_df = _reward_summary_frames(kwargs.get('reward_kwargs') or {})
330         if not scalar_reward_df.empty:
331             st.markdown('**Paràmetres simples**')
332             st.dataframe(scalar_reward_df, hide_index=True, use_container_width=True)

```

```

333     if not grouped_reward_df.empty:
334         st.markdown('**Paràmetres agrupats**')
335         st.dataframe(grouped_reward_df, hide_index=True, use_container_width=True)
336     if scalar_reward_df.empty and grouped_reward_df.empty:
337         st.info('No hi ha reward kwargs definits.')
338
339     with tab_misc:
340         weather_rows = []
341         for variable_name, values in (kwargs.get('weather_variability') or {}).items():
342             # La variabilitat pot arribar com a escalar o com a tuple sigma/mu/periode.
343             # Normalitzem la forma només per mostrar-la en taula.
344             if isinstance(values, (list, tuple)):
345                 sigma = values[0] if len(values) > 0 else '-'
346                 mu = values[1] if len(values) > 1 else '-'
347                 period = values[2] if len(values) > 2 else '-'
348             else:
349                 sigma = values
350                 mu = '-'
351                 period = '-'
352             weather_rows.append({
353                 'Variable': variable_name,
354                 'Sigma': _format_ui_value(sigma),
355                 'Mu': _format_ui_value(mu),
356                 'Periode': _format_ui_value(period),
357             })
358         weather_df = pd.DataFrame(weather_rows)
359         if not weather_df.empty:
360             st.markdown('**Variabilitat meteorològica**')
361             st.dataframe(weather_df, hide_index=True, use_container_width=True)
362
363         misc_df = pd.DataFrame([
364             {'Clau': 'building_config', 'Valor': _format_ui_value(kwargs.get('building_config'))},
365             {'Clau': 'context', 'Valor': _format_ui_value(kwargs.get('context'))},
366             {'Clau': 'initial_context', 'Valor': _format_ui_value(kwargs.get('initial_context'))},
367         ])
368         st.markdown('**Altres camps**')
369         st.dataframe(misc_df, hide_index=True, use_container_width=True)
370
371     with tab_raw:
372         st.json(kwargs)
373
374
375 def render_3d_viewer(
376     records: List[dict],
377     zones: List[str],
378     types: List[str],
379     z_clip_range: Tuple[float, float],
380     pv_list: List[dict],
381     name_index: Dict[str, int],
382 ) -> None:
383     """Renderitza la secció del visor de geometria de l'edifici en 3D.
384
385     Mostra la figura 3D interactiva Plotly. Totes les superfícies es mostren amb
386     per defecte amb tots els tipus i zones visibles.
387     """
388     if records:
389         vis_types = {surface_type: True for surface_type in types}
390         if pv_list:
391             vis_types["PV"] = True
392         vis_zones = {zone: True for zone in zones}
393         fig3d = plot_3d_model(
394             records=records,
395             visible_types=vis_types,
396             visible_zones=vis_zones,
397             z_clip=z_clip_range,
398             pv_list=pv_list,
399             name_index=name_index,
400         )
401         st.plotly_chart(fig3d, use_container_width=True)
402         st.caption("Passeu el cursor per veure zona, area i orientacio. Aspecte data i model recentrat.")
403     else:
404         st.warning("No s'han trobat superfícies amb coordenades.")
405
406
407 def render_environment_page() -> None:
408     """Renderitza la pàgina amb la presentació visual actualitzada."""
409
410     configure_studio_page(PAGE_TITLE, "pages/Mostrar_Entorn.py", layout=PAGE_LAYOUT)
411     inject_environment_styles()
412
413     render_environment_hero()
414     all_envs = sorted([env_name for env_name in gym.envs.registry.keys() if env_name.startswith("Eplus-")])
415     render_environment_section(
416         "Selecció de l'entorn",
417         "Catàleg",
418         "Filtra els entorns registrats i tria quin vols obrir abans de veure la representació geomètrica i les dades de configuració.",
419     ),
420
421     search_col, select_col = st.columns(2, gap="large")
422     with search_col:
423         search_query = st.text_input("Cerca un entorn per nom", placeholder="Ex: warehouse, hot, continuous...")

```



```

424 filtered_envs = [env_name for env_name in all_envs if search_query.lower() in env_name.lower()] if search_query else all_envs
425 with select_col:
426     env_selected = st.selectbox(
427         "Selecciona un entorn",
428         filtered_envs if filtered_envs else ["Cap coincidència"],
429         disabled=not filtered_envs,
430     )
431
432
433 st.caption(f"{len(filtered_envs)} entorns disponibles amb el filtre actual.")
434 if not filtered_envs:
435     st.warning("No s'ha trobat cap entorn amb aquest filtre.")
436     return
437
438 render_environment_tags(describe_environment_tags(env_selected))
439
440 try:
441     spec = gym.spec(env_selected)
442     kwargs = spec.kwargs
443     bfile = kwargs.get("building_file")
444
445     weather_file = None
446     if "weather_files" in kwargs:
447         raw = kwargs["weather_files"]
448         weather_file = raw if isinstance(raw, str) else raw[0]
449     elif "weather" in kwargs:
450         wf = kwargs["weather"].get("weather_files", [])
451         weather_file = wf if isinstance(wf, str) else (wf[0] if wf else None)
452
453     render_environment_section(
454         "Visualització de l'edifici",
455         "Geometria",
456         "La pàgina mostra la geometria 3D de l'edifici amb totes les superfícies visibles.",
457     )
458
459     records = []
460     name_index = {}
461     pv_list = []
462     storage_list = []
463     z_clip_range = (0.0, 0.0)
464     zones: List[str] = []
465     types: List[str] = []
466
467     if bfile and (BUILDINGS_DIR / bfile).exists():
468         with st.spinner("Carregant dades de l'entorn..."):
469             # load_environment_assets centralitza el parseig pesat de l'IDF/epJSON.
470             # La pàgina només decideix què es mostra i amb quins filtres.
471             ep_path = str(BUILDINGS_DIR / bfile)
472             assets = load_environment_assets(ep_path)
473             records = assets["records"]
474             zones = assets["zones"]
475             types = assets["types"]
476             z_clip_range = assets["z_clip_range"]
477             name_index = assets["name_index"]
478             pv_list = assets["pv"]
479             storage_list = assets["storage"]
480
481             render_3d_viewer(records, zones, types, z_clip_range, pv_list, name_index)
482     else:
483         st.info("No disponible per a aquest format o falta el fitxer d'edifici.")
484
485     render_environment_section(
486         "Configuració i dades associades",
487         "Context tècnic",
488         "Reuneix la configuració base de l'entorn, els paràmetres exposats per Sinergym i la informació del fitxer meteorològic associat.",
489     )
490     col1, col2 = st.columns(2)
491
492     with col1:
493         render_basic_config(spec, kwargs)
494
495         weather_loaded = False
496         if weather_file:
497             wpath = WEATHERS_DIR / weather_file
498             if wpath.exists():
499                 weather_loaded = True
500                 loc, avg_temp, cz = parse_epw_metadata_and_temperature(str(wpath))
501                 st.markdown("### Clima i localització")
502                 st.caption("Resum del fitxer meteorològic actiu i ubicació aproximada de l'entorn.")
503                 render_detail_card(
504                     "Informació general",
505                     [
506                         ("Fitxer clima", weather_file),
507                         ("Temp. mitjana", f"{avg_temp:.2f} C"),
508                         ("Zona climàtica", str(cz)),
509                         ("Ciutat", f"{loc['city']}, {loc['country']}"),
510                         ("Coordenades", f"{loc['latitude']:.3f}, {loc['longitude']:.3f}"),
511                     ],
512                 )
513                 st.caption("Ubicació aproximada")
514                 render_location_map(

```



```

515         loc["latitude"],
516         loc["longitude"],
517         zoom=4,
518         color=(255, 0, 0, 220),
519         radius=15000,
520     )
521
522     if not weather_loaded:
523         st.info("No hi ha cap fitxer de clima disponible per mostrar en aquesta configuració.")
524
525     st.markdown("<div style='height: 0.7rem;'></div>", unsafe_allow_html=True)
526     st.markdown("### Solar i bateria")
527     st.caption("Actius energètics detectats a partir de la geometria i dels objectes associats.")
528
529     if pv_list:
530         pv_summary = summarize_pv(records, name_index, pv_list)
531         surface_names = pv_summary["surfaces_with_pv"]
532         # Mostrem només unes quantes superfícies perquè alguns models tenen molts
533         # panells i la targeta quedaria il·legible.
534         surface_preview = ", ".join(surface_names[:4]) if surface_names else "Sense coincidències"
535         if len(surface_names) > 4:
536             surface_preview += f" +{len(surface_names) - 4} mes"
537         render_detail_card(
538             "Panells fotovoltaics",
539             [
540                 ("Generadors PV", str(pv_summary["count"])),
541                 ("Cares actives", str(len(surface_names))),
542                 ("Area aprox.", "0 m2" if pv_summary["area_m2"] is None else f"{pv_summary['area_m2']:.2f} m2"),
543                 ("Tilt mig", "0 deg" if pv_summary["avg_tilt"] is None else f"{pv_summary['avg_tilt']:.0f} deg"),
544                 ("Azimut mig", "0 deg" if pv_summary["avg_azimuth"] is None else f"{pv_summary['avg_azimuth']:.0f} deg"),
545                 ("Superfícies", surface_preview),
546             ],
547         )
548     else:
549         st.info("No s'han detectat panells fotovoltaics associats a superfícies.")
550
551     if storage_list:
552         render_battery_cards(storage_list)
553     else:
554         st.info("No s'han detectat objectes de bateria.")
555
556     with col2:
557         render_kwargs_overview(kwargs)
558
559     except Exception as error:
560         st.error(f"Error carregant l'entorn: {error}")
561
562 render_environment_page()
563 st.stop()
564

```

C.9 pages/Resultats.py

Listing 12: pages/Resultats.py

```

1  """Streamlit frontend per a la pàgina de resultats de BEMS-RL STUDIO."""
2
3  from __future__ import annotations
4
5  from html import escape
6  from pathlib import Path
7
8  import streamlit as st
9  from sidebar_nav import configure_studio_page
10
11  from backend.resultats_backend import (
12      DashboardData,
13      build_results_page_state,
14      get_run_artifacts,
15      load_dashboard_data,
16  )
17  from page_components.resultats_dashboard import render_inline_dashboard
18  from page_styles.resultats import inject_results_page_styles
19
20
21  PAGE_TITLE = "Resultats"
22  PAGE_LAYOUT = "wide"
23  INTRODUCTION_TEXT = (
24      "Explora els resultats d'una execució RL, descarrega els CSV principals "
25      "i obre el dashboard analític."
26  )
27
28  def build_download_filename(prefix: str, run_name: str) -> str:
29      """Crea un nom de fitxer de descàrrega estable CSV per a una execució seleccionada."""
30
31      sanitized_run_name = "".join(
32          c if c.isalnum() or c in {"-", "_"} else "-" for c in run_name.strip()
33      ).strip("-")

```

```

34     if not sanitized_run_name:
35         sanitized_run_name = "entrenament"
36     return f"{prefix}-{sanitized_run_name}.csv"
37
38
39 def render_hero() -> None:
40     """Bloc d'heroi de la pàgina de resultats de renderització a la UI de Streamlit."""
41
42     st.markdown(
43         f"""
44         <section class="results-hero">
45             <div class="hero-kicker">Analítica i exportació</div>
46             <h1 class="hero-title">{escape(PAGE_TITLE)}</h1>
47             <div class="hero-copy">{escape(INTRODUCTION_TEXT)}</div>
48         </section>
49         """,
50         unsafe_allow_html=True,
51     )
52
53 def render_results_page() -> None:
54     """Renderitza l'interpret d'ordres de la pàgina de resultats, el selector d'execució, el panell i les descàrregues CSV a la UI de Streamlit."""
55
56     configure_studio_page(PAGE_TITLE, "pages/Resultats.py", layout=PAGE_LAYOUT)
57     inject_results_page_styles()
58
59     base_dir = str(Path.cwd())
60     render_hero()
61
62     # La cerca de runs i la validació dels CSV queden fora de la UI principal perquè
63     # aquesta funció només decideixi què es mostra i quan es carrega el dashboard.
64     page_state = build_results_page_state(base_dir)
65
66     st.markdown("<div style='height: 1.15rem;'></div>", unsafe_allow_html=True)
67     st.markdown('<h2 class="section-title">Selecció</h2>', unsafe_allow_html=True)
68
69     if page_state.warning_message:
70         st.warning(page_state.warning_message)
71         st.stop()
72
73     try:
74         selection_cols = st.columns([10, 1.15], gap="small", vertical_alignment="center")
75     except TypeError:
76         selection_cols = st.columns([10, 1.15], gap="small")
77     with selection_cols[0]:
78         selected_run = st.selectbox("Selecciona l'execució", page_state.run_dirs)
79
80     run_artifacts = get_run_artifacts(selected_run.path)
81     if run_artifacts.error_message:
82         st.error(run_artifacts.error_message)
83         st.stop()
84
85     with selection_cols[1]:
86         try:
87             generate_requested = st.button(
88                 " ",
89                 icon="material/insert_chart:",
90                 help="Generar Dashboard",
91                 key="generate_dashboard_button",
92                 use_container_width=True,
93             )
94         except TypeError:
95             generate_requested = st.button(
96                 "\u25eb",
97                 help="Generar Dashboard",
98                 key="generate_dashboard_button",
99                 use_container_width=True,
100             )
101
102     if generate_requested:
103         with st.spinner("Carregant dades del dashboard..."):
104             try:
105                 # Guardem el dashboard a session_state perquè no es torni a carregar en cada
106                 # interacció menor, com descarregar un CSV o obrir una pestanya.
107                 dashboard_data = load_dashboard_data(
108                     run_artifacts.progress_path,
109                     run_artifacts.observations_path,
110                 )
111                 st.session_state["inline_dashboard_data"] = dashboard_data
112                 st.session_state["inline_dashboard_run_path"] = selected_run.path
113             except Exception as exc:
114                 st.error(f"Error carregant el dashboard: {exc}")
115
116     inline_data: DashboardData | None = st.session_state.get("inline_dashboard_data")
117     active_run_path: str | None = st.session_state.get("inline_dashboard_run_path")
118
119     if inline_data is not None and active_run_path == selected_run.path:
120         render_inline_dashboard(inline_data, page_state.run_dirs, selected_run.path)
121
122     download_cols = st.columns(2, gap="large")
123     with download_cols[0]:
124         st.download_button(

```

```

125         "Descarregar dades d'entrenament",
126         data=Path(run_artifacts.progress_path).read_bytes(),
127         file_name=build_download_filename("entrenament", selected_run.name),
128         mime="text/csv",
129         use_container_width=True,
130     )
131     with download_cols[1]:
132         st.download_button(
133             "Descarregar dades monitor",
134             data=Path(run_artifacts.observations_path).read_bytes(),
135             file_name=build_download_filename("monitor", selected_run.name),
136             mime="text/csv",
137             use_container_width=True,
138         )
139
140
141 render_results_page()

```

C.10 pages/Simular_Entorn.py

Listing 13: pages/Simular_Entorn.py

```

1 from __future__ import annotations
2
3 from functools import partial
4 from html import escape
5 import queue
6 import threading
7 import time
8 import traceback
9
10 import streamlit as st
11 from page_styles.simular_entorn import inject_baseline_styles
12 from sidebar_nav import configure_studio_page
13
14 from backend.simular_entorn_backend import (
15     ONE_YEAR_STEPS,
16     describe_environment,
17     list_environment_ids,
18     run_baseline_simulation,
19 )
20
21
22 PAGE_TITLE = "Simular Entorn"
23 PAGE_LAYOUT = "wide"
24 INTRODUCTION_TEXT = (
25     "Executa un baseline dels entorns amb els schedules normals d'EnergyPlus, "
26     "sense control RL, i compara'l amb execucions que ja teniu guardades."
27 )
28
29
30 def _key(name: str) -> str:
31     """Key."""
32     return f"baseline_{name}"
33
34
35 def render_baseline_hero() -> None:
36     """Mostra la secció d'heroi de referència a la UI de Streamlit."""
37     st.markdown(
38         f"""
39         <section class="results-hero">
40             <div class="hero-kicker">Simulacio baseline</div>
41             <h1 class="hero-title">{escape(PAGE_TITLE)}</h1>
42             <div class="hero-copy">{escape(INTRODUCTION_TEXT)}</div>
43         </section>
44         """,
45         unsafe_allow_html=True,
46     )
47
48
49
50 def render_section_header(title: str, description: str) -> None:
51     """Renderitza la secció de la capçalera de la secció a la UI de Streamlit."""
52     st.markdown(f'<h2 class="section-title">{escape(title)}</h2>', unsafe_allow_html=True)
53     st.markdown(
54         f"""
55         <section class="section-card" style="margin-bottom:0.85rem;">
56             <div class="section-copy">{escape(description)}</div>
57         </section>
58         """,
59         unsafe_allow_html=True,
60     )
61
62
63 def empty_baseline_runtime() -> dict[str, object]:
64     """Temps d'execució de base buit."""
65     return {
66         "running": False,

```

```

67         "completed": False,
68         "cancel_requested": False,
69         "needs_full_rerun": False,
70         "thread": None,
71         "queue": None,
72         "stop_event": None,
73         "result": None,
74         "error": None,
75         "traceback": None,
76         "env_id": None,
77         "steps_target": 1,
78         "latest_step": 0,
79         "status": "",
80         "started_at": None,
81         "stopped_at": None,
82         "frozen_elapsed_seconds": None,
83     }
84
85
86 def ensure_baseline_runtime() -> dict[str, object]:
87     """Assegureu-vos el temps d'execució de referència."""
88     runtime_key = _key("runtime")
89     if runtime_key not in st.session_state:
90         st.session_state[runtime_key] = empty_baseline_runtime()
91     return st.session_state[runtime_key]
92
93
94 def reset_baseline_runtime() -> dict[str, object]:
95     """Restableix el temps d'execució de referència."""
96     runtime = ensure_baseline_runtime()
97     previous_thread = runtime.get("thread")
98     if previous_thread is not None and getattr(previous_thread, "is_alive", lambda: False)():
99         return runtime
100
101     st.session_state[_key("runtime")] = empty_baseline_runtime()
102     return st.session_state[_key("runtime")]
103
104
105 def drain_baseline_runtime(runtime: dict[str, object]) -> None:
106     """Esgota el temps d'execució de la línia base."""
107     event_queue = runtime.get("queue")
108     if event_queue is None:
109         return
110
111     # El worker escriu esdeveniments en una cua i Streamlit els buida a cada rerun.
112     # Així la simulació pot continuar en segon pla sense bloquejar la interfície.
113     while True:
114         try:
115             event = event_queue.get_nowait()
116         except queue.Empty:
117             break
118
119         event_type = str(event.get("type") or "")
120         if event_type == "progress":
121             runtime["latest_step"] = max(
122                 int(runtime.get("latest_step") or 0),
123                 int(event.get("step_number") or 0),
124             )
125             runtime["steps_target"] = max(
126                 int(runtime.get("steps_target") or 1),
127                 int(event.get("total_steps") or 1),
128             )
129             runtime["status"] = str(event.get("status") or runtime.get("status") or "")
130         elif event_type == "result":
131             runtime["result"] = event.get("result")
132             result = runtime["result"]
133             if result is not None and runtime.get("frozen_elapsed_seconds") is None:
134                 runtime["frozen_elapsed_seconds"] = float(getattr(result, "elapsed_seconds", 0.0))
135             runtime["completed"] = True
136             runtime["running"] = False
137             runtime["needs_full_rerun"] = True
138         elif event_type == "error":
139             runtime["error"] = str(event.get("error") or "Error desconegut durant la simulació.")
140             runtime["traceback"] = event.get("traceback")
141             runtime["completed"] = True
142             runtime["running"] = False
143             runtime["needs_full_rerun"] = True
144         elif event_type == "finished":
145             runtime["completed"] = True
146             runtime["running"] = False
147             runtime["thread"] = None
148             runtime["needs_full_rerun"] = True
149
150     thread = runtime.get("thread")
151     if thread is not None and not thread.is_alive() and runtime.get("running"):
152         runtime["running"] = False
153         runtime["completed"] = True
154         runtime["thread"] = None
155         runtime["needs_full_rerun"] = True
156
157
158 def sync_baseline_runtime() -> dict[str, object]:

```

```

159 """Sincronitza el temps d'execució de referència."""
160 runtime = ensure_baseline_runtime()
161 drain_baseline_runtime(runtime)
162 return runtime
163
164
165 def consume_baseline_rerun_flag(runtime: dict[str, object] | None = None) -> bool:
166     """Consumeix la bandera de repetició de la línia de base."""
167     active_runtime = runtime if runtime is not None else ensure_baseline_runtime()
168     if active_runtime.get("needs_full_rerun") and not active_runtime.get("running"):
169         active_runtime["needs_full_rerun"] = False
170         return True
171     return False
172
173
174 def request_baseline_stop(runtime: dict[str, object] | None = None) -> bool:
175     """Sol·licita l'aturada de la línia de base."""
176     active_runtime = runtime if runtime is not None else ensure_baseline_runtime()
177     if not active_runtime.get("running") or active_runtime.get("stop_event") is None:
178         return False
179
180     active_runtime["cancel_requested"] = True
181     active_runtime["status"] = "Aturant simulacio..."
182     active_runtime["stopped_at"] = time.time()
183     started_at = active_runtime.get("started_at")
184     if started_at:
185         active_runtime["frozen_elapsed_seconds"] = max(0.0, float(active_runtime["stopped_at"]) - float(started_at))
186     active_runtime["stop_event"].set()
187     return True
188
189
190 def push_baseline_progress_event(
191     event_queue: "queue.Queue[dict[str, object]]",
192     job_config: dict[str, object],
193     step_number: int,
194     total_steps: int,
195 ) -> None:
196     """Envieu un esdeveniment de progrés de referència a la cua de temps d'execució de la pàgina."""
197     event_queue.put(
198         {
199             "type": "progress",
200             "step_number": int(step_number or 0),
201             "total_steps": int(total_steps or job_config["steps_target"]),
202             "status": f"Passos: {int(step_number or 0)}/{int(total_steps or job_config['steps_target'])}",
203         }
204     )
205
206
207 def baseline_worker(
208     job_config: dict[str, object],
209     event_queue: "queue.Queue[dict[str, object]]",
210     stop_event: threading.Event,
211 ) -> None:
212     """Executa la simulació baseline en segon pla."""
213     try:
214         result = run_baseline_simulation(
215             env_id=str(job_config["env_id"]),
216             steps_target=int(job_config["steps_target"]),
217             seed=int(job_config["seed"]) if job_config.get("seed") is not None else None,
218             should_stop=stop_event.is_set,
219             on_progress=partial(push_baseline_progress_event, event_queue, job_config),
220         )
221         event_queue.put({"type": "result", "result": result})
222     except Exception as exc:
223         event_queue.put(
224             {
225                 "type": "error",
226                 "error": str(exc),
227                 "traceback": traceback.format_exc(),
228             }
229         )
230     finally:
231         event_queue.put({"type": "finished"})
232
233
234 def start_baseline_run(request: dict[str, object]) -> dict[str, object]:
235     """Inicia l'execució de la línia de base."""
236     runtime = reset_baseline_runtime()
237     response: dict[str, object] = {
238         "started": False,
239         "errors": [],
240         "runtime": runtime,
241     }
242
243     if runtime.get("running"):
244         response["errors"].append("Ja hi ha una simulacio en curs.")
245         return response
246
247     env_id = str(request.get("env_id") or "").strip()
248     if not env_id:
249         response["errors"].append("Selecciona un entorn valid abans d'iniciar la simulacio.")
250         return response

```

```

251
252 if env_id not in list_environment_ids():
253     response["errors"].append("L'entorn seleccionat no esta registrat a Sinergym.")
254     return response
255
256 event_queue = queue.Queue[dict[str, object]] = queue.Queue()
257 stop_event = threading.Event()
258 job_config = {
259     "env_id": env_id,
260     "steps_target": int(request.get("steps_target") or ONE_YEAR_STEPS),
261     "seed": int(request.get("seed")) if request.get("seed") is not None else None,
262 }
263 worker_thread = threading.Thread(
264     target=baseline_worker,
265     args=(job_config, event_queue, stop_event),
266     daemon=True,
267 )
268
269 runtime.update(
270     {
271         "running": True,
272         "completed": False,
273         "cancel_requested": False,
274         "needs_full_rerun": False,
275         "thread": worker_thread,
276         "queue": event_queue,
277         "stop_event": stop_event,
278         "result": None,
279         "error": None,
280         "traceback": None,
281         "env_id": env_id,
282         "steps_target": int(job_config["steps_target"]),
283         "latest_step": 0,
284         "status": "Inicialitzant simulacio baseline...",
285         "started_at": time.time(),
286         "stopped_at": None,
287         "frozen_elapsed_seconds": None,
288     }
289 )
290 worker_thread.start()
291
292 response["started"] = True
293 response["runtime"] = runtime
294 return response
295
296
297 def render_environment_summary(env_id: str) -> None:
298     """Renderitza la secció de resum de l'entorn a la UI de Streamlit."""
299     summary = describe_environment(env_id)
300     step_text = f"{summary.step_minutes:g} min" if summary.step_minutes else "No disponible"
301
302     info_cols = st.columns(4)
303     with info_cols[0]:
304         st.metric("Variables observades", summary.variables_count)
305     with info_cols[1]:
306         st.metric("Meters", summary.meters_count)
307     with info_cols[2]:
308         st.metric("Actuadors RL", summary.actuators_count)
309     with info_cols[3]:
310         st.metric("Pas de simulacio", step_text)
311
312     st.markdown(
313         (
314             '<div class="baseline-chip-row">'
315             f'<span class="baseline-chip"><strong>Building</strong> {escape(summary.building_file or "No disponible")}</span>'
316             f'<span class="baseline-chip"><strong>Weathers</strong> {summary.weather_count}</span>'
317             f'<span class="baseline-chip"><strong>Mode</strong> schedules EnergyPlus</span>'
318             f'<span class="baseline-chip"><strong>Resolucio</strong> pas fix de {step_text}</span>'
319             '</div>'
320         ),
321         unsafe_allow_html=True,
322     )
323
324
325 def render_completed_baseline(runtime: dict[str, object]) -> None:
326     """Mostra la secció de referència completada a la UI de Streamlit."""
327     result = runtime.get("result")
328     if result is None or runtime.get("running"):
329         return
330
331     if bool(getattr(result, "cancelled", False)):
332         st.warning(
333             f"Simulacio aturada manualment despres de {float(getattr(result, 'elapsed_seconds', 0.0)):.2f} s "
334             f"i {int(getattr(result, 'completed_steps', 0) or 0)} passos."
335         )
336     else:
337         st.success(
338             f"Simulacio finalitzada en {float(getattr(result, 'elapsed_seconds', 0.0)):.2f} s "
339             f"i {int(getattr(result, 'completed_steps', 0) or 0)} passos."
340         )
341
342     run_path = getattr(result, "run_path", "")

```

```

343 if run_path:
344     st.caption(f"Run guardada a: {run_path}")
345 if hasattr(st, "page_link"):
346     st.page_link("pages/Resultats.py", label="Veure resultats a Resultats", icon=":material/insights:")
347
348
349 def render_baseline_runtime(runtime: dict[str, object]) -> None:
350     """Mostra la secció de temps d'execució de la línia de base a la UI de Streamlit."""
351     render_section_header(
352         "Estat de la simulació",
353         "Segueix el progrés del baseline i obre les visualitzacions quan la run estigui llesta.",
354     )
355
356     steps_target = max(int(runtime.get("steps_target") or 1), 1)
357     latest_step = min(int(runtime.get("latest_step") or 0), steps_target)
358     progress_value = min(latest_step / steps_target, 1.0)
359     elapsed_seconds = 0
360     started_at = runtime.get("started_at")
361     result = runtime.get("result")
362     # Quan la simulació acaba congelem el temps; si no, el rellotge seguiria pujant a cada rerun.
363     if result is not None:
364         elapsed_seconds = int(float(getattr(result, "elapsed_seconds", 0.0)))
365     elif runtime.get("frozen_elapsed_seconds") is not None:
366         elapsed_seconds = int(float(runtime.get("frozen_elapsed_seconds") or 0.0))
367     elif started_at:
368         elapsed_seconds = max(0, int(time.time() - float(started_at)))
369
370     st.progress(progress_value)
371
372     metric_cols = st.columns(3)
373     with metric_cols[0]:
374         st.metric("Passos", f"{latest_step}/{steps_target}")
375     with metric_cols[1]:
376         st.metric("Progrés", f"{progress_value * 100:.0f}%")
377     with metric_cols[2]:
378         st.metric("Temps", f"{elapsed_seconds // 60}m {elapsed_seconds % 60:02d}s")
379
380     status_text = str(runtime.get("status") or "").strip()
381     if status_text:
382         st.caption(status_text)
383
384     if runtime.get("cancel_requested") and runtime.get("running"):
385         st.warning("S'ha demanat l'aturada. La simulació es tancarà al proper punt segur.")
386
387     if runtime.get("error"):
388         st.error(str(runtime.get("error")))
389         if runtime.get("traceback"):
390             with st.expander("Traceback intern", expanded=False):
391                 st.code(str(runtime.get("traceback")), language="text")
392     return
393
394 render_completed_baseline(runtime)
395
396
397 def render_baseline_runtime_frame() -> None:
398     """Renderitza la secció de marc de temps d'execució de referència a la UI de Streamlit."""
399     runtime = sync_baseline_runtime()
400     if runtime.get("running") or runtime.get("completed") or runtime.get("error") or runtime.get("result"):
401         st.divider()
402         render_baseline_runtime(runtime)
403
404     if consume_baseline_rerun_flag(runtime):
405         st.rerun()
406
407
408 if hasattr(st, "fragment"):
409     render_baseline_runtime_fragment = st.fragment(run_every="1s")(render_baseline_runtime_frame)
410 else:
411     def render_baseline_runtime_fragment() -> None:
412         """Renderitza la secció del fragment de temps d'execució de la línia de base a la UI de Streamlit."""
413         render_baseline_runtime_frame()
414         runtime = sync_baseline_runtime()
415         if runtime.get("running"):
416             time.sleep(1.0)
417             st.rerun()
418
419
420 configure_studio_page(PAGE_TITLE, "pages/Simular_Entorn.py", layout=PAGE_LAYOUT)
421 inject_baseline_styles()
422
423 environment_ids = list_environment_ids()
424 if not environment_ids:
425     st.error("No hi ha entorns Sinergym registrats disponibles per executar un baseline.")
426     st.stop()
427
428 st.session_state.setdefault(_key("selected_env"), environment_ids[0])
429 st.session_state.setdefault(_key("steps"), ONE_YEAR_STEPS)
430 st.session_state.setdefault(_key("seed"), 0)
431
432 render_baseline_hero()
433
434 runtime = sync_baseline_runtime()

```

```

435 controls_disabled = bool(runtime.get("running"))
436
437 st.markdown("<div style='height: 1.15rem;'></div>", unsafe_allow_html=True)
438 st.markdown('<h2 class="section-title">Configuracio del baseline</h2>', unsafe_allow_html=True)
439
440 selected_env = st.selectbox(
441     "Entorn Sinergym",
442     options=environment_ids,
443     index=environment_ids.index(st.session_state[_key("selected_env")]) if st.session_state[_key("selected_env")] in environment_ids
444     else 0,
445     disabled=controls_disabled,
446 )
447 st.session_state[_key("selected_env")] = selected_env
448
449 render_environment_summary(selected_env)
450
451 config_cols = st.columns(2, gap="medium")
452 with config_cols[0]:
453     steps_limit = st.number_input(
454         "Limit de passos",
455         min_value=1,
456         max_value=5_000_000,
457         value=int(st.session_state[_key("steps")]),
458         step=35040,
459         disabled=controls_disabled,
460     )
461 with config_cols[1]:
462     seed_value = st.number_input(
463         "Seed inicial",
464         min_value=0,
465         max_value=999_999,
466         value=int(st.session_state[_key("seed")]),
467         step=1,
468         disabled=controls_disabled,
469     )
470
471 st.markdown(
472     f'<div class="baseline-inline-note">Recomanacio: usa {ONE_YEAR_STEPS} passos per generar una passada anual comparable amb una run
473     d\'agent.</div>',
474     unsafe_allow_html=True,
475 )
476
477 st.divider()
478 st.markdown('<h2 class="section-title">Control</h2>', unsafe_allow_html=True)
479
480 run_cols = st.columns(2, gap="medium")
481 with run_cols[0]:
482     start_requested = st.button(
483         "Iniciar simulacio baseline",
484         type="primary",
485         use_container_width=True,
486         disabled=controls_disabled,
487     )
488 with run_cols[1]:
489     cancel_requested = st.button(
490         "Cancelar",
491         use_container_width=True,
492         disabled=not bool(runtime.get("running")),
493     )
494
495 if cancel_requested and request_baseline_stop(runtime):
496     st.rerun()
497
498 if start_requested:
499     start_result = start_baseline_run(
500         {
501             "env_id": selected_env,
502             "steps_target": int(steps_limit),
503             "seed": int(seed_value),
504         }
505     )
506     for error in start_result.get("errors") or []:
507         st.error(str(error))
508     if start_result.get("started"):
509         st.rerun()
510
511 render_baseline_runtime_fragment()

```

C.11 pages/Visor_Climes_EPW.py

Listing 14: pages/Visor_Climes_EPW.py

```

1 from __future__ import annotations
2
3 import importlib
4 from dataclasses import dataclass
5 from html import escape
6

```



```

7 import pandas as pd
8 import streamlit as st
9
10 from page_styles.visor_climes_epw import inject_epw_viewer_styles
11 from sidebar_nav import configure_studio_page
12
13 from backend import visor_epw_backend as epw_backend
14 from backend.epw_figures import (
15     build_active_month_axis,
16     build_annual_heatmap_figure,
17     build_comfort_radiation_figure,
18     build_daily_temperature_band_figure,
19     build_focus_timeseries_figure,
20     build_heatmap_figure,
21     build_hourly_profile_figure,
22     build_monthly_climate_figure,
23     build_monthly_solar_figure,
24     build_monthly_wind_rose_grid_figure,
25 )
26 from backend.mapa_backend import render_location_map
27
28 epw_backend = importlib.reload(epw_backend)
29
30 WEATHER_ROOT = epw_backend.WEATHER_ROOT
31 DEG = epw_backend.DEG
32 DEG_C = epw_backend.DEG_C
33 KWH_PER_M2 = epw_backend.KWH_PER_M2
34 aggregate_epw_timeseries = epw_backend.aggregate_epw_timeseries
35 build_annual_hourly_heatmap_table = epw_backend.build_annual_hourly_heatmap_table
36 build_comfort_radiation_table = epw_backend.build_comfort_radiation_table
37 build_daily_temperature_profile = epw_backend.build_daily_temperature_profile
38 build_download_frame = epw_backend.build_download_frame
39 build_heatmap_table = epw_backend.build_heatmap_table
40 build_hourly_profile = epw_backend.build_hourly_profile
41 build_monthly_summary = epw_backend.build_monthly_summary
42 build_monthly_wind_rose_tables = epw_backend.build_monthly_wind_rose_tables
43 list_epw_catalog = epw_backend.list_epw_catalog
44 load_epw_bundle = epw_backend.load_epw_bundle
45 summarize_epw_metrics = epw_backend.summarize_epw_metrics
46
47
48 PAGE_TITLE = "Visor de Climes EPW"
49 PAGE_LAYOUT = "wide"
50 INTRODUCTION_TEXT = (
51     "Selecciona un fitxer climàtic EPW i explora'n les sèries temporals, "
52     "els patrons mensuals i els indicadors ambientals des d'un dashboard integrat."
53 )
54
55
56 PLOTLY_CHART_CONFIG = {"displayModeBar": False}
57
58
59 @dataclass(frozen=True)
60 class EpwDashboardState:
61     """Dades preparades necessàries per representar les pestanyes del panell EPW."""
62
63     selected_entry: dict[str, str]
64     metadata: dict[str, object]
65     full_metrics: dict[str, float]
66     filtered_metrics: dict[str, float]
67     month_tick_values: list[int]
68     month_tick_labels: list[str]
69     series_variable: str
70     series_aggregation: str
71     heatmap_variable: str
72     secondary_heatmap_variable: str
73     records_message: str
74     monthly_frame: pd.DataFrame
75     focus_series: pd.DataFrame
76     daily_temperature: pd.DataFrame
77     hourly_profile: pd.DataFrame
78     heatmap_frame: pd.DataFrame
79     radiation_heatmap_frame: pd.DataFrame
80     annual_temperature_heatmap: pd.DataFrame
81     annual_direct_radiation_heatmap: pd.DataFrame
82     annual_wind_heatmap: pd.DataFrame
83     annual_humidity_heatmap: pd.DataFrame
84     comfort_radiation_frame: pd.DataFrame
85     monthly_wind_rose_tables: dict[str, pd.DataFrame]
86     download_frame: pd.DataFrame
87     metric_cards: list[tuple[str, str]]
88
89
90 def ui_text(value: object) -> str:
91     """Retorna el text segur per a la pantalla, reparant el mojibake habitual de les metadades EPW."""
92
93     text = "" if value is None else str(value)
94     for _ in range(2):
95         if not any(marker in text for marker in ("Ã", "À", "à")):
96             break
97         # Algunes capçaleres EPW arriben com a text UTF-8 descodificat com a latin-1.
98         try:

```

```

99         repaired = text.encode("latin-1").decode("utf-8")
100     except (UnicodeEncodeError, UnicodeDecodeError):
101         break
102     if repaired == text:
103         break
104     text = repaired
105     return text
106
107
108 def render_hero() -> None:
109     """Renderitza l'heroi de la pàgina amb el títol del visor EPW i la introducció a la UI de Streamlit."""
110
111     hero_markup = f"""
112     <section class="epw-hero">
113         <div class="hero-kicker">Visualització climàtica</div>
114         <h1 class="hero-title">{escape(PAGE_TITLE)}</h1>
115         <div class="hero-copy">{escape(ui_text(INTRODUCTION_TEXT))}</div>
116     </section>
117     """
118     st.markdown(
119         hero_markup,
120         unsafe_allow_html=True,
121     )
122
123
124 def format_number(value: float | int | None, *, decimals: int = 1, suffix: str = "") -> str:
125     """Formata un valor numèric amb separadors catalans i un sufix opcional."""
126
127     if value is None or pd.isna(value):
128         return "-"
129     if decimals == 0:
130         return f"{int(round(float(value))):,}{suffix}".replace(",", ".")
131     return (
132         f"{float(value):.{decimals}f}{suffix}"
133         .replace(",", "X")
134         .replace(".", ",")
135         .replace("X", ".")
136     )
137
138
139 def data_frame_has_plot_values(
140     data_frame: pd.DataFrame,
141     columns: list[str] | None = None,
142     *,
143     require_non_zero: bool = False,
144 ) -> bool:
145     """Retorna si un DataFrame conté valors numèrics que val la pena dibuixar."""
146
147     if data_frame.empty:
148         return False
149
150     if columns is None:
151         values = data_frame
152     else:
153         available_columns = [column for column in columns if column in data_frame.columns]
154         if not available_columns:
155             return False
156         values = data_frame[available_columns]
157
158     numeric_values = values.select_dtypes(include="number")
159     if numeric_values.empty:
160         numeric_values = values.apply(pd.to_numeric, errors="coerce")
161
162     valid_values = numeric_values.stack().dropna()
163     if valid_values.empty:
164         return False
165
166     if require_non_zero:
167         return bool((valid_values.abs() > 0).any())
168
169     return True
170
171
172 def wind_rose_tables_have_plot_values(monthly_rose_tables: dict[str, pd.DataFrame]) -> bool:
173     """Indica si almenys una taula mensual de roses dels vents conté freqüències de vent."""
174
175     for rose_frame in monthly_rose_tables.values():
176         if data_frame_has_plot_values(rose_frame, ["frequency_pct"], require_non_zero=True):
177             return True
178     return False
179
180
181 def render_metric_cards(metrics: list[tuple[str, str]]) -> None:
182     """Renderitza les targetes KPI que es mostren a la pestanya de visió general de la UI de Streamlit."""
183
184     if not metrics:
185         return
186
187     metric_html = "".join(
188         (
189             '<section class="epw-metric-card">'
190             f'<div class="epw-metric-label">{escape(ui_text(label))}</div>'

```

```

191         f'<div class="epw-metric-value">{escape(ui_text(value))}</div>'
192         "</section>"
193     )
194     for label, value in metrics
195 )
196 st.markdown(
197     f'<section class="section-card epw-metric-shell">{metric_html}</section>',
198     unsafe_allow_html=True,
199 )
200
201
202 def render_detail_card(
203     metadata: dict[str, object],
204     selected_entry: dict[str, str],
205     total_records: int,
206     filtered_records: int,
207 ) -> None:
208     """Mostra la targeta de metadades del fitxer EPW seleccionada a la UI de Streamlit."""
209
210     detail_rows = [
211         ("Fitxer", ui_text(metadata.get("file_name", "-"))),
212         ("Ruta relativa", ui_text(selected_entry.get("relative_path", "-"))),
213         ("Ciutat", ui_text(metadata.get("city", "-"))),
214         ("País", ui_text(metadata.get("country", "-"))),
215         ("Font", ui_text(metadata.get("source", "-"))),
216         ("WMO", ui_text(metadata.get("wmo", "-"))),
217         ("Latitud", format_number(metadata.get("latitude"), decimals=3)),
218         ("Longitud", format_number(metadata.get("longitude"), decimals=3)),
219         ("Fus horari", f"UTC {format_number(metadata.get('timezone'), decimals=1)}"),
220         ("Elevació", f"{format_number(metadata.get('elevation_m'), decimals=0)} m"),
221         ("Registres", f"{filtered_records:,} / {total_records:,}" .replace(",", ".")),
222         ("Classificació", ui_text(metadata.get("climate_family", "-"))),
223     ]
224     detail_html = "".join(
225         f"""
226         <div class="epw-detail-row">
227             <div class="epw-detail-label">{escape(label)}</div>
228             <div class="epw-detail-value">{escape(str(value))}</div>
229         </div>
230         """
231         for label, value in detail_rows
232     )
233     st.markdown(
234         f"""
235         <section class="epw-detail-card">
236             <div class="epw-card-title">Context del fitxer</div>
237             <div class="epw-detail-list">{detail_html}</div>
238         </section>
239         """
240         ,
241         unsafe_allow_html=True,
242     )
243
244 def build_map_chart(metadata: dict[str, object]) -> None:
245     """Renderitza un mapa centrat a les coordenades de l'estació meteorològica EPW."""
246
247     latitude = metadata.get("latitude")
248     longitude = metadata.get("longitude")
249     try:
250         latitude = float(latitude)
251         longitude = float(longitude)
252     except (TypeError, ValueError):
253         st.info("Aquest fitxer EPW no té coordenades vàlides per mostrar al mapa.")
254         return
255
256     city = ui_text(metadata.get("city", "-"))
257     country = ui_text(metadata.get("country", "-"))
258     file_name = ui_text(metadata.get("file_name", "-"))
259     render_location_map(
260         latitude,
261         longitude,
262         zoom=4.5,
263         color=(95, 84, 249, 220),
264         radius=22000,
265         tooltip_html=f"<b>{file_name}</b><br>{city}, {country}",
266     )
267
268
269 def configure_epw_dashboard_page() -> None:
270     """Configura Chrome de la pàgina global i renderitza l'heroi EPW."""
271
272     configure_studio_page(PAGE_TITLE, "pages/Visor_Climes_EPW.py", layout=PAGE_LAYOUT)
273     inject_epw_viewer_styles()
274     render_hero()
275
276
277 def select_epw_entry() -> tuple[dict[str, str], dict[str, object], pd.DataFrame]:
278     """Renderitza els controls de cerca del catàleg i carrega el paquet EPW seleccionat a la UI de Streamlit."""
279
280     weather_root = WEATHER_ROOT
281     catalog = list_epw_catalog(str(weather_root))
282     if not catalog:

```

```

283     st.warning(f"No s'han trobat fitxers EPW a `{weather_root}`.")
284     st.stop()
285
286 search_cols = st.columns([1.15, 2.1], gap="medium")
287 with search_cols[0]:
288     search_query = st.text_input(
289         "Cerca un fitxer EPW",
290         placeholder="Ex: Granada, Lisboa, IWE...",
291     ).strip().lower()
292
293 filtered_catalog = tuple(
294     entry
295     for entry in catalog
296     if not search_query or search_query in entry["search_blob"]
297 )
298 if not filtered_catalog:
299     st.warning("Cap fitxer EPW coincideix amb el filtre de cerca.")
300     st.stop()
301
302 entry_by_path = {entry["path"]: entry for entry in filtered_catalog}
303 with search_cols[1]:
304     selected_path = st.selectbox(
305         "Fitxer climàtic",
306         list(entry_by_path),
307         format_func=lambda option: ui_text(entry_by_path[option]["display_name"]),
308     )
309
310 bundle = load_epw_bundle(selected_path)
311 return entry_by_path[selected_path], bundle["metadata"], bundle["data"]
312
313
314 def build_metric_card_values(filtered_metrics: dict[str, float]) -> list[tuple[str, str]]:
315     """Crea valors de visualització per a les targetes EPW de resum KPI."""
316
317     return [
318         (
319             "Temperatura mitjana",
320             f"{format_number(filtered_metrics['mean_temp'], decimals=1)} {DEG_C}",
321         ),
322         (
323             "Interval tèrmic",
324             f"{format_number(filtered_metrics['min_temp'], decimals=1)} - "
325             f"{format_number(filtered_metrics['max_temp'], decimals=1)} {DEG_C}",
326         ),
327         ("Humitat mitjana", f"{format_number(filtered_metrics['mean_rh'], decimals=1)} %"),
328         ("Vent mitjà", f"{format_number(filtered_metrics['mean_wind'], decimals=2)} m/s"),
329         (
330             "GHI acumulada",
331             f"{format_number(filtered_metrics['ghi_total_kwh_m2'], decimals=1)} {KWH_PER_M2}",
332         ),
333         ("Vent màxim", f"{format_number(filtered_metrics['max_wind'], decimals=2)} m/s"),
334         (
335             "DNI acumulada",
336             f"{format_number(filtered_metrics['dni_total_kwh_m2'], decimals=1)} {KWH_PER_M2}",
337         ),
338         (
339             "DHI acumulada",
340             f"{format_number(filtered_metrics['dhi_total_kwh_m2'], decimals=1)} {KWH_PER_M2}",
341         ),
342     ]
343
344
345 def prepare_epw_dashboard_state(
346     selected_entry: dict[str, str],
347     metadata: dict[str, object],
348     epw_data: pd.DataFrame,
349 ) -> EpwDashboardState:
350     """Crea totes les taules i etiquetes derivades necessàries per al panell EPW."""
351
352     filtered_data = epw_data.copy()
353     full_metrics = summarize_epw_metrics(epw_data)
354     filtered_metrics = summarize_epw_metrics(filtered_data)
355     month_tick_values, month_tick_labels = build_active_month_axis(filtered_data)
356
357     series_variable = "dry_bulb_c"
358     series_aggregation = "Dia"
359     heatmap_variable = "dry_bulb_c"
360     secondary_heatmap_variable = "direct_normal_radiation_wh_m2"
361     records_message = (
362         f"Mostrant {int(filtered_metrics['records']):,} registres de "
363         f"{int(full_metrics['records']):,} sobre l'any EPW complet."
364     ).replace(",", ".")
365
366     return EpwDashboardState(
367         selected_entry=selected_entry,
368         metadata=metadata,
369         full_metrics=full_metrics,
370         filtered_metrics=filtered_metrics,
371         month_tick_values=month_tick_values,
372         month_tick_labels=month_tick_labels,
373         series_variable=series_variable,
374         series_aggregation=series_aggregation,

```

```

375     heatmap_variable=heatmap_variable,
376     secondary_heatmap_variable=secondary_heatmap_variable,
377     records_message=records_message,
378     monthly_frame=build_monthly_summary(filtered_data),
379     focus_series=aggregate_epw_timeseries(
380         filtered_data,
381         series_variable,
382         series_aggregation,
383     ),
384     daily_temperature=build_daily_temperature_profile(filtered_data),
385     hourly_profile=build_hourly_profile(filtered_data),
386     heatmap_frame=build_heatmap_table(filtered_data, heatmap_variable),
387     radiation_heatmap_frame=build_heatmap_table(
388         filtered_data,
389         secondary_heatmap_variable,
390     ),
391     annual_temperature_heatmap=build_annual_hourly_heatmap_table(
392         filtered_data,
393         "dry_bulb_c",
394     ),
395     annual_direct_radiation_heatmap=build_annual_hourly_heatmap_table(
396         filtered_data,
397         "direct_normal_radiation_wh_m2",
398     ),
399     annual_wind_heatmap=build_annual_hourly_heatmap_table(
400         filtered_data,
401         "wind_speed_m_s",
402     ),
403     annual_humidity_heatmap=build_annual_hourly_heatmap_table(
404         filtered_data,
405         "relative_humidity_pct",
406     ),
407     comfort_radiation_frame=build_comfort_radiation_table(filtered_data, metadata),
408     monthly_wind_rose_tables=build_monthly_wind_rose_tables(filtered_data),
409     download_frame=build_download_frame(filtered_data),
410     metric_cards=build_metric_card_values(filtered_metrics),
411 )
412
413
414 def render_tab_intro(text: str) -> None:
415     """Feu una còpia explicativa a la part superior de cada pestanya EPW a la UI de Streamlit."""
416
417     st.markdown(
418         f"<div class='epw-tab-copy'>{escape(ui_text(text))}</div>",
419         unsafe_allow_html=True,
420     )
421
422
423 def render_annual_heatmap(
424     frame: pd.DataFrame,
425     variable_key: str,
426     state: EpwDashboardState,
427     empty_message: str,
428     *,
429     require_non_zero: bool = False,
430 ) -> None:
431     """Representeu un mapa de calor anual quan la taula d'origen contingui dades gràfics."""
432
433     if data_frame_has_plot_values(frame, require_non_zero=require_non_zero):
434         st.plotly_chart(
435             build_annual_heatmap_figure(
436                 frame,
437                 variable_key,
438                 tick_values=state.month_tick_values,
439                 tick_labels=state.month_tick_labels,
440             ),
441             use_container_width=True,
442             config=PLOTLY_CHART_CONFIG,
443         )
444     else:
445         st.info(empty_message)
446
447
448 def render_overview_tab(state: EpwDashboardState) -> None:
449     """Mostra la pestanya de visió general de l'EPW a la UI de Streamlit."""
450
451     render_tab_intro(
452         "Resum executiu del fitxer seleccionat, amb indicadors clau, context i ubicació."
453     )
454     render_metric_cards(state.metric_cards)
455     overview_cols = st.columns([1.18, 0.82], gap="large")
456     with overview_cols[0]:
457         render_detail_card(
458             state.metadata,
459             state.selected_entry,
460             total_records=int(state.full_metrics["records"]),
461             filtered_records=int(state.filtered_metrics["records"]),
462         )
463     with overview_cols[1]:
464         st.markdown("<div class='epw-card-title'>Ubicació</div>", unsafe_allow_html=True)
465         build_map_chart(state.metadata)
466

```

```

467
468 def render_climate_dashboard_tab(state: EpwDashboardState) -> None:
469     """Renderitza mapes de calor climàtics anuals i gràfics d'orientació/vents."""
470
471     render_tab_intro(
472         "Tauler visual anual per detectar patrons ràpids de temperatura, radiació, vent i humitat."
473     )
474     dashboard_heatmaps = st.columns(2, gap="medium")
475     with dashboard_heatmaps[0]:
476         render_annual_heatmap(
477             state.annual_temperature_heatmap,
478             "dry_bulb_c",
479             state,
480             "No hi ha prou dades de temperatura per construir el mapa anual.",
481         )
482         render_annual_heatmap(
483             state.annual_direct_radiation_heatmap,
484             "direct_normal_radiation_wh_m2",
485             state,
486             "No hi ha prou dades de radiació directa per construir el mapa anual.",
487             require_non_zero=True,
488         )
489
490     with dashboard_heatmaps[1]:
491         render_annual_heatmap(
492             state.annual_wind_heatmap,
493             "wind_speed_m_s",
494             state,
495             "No hi ha prou dades de vent per construir el mapa anual.",
496             require_non_zero=True,
497         )
498         render_annual_heatmap(
499             state.annual_humidity_heatmap,
500             "relative_humidity_pct",
501             state,
502             "No hi ha prou dades d'humitat per construir el mapa anual.",
503         )
504
505     render_comfort_radiation_chart(state)
506     render_monthly_wind_rose_chart(state)
507
508
509 def render_comfort_radiation_chart(state: EpwDashboardState) -> None:
510     """Genera un gràfic de radiació orientat a la comoditat quan hi hagi prou dades a la UI de Streamlit."""
511
512     if not data_frame_has_plot_values(
513         state.comfort_radiation_frame,
514         ["comfort_radiation_kwh_m2", "incident_radiation_kwh_m2"],
515         require_non_zero=True,
516     ):
517         st.info("No hi ha prou dades per estimar la radiació orientada d'aquest fitxer.")
518         return
519
520     st.plotly_chart(
521         build_comfort_radiation_figure(state.comfort_radiation_frame),
522         use_container_width=True,
523         config=PLOTLY_CHART_CONFIG,
524     )
525     st.caption(
526         "Estimació sobre façanes verticals a partir de la radiació EPW "
527         f"i una banda de confort tèrmic de 18-24 {DEG_C}."
528     )
529
530
531 def render_monthly_wind_rose_chart(state: EpwDashboardState) -> None:
532     """Representeu les roses dels vents mensuals quan les taules de vent continguin valors útils."""
533
534     if wind_rose_tables_have_plot_values(state.monthly_wind_rose_tables):
535         st.plotly_chart(
536             build_monthly_wind_rose_grid_figure(state.monthly_wind_rose_tables),
537             use_container_width=True,
538             config=PLOTLY_CHART_CONFIG,
539         )
540     else:
541         st.info("No hi ha prou dades de vent per construir les roses mensuals.")
542
543
544 def render_series_tab(state: EpwDashboardState) -> None:
545     """Mostra la pestanya de sèries temporals EPW a la UI de Streamlit."""
546
547     render_tab_intro(
548         "Evolució temporal anual de la temperatura, amb suport de rangs diaris i perfil horari mitjà."
549     )
550     if data_frame_has_plot_values(state.focus_series, [state.series_variable]):
551         st.plotly_chart(
552             build_focus_timeseries_figure(
553                 state.focus_series,
554                 state.series_variable,
555                 state.series_aggregation,
556             ),
557             use_container_width=True,
558             config=PLOTLY_CHART_CONFIG,

```

```

559     )
560 else:
561     st.info("No hi ha prou dades de temperatura per construir la sèrie temporal.")
562
563 series_cols = st.columns(2, gap="medium")
564 with series_cols[0]:
565     if data_frame_has_plot_values(
566         state.daily_temperature,
567         ["dry_bulb_min", "dry_bulb_mean", "dry_bulb_max"],
568     ):
569         st.plotly_chart(
570             build_daily_temperature_band_figure(state.daily_temperature),
571             use_container_width=True,
572             config=PLOTLY_CHART_CONFIG,
573         )
574     else:
575         st.info("No hi ha prou dades per construir la banda diària de temperatura.")
576
577 with series_cols[1]:
578     if data_frame_has_plot_values(
579         state.hourly_profile,
580         ["dry_bulb_mean", "dew_point_mean", "relative_humidity_mean"],
581     ):
582         st.plotly_chart(
583             build_hourly_profile_figure(state.hourly_profile),
584             use_container_width=True,
585             config=PLOTLY_CHART_CONFIG,
586         )
587     else:
588         st.info("No hi ha prou dades per construir el perfil horari mitjà.")
589
590 def render_patterns_tab(state: EpwDashboardState) -> None:
591     """Representeu gràfics de patró climàtic mensual i intradia."""
592
593     render_tab_intro("Comparatives mensuals i patrons intradiaris del fitxer climàtic.")
594     pattern_cols_monthly = st.columns(2, gap="medium")
595     with pattern_cols_monthly[0]:
596         if data_frame_has_plot_values(
597             state.monthly_frame,
598             ["dry_bulb_mean", "dry_bulb_min", "dry_bulb_max", "relative_humidity_mean"],
599         ):
600             st.plotly_chart(
601                 build_monthly_climate_figure(state.monthly_frame),
602                 use_container_width=True,
603                 config=PLOTLY_CHART_CONFIG,
604             )
605         else:
606             st.info("No hi ha prou dades climàtiques mensuals per construir aquest gràfic.")
607
608     with pattern_cols_monthly[1]:
609         if data_frame_has_plot_values(
610             state.monthly_frame,
611             [
612                 "global_horizontal_radiation_kwh_m2",
613                 "direct_normal_radiation_kwh_m2",
614                 "diffuse_horizontal_radiation_kwh_m2",
615             ],
616             require_non_zero=True,
617         ):
618             st.plotly_chart(
619                 build_monthly_solar_figure(state.monthly_frame),
620                 use_container_width=True,
621                 config=PLOTLY_CHART_CONFIG,
622             )
623         else:
624             st.info("No hi ha prou dades de radiació mensual per construir aquest gràfic.")
625
626     render_pattern_heatmaps(state)
627
628
629 def render_pattern_heatmaps(state: EpwDashboardState) -> None:
630     """Renderitza els mapes de calor de la pestanya del patró a la UI de Streamlit."""
631
632     pattern_cols_heatmaps = st.columns(2, gap="medium")
633     with pattern_cols_heatmaps[0]:
634         if data_frame_has_plot_values(state.heatmap_frame):
635             st.plotly_chart(
636                 build_heatmap_figure(state.heatmap_frame, state.heatmap_variable),
637                 use_container_width=True,
638                 config=PLOTLY_CHART_CONFIG,
639             )
640         else:
641             st.info("No hi ha prou dades de temperatura per construir el mapa de calor.")
642
643     with pattern_cols_heatmaps[1]:
644         if data_frame_has_plot_values(state.radiation_heatmap_frame, require_non_zero=True):
645             st.plotly_chart(
646                 build_heatmap_figure(
647                     state.radiation_heatmap_frame,
648                     state.secondary_heatmap_variable,
649                 ),
650             ),

```

```

651         use_container_width=True,
652         config=PLOTLY_CHART_CONFIG,
653     )
654     else:
655         st.info("No hi ha prou dades de radiació directa per construir el mapa de calor.")
656
657
658 def render_epw_tabs(state: EpwDashboardState) -> None:
659     """Renderitza totes les pestanyes del panell EPW."""
660
661     tab_overview, tab_dashboard, tab_series, tab_patterns = st.tabs(
662         ["Resum", "Tauler climàtic", "Sèries", "Patrons"]
663     )
664     with tab_overview:
665         render_overview_tab(state)
666     with tab_dashboard:
667         render_climate_dashboard_tab(state)
668     with tab_series:
669         render_series_tab(state)
670     with tab_patterns:
671         render_patterns_tab(state)
672
673
674 def render_epw_download(state: EpwDashboardState) -> None:
675     """Mostra els controls de baixada CSV per al fitxer EPW seleccionat."""
676
677     st.markdown("<div style='height: 1.25rem;'></div>", unsafe_allow_html=True)
678     st.download_button(
679         "Descarregar dades del fitxer (CSV)",
680         data=state.download_frame.to_csv(index=False).encode("utf-8"),
681         file_name=f"{state.metadata.get('stem', 'clima')}_complet.csv",
682         mime="text/csv",
683         use_container_width=True,
684         type="primary",
685     )
686     st.caption(state.records_message)
687
688
689 def render_epw_dashboard() -> None:
690     """Mostra el flux complet del visor EPW a la UI de Streamlit."""
691
692     configure_epw_dashboard_page()
693     selected_entry, metadata, epw_data = select_epw_entry()
694     st.markdown(
695         "<div class='epw-card-title epw-global-title'>Lectura global del fitxer</div>",
696         unsafe_allow_html=True,
697     )
698     state = prepare_epw_dashboard_state(selected_entry, metadata, epw_data)
699     render_epw_tabs(state)
700     render_epw_download(state)
701
702
703 render_epw_dashboard()

```

D Codi: Components reutilitzables

D.1 page_components/__init__.py

Listing 15: page_components/__init__.py

```

1  """Components Streamlit reutilitzables per a pàgines BEMS-RL Studio.
2
3  Grup de mòduls de components repetits UI seccions com ara formularis de entrenament, recompensa
4  controls de configuració, editors d'embolcalls i panells de resultats en línia. Es conserven
5  separat dels punts d'entrada de la pàgina de manera que es puguin documentar fragments complexos UI i
6  reutilitzat sense importar pàgines completes de Streamlit.
7  """

```

D.2 page_components/resultats_dashboard.py

Listing 16: page_components/resultats_dashboard.py

```

1  """Component del panell en línia per a la pàgina de resultats."""
2
3  from __future__ import annotations
4
5  from html import escape
6  from pathlib import Path
7
8  import pandas as pd
9  import plotly.graph_objects as go
10 import streamlit as st

```



```

11
12 from backend.resultats_backend import (
13     DashboardData,
14     add_comfort_percentage_kpi,
15     apply_real_period_axis,
16     build_real_period_options,
17     has_action_figure_data,
18     has_figure_data,
19     is_radiant_run,
20     load_comparison_data,
21     overlay_comparison_traces,
22     select_actions_for_obs,
23     select_radiant_action_data,
24     slice_obs_for_real_period,
25 )
26 from backend.resultats_report_backend import generate_report_bytes
27 _AGG_LABELS = {
28     "hour": "Horaria (0..23)",
29     "day": "Diaria (1..365)",
30     "month": "Mensual (1..12)",
31     "raw": "Sense Agregació (Crua)",
32 }
33 _SEASON_LABELS = {
34     "All": "Totes",
35     "Winter": "Hivern (Des, Gen, Feb)",
36     "Spring": "Primavera (Mar, Abr, Mai)",
37     "Summer": "Estiu (Jun, Jul, Ago)",
38     "Autumn": "Tardor (Set, Oct, Nov)",
39 }
40 _VIEW_MODE_LABELS = {
41     "aggregate": "Mitjana / Agregat",
42     "real": "Serie real",
43 }
44 _REAL_PERIOD_LABELS = {
45     "day": "Un dia",
46     "week": "Una setmana",
47     "month": "Un mes",
48 }
49
50
51 def _file_signature(path: str) -> tuple[str, int, int]:
52     """Retorna una signatura de memòria cau estable per a la ruta d'un fitxer."""
53
54     file_path = Path(path)
55     try:
56         stat = file_path.stat()
57     except OSError:
58         return str(file_path), -1, -1
59     return str(file_path), int(stat.st_mtime_ns), int(stat.st_size)
60
61
62 def _run_data_signature(progress_path: str, observations_path: str) -> tuple[object, ...]:
63     """Retorna les entrades de memòria cau que canvien cada vegada que canvien les dades d'execució seleccionades."""
64
65     run_dir = Path(progress_path).parent
66     yaml_signatures = []
67     try:
68         yaml_signatures = [
69             _file_signature(str(path))
70             for path in sorted(run_dir.glob("*.yaml"))
71         ]
72     except OSError:
73         yaml_signatures = []
74     return (
75         _file_signature(progress_path),
76         _file_signature(observations_path),
77         tuple(yaml_signatures),
78     )
79
80
81 def _load_comparison_data_cached(run_path: str) -> DashboardData | None:
82     """Carrega i emmagatzema a la memòria cau una comparació executada a l'estat de sessió Streamlit."""
83
84     cache_key = f"_comp_data_{hash(run_path)}"
85     if cache_key in st.session_state:
86         return st.session_state[cache_key]
87
88     data = load_comparison_data(run_path)
89     if data is None:
90         return None
91
92     st.session_state[cache_key] = data
93     return data
94
95 @st.cache_data(show_spinner="Generant informe...", max_entries=10)
96 def _cached_report_bytes(
97     progress_path: str,
98     observations_path: str,
99     agg_mode: str,
100     season: str,
101     comfort_scope: str,
102     zone_str: str,

```

```

103     data_signature: tuple[object, ...],
104 ) -> tuple[bytes, str]:
105     """Retorna bytes d'informes a la memòria cau per als filtres del panell seleccionats."""
106
107     return generate_report_bytes(
108         progress_path=progress_path,
109         observations_path=observations_path,
110         agg_mode=agg_mode,
111         season=season,
112         comfort_scope=comfort_scope,
113         zone_str=zone_str,
114     )
115
116
117 def _plot_if_available(fig: go.Figure | None) -> None:
118     """Mostra un gràfic Plotly només quan conté traces visibles."""
119
120     if has_figure_data(fig):
121         st.plotly_chart(fig, use_container_width=True)
122
123
124 def _render_report_export(
125     report_action_slot,
126     data: DashboardData,
127     *,
128     plot_mode: str,
129     plot_season: str,
130     comfort_scope: str,
131     zone_val: str | None,
132 ) -> None:
133     """Mostra els controls de generació i baixada d'informes."""
134
135     with report_action_slot.container():
136         zone_str = zone_val if zone_val else "ALL"
137         data_signature = _run_data_signature(data.progress_path, data.observations_path)
138         report_key = (
139             data.progress_path,
140             data.observations_path,
141             plot_mode,
142             plot_season,
143             comfort_scope,
144             zone_str,
145             data_signature,
146         )
147         if st.session_state.get("_report_cache_key") != report_key:
148             st.session_state.pop("_report_cache", None)
149             st.session_state["_report_cache_key"] = report_key
150
151         if "_report_cache" not in st.session_state:
152             if st.button(
153                 "Generar informe",
154                 icon="material/summarize:",
155                 key="btn_gen_report",
156                 use_container_width=True,
157             ):
158                 with st.spinner("Generant informe..."):
159                     st.session_state["_report_cache"] = _cached_report_bytes(
160                         data.progress_path,
161                         data.observations_path,
162                         plot_mode,
163                         plot_season,
164                         comfort_scope,
165                         zone_str,
166                         data_signature,
167                     )
168                 st.rerun()
169             return
170
171         report_bytes, report_ext = st.session_state["_report_cache"]
172         mime = "application/pdf" if report_ext == "pdf" else "text/html"
173         st.download_button(
174             f"Descarregar informe ({report_ext})",
175             data=report_bytes,
176             file_name=f"Sinergym_Report_{data.env_name}_{plot_season}.{report_ext}",
177             mime=mime,
178             key="btn_download_report",
179             use_container_width=True,
180         )
181
182
183 def _render_dashboard_tabs(figures: dict[str, go.Figure], *, view_mode: str) -> None:
184     """Renderitza els gràfics de resultats a la mateixa estructura de pestanyes que utilitza el panell."""
185
186     # Només obrim pestanyes de control o bateria si hi ha dades. Això evita pestanyes buides
187     # quan l'entorn no té actuadors radiants, bateria o accions registrades.
188     has_hvac_control = any(
189         has_figure_data(figures[key])
190         for key in ["setpoints", "setpoints_vs_indoor", "radiant", "actions"]
191     )
192     has_battery = any(
193         has_figure_data(figures[key])
194         for key in ["battery_power", "battery_soc", "battery_vs_grid", "battery_charge_price"]

```

```

195 )
196
197 tab_defs = [
198     ("Temperatura", True),
199     ("Consum i Potència", True),
200     ("Control HVAC", has_hvac_control),
201     ("Bateria", has_battery),
202     ("Episodi", True),
203 ]
204 active_tabs = st.tabs([label for label, show in tab_defs if show])
205 tab_iter = iter(active_tabs)
206 tab_temp = next(tab_iter)
207 tab_consum = next(tab_iter)
208 tab_control = next(tab_iter) if has_hvac_control else None
209 tab_battery = next(tab_iter) if has_battery else None
210 tab_episodi = next(tab_iter)
211
212 with tab_temp:
213     st.markdown(
214         "<div class='results-tab-copy'>Temperatura interior i exterior, percentatge d'hores en confort, "
215         "infraccions tèrmiques i distribució de temperatura per zona i hora.</div>",
216         unsafe_allow_html=True,
217     )
218     if view_mode == "aggregate":
219         _plot_if_available(figures["comfort"])
220         _plot_if_available(figures["indoor"])
221         _plot_if_available(figures["humidity"])
222         _plot_if_available(figures["violation"])
223     if view_mode == "aggregate":
224         _plot_if_available(figures["zone_heatmap"])
225
226 with tab_consum:
227     st.markdown(
228         "<div class='results-tab-copy'>Consum HVAC agregat, desglosament per meter, preu de l'energia "
229         "i mapa de calor global del consum al llarg de l'any.</div>",
230         unsafe_allow_html=True,
231     )
232     _plot_if_available(figures["hvac"])
233     _plot_if_available(figures["hvac_breakdown"])
234     _plot_if_available(figures["energy_price"])
235     if view_mode == "aggregate":
236         _plot_if_available(figures["heatmap"])
237
238 if tab_control is not None:
239     with tab_control:
240         st.markdown(
241             "<div class='results-tab-copy'>Setpoints tèrmics, actuació del control radiant "
242             "i accions que l'agent RL ha aplicat al sistema HVAC.</div>",
243             unsafe_allow_html=True,
244         )
245         _plot_if_available(figures["setpoints"])
246         _plot_if_available(figures["setpoints_vs_indoor"])
247         _plot_if_available(figures["radiant"])
248         _plot_if_available(figures["actions"])
249
250 if tab_battery is not None:
251     with tab_battery:
252         st.markdown(
253             "<div class='results-tab-copy'>Cicles de càrrega i descàrrega, estat de càrrega (SOC), "
254             "interacció amb la xarxa elèctrica i correlació amb el preu de l'energia.</div>",
255             unsafe_allow_html=True,
256         )
257         for key in ["battery_power", "battery_soc", "battery_vs_grid", "battery_charge_price"]:
258             _plot_if_available(figures[key])
259
260 with tab_episodi:
261     st.markdown(
262         "<div class='results-tab-copy'>Mètriques de rendiment acumulades per episodi "
263         "d'entrenament o avaluació.</div>",
264         unsafe_allow_html=True,
265     )
266     if view_mode == "aggregate" and has_figure_data(figures["episode"]):
267         st.plotly_chart(figures["episode"], use_container_width=True)
268     else:
269         st.info("Les mètriques d'episodi només estan disponibles en mode agregat.")
270
271
272 def _build_dashboard_figures(
273     data: DashboardData,
274     zobs: pd.DataFrame,
275     action_data: pd.DataFrame,
276     *,
277     plot_mode: str,
278     plot_season: str,
279     comfort_scope: str,
280     view_mode: str,
281     real_period_kind: str,
282 ) -> dict[str, go.Figure]:
283     """Crea totes les figures Plotly utilitzades pel panell de resultats en línia."""
284
285     from backend.grafics import figures_common as figures_common

```

```

287 comfort_mode = "raw" if plot_mode in ("day", "raw") else plot_mode
288 fix_colors = figures_common._fix_colors_for_visibility
289 hvac_raw_as_line = view_mode == "real" and real_period_kind == "month"
290
291 # Generem totes les figures en un sol lloc perquè la comparació pugui aplicar el mateix
292 # tractament a la run principal i a la run de referència.
293 figures: dict[str, go.Figure] = {
294     "comfort": (
295         figures_common.make_comfort_compliance(
296             zobs,
297             comfort_mode,
298             plot_season,
299             comfort_scope=comfort_scope,
300             comfort_config=data.yaml_cfg,
301         )
302         if view_mode == "aggregate"
303         else go.Figure()
304     ),
305     "indoor": figures_common.make_indoor_temperature_plot(zobs, plot_mode, plot_season),
306     "humidity": figures_common.make_indoor_humidity_plot(zobs, plot_mode, plot_season),
307     "setpoints": figures_common.make_setpoints_plot(zobs, plot_mode, plot_season),
308     "setpoints_vs_indoor": figures_common.make_setpoints_vs_indoor_plot(zobs, plot_mode, plot_season),
309     "radiant": figures_common.make_radiant_control_plot(zobs, plot_mode, plot_season),
310     "actions": figures_common.make_agent_actions_plot(action_data, plot_mode, plot_season),
311     "hvac": fix_colors(
312         figures_common.make_hvac_consumption_plot(
313             zobs,
314             plot_mode,
315             plot_season,
316             raw_as_line=hvac_raw_as_line,
317         ),
318         kind="hvac",
319         mode=plot_mode,
320     ),
321     "hvac_breakdown": figures_common.make_hvac_meter_breakdown_plot(zobs, plot_mode, plot_season),
322     "energy_price": figures_common.make_energy_price_plot(zobs, plot_mode, plot_season),
323     "battery_power": figures_common.make_battery_power_plot(zobs, plot_mode, plot_season),
324     "battery_soc": figures_common.make_battery_soc_plot(zobs, plot_mode, plot_season),
325     "battery_vs_grid": figures_common.make_battery_vs_grid_plot(zobs, plot_mode, plot_season),
326     "battery_charge_price": figures_common.make_battery_charge_with_price_plot(zobs, plot_mode, plot_season),
327     "zone_heatmap": go.Figure(),
328     "heatmap": go.Figure(),
329     "episode": figures_common.make_episode_metrics(data.progress)
330     if view_mode == "aggregate"
331     else go.Figure(),
332 }
333
334 if not has_action_figure_data(figures["actions"]):
335     figures["actions"] = go.Figure()
336
337 if view_mode == "aggregate":
338     figures["violation"] = fix_colors(
339         figures_common.make_violation_bars(
340             zobs,
341             plot_mode,
342             plot_season,
343             comfort_scope=comfort_scope,
344             comfort_config=data.yaml_cfg,
345         ),
346         kind="violation",
347         mode=plot_mode,
348     )
349     figures["zone_heatmap"] = figures_common.make_zone_temperature_heatmaps(
350         zobs, season=plot_season, agg="mean", max_cols=3
351     )
352     pivot = data.metrics_dict.get("pivot_consumption")
353     if pivot is not None and not pivot.empty:
354         figures["heatmap"] = figures_common.make_heatmap(pivot, "Total HVAC Consumption (kWh)")
355 else:
356     figures["violation"] = figures_common.make_violation_single_line(
357         zobs,
358         plot_mode,
359         plot_season,
360         comfort_config=data.yaml_cfg,
361     )
362     for key in [
363         "indoor",
364         "humidity",
365         "setpoints",
366         "setpoints_vs_indoor",
367         "radiant",
368         "actions",
369         "hvac",
370         "hvac_breakdown",
371         "energy_price",
372         "battery_power",
373         "battery_soc",
374         "battery_vs_grid",
375         "battery_charge_price",
376         "violation",
377     ]:
378         axis_data = action_data if key == "actions" else zobs

```

```

379         figures[key] = apply_real_period_axis(figures[key], axis_data, real_period_kind)
380
381     return figures
382
383
384 def _overlay_dashboard_figures(
385     figures: dict[str, go.Figure],
386     comparison_figures: dict[str, go.Figure],
387     *,
388     data: DashboardData,
389     view_mode: str,
390 ) -> dict[str, go.Figure]:
391     """Superposeu traces de comparació a les xifres actives del panell."""
392
393     overlay_keys = [
394         "indoor",
395         "humidity",
396         "setpoints",
397         "setpoints_vs_indoor",
398         "radiant",
399         "hvac",
400         "hvac_breakdown",
401         "energy_price",
402         "battery_power",
403         "battery_soc",
404         "battery_vs_grid",
405         "battery_charge_price",
406         "violation",
407     ]
408     if view_mode == "aggregate":
409         overlay_keys.extend(["comfort", "episode"])
410     if is_radiant_run(data):
411         overlay_keys.append("actions")
412
413     for key in overlay_keys:
414         figures[key] = overlay_comparison_traces(figures[key], comparison_figures[key])
415     return figures
416
417
418 def _style_dashboard_figures(
419     figures: dict[str, go.Figure],
420     *,
421     view_mode: str,
422 ) -> dict[str, go.Figure]:
423     """Aplica la semàntica comuna de Plotly a les figures del panell abans de representar-les."""
424
425     from backend.grafics import figures_common as figures_common
426
427     style_figure_semantics = getattr(figures_common, "style_figure_semantics", lambda fig: fig)
428     styled = dict(figures)
429     for key in [
430         "indoor",
431         "humidity",
432         "setpoints",
433         "setpoints_vs_indoor",
434         "radiant",
435         "actions",
436         "hvac",
437         "energy_price",
438         "battery_power",
439         "battery_soc",
440         "battery_vs_grid",
441         "battery_charge_price",
442         "violation",
443         "zone_heatmap",
444         "heatmap",
445     ]:
446         styled[key] = style_figure_semantics(styled[key])
447     if view_mode == "aggregate":
448         styled["comfort"] = style_figure_semantics(styled["comfort"])
449         styled["episode"] = style_figure_semantics(styled["episode"])
450     return styled
451
452
453 _KPI_HIGHER_IS_BETTER = frozenset({"% Hores en Confort"})
454
455
456 def _kpi_delta_color(key: str) -> str:
457     """Retorna el mode de color delta utilitzat per les targetes de comparació KPI."""
458     if key in _KPI_HIGHER_IS_BETTER:
459         return "normal"
460     key_lower = key.lower()
461     if "hvac" in key_lower or "violation" in key_lower:
462         return "inverse"
463     return "off"
464
465
466 def _render_kpis(kpis: dict, comp_kpis: dict | None) -> None:
467     """Renderitza les targetes KPI amb deltes de comparació opcionals."""
468     kpi_items = [(k, v) for k, v in kpis.items()]
469     if not kpi_items:
470         return

```

```

471 cards: list[str] = []
472
473 for k, v in kpi_items:
474     if v is None:
475         val_str = "N/D"
476     elif isinstance(v, float) and k.startswith("%"):
477         val_str = f"{v:.1f}%"
478     elif isinstance(v, (int, float)):
479         val_str = f"{float(v):.2f}"
480     else:
481         val_str = str(v)
482
483 delta_html = ""
484 if comp_kpis is not None and k in comp_kpis:
485     # El signe bo del delta depèn del KPI: menys cost o violació és millor,
486     # però més confort pot ser positiu. _kpi_delta_color encapsula aquesta regla.
487     cv = comp_kpis[k]
488     if (
489         isinstance(v, (int, float)) and isinstance(cv, (int, float))
490         and v is not None and cv is not None
491     ):
492         delta = round(float(v) - float(cv), 2)
493         color_hint = _kpi_delta_color(k)
494         if color_hint == "inverse":
495             is_good = delta < 0
496         elif color_hint == "normal":
497             is_good = delta > 0
498         else:
499             is_good = None
500         if delta == 0:
501             d_color = "#6b7280"
502             arrow = ""
503         elif is_good is None:
504             d_color = "#6b7280"
505             arrow = " " if delta > 0 else " "
506         elif is_good:
507             d_color = "#22c55e"
508             arrow = " " if delta > 0 else " "
509         else:
510             d_color = "#ef4444"
511             arrow = " " if delta > 0 else " "
512         sign = "+" if delta > 0 else ""
513         delta_html = (
514             f'<div class="results-kpi-delta" style="color:{d_color};">'
515             f'{arrow} {sign}{delta:.2f}</div>'
516         )
517
518 k_esc = escape(k)
519 v_esc = escape(val_str)
520 cards.append(
521     f'<div class="results-kpi-card">'
522     f'<div class="results-kpi-label" title="{k_esc}">{k_esc}</div>'
523     f'<div class="results-kpi-value">{v_esc}</div>'
524     f'{delta_html}</div>'
525 )
526
527
528 st.markdown(
529     '<section class="section-card results-kpi-shell">' + "".join(cards) + "</section>",
530     unsafe_allow_html=True,
531 )
532
533 def render_inline_dashboard(
534     data: DashboardData,
535     all_run_dirs: tuple,
536     current_run_path: str,
537 ) -> None:
538     """Mostra el panell en línia complet per a una execució carregada a la UI de Streamlit."""
539
540     from backend.grafics.figures_zones import filter_obs_by_zone
541     from backend.grafics.kpis import compute_kpis
542
543     st.markdown("----")
544
545     # --- Capçalera del panell ---
546     try:
547         head_cols = st.columns([8, 2], gap="small", vertical_alignment="center")
548     except TypeError:
549         head_cols = st.columns([8, 2], gap="small")
550     with head_cols[0]:
551         st.markdown('<h2 class="section-title">Dashboard</h2>', unsafe_allow_html=True)
552     with head_cols[1]:
553         report_action_slot = st.empty()
554
555     st.markdown(
556         f"""
557         <div class="results-dashboard-header">
558             <span class="results-dashboard-tag">
559                 <strong>Entorn</strong>&nbsp; {escape(data.env_name)}
560             </span>
561             <span class="results-dashboard-tag">
562                 <strong>Timesteps</strong>&nbsp; {data.timesteps_num:,}

```

```

563         </span>
564         <span class="results-dashboard-tag">
565             <strong>Model</strong>&nbsp;   {escape(data.model_name)}
566         </span>
567     </div>
568     """
569     unsafe_allow_html=True,
570 )
571
572 st.markdown("<div style='height:0.75rem;'></div>", unsafe_allow_html=True)
573
574 # --- Selector de comparació ---
575 comp_options = [("", "Cap (sense comparació)")] + [
576     (r.path, str(r)) for r in all_run_dirs if r.path != current_run_path
577 ]
578 comp_path = st.selectbox(
579     "Execució de referència per a comparació",
580     options=[p for p, _ in comp_options],
581     format_func=lambda x: dict(comp_options).get(x, x),
582     key="dash_comp_run",
583 )
584
585 comp_data: DashboardData | None = None
586 if comp_path:
587     with st.spinner("Carregant run de comparació..."):
588         comp_data = _load_comparison_data_cached(comp_path)
589     if comp_data is None:
590         st.warning("No s'han pogut carregar les dades de la run de referència.")
591
592 st.markdown("<div style='height:0.75rem;'></div>", unsafe_allow_html=True)
593
594 # Llegim els filtres des de session_state abans de pintar els widgets. Així els KPI
595 # surten a dalt de tot, però sempre calculats amb l'última selecció real de l'usuari.
596 view_mode = st.session_state.get("dash_view_mode", "aggregate")
597 agg_mode = "hour"
598 season = "All"
599 real_period_kind = st.session_state.get("dash_real_period_kind", "day")
600 if real_period_kind not in _REAL_PERIOD_LABELS:
601     real_period_kind = "month"
602     st.session_state["dash_real_period_kind"] = real_period_kind
603 real_period_value = ""
604 real_period_label = None
605
606 if view_mode == "aggregate":
607     agg_mode = st.session_state.get("dash_agg_mode", "hour")
608     season = st.session_state.get("dash_season", "All")
609 else:
610     agg_mode = "raw"
611     real_period_value = st.session_state.get(
612         f"dash_real_period_value_{real_period_kind}", ""
613     )
614     if not real_period_value:
615         _period_opts = build_real_period_options(data.obs, real_period_kind)
616         real_period_value = _period_opts[0][0] if _period_opts else ""
617
618 _default_comfort_scope = (
619     data.comfort_scope_options[0]["value"] if data.comfort_scope_options else "all"
620 )
621 comfort_scope = st.session_state.get("dash_comfort_scope", _default_comfort_scope)
622 zone_val = st.session_state.get("dash_zone", "ALL")
623
624 sel_zone = None if (not data.has_zones or zone_val == "ALL") else zone_val
625 zobs = filter_obs_by_zone(data.obs, sel_zone, data.yaml_cfg)
626 if view_mode == "real":
627     zobs, real_period_label = slice_obs_for_real_period(
628         zobs,
629         real_period_kind,
630         real_period_value,
631     )
632     if zobs.empty:
633         st.warning("No hi ha dades reals disponibles per al període seleccionat.")
634         return
635     st.caption(f"Comparant dades reals del període: {real_period_label}")
636 action_data = select_actions_for_obs(data.actions, zobs)
637 action_data = select_radiant_action_data(action_data, data)
638
639 # KPI principals del període i la zona actius.
640 kpis = compute_kpis(
641     obs=zobs,
642     cost_hourly=data.metrics_dict.get("cost_hourly"),
643     selected_zone=sel_zone,
644     comfort_scope=comfort_scope,
645     comfort_config=data.yaml_cfg,
646 )
647 kpis.pop("Avg Energy Cost (EUR/h)", None)
648
649 add_comfort_percentage_kpi(kpis, zobs, data.yaml_cfg)
650
651 comp_kpis: dict | None = None
652 comp_sel_zone = None
653 comp_zobs = None
654 comp_action_data = pd.DataFrame()

```

```

655 if comp_data is not None:
656     # La comparació intenta aplicar exactament el mateix filtre temporal i de zona.
657     # Si la run de referència no té aquell any concret, fem fallback per mes/dia equivalent.
658     comp_sel_zone = sel_zone if comp_data.has_zones else None
659     comp_zobs = filter_obs_by_zone(comp_data.obs, comp_sel_zone, comp_data.yaml_cfg)
660     if view_mode == "real":
661         comp_zobs, comp_period_label = slice_obs_for_real_period(
662             comp_zobs,
663             real_period_kind,
664             real_period_value,
665             allow_fallback=True,
666         )
667         if comp_zobs.empty:
668             st.warning("La run de referència no té dades per al període real seleccionat.")
669             comp_data = None
670         elif comp_period_label and comp_period_label != real_period_label:
671             st.caption(f"La referència s'ha alineat amb el període: {comp_period_label}")
672     if comp_data is None:
673         comp_zobs = None
674     else:
675         comp_action_data = select_actions_for_obs(comp_data.actions, comp_zobs)
676         comp_action_data = select_radiant_action_data(comp_action_data, comp_data)
677         comp_kpis = compute_kpis(
678             obs=comp_zobs,
679             cost_hourly=comp_data.metrics_dict.get("cost_hourly"),
680             selected_zone=comp_sel_zone,
681             comfort_scope=comfort_scope,
682             comfort_config=comp_data.yaml_cfg,
683         )
684         comp_kpis.pop("Avg Energy Cost (EUR/h)", None)
685         add_comfort_percentage_kpi(comp_kpis, comp_zobs, comp_data.yaml_cfg)
686         st.caption(
687             f"Delta = execucio actual - referencia ({dict(comp_options).get(comp_path, comp_path)})"
688         )
689
690 # Pintem els KPI abans dels filtres: és una decisió de lectura, no de càlcul.
691 _render_kpis(kpis, comp_kpis)
692
693 st.markdown("<div style='height:0.75rem;'></div>", unsafe_allow_html=True)
694
695 # Els filtres es pinten sota els KPI perquè el panell funcioni com un resum primer
696 # i com una eina d'exploració just després.
697 filter_cols = st.columns(5, gap="small")
698 with filter_cols[0]:
699     st.selectbox(
700         "Vista Temporal",
701         options=list(_VIEW_MODE_LABELS.keys()),
702         format_func=lambda x: _VIEW_MODE_LABELS[x],
703         key="dash_view_mode",
704     )
705 with filter_cols[1]:
706     if view_mode == "aggregate":
707         st.selectbox(
708             "Agregació Temporal",
709             options=["hour", "day", "month"],
710             format_func=lambda x: _AGG_LABELS[x],
711             key="dash_agg_mode",
712         )
713     else:
714         st.selectbox(
715             "Període Real",
716             options=list(_REAL_PERIOD_LABELS.keys()),
717             format_func=lambda x: _REAL_PERIOD_LABELS[x],
718             key="dash_real_period_kind",
719         )
720 with filter_cols[2]:
721     if view_mode == "aggregate":
722         st.selectbox(
723             "Filtrar per Estació",
724             options=list(_SEASON_LABELS.keys()),
725             format_func=lambda x: _SEASON_LABELS[x],
726             key="dash_season",
727         )
728     else:
729         real_period_options = build_real_period_options(data.obs, real_period_kind)
730         if real_period_options:
731             option_map = dict(real_period_options)
732             st.selectbox(
733                 "Selecciona el Període",
734                 options=[value for value, _ in real_period_options],
735                 format_func=lambda x: option_map.get(x, x),
736                 key=f"dash_real_period_value_{real_period_kind}",
737             )
738         else:
739             st.selectbox(
740                 "Selecciona el Període",
741                 options=["Sense dades disponibles"],
742                 disabled=True,
743                 key=f"dash_real_period_empty_{real_period_kind}",
744             )
745 with filter_cols[3]:
746     scope_labels = {opt["value"]: opt["label"] for opt in data.comfort_scope_options}

```



```

747     st.selectbox(
748         "Àmbit de Confort",
749         options=[opt["value"] for opt in data.comfort_scope_options],
750         format_func=lambda x: scope_labels[x],
751         key="dash_comfort_scope",
752     )
753     with filter_cols[4]:
754         if data.has_zones:
755             zone_labels = {opt["value"]: opt["label"] for opt in data.all_zone_options}
756             st.selectbox(
757                 "Zona",
758                 options=[opt["value"] for opt in data.all_zone_options],
759                 format_func=lambda x: zone_labels[x],
760                 key="dash_zone",
761             )
762         else:
763             st.selectbox(
764                 "Zona",
765                 options=["Global (Totes)"],
766                 key="dash_zone",
767                 disabled=True,
768             )
769
770     st.markdown("<div style='height:1rem;'></div>", unsafe_allow_html=True)
771
772     # --- Construeix totes les figures del panell ---
773     plot_mode = agg_mode
774     plot_season = season
775     figures = _build_dashboard_figures(
776         data,
777         zobs,
778         action_data,
779         plot_mode=plot_mode,
780         plot_season=plot_season,
781         comfort_scope=comfort_scope,
782         view_mode=view_mode,
783         real_period_kind=real_period_kind,
784     )
785
786     if comp_data is not None and comp_zobs is not None:
787         comp_action_data = select_radiant_action_data(comp_action_data, comp_data)
788         comparison_figures = _build_dashboard_figures(
789             comp_data,
790             comp_zobs,
791             comp_action_data,
792             plot_mode=plot_mode,
793             plot_season=plot_season,
794             comfort_scope=comfort_scope,
795             view_mode=view_mode,
796             real_period_kind=real_period_kind,
797         )
798         figures = _overlay_dashboard_figures(
799             figures,
800             comparison_figures,
801             data=data,
802             view_mode=view_mode,
803         )
804
805     figures = _style_dashboard_figures(figures, view_mode=view_mode)
806
807     _render_report_export(
808         report_action_slot,
809         data,
810         plot_mode=plot_mode,
811         plot_season=plot_season,
812         comfort_scope=comfort_scope,
813         zone_val=zone_val,
814     )
815
816     _render_dashboard_tabs(figures, view_mode=view_mode)

```

D.3 page_components/training_agent.py

Listing 17: page_components/training_agent.py

```

1  """Controls d'hiperparàmetres de l'agent per a la pàgina d'entrenament."""
2
3  from __future__ import annotations
4
5  import streamlit as st
6  from backend.entrenar_agent_session import training_form_key
7  def render_agent_params_section(algo_name: str) -> dict:
8      """Renderitza els ginyos d'hiperparàmetres de l'agent i retorna els seus valors recollits.
9
10     Cobreix la taxa d'aprenentatge, n_steps, els passos de temps totals i tots els SB3 avançats
11     paràmetres (batch_size, gamma, n_epochs, gae_lambda, clip_range,
12     ent_coef, vf_coef, max_grad_norm).
13

```

```

14 Retorna:
15     Un dictat amb tots els valors dels paràmetres d'agent recollits dels ginyes.
16     """
17     # Els tres primers camps són els que més es toquen en proves ràpides; els avançats
18     # queden agrupats perquè la pantalla no sembli una paret de paràmetres.
19     agent_col1, agent_col2, agent_col3 = st.columns(3)
20     with agent_col1:
21         learning_rate = st.number_input(
22             "learning_rate",
23             1e-5,
24             1e-2,
25             3e-4,
26             format="%.5f",
27             key=training_form_key("learning_rate"),
28         )
29     with agent_col2:
30         n_steps = st.number_input(
31             "n_steps (PPO/A2C)",
32             256,
33             4096,
34             2048,
35             step=256,
36             key=training_form_key("n_steps"),
37         )
38     with agent_col3:
39         timesteps_per_year = st.number_input(
40             "total_timesteps",
41             35040,
42             10_000_000,
43             35040,
44             step=35040,
45             key=training_form_key("timesteps_per_year"),
46         )
47
48     with st.container():
49         st.markdown("***Parametres SB3 avançats**")
50         # Aquests valors tenen defaults raonables per PPO/A2C. Si un algorisme no els usa,
51         # assemble_training_payload els filtra abans de crear el model.
52         sb3_col1, sb3_col2, sb3_col3, sb3_col4 = st.columns(4)
53         with sb3_col1:
54             batch_size = st.number_input(
55                 "batch_size",
56                 8,
57                 8192,
58                 256,
59                 step=8,
60                 key=training_form_key("batch_size"),
61             )
62             gamma = st.number_input(
63                 "gamma",
64                 0.0,
65                 0.9999,
66                 0.995,
67                 step=0.0005,
68                 format="%.4f",
69                 key=training_form_key("gamma"),
70             )
71         with sb3_col2:
72             n_epochs = st.number_input(
73                 "n_epochs",
74                 1,
75                 50,
76                 10,
77                 step=1,
78                 key=training_form_key("n_epochs"),
79             )
80             gae_lambda = st.number_input(
81                 "gae_lambda",
82                 0.0,
83                 1.0,
84                 0.95,
85                 step=0.01,
86                 format="%.2f",
87                 key=training_form_key("gae_lambda"),
88             )
89         with sb3_col3:
90             clip_range = st.number_input(
91                 "clip_range",
92                 0.01,
93                 1.0,
94                 0.2,
95                 step=0.01,
96                 format="%.2f",
97                 key=training_form_key("clip_range"),
98             )
99             ent_coef = st.number_input(
100                 "ent_coef",
101                 0.0,
102                 1.0,
103                 0.005,
104                 step=0.001,
105                 format="%.4f",

```

```

106         key=training_form_key("ent_coef"),
107     )
108     with sb3_col4:
109         vf_coef = st.number_input(
110             "vf_coef",
111             0.0,
112             2.0,
113             0.5,
114             step=0.05,
115             format="%.2f",
116             key=training_form_key("vf_coef"),
117         )
118         max_grad_norm = st.number_input(
119             "max_grad_norm",
120             0.0,
121             10.0,
122             0.5,
123             step=0.1,
124             format="%.1f",
125             key=training_form_key("max_grad_norm"),
126         )
127
128     return {
129         "learning_rate": learning_rate,
130         "n_steps": n_steps,
131         "timesteps_per_year": timesteps_per_year,
132         "batch_size": batch_size,
133         "gamma": gamma,
134         "n_epochs": n_epochs,
135         "gae_lambda": gae_lambda,
136         "clip_range": clip_range,
137         "ent_coef": ent_coef,
138         "vf_coef": vf_coef,
139         "max_grad_norm": max_grad_norm,
140     }
141
142
143 # Pàgina principal

```

D.4 page_components/training_page.py

Listing 18: page_components/training_page.py

```

1  """Façana component públic per a la pàgina de entrenament."""
2
3  from __future__ import annotations
4
5  from page_components.training_agent import render_agent_params_section
6  from page_components.training_rewards import render_reward_kwargs_section
7  from page_components.training_shared import (
8      PAGE_LAYOUT,
9      PAGE_TITLE,
10     inject_training_styles,
11     render_saved_training_library,
12     render_training_hero,
13     render_training_section,
14 )
15 from page_components.training_wrappers import render_wrappers_section

```

D.5 page_components/training_rewards.py

Listing 19: page_components/training_rewards.py

```

1  """Controls de configuració de recompenses per a la pàgina de entrenament."""
2
3  from __future__ import annotations
4
5  import streamlit as st
6
7  from backend.entrenar_agent_backend import COST_REWARDS, HOURLY_REWARDS, MULTIZONE_REWARDS
8  from backend.entrenar_agent_session import (
9      BATTERY_REWARDS,
10     ensure_select_state,
11     sanitize_multiselect_state,
12     training_form_key,
13 )
14 def render_reward_kwargs_section(
15     reward_name: str,
16     observation_variables: list[str],
17     candidate_grid_energy_vars: list[str],
18     candidate_battery_charge_vars: list[str],
19     candidate_battery_discharge_vars: list[str],
20     candidate_battery_loss_vars: list[str],
21 ) -> dict:

```

```

22 """Mostra tots els widgets de recompensa i retorna els valors recollits.
23
24 Cobreix pesos d'energia/confort, rangs de confort estacionals, específics de la bateria
25 paràmetres, paràmetres de recompensa de cost i configuració d'hores ocupades.
26
27 Retorna:
28     Un dictat que conté tots els valors kwargs de recompensa recollits dels ginys.
29 """
30 # Defaults de partida. Els ajustem després segons la reward escollida perquè una
31 # bateria, un cost horari i una reward simple no tenen la mateixa escala.
32 temperature_weight = 0.4
33 lambda_energy_cost = 1.0
34 comfort_threshold = 0.5
35 range_comfort_hours = (9, 19)
36 occupied_hours = (8, 18)
37 occupied_discomfort_multiplier = 20.0
38 off_hours_energy_multiplier = 5.0
39 occupied_comfort_multiplier = 2.0
40 unoccupied_comfort_multiplier = 0.3
41 energy_cost_weight = 0.20
42 grid_energy_weight = 0.2
43 battery_cycle_weight = 0.04
44 battery_loss_weight = 0.005
45 simultaneous_battery_weight = 0.005
46 lambda_grid = 0.0001
47 lambda_battery = 0.0001
48 grid_energy_variables = [v for v in candidate_grid_energy_vars if v in observation_variables]
49 battery_charge_variables = [v for v in candidate_battery_charge_vars if v in observation_variables]
50 battery_discharge_variables = [v for v in candidate_battery_discharge_vars if v in observation_variables]
51 battery_loss_variables = [v for v in candidate_battery_loss_vars if v in observation_variables]
52
53 # Aquestes banderes fan que la funció sigui llarga, però mantenen la UI en una sola
54 # passada: només es pinten els camps que realment necessita la reward seleccionada.
55 occupied_reward_selected = reward_name == "OccupiedHoursDiscomfortReward"
56 battery_reward_selected = reward_name in BATTERY_REWARDS
57 new_battery_cost_selected = reward_name == "MultizoneEnergyCostBatteryHVACReward"
58 energy_weight_default = 0.2 if (occupied_reward_selected or battery_reward_selected) else 0.5
59 lambda_energy_default = 0.001 if occupied_reward_selected else 0.0001
60 temperature_weight_default = 0.40 if new_battery_cost_selected else (0.55 if battery_reward_selected else 0.4)
61 sanitize_multiselect_state("grid_energy_variables", observation_variables, grid_energy_variables)
62 sanitize_multiselect_state("battery_charge_variables", observation_variables, battery_charge_variables)
63 sanitize_multiselect_state("battery_discharge_variables", observation_variables, battery_discharge_variables)
64 sanitize_multiselect_state("battery_loss_variables", observation_variables, battery_loss_variables)
65
66 _p1, _p2, _p3 = st.columns(3)
67 with _p1:
68     energy_weight = st.number_input(
69         "energy_weight",
70         0.0,
71         1.0,
72         energy_weight_default,
73         format="%.2f",
74         key=training_form_key("energy_weight"),
75     )
76 with _p2:
77     lambda_energy = st.number_input(
78         "lambda_energy",
79         0.0,
80         10.0,
81         lambda_energy_default,
82         format="%.5f",
83         key=training_form_key("lambda_energy"),
84     )
85 with _p3:
86     lambda_temperature = st.number_input(
87         "lambda_temperature",
88         0.0,
89         10.0,
90         1.0,
91         format="%.2f",
92         key=training_form_key("lambda_temperature"),
93     )
94
95 if reward_name in COST_REWARDS:
96     _c1, _c2 = st.columns(2)
97     with _c1:
98         temperature_weight = st.number_input(
99             "temperature_weight",
100             0.0,
101             1.0,
102             0.4,
103             format="%.2f",
104             key=training_form_key("temperature_weight"),
105         )
106     with _c2:
107         lambda_energy_cost = st.number_input(
108             "lambda_energy_cost",
109             0.0,
110             10.0,
111             0.0001,
112             format="%.5f",
113             key=training_form_key("lambda_energy_cost"),

```

```

114 )
115
116 if reward_name in BATTERY_REWARDS:
117     # En bateria és fàcil confondre potència de xarxa, càrrega i descàrrega. Per això
118     # deixem els multiselect visibles i amb candidats ja filtrats per l'entorn actiu.
119     _b1, _b2, _b3 = st.columns(3)
120     with _b1:
121         temperature_weight = st.number_input(
122             "temperature_weight",
123             0.0,
124             1.0,
125             temperature_weight_default,
126             format="%.2f",
127             key=training_form_key("temperature_weight"),
128         )
129     with _b2:
130         grid_energy_weight = st.number_input(
131             "grid_energy_weight",
132             0.0,
133             1.0,
134             0.2,
135             format="%.2f",
136             key=training_form_key("grid_energy_weight"),
137         )
138     with _b3:
139         battery_cycle_weight = st.number_input(
140             "battery_cycle_weight",
141             0.0,
142             1.0,
143             0.04,
144             format="%.3f",
145             key=training_form_key("battery_cycle_weight"),
146         )
147
148     _b4, _b5, _b6 = st.columns(3)
149     with _b4:
150         battery_loss_weight = st.number_input(
151             "battery_loss_weight",
152             0.0,
153             1.0,
154             0.005,
155             format="%.3f",
156             key=training_form_key("battery_loss_weight"),
157         )
158     with _b5:
159         simultaneous_battery_weight = st.number_input(
160             "simultaneous_battery_weight",
161             0.0,
162             1.0,
163             0.005,
164             format="%.3f",
165             key=training_form_key("simultaneous_battery_weight"),
166         )
167     with _b6:
168         lambda_grid = st.number_input(
169             "lambda_grid",
170             0.0,
171             10.0,
172             0.0001,
173             format="%.5f",
174             key=training_form_key("lambda_grid"),
175         )
176
177     lambda_battery = st.number_input(
178         "lambda_battery",
179         0.0,
180         10.0,
181         0.0001,
182         format="%.5f",
183         key=training_form_key("lambda_battery"),
184     )
185     grid_energy_variables = st.multiselect(
186         "Variables xarxa",
187         observation_variables,
188         default=grid_energy_variables,
189         key=training_form_key("grid_energy_variables"),
190     )
191     battery_charge_variables = st.multiselect(
192         "Variables carrega bateria",
193         observation_variables,
194         default=battery_charge_variables,
195         key=training_form_key("battery_charge_variables"),
196     )
197     battery_discharge_variables = st.multiselect(
198         "Variables descarrega bateria",
199         observation_variables,
200         default=battery_discharge_variables,
201         key=training_form_key("battery_discharge_variables"),
202     )
203     battery_loss_variables = st.multiselect(
204         "Variables perdues bateria",
205         observation_variables,

```

```

206         default=battery_loss_variables,
207         key=training_form_key("battery_loss_variables"),
208     )
209
210 if new_battery_cost_selected:
211     # Aquesta reward barreja confort, cost econòmic i ocupació; separar el bloc evita
212     # que aquests multiplicadors es vegin com a paràmetres genèrics de totes les rewards.
213     st.markdown("**Cost energètic i hores ocupades**")
214     _nc1, _nc2, _nc3 = st.columns(3)
215     with _nc1:
216         energy_cost_weight = st.number_input(
217             "energy_cost_weight", 0.0, 1.0, 0.20, format="%.2f",
218             help="Pes del terme de cost econòmic (grid_kW * EUR/kWh).",
219             key=training_form_key("energy_cost_weight"),
220         )
221     with _nc2:
222         lambda_energy_cost = st.number_input(
223             "lambda_energy_cost", 0.0, 10.0, 1.0, format="%.4f",
224             help="Escala del terme de cost econòmic.",
225             key=training_form_key("lambda_energy_cost"),
226         )
227     with _nc3:
228         occupied_hours = st.slider(
229             "occupied_hours",
230             0,
231             23,
232             (8, 18),
233             key=training_form_key("occupied_hours"),
234         )
235
236     _nm1, _nm2 = st.columns(2)
237     with _nm1:
238         occupied_comfort_multiplier = st.number_input(
239             "occupied_comfort_multiplier", 0.0, 20.0, 2.0, step=0.5, format="%.1f",
240             help="Multiplicador del terme de confort durant hores ocupades.",
241             key=training_form_key("occupied_comfort_multiplier"),
242         )
243     with _nm2:
244         unoccupied_comfort_multiplier = st.number_input(
245             "unoccupied_comfort_multiplier", 0.0, 5.0, 0.3, step=0.1, format="%.1f",
246             help="Multiplicador del terme de confort fora d'hores ocupades.",
247             key=training_form_key("unoccupied_comfort_multiplier"),
248         )
249
250 if reward_name in MULTIZONE_REWARDS:
251     comfort_threshold = st.number_input(
252         "comfort_threshold (C)",
253         0.0,
254         5.0,
255         0.5,
256         step=0.1,
257         format="%.1f",
258         key=training_form_key("comfort_threshold"),
259     )
260
261 if reward_name in HOURLY_REWARDS:
262     range_comfort_hours = st.slider(
263         "range_comfort_hours",
264         0,
265         23,
266         (9, 19),
267         key=training_form_key("range_comfort_hours"),
268     )
269
270 if reward_name == "OccupiedHoursDiscomfortReward":
271     _o1, _o2, _o3 = st.columns(3)
272     with _o1:
273         occupied_hours = st.slider(
274             "occupied_hours",
275             0,
276             23,
277             (8, 18),
278             key=training_form_key("occupied_hours"),
279         )
280     with _o2:
281         occupied_discomfort_multiplier = st.number_input(
282             "occupied_discomfort_multiplier",
283             min_value=1.0,
284             max_value=100.0,
285             value=20.0,
286             step=1.0,
287             key=training_form_key("occupied_discomfort_multiplier"),
288         )
289     with _o3:
290         off_hours_energy_multiplier = st.number_input(
291             "off_hours_energy_multiplier",
292             min_value=1.0,
293             max_value=100.0,
294             value=5.0,
295             step=0.5,
296             key=training_form_key("off_hours_energy_multiplier"),
297         )

```

```

298
299 st.divider()
300 _MONTHS = ["Gen", "Feb", "Mar", "Abr", "Mai", "Jun", "Jul", "Ago", "Set", "Oct", "Nov", "Des"]
301 # Les rewards de confort separen hivern i estiu; deixem la frontera editable perquè
302 # el mateix edifici pot treballar amb climes molt diferents.
303 range_winter = st.slider(
304     "range_comfort_winter",
305     0.0,
306     50.0,
307     (20.0, 23.5),
308     step=0.5,
309     key=training_form_key("range_winter"),
310 )
311 range_summer = st.slider(
312     "range_comfort_summer",
313     0.0,
314     50.0,
315     (23.0, 26.0),
316     step=0.5,
317     key=training_form_key("range_summer"),
318 )
319
320 st.divider()
321 _mc1, _mc2 = st.columns(2)
322 with _mc1:
323     month_options = list(range(1, 13))
324     ensure_select_state("summer_start_m", month_options, fallback=6)
325     summer_start_m = st.selectbox(
326         "Inici estiu (mes)",
327         month_options,
328         index=month_options.index(st.session_state[training_form_key("summer_start_m")]),
329         format_func=lambda x: _MONTHS[x - 1],
330         key=training_form_key("summer_start_m"),
331     )
332 with _mc2:
333     ensure_select_state("summer_final_m", month_options, fallback=9)
334     summer_final_m = st.selectbox(
335         "Final estiu (mes)",
336         month_options,
337         index=month_options.index(st.session_state[training_form_key("summer_final_m")]),
338         format_func=lambda x: _MONTHS[x - 1],
339         key=training_form_key("summer_final_m"),
340     )
341 _dc1, _dc2 = st.columns(2)
342 with _dc1:
343     summer_start_d = st.number_input(
344         "Dia inici",
345         1,
346         31,
347         1,
348         key=training_form_key("summer_start_d"),
349     )
350 with _dc2:
351     summer_final_d = st.number_input(
352         "Dia final",
353         1,
354         31,
355         30,
356         key=training_form_key("summer_final_d"),
357     )
358
359 return {
360     "energy_weight": energy_weight,
361     "lambda_energy": lambda_energy,
362     "lambda_temperature": lambda_temperature,
363     "temperature_weight": temperature_weight,
364     "lambda_energy_cost": lambda_energy_cost,
365     "comfort_threshold": comfort_threshold,
366     "range_comfort_hours": range_comfort_hours,
367     "occupied_hours": occupied_hours,
368     "occupied_discomfort_multiplier": occupied_discomfort_multiplier,
369     "off_hours_energy_multiplier": off_hours_energy_multiplier,
370     "occupied_comfort_multiplier": occupied_comfort_multiplier,
371     "unoccupied_comfort_multiplier": unoccupied_comfort_multiplier,
372     "energy_cost_weight": energy_cost_weight,
373     "grid_energy_weight": grid_energy_weight,
374     "battery_cycle_weight": battery_cycle_weight,
375     "battery_loss_weight": battery_loss_weight,
376     "simultaneous_battery_weight": simultaneous_battery_weight,
377     "lambda_grid": lambda_grid,
378     "lambda_battery": lambda_battery,
379     "grid_energy_variables": grid_energy_variables,
380     "battery_charge_variables": battery_charge_variables,
381     "battery_discharge_variables": battery_discharge_variables,
382     "battery_loss_variables": battery_loss_variables,
383     "range_winter": range_winter,
384     "range_summer": range_summer,
385     "summer_start_m": summer_start_m,
386     "summer_start_d": summer_start_d,
387     "summer_final_m": summer_final_m,
388     "summer_final_d": summer_final_d,
389 }

```

D.6 page_components/training_shared.py

Listing 20: page_components/training_shared.py

```
1  """Constants de pàgina d'entrenament compartides i components de capçalera reutilitzables."""
2
3  from __future__ import annotations
4
5  from html import escape
6  from pathlib import Path
7
8  import streamlit as st
9
10 from ui_theme import inject_studio_theme
11
12 from backend.entrenar_agent_backend import TRAINING_RESULT_KEY, list_saved_training_runs
13 from backend.entrenar_agent_session import (
14     TRAINING_LOADED_ARTIFACT_KEY,
15     TRAINING_SAVED_RUN_KEY,
16     apply_saved_training_ui_state,
17 )
18 PAGE_TITLE = "Entrenar model RL"
19 PAGE_LAYOUT = "wide"
20 INTRODUCTION_TEXT = (
21     "Configura l'entorn, la reward, els wrappers i els parametres de l'agent "
22     "per llançar entrenaments SB3 amb una presentació més clara i ordenada."
23 )
24
25 # Estils
26
27
28 def inject_training_styles() -> None:
29     """Aplica el tema global d'estudi."""
30     inject_studio_theme(max_width=1220)
31
32
33 # Components de renderització
34
35
36 def render_training_hero() -> None:
37     """Mostra el bloc de capçalera principal de la pàgina a la UI de Streamlit."""
38     st.markdown(
39         f"""
40         <section class="training-hero">
41             <div class="hero-kicker">Entrenament i configuració</div>
42             <h1 class="hero-title">{escape(PAGE_TITLE)}</h1>
43             <div class="hero-copy">{escape(INTRODUCTION_TEXT)}</div>
44         </section>
45         """,
46         unsafe_allow_html=True,
47     )
48
49
50 def render_training_section(title: str, kicker: str, description: str) -> None:
51     """Renderitza la capçalera visual d'una secció de pàgina a la UI de Streamlit."""
52     st.markdown(f'<h2 class="page-section-title">{escape(title)}</h2>', unsafe_allow_html=True)
53
54
55 def format_saved_training_label(run: dict) -> str:
56     """Retorna una etiqueta llegible per persones per a una cursa d'entrenament desada."""
57     created_at = run.get("created_at") or "-"
58     env_id = run.get("env_id") or "-"
59     reward_name = run.get("reward_name") or "-"
60     return f"{run.get('artifact_name', 'training')} | {env_id} | {reward_name} | {created_at}"
61
62
63 def render_saved_training_field(label: str, value: object) -> None:
64     """Mostra un únic camp etiquetat dins de la biblioteca de entrenament desada."""
65     st.markdown(
66         f"""
67         <div style="margin:0.1rem 0 0.9rem;">
68             <div style="font-size:0.78rem;color:var(--studio-text-soft);margin-bottom:0.15rem;">
69                 {escape(label)}
70             </div>
71             <div style="font-size:0.98rem;font-weight:600;line-height:1.35;color:var(--studio-text);overflow-wrap:anywhere;">
72                 {escape(str(value or "-"))}
73             </div>
74         </div>
75         """,
76         unsafe_allow_html=True,
77     )
78
79
80 def render_saved_training_library(envs: list[str]) -> None:
81     """Renderitza la secció de la biblioteca d'entrenament desada amb controls de càrrega i descàrrega a la UI de Streamlit."""
82     render_training_section(
83         "Scripts guardats",
84         "Historial",
85         "Recarrega configuracions anteriors i descarrega l'script literal de cada experiment.",
86     )
87
```



```

88 saved_runs = list_saved_training_runs()
89 last_saved_run = st.session_state.get(TRAINING_RESULT_KEY)
90 if last_saved_run:
91     st.success(f"S'ha guardat l'entrenament `{last_saved_run.get('artifact_name', 'training')}`.")
92
93 if not saved_runs:
94     st.info("Encara no hi ha scripts d'entrenament guardats.")
95     return
96
97 # Fem servir artifact_name com a clau estable: és el que també apareix al nom del model
98 # i evita confondre runs amb el mateix entorn però configuracions diferents.
99 run_map = {run["artifact_name"]: run for run in saved_runs if run.get("artifact_name")}
100 run_names = list(run_map.keys())
101 selected_run_name = st.session_state.get(TRAINING_SAVED_RUN_KEY)
102 if selected_run_name not in run_map and run_names:
103     st.session_state[TRAINING_SAVED_RUN_KEY] = run_names[0]
104
105 selected_run_name = st.selectbox(
106     "Entrenament guardat",
107     run_names,
108     key=TRAINING_SAVED_RUN_KEY,
109     format_func=lambda name: format_saved_training_label(run_map[name]),
110 )
111 selected_run = run_map[selected_run_name]
112
113 info_col1, info_col2 = st.columns(2)
114 with info_col1:
115     render_saved_training_field("Entorn", selected_run.get("env_id", "-"))
116 with info_col2:
117     render_saved_training_field("Reward", selected_run.get("reward_name", "-"))
118
119 script_path = Path(selected_run.get("training_script_path", ""))
120 controls_col1, controls_col2 = st.columns(2)
121 if controls_col1.button("Carregar configuració", use_container_width=True):
122     ui_state = selected_run.get("ui_state") or {}
123     if not ui_state:
124         st.warning("Aquest entrenament no té una configuració recarregable.")
125     else:
126         # Recarregar només toca l'estat dels widgets; l'entrenament encara no comença
127         # fins que l'usuari prem el botó principal.
128         apply_saved_training_ui_state(ui_state, envs)
129         st.session_state[TRAINING_LOADED_ARTIFACT_KEY] = selected_run.get("artifact_name")
130         st.rerun()
131
132 if script_path.is_file():
133     controls_col2.download_button(
134         "Descarregar script",
135         data=script_path.read_bytes(),
136         file_name=script_path.name,
137         mime="text/x-python",
138         use_container_width=True,
139     )
140 else:
141     controls_col2.warning("No s'ha trobat el fitxer .py d'aquest entrenament.")
142
143 loaded_artifact = st.session_state.get(TRAINING_LOADED_ARTIFACT_KEY)
144 if loaded_artifact:
145     st.info(f"Configuració carregada a la UI: `{loaded_artifact}`")

```

D.7 page_components/training_wrappers.py

Listing 21: page_components/training_wrappers.py

```

1 """Controls de wrappers i registre per a la pàgina d'entrenament."""
2
3 from __future__ import annotations
4
5 import streamlit as st
6
7 from backend.entrenar_agent_backend import DEFAULT_FORECAST_COLUMNS, ENERGY_COST_DIR, list_files
8 from backend.entrenar_agent_session import (
9     reset_widget_state_if_disabled,
10     sanitize_multiselect_state,
11     seed_discrete_incremental_widget_defaults,
12     seed_incremental_widget_defaults,
13     training_form_key,
14 )
15
16
17 def render_observation_wrappers(
18     observation_variables: list[str],
19     context_variables: list[str],
20     candidate_temp_vars: list[str],
21     candidate_setpoint_vars: list[str],
22 ) -> dict:
23     """Pinta els controls que modifiquen les observacions abans d'arribar a l'agent."""
24     st.caption("OBSERVACIÓ")
25

```

```

26 # Streamlit conserva valors antics encara que canviï l'entorn; sanejar-los aquí
27 # evita que un multiselect apunti a variables que ja no existeixen.
28 sanitize_multiselect_state("previous_variables", observation_variables, candidate_temp_vars[:1])
29 sanitize_multiselect_state("weather_forecast_columns", DEFAULT_FORECAST_COLUMNS, list(DEFAULT_FORECAST_COLUMNS))
30 sanitize_multiselect_state("delta_temp_variables", observation_variables, candidate_temp_vars)
31 sanitize_multiselect_state("delta_setpoint_variables", observation_variables, candidate_setpoint_vars[:1])
32 sanitize_multiselect_state("reduced_observations", observation_variables, [])
33
34 enable_datetime_wrapper = st.checkbox(
35     "DatetimeWrapper",
36     False,
37     key=training_form_key("enable_datetime_wrapper"),
38 )
39
40 enable_previous_wrapper = st.checkbox(
41     "PreviousObservationWrapper",
42     False,
43     key=training_form_key("enable_previous_wrapper"),
44 )
45 previous_variables = []
46 if enable_previous_wrapper:
47     previous_variables = st.multiselect(
48         "Variables pas anterior",
49         observation_variables,
50         default=candidate_temp_vars[:1],
51         key=training_form_key("previous_variables"),
52     )
53
54 enable_multi_obs_wrapper = st.checkbox(
55     "MultiObsWrapper",
56     False,
57     key=training_form_key("enable_multi_obs_wrapper"),
58 )
59 multi_obs_n = 5
60 multi_obs_flatten = True
61 if enable_multi_obs_wrapper:
62     multi_obs_n = st.slider(
63         "Observacions apilades",
64         2,
65         12,
66         5,
67         key=training_form_key("multi_obs_n"),
68     )
69     multi_obs_flatten = st.checkbox(
70         "Aplanar observacio",
71         True,
72         key=training_form_key("multi_obs_flatten"),
73     )
74
75 enable_normalize_obs_wrapper = st.checkbox(
76     "NormalizeObservation",
77     False,
78     key=training_form_key("enable_normalize_obs_wrapper"),
79 )
80 normalize_obs_auto = True
81 normalize_obs_epsilon = 1e-8
82 normalize_obs_mean = ""
83 normalize_obs_var = ""
84 if enable_normalize_obs_wrapper:
85     normalize_obs_auto = st.checkbox(
86         "Actualitzacio automatica mean/var",
87         True,
88         key=training_form_key("normalize_obs_auto"),
89     )
90     normalize_obs_epsilon = st.number_input(
91         "epsilon",
92         1e-10,
93         1e-2,
94         1e-8,
95         format="%.10f",
96         key=training_form_key("normalize_obs_epsilon"),
97     )
98     normalize_obs_mean = st.text_input(
99         "Ruta mean.txt (opcional)",
100         "",
101         key=training_form_key("normalize_obs_mean"),
102     )
103     normalize_obs_var = st.text_input(
104         "Ruta var.txt (opcional)",
105         "",
106         key=training_form_key("normalize_obs_var"),
107     )
108
109 enable_weather_forecast_wrapper = st.checkbox(
110     "WeatherForecastingWrapper",
111     False,
112     key=training_form_key("enable_weather_forecast_wrapper"),
113 )
114 weather_forecast_n = 5
115 weather_forecast_delta = 1
116 weather_forecast_columns = list(DEFAULT_FORECAST_COLUMNS)
117 if enable_weather_forecast_wrapper:

```

```

118     weather_forecast_n = st.slider(
119         "Mostres de previsi",
120         1,
121         24,
122         5,
123         key=training_form_key("weather_forecast_n"),
124     )
125     weather_forecast_delta = st.slider(
126         "Separacio entre mostres",
127         1,
128         24,
129         1,
130         key=training_form_key("weather_forecast_delta"),
131     )
132     weather_forecast_columns = st.multiselect(
133         "Variables de previsi",
134         DEFAULT_FORECAST_COLUMNS,
135         default=DEFAULT_FORECAST_COLUMNS,
136         key=training_form_key("weather_forecast_columns"),
137     )
138
139     enable_delta_temp_wrapper = st.checkbox(
140         "DeltaTempWrapper",
141         False,
142         key=training_form_key("enable_delta_temp_wrapper"),
143     )
144     delta_temp_variables = []
145     delta_setpoint_variables = []
146     if enable_delta_temp_wrapper:
147         delta_temp_variables = st.multiselect(
148             "Variables temperatura",
149             observation_variables,
150             default=candidate_temp_vars,
151             key=training_form_key("delta_temp_variables"),
152         )
153         delta_setpoint_variables = st.multiselect(
154             "Variables setpoint",
155             observation_variables,
156             default=candidate_setpoint_vars[:1],
157             key=training_form_key("delta_setpoint_variables"),
158         )
159
160     enable_reduce_obs_wrapper = st.checkbox(
161         "ReduceObservationWrapper",
162         False,
163         key=training_form_key("enable_reduce_obs_wrapper"),
164     )
165     reduced_observations = []
166     if enable_reduce_obs_wrapper:
167         reduced_observations = st.multiselect(
168             "Variables a excloure",
169             observation_variables,
170             default=[],
171             key=training_form_key("reduced_observations"),
172         )
173
174     variability_disabled = not bool(context_variables)
175     # Si l'entorn no exposa variables de context, el wrapper es desactiva i també netegem
176     # l'estat antic perquè no torni a aparèixer marcat en una reexecució.
177     reset_widget_state_if_disabled("enable_variability_context_wrapper", variability_disabled)
178     enable_variability_context_wrapper = st.checkbox(
179         "VariabilityContextWrapper",
180         False,
181         disabled=variability_disabled,
182         key=training_form_key("enable_variability_context_wrapper"),
183     )
184     variability_context_low = []
185     variability_context_high = []
186     variability_delta_value = 1.0
187     variability_step_frequency_min = 96
188     variability_step_frequency_max = 96 * 7
189     if enable_variability_context_wrapper:
190         st.caption("Rang de variabilitat del context")
191         for context_variable in context_variables:
192             low_col, high_col = st.columns(2)
193             with low_col:
194                 variability_context_low.append(
195                     st.number_input(
196                         f"{context_variable} min",
197                         value=0.0,
198                         key=training_form_key(f"variability_context_low_{context_variable}"),
199                     )
200             )
201             with high_col:
202                 variability_context_high.append(
203                     st.number_input(
204                         f"{context_variable} max",
205                         value=2.0,
206                         key=training_form_key(f"variability_context_high_{context_variable}"),
207                     )
208             )
209     variability_delta_value = st.number_input(

```

```

210         "Delta context",
211         min_value=0.01,
212         value=1.0,
213         step=0.1,
214         key=training_form_key("variability_delta_value"),
215     )
216     freq_col1, freq_col2 = st.columns(2)
217     with freq_col1:
218         variability_step_frequency_min = st.number_input(
219             "Freq. min passos",
220             min_value=1,
221             value=96,
222             step=1,
223             key=training_form_key("variability_step_frequency_min"),
224         )
225     with freq_col2:
226         variability_step_frequency_max = st.number_input(
227             "Freq. max passos",
228             min_value=2,
229             value=96 * 7,
230             step=1,
231             key=training_form_key("variability_step_frequency_max"),
232         )
233
234     return {
235         "enable_datetime_wrapper": enable_datetime_wrapper,
236         "enable_previous_wrapper": enable_previous_wrapper,
237         "previous_variables": previous_variables,
238         "enable_multi_obs_wrapper": enable_multi_obs_wrapper,
239         "multi_obs_n": multi_obs_n,
240         "multi_obs_flatten": multi_obs_flatten,
241         "enable_normalize_obs_wrapper": enable_normalize_obs_wrapper,
242         "normalize_obs_auto": normalize_obs_auto,
243         "normalize_obs_epsilon": normalize_obs_epsilon,
244         "normalize_obs_mean": normalize_obs_mean,
245         "normalize_obs_var": normalize_obs_var,
246         "enable_weather_forecast_wrapper": enable_weather_forecast_wrapper,
247         "weather_forecast_n": weather_forecast_n,
248         "weather_forecast_delta": weather_forecast_delta,
249         "weather_forecast_columns": weather_forecast_columns,
250         "enable_delta_temp_wrapper": enable_delta_temp_wrapper,
251         "delta_temp_variables": delta_temp_variables,
252         "delta_setpoint_variables": delta_setpoint_variables,
253         "enable_reduce_obs_wrapper": enable_reduce_obs_wrapper,
254         "reduced_observations": reduced_observations,
255         "enable_variability_context_wrapper": enable_variability_context_wrapper,
256         "variability_context_low": variability_context_low,
257         "variability_context_high": variability_context_high,
258         "variability_delta_value": variability_delta_value,
259         "variability_step_frequency_min": variability_step_frequency_min,
260         "variability_step_frequency_max": variability_step_frequency_max,
261     }
262
263
264 def render_action_wrappers(
265     env_meta: dict,
266     action_variables: list[str],
267     action_space,
268 ) -> dict:
269     """Pinta els controls que canvien com l'agent envia accions a l'entorn."""
270     st.caption("ACCIO")
271
272     # Els wrappers incrementals només tenen sentit quan l'acció és contínua: converteixen
273     # una ordre absoluta en canvis petits, més semblants a moure un termòstat real.
274     sanitize_multiselect_state("incremental_variables", action_variables, action_variables[:1])
275     enable_incremental_wrapper = st.checkbox(
276         "IncrementalWrapper",
277         False,
278         disabled=not env_meta["is_continuous"],
279         key=training_form_key("enable_incremental_wrapper"),
280     )
281     incremental_variables = []
282     incremental_definition = {}
283     incremental_initial_values = []
284     if enable_incremental_wrapper:
285         seed_incremental_widget_defaults(action_variables)
286         incremental_variables = st.multiselect(
287             "Variables d'accio incrementals",
288             action_variables,
289             default=action_variables[:1],
290             key=training_form_key("incremental_variables"),
291         )
292     for variable_name in incremental_variables:
293         action_index = action_variables.index(variable_name)
294         low_value = float(action_space.low[action_index])
295         high_value = float(action_space.high[action_index])
296         default_initial = float((low_value + high_value) / 2.0)
297         st.markdown(f"***{variable_name}***")
298         init_col, delta_col, step_col = st.columns(3)
299         with init_col:
300             initial_value = st.number_input(
301                 "Valor inicial",

```

```

302         value=default_initial,
303         key=training_form_key(f"incr_init_{variable_name}"),
304     )
305     with delta_col:
306         delta_value = st.number_input(
307             "Delta max",
308             min_value=0.1,
309             value=max((high_value - low_value) / 4.0, 0.5),
310             step=0.1,
311             key=training_form_key(f"incr_delta_{variable_name}"),
312         )
313     with step_col:
314         step_value = st.number_input(
315             "Pas",
316             min_value=0.1,
317             value=max((high_value - low_value) / 20.0, 0.1),
318             step=0.1,
319             key=training_form_key(f"incr_step_{variable_name}"),
320         )
321     incremental_definition[variable_name] = (delta_value, step_value)
322     incremental_initial_values.append(initial_value)
323     st.session_state[training_form_key("incremental_definition")] = incremental_definition
324     st.session_state[training_form_key("incremental_initial_values")] = incremental_initial_values
325
326     enable_discrete_incremental_wrapper = st.checkbox(
327         "DiscreteIncrementalWrapper",
328         False,
329         disabled=not env_meta["is_continuous"],
330         key=training_form_key("enable_discrete_incremental_wrapper"),
331     )
332     discrete_incremental_initial_values = []
333     discrete_incremental_delta = 2.0
334     discrete_incremental_step = 0.5
335     if enable_discrete_incremental_wrapper:
336         seed_discrete_incremental_widget_defaults(action_variables)
337         for action_index, variable_name in enumerate(action_variables):
338             low_value = float(action_space.low[action_index])
339             high_value = float(action_space.high[action_index])
340             default_initial = float((low_value + high_value) / 2.0)
341             discrete_incremental_initial_values.append(
342                 st.number_input(
343                     f"Valor inicial {variable_name}",
344                     value=default_initial,
345                     key=training_form_key(f"disc_incr_init_{variable_name}"),
346                 )
347             )
348     discrete_incremental_delta = st.number_input(
349         "Delta discret max",
350         min_value=0.1,
351         value=2.0,
352         step=0.1,
353         key=training_form_key("discrete_incremental_delta"),
354     )
355     discrete_incremental_step = st.number_input(
356         "Pas discret",
357         min_value=0.1,
358         value=0.5,
359         step=0.1,
360         key=training_form_key("discrete_incremental_step"),
361     )
362     st.session_state[training_form_key("discrete_incremental_initial_values")] = (
363         discrete_incremental_initial_values
364     )
365
366     normalize_action_disabled = (
367         not env_meta["is_continuous"]
368         or enable_incremental_wrapper
369         or enable_discrete_incremental_wrapper
370     )
371     # NormalizeAction és còmode com a default en Box, però no el deixem conviure amb
372     # wrappers incrementals perquè llavors l'escala de control queda doblement transformada.
373     normalize_action_key = training_form_key("enable_normalize_action")
374     normalize_action_seed_key = training_form_key("enable_normalize_action_default_seeded")
375     reset_widget_state_if_disabled("enable_normalize_action", normalize_action_disabled)
376     if normalize_action_disabled:
377         st.session_state.pop(normalize_action_seed_key, None)
378     elif not st.session_state.get(normalize_action_seed_key):
379         st.session_state[normalize_action_key] = True
380         st.session_state[normalize_action_seed_key] = True
381     enable_normalize_action = st.checkbox(
382         "NormalizeAction",
383         value=not normalize_action_disabled,
384         disabled=normalize_action_disabled,
385         key=normalize_action_key,
386     )
387     if normalize_action_disabled:
388         enable_normalize_action = False
389
390     storage_smoothing_disabled = env_meta.get("building_name") != "OfficeGridStorageSmoothing.epJSON"
391     # Aquest wrapper està escrit per a un edifici concret amb bateria i restriccions de xarxa;
392     # en la resta d'entorns el mostrem desactivat per evitar errors difícils d'entendre.
393     reset_widget_state_if_disabled("enable_storage_smoothing_wrapper", storage_smoothing_disabled)

```

```

394 enable_storage_smoothing_wrapper = st.checkbox(
395     "OfficeGridStorageSmoothingActionConstraintsWrapper",
396     False,
397     disabled=storage_smoothing_disabled,
398     key=training_form_key("enable_storage_smoothing_wrapper"),
399 )
400
401 return {
402     "enable_incremental_wrapper": enable_incremental_wrapper,
403     "incremental_variables": incremental_variables,
404     "incremental_definition": incremental_definition,
405     "incremental_initial_values": incremental_initial_values,
406     "enable_discrete_incremental_wrapper": enable_discrete_incremental_wrapper,
407     "discrete_incremental_initial_values": discrete_incremental_initial_values,
408     "discrete_incremental_delta": discrete_incremental_delta,
409     "discrete_incremental_step": discrete_incremental_step,
410     "enable_normalize_action": enable_normalize_action,
411     "enable_storage_smoothing_wrapper": enable_storage_smoothing_wrapper,
412 }
413
414
415 def render_energy_logging_wrappers(
416     energy_cost_files: list[str],
417     initial_energy_cost_path: str,
418     initial_file_logger_name: str,
419 ) -> dict:
420     """Pinta els controls de cost energètic i de fitxer de registre."""
421     st.caption("ENERGIA I LOGS")
422
423     energy_cost_path = initial_energy_cost_path
424     file_logger_name = initial_file_logger_name
425
426     use_energy_cost = st.checkbox(
427         "EnergyCostWrapper",
428         False,
429         key=training_form_key("use_energy_cost"),
430     )
431     if use_energy_cost:
432         if energy_cost_files:
433             selected_energy_cost = st.selectbox(
434                 "Fitxer cost",
435                 energy_cost_files,
436                 index=energy_cost_files.index(energy_cost_path) if energy_cost_path in energy_cost_files else 0,
437             )
438             energy_cost_path = selected_energy_cost
439         energy_cost_path = st.text_input(
440             "Ruta fitxer cost",
441             energy_cost_path,
442             key=training_form_key("energy_cost_path"),
443         )
444
445     use_file_logger = st.checkbox(
446         "EnergyCostFileLogger",
447         False,
448         key=training_form_key("use_file_logger"),
449     )
450     if use_file_logger:
451         file_logger_name = st.text_input(
452             "Nom fitxer registre",
453             "energy_cost_log.csv",
454             key=training_form_key("file_logger_name"),
455         )
456
457     return {
458         "energy_cost_path": energy_cost_path,
459         "file_logger_name": file_logger_name,
460         "use_energy_cost": use_energy_cost,
461         "use_file_logger": use_file_logger,
462     }
463
464
465 def render_wrappers_section(
466     env_meta: dict,
467     observation_variables: list[str],
468     action_variables: list[str],
469     action_space,
470     context_variables: list[str],
471     candidate_temp_vars: list[str],
472     candidate_setpoint_vars: list[str],
473 ) -> dict:
474     """Agrupa els controls de wrappers i retorna una configuració plana per al backend."""
475     energy_cost_files = list_files(ENERGY_COST_DIR, {".csv"})
476     default_energy_cost_path = (
477         energy_cost_files[0] if energy_cost_files
478         else str(ENERGY_COST_DIR / "PVPC_active_energy_billing_Iberian_Peninsula_2023.csv")
479     )
480     default_file_logger_name = "energy_cost_log.csv"
481     initial_energy_cost_path = st.session_state.get(
482         training_form_key("energy_cost_path"),
483         default_energy_cost_path,
484     )
485     initial_file_logger_name = st.session_state.get(

```

```

486     training_form_key("file_logger_name"),
487     default_file_logger_name,
488 )
489
490 observation_column, action_column = st.columns(2)
491 with observation_column:
492     observation_wrapper_values = render_observation_wrappers(
493         observation_variables=observation_variables,
494         context_variables=context_variables,
495         candidate_temp_vars=candidate_temp_vars,
496         candidate_setpoint_vars=candidate_setpoint_vars,
497     )
498 with action_column:
499     action_wrapper_values = render_action_wrappers(
500         env_meta=env_meta,
501         action_variables=action_variables,
502         action_space=action_space,
503     )
504 energy_logging_values = render_energy_logging_wrappers(
505     energy_cost_files=energy_cost_files,
506     initial_energy_cost_path=initial_energy_cost_path,
507     initial_file_logger_name=initial_file_logger_name,
508 )
509
510 # Es retorna un diccionari pla perquè el muntatge final de training pugui barrejar
511 # reward, wrappers i paràmetres de l'agent sense dependre de l'ordre visual de la pàgina.
512 return {
513     **energy_logging_values,
514     **observation_wrapper_values,
515     **action_wrapper_values,
516 }

```

E Codi: Backend funcional

E.1 backend/___init___py

Listing 22: backend/___init___py

```

1  """Lògica d'aplicació de BEMS-RL Studio segura per importar.
2
3  El paquet backend conté el codi que dona suport a la interfície Streamlit:
4  creació d'entorns, anàlisi climàtica, orquestració de l'entrenament, avaluació,
5  control en directe, agregació de resultats i generació d'informes. Els mòduls
6  d'aquest paquet eviten renderitzar la UI directament perquè es puguin importar
7  des de proves, scripts i Sphinx autodoc.
8  """

```

E.2 backend/afegir_entorn_analysis.py

Listing 23: backend/afegir_entorn_analysis.py

```

1  """Lectura i anàlisi d'edificis per preparar la configuració dels entorns."""
2
3  from __future__ import annotations
4
5  import csv
6  import io
7  import json
8  from pathlib import Path
9  from typing import Any, Dict, List, Optional, Sequence, Tuple
10
11  import gymnasium as gym
12  import numpy as np
13  from sinergym.envs.eplus_env import EplusEnv
14  from sinergym.utils.rewards import BaseReward
15
16  from backend.afegir_entorn_constants import DEFAULT_TIME_VARIABLES
17  from backend.afegir_entorn_types import ActuatorOption, BuildingTrainingAnalysis
18  from backend.afegir_entorn_utils import normalize_token, sanitize_identifier
19
20  class ProbeReward(BaseReward):
21     """Reward nul usat només per inspeccionar handlers disponibles."""
22
23     def __call__(self, obs_dict: Dict[str, Any]) -> Tuple[float, Dict[str, Any]]:
24         """Executa l'objecte cridable."""
25         return 0.0, {}
26
27
28  def load_building_json(building_path: Path) -> Dict[str, Any]:
29     """Carrega i analitza un fitxer d'edifici de format eJSON en un diccionari.
30
31     Paràmetres:

```

```

32     building_path (Path): camí cap al fitxer epJSON.
33
34     Retorna:
35         Dict[str, Any]: el diccionari carregat que renderitza les dades de l'edifici.
36     """
37     with open(building_path, "r", encoding="utf-8") as file_handle:
38         return json.load(file_handle)
39
40
41 def extract_schedule_type_index(building_data: Dict[str, Any]) -> Dict[str, str]:
42     """Extreu una assignació de noms d'objectes de planificació als seus respectius tipus de blocs de planificació.
43
44     Paràmetres:
45         building_data (Dict[str, Any]): les dades de l'edifici epJSON carregades.
46
47     Retorna:
48         Dict[str, str]: un diccionari que assigna noms de planificació als seus tipus de bloc d'objectes
49         (e.g., 'Programació:Compact').
50     """
51     schedule_types: Dict[str, str] = {}
52     for object_type, objects in building_data.items():
53         if not object_type.startswith("Schedule:") or not isinstance(objects, dict):
54             continue
55         for schedule_name in objects.keys():
56             schedule_types[schedule_name] = object_type
57     return schedule_types
58
59
60 def resolve_schedule_name(
61     schedule_types: Dict[str, str],
62     schedule_name: str,
63 ) -> Optional[str]:
64     """Resol les referències de planificació amb noms d'estil EnergyPlus que no distingeixen entre majúscules i minúscules."""
65     if not schedule_name:
66         return None
67     if schedule_name in schedule_types:
68         return schedule_name
69
70     target_casefold = schedule_name.casefold()
71     for actual_name in schedule_types.keys():
72         if actual_name.casefold() == target_casefold:
73             return actual_name
74
75     target_normalized = normalize_token(schedule_name)
76     for actual_name in schedule_types.keys():
77         if normalize_token(actual_name) == target_normalized:
78             return actual_name
79
80     return None
81
82
83 def _register_thermostat_candidate(
84     detected: Dict[Tuple[str, str], Dict[str, Any]],
85     schedule_types: Dict[str, str],
86     schedule_name: str,
87     category: str,
88     zone_name: Optional[str],
89 ) -> None:
90     """Registra una planificació de termòstat com a entrada de controlador candidat si encara no s'ha fet un seguiment.
91
92     Crea una nova entrada a `detected` per a la clau donada (categoria, schedule_name) si no hi ha,
93     aleshores hi associa el nom de la zona.
94
95     Paràmetres:
96         detected (Dict): Registre de candidats acumulat clausurat per (categoria, schedule_name).
97         schedule_types (Dict[str, str]): Mapeig de noms de planificació amb les seves cadenes del tipus de bloc.
98         schedule_name (str): el nom del programa EnergyPlus per registrar-se.
99         category (str): categoria de consigna, e.g. 'heating_setpoint' o 'cooling_setpoint'.
100         zone_name (Optional[str]): Nom de la zona a associar amb aquest candidat.
101     """
102     resolved_schedule_name = resolve_schedule_name(schedule_types, schedule_name)
103     if resolved_schedule_name is None:
104         return
105     key = (category, resolved_schedule_name)
106     if key not in detected:
107         detected[key] = {
108             "category": category,
109             "schedule_name": resolved_schedule_name,
110             "zones": set(),
111             "references": set(),
112             "source_kind": "thermostat",
113             "element_type": schedule_types[resolved_schedule_name],
114         }
115     if zone_name:
116         detected[key]["zones"].add(zone_name)
117         detected[key]["references"].add(zone_name)
118
119
120 def extract_thermostat_schedule_candidates(
121     building_data: Dict[str, Any],
122     schedule_types: Dict[str, str],
123 ) -> Tuple[List[str], List[Dict[str, Any]]]:

```



```

124 """Detecta programes de termòstat implícits basats en els punts de consigna Dual i Single del model.
125
126 Paràmetres:
127     building_data (Dict[str, Any]): les dades epJSON carregades.
128     schedule_types (Dict[str, str]): una assignació de noms de planificació a tipus de bloc de planificació.
129
130 Retorna:
131     Tuple[List[str], List[Dict[str, Any]]]: una tupla que conté una llista de detectats
132     zones del termòstat i una llista de diccionaris estructurats amb detalls dels actuadors candidats.
133 """
134 thermostat_controls = building_data.get("ZoneControl:Thermostat", {})
135 dual_setpoints = building_data.get("ThermostatSetpoint:DualSetpoint", {})
136 single_heating = building_data.get("ThermostatSetpoint:SingleHeating", {})
137 single_cooling = building_data.get("ThermostatSetpoint:SingleCooling", {})
138
139 thermostat_zones: List[str] = []
140 detected: Dict[Tuple[str, str], Dict[str, Any]] = {}
141
142 for _, thermostat_spec in thermostat_controls.items():
143     zone_name = thermostat_spec.get("zone_or_zonelist_name") or thermostat_spec.get("zone_name")
144     if zone_name and zone_name not in thermostat_zones:
145         thermostat_zones.append(zone_name)
146
147     # EnergyPlus desa fins a cinc controls numerats per termòstat; cal revisar-los tots
148     # perquè un mateix objecte pot combinar consignes Dual i Single.
149     for control_index in range(1, 6):
150         object_type = thermostat_spec.get(f"control_{control_index}_object_type")
151         object_name = thermostat_spec.get(f"control_{control_index}_name")
152         if not object_type or not object_name:
153             continue
154
155         if object_type == "ThermostatSetpoint:DualSetpoint":
156             dual_spec = dual_setpoints.get(object_name, {})
157             _register_thermostat_candidate(
158                 detected,
159                 schedule_types,
160                 dual_spec.get("heating_setpoint_temperature_schedule_name", ""),
161                 "heating_setpoint",
162                 zone_name,
163             )
164             _register_thermostat_candidate(
165                 detected,
166                 schedule_types,
167                 dual_spec.get("cooling_setpoint_temperature_schedule_name", ""),
168                 "cooling_setpoint",
169                 zone_name,
170             )
171         elif object_type == "ThermostatSetpoint:SingleHeating":
172             single_spec = single_heating.get(object_name, {})
173             _register_thermostat_candidate(
174                 detected,
175                 schedule_types,
176                 single_spec.get("setpoint_temperature_schedule_name", ""),
177                 "heating_setpoint",
178                 zone_name,
179             )
180         elif object_type == "ThermostatSetpoint:SingleCooling":
181             single_spec = single_cooling.get(object_name, {})
182             _register_thermostat_candidate(
183                 detected,
184                 schedule_types,
185                 single_spec.get("setpoint_temperature_schedule_name", ""),
186                 "cooling_setpoint",
187                 zone_name,
188             )
189
190 ordered_candidates: List[Dict[str, Any]] = []
191 # Ordenem els candidats perquè la UI i els YAML generats siguin estables entre execucions.
192 for key in sorted(detected.keys(), key=lambda item: (item[0], item[1].lower())):
193     candidate = detected[key]
194     ordered_candidates.append(
195         {
196             "schedule_name": candidate["schedule_name"],
197             "category": candidate["category"],
198             "zones": tuple(sorted(candidate["zones"])),
199             "references": tuple(sorted(candidate["references"])),
200             "source_kind": candidate["source_kind"],
201             "element_type": candidate["element_type"],
202         }
203     )
204
205 return thermostat_zones, ordered_candidates
206
207
208 def _collect_schedule_references_from_value(
209     value: Any,
210     schedule_types: Dict[str, str],
211 ) -> List[str]:
212     """Recollíu referències de calendari a partir del valor."""
213     references: List[str] = []
214     if isinstance(value, str):
215         resolved_schedule_name = resolve_schedule_name(schedule_types, value)

```

```

216     if resolved_schedule_name is not None:
217         references.append(resolved_schedule_name)
218     elif isinstance(value, dict):
219         for nested_value in value.values():
220             references.extend(_collect_schedule_references_from_value(nested_value, schedule_types))
221     elif isinstance(value, list):
222         for nested_value in value:
223             references.extend(_collect_schedule_references_from_value(nested_value, schedule_types))
224     return references
225
226
227 def extract_schedule_numeric_values(
228     building_data: Dict[str, Any],
229     schedule_types: Dict[str, str],
230     schedule_name: str,
231     visited: Optional[set[str]] = None,
232 ) -> List[float]:
233     """Extreu valors numèrics de planificació."""
234     if visited is None:
235         visited = set()
236
237     # Les planificacions poden referenciar altres planificacions; aquest conjunt evita bucles
238     # quan EnergyPlus encadena objectes Schedule:* de manera recursiva.
239     resolved_schedule_name = resolve_schedule_name(schedule_types, schedule_name)
240     if resolved_schedule_name is None:
241         return []
242     if resolved_schedule_name in visited:
243         return []
244
245     object_type = schedule_types.get(resolved_schedule_name)
246     if object_type is None:
247         return []
248
249     schedule_objects = building_data.get(object_type, {})
250     schedule_spec = schedule_objects.get(resolved_schedule_name)
251     if not isinstance(schedule_spec, dict):
252         return []
253
254     visited.add(resolved_schedule_name)
255
256     if object_type == "Schedule:Compact":
257         values: List[float] = []
258         for entry in schedule_spec.get("data", []):
259             if not isinstance(entry, dict):
260                 continue
261             field_value = entry.get("field")
262             if isinstance(field_value, (int, float)) and not isinstance(field_value, bool):
263                 values.append(float(field_value))
264         return values
265
266     if object_type == "Schedule:Constant":
267         values = []
268         for key, value in schedule_spec.items():
269             if key == "schedule_type_limits_name":
270                 continue
271             if isinstance(value, (int, float)) and not isinstance(value, bool):
272                 values.append(float(value))
273         return values
274
275     if object_type == "Schedule:Day:Hourly":
276         values = []
277         for key, value in schedule_spec.items():
278             if key.lower().startswith("hour_") and isinstance(value, (int, float)) and not isinstance(value, bool):
279                 values.append(float(value))
280         return values
281
282     if object_type in {"Schedule:Year", "Schedule:Week:Daily", "Schedule:Week:Compact"}:
283         values = []
284         for nested_schedule_name in _collect_schedule_references_from_value(schedule_spec, schedule_types):
285             values.extend(
286                 extract_schedule_numeric_values(
287                     building_data,
288                     schedule_types,
289                     nested_schedule_name,
290                     visited,
291                 )
292             )
293         return values
294
295     return []
296
297
298 def extract_schedule_value_range(
299     building_data: Dict[str, Any],
300     schedule_types: Dict[str, str],
301     schedule_name: str,
302 ) -> Tuple[Optional[float], Optional[float]]:
303     """Extreu l'interval de valors de la planificació."""
304     values = extract_schedule_numeric_values(building_data, schedule_types, schedule_name)
305     if not values:
306         return None, None
307     return min(values), max(values)

```

```

308
309
310 def summarize_references(prefix: str, references: Sequence[str]) -> str:
311     """Summarize references."""
312     cleaned = [reference for reference in references if reference]
313     if not cleaned:
314         return prefix
315     if len(cleaned) == 1:
316         return f"{prefix}: {cleaned[0]}"
317     if len(cleaned) == 2:
318         return f"{prefix}: {cleaned[0]}, {cleaned[1]}"
319     return f"{prefix}: {cleaned[0]}, {cleaned[1]} +{len(cleaned) - 2}"
320
321
322 def extract_scheduled_hvac_controller_candidates(
323     building_data: Dict[str, Any],
324     schedule_types: Dict[str, str],
325     excluded_schedule_names: Sequence[str],
326 ) -> List[Dict[str, Any]]:
327     """Identifica els punts de consigna programats i els gestors de disponibilitat a partir del model d'edifici que
328     encara no es gestionen com a punts de consigna explícits del termòstat.
329
330     Paràmetres:
331         building_data (Dict[str, Any]): les dades epJSON carregades.
332         schedule_types (Dict[str, str]): una assignació de noms de planificació a tipus de planificació.
333         excluded_schedule_names (Sequence[str]): Noms de les planificacions per ignorar
334         (normalment ja està assignat als termòstats).
335
336     Retorna:
337         List[Dict[str, Any]]: una llista de diccionaris genèrics HVAC candidats.
338     """
339     excluded = {normalize_token(name) for name in excluded_schedule_names}
340     detected: Dict[Tuple[str, str], Dict[str, Any]] = {}
341
342     scheduled_setpoints = building_data.get("SetpointManager:Scheduled", {})
343     for _, manager_spec in scheduled_setpoints.items():
344         if not isinstance(manager_spec, dict):
345             continue
346
347         schedule_name = resolve_schedule_name(schedule_types, manager_spec.get("schedule_name", ""))
348         if schedule_name is None:
349             continue
350         if normalize_token(schedule_name) in excluded:
351             continue
352         if normalize_token(str(manager_spec.get("control_variable", ""))) != "temperature":
353             continue
354
355         # Aquests setpoints no pengen directament del termòstat, però també poden ser
356         # bons actuadors RL si EnergyPlus els controla amb una schedule editable.
357         key = ("scheduled_temperature_setpoint", schedule_name)
358         if key not in detected:
359             detected[key] = {
360                 "category": "scheduled_temperature_setpoint",
361                 "schedule_name": schedule_name,
362                 "zones": set(),
363                 "references": set(),
364                 "source_kind": "setpoint_manager",
365                 "element_type": schedule_types[schedule_name],
366             }
367
368         setpoint_target = manager_spec.get("setpoint_node_or_nodelist_name")
369         if isinstance(setpoint_target, str) and setpoint_target:
370             detected[key]["references"].add(setpoint_target)
371
372     scheduled_availability = building_data.get("AvailabilityManager:Scheduled", {})
373     for manager_name, manager_spec in scheduled_availability.items():
374         if not isinstance(manager_spec, dict):
375             continue
376
377         schedule_name = resolve_schedule_name(schedule_types, manager_spec.get("schedule_name", ""))
378         if schedule_name is None:
379             continue
380         if normalize_token(schedule_name) in excluded:
381             continue
382
383         key = ("availability", schedule_name)
384         if key not in detected:
385             detected[key] = {
386                 "category": "availability",
387                 "schedule_name": schedule_name,
388                 "zones": set(),
389                 "references": set(),
390                 "source_kind": "availability_manager",
391                 "element_type": schedule_types[schedule_name],
392             }
393         if manager_name:
394             detected[key]["references"].add(manager_name)
395
396     ordered_candidates: List[Dict[str, Any]] = []
397     for key in sorted(detected.keys(), key=lambda item: (item[0], item[1].lower())):
398         candidate = detected[key]
399         ordered_candidates.append(

```

```

400         {
401             "schedule_name": candidate["schedule_name"],
402             "category": candidate["category"],
403             "zones": tuple(sorted(candidate["zones"])),
404             "references": tuple(sorted(candidate["references"])),
405             "source_kind": candidate["source_kind"],
406             "element_type": candidate["element_type"],
407         }
408     )
409
410     return ordered_candidates
411
412
413 def extract_storage_controller_candidates(
414     building_data: Dict[str, Any],
415     schedule_types: Dict[str, str],
416     excluded_schedule_names: Sequence[str],
417 ) -> List[Dict[str, Any]]:
418     """Identifica els horaris de càrrega/descàrrega de la bateria que es poden exposar com a accions."""
419
420     excluded = {normalize_token(name) for name in excluded_schedule_names}
421     detected: Dict[Tuple[str, str], Dict[str, Any]] = {}
422
423     controller_fields = (
424         (
425             "storage_charge_power_fraction_schedule_name",
426             "battery_charge",
427         ),
428         (
429             "storage_discharge_power_fraction_schedule_name",
430             "battery_discharge",
431         ),
432     )
433
434     for distribution_name, distribution_spec in building_data.get("ElectricLoadCenter:Distribution", {}).items():
435         if not isinstance(distribution_spec, dict):
436             continue
437
438         # La distribució pot apuntar a un objecte Storage; guardem tots dos noms perquè
439         # després la UI pugui explicar d'on surt cada actuator de bateria.
440         storage_name = distribution_spec.get("electrical_storage_object_name")
441         reference = distribution_name
442         if storage_name:
443             reference = f"{distribution_name} -> {storage_name}"
444
445         for field_name, category in controller_fields:
446             # EnergyPlus separa càrrega i descàrrega en camps diferents; els exposem com
447             # actuadors diferents per no barrejar dues decisions físiques oposades.
448             schedule_name = resolve_schedule_name(schedule_types, distribution_spec.get(field_name, ""))
449             if schedule_name is None:
450                 continue
451             if normalize_token(schedule_name) in excluded:
452                 continue
453
454             key = (category, schedule_name)
455             if key not in detected:
456                 detected[key] = {
457                     "category": category,
458                     "schedule_name": schedule_name,
459                     "zones": set(),
460                     "references": set(),
461                     "source_kind": "electric_storage",
462                     "element_type": schedule_types[schedule_name],
463                 }
464             detected[key]["references"].add(reference)
465
466     ordered_candidates: List[Dict[str, Any]] = []
467     for key in sorted(detected.keys(), key=lambda item: (item[0], item[1].lower())):
468         candidate = detected[key]
469         ordered_candidates.append(
470             {
471                 "schedule_name": candidate["schedule_name"],
472                 "category": candidate["category"],
473                 "zones": tuple(sorted(candidate["zones"])),
474                 "references": tuple(sorted(candidate["references"])),
475                 "source_kind": candidate["source_kind"],
476                 "element_type": candidate["element_type"],
477             }
478         )
479
480     return ordered_candidates
481
482
483 def parse_available_handlers(available_data: str) -> Tuple[List[Tuple[str, str, str]], List[Tuple[str, str]], List[str]]:
484     """Analitzar els controladors disponibles."""
485     actuators: List[Tuple[str, str, str]] = []
486     variables: List[Tuple[str, str]] = []
487     meters: List[str] = []
488
489     reader = csv.reader(io.StringIO(available_data or ""))
490     for row in reader:
491         cleaned = [cell.strip().strip(";") for cell in row if cell is not None]

```

```

492     if not cleaned:
493         continue
494     entry_type = cleaned[0]
495     if entry_type == "Actuator" and len(cleaned) >= 4:
496         actuators.append((cleaned[1], cleaned[2], cleaned[3]))
497     elif entry_type == "OutputVariable" and len(cleaned) >= 3:
498         variables.append((cleaned[1], cleaned[2]))
499     elif entry_type == "OutputMeter" and len(cleaned) >= 2:
500         meters.append(cleaned[1])
501
502     return actuators, variables, meters
503
504
505 def probe_available_handlers(building_path: Path, weather_path: Path) -> Tuple[bool, str, Optional[str]]:
506     """Inicia un bucle mínim Sinergym per extreure exactament quins controladors EnergyPlus estan disponibles.
507
508     Paràmetres:
509         building_path (Path): camí cap al model d'edifici IDF o epJSON.
510         weather_path (Path): camí cap al fitxer meteorològic EPW.
511
512     Retorna:
513         Tuple[bool, str, Optional[str]]: una tupla que conté un booleà d'èxit, un text csv de controladors sense processar i una
514         cadena d'error si escau.
515     """
516     env: Optional[EplusEnv] = None
517     try:
518         env = EplusEnv(
519             building_file=str(building_path),
520             weather_files=str(weather_path),
521             action_space=gym.spaces.Box(low=0, high=0, shape=(0,), dtype=np.float32),
522             time_variables=DEFAULT_TIME_VARIABLES,
523             variables={},
524             meters={},
525             actuators={},
526             context={},
527             reward=ProbeReward,
528             reward_kwargs={},
529             env_name=f"probe-{sanitize_identifider(building_path.stem, 'env')}",
530             building_config={
531                 "runperiod": (1, 1, 1991, 1, 1, 1991),
532                 "timesteps_per_hour": 1,
533             },
534         )
535         env.reset()
536         return True, env.available_handlers or "", None
537     except Exception as error:
538         return False, "", str(error)
539     finally:
540         if env is not None:
541             try:
542                 env.close()
543             except Exception:
544                 pass
545
546 def find_available_output_variable(
547     available_variables: Sequence[Tuple[str, str]],
548     variable_name: str,
549     variable_key: str,
550 ) -> Optional[Tuple[str, str]]:
551     """Troba la variable de sortida disponible."""
552     target_name = normalize_token(variable_name)
553     target_key = normalize_token(variable_key)
554     for actual_name, actual_key in available_variables:
555         if normalize_token(actual_name) == target_name and normalize_token(actual_key) == target_key:
556             return actual_name, actual_key
557     return None
558
559
560 def find_available_meter(
561     available_meters: Sequence[str],
562     meter_name: str,
563 ) -> Optional[str]:
564     """Troba el comptador disponible."""
565     target_name = normalize_token(meter_name)
566     for actual_name in available_meters:
567         if normalize_token(actual_name) == target_name:
568             return actual_name
569     return None
570
571
572 def default_bounds_for_category(
573     category: str,
574     actuator_key: str,
575     current_low: Optional[float],
576     current_high: Optional[float],
577 ) -> Tuple[float, float]:
578     """Limits per defecte per a la categoria."""
579     if category == "heating_setpoint":
580         return 15.0, 22.5
581     if category == "cooling_setpoint":
582         return 22.5, 30.0

```

```

583 if category == "availability":
584     return 0.0, 1.0
585 if category in {"battery_charge", "battery_discharge"}:
586     return 0.0, 1.0
587 if category == "scheduled_temperature_setpoint":
588     if current_low is not None and current_high is not None:
589         if abs(current_high - current_low) < 1e-6:
590             margin = max(1.0, abs(current_low) * 0.05)
591             return float(current_low - margin), float(current_high + margin)
592         margin = max(0.5, (current_high - current_low) * 0.1)
593         return float(current_low - margin), float(current_high + margin)
594     return 15.0, 80.0
595 return 0.0, 1.0
596
597
598 def build_actuator_label(category: str) -> str:
599     """Crea una etiqueta de l'actuador per al flux Studio."""
600     if category == "heating_setpoint":
601         return "Setpoint de calefacció"
602     if category == "cooling_setpoint":
603         return "Setpoint de refrigeració"
604     if category == "availability":
605         return "Disponibilitat HVAC"
606     if category == "scheduled_temperature_setpoint":
607         return "Setpoint de sistema HVAC"
608     if category == "battery_charge":
609         return "Càrrega de bateria"
610     if category == "battery_discharge":
611         return "Descàrrega de bateria"
612     return "Control HVAC"
613
614
615 def _build_actuator_option(
616     candidate: Dict[str, Any],
617     building_data: Dict[str, Any],
618     schedule_types: Dict[str, str],
619 ) -> ActuatorOption:
620     """Construeix un ActuatorOption escrit a partir d'un diccionari candidat en brut.
621
622     Extreu l'interval de valors de la planificació de les dades de l'edifici i calcula sensiblement
623     límits d'acció per defecte basats en la categoria de l'actuador.
624
625     Paràmetres:
626         candidate (Dict[str, Any]): Dicte de candidat en brut que conté schedule_name, categoria,
627         zones, referències, camps element_type i source_kind.
628         building_data (Dict[str, Any]): dades completes de l'edifici epJSON per a l'extracció de l'interval de planificació.
629         schedule_types (Dict[str, str]): Assignació de noms de planificació a tipus de blocs de planificació.
630
631     Retorna:
632         ActuatorOption: una opció d'actuador estructurada i immutable preparada per a la representació de UI i
633         configuració de l'entorn.
634     """
635     schedule_name = candidate["schedule_name"]
636     category = candidate["category"]
637     zones = tuple(candidate["zones"])
638     references = tuple(candidate["references"])
639     current_low, current_high = extract_schedule_value_range(
640         building_data,
641         schedule_types,
642         schedule_name,
643     )
644     default_low, default_high = default_bounds_for_category(
645         category,
646         schedule_name,
647         current_low,
648         current_high,
649     )
650     if category in {"heating_setpoint", "cooling_setpoint"}:
651         reference = summarize_references("Zones", zones)
652     elif category == "scheduled_temperature_setpoint":
653         reference = summarize_references("Nodes", references)
654     elif category in {"battery_charge", "battery_discharge"}:
655         reference = summarize_references("Bateria", references)
656     else:
657         reference = summarize_references("Sistemes", references)
658
659     return ActuatorOption(
660         option_id=f'{candidate["element_type"]} |Schedule Value|{schedule_name}',
661         actuator_key=schedule_name,
662         element_type=candidate["element_type"],
663         value_type="Schedule Value",
664         label=build_actuator_label(category),
665         reference=reference,
666         category=category,
667         zones=zones,
668         current_low=current_low,
669         current_high=current_high,
670         default_low=default_low,
671         default_high=default_high,
672     )
673
674

```

```

675 def build_training_analysis(
676     building_path: Path,
677     weather_path: Path,
678     probe_handlers: bool = False,
679 ) -> BuildingTrainingAnalysis:
680     """Realitza una anàlisi completa d'un model d'edifici per extreure controladors HVAC entrenables.
681
682     Paràmetres:
683         building_path (Path): camí cap al model d'edifici.
684         weather_path (Path): Camí al fitxer meteorològic per resoldre les restriccions de l'entorn.
685         probe_handlers (bool, optional): Si True, un entorn de sonda Sinergym s'executa exactament
686             determinar els components de sortida. El valor predeterminat és False.
687
688     Retorna:
689         BuildingTrainingAnalysis: una classe de dades estructurada que conté resultats de detecció per
690             termòstats, variables, comptadors i actuadors.
691     """
692     building_data = load_building_json(building_path)
693     schedule_types = extract_schedule_type_index(building_data)
694     thermostat_zones, thermostat_candidates = extract_thermostat_schedule_candidates(
695         building_data,
696         schedule_types,
697     )
698     thermostat_schedule_names = [
699         candidate["schedule_name"]
700         for candidate in thermostat_candidates
701     ]
702     hvac_schedule_candidates = extract_scheduled_hvac_controller_candidates(
703         building_data,
704         schedule_types,
705         thermostat_schedule_names,
706     )
707     storage_schedule_candidates = extract_storage_controller_candidates(
708         building_data,
709         schedule_types,
710         thermostat_schedule_names,
711     )
712
713     if probe_handlers:
714         probe_success, available_raw, probe_error = probe_available_handlers(building_path, weather_path)
715     else:
716         probe_success, available_raw, probe_error = False, "", None
717     _, available_output_variables, available_meters = parse_available_handlers(available_raw)
718
719     thermostat_actuators: List[ActuatorOption] = []
720     used_schedule_keys: set[str] = set()
721     for candidate in thermostat_candidates:
722         option = _build_actuator_option(candidate, building_data, schedule_types)
723         thermostat_actuators.append(option)
724         used_schedule_keys.add(normalize_token(option.actuator_key))
725
726     hvac_actuators: List[ActuatorOption] = []
727     for candidate in [*hvac_schedule_candidates, *storage_schedule_candidates]:
728         normalized_key = normalize_token(candidate["schedule_name"])
729         if normalized_key in used_schedule_keys:
730             continue
731         hvac_actuators.append(_build_actuator_option(candidate, building_data, schedule_types))
732         used_schedule_keys.add(normalized_key)
733
734     return BuildingTrainingAnalysis(
735         thermostat_zones=tuple(thermostat_zones),
736         thermostat_actuators=tuple(thermostat_actuators),
737         hvac_actuators=tuple(hvac_actuators),
738         available_output_variables=tuple(available_output_variables),
739         available_meters=tuple(available_meters),
740         probe_success=probe_success,
741         probe_error=probe_error,
742     )

```

E.3 backend/afegir_entorn_backend.py

Listing 24: backend/afegir_entorn_backend.py

```

1  """Façana de compatibilitat per al backend afegir entorns.
2
3  La implementació es divideix en mòduls centrats, però aquest fitxer manté el fitxer
4  públic API utilitzat per la pàgina estable Streamlit.
5  """
6
7  from __future__ import annotations
8
9  from backend.afegir_entorn_analysis import (
10     ProbeReward,
11     _build_actuator_option,
12     _collect_schedule_references_from_value,
13     _register_thermostat_candidate,
14     build_actuator_label,
15     build_training_analysis,

```

```

16     default_bounds_for_category,
17     extract_schedule_numeric_values,
18     extract_schedule_type_index,
19     extract_schedule_value_range,
20     extract_scheduled_hvac_controller_candidates,
21     extract_storage_controller_candidates,
22     extract_thermostat_schedule_candidates,
23     find_available_meter,
24     find_available_output_variable,
25     load_building_json,
26     parse_available_handlers,
27     probe_available_handlers,
28     resolve_schedule_name,
29     summarize_references,
30 )
31 from backend.afegir_entorn_config import (
32     add_variable_spec,
33     build_action_space_expression,
34     build_environment_config,
35     build_registered_env_id,
36     build_training_observation_config,
37     build_variable_name_for_actuator,
38     build_variable_output_names,
39     cleanup_failed_environment,
40     register_environment_from_yaml,
41     validate_registered_environment,
42     write_yaml_config,
43 )
44 from backend.afegir_entorn_constants import (
45     DEFAULT_TIME_VARIABLES,
46     DEFAULT_WEATHER_VARIABILITY,
47     DETAILED_HVAC_METER_CANDIDATES,
48     ENERGY_METER_CANDIDATES,
49     ENERGY_VARIABLE_CANDIDATES,
50     ENERGYPLUS_SUBPROCESS_TIMEOUT_SECONDS,
51     ENV_VALIDATION_TIMEOUT_SECONDS,
52     FALLBACK_UPDATER_DIR,
53     STANDARD_WEATHER_VARIABLES,
54     STANDARD_ZONE_VARIABLES,
55     TARGET_EPLUS_VERSION,
56     TRANSITION_DOWNLOAD_URL,
57 )
58 from backend.afegir_entorn_conversion import (
59     convert_idf_to_epjson,
60     detect_idf_version,
61     download_transition_tools,
62     get_transition_updater_dir,
63     needs_idf_upgrade,
64     upgrade_idf_version,
65 )
66 from backend.afegir_entorn_types import (
67     ActuatorOption,
68     BuildingTrainingAnalysis,
69     EnvironmentValidationResult,
70     UpgradeResult,
71 )
72 from backend.afegir_entorn_utils import (
73     normalize_token,
74     sanitize_identifier,
75     save_uploaded_bytes,
76     unique_identifier,
77 )
78 from backend.paths import BUILDINGS_DIR, CFG_DIR, PKG_DIR, WEATHERS_DIR
79 from backend.weather_profiles import (
80     WeatherProfileSuggestion,
81     build_weather_profile_suggestions,
82     build_weather_temperature_preview,
83     build_weather_temperature_preview_figure,
84     load_epw_dry_bulb_series,
85     summarize_weather_file,
86 )
87
88 CFG_DIR.mkdir(parents=True, exist_ok=True)
89
90 __all__ = [
91     "ActuatorOption",
92     "BUILDINGS_DIR",
93     "BuildingTrainingAnalysis",
94     "CFG_DIR",
95     "DEFAULT_TIME_VARIABLES",
96     "DEFAULT_WEATHER_VARIABILITY",
97     "DETAILED_HVAC_METER_CANDIDATES",
98     "ENERGYPLUS_SUBPROCESS_TIMEOUT_SECONDS",
99     "ENERGY_METER_CANDIDATES",
100     "ENERGY_VARIABLE_CANDIDATES",
101     "ENV_VALIDATION_TIMEOUT_SECONDS",
102     "EnvironmentValidationResult",
103     "FALLBACK_UPDATER_DIR",
104     "PKG_DIR",
105     "ProbeReward",
106     "STANDARD_WEATHER_VARIABLES",
107     "STANDARD_ZONE_VARIABLES",

```



```

108 "TARGET_EPLUS_VERSION",
109 "TRANSITION_DOWNLOAD_URL",
110 "UpgradeResult",
111 "WEATHERS_DIR",
112 "WeatherProfileSuggestion",
113 "add_variable_spec",
114 "build_action_space_expression",
115 "build_actuator_label",
116 "build_environment_config",
117 "build_registered_env_id",
118 "build_training_analysis",
119 "build_training_observation_config",
120 "build_variable_name_for_actuator",
121 "build_variable_output_names",
122 "build_weather_profile_suggestions",
123 "build_weather_temperature_preview",
124 "build_weather_temperature_preview_figure",
125 "cleanup_failed_environment",
126 "convert_idf_to_epjson",
127 "default_bounds_for_category",
128 "detect_idf_version",
129 "download_transition_tools",
130 "extract_schedule_numeric_values",
131 "extract_schedule_type_index",
132 "extract_schedule_value_range",
133 "extract_scheduled_hvac_controller_candidates",
134 "extract_storage_controller_candidates",
135 "extract_thermostat_schedule_candidates",
136 "find_available_meter",
137 "find_available_output_variable",
138 "get_transition_updater_dir",
139 "load_building_json",
140 "load_epw_dry_bulb_series",
141 "needs_idf_upgrade",
142 "normalize_token",
143 "parse_available_handlers",
144 "probe_available_handlers",
145 "register_environment_from_yaml",
146 "resolve_schedule_name",
147 "sanitize_identifier",
148 "save_uploaded_bytes",
149 "summarize_references",
150 "summarize_weather_file",
151 "unique_identifier",
152 "upgrade_idf_version",
153 "validate_registered_environment",
154 "write_yaml_config",
155 ]

```

E.4 backend/afegir_entorn_config.py

Listing 25: backend/afegir_entorn_config.py

```

1  """Crea i valideu les configuracions de Sinergym des de l'entrada del formulari Studio.
2
3  Aquest mòdul és l'últim pas del flux Afegeix un entorn. Rep el
4  anàlisi d'edificis, variables seleccionades, comptadors, actuadors i paràmetres de recompensa,
5  després escriu una configuració YAML que es pot registrar com a Gymnasium
6  entorn.
7  """
8
9  from __future__ import annotations
10
11  import json
12  import re
13  import subprocess
14  import sys
15  from pathlib import Path
16  from typing import Any, Dict, List, Optional, Sequence, Tuple
17
18  import gymnasium as gym
19  import sinergym
20  import yaml
21  from sinergym.utils.common import convert_conf_to_env_parameters
22
23  from backend.afegir_entorn_analysis import find_available_meter, find_available_output_variable
24  from backend.afegir_entorn_constants import (
25      DEFAULT_TIME_VARIABLES,
26      DETAILED_HVAC_METER_CANDIDATES,
27      ENERGY_METER_CANDIDATES,
28      ENERGY_VARIABLE_CANDIDATES,
29      ENV_VALIDATION_TIMEOUT_SECONDS,
30      STANDARD_WEATHER_VARIABLES,
31      STANDARD_ZONE_VARIABLES,
32  )
33  from backend.afegir_entorn_types import (
34      ActuatorOption,
35      BuildingTrainingAnalysis,

```

```

36     EnvironmentValidationResult,
37 )
38 from backend.afegir_entorn_utils import sanitize_identifier, unique_identifier
39
40 def build_variable_output_names(base_name: str, keys: Sequence[str] | str) -> List[str]:
41     """Retorna àlies de variable de sortida estables per a una o més claus EnergyPlus."""
42     if isinstance(keys, str):
43         return [base_name]
44     return [f"{key.lower().replace(' ', '_')}-{base_name}" for key in keys]
45
46
47 def add_variable_spec(
48     variables: Dict[str, Dict[str, Any]],
49     available_output_variables: Sequence[Tuple[str, str]],
50     canonical_variable_name: str,
51     alias_name: str,
52     keys: Sequence[str] | str,
53 ) -> List[str]:
54     """Afegeix una especificació variable."""
55     requested_keys = [keys] if isinstance(keys, str) else list(keys)
56     matched_keys: List[str] = []
57     actual_variable_name = canonical_variable_name
58
59     for requested_key in requested_keys:
60         matched = find_available_output_variable(
61             available_output_variables,
62             canonical_variable_name,
63             requested_key,
64         )
65         if matched is None:
66             continue
67         actual_variable_name, actual_key = matched
68         matched_keys.append(actual_key)
69
70     if not matched_keys:
71         if isinstance(keys, str):
72             variables[canonical_variable_name] = {
73                 "variable_names": alias_name,
74                 "keys": keys,
75             }
76             return [alias_name]
77
78         variables[canonical_variable_name] = {
79             "variable_names": alias_name,
80             "keys": list(keys),
81         }
82         return build_variable_output_names(alias_name, list(keys))
83
84     output_keys: Sequence[str] | str
85     if len(matched_keys) == 1:
86         output_keys = matched_keys[0]
87     else:
88         output_keys = matched_keys
89
90     variables[actual_variable_name] = {
91         "variable_names": alias_name,
92         "keys": output_keys,
93     }
94     return build_variable_output_names(alias_name, output_keys)
95
96
97 def build_training_observation_config(
98     analysis: BuildingTrainingAnalysis,
99 ) -> Tuple[Dict[str, Dict[str, Any]], Dict[str, str], List[str], List[str]]:
100     """Crea la configuració d'observació d'entrenament per al flux Studio."""
101     variables: Dict[str, Dict[str, Any]] = {}
102     meters: Dict[str, str] = {}
103
104     # Primer afegim variables meteorològiques i de zona "normals"; són les que després
105     # espera la UI encara que el fitxer EnergyPlus tingui noms llargs o lleugerament diferents.
106     for variable_name, alias_name, variable_key in STANDARD_WEATHER_VARIABLES:
107         add_variable_spec(
108             variables,
109             analysis.available_output_variables,
110             variable_name,
111             alias_name,
112             variable_key,
113         )
114
115     temperature_variable_names: List[str] = []
116     for variable_name, alias_name in STANDARD_ZONE_VARIABLES:
117         generated_names = add_variable_spec(
118             variables,
119             analysis.available_output_variables,
120             variable_name,
121             alias_name,
122             analysis.thermostat_zones,
123         )
124         if alias_name == "air_temperature":
125             temperature_variable_names = generated_names
126
127     energy_variable_names: List[str] = []

```

```

128 for variable_name, alias_name, variable_key in ENERGY_VARIABLE_CANDIDATES:
129     matched = find_available_output_variable(
130         analysis.available_output_variables,
131         variable_name,
132         variable_key,
133     )
134     if matched is None and analysis.probe_success:
135         continue
136
137     add_variable_spec(
138         variables,
139         analysis.available_output_variables,
140         variable_name,
141         alias_name,
142         variable_key,
143     )
144     energy_variable_names = [alias_name]
145     break
146
147 if not energy_variable_names:
148     # Alguns models no publiquen energia HVAC com a output variable, però sí com a meter.
149     # En aquest cas fem servir el meter equivalent i mantenim el mateix alias intern.
150     for meter_name, alias_name in ENERGY_METER_CANDIDATES:
151         matched_meter = find_available_meter(analysis.available_meters, meter_name)
152         if matched_meter is None and analysis.probe_success:
153             continue
154         meters[matched_meter or meter_name] = alias_name
155         energy_variable_names = [alias_name]
156         break
157
158 for meter_name, alias_name in DETAILED_HVAC_METER_CANDIDATES:
159     # Els meters detallats no són obligatoris per entrenar, però si hi són donen
160     # gràfics molt més útils al dashboard.
161     matched_meter = find_available_meter(analysis.available_meters, meter_name)
162     if matched_meter is None:
163         continue
164     meters.setdefault(matched_meter, alias_name)
165
166 if not temperature_variable_names:
167     raise ValueError(
168         "No s'han pogut construir variables de temperatura de zona a partir dels termòstats detectats."
169     )
170 if not energy_variable_names:
171     raise ValueError(
172         "No s'ha detectat cap variable o meter energètic vàlid per construir la reward."
173     )
174
175 return variables, meters, temperature_variable_names, energy_variable_names
176
177
178 def build_variable_name_for_actuator(
179     option: ActuatorOption,
180     selected_options: Sequence[ActuatorOption],
181     used_names: set[str],
182 ) -> str:
183     """Crea el nom de la variable per a l'actuador per al flux Studio."""
184     heating_count = sum(item.category == "heating_setpoint" for item in selected_options)
185     cooling_count = sum(item.category == "cooling_setpoint" for item in selected_options)
186
187     # Si només hi ha una consigna de fred/calor mantenim noms simples. Amb més zones,
188     # el nom ha de portar zona o schedule per no crear variables duplicades.
189     if option.category == "heating_setpoint":
190         if heating_count == 1:
191             base_name = "Heating_Setpoint_RL"
192         elif len(option.zones) == 1:
193             base_name = f"heating_setpoint_{sanitize_identifer(option.zones[0], 'zone')}}"
194         else:
195             base_name = f"heating_setpoint_{sanitize_identifer(option.actuator_key, 'schedule')}}"
196     elif option.category == "cooling_setpoint":
197         if cooling_count == 1:
198             base_name = "Cooling_Setpoint_RL"
199         elif len(option.zones) == 1:
200             base_name = f"cooling_setpoint_{sanitize_identifer(option.zones[0], 'zone')}}"
201         else:
202             base_name = f"cooling_setpoint_{sanitize_identifer(option.actuator_key, 'schedule')}}"
203     elif option.category == "scheduled_temperature_setpoint":
204         base_name = f"hvac_setpoint_{sanitize_identifer(option.actuator_key, 'schedule')}}"
205     elif option.category == "availability":
206         base_name = f"hvac_availability_{sanitize_identifer(option.actuator_key, 'schedule')}}"
207     elif option.category == "battery_charge":
208         base_name = "Battery_Charge_Rate_RL"
209     elif option.category == "battery_discharge":
210         base_name = "Battery_Discharge_Rate_RL"
211     else:
212         base_name = f"hvac_control_{sanitize_identifer(option.actuator_key, 'actuator')}}"
213
214     return unique_identifer(base_name, used_names)
215
216
217 def build_action_space_expression(bounds: Sequence[Tuple[float, float]]) -> str:
218     """Crea una expressió d'espai d'acció per al flux Studio."""
219     lows = ", ".join(f"{low:.4f}" for low, _ in bounds)

```

```

220 highs = ", ".join(f"{high:.4f}" for _, high in bounds)
221 return (
222     "gym.spaces.Box("
223     f"low=np.array([{{lows}}], dtype=np.float32), "
224     f"high=np.array([{{highs}}], dtype=np.float32), "
225     f"shape={{len(bounds)}}, dtype=np.float32)"
226 )
227
228
229 def build_environment_config(
230     id_base: str,
231     building_file_name: str,
232     weather_profiles: Sequence[Dict[str, str]],
233     analysis: BuildingTrainingAnalysis,
234     selected_actuators: Sequence[ActuatorOption],
235     actuator_bounds: Dict[str, Tuple[float, float]],
236     weather_variability: Optional[Dict[str, Sequence[float]]] = None,
237 ) -> Dict[str, Any]:
238     """Construeix el diccionari de configuració de l'entorn arrel Sinergym YAML.
239
240     Paràmetres:
241         id_base (str): nom base per al registre de l'entorn.
242         building_file_name (str): el nom del fitxer d'estructura.
243         weather_profiles (Sequence[Dict[str, str]]): Llista de mapes meteorològics (fitxa i claus).
244         analysis (BuildingTrainingAnalysis): propietats detectades.
245         selected_actuators (Sequence[ActuatorOption]): actuadors seleccionats del UI.
246         actuator_bounds (Dict[str, Tuple[float, float]]): els límits continus de cada actuator.
247         weather_variability (Optional[Dict[str, Sequence[float]]]): Diccionari de variabilitat opcional.
248
249     Retorna:
250         Dict[str, Any]: La configuració totalment estructurada està preparada per escriure com a YAML.
251     """
252     variables, meters, temperature_variable_names, energy_variable_names = (
253         build_training_observation_config(analysis)
254     )
255
256     # Ordenem actuadors perquè el Box tingui un ordre estable: calor, fred, bateria i resta.
257     # Sense això, dos registres del mateix edifici podrien generar espais d'acció diferents.
258     ordered_actuators = sorted(
259         selected_actuators,
260         key=lambda option: (
261             {
262                 "heating_setpoint": 0,
263                 "cooling_setpoint": 1,
264                 "battery_charge": 2,
265                 "battery_discharge": 3,
266             }.get(option.category, 4),
267             option.label.lower(),
268         ),
269     )
270
271     used_variable_names: set[str] = set()
272     actuators: Dict[str, Dict[str, str]] = {}
273     bounds: List[Tuple[float, float]] = []
274     for option in ordered_actuators:
275         # actuator_key apunta a l'objecte EnergyPlus real; variable_name és el nom curt que
276         # veurà Sinergym i després el model RL.
277         low, high = actuator_bounds[option.option_id]
278         variable_name = build_variable_name_for_actuator(option, ordered_actuators, used_variable_names)
279         actuators[option.actuator_key] = {
280             "variable_name": variable_name,
281             "element_type": option.element_type,
282             "value_type": option.value_type,
283         }
284         bounds.append((low, high))
285
286     env_cfg: Dict[str, Any] = {
287         "id_base": id_base,
288         "building_file": building_file_name,
289         "weather_specification": {
290             "weather_files": [profile["weather_file"] for profile in weather_profiles],
291             "keys": [profile["key"] for profile in weather_profiles],
292         },
293         "building_config": None,
294         "max_ep_store": 3,
295         "time_variables": list(DEFAULT_TIME_VARIABLES),
296         "variables": variables,
297         "meters": meters,
298         "actuators": actuators,
299         "context": {},
300         "action_space": build_action_space_expression(bounds),
301         "reward": "sinergym.utils.rewards.LinearReward",
302         "reward_kwargs": {
303             "temperature_variables": temperature_variable_names,
304             "energy_variables": energy_variable_names,
305             "range_comfort_winter": [20.0, 23.5],
306             "range_comfort_summer": [23.0, 26.0],
307             "summer_start": [6, 1],
308             "summer_final": [9, 30],
309             "energy_weight": 0.5,
310             "lambda_energy": 1e-4,
311             "lambda_temperature": 1.0,

```

```

312     },
313 }
314
315 if weather_variability:
316     # Només escrivim weather_variability quan l'usuari l'ha configurat; deixar-ho buit
317     # pot activar comportaments diferents en versions de Sinergym.
318     env_cfg["weather_variability"] = weather_variability
319
320 return env_cfg
321
322
323 def write_yaml_config(cfg_path: Path, env_cfg: Dict[str, Any]) -> Path:
324     """Escriu el diccionari de configuració de l'entorn generat en un fitxer YAML.
325
326     Paràmetres:
327         cfg_path (Path): Camí al fitxer de sortida YAML.
328         env_cfg (Dict[str, Any]): dades de configuració del diccionari.
329
330     Retorna:
331         Path: la ruta del fitxer de sortida confirmada.
332     """
333     with open(cfg_path, "w", encoding="utf-8") as file_handle:
334         yaml.dump(env_cfg, file_handle, sort_keys=False, allow_unicode=True)
335     return cfg_path
336
337
338 def register_environment_from_yaml(cfg_path: Path) -> None:
339     """Registra o torna a registrar els entorns des d'una configuració Sinergym YAML mitjançant Gym.
340
341     Paràmetres:
342         cfg_path (Path): camí cap a la configuració vàlida Sinergym YAML.
343     """
344     with open(cfg_path, "r", encoding="utf-8") as yaml_conf:
345         conf = yaml.safe_load(yaml_conf)
346
347     configurations = convert_conf_to_env_parameters(conf)
348     registry = gym.envs.registration.registry
349     for env_id in configurations.keys():
350         registry.pop(env_id, None)
351         registry.pop(env_id.replace("continuous", "discrete"), None)
352
353     sinergym.register_envs_from_yaml(str(cfg_path))
354
355
356 def build_registered_env_id(id_base: str, weather_key: str, stochastic: bool = False) -> str:
357     """Crea l'ID de l'entorn del Gym canònic basat en les convencions de denominació Sinergym.
358
359     Paràmetres:
360         id_base (str): Nom base donat a l'element d'edifici.
361         weather_key (str): descriptor breu de la cadena meteorològica.
362         stochastic (bool, optional): si la variant pretén incloure variabilitat.
363
364     Retorna:
365         str: identificador d'entorn OpenAI Gym vàlid.
366     """
367     suffix = "-continuous-stochastic-v1" if stochastic else "-continuous-v1"
368     return f"Eplus-{id_base}-{weather_key}{suffix}"
369
370
371 def validate_registered_environment(env_id: str, cfg_path: Optional[Path] = None) -> EnvironmentValidationResult:
372     """Executa un subprocés aïllat per activar i validar que l'entorn es registra i passa correctament.
373
374     Paràmetres:
375         env_id (str): cadena d'ID d'entorn Gym.
376         cfg_path (Optional[Path], optional): camí cap a yaml si és necessari per al registre diferit.
377
378     Retorna:
379         EnvironmentValidationResult: un dictat de verificació estructural que indica la matriu d'observació i els espais.
380     """
381     validation_script = """
382 import json
383 import sys
384
385 import gymnasium as gym
386 import sinergym
387
388 env_id = sys.argv[1]
389 cfg_path = sys.argv[2] if len(sys.argv) > 2 else None
390 if cfg_path:
391     sinergym.register_envs_from_yaml(cfg_path)
392
393 env = gym.make(env_id)
394 try:
395     observation = env.reset()[0]
396     if hasattr(observation, "tolist"):
397         observation = observation.tolist()
398     payload = {
399         "observation_space": repr(env.observation_space),
400         "action_space": repr(env.action_space),
401         "initial_observation": observation,
402     }
403     print(f"---JSON_START---\\n{json.dumps(payload)}\\n---JSON_END---")

```

```

404 finally:
405     env.close()
406     """
407
408     try:
409         result = subprocess.run(
410             [sys.executable, "-m", env_id, *( [str(cfg_path)] if cfg_path else [] )],
411             input=validation_script,
412             check=True,
413             capture_output=True,
414             text=True,
415             timeout=ENV_VALIDATION_TIMEOUT_SECONDS,
416         )
417         match = re.search(r"---JSON_START---\n(?:.*)\n---JSON_END---", result.stdout, re.DOTALL)
418         if not match:
419             raise RuntimeError(f"No s'ha trobat l'estructura de validació a l'output. Output obtingut:\n{result.stdout}\nErrors:\n{result.stderr}")
420
421         payload = json.loads(match.group(1))
422         return EnvironmentValidationResult(
423             observation_space=payload["observation_space"],
424             action_space=payload["action_space"],
425             initial_observation=payload["initial_observation"],
426         )
427     except subprocess.TimeoutExpired as error:
428         raise TimeoutError(
429             "La validació de l'entorn ha superat el temps limit. "
430             "Revisa la configuració del model o prova un edifici més lleuger."
431         ) from error
432     except subprocess.CalledProcessError as error:
433         stderr = (error.stderr or "").strip()
434         stdout = (error.stdout or "").strip()
435         raise RuntimeError(stderr or stdout or "La validació ha fallat en arrencar l'entorn.") from error
436
437 def cleanup_failed_environment(cfg_path: Path, tmp_path: Path) -> List[Tuple[str, str]]:
438     """Intenta eliminar les estructures de configuració no enllaçades o incorrectes com a resultat d'un registre fallit.
439
440     Paràmetres:
441         cfg_path (Path): Configuració interrompuda de YAML.
442         tmp_path (Path): fitxer epJSON intermedi generat.
443
444     Retorna:
445         List[Tuple[str, str]]: Tuples de missatgeria de nivell de diagnòstic que mapegen quins recursos es van revertir.
446     """
447     messages: List[Tuple[str, str]] = []
448     try:
449         if cfg_path.exists():
450             cfg_path.unlink()
451             messages.append(("warning", "Fitxer de registre YAML revertit.))
452         if tmp_path.suffix.lower() == ".idf":
453             epjson_path = tmp_path.with_suffix(".epJSON")
454             if epjson_path.exists():
455                 epjson_path.unlink()
456                 messages.append(("warning", "Fitxer estructural epJSON revertit.))
457     except Exception as cleanup_error:
458         messages.append(("error", f"Error eliminant fitxers: {cleanup_error}"))
459     return messages
460

```

E.5 backend/afegir_entorn_constants.py

Listing 26: backend/afegir_entorn_constants.py

```

1  """Constants i valors per defecte del flux per afegir entorns."""
2
3  from __future__ import annotations
4
5  from pathlib import Path
6
7  FALLBACK_UPDATER_DIR = Path("/workspaces/sinergym/IDFVersionUpdater")
8  TRANSITION_DOWNLOAD_URL = (
9      "https://github.com/NREL/EnergyPlus/releases/download/v24.1.0/"
10     "EnergyPlus-24.1.0-9d7789a3ac-Linux-Ubuntu22.04-x86_64.tar.gz"
11 )
12 TARGET_EPLUS_VERSION = (24, 1)
13 ENERGYPLUS_SUBPROCESS_TIMEOUT_SECONDS = 300
14 ENV_VALIDATION_TIMEOUT_SECONDS = 180
15
16 DEFAULT_TIME_VARIABLES = ["month", "day_of_month", "hour"]
17 DEFAULT_WEATHER_VARIABILITY = {
18     "Dry Bulb Temperature": [1.0, 0.0, 24.0],
19 }
20 STANDARD_WEATHER_VARIABLES = (
21     ("Site Outdoor Air DryBulb Temperature", "outdoor_temperature", "Environment"),
22     ("Site Outdoor Air Relative Humidity", "outdoor_humidity", "Environment"),
23     ("Site Wind Speed", "wind_speed", "Environment"),
24     ("Site Wind Direction", "wind_direction", "Environment"),
25     ("Site Diffuse Solar Radiation Rate per Area", "diffuse_solar_radiation", "Environment"),

```

```

26     ("Site Direct Solar Radiation Rate per Area", "direct_solar_radiation", "Environment"),
27 )
28 STANDARD_ZONE_VARIABLES = (
29     ("Zone Thermostat Heating Setpoint Temperature", "htg_setpoint"),
30     ("Zone Thermostat Cooling Setpoint Temperature", "clg_setpoint"),
31     ("Zone Air Temperature", "air_temperature"),
32     ("Zone Air Relative Humidity", "air_humidity"),
33     ("Zone People Occupant Count", "people_occupant"),
34 )
35 ENERGY_VARIABLE_CANDIDATES = (
36     ("Facility Total HVAC Electricity Demand Rate", "HVAC_electricity_demand_rate", "Whole Building"),
37     ("Facility Total Electricity Demand Rate", "facility_electricity_demand_rate", "Whole Building"),
38 )
39 ENERGY_METER_CANDIDATES = (
40     ("Electricity:HVAC", "total_electricity_hvac"),
41     ("Electricity:Facility", "total_electricity_facility"),
42     ("Electricity:Building", "total_electricity_building"),
43 )
44 DETAILED_HVAC_METER_CANDIDATES = (
45     ("NaturalGas:HVAC", "natural_gas_hvac"),
46     ("Heating:Electricity", "heating_electricity"),
47     ("Cooling:Electricity", "cooling_electricity"),
48     ("Fans:Electricity", "fans_electricity"),
49     ("Pumps:Electricity", "pumps_electricity"),
50     ("HeatRejection:Electricity", "heat_rejection_electricity"),
51     ("Humidifier:Electricity", "humidifier_electricity"),
52     ("HeatRecovery:Electricity", "heat_recovery_electricity"),
53 )

```

E.6 backend/afegir_entorn_conversion.py

Listing 27: backend/afegir_entorn_conversion.py

```

1  """Conversió i actualització d'IDF/epJSON dins del flux per afegir entorns."""
2
3  from __future__ import annotations
4
5  import os
6  import re
7  import shutil
8  import ssl
9  import subprocess
10 import tarfile
11 import urllib.request
12 from pathlib import Path
13 from typing import Optional, Tuple
14
15 from backend.afegir_entorn_constants import (
16     ENERGYPLUS_SUBPROCESS_TIMEOUT_SECONDS,
17     FALLBACK_UPDATER_DIR,
18     TARGET_EPLUS_VERSION,
19     TRANSITION_DOWNLOAD_URL,
20 )
21 from backend.afegir_entorn_types import UpgradeResult
22
23 def detect_idf_version(idf_path: Path) -> Optional[Tuple[int, int]]:
24     """Llegeix un fitxer IDF per extreure la seva versió de destinació EnergyPlus.
25
26     Paràmetres:
27         idf_path (Path): camí cap al fitxer IDF.
28
29     Retorna:
30         Optional[Tuple[int, int]]: una tupla de nombres enters majors i menors si es troba, sinó None.
31     """
32     with open(idf_path, "r", encoding="utf-8", errors="ignore") as file_handle:
33         content = file_handle.read()
34
35     match = re.search(r"Version,\s*([\d\.]+\s*)\s*;", content, re.IGNORECASE)
36     if not match:
37         return None
38
39     parts = match.group(1).strip().split(".")
40     if len(parts) < 2:
41         return None
42
43     return int(parts[0]), int(parts[1])
44
45
46 def needs_idf_upgrade(version: Tuple[int, int]) -> bool:
47     """Avalua si una versió IDF requereix l'actualització a la versió del simulador de destinació.
48
49     Paràmetres:
50         version (Tuple[int, int]): versió major i menor de IDF.
51
52     Retorna:
53         bool: True si la versió és estrictament més antiga que la `TARGET_EPLUS_VERSION`.
54     """
55     major, minor = version

```

```

56 target_major, target_minor = TARGET_EPLUS_VERSION
57 return not (major > target_major or (major == target_major and minor >= target_minor))
58
59
60 def get_transition_updater_dir() -> Optional[Path]:
61     """Recupera el directori local que conté EnergyPlus IDFVersionUpdater binaris.
62
63     Retorna:
64         Optional[Path]: camí cap a les eines d'actualització, o None si no es troba localment.
65     """
66     ep_base_dir = Path(os.environ.get("EPLUS_PATH", "/usr/local/EnergyPlus-24-1-0"))
67     updater_dir = ep_base_dir / "PreProcess" / "IDFVersionUpdater"
68     if updater_dir.exists():
69         return updater_dir
70     if FALLBACK_UPDATER_DIR.exists():
71         return FALLBACK_UPDATER_DIR
72     return None
73
74
75 def download_transition_tools() -> Path:
76     """Baixa i extreu EnergyPlus Eines de transició al directori alternatiu.
77
78     En lloc de descarregar tota la suite EnergyPlus, només extreu l'actualitzador PreProcess.
79
80     Retorna:
81         Path: el directori local que conté les eines de transició descarregades.
82     """
83     ssl._create_default_https_context = ssl._create_unverified_context
84     tar_path, _ = urllib.request.urlretrieve(TRANSITION_DOWNLOAD_URL, "/tmp/eplusr.tar.gz")
85     FALLBACK_UPDATER_DIR.mkdir(parents=True, exist_ok=True)
86     with tarfile.open(tar_path, "r:gz") as tar:
87         members = []
88         for member in tar.getmembers():
89             if "PreProcess/IDFVersionUpdater/" in member.name:
90                 member.name = member.name.split("PreProcess/IDFVersionUpdater/")[1]
91                 if member.name:
92                     members.append(member)
93         tar.extractall(path=FALLBACK_UPDATER_DIR, members=members)
94     Path("/tmp/eplusr.tar.gz").unlink(missing_ok=True)
95     return FALLBACK_UPDATER_DIR
96
97
98 def upgrade_idf_version(idf_path: Path, updater_dir: Path) -> UpgradeResult:
99     """Actualitza seqüencialment un fitxer IDF mitjançant cadenes de transició EnergyPlus.
100
101     Paràmetres:
102         idf_path (Path): camí cap al fitxer IDF de destinació.
103         updater_dir (Path): Directori que conté les eines de transició.
104
105     Retorna:
106         UpgradeResult: Estructura resumida de l'operació d'actualització.
107     """
108     initial_version = detect_idf_version(idf_path)
109     if initial_version is None or not needs_idf_upgrade(initial_version):
110         return UpgradeResult(upgraded=False, final_version=initial_version)
111
112     current_major, current_minor = initial_version
113     working_dir = Path("/tmp/eplusr_upgrade")
114     working_dir.mkdir(parents=True, exist_ok=True)
115     working_file = working_dir / idf_path.name
116     shutil.copy(idf_path, working_file)
117
118     loops = 0
119     failed_transition_version = None
120     while loops < 20:
121         prefix = f"Transition-V{current_major}-{current_minor}-0-to-V"
122         tool = next(
123             (
124                 file_path
125                 for file_path in updater_dir.glob("Transition-*")
126                 if file_path.name.startswith(prefix)
127             ),
128             None,
129         )
130
131         if not tool:
132             break
133
134         try:
135             subprocess.run(
136                 [str(tool), str(working_file)],
137                 check=True,
138                 cwd=str(updater_dir),
139                 capture_output=True,
140                 timeout=ENERGYPLUS_SUBPROCESS_TIMEOUT_SECONDS,
141             )
142         except subprocess.CalledProcessError:
143             failed_transition_version = (current_major, current_minor)
144             break
145
146     next_version = detect_idf_version(working_file)
147     if next_version is None:

```



```

148         break
149
150         current_major, current_minor = next_version
151         loops += 1
152
153     shutil.copy(working_file, idf_path)
154     final_version = detect_idf_version(idf_path)
155
156     upgraded = False
157     if initial_version is not None and final_version is not None:
158         upgraded = final_version[0] > initial_version[0] or final_version[1] > initial_version[1]
159
160     return UpgradeResult(
161         upgraded=upgraded,
162         final_version=final_version,
163         failed_transition_version=failed_transition_version,
164     )
165
166
167 def convert_idf_to_epjson(idf_path: Path) -> Path:
168     """Converteix un fitxer EnergyPlus IDF al format epJSON mitjançant les eines CLI.
169
170     Paràmetres:
171         idf_path (Path): camí cap al fitxer estructural IDF.
172
173     Retorna:
174         Path: el camí del fitxer epJSON convertit resultant.
175     """
176     epjson_path = idf_path.with_suffix(".epJSON")
177     subprocess.run(
178         [
179             "energyplus",
180             "--convert-only",
181             "--output-directory",
182             str(idf_path.parent),
183             str(idf_path),
184         ],
185         check=True,
186         timeout=ENERGYPLUS_SUBPROCESS_TIMEOUT_SECONDS,
187     )
188     return epjson_path

```

E.7 backend/afegir_entorn_types.py

Listing 28: backend/afegir_entorn_types.py

```

1  """Tipus de dades compartits pel flux per afegir entorns."""
2
3  from __future__ import annotations
4
5  from dataclasses import dataclass
6  from typing import Optional, Tuple
7
8  @dataclass(frozen=True)
9  class UpgradeResult:
10     """Contenidor de dades per a UpgradeResult."""
11     upgraded: bool
12     final_version: Optional[Tuple[int, int]]
13     failed_transition_version: Optional[Tuple[int, int]] = None
14
15
16  @dataclass(frozen=True)
17  class EnvironmentValidationResult:
18     """Contenidor de dades per a EnvironmentValidationResult."""
19     observation_space: object
20     action_space: object
21     initial_observation: object
22
23
24  @dataclass(frozen=True)
25  class ActuatorOption:
26     """Contenidor de dades per a ActuatorOption."""
27     option_id: str
28     actuator_key: str
29     element_type: str
30     value_type: str
31     label: str
32     reference: str
33     category: str
34     zones: Tuple[str, ...]
35     current_low: Optional[float]
36     current_high: Optional[float]
37     default_low: float
38     default_high: float
39
40
41  @dataclass(frozen=True)
42  class BuildingTrainingAnalysis:

```

```

43 """Contenedor de dades per a BuildingTrainingAnalysis."""
44 thermostat_zones: Tuple[str, ...]
45 thermostat_actuators: Tuple[ActuatorOption, ...]
46 hvac_actuators: Tuple[ActuatorOption, ...]
47 available_output_variables: Tuple[Tuple[str, str], ...]
48 available_meters: Tuple[str, ...]
49 probe_success: bool
50 probe_error: Optional[str] = None

```

E.8 backend/afegir_entorn_utils.py

Listing 29: backend/afegir_entorn_utils.py

```

1  """Funcions comunes per desar fitxers i normalitzar noms dins del flux per afegir entorns."""
2
3  from __future__ import annotations
4
5  import re
6  from pathlib import Path
7
8  def save_uploaded_bytes(target_path: Path, content: bytes) -> Path:
9      """Desa el contingut en bytes en brut a una ruta de fitxer especificada.
10
11      Paràmetres:
12          target_path (Path): camí del fitxer de destinació.
13          contingut (bytes): bytes en brut per escriure.
14
15      Retorna:
16          Path: S'ha passat pel mateix camí, per encadenar.
17      """
18      target_path.parent.mkdir(parents=True, exist_ok=True)
19      with open(target_path, "wb") as file_handle:
20          file_handle.write(content)
21      return target_path
22
23
24  def sanitize_identifider(raw_value: str, fallback: str) -> str:
25      """Neteja una cadena per utilitzar-la com a identificador segur, substituint els caràcters no alfanumèrics.
26
27      Paràmetres:
28          raw_value (str): la cadena inicial.
29          fallback (str): es retorna una cadena de retorn si la cadena resultant està buida.
30
31      Retorna:
32          str: una cadena alfanumèrica netejada amb guions baixos.
33      """
34      sanitized = re.sub(r"[^a-zA-Z0-9]+", "_", (raw_value or "").strip().lower()).strip("_")
35      return sanitized or fallback
36
37
38  def normalize_token(value: str) -> str:
39      """Normalitza una cadena per a la comparació utilitzant minúscules i estandarditzant l'espaiat.
40
41      Paràmetres:
42          value (str): el testimoni en brut a normalitzar.
43
44      Retorna:
45          str: un testimoni net i en minúscules dividit per espais individuals.
46      """
47      normalized = (value or "").lower()
48      normalized = normalized.replace("drybulb", "dry bulb")
49      normalized = re.sub(r"[^a-zA-Z0-9]+", " ", normalized)
50      return re.sub(r"\s+", " ", normalized).strip()
51
52
53  def unique_identifider(base: str, used: set[str]) -> str:
54      """Genera un identificador únic afegint un sufix incremental si cal.
55
56      Paràmetres:
57          base (str): la cadena base desitjada.
58          (set[str]) utilitzat: un conjunt d'identificadors ja pres.
59
60      Retorna:
61          str: un identificador únic que no existeix a `used`. L'identificador també s'afegeix al conjunt.
62      """
63      candidate = base
64      suffix = 2
65      while candidate in used:
66          candidate = f"{base}_{suffix}"
67          suffix += 1
68      used.add(candidate)
69      return candidate

```

E.9 backend/avaluar_agent_backend.py

Listing 30: backend/avaluar_agent_backend.py

```
1  """Temps d'execució d'avaluació asíncrona per a agents Studio entrenats.
2
3  L'avaluació es pot executar per a molts passos de simulació, de manera que aquest backend aïlla el
4  bucle d'execució, cua de progrés i estat de cancel·lació des de la pàgina Streamlit. Això
5  també centralitza la càrrega de models, la creació d'entorns i la validació de l'espai d'acció
6  de manera que el control i l'avaluació en directe segueixen el mateix contracte.
7  """
8
9  from __future__ import annotations
10
11  import json
12  import queue
13  import threading
14  import time
15  import traceback
16  from dataclasses import dataclass
17  from functools import partial
18  from pathlib import Path
19  from typing import Any, Callable, Dict, List, Optional, Tuple
20
21  import gymnasium as gym
22  import numpy as np
23  import streamlit as st
24  from stable_baselines3.common.monitor import Monitor
25  from stable_baselines3.common.vec_env import DummyVecEnv, VecEnvWrapper
26
27  import sinergym # noga: F401
28  from sinergym.utils.wrappers import CSVLogger, LoggerWrapper, NormalizeAction
29
30  from backend.paths import ONE_YEAR_STEPS
31  from backend.sb3_utils import (
32      action_spaces_compatible,
33      candidate_vecnorm,
34      describe_action_space,
35      env_id_from_meta_or_name,
36      format_action_for_env,
37      load_sb3_model_bytes,
38      load_vecnormalize,
39      scan_model_zips,
40      should_apply_normalize_action,
41  )
42
43  EVALUATION_RUNTIME_STATE_KEY = "evaluation_runtime_state"
44
45
46  @dataclass(frozen=True)
47  class EvaluationResult:
48      """Resum de resultats d'una execució d'un treballador d'avaluació."""
49      elapsed_seconds: float
50      cancelled: bool
51      warnings: Tuple[str, ...] = ()
52
53
54  def push_evaluation_progress(
55      event_queue: "queue.Queue[Dict[str, Any]]",
56      job_config: Dict[str, Any],
57      step_number: int,
58      total_steps: int,
59  ) -> None:
60      """Envieu un esdeveniment de progrés d'avaluació a la cua d'execució."""
61      push_evaluation_event(
62          event_queue,
63          {
64              "type": "progress",
65              "step_number": int(step_number or 0),
66              "total_steps": int(total_steps or job_config["steps_target"]),
67              "status": f"Passos: {int(step_number or 0)}/{int(total_steps or job_config['steps_target'])}",
68          },
69      )
70
71
72  def build_evaluation_env(
73      env_id: str,
74      gym_kwargs: Dict[str, Any],
75      model_action_space: Any,
76      wrapper_configs: Optional[List[Dict[str, Any]]],
77  ) -> Any:
78      """Crea un entorn embolicat per a una execució d'avaluació d'agent."""
79      env = make_env(env_id, seed=0, gym_kwargs=gym_kwargs)
80      env = apply_evaluation_action_contract(env, model_action_space, wrapper_configs)
81      env = LoggerWrapper(env)
82      env = CSVLogger(env)
83      env = Monitor(env)
84      return env
85
86
87
```

```

88 def empty_evaluation_runtime() -> Dict[str, Any]:
89     """Temps d'execució d'avaluació buit."""
90     return {
91         "running": False,
92         "completed": False,
93         "cancel_requested": False,
94         "needs_full_rerun": False,
95         "thread": None,
96         "queue": None,
97         "stop_event": None,
98         "result": None,
99         "error": None,
100         "traceback": None,
101         "model_path": None,
102         "env_id": None,
103         "steps_target": 1,
104         "latest_step": 0,
105         "status": "",
106         "started_at": None,
107     }
108
109
110 def ensure_evaluation_runtime() -> Dict[str, Any]:
111     """Assegura el temps d'execució de l'avaluació."""
112     if EVALUATION_RUNTIME_STATE_KEY not in st.session_state:
113         st.session_state[EVALUATION_RUNTIME_STATE_KEY] = empty_evaluation_runtime()
114     return st.session_state[EVALUATION_RUNTIME_STATE_KEY]
115
116
117 def reset_evaluation_runtime() -> Dict[str, Any]:
118     """Restableix el temps d'execució de l'avaluació."""
119     runtime = ensure_evaluation_runtime()
120     previous_thread = runtime.get("thread")
121     if previous_thread is not None and getattr(previous_thread, "is_alive", lambda: False)():
122         return runtime
123
124     st.session_state[EVALUATION_RUNTIME_STATE_KEY] = empty_evaluation_runtime()
125     return st.session_state[EVALUATION_RUNTIME_STATE_KEY]
126
127
128 def push_evaluation_event(event_queue: "queue.Queue[Dict[str, Any]]", payload: Dict[str, Any]) -> None:
129     """Esdeveniment d'avaluació push."""
130     event_queue.put(dict(payload))
131
132
133 def evaluation_worker(
134     job_config: Dict[str, Any],
135     event_queue: "queue.Queue[Dict[str, Any]]",
136     stop_event: threading.Event,
137 ) -> None:
138     """Executa l'avaluació en segon pla."""
139     try:
140         result = run_agent_evaluation(
141             model_path=job_config["model_path"],
142             env_id=job_config["env_id"],
143             use_vecnorm=bool(job_config["use_vecnorm"]),
144             vecnorm_path=job_config["vecnorm_path"],
145             steps_target=int(job_config["steps_target"]),
146             deterministic=bool(job_config["deterministic"]),
147             should_stop=stop_event.is_set,
148             on_progress=partial(push_evaluation_progress, event_queue, job_config),
149         )
150         push_evaluation_event(event_queue, {"type": "result", "result": result})
151     except Exception as exc:
152         push_evaluation_event(
153             event_queue,
154             {
155                 "type": "error",
156                 "error": str(exc),
157                 "traceback": traceback.format_exc(),
158             },
159         )
160     finally:
161         push_evaluation_event(event_queue, {"type": "finished"})
162
163
164 def drain_evaluation_runtime(runtime: Dict[str, Any]) -> None:
165     """Temps d'execució de l'avaluació del buidatge."""
166     event_queue = runtime.get("queue")
167     if event_queue is None:
168         return
169
170     # La UI consulta aquest runtime sovint; buidem la cua sencera en cada passada perquè
171     # no quedin esdeveniments antics fent saltar el progrés enrere.
172     while True:
173         try:
174             event = event_queue.get_nowait()
175         except queue.Empty:
176             break
177
178         event_type = str(event.get("type") or "")
179         if event_type == "progress":

```

```

180         runtime["latest_step"] = max(
181             int(runtime.get("latest_step") or 0),
182             int(event.get("step_number") or 0),
183         )
184         runtime["steps_target"] = max(
185             int(runtime.get("steps_target") or 1),
186             int(event.get("total_steps") or 1),
187         )
188         runtime["status"] = str(event.get("status") or runtime.get("status") or "")
189     elif event_type == "result":
190         runtime["result"] = event.get("result")
191         runtime["completed"] = True
192         runtime["running"] = False
193         runtime["needs_full_rerun"] = True
194     elif event_type == "error":
195         runtime["error"] = str(event.get("error") or "Error desconegut durant l'avaluació.")
196         runtime["traceback"] = event.get("traceback")
197         runtime["completed"] = True
198         runtime["running"] = False
199         runtime["needs_full_rerun"] = True
200     elif event_type == "finished":
201         runtime["completed"] = True
202         runtime["running"] = False
203         runtime["thread"] = None
204         runtime["needs_full_rerun"] = True
205
206     thread = runtime.get("thread")
207     if thread is not None and not thread.is_alive() and runtime.get("running"):
208         runtime["running"] = False
209         runtime["completed"] = True
210         runtime["thread"] = None
211         runtime["needs_full_rerun"] = True
212
213
214 def sync_evaluation_runtime() -> Dict[str, Any]:
215     """Sincronitza el temps d'execució de l'avaluació."""
216     runtime = ensure_evaluation_runtime()
217     drain_evaluation_runtime(runtime)
218     return runtime
219
220
221 def consume_evaluation_runtime_rerun_flag(runtime: Optional[Dict[str, Any]] = None) -> bool:
222     """Indicador de reexecució del clima d'execució de l'avaluació."""
223     active_runtime = runtime if runtime is not None else ensure_evaluation_runtime()
224     if active_runtime.get("needs_full_rerun") and not active_runtime.get("running"):
225         active_runtime["needs_full_rerun"] = False
226         return True
227     return False
228
229
230 def request_evaluation_stop(runtime: Optional[Dict[str, Any]] = None) -> bool:
231     """Sol·licitar l'aturada de l'avaluació."""
232     active_runtime = runtime if runtime is not None else ensure_evaluation_runtime()
233     if not active_runtime.get("running") or active_runtime.get("stop_event") is None:
234         return False
235
236     active_runtime["cancel_requested"] = True
237     active_runtime["status"] = "Aturant avaluació..."
238     active_runtime["stop_event"].set()
239     return True
240
241
242 def start_evaluation_run(request: Dict[str, Any]) -> Dict[str, Any]:
243     """Iniciar l'execució d'avaluació."""
244     runtime = reset_evaluation_runtime()
245     response: Dict[str, Any] = {
246         "started": False,
247         "errors": [],
248         "runtime": runtime,
249     }
250
251     if runtime.get("running"):
252         response["errors"].append("Ja hi ha una avaluació en curs.")
253         return response
254
255     model_path = Path(request.get("model_path") or "")
256     env_id = str(request.get("env_id") or "").strip()
257     vecnorm_path = request.get("vecnorm_path")
258
259     if not model_path.exists():
260         response["errors"].append("No s'ha trobat el model seleccionat.")
261         return response
262     if not env_id:
263         response["errors"].append("No s'ha pogut determinar un env_id vàlid.")
264         return response
265
266     event_queue: "queue.Queue[Dict[str, Any]]" = queue.Queue()
267     stop_event = threading.Event()
268     job_config = {
269         "model_path": model_path,
270         "env_id": env_id,
271         "use_vecnorm": bool(request.get("use_vecnorm")),

```

```

272     "vecnorm_path": Path(vecnorm_path) if vecnorm_path else None,
273     "steps_target": int(request.get("steps_target") or ONE_YEAR_STEPS),
274     "deterministic": bool(request.get("deterministic", True)),
275 }
276 worker_thread = threading.Thread(
277     target=evaluation_worker,
278     args=(job_config, event_queue, stop_event),
279     daemon=True,
280 )
281
282 runtime.update(
283     {
284         "running": True,
285         "completed": False,
286         "cancel_requested": False,
287         "needs_full_rerun": False,
288         "thread": worker_thread,
289         "queue": event_queue,
290         "stop_event": stop_event,
291         "result": None,
292         "error": None,
293         "traceback": None,
294         "model_path": str(model_path),
295         "env_id": env_id,
296         "steps_target": int(job_config["steps_target"]),
297         "latest_step": 0,
298         "status": "Inicialitzant avaluació...",
299         "started_at": time.time(),
300     }
301 )
302 worker_thread.start()
303
304 response["started"] = True
305 response["runtime"] = runtime
306 return response
307
308
309 def load_model_metadata(model_path: Path) -> Dict[str, Any]:
310     """Carrega les metadades del model."""
311     metadata_path = model_path.with_name(model_path.stem + "_metadata.json")
312     if metadata_path.exists():
313         # Format nou: metadata separat al costat del .zip. Si hi falta algun camp,
314         # el completem des de training_config per compatibilitat.
315         with open(metadata_path, "r", encoding="utf-8") as handle:
316             metadata = json.load(handle)
317             training_config = metadata.get("training_config") if isinstance(metadata.get("training_config"), dict) else {}
318             if not metadata.get("env_id"):
319                 metadata["env_id"] = training_config.get("env_id")
320             if not metadata.get("reward_name"):
321                 metadata["reward_name"] = training_config.get("reward_name")
322             if "wrapper_configs" not in metadata and "wrapper_configs" in training_config:
323                 metadata["wrapper_configs"] = training_config.get("wrapper_configs", [])
324             if "meters" not in metadata and "meters" in training_config:
325                 metadata["meters"] = training_config.get("meters")
326             if "reward_kwargs" not in metadata and "reward_kwargs" in training_config:
327                 metadata["reward_kwargs"] = training_config.get("reward_kwargs")
328             metadata["metadata_source"] = str(metadata_path)
329             return metadata
330
331     training_config_path = model_path.parent / "training_config.json"
332     if training_config_path.exists():
333         # Format antic: només hi havia training_config.json dins la carpeta de la run.
334         with open(training_config_path, "r", encoding="utf-8") as handle:
335             payload = json.load(handle)
336             training_config = dict(payload.get("training_config") or {})
337             return {
338                 "env_id": payload.get("env_id") or training_config.get("env_id"),
339                 "reward_name": payload.get("reward_name") or training_config.get("reward_name"),
340                 "wrapper_configs": training_config.get("wrapper_configs", []),
341                 "meters": training_config.get("meters"),
342                 "reward_kwargs": training_config.get("reward_kwargs"),
343                 "training_config": training_config,
344                 "artifact_name": payload.get("artifact_name"),
345                 "metadata_source": str(training_config_path),
346             }
347     return {}
348
349
350 def apply_evaluation_action_contract(
351     env: gym.Env,
352     model_action_space: Any,
353     wrapper_configs: List[Dict[str, Any]],
354 ) -> gym.Env:
355     """Aplica contracte d'acció d'avaluació."""
356     from backend.entrenar_agent_backend import apply_training_wrappers
357
358     if wrapper_configs:
359         return apply_training_wrappers(env, wrapper_configs)
360     if should_apply_normalize_action(model_action_space, env.action_space):
361         return NormalizeAction(env)
362     return env
363

```

```

364
365 def validate_action_contract(model_action_space: Any, env_action_space: Any, metadata: Dict[str, Any]) -> None:
366     """Validació del contracte d'actuació."""
367     if action_spaces_compatible(model_action_space, env_action_space):
368         return
369
370     wrappers = metadata.get("wrapper_configs") or []
371     wrapper_names = ", ".join(str(item.get("name")) for item in wrappers if isinstance(item, dict)) or "cap detectat"
372     source = metadata.get("metadata_source") or "metadades no trobades"
373     raise ValueError(
374         "L'espai d'accions de l'entorn no coincideix amb el que espera el model. "
375         f"Model: {describe_action_space(model_action_space)}; "
376         f"entorn: {describe_action_space(env_action_space)}."
377         f"Wrappers detectats: {wrapper_names}. Font: {source}."
378     )
379
380
381 def _build_gym_kwargs(metadata: Dict[str, Any]) -> Dict[str, Any]:
382     """Construeix kwargs de Gym."""
383     training_config = metadata.get("training_config") or {}
384     meters = metadata.get("meters") or training_config.get("meters")
385     reward_name = metadata.get("reward_name") or training_config.get("reward_name")
386     reward_kwargs = metadata.get("reward_kwargs") or training_config.get("reward_kwargs") or {}
387
388     kwargs: Dict[str, Any] = {}
389     if not reward_name and not meters:
390         return kwargs
391
392     try:
393         # Quan avaluem un model, recuperem reward i meters del training perquè l'entorn
394         # tingui el mateix espai d'observació i la mateixa telemetria.
395         from backend.entrenar_agent_backend import REWARD_CLASSES, with_detailed_hvac_meters
396         kwargs["meters"] = with_detailed_hvac_meters(meters or {})
397         reward_cls = REWARD_CLASSES.get(reward_name) if reward_name else None
398         if reward_cls is not None:
399             kwargs["reward"] = reward_cls
400             if reward_kwargs:
401                 kwargs["reward_kwargs"] = reward_kwargs
402     except Exception:
403         pass
404     return kwargs
405
406
407 def make_env(env_id: str, seed: int = 0, gym_kwargs: Optional[Dict[str, Any]] = None) -> gym.Env:
408     """Crea un env per al flux Studio."""
409     env = gym.make(env_id, **(gym_kwargs or {}))
410     env.reset(seed=seed)
411     return env
412
413
414 def run_agent_evaluation(
415     model_path: Path,
416     env_id: str,
417     use_vecnorm: bool,
418     vecnorm_path: Optional[Path],
419     steps_target: int,
420     deterministic: bool,
421     should_stop: Callable[[int, bool], bool],
422     on_progress: Callable[[int, int], None],
423     override_wrapper_configs: Optional[List[Dict[str, Any]]] = None,
424 ) -> EvaluationResult:
425     """Executa l'avaluació i retorna el resum d'execució."""
426
427     with open(model_path, "rb") as file_handle:
428         model, _ = load_sb3_model_bytes(file_handle.read(), device="cpu")
429
430     metadata = load_model_metadata(model_path)
431     if override_wrapper_configs is not None:
432         wrapper_configs = override_wrapper_configs
433     else:
434         wrapper_configs = metadata.get("wrapper_configs", [])
435     model_action_space = getattr(model, "action_space", None)
436
437     gym_kwargs = _build_gym_kwargs(metadata)
438
439     env_factory = partial(build_evaluation_env, env_id, gym_kwargs, model_action_space, wrapper_configs)
440
441     eval_warnings: List[str] = []
442
443     venv = None
444     if use_vecnorm and vecnorm_path and vecnorm_path.exists():
445         # VecNormalize s'ha d'enganxar al mateix tipus d'entorn amb què es va entrenar.
446         # Si no encaixa, seguim sense normalització però deixem l'avís visible.
447         try:
448             venv = load_vecnormalize(str(vecnorm_path), env_factory)
449         except ValueError as exc:
450             eval_warnings.append(
451                 f"VecNormalize no aplicat: {exc} "
452                 "Continuant sense normalització d'observacions; les prediccions poden ser menys precises."
453             )
454     if venv is None:
455         venv = DummyVecEnv([env_factory])

```

```

456
457     try:
458         validate_action_contract(model_action_space, venv.action_space, metadata)
459     except ValueError as exc:
460         eval_warnings.append(f"Avis de contracte d'accions: {exc}")
461
462 model_obs_shape = getattr(getattr(model, "observation_space", None), "shape", None)
463 env_obs_shape = getattr(getattr(venv, "observation_space", None), "shape", None)
464 if model_obs_shape and env_obs_shape and tuple(model_obs_shape) != tuple(env_obs_shape):
465     # Aquest error és millor aturar-lo aquí: si les observacions no tenen la mateixa
466     # forma, el model podria predir accions aparentment vàlides però sense sentit.
467     model_size = int(np.prod(model_obs_shape))
468     env_size = int(np.prod(env_obs_shape))
469     raise ValueError(
470         f"L'espai d'observacions del model ({model_size} variables, forma {model_obs_shape}) "
471         f"no coincideix amb l'entorn seleccionat ({env_size} variables, forma {env_obs_shape}). "
472         "Assegura't d'avaluar el model amb el mateix entorn i configuració de wrappers amb que va ser entrenat."
473     )
474
475 cancelled = False
476 start_time = time.time()
477 obs = venv.reset()
478
479 try:
480     for step_number in range(steps_target):
481         if should_stop():
482             cancelled = True
483             break
484
485         action, _ = model.predict(obs, deterministic=deterministic)
486         formatted_action = format_action_for_env(action, venv)
487
488         next_obs, rewards, dones, infos = venv.step(formatted_action)
489         obs = next_obs
490
491         if bool(np.array(dones).reshape(-1)[0]):
492             # Alguns entorns acaben episodi abans dels passos demanats; reiniciem per
493             # mantenir l'avaluació viva fins al límit triat per l'usuari.
494             obs = venv.reset()
495
496         if (step_number + 1) % 200 == 0 or (step_number + 1) == steps_target:
497             on_progress(step_number + 1, steps_target)
498 finally:
499     try:
500         base = venv
501         if isinstance(base, VecEnvWrapper):
502             base = base.venv
503         if hasattr(base, "envs") and base.envs:
504             base.envs[0].close()
505     except Exception:
506         pass
507
508 return EvaluationResult(
509     elapsed_seconds=time.time() - start_time,
510     cancelled=cancelled,
511     warnings=tuple(eval_warnings),
512 )

```

E.10 backend/entrenar_agent_artifacts.py

Listing 31: backend/entrenar_agent_artifacts.py

```

1  """Rutes d'artefacte d'entrenament, metadades i postprocessament CSV."""
2
3  from __future__ import annotations
4
5  import copy
6  import csv
7  import json
8  import os
9  import re
10 from pathlib import Path
11 from typing import Any, Dict, List
12
13 from backend.entrenar_agent_constants import (
14     DETAILED_HVAC_METERS,
15     JOULES_PER_KWH,
16     TRAINING_ARTIFACTS_DIR_NAME,
17     TRAINING_CONFIG_FILENAME,
18 )
19
20
21 def sanitize_training_name_component(value: str) -> str:
22     """Higienitzar el component del nom de la entrenament."""
23     cleaned = re.sub(r"[^A-Za-z0-9]+", "-", value).strip("-").lower()
24     return cleaned or "sense-nom"
25
26

```



```

27 def get_training_artifacts_root(base_path: Path | None = None) -> Path:
28     """Retorna l'arrel dels artefactes d'entrenament."""
29     return (base_path or Path.cwd()).resolve() / TRAINING_ARTIFACTS_DIR_NAME
30
31
32 def build_training_artifact_paths(
33     training_config: Dict[str, Any],
34     base_path: Path | None = None,
35 ) -> Dict[str, Path | str]:
36     """Crea rutes d'artefactes d'entrenament per al flux Studio."""
37     artifacts_root = get_training_artifacts_root(base_path)
38     env_slug = sanitize_training_name_component(training_config["env_id"])
39     reward_slug = sanitize_training_name_component(training_config["reward_name"])
40     family_dir = artifacts_root / env_slug / reward_slug
41     prefix = f"training-{env_slug}-{reward_slug}-"
42     max_index = 0
43
44     if family_dir.exists():
45         for child in family_dir.iterdir():
46             if not child.is_dir():
47                 continue
48             match = re.fullmatch(rf"{re.escape(prefix)}(\d+)", child.name)
49             if match:
50                 max_index = max(max_index, int(match.group(1)))
51
52     artifact_name = f"{prefix}{max_index + 1:03d}"
53     artifact_dir = family_dir / artifact_name
54     return {
55         "artifact_name": artifact_name,
56         "artifact_dir": artifact_dir,
57         "model_path": artifact_dir / f"{artifact_name}.zip",
58         "vecnorm_path": artifact_dir / f"{artifact_name}_vecnorm.pkl",
59         "eval_script_path": artifact_dir / f"{artifact_name}_eval.py",
60         "training_script_path": artifact_dir / f"{artifact_name}.py",
61         "config_path": artifact_dir / TRAINING_CONFIG_FILENAME,
62     }
63
64
65 def append_detailed_meter_kwh_columns(observations_path: str | Path) -> None:
66     """Afegeix columnes detallades del comptador de kWh."""
67     observations_file = Path(observations_path) if observations_path else None
68     if observations_file is None or not observations_file.is_file():
69         return
70
71     with open(observations_file, "r", newline="", encoding="utf-8") as file_handle:
72         reader = csv.DictReader(file_handle)
73         fieldnames = list(reader.fieldnames or [])
74         rows = list(reader)
75
76     if not fieldnames or not rows:
77         return
78
79     source_columns: Dict[str, str] = {}
80     for alias, meter_name in DETAILED_HVAC_METERS.items():
81         target_column = f"{alias}_kwh"
82         if target_column in fieldnames:
83             continue
84         if alias in fieldnames:
85             source_columns[alias] = alias
86         elif meter_name in fieldnames:
87             source_columns[alias] = meter_name
88
89     if not source_columns:
90         return
91
92     # EnergyPlus escriu molts meters en joules. Afegim columnes *_kwh perquè els gràfics
93     # posteriors no hagin de repetir aquesta conversió cada vegada.
94     derived_columns = {source_alias: f"{source_alias}_kwh" for source_alias in source_columns}
95     for row in rows:
96         for source_alias, source_column in source_columns.items():
97             target_column = derived_columns[source_alias]
98             try:
99                 row[target_column] = str(float(row.get(source_column, "")) / JOULES_PER_KWH)
100             except (TypeError, ValueError):
101                 row[target_column] = ""
102
103     output_fields = fieldnames + list(derived_columns.values())
104     with open(observations_file, "w", newline="", encoding="utf-8") as file_handle:
105         writer = csv.DictWriter(file_handle, fieldnames=output_fields)
106         writer.writeheader()
107         writer.writerows(rows)
108
109
110 def append_detailed_meter_kwh_columns_in_workspace(workspace_path: str | Path | None) -> None:
111     """Afegeix columnes detallades del comptador de kWh a l'espai de treball."""
112     if not workspace_path:
113         return
114
115     workspace = Path(workspace_path)
116     if not workspace.is_dir():
117         return
118

```

```

119 for observations_file in workspace.rglob("observations.csv"):
120     try:
121         append_detailed_meter_kwh_columns(observations_file)
122     except Exception:
123         continue
124
125
126 def get_runtime_workspace_path(runtime: Dict[str, Any]) -> str | None:
127     """Retorna la ruta de l'espai de treball en temps d'execució."""
128     if not runtime:
129         return None
130
131     try:
132         paths = runtime["vec"].get_attr("workspace_path")
133         if paths and paths[0]:
134             return str(paths[0])
135     except Exception:
136         pass
137
138     # Segon intent: alguns models guarden el VecEnv dins model.env, depenent de com s'ha
139     # creat la run i de la versió d'SB3.
140     try:
141         paths = runtime["model"].env.get_attr("workspace_path")
142         if paths and paths[0]:
143             return str(paths[0])
144     except Exception:
145         pass
146
147     try:
148         # Últim recurs per a runs antigues: busquem la carpeta més recent que coincideixi
149         # amb l'env_id i que contingui els CSV esperats.
150         env_id = runtime["config"]["env_id"]
151         candidates = sorted(
152             [
153                 directory
154                 for directory in Path(os.getcwd()).iterdir()
155                 if directory.is_dir() and directory.name.startswith(f"{env_id}-res")
156             ],
157             key=lambda directory: directory.stat().st_mtime,
158             reverse=True,
159         )
160         if candidates:
161             return str(candidates[0])
162     except Exception:
163         pass
164
165     return None
166
167
168 def build_training_ui_state(options: Dict[str, Any]) -> Dict[str, Any]:
169     """Crea l'estat de la interfície de l'entrenament per al flux Studio."""
170     excluded_keys = {"spec", "variables"}
171     return {
172         key: copy.deepcopy(value)
173         for key, value in options.items()
174         if key not in excluded_keys
175     }
176
177
178 def list_saved_training_runs(base_path: Path | None = None) -> List[Dict[str, Any]]:
179     """Llista les sessions d'entrenament desades."""
180     artifacts_root = get_training_artifacts_root(base_path)
181     if not artifacts_root.exists():
182         return []
183
184     runs: List[Dict[str, Any]] = []
185     for config_path in artifacts_root.rglob(TRAINING_CONFIG_FILENAME):
186         try:
187             with open(config_path, "r", encoding="utf-8") as handle:
188                 payload = json.load(handle)
189         except Exception:
190             continue
191
192         artifact_name = payload.get("artifact_name") or config_path.parent.name
193         artifact_dir = Path(payload.get("artifact_dir") or config_path.parent)
194         files = payload.get("files") or {}
195         training_script_name = files.get("training_script") or f"{artifact_name}.py"
196         training_script_path = artifact_dir / training_script_name
197
198         payload["artifact_name"] = artifact_name
199         payload["artifact_dir"] = str(artifact_dir)
200         payload["config_path"] = str(config_path)
201         payload["training_script_path"] = str(training_script_path)
202         payload["training_script_exists"] = training_script_path.exists()
203         runs.append(payload)
204
205     runs.sort(
206         key=lambda item: (
207             item.get("created_at") or "",
208             item.get("artifact_name") or "",
209         ),
210         reverse=True,

```

```

211 )
212 return runs

```

E.11 backend/entrenar_agent_backend.py

Listing 32: backend/entrenar_agent_backend.py

```

1  """Façana pública per al backend de entrenament Studio.
2
3  La entrenament s'implementa a través de mòduls centrats en artefactes, entorn
4  resolució, configuració de recompenses, embolcalls, estat d'execució i sessió UI
5  normalització. Aquesta façana conserva el camí històric d'importació utilitzat per Streamlit
6  pàgines tot donant a Sphinx un sol lloc per exposar la superfície de entrenament API.
7  """
8
9  from backend.entrenar_agent_artifacts import (
10     append_detailed_meter_kwh_columns,
11     append_detailed_meter_kwh_columns_in_workspace,
12     build_training_artifact_paths,
13     build_training_ui_state,
14     get_runtime_workspace_path,
15     get_training_artifacts_root,
16     list_saved_training_runs,
17     sanitize_training_name_component,
18 )
19 from backend.entrenar_agent_constants import (
20     BATTERY_REWARDS,
21     COST_REWARDS,
22     DEFAULT_FORECAST_COLUMNS,
23     DEFAULT_MINUTES_PER_STEP,
24     DETAILED_HVAC_METERS,
25     DISCRETE_ACTION_SPACES,
26     ENERGY_COST_DIR,
27     FIELD_HELP,
28     FIXED_WRAPPER_ROWS,
29     HOURLY_REWARDS,
30     JOULES_PER_KWH,
31     MULTIZONE_REWARDS,
32     PACKAGE_DATA_DIR,
33     POLICIES,
34     REWARD_CLASSES,
35     TRAINING_ARTIFACTS_DIR_NAME,
36     TRAINING_CONFIG_FILENAME,
37     TRAINING_REPRO_SCRIPT_FILENAME,
38     TRAINING_RESULT_KEY,
39     TRAINING_RUNTIME_KEY,
40 )
41 from backend.entrenar_agent_env import (
42     _effective_action_space_from_spec,
43     default_battery_variables,
44     default_comfort_temperature_variables,
45     default_grid_energy_variables,
46     get_env_metadata,
47     get_training_meters_for_env,
48     is_comfort_temperature_variable,
49     is_energy_variable,
50     list_files,
51     list_registered_envs,
52     load_default_reward_variables,
53     with_detailed_hvac_meters,
54 )
55 from backend.entrenar_agent_rewards import (
56     assemble_training_payload,
57     build_algo_kwargs,
58     build_multizone_occupancy_mapping,
59     build_multizone_temp_mapping,
60     build_reward_kwargs,
61     reward_summary_frames,
62     validate_training_configuration,
63 )
64 from backend.entrenar_agent_runtime import (
65     build_training_env_instance,
66     clear_training_runtime,
67     create_training_runtime,
68     prepare_incremental_learn,
69     save_training_artifacts,
70 )
71 from backend.entrenar_agent_utils import filter_supported_kwargs, format_ui_value
72 from backend.entrenar_agent_wrappers import (
73     EnergyCostFileLogger,
74     apply_training_wrappers,
75     build_energy_cost_wrapper_reward_kwargs,
76     build_wrapper_configs,
77     build_wrapper_summary_rows,
78     wrapper_details,
79 )
80 from backend.sb3_utils import ALGOS

```

E.12 backend/entrenar_agent_charts.py

Listing 33: backend/entrenar_agent_charts.py

```
1  """Renderització de gràfics d'entrenament en directe per a la pàgina entrenar l'agent."""
2
3  from __future__ import annotations
4
5  import os
6  from pathlib import Path
7
8  import pandas as pd
9  import streamlit as st
10
11  from backend.entrenar_agent_session import TRAINING_WORKSPACE_KEY
12  from backend.grafics.data_loader import _repair_power_metrics_from_progress
13  from backend.grafics.figures_common import make_episode_metrics
14
15
16  def _get_workspace_from_runtime(runtime: dict) -> str | None:
17      """Extreu el workspace_path del VecEnv actiu.
18
19      Intenta tres estratègies per ordre de preferència:
20      1. ``VecEnv.get_attr`` (travessa la cadena d'embolcall Gymnasium).
21      2. ``model.env.get_attr`` (accés directe a l'entorn intern del model).
22      3. Exploració de CWD per trobar directoris que coincideixin amb ``{env_id}-res*`` (retrocés).
23
24      Retorna:
25          Camí absolut a l'espai de treball com a cadena, o ``None`` si no
26          l'estratègia té èxit.
27      """
28      # Estratègia 1: via VecEnv.get_attr
29      try:
30          paths = runtime["vec"].get_attr("workspace_path")
31          if paths and paths[0]:
32              return str(paths[0])
33      except Exception:
34          pass
35
36      # Estratègia 2: via model.env
37      try:
38          paths = runtime["model"].env.get_attr("workspace_path")
39          if paths and paths[0]:
40              return str(paths[0])
41      except Exception:
42          pass
43
44      # Estratègia 3: directori més recent a CWD que coincideix amb env_id
45      try:
46          env_id = runtime["config"]["env_id"]
47          cwd = Path(os.getcwd())
48          candidates = sorted(
49              [d for d in cwd.iterdir() if d.is_dir() and d.name.startswith(f"{env_id}-res")],
50              key=lambda d: d.stat().st_mtime,
51              reverse=True,
52          )
53          if candidates:
54              return str(candidates[0])
55      except Exception:
56          pass
57
58      return None
59
60
61  def render_live_training_charts() -> None:
62      """Mostra els gràfics d'evolució per episodi llegint ``progress.csv``.
63
64      Llegeix ``progress.csv`` de l'espai de treball actiu i en mostra tres
65      subgràfics (recompensa, infracció de temperatura i demanda d'energia). Si no
66      l'episodi encara s'ha completat, es mostra un missatge d'espera.
67      """
68      workspace_str = st.session_state.get(TRAINING_WORKSPACE_KEY)
69
70      if not workspace_str:
71          st.caption("Esperant la inicialització del workspace...")
72          return
73
74      progress_path = Path(workspace_str) / "progress.csv"
75      if not progress_path.exists():
76          st.markdown(
77              """
78              <div style="
79                  padding:0.75rem 1rem; border-radius:10px;
80                  background:#f8fafd; border:1px solid #e0e7ff;
81                  color:#6366f1; font-size:0.88rem;
82              ">
83                  Els grafics apareixeran en completar-se el primer episodi.
84              </div>
85              """,
86              unsafe_allow_html=True,
87          )
```

```

88     return
89
90     try:
91         progress = pd.read_csv(progress_path)
92     except Exception:
93         return
94
95     if progress.empty:
96         return
97
98     try:
99         progress = _repair_power_metrics_from_progress(progress)
100     except Exception:
101         pass
102
103     n_episodes = len(progress)
104     st.markdown(
105         f"""
106         <div style="display:flex; align-items:center; gap:0.6rem; margin-bottom:0.75rem;">
107             <span style="
108                 display:inline-flex; align-items:center; gap:0.35rem;
109                 background:#ecfdf5; color:#059669;
110                 border:1px solid #a7f3d0; border-radius:999px;
111                 padding:0.18rem 0.75rem; font-size:0.78rem; font-weight:700;
112                 letter-spacing:0.06em;
113             ">
114                 <span style="width:7px;height:7px;border-radius:50%;
115                     background:#10b981;display:inline-block;
116                     box-shadow:0 0 3px rgba(16,185,129,.25);"></span>
117                     EN VIU
118                 </span>
119                 <span style="color:var(--studio-text-soft);font-size:0.85rem;">
120                     {n_episodes} episodi{"s" if n_episodes != 1 else ""} completat{"s" if n_episodes != 1 else ""}
121                 </span>
122             </div>
123             """,
124         unsafe_allow_html=True,
125     )
126
127     fig = make_episode_metrics(progress)
128
129     # Paleta de colors per a cada subgràfic
130     _TRACE_COLORS = [
131         "#6366f1", # recompensa -> indigo
132         "#f59e0b", # violation -> amber
133         "#10b981", # power -> green
134     ]
135     _FILL_COLORS = [
136         "rgba(99,102,241,0.08)",
137         "rgba(245,158,11,0.08)",
138         "rgba(16,185,129,0.08)",
139     ]
140
141     # Estil cada traça: color, amplada, marcadors i àrea de farciment
142     for i, trace in enumerate(fig.data):
143         trace.update(
144             line=dict(color=_TRACE_COLORS[i], width=2.5),
145             marker=dict(
146                 color="#ffffff",
147                 size=8,
148                 line=dict(color=_TRACE_COLORS[i], width=2),
149             ),
150             fill="tozeroy",
151             fillcolor=_FILL_COLORS[i],
152         )
153
154     # Distribució manual de dominis amb prou espai per als títols de subgràfics
155     row_domains = [
156         [0.69, 1.00],
157         [0.34, 0.60],
158         [0.00, 0.25],
159     ]
160
161     # Propietats d'eix comuns
162     _axis_style = dict(
163         showgrid=True,
164         gridcolor="rgba(0,0,0,0.05)",
165         gridwidth=1,
166         zeroline=False,
167         linecolor="rgba(0,0,0,0.12)",
168         linewidth=1,
169         tickfont=dict(size=11, color="#64748b"),
170         title_font=dict(size=11, color="#94a3b8"),
171     )
172
173     fig.update_layout(
174         title_text="",
175         height=680,
176         paper_bgcolor="rgba(0,0,0,0)",
177         plot_bgcolor="rgba(248,250,255,0.6)",
178         margin=dict(t=30, b=20, l=50, r=10),
179         font=dict(size=12, color="#334155"),

```

```

180     yaxis=dict(domain=row_domains[0], **axis_style),
181     yaxis2=dict(domain=row_domains[1], **axis_style),
182     yaxis3=dict(domain=row_domains[2], **axis_style),
183     xaxis=dict(title_text="", **axis_style),
184     xaxis2=dict(title_text="", **axis_style),
185     xaxis3=dict(title_text="Episodi", **axis_style),
186 )
187
188 # Encoixinat de l'eix X perquè els marcadors dels extrems no es retallin
189 x_pad = 0.5
190 x_min = float(progress["episode_num"].min()) if "episode_num" in progress.columns else 1.0
191 x_max = float(progress["episode_num"].max()) if "episode_num" in progress.columns else float(len(progress))
192 fig.update_xaxes(range=[x_min - x_pad, x_max + x_pad])
193
194 # Reposicionar i estilitzar les anotacions del títol del subgràfic
195 for i, annotation in enumerate(fig.layout.annotations):
196     annotation.update(
197         y=row_domains[i][1],
198         yanchor="bottom",
199         font=dict(size=12, color="#475569", family="sans-serif"),
200     )
201
202 st.plotly_chart(fig, use_container_width=True, key="live_training_chart")

```

E.13 backend/entrenar_agent_constants.py

Listing 34: backend/entrenar_agent_constants.py

```

1  """Constants i registres per al backend de l'agent de entrenament."""
2
3  from __future__ import annotations
4
5  from pathlib import Path
6
7  import sinergym
8  from gymnasium.spaces import Discrete, MultiBinary, MultiDiscrete
9  from sinergym.utils.rewards import (
10     BatteryHVACReward,
11     EnergyCostHourlyReward,
12     EnergyCostLinearReward,
13     ExpReward,
14     HourlyLinearReward,
15     LinearReward,
16     MultiZoneEnergyCostHourlyReward,
17     MultiZoneEnergyCostReward,
18     MultiZoneBatteryHVACReward,
19     MultizoneEnergyCostBatteryHVACReward,
20     MultiZoneHourlyReward,
21     MultiZoneReward,
22     NormalizedLinearReward,
23     OccupancyMultiZoneReward,
24     OccupiedHoursDiscomfortReward,
25 )
26
27 from backend.paths import ONE_YEAR_STEPS
28
29
30 POLICIES = {
31     "PPO": ["MlpPolicy"],
32     "A2C": ["MlpPolicy"],
33     "DQN": ["MlpPolicy"],
34     "SAC": ["MlpPolicy"],
35     "TD3": ["MlpPolicy"],
36     "DDPG": ["MlpPolicy"],
37 }
38
39 REWARD_CLASSES = {
40     "LinearReward": LinearReward,
41     "ExpReward": ExpReward,
42     "HourlyLinearReward": HourlyLinearReward,
43     "EnergyCostLinearReward": EnergyCostLinearReward,
44     "EnergyCostHourlyReward": EnergyCostHourlyReward,
45     "NormalizedLinearReward": NormalizedLinearReward,
46     "BatteryHVACReward": BatteryHVACReward,
47     "MultiZoneReward": MultiZoneReward,
48     "OccupancyMultiZoneReward": OccupancyMultiZoneReward,
49     "MultiZoneHourlyReward": MultiZoneHourlyReward,
50     "MultiZoneEnergyCostReward": MultiZoneEnergyCostReward,
51     "MultiZoneEnergyCostHourlyReward": MultiZoneEnergyCostHourlyReward,
52     "MultiZoneBatteryHVACReward": MultiZoneBatteryHVACReward,
53     "MultizoneEnergyCostBatteryHVACReward": MultizoneEnergyCostBatteryHVACReward,
54     "OccupiedHoursDiscomfortReward": OccupiedHoursDiscomfortReward,
55 }
56
57 COST_REWARDS = {
58     "EnergyCostLinearReward",
59     "EnergyCostHourlyReward",
60     "MultiZoneEnergyCostReward",
61     "MultiZoneEnergyCostHourlyReward",

```

```

61 }
62 MULTIZONE_REWARDS = {
63     "MultiZoneReward",
64     "OccupancyMultiZoneReward",
65     "MultiZoneHourlyReward",
66     "MultiZoneEnergyCostReward",
67     "MultiZoneEnergyCostHourlyReward",
68     "MultiZoneBatteryHVACReward",
69     "MultizoneEnergyCostBatteryHVACReward",
70 }
71 HOURLY_REWARDS = {
72     "HourlyLinearReward",
73     "EnergyCostHourlyReward",
74     "MultiZoneHourlyReward",
75     "MultiZoneEnergyCostHourlyReward",
76     "OccupiedHoursDiscomfortReward",
77 }
78 BATTERY_REWARDS = {
79     "BatteryHVACReward",
80     "MultiZoneBatteryHVACReward",
81     "MultizoneEnergyCostBatteryHVACReward",
82 }
83
84 DEFAULT_MINUTES_PER_STEP = int((365 * 24 * 60) / ONE_YEAR_STEPS)
85
86 TRAINING_RUNTIME_KEY = "training_runtime"
87 TRAINING_RESULT_KEY = "training_result"
88 TRAINING_ARTIFACTS_DIR_NAME = "trainings"
89 TRAINING_CONFIG_FILENAME = "training_config.json"
90 TRAINING_REPRO_SCRIPT_FILENAME = "train_reproduce.py"
91 DETAILED_HVAC_METERS = {
92     "natural_gas_hvac": "NaturalGas:HVAC",
93     "heating_electricity": "Heating:Electricity",
94     "cooling_electricity": "Cooling:Electricity",
95     "fans_electricity": "Fans:Electricity",
96     "pumps_electricity": "Pumps:Electricity",
97     "heat_rejection_electricity": "HeatRejection:Electricity",
98     "humidifier_electricity": "Humidifier:Electricity",
99     "heat_recovery_electricity": "HeatRecovery:Electricity",
100 }
101 JOULES_PER_KWH = 3_600_000.0
102
103 FIELD_HELP = {
104     "env_id": "Entorn de Sinergym que vols entrenar. Defineix edifici, clima, observacions i controls.",
105     "algo_name": "Algorisme de Stable Baselines3 que farà l'entrenament del model.",
106     "policy_name": "Política o arquitectura base que farà servir l'algorisme.",
107     "reward_name": "Funció de recompensa que transforma l'estat de l'entorn en reward a cada pas.",
108     "range_winter": "Rang de confort d'hivern en C. Sortir d'aquest rang penalitza la reward.",
109     "range_summer": "Rang de confort d'estiu en C. Sortir d'aquest rang penalitza la reward.",
110     "energy_weight": "Pes relatiu de l'energia dins de la reward. Més alt = més pes energètic. A OccupiedHoursDiscomfortReward aquest pes s'aplica a les hores ocupades.",
111     "temperature_weight": "Pes del terme de confort en rewards amb cost. El pes de cost es calcula com 1 - energia - confort.",
112     "lambda_energy": "Factor base d'escala de la penalització energètica.",
113     "lambda_temperature": "Factor d'escala de la penalització de temperatura.",
114     "grid_energy_weight": "Pes de la potencia importada de xarxa dins de les rewards amb bateria.",
115     "battery_cycle_weight": "Pes del cicle de bateria: suma de potencia de carrega i descarrega.",
116     "battery_loss_weight": "Pes de les pèrdues tèrmiques de la bateria si l'entorn les exposa.",
117     "simultaneous_battery_weight": "Pes extra per penalitzar carrega i descarrega simultànies.",
118     "lambda_grid": "Factor d'escala de la penalització per importació de xarxa.",
119     "lambda_battery": "Factor d'escala de les penalitzacions de bateria.",
120     "summer_start": "Mes d'inici del període d'estiu usat per la reward.",
121     "summer_start_d": "Dia d'inici del període d'estiu usat per la reward.",
122     "summer_final": "Mes final del període d'estiu usat per la reward.",
123     "summer_final_d": "Dia final del període d'estiu usat per la reward.",
124     "lambda_energy_cost": "Factor d'escala del cost econòmic de l'energia dins de la reward.",
125     "comfort_threshold": "Marge de desviació acceptable respecte al setpoint en rewards multizona.",
126     "range_comfort_hours": "Franja horària en que el confort té més pes dins de la reward.",
127     "occupied_hours": "Franja horària considerada d'ocupació. Dins d'aquesta franja es considera confort; fora d'ella la reward prioritza reduir consum.",
128     "occupied_discomfort_multiplier": "Multiplicador aplicat a la penalització de confort durant les hores d'ocupació.",
129     "off_hours_energy_multiplier": "Multiplicador extra del terme energètic fora d'ocupació. Més alt = menys consum nocturn i menys preconditioning.",
130     "use_energy_cost": "Activa el càlcul del cost energètic amb el fitxer de preus seleccionat.",
131     "use_file_logger": "Desa un CSV extra amb el cost estimat a cada pas.",
132     "energy_cost_path": "Ruta del fitxer CSV amb els preus de l'energia que farà servir el wrapper de cost.",
133     "file_logger_name": "Nom del fitxer CSV on es desa el cost pas a pas.",
134     "datetime_wrapper": "Afegeix variables temporals com el mes, el dia i l'hora a l'observació.",
135     "previous_wrapper": "Afegeix a l'observació els valors del pas anterior per a les variables seleccionades.",
136     "previous_variables": "Variables de l'observació actual que es copiaran des del pas anterior.",
137     "multi_obs_wrapper": "Construeix una observació amb diversos passos consecutius per donar context temporal a l'agent.",
138     "multi_obs_n": "Nombre de finestres temporals consecutives que es conservaran a l'observació.",
139     "multi_obs_flatten": "Si està activat, la pila temporal es converteix en un vector pla.",
140     "normalize_obs_wrapper": "Normalitza l'observació per estabilitzar l'entrenament de l'agent.",
141     "normalize_obs_auto": "Actualitza automàticament la mitjana i la variància durant l'entrenament.",
142     "normalize_obs_epsilon": "Petit valor de seguretat per evitar divisions per zero en la normalització.",
143     "normalize_obs_mean": "Fitxer opcional amb la mitjana precomputada de les observacions.",
144     "normalize_obs_var": "Fitxer opcional amb la variància precomputada de les observacions.",
145     "weather_forecast_wrapper": "Afegeix a l'observació una previsió meteorològica construïda a partir del fitxer climàtic de l'entorn.",
146     "weather_forecast_n": "Nombre de mostres futures de previsió que s'afegiran a cada observació.",
147     "weather_forecast_delta": "Separació en passos entre mostres consecutives de previsió.",
148     "weather_forecast_columns": "Variables meteorològiques que es faràn servir per construir la previsió.",

```



```

149 "delta_temp_wrapper": "Calcula diferències entre temperatures mesurades i setpoints per simplificar el control.",
150 "delta_temp_variables": "Variables de temperatura reals que es compararan amb els setpoints.",
151 "delta_setpoint_variables": "Variables de consigna que es faràn servir de referència per calcular el delta.",
152 "reduce_obs_wrapper": "Elimina variables de l'observació per reduir-ne la dimensió.",
153 "reduced_observations": "Variables que s'exclouran de l'observació final.",
154 "incremental_wrapper": "Transforma accions contínues en increments limitats per variable.",
155 "incremental_variables": "Variables d'acció que es controlaran de forma incremental.",
156 "incremental_initial_value": "Valor inicial de la variable abans d'aplicar increments.",
157 "incremental_delta": "Canvi màxim admissible respecte al valor actual de la variable.",
158 "incremental_step": "Pas discret intern amb que es representen els increments.",
159 "discrete_incremental_wrapper": "Transforma l'espai d'accions en decisions discretes d'augment, manténiment o reducció.",
160 "discrete_incremental_initial": "Valor inicial per a cada variable controlada en mode discret incremental.",
161 "discrete_incremental_delta": "Canvi màxim que es pot acumular en mode discret incremental.",
162 "discrete_incremental_step": "Mida del pas aplicat a cada decisió discreta.",
163 "normalize_action": "Reescaleta l'espai d'accions continu a un rang fix per facilitar l'entrenament.",
164 "learning_rate": "Velocitat d'aprenentatge del model. Massa alt pot fer l'entrenament inestable.",
165 "n_steps": "Nombre de passos recollits abans de cada actualització en PPO i A2C.",
166 "timesteps_per_year": f"Nombre total de passos d'entrenament. Amb els valors per defecte, 1 step = {DEFAULT_MINUTES_PER_STEP} minuts i {ONE_YEAR_STEPS} steps = 1 any.",
167 "start_training": "Inicia l'entrenament amb la configuració actual.",
168 "stop_training": "Atura l'entrenament al final del bloc actual i desa l'estat parcial.",
169 }
170
171 DEFAULT_FORECAST_COLUMNS = [
172     "Dry Bulb Temperature",
173     "Relative Humidity",
174     "Wind Direction",
175     "Wind Speed",
176     "Direct Normal Radiation",
177     "Diffuse Horizontal Radiation",
178 ]
179
180 PACKAGE_DATA_DIR = Path(sinergym.__file__).resolve().parent / "data"
181 ENERGY_COST_DIR = PACKAGE_DATA_DIR / "energy_cost"
182
183 FIXED_WRAPPER_ROWS = [
184     {"Wrapper": "Monitor", "Actiu": "Si", "Detall": "Sempre actiu"},
185     {"Wrapper": "LoggerWrapper", "Actiu": "Si", "Detall": "Sempre actiu"},
186     {"Wrapper": "CSVLogger", "Actiu": "Si", "Detall": "Sempre actiu"},
187 ]
188
189 DISCRETE_ACTION_SPACES = (Discrete, MultiDiscrete, MultiBinary)

```

E.14 backend/entrenar_agent_env.py

Listing 35: backend/entrenar_agent_env.py

```

1  """Descobrimet d'entorns i metadades per configurar entrenaments."""
2
3  from __future__ import annotations
4
5  import os
6  from pathlib import Path
7  from typing import Any, Dict, List, Sequence
8
9  import gymnasium as gym
10 import sinergym
11 import yaml
12 from gymnasium.spaces import Box
13
14 from backend.entrenar_agent_constants import DISCRETE_ACTION_SPACES
15
16
17 def list_registered_envs() -> List[str]:
18     """Llista d'envs registrats."""
19     return sorted([env_name for env_name in gym.envs.registry.keys() if env_name.startswith("Eplus-")])
20
21
22 def list_files(directory: Path, suffixes: Sequence[str]) -> List[str]:
23     """Llista de fitxers."""
24     if not directory.exists():
25         return []
26     return sorted(
27         str(path)
28         for path in directory.iterdir()
29         if path.is_file() and path.suffix.lower() in suffixes
30     )
31
32
33 def with_detailed_hvac_meters(meters: Dict[str, str] | None) -> Dict[str, str]:
34     """Retorna només comptadors configurats.
35
36     Inventar els metres HVAC predeterminats fa que EnergyPlus retorni els identificadors API no vàlids
37     per a edificis que no els exposen, omplint eplusout.err amb errors `Severe getMeterValue`
38     errors getMeterValue. Els comptadors detallats encara s'utilitzen quan es generen
39     la configuració de l'entorn els va detectar explícitament.
40     """
41     return dict(meters or {})

```



```

42
43
44 def get_training_meters_for_env(env_id: str) -> Dict[str, str]:
45     """Retorna els mesuradors d'entrenament per a env."""
46     try:
47         spec = gym.spec(env_id)
48         env_kwargs = spec.kwargs or {}
49         meters = env_kwargs.get("meters", {}) or {}
50     except Exception:
51         meters = {}
52     return with_detailed_hvac_meters(meters)
53
54
55 def _effective_action_space_from_spec(spec: gym.envs.registration.EnvSpec, fallback: Any) -> Any:
56     """Espai d'acció efectiu des de les especificacions."""
57     for wrapper_spec in getattr(spec, "additional_wrappers", ()) or ():
58         if getattr(wrapper_spec, "name", "") != "DiscretizeEnv":
59             continue
60         wrapper_kwargs = getattr(wrapper_spec, "kwargs", {}) or {}
61         discrete_space = wrapper_kwargs.get("discrete_space")
62         if discrete_space is not None:
63             return discrete_space
64     return fallback
65
66
67 def get_env_metadata(env_id: str) -> Dict[str, Any]:
68     """Retorna les metadades de l'env."""
69     spec = gym.spec(env_id)
70     env_kwargs = dict(spec.kwargs or {})
71     variables = env_kwargs.get("variables", {}) or {}
72     meters = with_detailed_hvac_meters(env_kwargs.get("meters", {}) or {})
73     env_kwargs["meters"] = meters
74     actuators = env_kwargs.get("actuators", {}) or {}
75     context = env_kwargs.get("context", {}) or {}
76     time_variables = env_kwargs.get("time_variables", []) or []
77     observation_variables = time_variables + list(variables.keys()) + list(meters.keys())
78     action_variables = list(actuators.keys())
79     context_variables = list(context.keys())
80     base_action_space = env_kwargs.get("action_space")
81     action_space = _effective_action_space_from_spec(spec, base_action_space)
82     building_file = env_kwargs.get("building_file")
83     building_name = os.path.basename(building_file) if building_file else ""
84     return {
85         "spec": spec,
86         "env_kwargs": env_kwargs,
87         "variables": variables,
88         "meters": meters,
89         "actuators": actuators,
90         "context": context,
91         "time_variables": time_variables,
92         "observation_variables": observation_variables,
93         "action_variables": action_variables,
94         "context_variables": context_variables,
95         "action_space": action_space,
96         "base_action_space": base_action_space,
97         "is_continuous": isinstance(action_space, Box),
98         "is_discrete": isinstance(action_space, DISCRETE_ACTION_SPACES),
99         "building_file": building_file,
100         "building_name": building_name,
101         "candidate_temp_vars": default_comfort_temperature_variables(observation_variables, variables),
102         "candidate_energy_vars": [
103             key for key in observation_variables if is_energy_variable(key, variables)
104         ],
105         "candidate_grid_energy_vars": default_grid_energy_variables(observation_variables),
106         "candidate_battery_charge_vars": default_battery_variables(observation_variables, "charge"),
107         "candidate_battery_discharge_vars": default_battery_variables(observation_variables, "discharge"),
108         "candidate_battery_loss_vars": default_battery_variables(observation_variables, "loss"),
109         "candidate_cost_vars": [key for key in observation_variables if ("cost" in key.lower() or "billing" in key.lower())],
110         "candidate_setpoint_vars": [key for key in variables if "setpoint" in key.lower()],
111     }
112
113
114 def is_comfort_temperature_variable(variable_name: str, variables: Dict[str, Any]) -> bool:
115     """Es variable la temperatura de confort."""
116     variable_name_lower = str(variable_name).lower()
117     if "temperature" not in variable_name_lower:
118         return False
119     if "outdoor" in variable_name_lower or "setpoint" in variable_name_lower:
120         return False
121
122     variable_meta = variables.get(variable_name)
123     if isinstance(variable_meta, (list, tuple)):
124         output_name = str(variable_meta[0]).lower() if len(variable_meta) > 0 else ""
125         output_key = str(variable_meta[1]).lower() if len(variable_meta) > 1 else ""
126         if "outdoor" in output_name:
127             return False
128         if output_key in {"environment", "site", "whole building"}:
129             return False
130         if "zone air temperature" in output_name:
131             return True
132
133     return "air_temperature" in variable_name_lower

```

```

134
135
136 def default_comfort_temperature_variables(
137     variable_names: Sequence[str], variables: Dict[str, Any]
138 ) -> List[str]:
139     """Variables de temperatura de confort per defecte."""
140     return [
141         variable_name
142         for variable_name in variable_names
143         if variable_name in variables and is_comfort_temperature_variable(variable_name, variables)
144     ]
145
146
147 def is_energy_variable(variable_name: str, variables: Dict[str, Any]) -> bool:
148     """Es l'energia variable."""
149     variable_name_lower = str(variable_name).lower()
150     if any(
151         token in variable_name_lower
152         for token in ("energy", "electricity", "elèctricity", "power", "hvac", "demand")
153     ):
154         return True
155
156     variable_meta = variables.get(variable_name)
157     if isinstance(variable_meta, (list, tuple)):
158         output_name = str(variable_meta[0]).lower() if len(variable_meta) > 0 else ""
159         output_key = str(variable_meta[1]).lower() if len(variable_meta) > 1 else ""
160         if any(
161             token in output_name
162             for token in ("electricity", "power", "demand rate", "hvac")
163         ):
164             return True
165         if any(token in output_key for token in ("whole building", "hvac")):
166             return True
167
168     return False
169
170
171 def default_grid_energy_variables(variable_names: Sequence[str]) -> List[str]:
172     """Variables d'energia de xarxa per defecte."""
173     available = list(variable_names)
174     preferred = [
175         "facility_net_purchased_electricity_rate",
176         "facility_total_purchased_electricity_rate",
177         "facility_total_electricity_demand_rate",
178         "facility_total_building_electricity_demand_rate",
179     ]
180     for variable_name in preferred:
181         if variable_name in available:
182             return [variable_name]
183     return [
184         variable_name
185         for variable_name in available
186         if "purchased" in variable_name.lower() or "grid" in variable_name.lower()
187     ][:1]
188
189
190 def default_battery_variables(variable_names: Sequence[str], kind: str) -> List[str]:
191     """Variables per defecte de la bateria."""
192     available = list(variable_names)
193     kind_tokens = {
194         "charge": ("storage_charge_power", "battery_charge"),
195         "discharge": ("storage_discharge_power", "battery_discharge"),
196         "loss": ("storage_thermal_loss_rate", "battery_loss"),
197     }
198     tokens = kind_tokens.get(kind, ())
199     matches = [
200         variable_name
201         for variable_name in available
202         if any(token in variable_name.lower() for token in tokens)
203     ]
204     if matches:
205         return matches[:1]
206
207     fallback_tokens = {
208         "charge": ("charge",),
209         "discharge": ("discharge",),
210         "loss": ("loss", "thermal"),
211     }.get(kind, ())
212     return [
213         variable_name
214         for variable_name in available
215         if "storage" in variable_name.lower()
216         and any(token in variable_name.lower() for token in fallback_tokens)
217     ][:1]
218
219
220 def load_default_reward_variables(
221     spec: Any, env_id: str, variables: Dict[str, Any]
222 ) -> Tuple[Dict[str, Any], List[str], List[str], List[str]]:
223     """Carrega les variables de recompensa predeterminades."""
224     resolved_variables = variables
225     try:

```

```

226 building_file = spec.kwargs.get("building_file")
227 base_name = os.path.splitext(os.path.basename(building_file))[0] if building_file else env_id
228 cfg_file = os.path.join(
229     os.path.dirname(sinergym.__file__),
230     "data/default_configuration",
231     f"{base_name}.yaml",
232 )
233
234 temp_vars: List[str] = []
235 energy_vars: List[str] = []
236 cost_vars: List[str] = []
237 if os.path.isfile(cfg_file):
238     # Preferim la configuració oficial de Sinergym quan existeix; porta els noms
239     # de variables que l'entorn espera per defecte.
240     with open(cfg_file, "r", encoding="utf-8") as cfg_handle:
241         cfg = yaml.safe_load(cfg_handle) or {}
242         reward_cfg = cfg.get("reward_kwargs", {}) if isinstance(cfg.get("reward_kwargs"), dict) else {}
243         temp_vars = (
244             reward_cfg.get("temperature_variables")
245             or cfg.get("temperature_variables")
246             or []
247         )
248         energy_vars = (
249             reward_cfg.get("energy_variables")
250             or cfg.get("energy_variables")
251             or []
252         )
253         cost_vars = (
254             reward_cfg.get("energy_cost_variables")
255             or cfg.get("energy_cost_variables")
256             or []
257         )
258
259 if not temp_vars:
260     # Si el YAML no ajuda, fem una inferència prudent a partir dels noms de variables.
261     temp_vars = default_comfort_temperature_variables(variables.keys(), variables)
262 if not energy_vars:
263     energy_vars = [key for key in variables if is_energy_variable(key, variables)]
264 if not cost_vars:
265     cost_vars = [key for key in variables if ("cost" in key or "billing" in key)]
266
267 temp_vars = [value for value in temp_vars if value in variables]
268 energy_vars = [value for value in energy_vars if value in variables]
269 cost_vars = [value for value in cost_vars if value in variables]
270 # Tornem a sanejar temperatures per evitar setpoints o variables globals que hagin
271 # entrat per heurística però no siguin temperatura interior real.
272 sanitized_temp_vars = default_comfort_temperature_variables(temp_vars, variables)
273 if sanitized_temp_vars:
274     temp_vars = sanitized_temp_vars
275 return resolved_variables, temp_vars, energy_vars, cost_vars
276 except Exception:
277     return {}, [], [], []

```

E.15 backend/entrenar_agent_rewards.py

Listing 36: backend/entrenar_agent_rewards.py

```

1  """Recompenses, algorismes i creadors de càrrega útil de entrenament completa."""
2
3  from __future__ import annotations
4
5  from typing import Any, Dict, List, Sequence, Tuple
6
7  import pandas as pd
8
9  from backend.entrenar_agent_artifacts import build_training_ui_state
10 from backend.entrenar_agent_constants import (
11     BATTERY_REWARDS,
12     COST_REWARDS,
13     DEFAULT_MINUTES_PER_STEP,
14     MULTIZONE_REWARDS,
15     REWARD_CLASSES,
16 )
17 from backend.entrenar_agent_env import (
18     default_battery_variables,
19     default_grid_energy_variables,
20     get_training_meters_for_env,
21     load_default_reward_variables,
22 )
23 from backend.entrenar_agent_utils import filter_supported_kwargs, format_ui_value
24 from backend.entrenar_agent_wrappers import (
25     build_energy_cost_wrapper_reward_kwargs,
26     build_wrapper_configs,
27     build_wrapper_summary_rows,
28 )
29 from backend.sb3_utils import ALGOS
30
31

```

```

32 def reward_summary_frames(reward_kwargs: Dict[str, Any]) -> Tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
33     """Marcs de resum de recompensa."""
34     scalar_rows: List[Dict[str, str]] = []
35     list_rows: List[Dict[str, str]] = []
36     mapping_rows: List[Dict[str, str]] = []
37
38     for key, value in (reward_kwargs or {}).items():
39         if isinstance(value, dict):
40             if value:
41                 for inner_key, inner_value in value.items():
42                     mapping_rows.append(
43                         {"Bloc": key, "Element": inner_key, "Valor": format_ui_value(inner_value)}
44                     )
45             else:
46                 scalar_rows.append({"Parametre": key, "Valor": "-"})
47         elif isinstance(value, (list, tuple, set)):
48             list_rows.append({"Parametre": key, "Valor": format_ui_value(value)})
49         else:
50             scalar_rows.append({"Parametre": key, "Valor": format_ui_value(value)})
51
52     return pd.DataFrame(scalar_rows), pd.DataFrame(list_rows), pd.DataFrame(mapping_rows)
53
54
55 def build_multizone_temp_mapping(
56     variables: Dict[str, Any], temp_vars: Sequence[str], pair_setpoints: bool = False
57 ) -> Dict[str, Any]:
58     """Crea un mapa temporal multizona per al flux Studio."""
59     # Les variables de Sinergym guarden el domini/zona a la segona posició. L'aprofitem
60     # per emparellar cada temperatura amb els seus setpoints sense dependre només del nom.
61     setpoint_vars = [key for key in variables if "setpoint" in key.lower()]
62     heating_setpoints = [
63         key for key in setpoint_vars if "htg" in key.lower() or "heating" in key.lower()
64     ]
65     cooling_setpoints = [
66         key for key in setpoint_vars if "clg" in key.lower() or "cooling" in key.lower()
67     ]
68     temp_mapping: Dict[str, Any] = {}
69     for temp_var in temp_vars:
70         domain = variables[temp_var][1] if temp_var in variables else None
71         matches = [
72             setpoint_var
73             for setpoint_var in setpoint_vars
74             if domain and variables.get(setpoint_var, [None, None])[1] == domain
75         ]
76         if pair_setpoints:
77             # Algunes rewards volen parella [calor, fred]. Si no trobem coincidència de zona,
78             # fem fallback a la primera parella global disponible.
79             heating_matches = [match for match in matches if match in heating_setpoints]
80             cooling_matches = [match for match in matches if match in cooling_setpoints]
81             if heating_matches and cooling_matches:
82                 temp_mapping[temp_var] = [heating_matches[0], cooling_matches[0]]
83             elif matches:
84                 temp_mapping[temp_var] = matches[0]
85             elif heating_setpoints and cooling_setpoints:
86                 temp_mapping[temp_var] = [heating_setpoints[0], cooling_setpoints[0]]
87             elif setpoint_vars:
88                 temp_mapping[temp_var] = setpoint_vars[0]
89             elif matches:
90                 temp_mapping[temp_var] = matches[0]
91             elif len(setpoint_vars) == 1:
92                 temp_mapping[temp_var] = setpoint_vars[0]
93     return temp_mapping
94
95
96 def build_multizone_occupancy_mapping(
97     variables: Dict[str, Any], temp_vars: Sequence[str]
98 ) -> Dict[str, str]:
99     """Crea un mapa d'ocupació multizona per al flux Studio."""
100     occupancy_vars = [
101         key
102         for key in variables
103         if "occupant" in key.lower() or "occupancy" in key.lower()
104     ]
105     occupancy_mapping: Dict[str, str] = {}
106     for temp_var in temp_vars:
107         temp_meta = variables.get(temp_var)
108         temp_domain = temp_meta[1] if isinstance(temp_meta, (list, tuple)) and len(temp_meta) > 1 else None
109         matches = [
110             occupancy_var
111             for occupancy_var in occupancy_vars
112             if temp_domain
113             and isinstance(variables.get(occupancy_var), (list, tuple))
114             and len(variables[occupancy_var]) > 1
115             and variables[occupancy_var][1] == temp_domain
116         ]
117         if not matches:
118             temp_prefix = temp_var.lower().replace("_air_temperature", "")
119             matches = [
120                 occupancy_var
121                 for occupancy_var in occupancy_vars
122                 if occupancy_var.lower().startswith(temp_prefix)
123             ]

```

```

124     if matches:
125         occupancy_mapping[temp_var] = matches[0]
126     return occupancy_mapping
127
128
129 def _build_common_reward_kwargs(
130     options: Dict[str, Any],
131     temp_vars: Sequence[str],
132     energy_vars: Sequence[str],
133 ) -> Dict[str, Any]:
134     """Crea els paràmetres compartits per les rewards lineals i de bateria."""
135     return {
136         "temperature_variables": list(temp_vars),
137         "energy_variables": list(energy_vars),
138         "range_comfort_winter": tuple(options["range_winter"]),
139         "range_comfort_summer": tuple(options["range_summer"]),
140         "summer_start": (options["summer_start_m"], options["summer_start_d"]),
141         "summer_final": (options["summer_final_m"], options["summer_final_d"]),
142         "energy_weight": options["energy_weight"],
143         "lambda_energy": options["lambda_energy"],
144         "lambda_temperature": options["lambda_temperature"],
145     }
146
147
148 def _build_battery_reward_kwargs(
149     options: Dict[str, Any],
150     variables: Dict[str, Any],
151 ) -> Dict[str, Any]:
152     """Crea la part comuna de les rewards que penalitzen xarxa i bateria."""
153     available_variables = variables.keys()
154     grid_energy_variables = (
155         list(options["grid_energy_variables"])
156         if "grid_energy_variables" in options
157         else default_grid_energy_variables(available_variables)
158     )
159     battery_charge_variables = (
160         list(options["battery_charge_variables"])
161         if "battery_charge_variables" in options
162         else default_battery_variables(available_variables, "charge")
163     )
164     battery_discharge_variables = (
165         list(options["battery_discharge_variables"])
166         if "battery_discharge_variables" in options
167         else default_battery_variables(available_variables, "discharge")
168     )
169     battery_loss_variables = (
170         list(options["battery_loss_variables"])
171         if "battery_loss_variables" in options
172         else default_battery_variables(available_variables, "loss")
173     )
174
175     return {
176         "grid_energy_variables": grid_energy_variables,
177         "battery_charge_variables": battery_charge_variables,
178         "battery_discharge_variables": battery_discharge_variables,
179         "battery_loss_variables": battery_loss_variables,
180         "grid_energy_weight": options.get("grid_energy_weight", 0.2),
181         "battery_cycle_weight": options.get("battery_cycle_weight", 0.04),
182         "battery_loss_weight": options.get("battery_loss_weight", 0.005),
183         "simultaneous_battery_weight": options.get("simultaneous_battery_weight", 0.005),
184         "lambda_grid": options.get("lambda_grid", options["lambda_energy"]),
185         "lambda_battery": options.get("lambda_battery", options["lambda_energy"]),
186     }
187
188
189 def _build_multizone_base_kwargs(
190     options: Dict[str, Any],
191     energy_vars: Sequence[str],
192     temp_mapping: Dict[str, Any],
193 ) -> Dict[str, Any]:
194     """Crea la base de kwargs que comparteixen les rewards multizona."""
195     return {
196         "energy_variables": list(energy_vars),
197         "temperature_and_setpoints_conf": temp_mapping,
198         "comfort_threshold": options["comfort_threshold"],
199         "energy_weight": options["energy_weight"],
200         "lambda_energy": options["lambda_energy"],
201         "lambda_temperature": options["lambda_temperature"],
202     }
203
204
205 def _add_energy_cost_file_kwargs(
206     reward_kwargs: Dict[str, Any],
207     options: Dict[str, Any],
208     cost_vars: Sequence[str],
209 ) -> None:
210     """Afegeix cost horari i fitxer de preus quan la reward ho necessita."""
211     reward_kwargs.update(
212         {
213             "lambda_energy_cost": options["lambda_energy_cost"],
214             "energy_cost_variables": list(cost_vars),
215             "energy_cost_path": options["energy_cost_path"],

```

```

216         "seconds_per_step": DEFAULT_MINUTES_PER_STEP * 60,
217     }
218 )
219
220
221 def _build_single_zone_reward_kwargs(
222     reward_name: str,
223     options: Dict[str, Any],
224     reward_common: Dict[str, Any],
225     battery_kwargs: Dict[str, Any],
226     temp_vars: Sequence[str],
227     energy_vars: Sequence[str],
228     cost_vars: Sequence[str],
229 ) -> Dict[str, Any] | None:
230     """Crea kwargs per rewards que treballen amb variables globals o monozonea."""
231     if reward_name == "EnergyCostLinearReward":
232         reward_kwargs = dict(reward_common)
233         reward_kwargs["energy_cost_variables"] = list(cost_vars)
234         reward_kwargs["temperature_weight"] = options["temperature_weight"]
235         reward_kwargs["lambda_energy_cost"] = options["lambda_energy_cost"]
236         return reward_kwargs
237
238     if reward_name == "BatteryHVACReward":
239         reward_kwargs = dict(reward_common)
240         reward_kwargs["temperature_weight"] = options["temperature_weight"]
241         reward_kwargs.update(battery_kwargs)
242         return reward_kwargs
243
244     if reward_name == "HourlyLinearReward":
245         return {
246             "temperature_variables": list(temp_vars),
247             "energy_variables": list(energy_vars),
248             "range_comfort_winter": tuple(options["range_winter"]),
249             "range_comfort_summer": tuple(options["range_summer"]),
250             "summer_start": (options["summer_start_m"], options["summer_start_d"]),
251             "summer_final": (options["summer_final_m"], options["summer_final_d"]),
252             "lambda_energy": options["lambda_energy"],
253             "lambda_temperature": options["lambda_temperature"],
254             "range_comfort_hours": tuple(options["range_comfort_hours"]),
255         }
256
257     if reward_name == "EnergyCostHourlyReward":
258         reward_kwargs = dict(reward_common)
259         reward_kwargs["temperature_weight"] = options["temperature_weight"]
260         reward_kwargs["range_comfort_hours"] = tuple(options["range_comfort_hours"])
261         _add_energy_cost_file_kwargs(reward_kwargs, options, cost_vars)
262         return reward_kwargs
263
264     if reward_name == "OccupiedHoursDiscomfortReward":
265         reward_kwargs = dict(reward_common)
266         reward_kwargs["occupied_hours"] = tuple(options["occupied_hours"])
267         reward_kwargs["occupied_discomfort_multiplier"] = options["occupied_discomfort_multiplier"]
268         reward_kwargs["off_hours_energy_multiplier"] = options["off_hours_energy_multiplier"]
269         return reward_kwargs
270
271     return None
272
273
274 def _build_multizone_reward_kwargs(
275     reward_name: str,
276     options: Dict[str, Any],
277     energy_vars: Sequence[str],
278     cost_vars: Sequence[str],
279     temp_mapping: Dict[str, Any],
280     occupancy_mapping: Dict[str, str],
281     battery_kwargs: Dict[str, Any],
282 ) -> Dict[str, Any] | None:
283     """Crea kwargs per rewards que separen temperatura i consignes per zona."""
284     if reward_name == "MultiZoneReward":
285         return _build_multizone_base_kwargs(options, energy_vars, temp_mapping)
286
287     if reward_name == "OccupancyMultiZoneReward":
288         reward_kwargs = _build_multizone_base_kwargs(options, energy_vars, temp_mapping)
289         reward_kwargs["occupancy_variables_conf"] = occupancy_mapping
290         reward_kwargs["occupancy_threshold"] = 0.0
291         return reward_kwargs
292
293     if reward_name == "MultiZoneHourlyReward":
294         reward_kwargs = _build_multizone_base_kwargs(options, energy_vars, temp_mapping)
295         reward_kwargs["default_energy_weight"] = reward_kwargs.pop("energy_weight")
296         reward_kwargs["range_comfort_hours"] = tuple(options["range_comfort_hours"])
297         return reward_kwargs
298
299     if reward_name == "MultiZoneEnergyCostReward":
300         reward_kwargs = _build_multizone_base_kwargs(options, energy_vars, temp_mapping)
301         reward_kwargs["temperature_weight"] = options["temperature_weight"]
302         _add_energy_cost_file_kwargs(reward_kwargs, options, cost_vars)
303         return reward_kwargs
304
305     if reward_name == "MultiZoneEnergyCostHourlyReward":
306         reward_kwargs = _build_multizone_base_kwargs(options, energy_vars, temp_mapping)
307         reward_kwargs["temperature_weight"] = options["temperature_weight"]

```

```

308     reward_kwargs["range_comfort_hours"] = tuple(options["range_comfort_hours"])
309     _add_energy_cost_file_kwargs(reward_kwargs, options, cost_vars)
310     return reward_kwargs
311
312     if reward_name == "MultiZoneBatteryHVACReward":
313         reward_kwargs = _build_multizone_base_kwargs(options, energy_vars, temp_mapping)
314         reward_kwargs["temperature_weight"] = options["temperature_weight"]
315         reward_kwargs.update(battery_kwargs)
316         return reward_kwargs
317
318     if reward_name == "MultizoneEnergyCostBatteryHVACReward":
319         reward_kwargs = _build_multizone_base_kwargs(options, energy_vars, temp_mapping)
320         reward_kwargs["temperature_weight"] = options["temperature_weight"]
321         reward_kwargs["lambda_energy_cost"] = options["lambda_energy_cost"]
322         reward_kwargs["energy_cost_weight"] = options.get("energy_cost_weight", 0.20)
323         reward_kwargs["energy_cost_variables"] = list(cost_vars)
324         reward_kwargs["occupied_hours"] = tuple(options.get("occupied_hours", (8, 18)))
325         reward_kwargs["occupied_comfort_multiplier"] = options.get("occupied_comfort_multiplier", 2.0)
326         reward_kwargs["unoccupied_comfort_multiplier"] = options.get("unoccupied_comfort_multiplier", 0.3)
327         reward_kwargs.update(battery_kwargs)
328         return reward_kwargs
329
330     return None
331
332
333 def build_reward_kwargs(
334     options: Dict[str, Any],
335     variables: Dict[str, Any],
336     temp_vars: Sequence[str],
337     energy_vars: Sequence[str],
338     cost_vars: Sequence[str],
339 ) -> Tuple[Dict[str, Any], Dict[str, str], List[str]]:
340     """Crea kwargs de recompensa per al flux Studio."""
341     reward_name = options["reward_name"]
342     reward_common = _build_common_reward_kwargs(options, temp_vars, energy_vars)
343
344     # Les rewards multizona volen un diccionari zona -> consignes. El construïm abans
345     # de seleccionar la reward perquè també serveix per validar la configuració.
346     temp_mapping = build_multizone_temp_mapping(
347         variables,
348         temp_vars,
349         pair_setpoints=reward_name in ("MultiZoneBatteryHVACReward", "MultizoneEnergyCostBatteryHVACReward"),
350     )
351     occupancy_mapping = build_multizone_occupancy_mapping(variables, temp_vars)
352     battery_kwargs = _build_battery_reward_kwargs(options, variables)
353
354     reward_kwargs = _build_single_zone_reward_kwargs(
355         reward_name=reward_name,
356         options=options,
357         reward_common=reward_common,
358         battery_kwargs=battery_kwargs,
359         temp_vars=temp_vars,
360         energy_vars=energy_vars,
361         cost_vars=cost_vars,
362     )
363     if reward_kwargs is None:
364         reward_kwargs = _build_multizone_reward_kwargs(
365             reward_name=reward_name,
366             options=options,
367             energy_vars=energy_vars,
368             cost_vars=cost_vars,
369             temp_mapping=temp_mapping,
370             occupancy_mapping=occupancy_mapping,
371             battery_kwargs=battery_kwargs,
372         )
373     if reward_kwargs is None:
374         reward_kwargs = dict(reward_common)
375
376     reward_cls = REWARD_CLASSES[reward_name]
377     reward_kwargs, dropped_reward_kwargs = filter_supported_kwargs(reward_cls, reward_kwargs)
378     return reward_kwargs, temp_mapping, dropped_reward_kwargs
379
380
381 def validate_training_configuration(
382     options: Dict[str, Any], temp_mapping: Dict[str, str]
383 ) -> List[str]:
384     """Valideu la recompensa, l'algorisme, l'espai d'acció i la compatibilitat de l'embolcall."""
385     validation_errors: List[str] = []
386     reward_name = options["reward_name"]
387     algo_name = options["algo_name"]
388     action_space_type = options.get("action_space_type")
389
390     # Primer descartem combinacions que SB3 no accepta, abans de crear entorns o models.
391     if options.get("is_discrete") and algo_name in {"SAC", "TD3", "DDPG"}:
392         validation_errors.append(
393             f"{algo_name} només es pot entrenar amb espais d'acció continus. Fes servir PPO o A2C en aquest entorn."
394         )
395     if action_space_type in {"MultiDiscrete", "MultiBinary"} and algo_name == "DQN":
396         validation_errors.append(
397             "DQN només admet un Discrete simple; aquest entorn fa servir un espai d'acció multidiscret."
398         )
399

```



```

400 if reward_name in COST_REWARDS and (options["energy_weight"] + options["temperature_weight"] > 1.0):
401     validation_errors.append("En rewards amb cost cal complir energy_weight + temperature_weight <= 1.")
402 if reward_name in MULTIZONE_REWARDS and not temp_mapping:
403     validation_errors.append(
404         "No s'han pogut inferir setpoints compatibles per construir la reward multizona."
405     )
406 if reward_name in BATTERY_REWARDS:
407     # Les rewards amb bateria necessiten com a mínim saber què entra de la xarxa
408     # i alguna pista de càrrega o descàrrega; si no, la penalització queda buida.
409     available_variables = list((options.get("variables") or {}).keys())
410     grid_vars = (
411         options["grid_energy_variables"]
412         if "grid_energy_variables" in options
413         else default_grid_energy_variables(available_variables)
414     )
415     charge_vars = (
416         options["battery_charge_variables"]
417         if "battery_charge_variables" in options
418         else default_battery_variables(available_variables, "charge")
419     )
420     discharge_vars = (
421         options["battery_discharge_variables"]
422         if "battery_discharge_variables" in options
423         else default_battery_variables(available_variables, "discharge")
424     )
425     if not grid_vars:
426         validation_errors.append(
427             "No s'han detectat variables de xarxa per a la reward amb bateria."
428         )
429     if not charge_vars and not discharge_vars:
430         validation_errors.append(
431             "No s'han detectat variables de càrrega o descàrrega de bateria."
432         )
433     # A partir d'aquí validem dependències entre wrappers. Són regles de UI, no errors
434     # de Python: millor avisar amb missatges clars que deixar que falli al mig del training.
435     if options["enable_previous_wrapper"] and not options["previous_variables"]:
436         validation_errors.append("PreviousObservationWrapper requereix almenys una variable.")
437     if options["enable_weather_forecast_wrapper"] and not options["weather_forecast_columns"]:
438         validation_errors.append("WeatherForecastingWrapper requereix almenys una variable de previsió.")
439     if options["enable_delta_temp_wrapper"] and (
440         not options["delta_temp_variables"] or not options["delta_setpoint_variables"]
441     ):
442         validation_errors.append("DeltaTempWrapper requereix variables de temperatura i almenys un setpoint.")
443     if options["enable_reduce_obs_wrapper"] and options["enable_datetime_wrapper"]:
444         reserved_datetime_vars = {"month", "day_of_month", "hour"}
445         invalid_reduction = reserved_datetime_vars.intersection(options["reduced_observations"])
446         if invalid_reduction:
447             validation_errors.append(
448                 "No pots reduir month/day_of_month/hour mentre DatetimeWrapper estigui actiu."
449             )
450     if options["enable_incremental_wrapper"] and not options["incremental_variables"]:
451         validation_errors.append("IncrementalWrapper requereix almenys una variable d'acció.")
452     if options["enable_incremental_wrapper"] and options["enable_discrete_incremental_wrapper"]:
453         validation_errors.append("IncrementalWrapper i DiscreteIncrementalWrapper no es poden activar alhora.")
454     if (
455         options["enable_incremental_wrapper"] or options["enable_discrete_incremental_wrapper"]
456     ) and options["enable_normalize_action"]:
457         validation_errors.append("NormalizeAction no es pot combinar amb els wrappers incrementals en aquesta UI.")
458     if options["enable_normalize_action"] and not options.get("is_continuous", True):
459         validation_errors.append("NormalizeAction només es pot aplicar a entorns amb action_space continu.")
460     if options.get("enable_variability_context_wrapper"):
461         low_values = list(options.get("variability_context_low") or [])
462         high_values = list(options.get("variability_context_high") or [])
463         if not low_values or len(low_values) != len(high_values):
464             validation_errors.append("VariabilityContextWrapper requereix límits min/max coherents.")
465         if any(high <= low for low, high in zip(low_values, high_values)):
466             validation_errors.append("A VariabilityContextWrapper cada màxim ha de ser superior al mínim.")
467         if options.get("variability_step_frequency_max", 0) <= options.get("variability_step_frequency_min", 0):
468             validation_errors.append("La freqüència màxima del context ha de ser superior a la mínima.")
469     if (
470         options.get("enable_storage_smoothing_wrapper")
471         and options.get("building_name") != "OfficeGridStorageSmoothing.epJSON"
472     ):
473         validation_errors.append(
474             "OfficeGridStorageSmoothingActionConstraintsWrapper només és vàlid per OfficeGridStorageSmoothing.epJSON."
475         )
476
477     return validation_errors
478
479 # Àlies compatibles enrere per a l'identificador públic amb accent anterior.
480 globals()["validate_training_configuration"] = validate_training_configuration
481 globals()["avlidate_training_configuration"] = validate_training_configuration
482
483
484
485 def build_algo_kwargs(options: Dict[str, Any]) -> Tuple[Dict[str, Any], List[str]]:
486     """Crea algo kwargs per al flux Studio."""
487     candidate_kwargs = {
488         "learning_rate": options["learning_rate"],
489         "n_steps": options.get("n_steps"),
490         "batch_size": options.get("batch_size"),
491         "n_epochs": options.get("n_epochs"),

```



```

492     "gamma": options.get("gamma"),
493     "gae_lambda": options.get("gae_lambda"),
494     "clip_range": options.get("clip_range"),
495     "ent_coef": options.get("ent_coef"),
496     "vf_coef": options.get("vf_coef"),
497     "max_grad_norm": options.get("max_grad_norm"),
498 }
499 candidate_kwargs = {
500     key: value
501     for key, value in candidate_kwargs.items()
502     if value is not None
503 }
504 return filter_supported_kwargs(ALGOS[options["algo_name"]], candidate_kwargs)
505
506
507 def assemble_training_payload(options: Dict[str, Any]) -> Dict[str, Any]:
508     """Munta tota la configuració d'entrenament a partir de la UI."""
509     meters = get_training_meters_for_env(options["env_id"])
510     variables, temp_vars, energy_vars, cost_vars = load_default_reward_variables(
511         spec=options["spec"],
512         env_id=options["env_id"],
513         variables=options["variables"],
514     )
515     reward_kwargs, temp_mapping, dropped_reward_kwargs = build_reward_kwargs(
516         options=options,
517         variables=variables,
518         temp_vars=temp_vars,
519         energy_vars=energy_vars,
520         cost_vars=cost_vars,
521     )
522     validation_errors = validate_training_configuration(options, temp_mapping)
523     if not energy_vars:
524         validation_errors.append(
525             "No s'han pogut inferir variables d'energia per a la reward. Revisa la configuració de l'entorn."
526         )
527     if options["reward_name"] == "OccupancyMultiZoneReward":
528         # Aquesta reward necessita ocupació per cada temperatura/zona. Si falta algun
529         # mapa, millor fallar abans que entrenar amb una penalització incompleta.
530         occupancy_mapping = reward_kwargs.get("occupancy_variables_conf") or {}
531         missing_occupancy_mapping = [
532             temp_var
533             for temp_var in temp_mapping
534             if temp_var not in occupancy_mapping
535         ]
536         if missing_occupancy_mapping:
537             validation_errors.append(
538                 "No s'han pogut inferir variables d'ocupació per a totes les zones: "
539                 + ", ".join(missing_occupancy_mapping)
540             )
541     energy_cost_wrapper_reward_kwargs = build_energy_cost_wrapper_reward_kwargs(
542         options,
543         temp_vars,
544         energy_vars,
545     )
546     recalculate_energy_cost_reward = options["reward_name"] in {
547         "LinearReward",
548         "EnergyCostLinearReward",
549     }
550     # El wrapper de cost pot recalculer reward només en rewards lineals; en les altres,
551     # el cost ja entra dins la pròpia classe de reward.
552     wrapper_configs = build_wrapper_configs(
553         options,
554         energy_cost_reward_kwargs=energy_cost_wrapper_reward_kwargs,
555         recalculate_energy_cost_reward=recalculate_energy_cost_reward,
556     )
557     wrapper_rows = build_wrapper_summary_rows(wrapper_configs)
558     algo_kwargs, dropped_algo_kwargs = build_algo_kwargs(options)
559
560     # Aquest dict és el que es desa amb l'artefacte. Ha de ser suficient per repetir,
561     # avaluar o inspeccionar l'entrenament sense dependre de l'estat viu de Streamlit.
562     training_config = {
563         "env_id": options["env_id"],
564         "algo_name": options["algo_name"],
565         "policy_name": options["policy_name"],
566         "reward_name": options["reward_name"],
567         "reward_kwargs": reward_kwargs,
568         "variables": variables,
569         "meters": meters,
570         "temp_vars": temp_vars,
571         "energy_vars": energy_vars,
572         "cost_vars": cost_vars,
573         "wrapper_configs": wrapper_configs,
574         "wrapper_rows": wrapper_rows,
575         "learning_rate": options["learning_rate"],
576         "n_steps": options["n_steps"],
577         "batch_size": options.get("batch_size"),
578         "n_epochs": options.get("n_epochs"),
579         "gamma": options.get("gamma"),
580         "gae_lambda": options.get("gae_lambda"),
581         "clip_range": options.get("clip_range"),
582         "ent_coef": options.get("ent_coef"),
583         "vf_coef": options.get("vf_coef"),

```

```

584     "max_grad_norm": options.get("max_grad_norm"),
585     "algo_kwargs": algo_kwargs,
586     "timesteps_per_year": options["timesteps_per_year"],
587 }
588
589 return {
590     "dropped_reward_kwargs": dropped_reward_kwargs,
591     "dropped_algo_kwargs": dropped_algo_kwargs,
592     "ui_state": build_training_ui_state(options),
593     "validation_errors": validation_errors,
594     "training_config": training_config,
595 }

```

E.16 backend/entrenar_agent_runtime.py

Listing 37: backend/entrenar_agent_runtime.py

```

1  """Cicle de vida d'execució per a sessions d'entrenament en directe."""
2
3  from __future__ import annotations
4
5  import copy
6  import json
7  from datetime import datetime
8  from functools import partial
9  from pathlib import Path
10 from typing import Any, Dict, Sequence
11
12 import gymnasium as gym
13 import numpy as np
14 import streamlit as st
15 from stable_baselines3.common.monitor import Monitor
16 from stable_baselines3.common.vec_env import DummyVecEnv, VecNormalize
17 from sinergym.utils.wrappers import CSVLogger, LoggerWrapper
18
19 from backend.entrenar_agent_artifacts import (
20     append_detailed_meter_kwh_columns_in_workspace,
21     build_training_artifact_paths,
22     get_runtime_workspace_path,
23 )
24 from backend.entrenar_agent_constants import REWARD_CLASSES, TRAINING_RUNTIME_KEY
25 from backend.entrenar_agent_env import get_training_meters_for_env, with_detailed_hvac_meters
26 from backend.entrenar_agent_wrappers import apply_training_wrappers
27 from backend.sb3_utils import ALGOS
28 from backend.training_scripts import build_training_eval_script, build_training_repro_script
29
30
31 def clear_training_runtime() -> None:
32     """Neteja el temps d'execució de l'entrenament."""
33     runtime = st.session_state.pop(TRAINING_RUNTIME_KEY, None)
34     if not runtime:
35         return
36
37     workspace_path = get_runtime_workspace_path(runtime)
38     vec = runtime.get("vec")
39     if vec is not None:
40         try:
41             vec.envs[0].close()
42         except Exception:
43             try:
44                 vec.close()
45             except Exception:
46                 pass
47
48     append_detailed_meter_kwh_columns_in_workspace(workspace_path)
49
50
51 def prepare_incremental_learn(model: Any, vec_env: Any) -> None:
52     """Prepara l'aprenentatge incremental."""
53     rollout_buffer = getattr(model, "rollout_buffer", None)
54     if rollout_buffer is not None and hasattr(rollout_buffer, "reset"):
55         rollout_buffer.reset()
56
57     if getattr(model, "_last_obs", None) is None and vec_env is not None:
58         model._last_obs = vec_env.reset()
59         model._last_episode_starts = np.ones((vec_env.num_envs,), dtype=bool)
60
61
62 def save_training_artifacts(runtime: Dict[str, Any]) -> Dict[str, Any]:
63     """Deseu els artefactes d'entrenament."""
64     config = runtime["config"]
65     artifact_dir = Path(runtime["artifact_dir"])
66     artifact_dir.mkdir(parents=True, exist_ok=False)
67
68     runtime["model"].save(str(runtime["save_path"]))
69     runtime["vec"].save(str(runtime["vecnorm_path"]))
70
71     result = {

```

```

72     "artifact_name": runtime["artifact_name"],
73     "created_at": datetime.now().isoformat(timespec="seconds"),
74     "artifact_dir": str(artifact_dir),
75     "env_id": config["env_id"],
76     "reward_name": config["reward_name"],
77     "algo_name": config["algo_name"],
78     "policy_name": config["policy_name"],
79     "files": {
80         "training_script": Path(runtime["training_script_path"]).name,
81         "eval_script": Path(runtime["eval_script_path"]).name,
82         "model": Path(runtime["save_path"]).name,
83         "vecnorm": Path(runtime["vecnorm_path"]).name,
84         "config": Path(runtime["config_path"]).name,
85     },
86     "ui_state": runtime.get("ui_state", {}),
87     "training_config": config,
88 }
89
90 with open(runtime["config_path"], "w", encoding="utf-8") as handle:
91     json.dump(result, handle, indent=2, ensure_ascii=False)
92
93 with open(runtime["eval_script_path"], "w", encoding="utf-8") as handle:
94     handle.write(build_training_eval_script(config))
95
96 with open(runtime["training_script_path"], "w", encoding="utf-8") as handle:
97     handle.write(build_training_repro_script(runtime["artifact_name"], config))
98
99 return result
100
101
102 def build_training_env_instance(
103     env_id: str,
104     reward_name: str,
105     reward_kwargs: Dict[str, Any],
106     meters: Dict[str, str],
107     wrapper_configs: Sequence[Dict[str, Any]],
108 ) -> Any:
109     """Crea una instància d'entorn embolicat per a l'entrenament en directe."""
110     reward_cls = REWARD_CLASSES[reward_name]
111     env = gym.make(env_id, meters=meters, reward=reward_cls, reward_kwargs=reward_kwargs)
112     env = apply_training_wrappers(env, wrapper_configs)
113     env = LoggerWrapper(env)
114     env = CSVLogger(env)
115     env = Monitor(env)
116     return env
117
118
119 def create_training_runtime(
120     training_config: Dict[str, Any],
121     ui_state: Dict[str, Any] | None = None,
122 ) -> Dict[str, Any]:
123     """Crea temps d'execució de la entrenament."""
124     env_id = training_config["env_id"]
125     reward_name = training_config["reward_name"]
126     reward_kwargs = training_config["reward_kwargs"]
127     meters = with_detailed_hvac_meters(training_config.get("meters") or get_training_meters_for_env(env_id))
128     wrapper_configs = training_config["wrapper_configs"]
129     algo_name = training_config["algo_name"]
130     policy_name = training_config["policy_name"]
131     learning_rate = training_config["learning_rate"]
132     n_steps = training_config["n_steps"]
133     timesteps_per_year = training_config["timesteps_per_year"]
134
135     env_factory = partial(
136         build_training_env_instance,
137         env_id,
138         reward_name,
139         reward_kwargs,
140         meters,
141         wrapper_configs,
142     )
143     vec = DummyVecEnv([env_factory])
144     vec = VecNormalize(vec, norm_obs=True, norm_reward=True)
145
146     model_cls = ALGOS[algo_name]
147     algo_kwargs = dict(training_config.get("algo_kwargs") or {})
148     algo_kwargs.setdefault("learning_rate", learning_rate)
149     if algo_name in ["PP0", "A2C"] and n_steps is not None:
150         algo_kwargs.setdefault("n_steps", n_steps)
151     algo_kwargs["verbose"] = 0
152
153     model = model_cls(policy_name, vec, **algo_kwargs)
154     artifact_paths = build_training_artifact_paths(training_config)
155
156     return {
157         "model": model,
158         "vec": vec,
159         "config": training_config,
160         "ui_state": copy.deepcopy(ui_state or {}),
161         "artifact_name": artifact_paths["artifact_name"],
162         "artifact_dir": artifact_paths["artifact_dir"],
163         "save_path": artifact_paths["model_path"],

```

```

164     "vecnorm_path": artifact_paths["vecnorm_path"],
165     "eval_script_path": artifact_paths["eval_script_path"],
166     "training_script_path": artifact_paths["training_script_path"],
167     "config_path": artifact_paths["config_path"],
168     "chunk_timesteps": int(algo_kwargs.get("n_steps") or n_steps) if algo_name in ["PP0", "A2C"] else min(512, timesteps_per_year
169 ),
170 }

```

E.17 backend/entrenar_agent_session.py

Listing 38: backend/entrenar_agent_session.py

```

1  """Gestió de l'estat de sessió per a la pàgina d'entrenament de l'agent."""
2
3  from __future__ import annotations
4
5  import streamlit as st
6
7  from backend.entrenar_agent_backend import TRAINING_RUNTIME_KEY, clear_training_runtime
8
9  BATTERY_REWARDS = {
10     "BatteryHVACReward",
11     "MultiZoneBatteryHVACReward",
12     "MultizoneEnergyCostBatteryHVACReward",
13 }
14
15 TRAINING_WORKSPACE_KEY = "training_live_workspace_path"
16 TRAINING_FORM_PREFIX = "training_form_"
17 TRAINING_SAVED_RUN_KEY = "training_saved_run"
18 TRAINING_LOADED_ARTIFACT_KEY = "training_loaded_artifact"
19
20 TRAINING_RANGE_FIELDS = {
21     "range_winter",
22     "range_summer",
23     "range_comfort_hours",
24     "occupied_hours",
25 }
26
27 def training_form_key(field: str) -> str:
28     """Retorna la clau d'estat de sessió per a un camp de formulari d'entrenament."""
29     return f"{TRAINING_FORM_PREFIX}{field}"
30
31 def reset_widget_state_if_disabled(field: str, disabled: bool, value=False) -> None:
32     """Restableix l'estat de sessió d'un giny a un valor predeterminat quan el giny està desactivat."""
33     if disabled:
34         st.session_state[training_form_key(field)] = value
35
36
37 def ensure_select_state(field: str, options: list[str], fallback: str | None = None) -> str:
38     """Assegureu-vos que un camp de casella de selecció tingui un valor vàlid en estat de sessió.
39
40     Si el valor actual no es troba a les opcions, se substitueix per una alternativa (si és vàlid)
41     o per la primera opció.
42     """
43     key = training_form_key(field)
44     if not options:
45         st.session_state[key] = ""
46         return ""
47
48     current_value = st.session_state.get(key)
49     if current_value not in options:
50         st.session_state[key] = fallback if fallback in options else options[0]
51     return st.session_state[key]
52
53
54 def sanitize_multiselect_state(
55     field: str,
56     options: list[str],
57     fallback: list[str] | None = None,
58 ) -> None:
59     """Elimineu els valors obsolets d'un camp de selecció múltiple en estat de sessió.
60
61     Els valors que ja no estan presents a les opcions s'eliminen. Si el resultat és
62     buit i es proporciona una alternativa, s'utilitzen elements de reserva vàlids.
63     """
64     key = training_form_key(field)
65     current_value = st.session_state.get(key)
66     if current_value is None:
67         return
68
69     if not isinstance(current_value, (list, tuple, set)):
70         current_value = fallback or []
71
72     sanitized = [item for item in current_value if item in options]
73     if not sanitized and fallback is not None:
74         sanitized = [item for item in fallback if item in options]
75     st.session_state[key] = sanitized
76

```

```

77
78 def normalize_training_range_state() -> None:
79     """Converteix els camps d'interval codificats per llista en tuples en estat de sessió.
80
81     Streamlit serialitza les tuples lliscants com a llistes quan es restaura l'estat de la sessió
82     d'una carrera guardada; aquesta funció els torna a convertir en tuples de manera que
83     ``st.slider`` no genera cap desajust de tipus.
84     """
85     for field in TRAINING_RANGE_FIELDS:
86         key = training_form_key(field)
87         value = st.session_state.get(key)
88         if isinstance(value, list) and len(value) == 2:
89             st.session_state[key] = tuple(value)
90
91
92 def clear_incremental_widget_state() -> None:
93     """Elimineu totes les claus del widget d'embolcall incremental dinàmic de l'estat de sessió.
94
95     Les claus s'identifiquen pel seu patró de prefix, que és generat per
96     ``training_form_key`` per a cada widget incremental per variable.
97     """
98     dynamic_prefixes = (
99         training_form_key("incr_init_"),
100         training_form_key("incr_delta_"),
101         training_form_key("incr_step_"),
102         training_form_key("disc_incr_init_"),
103     )
104     for session_key in list(st.session_state.keys()):
105         if session_key.startswith(dynamic_prefixes):
106             st.session_state.pop(session_key, None)
107
108
109 def apply_saved_training_ui_state(ui_state: dict, envs: list[str]) -> None:
110     """Restaura un estat d'entrenament desat anteriorment UI a l'estat de sessió.
111
112     Primer esborra totes les claus del widget incrementals i després escriu cada camp des de
113     ``ui_state``, converteix els camps d'interval en tuples i restableix l'entrenament
114     temps d'execució perquè la configuració carregada tingui efecte a la següent execució.
115     """
116     clear_incremental_widget_state()
117     for field, value in (ui_state or {}).items():
118         normalized_value = tuple(value) if field in TRAINING_RANGE_FIELDS and isinstance(value, list) else value
119         st.session_state[training_form_key(field)] = normalized_value
120
121     ensure_select_state("env_id", envs)
122     clear_training_runtime()
123     st.session_state.is_training = False
124     st.session_state.stop_training = False
125     st.session_state.pop(TRAINING_RUNTIME_KEY, None)
126     st.session_state.pop(TRAINING_WORKSPACE_KEY, None)
127
128
129 def seed_incremental_widget_defaults(selected_variables: list[str]) -> None:
130     """Empleneu prèviament els ginyos d'embolcall incrementals per variable des de l'estat desat.
131
132     Només escriu a claus d'estat de sessió que encara no existeixen, de manera que
133     Els valors dels widgets en directe mai es sobreescrueixen en tornar a renderitzar.
134     """
135     stored_variables = st.session_state.get(training_form_key("incremental_variables"), []) or []
136     stored_initial_values = st.session_state.get(training_form_key("incremental_initial_values"), []) or []
137     stored_definition = st.session_state.get(training_form_key("incremental_definition"), {}) or {}
138     initial_values_map = {
139         variable_name: stored_initial_values[index]
140         for index, variable_name in enumerate(stored_variables)
141         if index < len(stored_initial_values)
142     }
143
144     for variable_name in selected_variables:
145         init_key = training_form_key(f"incr_init_{variable_name}")
146         delta_key = training_form_key(f"incr_delta_{variable_name}")
147         step_key = training_form_key(f"incr_step_{variable_name}")
148         if init_key not in st.session_state and variable_name in initial_values_map:
149             st.session_state[init_key] = initial_values_map[variable_name]
150
151         definition_values = stored_definition.get(variable_name)
152         if isinstance(definition_values, (list, tuple)) and len(definition_values) == 2:
153             if delta_key not in st.session_state:
154                 st.session_state[delta_key] = definition_values[0]
155             if step_key not in st.session_state:
156                 st.session_state[step_key] = definition_values[1]
157
158
159 def seed_discrete_incremental_widget_defaults(action_variables: list[str]) -> None:
160     """Empleneu prèviament els ginyos d'embolcall incrementals discrets per variable des de l'estat desat.
161
162     Només escriu a claus d'estat de sessió que encara no existeixen.
163     """
164     stored_initial_values = st.session_state.get(
165         training_form_key("discrete_incremental_initial_values"),
166         [],
167     ) or []
168     for action_index, variable_name in enumerate(action_variables):

```

```

169 widget_key = training_form_key(f"disc_incr_init_{variable_name}")
170 if widget_key not in st.session_state and action_index < len(stored_initial_values):
171     st.session_state[widget_key] = stored_initial_values[action_index]

```

E.18 backend/entrenar_agent_utils.py

Listing 39: backend/entrenar_agent_utils.py

```

1 """Funcions comunes per preparar arguments i valors que es mostren durant l'entrenament."""
2
3 from __future__ import annotations
4
5 import inspect
6 from typing import Any, Dict, Tuple
7
8
9 def filter_supported_kwargs(callable_obj: Any, kwargs: Dict[str, Any]) -> Tuple[Dict[str, Any], List[str]]:
10     """Kwargs compatibles amb el filtre."""
11     try:
12         signature_target = callable_obj.__init__ if inspect.isclass(callable_obj) else callable_obj
13         signature = inspect.signature(signature_target)
14     except (TypeError, ValueError):
15         return dict(kwargs), []
16
17     parameters = signature.parameters
18     if any(parameter.kind == inspect.Parameter.VAR_KEYWORD for parameter in parameters.values()):
19         return dict(kwargs), []
20
21     allowed_kwargs = {
22         name
23         for name, parameter in parameters.items()
24         if name != "self"
25         and parameter.kind in (inspect.Parameter.POSITIONAL_OR_KEYWORD, inspect.Parameter.KEYWORD_ONLY)
26     }
27     filtered_kwargs = {key: value for key, value in kwargs.items() if key in allowed_kwargs}
28     dropped_kwargs = [key for key in kwargs if key not in allowed_kwargs]
29     return filtered_kwargs, dropped_kwargs
30
31
32 def format_ui_value(value: Any) -> str:
33     """Formata el valor ui."""
34     if value is None:
35         return "-"
36     if isinstance(value, dict):
37         return ", ".join(f"{key}: {format_ui_value(item)}" for key, item in value.items()) if value else "-"
38     if isinstance(value, (list, tuple, set)):
39         return ", ".join(format_ui_value(item) for item in value) if value else "-"
40     return str(value)

```

E.19 backend/entrenar_agent_wrappers.py

Listing 40: backend/entrenar_agent_wrappers.py

```

1 """Configuració i aplicació de wrapper per a entorns d'entrenament."""
2
3 from __future__ import annotations
4
5 import csv
6 from typing import Any, Dict, List, Sequence
7
8 import numpy as np
9 from gymnasium import Wrapper
10 from gymnasium.spaces import Box
11 from sinergym.utils.wrappers import (
12     DatetimeWrapper,
13     DeltaTempWrapper,
14     DiscreteIncrementalWrapper,
15     EnergyCostWrapper,
16     IncrementalWrapper,
17     MultiObsWrapper,
18     NormalizeAction,
19     NormalizeObservation,
20     OfficeGridStorageSmoothingActionConstraintsWrapper,
21     PreviousObservationWrapper,
22     ReduceObservationWrapper,
23     VariabilityContextWrapper,
24     WeatherForecastingWrapper,
25 )
26
27 from backend.entrenar_agent_constants import FIXED_WRAPPER_ROWS
28 from backend.entrenar_agent_utils import format_ui_value
29
30
31 class EnergyCostFileLogger(Wrapper):

```

```

32 """Logger adicional (opcional) que escriu cost per pas en un CSV independent."""
33
34 def __init__(self, env, filename="energy_cost_log.csv"):
35     """Inicialitza la instància."""
36     super().__init__(env)
37     self.csvfile = open(filename, mode="w", newline="")
38     self.writer = csv.writer(self.csvfile)
39     self.writer.writerow(["step", "elèctricity_cost_eur_per_kwh", "energy_cost_eur"])
40     self.step_count = 0
41
42 def step(self, action):
43     """Step."""
44     obs, reward, terminated, truncated, info = self.env.step(action)
45     self.step_count += 1
46     cost_per_kwh = info.get("elèctricity_cost", 0.0)
47     energy_rate = info.get("HVAC_elèctricity_demand_rate", 0.0)
48     cost_total = cost_per_kwh * energy_rate
49     self.writer.writerow([self.step_count, cost_per_kwh, cost_total])
50     return obs, reward, terminated, truncated, info
51
52 def reset(self, **kwargs):
53     """Restableix."""
54     self.step_count = 0
55     return self.env.reset(**kwargs)
56
57 def close(self):
58     """Close."""
59     self.csvfile.close()
60     if hasattr(self.env, "close"):
61         self.env.close()
62
63 def wrapper_details(params: Dict[str, Any]) -> str:
64     """Detalls de l'embolcall."""
65     if not params:
66         return "-"
67     return "; ".join(f"{key}={format_ui_value(value)}" for key, value in params.items())
68
69
70 def build_wrapper_summary_rows(wrapper_configs: Sequence[Dict[str, Any]]) -> List[Dict[str, str]]:
71     """Crea files de resum d'embolcall per al flux Studio."""
72     rows = [dict(row) for row in FIXED_WRAPPER_ROWS]
73     for config in wrapper_configs:
74         rows.append(
75             {
76                 "Wrapper": config["name"],
77                 "Actiu": "Si",
78                 "Detall": wrapper_details(config.get("params", {})),
79             }
80         )
81     return rows
82
83
84 def apply_training_wrappers(env: Any, wrapper_configs: Sequence[Dict[str, Any]]) -> Any:
85     """Aplica embolcalls d'entrenament."""
86     # L'ordre ve guardat a la configuració i s'ha de respectar igual en training,
87     # avaluació i interacció. Si es canvia, canvia també l'espai d'observació/acció.
88     for wrapper_config in wrapper_configs:
89         name = wrapper_config["name"]
90         params = wrapper_config.get("params", {})
91         if name == "PreviousObservationWrapper":
92             env = PreviousObservationWrapper(env, **params)
93         elif name == "DatetimeWrapper":
94             env = DatetimeWrapper(env)
95         elif name == "WeatherForecastingWrapper":
96             env = WeatherForecastingWrapper(env, **params)
97         elif name == "EnergyCostWrapper":
98             env = EnergyCostWrapper(env, **params)
99         elif name == "DeltaTempWrapper":
100             env = DeltaTempWrapper(env, **params)
101         elif name == "ReduceObservationWrapper":
102             env = ReduceObservationWrapper(env, **params)
103         elif name == "MultiObsWrapper":
104             env = MultiObsWrapper(env, **params)
105         elif name == "NormalizeObservation":
106             env = NormalizeObservation(env, **params)
107         elif name == "VariabilityContextWrapper":
108             # Aquest wrapper espera un Box real, no només els límits serialitzats al YAML.
109             context_space = Box(
110                 low=np.array(params["context_low"], dtype=np.float32),
111                 high=np.array(params["context_high"], dtype=np.float32),
112                 dtype=np.float32,
113             )
114             env = VariabilityContextWrapper(
115                 env,
116                 context_space=context_space,
117                 delta_value=params["delta_value"],
118                 step_frequency_range=tuple(params["step_frequency_range"]),
119             )
120         elif name == "OfficeGridStorageSmoothingActionConstraintsWrapper":
121             env = OfficeGridStorageSmoothingActionConstraintsWrapper(env)
122         elif name == "IncrementalWrapper":

```



```

124     env = IncrementalWrapper(env, **params)
125 elif name == "DiscreteIncrementalWrapper":
126     env = DiscreteIncrementalWrapper(env, **params)
127 elif name == "NormalizeAction":
128     # NormalizeAction només té sentit en espais continus; així evitem embolicar
129     # un Discrete per accident quan es reutilitza una configuració antiga.
130     if isinstance(env.action_space, Box):
131         env = NormalizeAction(env, **params)
132 elif name == "EnergyCostFileLogger":
133     env = EnergyCostFileLogger(env, **params)
134 return env
135
136
137 def build_energy_cost_wrapper_reward_kwargs(
138     options: Dict[str, Any],
139     temp_vars: Sequence[str],
140     energy_vars: Sequence[str],
141 ) -> Dict[str, Any]:
142     """Crea kwargs de recompensa d'embolcall de costos energètics per al flux Studio."""
143     return {
144         "temperature_variables": list(temp_vars),
145         "energy_variables": list(energy_vars),
146         "energy_cost_variables": ["energy_cost"],
147         "range_comfort_winter": tuple(options["range_winter"]),
148         "range_comfort_summer": tuple(options["range_summer"]),
149         "energy_weight": options["energy_weight"],
150         "temperature_weight": options["temperature_weight"],
151         "lambda_energy": options["lambda_energy"],
152         "lambda_temperature": options["lambda_temperature"],
153         "lambda_energy_cost": options["lambda_energy_cost"],
154     }
155
156
157 def build_wrapper_configs(
158     options: Dict[str, Any],
159     energy_cost_reward_kwargs: Dict[str, Any] | None = None,
160     recalculate_energy_cost_reward: bool = True,
161 ) -> List[Dict[str, Any]]:
162     """Crea configuracions d'embolcall per al flux Studio."""
163     # Aquesta llista és el contracte que després es desa amb el model. Per això guardem
164     # noms i paràmetres simples, fàcils de reconstruir en avaluació.
165     wrapper_configs: List[Dict[str, Any]] = []
166     if options["enable_previous_wrapper"]:
167         wrapper_configs.append(
168             {
169                 "name": "PreviousObservationWrapper",
170                 "params": {"previous_variables": options["previous_variables"]},
171             }
172         )
173     if options["enable_datetime_wrapper"]:
174         wrapper_configs.append({"name": "DatetimeWrapper", "params": {}})
175     if options["enable_weather_forecast_wrapper"]:
176         wrapper_configs.append(
177             {
178                 "name": "WeatherForecastingWrapper",
179                 "params": {
180                     "n": options["weather_forecast_n"],
181                     "delta": options["weather_forecast_delta"],
182                     "columns": options["weather_forecast_columns"],
183                 },
184             }
185         )
186     if options["use_energy_cost"]:
187         energy_cost_params = {
188             "energy_cost_data_path": options["energy_cost_path"],
189             "recalculate_reward": recalculate_energy_cost_reward,
190         }
191         if energy_cost_reward_kwargs is not None:
192             energy_cost_params["reward_kwargs"] = energy_cost_reward_kwargs
193         wrapper_configs.append(
194             {
195                 "name": "EnergyCostWrapper",
196                 "params": energy_cost_params,
197             }
198         )
199     if options["enable_delta_temp_wrapper"]:
200         wrapper_configs.append(
201             {
202                 "name": "DeltaTempWrapper",
203                 "params": {
204                     "temperature_variables": options["delta_temp_variables"],
205                     "setpoint_variables": options["delta_setpoint_variables"],
206                 },
207             }
208         )
209     if options["enable_reduce_obs_wrapper"]:
210         wrapper_configs.append(
211             {
212                 "name": "ReduceObservationWrapper",
213                 "params": {"obs_reduction": options["reduced_observations"]},
214             }
215         )

```



```

216 if options["enable_multi_obs_wrapper"]:
217     wrapper_configs.append(
218         {
219             "name": "MultiObsWrapper",
220             "params": {"n": options["multi_obs_n"], "flatten": options["multi_obs_flatten"]},
221         }
222     )
223 if options["enable_normalize_obs_wrapper"]:
224     wrapper_configs.append(
225         {
226             "name": "NormalizeObservation",
227             "params": {
228                 "automatic_update": options["normalize_obs_auto"],
229                 "epsilon": options["normalize_obs_epsilon"],
230                 "mean": options["normalize_obs_mean"] or None,
231                 "var": options["normalize_obs_var"] or None,
232             },
233         }
234     )
235 if options.get("enable_variability_context_wrapper"):
236     # Els límits del context es guarden com a llistes perquè siguin serialitzables.
237     # El Box es reconstrueix quan s'apliquen els wrappers.
238     wrapper_configs.append(
239         {
240             "name": "VariabilityContextWrapper",
241             "params": {
242                 "context_low": list(options.get("variability_context_low") or []),
243                 "context_high": list(options.get("variability_context_high") or []),
244                 "delta_value": options.get("variability_delta_value", 1.0),
245                 "step_frequency_range": (
246                     options.get("variability_step_frequency_min", 96),
247                     options.get("variability_step_frequency_max", 96 * 7),
248                 ),
249             },
250         }
251     )
252 if options["enable_incremental_wrapper"]:
253     wrapper_configs.append(
254         {
255             "name": "IncrementalWrapper",
256             "params": {
257                 "incremental_variables_definition": options["incremental_definition"],
258                 "initial_values": options["incremental_initial_values"],
259             },
260         }
261     )
262 if options["enable_discrete_incremental_wrapper"]:
263     wrapper_configs.append(
264         {
265             "name": "DiscreteIncrementalWrapper",
266             "params": {
267                 "initial_values": options["discrete_incremental_initial_values"],
268                 "delta_temp": options["discrete_incremental_delta"],
269                 "step_temp": options["discrete_incremental_step"],
270             },
271         }
272     )
273 if (
274     options["enable_normalize_action"]
275     and options.get("is_continuous", True)
276     and not options["enable_incremental_wrapper"]
277     and not options["enable_discrete_incremental_wrapper"]
278 ):
279     # Els wrappers incrementals ja redefeixen l'escala de l'acció; combinar-los amb
280     # NormalizeAction fa que l'agent aprengui en una escala difícil d'interpretar.
281     wrapper_configs.append(
282         {
283             "name": "NormalizeAction",
284             "params": {"normalize_range": (-1.0, 1.0)},
285         }
286     )
287 if options.get("enable_storage_smoothing_wrapper"):
288     wrapper_configs.append(
289         {
290             "name": "OfficeGridStorageSmoothingActionConstraintsWrapper",
291             "params": {},
292         }
293     )
294 if options["use_file_logger"]:
295     wrapper_configs.append(
296         {
297             "name": "EnergyCostFileLogger",
298             "params": {"filename": options["file_logger_name"]},
299         }
300     )
301 return wrapper_configs

```

E.20 backend/epw_figures.py

Listing 41: backend/epw_figures.py

```
1  """Plotly creadors de figures per al visor de clima EPW."""
2
3  from __future__ import annotations
4
5  import math
6
7  import pandas as pd
8  import plotly.graph_objects as go
9  from plotly.subplots import make_subplots
10
11 from backend import visor_epw_backend as epw_backend
12
13
14 MONTH_ORDER = epw_backend.MONTH_ORDER
15 EPW_VARIABLES = epw_backend.EPW_VARIABLES
16 DEG = epw_backend.DEG
17 DEG_C = epw_backend.DEG_C
18 WH_PER_M2 = epw_backend.WH_PER_M2
19 KWH_PER_M2 = epw_backend.KWH_PER_M2
20 MID_DOT = epw_backend.MID_DOT
21 WIND_DIRECTION_LABELS = epw_backend.WIND_DIRECTION_LABELS
22 variable_label = epw_backend.variable_label
23
24 MONTH_TICK_POSITIONS = [1, 32, 60, 91, 121, 152, 182, 213, 244, 274, 305, 335]
25
26
27 def ui_text(value: object) -> str:
28     """Retorna el text segur per a la pantalla, reparant el mojibake habitual de les metadades EPW."""
29
30     text = "" if value is None else str(value)
31     for _ in range(2):
32         if not any(marker in text for marker in ("Ã", "À", "à")):
33             break
34         try:
35             repaired = text.encode("latin-1").decode("utf-8")
36         except (UnicodeEncodeError, UnicodeDecodeError):
37             break
38         if repaired == text:
39             break
40         text = repaired
41     return text
42
43
44 def build_active_month_axis(data_frame: pd.DataFrame) -> tuple[list[int], list[str]]:
45     """Retorna les posicions de marca del mes que coincideixen amb les dades EPW visibles actualment."""
46
47     month_frame = (
48         data_frame[["month", "month_label", "day_of_year"]]
49         .dropna(subset=["month", "day_of_year"])
50         .sort_values(["month", "day_of_year"])
51         .drop_duplicates(subset=["month"])
52     )
53     if month_frame.empty:
54         return MONTH_TICK_POSITIONS, MONTH_ORDER
55
56     tick_values = month_frame["day_of_year"].astype(int).tolist()
57     tick_labels = month_frame["month_label"].astype(str).tolist()
58     return tick_values, tick_labels
59
60
61 def apply_figure_style(fig: go.Figure, title: str, *, legend: bool = True, height: int = 400) -> go.Figure:
62     """Aplica l'estil visual compartit a una figura Plotly."""
63
64     top_margin = 82 if legend else 58
65     fig.update_layout(
66         title=dict(
67             text=ui_text(title),
68             x=0.02,
69             xanchor="left",
70             y=0.98,
71             yanchor="top",
72             font=dict(size=15),
73         ),
74         height=height,
75         margin=dict(l=12, r=12, t=top_margin, b=18),
76         paper_bgcolor="rgba(0,0,0,0)",
77         plot_bgcolor="rgba(247,249,254,0.75)",
78         hovermode="x unified",
79         legend=dict(
80             orientation="h",
81             yanchor="bottom",
82             y=1.02,
83             xanchor="right",
84             x=1,
85             font=dict(size=12),
86             bgcolor="rgba(255,255,255,0.78)",
87     ),
```

```

88     showlegend=legend,
89     font=dict(family="Bahnschrift, Aptos, Segoe UI, sans-serif", color="#17233c"),
90 )
91 fig.update_xaxes(showgrid=False, zeroline=False)
92 fig.update_yaxes(gridcolor="rgba(204, 214, 235, 0.55)", zeroline=False)
93 return fig
94
95
96 def build_focus_timeseries_figure(series_frame: pd.DataFrame, variable_key: str, aggregation_label: str) -> go.Figure:
97     """Crea la figura de sèrie temporal agregada principal per a la variable EPW seleccionada per al flux Studio."""
98
99     fig = go.Figure()
100     fig.add_trace(
101         go.Scatter(
102             x=series_frame["timestamp"],
103             y=series_frame[variable_key],
104             mode="lines",
105             name=ui_text(variable_label(variable_key, aggregated=aggregation_label != "Hora")),
106             line=dict(color=EPW_VARIABLES[variable_key]["color"], width=2.3),
107             fill="tozeroy" if EPW_VARIABLES[variable_key]["aggregate"] == "sum" else None,
108             fillcolor="rgba(95, 84, 249, 0.10)" if EPW_VARIABLES[variable_key]["aggregate"] == "sum" else None,
109             connectgaps=False,
110         )
111     )
112     fig.update_xaxes(title_text="Calendari EPW")
113     return apply_figure_style(fig, f"S\u00e8rie temporal {MID_DOT} {aggregation_label}", height=390)
114
115
116 def build_monthly_climate_figure(monthly_frame: pd.DataFrame) -> go.Figure:
117     """Crea una figura mensual de comparaci\u00f3 de temperatura i humitat per al flux Studio."""
118
119     fig = make_subplots(specs=[[{"secondary_y": True}]]))
120     fig.add_trace(
121         go.Scatter(
122             x=monthly_frame["month_label"],
123             y=monthly_frame["dry_bulb_max"],
124             mode="lines",
125             line=dict(color="rgba(242, 107, 91, 0.18)", width=0.1),
126             showlegend=False,
127             hoverinfo="skip",
128         ),
129         secondary_y=False,
130     )
131     fig.add_trace(
132         go.Scatter(
133             x=monthly_frame["month_label"],
134             y=monthly_frame["dry_bulb_min"],
135             mode="lines",
136             line=dict(color="rgba(242, 107, 91, 0.18)", width=0.1),
137             fill="tonexty",
138             fillcolor="rgba(242, 107, 91, 0.14)",
139             name="Rang",
140         ),
141         secondary_y=False,
142     )
143     fig.add_trace(
144         go.Scatter(
145             x=monthly_frame["month_label"],
146             y=monthly_frame["dry_bulb_mean"],
147             mode="lines+markers",
148             name="Temp. seca",
149             line=dict(color="#f26b5b", width=2.5),
150         ),
151         secondary_y=False,
152     )
153     fig.add_trace(
154         go.Bar(
155             x=monthly_frame["month_label"],
156             y=monthly_frame["relative_humidity_mean"],
157             name="Humitat",
158             marker_color="rgba(79, 142, 247, 0.65)",
159         ),
160         secondary_y=True,
161     )
162     apply_figure_style(fig, "Climatologia mensual", height=380)
163     fig.update_yaxes(title_text=f"Temperatura ({DEG_C})", secondary_y=False)
164     fig.update_yaxes(title_text="Humitat relativa (%)", secondary_y=True)
165     return fig
166
167
168 def build_monthly_solar_figure(monthly_frame: pd.DataFrame) -> go.Figure:
169     """Crea un gr\u00e0fic de barres de radiaci\u00f3 solar acumulada mensual per al flux Studio."""
170
171     fig = go.Figure()
172     fig.add_trace(
173         go.Bar(
174             x=monthly_frame["month_label"],
175             y=monthly_frame["global_horizontal_radiation_kwh_m2"],
176             name="GHI",
177             marker_color="#f3a738",
178         )
179     )

```

```

180 fig.add_trace(
181     go.Bar(
182         x=monthly_frame["month_label"],
183         y=monthly_frame["direct_normal_radiation_kwh_m2"],
184         name="DNI",
185         marker_color="#ef7d57",
186     )
187 )
188 fig.add_trace(
189     go.Bar(
190         x=monthly_frame["month_label"],
191         y=monthly_frame["diffuse_horizontal_radiation_kwh_m2"],
192         name="DHI",
193         marker_color="#f9cb40",
194     )
195 )
196 fig.update_layout(barmode="group")
197 fig.update_yaxes(title_text=KWH_PER_M2)
198 return apply_figure_style(fig, "Radiaci\u00f3 solar mensual", height=380)
199
200
201 def build_daily_temperature_band_figure(daily_frame: pd.DataFrame) -> go.Figure:
202     """Crea la figura de la banda de temperatura di\u00e0ria amb valors m\u00ednims, mitjans i m\u00e0xims per al flux Studio."""
203
204     fig = go.Figure()
205     fig.add_trace(
206         go.Scatter(
207             x=daily_frame["date"],
208             y=daily_frame["dry_bulb_max"],
209             mode="lines",
210             line=dict(color="rgba(242, 107, 91, 0.16)", width=0.1),
211             showlegend=False,
212             hoverinfo="skip",
213         )
214     )
215     fig.add_trace(
216         go.Scatter(
217             x=daily_frame["date"],
218             y=daily_frame["dry_bulb_min"],
219             mode="lines",
220             line=dict(color="rgba(79, 142, 247, 0.16)", width=0.1),
221             fill="tonexty",
222             fillcolor="rgba(95, 84, 249, 0.16)",
223             name="Rang diari",
224         )
225     )
226     fig.add_trace(
227         go.Scatter(
228             x=daily_frame["date"],
229             y=daily_frame["dry_bulb_mean"],
230             mode="lines",
231             name="Mitjana",
232             line=dict(color="#5f54f9", width=2),
233         )
234     )
235     fig.update_yaxes(title_text=DEG_C)
236     return apply_figure_style(fig, "Banda di\u00f0ria de temperatura", height=360)
237
238
239 def build_hourly_profile_figure(hourly_frame: pd.DataFrame) -> go.Figure:
240     """Crea un perfil horari mitj\u00e0 de temperatura, punt de rosada i humitat per al flux Studio."""
241
242     fig = make_subplots(specs=[[{"secondary_y": True}]]))
243     fig.add_trace(
244         go.Scatter(
245             x=hourly_frame["hour_start"],
246             y=hourly_frame["dry_bulb_mean"],
247             mode="lines+markers",
248             name="Temp.",
249             line=dict(color="#f26b5b", width=2.4),
250         ),
251         secondary_y=False,
252     )
253     fig.add_trace(
254         go.Scatter(
255             x=hourly_frame["hour_start"],
256             y=hourly_frame["dew_point_mean"],
257             mode="lines",
258             name="Rosada",
259             line=dict(color="#ffb347", width=2, dash="dot"),
260         ),
261         secondary_y=False,
262     )
263     fig.add_trace(
264         go.Scatter(
265             x=hourly_frame["hour_start"],
266             y=hourly_frame["relative_humidity_mean"],
267             mode="lines",
268             name="Humitat",
269             line=dict(color="#4f8ef7", width=2.2),
270         ),
271         secondary_y=True,

```

```

272 )
273 apply_figure_style(fig, "Perfil horari mitj\u00e0", height=360)
274 fig.update_xaxes(title_text="Hora del dia")
275 fig.update_yaxes(title_text=f"Temperatura ({DEG_C})", secondary_y=False)
276 fig.update_yaxes(title_text="Humitat relativa (%)", secondary_y=True)
277 return fig
278
279
280 def build_heatmap_figure(heatmap_frame: pd.DataFrame, variable_key: str) -> go.Figure:
281     """Crea un mapa de calor mes a hora per a la variable EPW seleccionada."""
282
283     fig = go.Figure(
284         data=[
285             go.Heatmap(
286                 z=heatmap_frame.values,
287                 x=[int(hour) for hour in heatmap_frame.columns],
288                 y=list(heatmap_frame.index),
289                 colorscale="Blues" if variable_key == "relative_humidity_pct" else "Turbo",
290                 colorbar=dict(title=ui_text(variable_label(variable_key))),
291                 hovertemplate="Mes {y}<br>Hora {x}:00<br>Valor {z:.2f}<extra></extra>",
292             )
293         ]
294     )
295     fig.update_xaxes(title_text="Hora del dia")
296     fig.update_yaxes(title_text="Mes")
297     return apply_figure_style(
298         fig,
299         f"Mapa de calor {MID_DOT} {ui_text(EPW_VARIABLES[variable_key]['label'])}",
300         legend=False,
301         height=390,
302     )
303
304
305 def build_annual_heatmap_figure(
306     heatmap_frame: pd.DataFrame,
307     variable_key: str,
308     *,
309     tick_values: list[int] | None = None,
310     tick_labels: list[str] | None = None,
311 ) -> go.Figure:
312     """Crea un mapa de calor anual hora a dia per a l'escaneig a escala del panell."""
313
314     title_map = {
315         "dry_bulb_c": f"Temperature ({DEG_C})",
316         "direct_normal_radiation_wh_m2": f"Direct Radiation ({WH_PER_M2})",
317         "wind_speed_m_s": "Wind Speed (m/s)",
318         "relative_humidity_pct": "Relative Humidity (%)",
319     }
320     color_scale_map = {
321         "dry_bulb_c": "RdBu_r",
322         "direct_normal_radiation_wh_m2": "YlOrRd",
323         "wind_speed_m_s": "Greens",
324         "relative_humidity_pct": "Blues",
325     }
326
327     # Redueix l'escala a quantils robusts perquè els valors atípics aïllats no aplanin el mapa de calor.
328     numeric_values = pd.DataFrame(heatmap_frame).to_numpy(dtype=float)
329     flattened = pd.Series(numeric_values.reshape(-1)).dropna()
330     if flattened.empty:
331         return go.Figure()
332
333     zmin = float(flattened.quantile(0.02))
334     zmax = float(flattened.quantile(0.98))
335     if variable_key == "dry_bulb_c":
336         bound = max(abs(zmin), abs(zmax), 1.0)
337         zmin = -bound
338         zmax = bound
339     elif zmin == zmax:
340         zmin -= 1.0
341         zmax += 1.0
342
343     x_values = [int(column) for column in heatmap_frame.columns]
344     fig = go.Figure(
345         data=[
346             go.Heatmap(
347                 z=heatmap_frame.values,
348                 x=x_values,
349                 y=list(heatmap_frame.index),
350                 zmin=zmin,
351                 zmax=zmax,
352                 colorscale=color_scale_map.get(variable_key, "Turbo"),
353                 colorbar=dict(title=ui_text(EPW_VARIABLES[variable_key]['unit']), len=0.78, thickness=11),
354                 hovertemplate="Dia {x}<br>Hora {y}:00<br>Valor {z:.2f}<extra></extra>",
355             )
356         ]
357     )
358     fig.update_layout(
359         title=ui_text(title_map.get(variable_key, variable_label(variable_key))),
360         height=220,
361         margin=dict(l=10, r=10, t=40, b=10),
362         paper_bgcolor="rgba(0,0,0,0)",
363         plot_bgcolor="rgba(247,249,254,0.75)",

```

```

364     font=dict(family="Bahnschrift, Aptos, Segoe UI, sans-serif", color="#17233c"),
365 )
366 fig.update_xaxes(
367     tickmode="array",
368     tickvals=tick_values or MONTH_TICK_POSITIONS,
369     ticktext=tick_labels or MONTH_ORDER,
370     showgrid=False,
371     zeroline=False,
372     range=[min(x_values), max(x_values)] if x_values else None,
373 )
374 fig.update_yaxes(
375     title_text="Hora",
376     tickmode="array",
377     tickvals=[0, 6, 12, 18, 23],
378     showgrid=False,
379     zeroline=False,
380     autorange="reversed",
381 )
382 return fig
383
384
385 def build_comfort_radiation_figure(radiation_frame: pd.DataFrame) -> go.Figure:
386     """Crea un gràfic de radiació polar benefici-dany per a les orientacions de façana per al flux Studio."""
387
388     if radiation_frame.empty:
389         return go.Figure()
390
391     ordered_frame = radiation_frame.sort_values(["radius_start", "theta_deg"]).reset_index(drop=True)
392     band_order = list(dict.fromkeys(ordered_frame["altitude_band"]).tolist())
393     # Utilitza una gamma de colors simètrica perquè els valors positius i negatius codifiquen efectes de confort oposats.
394     max_abs = max(
395         abs(float(ordered_frame["comfort_radiation_kwh_m2"].min())),
396         abs(float(ordered_frame["comfort_radiation_kwh_m2"].max())),
397         1.0,
398     )
399
400     fig = go.Figure()
401     for band_index, altitude_band in enumerate(band_order):
402         # Cada banda es dibuixa com un segment polar anular a l'altitud solar corresponent.
403         band_frame = ordered_frame[ordered_frame["altitude_band"] == altitude_band]
404         fig.add_trace(
405             go.Barpolar(
406                 r=band_frame["radius_size"],
407                 theta=band_frame["theta_deg"],
408                 base=band_frame["radius_start"],
409                 width=[22.5] * len(band_frame),
410                 marker=dict(
411                     color=band_frame["comfort_radiation_kwh_m2"],
412                     colorscale=[[0.0, "#1f9d55"], [0.5, "#f4ddaf"], [1.0, "#d94d41"]],
413                     cmin=-max_abs,
414                     cmax=max_abs,
415                     showscale=band_index == len(band_order) - 1,
416                     colorbar=dict(title="KWH_PER_M2", len=0.78, thickness=11),
417                 ),
418                 customdata=list(
419                     zip(
420                         band_frame["altitude_band"],
421                         band_frame["comfort_radiation_kwh_m2"],
422                         band_frame["incident_radiation_kwh_m2"],
423                     )
424                 ),
425                 hovertemplate=(
426                     "%{theta}<br>"
427                     "Banda solar %{customdata[0]}<br>"
428                     f"Balanç de confort %{customdata[1]:.1f}} {KWH_PER_M2}<br>"
429                     f"Radiació captada %{customdata[2]:.1f}} {KWH_PER_M2}<extra></extra>"
430                 ),
431                 opacity=0.94,
432                 showlegend=False,
433             )
434         )
435
436     fig.update_layout(
437         title=f"Benefit/Harm Radiation ({KWH_PER_M2})",
438         height=390,
439         margin=dict(l=10, r=10, t=46, b=10),
440         paper_bgcolor="rgba(0,0,0,0)",
441         font=dict(family="Bahnschrift, Aptos, Segoe UI, sans-serif", color="#17233c"),
442         polar=dict(
443             bgcolor="rgba(247,249,254,0.75)",
444             radialaxis=dict(
445                 range=[0, 90],
446                 tickmode="array",
447                 tickvals=[15, 30, 45, 60, 75, 90],
448                 ticktext=[f"15{DEG}", f"30{DEG}", f"45{DEG}", f"60{DEG}", f"75{DEG}", f"90{DEG}"],
449                 gridcolor="rgba(204,214,235,0.45)",
450             ),
451             angularaxis=dict(
452                 direction="clockwise",
453                 rotation=90,
454                 tickmode="array",
455                 tickvals=[0, 45, 90, 135, 180, 225, 270, 315],

```

```

456         ticktext=["N", "NE", "E", "SE", "S", "SW", "W", "NW"],
457     ),
458 ),
459 )
460 return fig
461
462
463 def build_monthly_wind_rose_grid_figure(monthly_rose_tables: dict[str, pd.DataFrame]) -> go.Figure:
464     """Construeix una graella compacta de roses dels vents mensuals."""
465
466     month_items = list(monthly_rose_tables.items())
467     if not month_items:
468         return go.Figure()
469
470     max_frequency = max(
471         (float(frame["frequency_pct"].max()) for _, frame in month_items if not frame.empty),
472         default=1.0,
473     )
474     max_speed = max((float(frame["mean_speed"].max()) for _, frame in month_items if not frame.empty), default=1.0)
475     if max_frequency <= 0:
476         max_frequency = 1.0
477     if max_speed <= 0:
478         max_speed = 1.0
479
480     # Fem una graella màxima de quatre columnes perquè els dotze mesos càpiguen sense
481     # convertir cada rosa del vent en una miniatura il·legible.
482     columns = min(4, max(1, len(month_items)))
483     rows = max(1, math.ceil(len(month_items) / columns))
484     specs = [{"type": "polar"} for _ in range(columns)] for _ in range(rows)]
485     subplot_titles = [month_label for month_label, _ in month_items]
486     empty_slots = rows * columns - len(subplot_titles)
487     if empty_slots > 0:
488         subplot_titles.extend([""] * empty_slots)
489
490     fig = make_subplots(
491         rows=rows,
492         cols=columns,
493         specs=specs,
494         subplot_titles=subplot_titles,
495         horizontal_spacing=0.05,
496         vertical_spacing=0.12,
497     )
498
499     for month_index, (month_label, rose_frame) in enumerate(month_items):
500         row = month_index // columns + 1
501         col = month_index % columns + 1
502         ordered_frame = rose_frame.copy()
503         # Ordenem les direccions manualment; si no, Plotly les podria ordenar alfabèticament.
504         ordered_frame["direction"] = pd.Categorical(
505             ordered_frame["direction"],
506             categories=list(WIND_DIRECTION_LABELS),
507             ordered=True,
508         )
509         ordered_frame = ordered_frame.sort_values("direction")
510
511         fig.add_trace(
512             go.BarPolar(
513                 r=ordered_frame["frequency_pct"],
514                 theta=ordered_frame["direction"].astype(str),
515                 marker=dict(
516                     color=ordered_frame["mean_speed"],
517                     colorscale="Turbo",
518                     cmin=0.0,
519                     cmx=max_speed,
520                     showscale=month_index == len(month_items) - 1,
521                     colorbar=dict(title="m/s", len=0.44, thickness=10, y=0.18),
522                 ),
523                 opacity=0.9,
524                 hovertemplate=(
525                     f"{month_label}<br>"
526                     f"%{theta}<br>"
527                     f"Freq. %{r:.1f}%<br>"
528                     f"Vent mitjà %{marker.color:.2f} m/s<extra></extra>"
529                 ),
530                 showlegend=False,
531             ),
532             row=row,
533             col=col,
534         )
535
536     fig.update_layout(
537         title="Wind Rose",
538         height=245 * rows + 105,
539         margin=dict(l=10, r=10, t=70, b=12),
540         paper_bgcolor="rgba(0,0,0,0)",
541         font=dict(family="Bahnschrift, Aptos, Segoe UI, sans-serif", color="#17233c"),
542     )
543     fig.update_annotations(
544         font=dict(size=11, color="#17233c"),
545         yshift=14,
546     )
547

```

```

548 for polar_index in range(1, len(month_items) + 1):
549     polar_name = "polar" if polar_index == 1 else f"polar{polar_index}"
550     fig.layout[polar_name].update(
551         bgcolor="rgba(247,249,254,0.75)",
552         radialaxis=dict(
553             range=[0, max_frequency],
554             showticklabels=False,
555             ticks="",
556             gridcolor="rgba(204,214,235,0.35)",
557         ),
558         angularaxis=dict(
559             direction="clockwise",
560             rotation=90,
561             tickmode="array",
562             tickvals=[0, 90, 180, 270],
563             ticktext=["N", "E", "S", "W"],
564             tickfont=dict(size=8),
565         ),
566     )
567
568 return fig

```

E.21 backend/gestionar_arxius_backend.py

Listing 42: backend/gestionar_arxius_backend.py

```

1  """Gestió de fitxers per a actius, models i dades meteorològiques de Studio.
2
3  Aquest mòdul fa una còpia de seguretat de la pàgina del navegador de fitxers. Formata metadades del sistema de fitxers,
4  resumeix els fitxers meteorològics EPW, descobreix artefactes de model entrenats i s'aplica
5  operacions de còpia/supressió limitades utilitzades per la UI de Streamlit.
6  """
7
8  import os
9  import shutil
10 from dataclasses import dataclass
11 from datetime import datetime
12 from pathlib import Path
13
14 import pandas as pd
15 import streamlit as st
16
17
18 ROOT_PATH = Path.cwd().resolve()
19 DATA_BTC_PATH = ROOT_PATH / 'sinergym' / 'data' / 'buildings'
20 DATA_WEA_PATH = ROOT_PATH / 'sinergym' / 'data' / 'weather'
21 MODEL_FILE_EXTENSIONS = (".zip", ".pkl")
22 MODEL_ROOT_DIRECTORY_NAMES = {"models", "trainings"}
23
24
25 @dataclass(frozen=True)
26 class ExplorerItem:
27     """Fila del navegador de fitxers serialitzable que es mostra a la pàgina de gestió d'actius."""
28     name: str
29     path: str
30     size: str
31     mtime: str
32     is_dir: bool
33
34
35 def get_size_format(b, factor=1024, suffix="B"):
36     """Retorna una mida compacta i llegible, com ara ``12.40KB``."""
37     for unit in ["", "K", "M", "G", "T", "P", "E", "Z"]:
38         if b < factor:
39             return f"{b:.2f}{unit}{suffix}"
40         b /= factor
41     return f"{b:.2f}Y{suffix}"
42
43
44 @st.cache_data(show_spinner=False)
45 def parse_epw_overview(epw_file):
46     """Llegeix les metadades d'ubicació i la temperatura mitjana del bulb sec d'un fitxer EPW."""
47     with open(epw_file, 'r', encoding='utf-8', errors='ignore') as handle:
48         lines = handle.readlines()
49
50     loc = lines[0].strip().split(',')
51     location = {
52         'city': loc[1] if len(loc) > 1 else '-',
53         'state': loc[2] if len(loc) > 2 else '-',
54         'country': loc[3] if len(loc) > 3 else '-',
55         'latitude': float(loc[6]) if len(loc) > 6 else 0.0,
56         'longitude': float(loc[7]) if len(loc) > 7 else 0.0,
57     }
58
59     df = pd.read_csv(epw_file, skiprows=8, header=None)
60     avg_temp = float(df.iloc[:, 6].mean())
61     climate_zone = 'hot' if avg_temp > 22 else 'cold' if avg_temp < 12 else 'cool'
62     return location, avg_temp, climate_zone

```



```

63
64
65 @st.cache_data(show_spinner=False)
66 def load_weather_map_rows(root_path_str):
67     """Retorna les files de fitxers meteorològics preparats per al mapa descobertes a continuació ``root_path_str``."""
68     root_path = Path(root_path_str)
69     rows = []
70     color_map = {
71         'hot': [230, 120, 40],
72         'cool': [70, 130, 180],
73         'cold': [80, 190, 220],
74     }
75
76     for epw_path in sorted(root_path.rglob('*.epw')):
77         try:
78             location, avg_temp, climate_zone = parse_epw_overview(str(epw_path))
79         except Exception:
80             continue
81
82         color = color_map.get(climate_zone, [120, 120, 120])
83         rows.append({
84             'file_name': epw_path.name,
85             'relative_path': str(epw_path.relative_to(root_path)),
86             'city': location['city'],
87             'country': location['country'],
88             'latitude': location['latitude'],
89             'longitude': location['longitude'],
90             'climate': climate_zone,
91             'avg_temp': round(avg_temp, 2),
92             'color_r': color[0],
93             'color_g': color[1],
94             'color_b': color[2],
95         })
96
97     return pd.DataFrame(rows)
98
99
100 def delete_item(path_str):
101     """Suprimeix l'element."""
102     path = Path(path_str)
103     try:
104         if path.is_dir():
105             shutil.rmtree(path)
106         else:
107             if path.exists():
108                 path.unlink()
109     except Exception as exc:
110         return str(exc)
111     return None
112
113
114 def list_explorer_items(current_path, root_path, filter_func=None):
115     """Llista els elements de l'explorador."""
116     try:
117         with os.scandir(current_path) as iterator:
118             entries = list(iterator)
119     except Exception as exc:
120         return (), str(exc)
121
122     folders = []
123     files = []
124
125     for entry in entries:
126         if filter_func:
127             if not filter_func(entry.name, entry.is_dir(), current_path == root_path):
128                 continue
129
130         try:
131             stat = entry.stat()
132             mtime = datetime.fromtimestamp(stat.st_mtime).strftime('%Y-%m-%d %H:%M')
133             size_str = "DIR" if entry.is_dir() else get_size_format(stat.st_size)
134
135             item = ExplorerItem(
136                 name=entry.name,
137                 path=entry.path,
138                 size=size_str,
139                 mtime=mtime,
140                 is_dir=entry.is_dir(),
141             )
142
143             if entry.is_dir():
144                 folders.append(item)
145             else:
146                 files.append(item)
147         except Exception:
148             continue
149
150     folders.sort(key=lambda item: item.name.lower())
151     files.sort(key=lambda item: item.name.lower())
152     return tuple(folders + files), None
153
154

```

```

155 def filter_trainings(name, is_dir, is_root):
156     """Entrenaments de filtre."""
157     if is_root:
158         return is_dir and (
159             name == "trainings"
160             or name.startswith("Eplus-")
161             or name.startswith("ppo_")
162         )
163     return True
164
165
166 def filter_models(name, is_dir, is_root):
167     """Models de filtre."""
168     name_lower = name.lower()
169     if not is_dir:
170         return name_lower.endswith(MODEL_FILE_EXTENSIONS)
171     if is_root:
172         return (
173             name_lower in MODEL_ROOT_DIRECTORY_NAMES
174             or name_lower.startswith("eplus-")
175             or name_lower.startswith("ppo_")
176             or name_lower.startswith("training-")
177         )
178     return True
179
180
181 def filter_weather(name, is_dir, is_root):
182     """Filtre el temps."""
183     if not is_dir:
184         return name.endswith(".epw")
185     return True
186
187
188 def filter_envs(name, is_dir, is_root):
189     """Filtre envs."""
190     if not is_dir:
191         return name.endswith(".epJSON") or name.endswith(".idf")
192     return True

```

E.22 backend/interaccionar_agent_backend.py

Listing 43: backend/interaccionar_agent_backend.py

```

1  """Execució en directe per provar agents entrenats dins d'un entorn Sinergym.
2
3  La pàgina de control en directe utilitza aquest mòdul per crear entorns Sinergym embolcallats,
4  carregueu les polítiques Stable-Baselines3, valideu la compatibilitat de les accions i traduïu-les
5  observacions/accions en estructures fàcils de mostrar. Evita intencionadament
6  Streamlit, de manera que les mateixes comprovacions poden donar suport a l'avaluació i la documentació.
7  """
8
9  import json
10 from functools import partial
11 from pathlib import Path
12 from typing import Any, Callable, Dict, List, Optional, Sequence, Tuple
13
14 import gymnasium as gym
15 import numpy as np
16 from stable_baselines3.common.base_class import BaseAlgorithm
17 from stable_baselines3.common.vec_env import DummyVecEnv, VecNormalize
18 from sinergym.utils.wrappers import CSVLogger, LoggerWrapper, NormalizeAction
19
20 from backend.sb3_utils import (
21     action_spaces_compatible,
22     candidate_vecnorm,
23     describe_action_space,
24     env_id_from_meta_or_name,
25     format_action_for_env,
26     load_sb3_model_bytes,
27     load_vecnormalize,
28     scan_model_zips,
29     should_apply_normalize_action,
30 )
31
32
33 def _make_runtime_env(
34     selected_env_id: str,
35     wrapper_configs: Optional[Sequence[Dict[str, Any]]] = None,
36     *,
37     trust_training_metadata: bool = False,
38     model_action_space: Any = None,
39     gym_kwargs: Optional[Dict[str, Any]] = None,
40 ) -> Any:
41     """Crea una instància d'entorn d'interacció en directe embolicada."""
42     env = gym.make(selected_env_id, **(gym_kwargs or {}))
43     if wrapper_configs:
44         from backend.entrenar_agent_backend import apply_training_wrappers

```

```

46     env = apply_training_wrappers(env, wrapper_configs)
47 elif not trust_training_metadata and should_apply_normalize_action(model_action_space, env.action_space):
48     env = NormalizeAction(env)
49 env = LoggerWrapper(env)
50 env = CSVLogger(env)
51 return env
52
53
54 def mode_requires_model(mode_label: str) -> bool:
55     """El mode requereix un model."""
56     lowered = (mode_label or "").lower()
57     return "interactiva" in lowered or "avaluaci" in lowered
58
59
60 def load_model_training_metadata(model_path: Path) -> Dict[str, Any]:
61     """Carrega les metadades d'entrenament del model."""
62     metadata_path = model_path.with_name(model_path.stem + "_metadata.json")
63     if metadata_path.exists():
64         # Format nou: el .zip va acompanyat d'un metadata explícit. Completam camps buits
65         # des de training_config per no perdre compatibilitat amb models anteriors.
66         with open(metadata_path, "r", encoding="utf-8") as handle:
67             metadata = json.load(handle)
68             training_config = metadata.get("training_config") if isinstance(metadata.get("training_config"), dict) else {}
69             if not metadata.get("env_id"):
70                 metadata["env_id"] = training_config.get("env_id")
71             if not metadata.get("reward_name"):
72                 metadata["reward_name"] = training_config.get("reward_name")
73             if "wrapper_configs" not in metadata and "wrapper_configs" in training_config:
74                 metadata["wrapper_configs"] = training_config.get("wrapper_configs", [])
75             if "meters" not in metadata and "meters" in training_config:
76                 metadata["meters"] = training_config.get("meters")
77             if "reward_kwargs" not in metadata and "reward_kwargs" in training_config:
78                 metadata["reward_kwargs"] = training_config.get("reward_kwargs")
79             metadata["metadata_source"] = str(metadata_path)
80             return metadata
81
82     training_config_path = model_path.parent / "training_config.json"
83     if training_config_path.exists():
84         # Format antic: la carpeta de training només tenia training_config.json.
85         with open(training_config_path, "r", encoding="utf-8") as handle:
86             payload = json.load(handle)
87             training_config = dict(payload.get("training_config") or {})
88             metadata = {
89                 "env_id": payload.get("env_id") or training_config.get("env_id"),
90                 "reward_name": payload.get("reward_name") or training_config.get("reward_name"),
91                 "wrapper_configs": training_config.get("wrapper_configs", []),
92                 "meters": training_config.get("meters"),
93                 "reward_kwargs": training_config.get("reward_kwargs"),
94                 "training_config": training_config,
95                 "artifact_name": payload.get("artifact_name"),
96                 "metadata_source": str(training_config_path),
97             }
98             return metadata
99
100     return {}
101
102
103 def _build_gym_kwargs_from_metadata(metadata: Dict[str, Any]) -> Dict[str, Any]:
104     """Crea kwargs de Gym a partir de metadades."""
105     training_config = metadata.get("training_config") or {}
106     meters = metadata.get("meters") or training_config.get("meters")
107     reward_name = metadata.get("reward_name") or training_config.get("reward_name")
108     reward_kwargs = metadata.get("reward_kwargs") or training_config.get("reward_kwargs") or {}
109
110     kwargs: Dict[str, Any] = {}
111     if not reward_name and not meters:
112         return kwargs
113
114     try:
115         # Reapliquem meters i reward de training per reconstruir l'entorn amb el mateix
116         # contracte que va veure el model.
117         from backend.entrenar_agent_backend import REWARD_CLASSES, with_detailed_hvac_meters
118         kwargs["meters"] = with_detailed_hvac_meters(meters or {})
119         reward_cls = REWARD_CLASSES.get(reward_name) if reward_name else None
120         if reward_cls is not None:
121             kwargs["reward"] = reward_cls
122             if reward_kwargs:
123                 kwargs["reward_kwargs"] = reward_kwargs
124     except Exception:
125         pass
126     return kwargs
127
128
129 def build_env_factory(
130     selected_env_id: str,
131     wrapper_configs: Optional[Sequence[Dict[str, Any]]] = None,
132     *,
133     trust_training_metadata: bool = False,
134     model_action_space: Any = None,
135     gym_kwargs: Optional[Dict[str, Any]] = None,
136 ) -> Callable[[], Any]:
137     """Crea una fàbrica d'env per al flux Studio."""

```

```

138     return partial(
139         _make_runtime_env,
140         selected_env_id,
141         wrapper_configs,
142         trust_training_metadata=trust_training_metadata,
143         model_action_space=model_action_space,
144         gym_kwargs=gym_kwargs,
145     )
146
147
148 def validate_action_contract(
149     model_obj: Optional[BaseAlgorithm],
150     vec_env: Any,
151     metadata: Dict[str, Any],
152 ) -> None:
153     """Validació del contracte d'actuació."""
154     if model_obj is None:
155         return
156
157     model_action_space = getattr(model_obj, "action_space", None)
158     env_action_space = getattr(vec_env, "action_space", None)
159     if action_spaces_compatible(model_action_space, env_action_space):
160         return
161
162     wrappers = metadata.get("wrapper_configs") or []
163     wrapper_names = ", ".join(str(item.get("name")) for item in wrappers if isinstance(item, dict)) or "cap detectat"
164     source = metadata.get("metadata_source") or "metadades no trobades"
165     raise ValueError(
166         "L'espai d'accions de l'entorn no coincideix amb el que espera el model. "
167         f"Model: {describe_action_space(model_action_space)}; "
168         f"entorn: {describe_action_space(env_action_space)}. "
169         f"Wrappers detectats: {wrapper_names}. Font: {source}."
170     )
171
172
173 def _space_shape(space: Any) -> Tuple[int, ...] | None:
174     """Space shape."""
175     shape = getattr(space, "shape", None)
176     if shape is None:
177         return None
178     try:
179         return tuple(int(value) for value in shape)
180     except Exception:
181         return None
182
183
184 def get_model_observation_size(model_obj: Optional[BaseAlgorithm]) -> Optional[int]:
185     """Retorna la mida d'observació del model."""
186     if model_obj is None:
187         return None
188     shape = _space_shape(getattr(model_obj, "observation_space", None))
189     if not shape:
190         return None
191     return int(np.prod(shape))
192
193
194 def get_observation_size_from_space(space: Any) -> Optional[int]:
195     """Retorna la mida d'observació des de l'espai."""
196     shape = _space_shape(space)
197     if not shape:
198         return None
199     return int(np.prod(shape))
200
201
202 def get_observation_size_from_array(obs: Any) -> Optional[int]:
203     """Retorna la mida de l'observació de la matriu."""
204     try:
205         arr = np.asarray(obs)
206         if arr.ndim > 1:
207             arr = arr.reshape(arr.shape[0], -1)[0]
208         return int(arr.size)
209     except Exception:
210         return None
211
212
213 def flatten_observation_values(values: Any) -> List[float]:
214     """Aplanar els valors d'observació."""
215     if values is None:
216         return []
217     try:
218         return [float(value) for value in np.asarray(values, dtype=np.float32).reshape(-1)]
219     except Exception:
220         flattened: List[float] = []
221         try:
222             iterator = list(values)
223         except Exception:
224             iterator = []
225         for value in iterator:
226             if isinstance(value, (list, tuple, np.ndarray)):
227                 flattened.extend(flatten_observation_values(value))
228             continue
229     try:

```

```

230         flattened.append(float(value))
231     except Exception:
232         pass
233     return flattened
234
235
236 def get_runtime_observation_variables(
237     core_env: Any,
238     obs: Any = None,
239     model_obj: Optional[BaseAlgorithm] = None,
240     vec_env: Any = None,
241 ) -> List[str]:
242     """Retorna les variables d'observació del clima d'execució."""
243     obs_vars = get_wrapper_variables(core_env, "observation_variables")
244     target_size = (
245         get_model_observation_size(model_obj)
246         or get_observation_size_from_space(getattr(vec_env, "observation_space", None))
247         or get_observation_size_from_array(obs)
248         or len(obs_vars)
249     )
250
251     if not target_size:
252         return obs_vars
253     if len(obs_vars) == target_size:
254         return obs_vars
255     if obs_vars and target_size % len(obs_vars) == 0:
256         repeats = target_size // len(obs_vars)
257         if repeats > 1:
258             return [
259                 f"{variable}_hist{block + 1}"
260                 for block in range(repeats)
261                 for variable in obs_vars
262             ]
263     return [f"obs_{index + 1:03d}" for index in range(target_size)]
264
265
266 def coerce_observation_values(values: Sequence[float], expected_size: Optional[int]) -> List[float]:
267     """Converteix els valors d'observació."""
268     coerced = flatten_observation_values(values)
269     if expected_size is None or len(coerced) == expected_size:
270         return coerced
271     if len(coerced) > expected_size:
272         return coerced[:expected_size]
273     fill_value = coerced[-1] if coerced else 0.0
274     return coerced + [fill_value] * (expected_size - len(coerced))
275
276
277 def initialize_runtime(
278     selected_env_id: str,
279     mode_label: str,
280     model_path: Optional[Path] = None,
281 ) -> Tuple[Any, Any, Optional[BaseAlgorithm], Any, List[str]]:
282     """Inicialitza l'execució de l'entorn."""
283     init_warnings: List[str] = []
284     model_obj: Optional[BaseAlgorithm] = None
285     vecnorm_path: Optional[Path] = None
286     metadata: Dict[str, Any] = {}
287     if mode_requires_model(mode_label) and model_path is not None:
288         with open(model_path, "rb") as file_handle:
289             model_obj, _ = load_sb3_model_bytes(file_handle.read(), device="cpu")
290             vecnorm_path = candidate_vecnorm(model_path)
291             metadata = load_model_training_metadata(model_path)
292
293     metadata_env_id = metadata.get("env_id")
294     if isinstance(metadata_env_id, str) and metadata_env_id in gym.envs.registry.keys():
295         # Si el model porta env_id al metadata, el fem prevaldre sobre el text escrit a mà.
296         # Això evita carregar una política entrenada amb un espai d'accions diferent.
297         selected_env_id = metadata_env_id
298
299     has_wrapper_contract = "wrapper_configs" in metadata
300     wrapper_configs = list(metadata.get("wrapper_configs") or [])
301     gym_kwargs = _build_gym_kwargs_from_metadata(metadata)
302     env_factory = build_env_factory(
303         selected_env_id,
304         wrapper_configs=wrapper_configs,
305         trust_training_metadata=has_wrapper_contract,
306         model_action_space=getattr(model_obj, "action_space", None),
307         gym_kwargs=gym_kwargs or None,
308     )
309
310     vec_env = None
311     if mode_requires_model(mode_label) and vecnorm_path and vecnorm_path.exists():
312         # VecNormalize només és segur si es reconstrueix sobre el mateix env_factory.
313         try:
314             vec_env = load_vecnormalize(str(vecnorm_path), env_factory)
315         except ValueError as exc:
316             init_warnings.append(
317                 f"VecNormalize no aplicat: {exc} "
318                 "Continuant sense normalització d'observacions; les prediccions del model poden ser menys precises."
319             )
320     if vec_env is None:
321         vec_env = DummyVecEnv([env_factory])

```

```

322
323     try:
324         validate_action_contract(model_obj, vec_env, metadata)
325     except ValueError as exc:
326         init_warnings.append(f"Avis de contracte d'accions: {exc}")
327
328     model_obs_shape = _space_shape(getattr(model_obj, "observation_space", None)) if model_obj else None
329     env_obs_shape = _space_shape(getattr(vec_env, "observation_space", None))
330     if model_obs_shape and env_obs_shape and model_obs_shape != env_obs_shape:
331         # En mode interactiu no aturem directament: aviseu i deixeu que l'usuari decideixi,
332         # perquè pot estar fent una prova ràpida de compatibilitat.
333         model_size = int(np.prod(model_obs_shape))
334         env_size = int(np.prod(env_obs_shape))
335         init_warnings.append(
336             f"Avis de contracte d'observacions: el model espera {model_size} variables {model_obs_shape} "
337             f"però l'entorn en dona {env_size} {env_obs_shape}. "
338             "Les prediccions de l'agent no seran fiables."
339         )
340
341     core_env = vec_env.envs[0] if hasattr(vec_env, "envs") else vec_env
342     obs = vec_env.reset()
343     return vec_env, core_env, model_obj, obs, init_warnings
344
345
346 def get_wrapper_variables(core_env: Any, attr_name: str) -> List[Any]:
347     """Retorna les variables de l'embolcall."""
348     if hasattr(core_env, "get_wrapper_attr"):
349         values = core_env.get_wrapper_attr(attr_name)
350         if values is None:
351             return []
352         return list(values)
353     return []
354
355
356 def extract_policy_test_defaults(
357     obs: Any,
358     info_dict: Dict[str, Any],
359     obs_vars: Sequence[str],
360     vec_env: Any,
361     core_env: Any,
362 ) -> List[float]:
363     """Extreu els valors predeterminats de la prova de política."""
364     raw_obs_values = obs[0] if getattr(obs, "ndim", 0) > 1 else obs
365     has_raw = False
366     if isinstance(info_dict, dict) and "observation" in info_dict:
367         # Si l'entorn ens dona l'observació crua a info, és la millor base per al formulari.
368         if len(info_dict["observation"]) == len(obs_vars):
369             raw_obs_values = info_dict["observation"]
370             has_raw = True
371
372     if not has_raw:
373         if isinstance(vec_env, VecNormalize):
374             # En avaluació estàtica volem valors humans; desfer VecNormalize quan és possible.
375             try:
376                 raw_obs_values = vec_env.unnormalize_obs(np.array([raw_obs_values]))[0]
377             except Exception:
378                 pass
379
380         temp = core_env
381         while hasattr(temp, "env"):
382             if type(temp).__name__ == "NormalizeObservation":
383                 # Alguns wrappers normalitzen sense VecNormalize; busquem aquest cas caminant
384                 # la cadena d'embolcalls.
385                 try:
386                     std = np.sqrt(temp.obs_rms.var + temp.epsilon)
387                     raw_obs_values = (raw_obs_values * std) + temp.obs_rms.mean
388                 except Exception:
389                     pass
390                 break
391         temp = temp.env
392
393     return flatten_observation_values(raw_obs_values)
394
395
396 def randomize_observation_values(obs_vars: Sequence[str]) -> List[float]:
397     """Aleatoritzar els valors d'observació."""
398     import random
399
400     new_vals: List[float] = []
401     for var_name in obs_vars:
402         lowered = var_name.lower()
403         if "temperature" in lowered or "temp" in lowered:
404             new_vals.append(random.uniform(5.0, 35.0))
405         elif "power" in lowered or "rate" in lowered:
406             new_vals.append(random.uniform(0.0, 5000.0))
407         elif "month" in lowered:
408             new_vals.append(float(random.randint(1, 12)))
409         elif "day" in lowered:
410             new_vals.append(float(random.randint(1, 31)))
411         elif "hour" in lowered:
412             new_vals.append(float(random.randint(0, 23)))
413         else:

```

```

414         new_vals.append(random.uniform(0.0, 1.0))
415     return new_vals
416
417
418 def infer_unit(var_name: str) -> str:
419     """Infer unit."""
420     lowered = var_name.lower()
421     if "temperature" in lowered or "temp" in lowered or "setpoint" in lowered:
422         return "°C"
423     if "power" in lowered or "rate" in lowered or "demand" in lowered:
424         return "W"
425     if "humidity" in lowered:
426         return "%"
427     if "speed" in lowered:
428         return "m/s"
429     if "direction" in lowered or "angle" in lowered:
430         return "°"
431     if "radiation" in lowered:
432         return "W/m²"
433     if "month" in lowered or "day" in lowered or "hour" in lowered:
434         return ""
435     return ""
436
437
438 def normalize_policy_observation(
439     custom_obs: Sequence[float],
440     core_env: Any,
441     vec_env: Any,
442     expected_size: Optional[int] = None,
443 ) -> np.ndarray:
444     """Normalitza l'observació de les polítiques."""
445     expected_shape = _space_shape(getattr(vec_env, "observation_space", None))
446     if expected_size is None and expected_shape:
447         expected_size = int(np.prod(expected_shape))
448
449     normalized_values = coerce_observation_values(custom_obs, expected_size)
450     if expected_shape and int(np.prod(expected_shape)) == len(normalized_values):
451         normalized = np.array(normalized_values, dtype=np.float32).reshape((1, *expected_shape))
452     else:
453         normalized = np.array([normalized_values], dtype=np.float32)
454
455     temp = core_env
456     while hasattr(temp, "env"):
457         if type(temp).__name__ == "NormalizeObservation":
458             try:
459                 std = np.sqrt(temp.obs_rms.var + temp.epsilon)
460                 normalized[0] = (normalized[0] - temp.obs_rms.mean) / std
461             except Exception:
462                 pass
463             break
464         temp = temp.env
465
466     if isinstance(vec_env, VecNormalize):
467         return vec_env.normalize_obs(normalized)
468     return normalized
469
470
471 def unnormalize_sinergym_action(action_array: Any, env_obj: Any) -> Any:
472     """Unnormalize synergym action."""
473     temp = env_obj
474     while hasattr(temp, "env"):
475         if type(temp).__name__ == "NormalizeAction":
476             return temp.action(action_array)
477         temp = temp.env
478     return action_array
479
480
481 def prepare_action_display(action: Any, vec_env: Any, core_env: Any) -> Any:
482     """Prepara la visualització de l'acció."""
483     action_formatted = format_action_for_env(action, vec_env)
484     if isinstance(action_formatted, np.ndarray) and len(action_formatted.shape) > 1:
485         action_display = action_formatted[0]
486     else:
487         action_display = action_formatted
488     return unnormalize_sinergym_action(action_display, core_env)
489
490
491 def extract_display_observation(
492     obs: Any,
493     info_dict: Dict[str, Any],
494     obs_vars: Sequence[str],
495     vec_env: Any,
496 ) -> Tuple[Any, Any]:
497     """Extreu l'observació de visualització."""
498     actual_obs = obs[0] if isinstance(obs, np.ndarray) and len(obs.shape) > 1 else obs
499     raw_obs_values: Any = []
500     has_raw = False
501
502     # Preferim l'observació crua que torna Sinergym a info; és la que té unitats humanes.
503     if isinstance(info_dict, dict) and "observation" in info_dict:
504         last_raw_obs = info_dict["observation"]
505         if len(last_raw_obs) == len(obs_vars):

```

```

506         raw_obs_values = last_raw_obs
507         has_raw = True
508
509     if not has_raw and isinstance(vec_env, VecNormalize):
510         # Si la run passa per VecNormalize, el model veu valors normalitzats. Per mostrar-los
511         # a pantalla els tornem a l'escala original quan es pot.
512         try:
513             raw_obs_values = vec_env.unnormalize_obs(np.array([actual_obs]))[0]
514             has_raw = True
515         except Exception:
516             pass
517
518     display_obs = flatten_observation_values(raw_obs_values if has_raw else actual_obs)
519     return actual_obs, display_obs
520
521
522 def extract_info_dict(infos: Any) -> Dict[str, Any]:
523     """Extreu el diccionari info d'un step de VecEnv o Gym."""
524     if isinstance(infos, (tuple, list, np.ndarray)):
525         return infos[0] if len(infos) > 0 else {}
526     if isinstance(infos, dict):
527         return infos
528     return {}
529
530
531 def build_history_entry(
532     step_number: int,
533     reward: float,
534     info_dict: Dict[str, Any],
535     next_obs: Any,
536     obs_vars: Sequence[str],
537     vec_env: Any,
538 ) -> Dict[str, Any]:
539     """Crea l'entrada de l'historial per al flux Studio."""
540     hist_entry: Dict[str, Any] = {"pas": step_number, "reward": reward}
541
542     raw_vals: Any = []
543     has_raw = False
544     if isinstance(info_dict, dict) and "observation" in info_dict:
545         # Guardem historial en escala real perquè els KPI del panell siguin interpretables.
546         last_raw = info_dict["observation"]
547         if len(last_raw) == len(obs_vars):
548             raw_vals = last_raw
549             has_raw = True
550
551     if not has_raw and isinstance(vec_env, VecNormalize):
552         try:
553             obs_2d = np.array([next_obs[0] if len(next_obs.shape) > 1 else next_obs])
554             raw_vals = vec_env.unnormalize_obs(obs_2d)[0]
555             has_raw = True
556         except Exception:
557             pass
558
559     vals_to_log = flatten_observation_values(
560         raw_vals if has_raw else (next_obs[0] if len(next_obs.shape) > 1 else next_obs)
561     )
562     if len(vals_to_log) == len(obs_vars):
563         for var_name, value in zip(obs_vars, vals_to_log):
564             hist_entry[var_name] = value
565
566     return hist_entry
567
568
569 def run_environment_steps(
570     vec_env: Any,
571     core_env: Any,
572     action_to_take: Any,
573     current_step: int,
574     repeat_n: int = 1,
575     obs_vars: Optional[Sequence[str]] = None,
576 ) -> Dict[str, Any]:
577     """Executa els passos de l'entorn."""
578     if hasattr(core_env, "action_space"):
579         action_formatted = format_action_for_env(action_to_take, vec_env)
580     else:
581         action_formatted = action_to_take
582
583     runtime_obs_vars = (
584         list(obs_vars)
585         if obs_vars is not None
586         else get_wrapper_variables(core_env, "observation_variables")
587     )
588     history_entries: List[Dict[str, Any]] = []
589     next_obs = None
590     reward = 0.0
591     info_dict: Dict[str, Any] = {}
592     episode_finished = False
593     reset_obs = None
594
595     for offset in range(repeat_n):
596         next_obs, rewards, dones, infos = vec_env.step(action_formatted)
597         reward = rewards[0] if isinstance(rewards, (list, np.ndarray)) else rewards

```



```

598     done = dones[0] if isinstance(dones, (list, np.ndarray)) else dones
599     info_dict = extract_info_dict(infos)
600
601     history_entries.append(
602         build_history_entry(
603             step_number=current_step + offset + 1,
604             reward=reward,
605             info_dict=info_dict,
606             next_obs=next_obs,
607             obs_vars=runtime_obs_vars,
608             vec_env=vec_env,
609         )
610     )
611
612     if done:
613         episode_finished = True
614         reset_obs = vec_env.reset()
615         next_obs = reset_obs
616         break
617
618     return {
619         "next_obs": next_obs,
620         "reward": reward,
621         "info_dict": info_dict,
622         "history_entries": history_entries,
623         "episode_finished": episode_finished,
624         "reset_obs": reset_obs,
625     }
626
627 def find_matching_column(row: Dict[str, Any], keywords: Sequence[str]) -> Optional[str]:
628     """Troba una columna compatible."""
629     for col_name in row.keys():
630         if any(keyword.lower() in col_name.lower() for keyword in keywords):
631             return col_name
632     return None
633

```

E.23 backend/mapa_backend.py

Listing 44: backend/mapa_backend.py

```

1  """Utilitats de representació de mapes compartides construïdes a pydeck i Streamlit."""
2
3  from __future__ import annotations
4
5  import pandas as pd
6  import pydeck as pdk
7  import streamlit as st
8
9
10 def render_location_map(
11     latitude: float,
12     longitude: float,
13     *,
14     zoom: float = 4.5,
15     color: tuple[int, int, int, int] = (95, 84, 249, 220),
16     radius: int = 22000,
17     tooltip_html: str | None = None,
18     use_container_width: bool = True,
19 ) -> None:
20     """Renderitza un mapa de dispersió d'un sol punt centrat en les coordenades donades.
21
22     tooltip_html accepta HTML preformatats (no calen referències de columna pydeck).
23     """
24
25     point_frame = pd.DataFrame([{"latitude": latitude, "longitude": longitude}])
26     tooltip = (
27         {"html": tooltip_html, "style": {"backgroundColor": "#17233c", "color": "white"}}
28         if tooltip_html
29         else None
30     )
31     st.pydeck_chart(
32         pdk.Deck(
33             initial_view_state=pdk.ViewState(
34                 latitude=latitude,
35                 longitude=longitude,
36                 zoom=zoom,
37                 pitch=0,
38             ),
39             layers=[
40                 pdk.Layer(
41                     "ScatterplotLayer",
42                     data=point_frame,
43                     get_position="[longitude, latitude]",
44                     get_fill_color=list(color),
45                     get_radius=radius,
46                     pickable=tooltip_html is not None,
47                 )
48             ]
49         )
50     )

```

```

48         ],
49         tooltip=tooltip,
50     ),
51     use_container_width=use_container_width,
52 )
53
54
55 def render_multipoint_map(
56     data_frame: pd.DataFrame,
57     *,
58     center_latitude: float | None = None,
59     center_longitude: float | None = None,
60     zoom: float = 2,
61     color: tuple[int, int, int, int] = (255, 0, 0, 200),
62     radius: int = 55000,
63     tooltip_html: str | None = None,
64     use_container_width: bool = True,
65 ) -> None:
66     """Renderitza un mapa de dispersió de diversos punts des d'un DataFrame amb columnes de latitud/longitud.
67
68     La vista es centra a les coordenades mitjanes tret que center_latitude/center_longitude
69     es proporcionen de manera explícita. tooltip_html admet marcadors de posició de nom de columna pydeck
70     (e.g. '<b>{file_name}</b><br/>{ciutat}').
71     """
72
73     lat_center = center_latitude if center_latitude is not None else float(data_frame["latitude"].mean())
74     lon_center = center_longitude if center_longitude is not None else float(data_frame["longitude"].mean())
75     tooltip = (
76         {"html": tooltip_html, "style": {"backgroundColor": "#14213d", "color": "white"}}
77         if tooltip_html
78         else None
79     )
80     st.pydeck_chart(
81         pdk.Deck(
82             initial_view_state=pdk.ViewState(
83                 latitude=lat_center,
84                 longitude=lon_center,
85                 zoom=zoom,
86                 pitch=0,
87             ),
88             layers=[
89                 pdk.Layer(
90                     "ScatterplotLayer",
91                     data=data_frame,
92                     get_position=["longitude", "latitude"],
93                     get_fill_color=list(color),
94                     get_radius=radius,
95                     pickable=tooltip_html is not None,
96                 )
97             ],
98             tooltip=tooltip,
99         ),
100         use_container_width=use_container_width,
101     )

```

E.24 backend/mostrar_entorn_backend.py

Listing 45: backend/mostrar_entorn_backend.py

```

1  """Backend de metadades de l'entorn: carrega actius, PV, emmagatzematge i dades per pintar la UI."""
2
3  from __future__ import annotations
4
5  import json
6  from typing import Any, Dict, List, Optional, Tuple
7
8  import pandas as pd
9  import streamlit as st
10
11  from backend.viewer_3d_backend import build_surface_records
12
13
14  @st.cache_data(show_spinner=False)
15  def load_epjson(path: str) -> dict:
16     """Llegeix i retorna un fitxer epJSON del disc (emmagatzemat a la memòria cau)."""
17     with open(path, 'r') as fh:
18         return json.load(fh)
19
20
21  def _safe_float(value: Any) -> Optional[float]:
22     """Retorna float(valor) o None si la conversió falla."""
23     try:
24         return None if value is None else float(value)
25     except Exception:
26         return None
27
28
29  def _getf(data: dict, *keys: str) -> Optional[float]:

```

```

30 """Retorna el primer valor numèric trobat entre les claus donades, o None."""
31 for key in keys:
32     value = data.get(key)
33     try:
34         if value is None:
35             continue
36         return float(value)
37     except Exception:
38         continue
39 return None
40
41
42 def _derive_kibam_metrics(
43     obj: dict,
44 ) -> Tuple[Optional[float], Optional[float], Optional[float]]:
45     """Deriva (energy_kwh, p_charge_kw, p_discharge_kw) d'un objecte de bateria KiBaM.
46
47     Processa ElectricLoadCenter:Emmagatzematge: camps de bateria per estimar el nivell de paquet
48     capacitat energètica i límits de potència de càrrega/descàrrega.
49     """
50     cap_ah = _getf(obj, "maximum_module_capacity")
51     voltage_full = _getf(obj, "fully_charged_module_open_circuit_voltage")
52     voltage_cut = _getf(obj, "module_cut_off_voltage")
53     c_rate = _getf(obj, "module_charge_rate_limit")
54     current_discharge = _getf(obj, "maximum_module_discharging_current")
55
56     n_series = int(_getf(obj, "number_of_battery_modules_in_series") or 1)
57     n_parallel = int(_getf(obj, "number_of_battery_modules_in_parallel") or 1)
58
59     if cap_ah is None:
60         return None, None, None
61
62     if voltage_full and voltage_cut:
63         voltage_nominal = 0.5 * (voltage_full + voltage_cut)
64     elif voltage_full:
65         voltage_nominal = 0.95 * voltage_full
66     elif voltage_cut:
67         voltage_nominal = 1.10 * voltage_cut
68     else:
69         voltage_nominal = 12.0
70
71     voltage_pack = voltage_nominal * n_series
72     cap_ah_pack = cap_ah * n_parallel
73     energy_kwh = (cap_ah_pack * voltage_pack) / 1000.0
74
75     p_charge_kw = None
76     if c_rate is not None:
77         p_charge_kw = (c_rate * cap_ah * n_parallel * voltage_pack) / 1000.0
78
79     p_discharge_kw = None
80     if current_discharge is not None:
81         p_discharge_kw = (current_discharge * n_parallel * voltage_pack) / 1000.0
82
83     return energy_kwh, p_charge_kw, p_discharge_kw
84
85
86 def parse_pv_and_storage(epjson: dict) -> Dict[str, Any]:
87     """Analitza generadors de PV i objectes d'emmagatzematge de la bateria des d'un epJSON dict.
88
89     Admet generador: fotovoltaic, generador: PVWatts,
90     ElectricLoadCenter:Emmagatzematge:Simple, :Bateria i :LiIonNMCBattery.
91     Retorna un dictat amb les claus 'pv' i 'emmagatzematge', cadascuna una llista de dicts normalitzats.
92     """
93     output: Dict[str, List[dict]] = {'pv': [], 'storage': []}
94
95     # EnergyPlus té dues famílies habituals de generadors solars. Les normalitzem a la
96     # mateixa estructura perquè el visor no hagi de conèixer cada tipus.
97     for key in ("Generator:Photovoltaic", "Generator:PVWatts"):
98         for name, obj in epjson.get(key, {}).items():
99             output['pv'].append({
100                 'name': name,
101                 'surface_name': obj.get("surface_name"),
102                 'type': key,
103                 'capacity_dc': obj.get("dc_system_capacity"),
104                 'num_series': obj.get("number_of_modules_in_series"),
105                 'num_strings': obj.get("number_of_series_strings"),
106                 'performance_obj': obj.get("module_performance_name"),
107                 'array_geometry_type': obj.get("array_geometry_type"),
108             })
109
110     for name, obj in epjson.get("ElectricLoadCenter:Storage:Simple", {}).items():
111         # En Storage:Simple l'eficiència de volta completa es pot estimar multiplicant
112         # càrrega i descàrrega, si totes dues existeixen.
113         eff_charge = _safe_float(obj.get("nominal_energètic_efficiency_for_charging"))
114         eff_discharge = _safe_float(obj.get("nominal_discharging_energètic_efficiency"))
115         eff_roundtrip = None
116         if eff_charge is not None and eff_discharge is not None:
117             try:
118                 eff_roundtrip = float(eff_charge) * float(eff_discharge)
119             except Exception:
120                 pass
121         output['storage'].append({

```

```

122         'name': name,
123         'type': "ElectricLoadCenter:Storage:Simple",
124         'energy_kwh': _safe_float(obj.get("maximum_storage_capacity")),
125         'p_charge_kw': _safe_float(obj.get("maximum_power_for_charging")),
126         'p_discharge_kw': _safe_float(obj.get("maximum_power_for_discharging")),
127         'eff_roundtrip': eff_roundtrip,
128         'zone_name': obj.get("zone_name"),
129     })
130
131     for name, obj in epjson.get("ElectricLoadCenter:Storage:Battery", {}).items():
132         energy_kwh, p_charge_kw, p_discharge_kw = _derive_kibam_metrics(obj)
133         output['storage'].append({
134             'name': name,
135             'type': "ElectricLoadCenter:Storage:Battery",
136             'energy_kwh': None if energy_kwh is None else round(energy_kwh, 2),
137             'p_charge_kw': None if p_charge_kw is None else round(p_charge_kw, 2),
138             'p_discharge_kw': None if p_discharge_kw is None else round(p_discharge_kw, 2),
139             'eff_roundtrip': None,
140             'zone_name': obj.get("zone_name"),
141         })
142
143     for name, obj in epjson.get("ElectricLoadCenter:Storage:LiIonNMCBattery", {}).items():
144         output['storage'].append({
145             'name': name,
146             'type': "ElectricLoadCenter:Storage:LiIonNMCBattery",
147             'energy_kwh': None,
148             'p_charge_kw': None,
149             'p_discharge_kw': None,
150             'eff_roundtrip': None,
151             'zone_name': obj.get("zone_name"),
152         })
153
154     return output
155
156
157 @St.cache_data(show_spinner=False)
158 def load_environment_assets(epjson_path: str) -> Dict[str, Any]:
159     """Carrega i prepara tots els actius principals per a un entorn Sinergym (emmagatzemats a la memòria cau).
160
161     Llegeix el fitxer epJSON, crea registres de superfície enriquits i analitza
162     PV/objectes d'emmagatzematge. Retorna un dictat unificat preparat per a UI.
163     """
164     data = load_epjson(epjson_path)
165     enriched = build_surface_records(data)
166     energy = parse_pv_and_storage(data)
167     return {
168         'records': enriched['records'],
169         'zones': enriched['zones'],
170         'types': enriched['types'],
171         'center': enriched['center'],
172         'z_clip_range': enriched['z_clip_range'],
173         'name_index': enriched['name_index'],
174         'pv': energy['pv'],
175         'storage': energy['storage'],
176     }
177
178
179 def summarize_pv(
180     records: List[dict],
181     name_index: Dict[str, int],
182     pv_list: List[dict],
183 ) -> Dict[str, Any]:
184     """Calculeu mètriques agregades per als panells PV associats a les superfícies de l'edifici.
185
186     Recompte de retorns, llista de noms de superfície actives, àrea total, inclinació mitjana i
187     azimuth mitjà. L'àrea i els angles són None quan no es troben superfícies coincidents.
188     """
189     used_surfaces = []
190     total_area = 0.0
191     tilts: List[float] = []
192     azimuths: List[float] = []
193     for pv in pv_list:
194         surface_name = pv.get('surface_name')
195         if surface_name in name_index:
196             record = records[name_index[surface_name]]
197             used_surfaces.append(surface_name)
198             total_area += record['area']
199             tilts.append(record['tilt'])
200             azimuths.append(record['azimuth'])
201     return {
202         'count': len(pv_list),
203         'surfaces_with_pv': sorted(set(used_surfaces)),
204         'area_m2': total_area,
205         'avg_tilt': sum(tilts) / len(tilts) if tilts else None,
206         'avg_azimuth': sum(azimuths) / len(azimuths) if azimuths else None,
207     }
208
209
210 @St.cache_data(show_spinner=False)
211 def parse_epw_metadata_and_temperature(epw_file: str):
212     """Llegeix les metadades d'ubicació i la temperatura anual mitjana d'un fitxer EPW (emmagatzemat a la memòria cau).
213

```

```

214     Retorna (location_dict, avg_temp_celsius, climate_zone_label).
215     """
216     with open(epw_file, 'r') as fh:
217         lines = fh.readlines()
218
219     loc = lines[0].strip().split(',')
220     loc_data = {
221         'city': loc[1], 'state': loc[2], 'country': loc[3],
222         'latitude': float(loc[6]), 'longitude': float(loc[7]),
223         'timezone': float(loc[8]), 'elevation': float(loc[9]),
224     }
225     df = pd.read_csv(epw_file, skiprows=8, header=None)
226     avg_temp = df.iloc[:, 6].mean()
227     climate_zone = "hot" if avg_temp > 22 else "cold" if avg_temp < 12 else "cool"
228     return loc_data, avg_temp, climate_zone
229
230
231 def _format_ui_value(value: Any) -> str:
232     """Formata qualsevol valor per mostrar-lo, retornant '-' per als valors buits/None."""
233     if value is None:
234         return '-'
235     if isinstance(value, (list, tuple, set, dict)) and len(value) == 0:
236         return '-'
237     return str(value)
238
239
240 def _tuple_mapping_to_df(mapping: Dict[str, Any], columns: List[str]) -> pd.DataFrame:
241     """Converteix un dictat d'entrades amb valors de tupla a una etiqueta DataFrame."""
242     rows = []
243     for alias, values in (mapping or {}).items():
244         flat = list(values) if isinstance(values, (list, tuple)) else [values]
245         padded = flat + [''] * max(0, len(columns) - len(flat))
246         row: Dict[str, Any] = {'Alias': alias}
247         for index, column in enumerate(columns):
248             row[column] = _format_ui_value(padded[index])
249         rows.append(row)
250     return pd.DataFrame(rows)
251
252
253 def _sequence_to_df(values: List[Any], column_name: str) -> pd.DataFrame:
254     """Converteix una llista plana de valors en una sola columna DataFrame."""
255     return pd.DataFrame([{'column_name': _format_ui_value(v)} for v in (values or [])])
256
257
258 def _reward_summary_frames(
259     reward_kwargs: Dict[str, Any],
260 ) -> Tuple[pd.DataFrame, pd.DataFrame]:
261     """Divideix els kwargs de recompensa en escalar i agrupa (list-valued) DataFrames."""
262     scalar_rows: List[dict] = []
263     grouped_rows: List[dict] = []
264     for key, value in (reward_kwargs or {}).items():
265         if isinstance(value, (list, tuple)):
266             grouped_rows.append({'Parametre': key, 'Valor': ', '.join(map(_format_ui_value, value))})
267         else:
268             scalar_rows.append({'Parametre': key, 'Valor': _format_ui_value(value)})
269     return pd.DataFrame(scalar_rows), pd.DataFrame(grouped_rows)

```

E.25 backend/paths.py

Listing 46: backend/paths.py

```

1  """Camins del sistema de fitxers canònics utilitzats pels mòduls de fons BEMS-RL Studio.
2
3  Les constants d'aquest mòdul apunten a les dades actives del paquet Sinergym
4  directoris dins dels temps d'execució local i Docker. Mantenir-los centralitzats evita
5  codi de pàgina des de codificació d'ubicacions d'edifici, temps i configuració.
6  """
7
8  from __future__ import annotations
9
10 from pathlib import Path
11
12 import sinergym
13
14 PKG_DIR = Path(sinergym.__file__).parent
15 BUILDINGS_DIR = PKG_DIR / "data" / "buildings"
16 WEATHERS_DIR = PKG_DIR / "data" / "weather"
17 CFG_DIR = PKG_DIR / "data" / "default_configuration"
18
19 ONE_YEAR_STEPS = 35040

```

E.26 backend/resultats_backend.py

Listing 47: backend/resultats_backend.py

```
1 """Càrrega de dades, preparació de KPI i descobriment d'execució per a la pàgina de resultats.
2
3 El backend de resultats converteix els artefactes de entrenament i avaluació persistents en el
4 Estructura ``DashboardData`` consumida per Streamlit i pel generador d'informes.
5 Manté deliberadament el descobriment del sistema de fitxers, la normalització CSV i les dades d'acció
6 alineació fora del fitxer de pàgina, de manera que el mateix comportament es pot reutilitzar de manera automatitzada
7 exportacions.
8 """
9
10 from __future__ import annotations
11
12 import copy
13 import glob
14 import json
15 import os
16 import re
17 from dataclasses import dataclass
18 from pathlib import Path
19 from typing import Any
20
21 import pandas as pd
22 import plotly.graph_objects as go
23
24 from backend.grafics.time_utils import _coerce_temporal_parts
25
26
27 @dataclass(frozen=True)
28 class RunOption:
29     """Directori d'execució seleccionable que es mostra a la pàgina de resultats."""
30
31     path: str
32     name: str
33
34     def __str__(self) -> str:
35         """Str."""
36         return self.name
37
38
39 @dataclass(frozen=True)
40 class ResultsPageState:
41     """S'han resolt les entrades necessàries abans que es pugui representar la pàgina de resultats."""
42
43     default_base_dir: str
44     run_dirs: tuple[RunOption, ...]
45     warning_message: str | None = None
46
47
48 @dataclass(frozen=True)
49 class RunArtifacts:
50     """Fitxers d'artefactes derivats d'una execució seleccionada."""
51
52     progress_path: str
53     observations_path: str
54     error_message: str | None = None
55
56
57 @dataclass
58 class DashboardData:
59     """Es requereix un paquet de dades carregat per representar el panell de resultats en línia."""
60
61     progress: pd.DataFrame
62     obs: pd.DataFrame
63     actions: pd.DataFrame
64     metrics_dict: dict
65     zone_opts: list
66     has_zones: bool
67     all_zone_options: list
68     has_occupied_scope: bool
69     comfort_scope_options: list
70     default_comfort_scope: str
71     env_name: str
72     run_mode: str
73     timesteps_num: int
74     model_name: str
75     progress_path: str = ""
76     observations_path: str = ""
77     actions_path: str = ""
78     yaml_cfg: Any = None
79
80
81 _IGNORED_RUN_PARTS = {"__pycache__", ".git", ".venv", "venv", "node_modules"}
82
83
84 def _is_valid_run_directory(run_directory: Path) -> bool:
85     """Retorna True si el directori conté tant progress.csv com almenys un observations.csv."""
86     if any(part in _IGNORED_RUN_PARTS for part in run_directory.parts):
87         return False
```

```

88     if not (run_directory / "progress.csv").exists():
89         return False
90     observation_files = glob.glob(
91         os.path.join(str(run_directory), "**", "observations.csv"),
92         recursive=True,
93     )
94     return bool(observation_files)
95
96
97 def _build_run_display_name(run_directory: Path, base_directory: Path) -> str:
98     """Retorna una etiqueta llegible per a un directori d'execució relativa al directori base."""
99     try:
100         relative_path = run_directory.relative_to(base_directory)
101         return str(relative_path)
102     except ValueError:
103         return run_directory.name
104
105
106 def build_results_page_state(base_dir: str | None = None) -> ResultsPageState:
107     """Crea l'estat inicial de la pàgina escanejant el directori base per trobar execucions vàlides per al flux Studio."""
108
109     resolved_base_dir = Path.cwd() if base_dir is None else Path(base_dir)
110     progress_files = glob.glob(
111         os.path.join(str(resolved_base_dir), "**", "progress.csv"),
112         recursive=True,
113     )
114     run_directories = sorted(
115         {
116             Path(path).parent
117             for path in progress_files
118             if _is_valid_run_directory(Path(path).parent)
119         },
120         key=os.path.getmtime,
121         reverse=True,
122     )
123
124     if not run_directories:
125         return ResultsPageState(
126             default_base_dir=str(Path.cwd()),
127             run_dirs=(),
128             warning_message="No s'ha trobat cap execució amb progress.csv i observations.csv.",
129         )
130
131     return ResultsPageState(
132         default_base_dir=str(Path.cwd()),
133         run_dirs=tuple(
134             RunOption(
135                 path=str(run_directory),
136                 name=_build_run_display_name(run_directory, resolved_base_dir),
137             )
138             for run_directory in run_directories
139         ),
140     )
141
142
143 def get_run_artifacts(selected_run: str) -> RunArtifacts:
144     """Resol els fitxers d'artefactes principals associats a una execució seleccionada."""
145
146     latest_progress = os.path.join(selected_run, "progress.csv")
147     observation_files = glob.glob(
148         os.path.join(selected_run, "**", "observations.csv"),
149         recursive=True,
150     )
151
152     if not (os.path.exists(latest_progress) and observation_files):
153         return RunArtifacts(
154             progress_path=latest_progress,
155             observations_path="",
156             error_message="Falten progress.csv o observations.csv en l'execució seleccionada.",
157         )
158
159     sorted_observation_files = sorted(
160         observation_files,
161         key=os.path.getmtime,
162         reverse=True,
163     )
164     selected_observation = (
165         sorted_observation_files[1]
166         if len(sorted_observation_files) > 1
167         else sorted_observation_files[0]
168     )
169
170     return RunArtifacts(
171         progress_path=latest_progress,
172         observations_path=selected_observation,
173     )
174
175
176 def load_action_data(observations_path: str) -> tuple[pd.DataFrame, str]:
177     """Carrega la primera acció admesa CSV que es troba al costat del fitxer d'observacions."""
178
179     monitor_dir = Path(observations_path).parent

```

```

180 for filename in ("simulated_actions.csv", "agent_actions.csv"):
181     candidate = monitor_dir / filename
182     if not candidate.is_file():
183         continue
184     try:
185         return pd.read_csv(candidate), str(candidate)
186     except Exception:
187         continue
188 return pd.DataFrame(), ""
189
190
191 def align_actions_with_observations(
192     actions: pd.DataFrame,
193     obs: pd.DataFrame,
194 ) -> pd.DataFrame:
195     """Alineu les files d'acció amb les marques de temps d'observació i columnes temporals útils."""
196
197     if actions.empty or obs.empty:
198         return actions
199
200     aligned = actions.copy()
201     obs_index = obs.index
202     action_count = len(aligned)
203     if len(obs_index) == action_count + 1:
204         context = obs.iloc[1:action_count + 1]
205     else:
206         context = obs.iloc[:action_count]
207
208     aligned = aligned.iloc[:len(context)].copy()
209     aligned.index = context.index
210     temporal_columns = (
211         "datetime",
212         "timestamp",
213         "month",
214         "day_of_month",
215         "hour",
216         "time_elapsed(hours)",
217     )
218     for column in temporal_columns:
219         if column in context.columns and column not in aligned.columns:
220             aligned[column] = context[column].to_numpy()
221     return aligned
222
223
224 def select_actions_for_obs(actions: pd.DataFrame, obs: pd.DataFrame) -> pd.DataFrame:
225     """Retorna les files d'acció que corresponen al sector d'observació proporcionat."""
226
227     if actions.empty or obs.empty:
228         return pd.DataFrame()
229     if isinstance(actions.index, pd.DatetimeIndex) and isinstance(obs.index, pd.DatetimeIndex):
230         selected = actions.loc[actions.index.isin(obs.index)].copy()
231         if not selected.empty:
232             return selected.sort_index(kind="mergesort")
233     return actions.iloc[:len(obs)].copy().sort_index(kind="mergesort")
234
235
236 def is_radiant_run(data: DashboardData) -> bool:
237     """Retorna si l'execució seleccionada pertany a una configuració d'edifici radiant."""
238
239     run_text = f"{data.env_name} {data.model_name} {data.yaml_cfg}".lower()
240     return "radiant" in run_text
241
242
243 def select_radiant_action_data(actions: pd.DataFrame, data: DashboardData) -> pd.DataFrame:
244     """Retorna les dades d'acció només quan l'execució seleccionada utilitza el control del sòl radiant."""
245
246     if actions.empty or not is_radiant_run(data):
247         return pd.DataFrame(index=actions.index)
248     return actions
249
250
251 def load_dashboard_data(progress_path: str, observations_path: str) -> DashboardData:
252     """Carrega i retorna el paquet de dades complet utilitzat pel panell de resultats."""
253
254     import yaml
255
256     from backend.grafics.comfort_scope import has_occupied_data
257     from backend.grafics.data_loader import load_data
258     from backend.grafics.figures_zones import get_zone_options
259     from backend.grafics.metrics import compute_metrics
260
261     yaml_cfg = None
262     try:
263         yaml_candidates = list(Path(progress_path).parent.glob("*.yaml"))
264         if yaml_candidates:
265             with open(yaml_candidates[0], "r", encoding="utf-8") as config_file:
266                 yaml_cfg = yaml.safe_load(config_file)
267     except Exception:
268         pass
269
270     progress, obs = load_data(progress_path, observations_path)
271     obs = _ensure_observation_time_columns(obs)

```



```

272 actions, actions_path = load_action_data(observations_path)
273 actions = align_actions_with_observations(actions, obs)
274
275
276 metrics_dict = compute_metrics(progress, obs, yaml_cfg=yaml_cfg)
277 zone_opts = get_zone_options(obs, yaml_cfg)
278 has_zones = len(zone_opts) > 1
279 all_zone_options = [{"label": "Global (Totes)", "value": "ALL"}] + zone_opts
280 has_occupied_scope = has_occupied_data(obs)
281 comfort_scope_options = [{"label": "Totes les hores", "value": "all"}]
282 if has_occupied_scope:
283     comfort_scope_options.append(
284         {"label": "Només hores ocupades", "value": "occupied"}
285     )
286 default_comfort_scope = "occupied" if has_occupied_scope else "all"
287
288 run_folder = Path(progress_path).parent
289 run_folder_name = run_folder.name
290 match_env = re.match(r"(.*)?(?:-res\d+)?$", run_folder_name)
291 env_name = match_env.group(1) if match_env else run_folder_name
292 is_eval = len(progress) < 10
293 run_mode = "Avaluacio" if is_eval else "Entrenament"
294
295 if "time/total_timesteps" in progress.columns:
296     timesteps_num = int(progress["time/total_timesteps"].max())
297 elif "length(timesteps)" in progress.columns:
298     timesteps_num = int(progress["length(timesteps)"].sum())
299 else:
300     timesteps_num = len(obs)
301
302 model_name = resolve_model_name_for_run(progress_path, env_name)
303
304 return DashboardData(
305     progress=progress,
306     obs=obs,
307     actions=actions,
308     metrics_dict=metrics_dict,
309     zone_opts=zone_opts,
310     has_zones=has_zones,
311     all_zone_options=all_zone_options,
312     has_occupied_scope=has_occupied_scope,
313     comfort_scope_options=comfort_scope_options,
314     default_comfort_scope=default_comfort_scope,
315     env_name=env_name,
316     run_mode=run_mode,
317     timesteps_num=timesteps_num,
318     model_name=model_name,
319     progress_path=progress_path,
320     observations_path=observations_path,
321     actions_path=actions_path,
322     yaml_cfg=yaml_cfg,
323 )
324
325
326 def resolve_model_name_for_run(progress_path: str, env_name: str) -> str:
327     """Retorna el nom del model desat més probable per a una execució de resultats."""
328
329     run_folder = Path(progress_path).parent
330     env_slug = _slugify_model_lookup(env_name)
331     fallback = "Desconegut / No Especificat"
332
333     metadata_candidate = _model_name_from_training_metadata(run_folder, env_name, env_slug)
334     if metadata_candidate:
335         return metadata_candidate
336
337     zip_candidate = _model_name_from_zip_search(run_folder, env_name, env_slug)
338     if zip_candidate:
339         return zip_candidate
340
341     return fallback
342
343
344 def _slugify_model_lookup(value: str) -> str:
345     """Normalitza els noms de la mateixa manera que ho fan els directoris d'artefactes d'entrenament."""
346
347     cleaned = re.sub(r"[^A-Za-z0-9]+", "-", str(value)).strip("-").lower()
348     return cleaned or "sense-nom"
349
350
351 def _candidate_model_roots(run_folder: Path) -> list[Path]:
352     """Retorna les arrels probables que continguin artefactes de model desats."""
353
354     cwd = Path.cwd()
355     roots = [
356         run_folder,
357         run_folder.parent,
358         cwd / "trainings",
359         cwd / "models",
360         cwd,
361     ]
362     unique_roots: list[Path] = []
363     seen: set[Path] = set()

```

```

364 for root in roots:
365     try:
366         resolved = root.resolve()
367     except Exception:
368         resolved = root
369     if resolved in seen or not root.exists():
370         continue
371     seen.add(resolved)
372     unique_roots.append(root)
373 return unique_roots
374
375
376 def _model_name_from_training_metadata(
377     run_folder: Path,
378     env_name: str,
379     env_slug: str,
380 ) -> str | None:
381     """Troba un training_config.json que coincideixi i retorna la base del fitxer del model."""
382
383     config_candidates: list[tuple[int, float, str]] = []
384     progress_mtime = _safe_mtime(run_folder / "progress.csv")
385
386     # Busquem training_config.json propers a la run. Si hi ha diversos candidats, guanya
387     # el que coincideix millor amb l'env_id i amb una data de modificació més propera.
388     for root in _candidate_model_roots(run_folder):
389         if root.name in {".git", "__pycache__", "node_modules", ".venv", "venv"}:
390             continue
391         try:
392             config_paths = (
393                 root.rglob("training_config.json")
394                 if root.name in {"trainings", "models"}
395                 else root.glob("training_config.json")
396             )
397             for config_path in config_paths:
398                 try:
399                     with open(config_path, "r", encoding="utf-8") as handle:
400                         payload = json.load(handle)
401                 except Exception:
402                     continue
403
404                 payload_env = str(payload.get("env_id") or "")
405                 payload_slug = _slugify_model_lookup(payload_env)
406                 if payload_env != env_name and payload_slug != env_slug:
407                     continue
408
409                 artifact_name = str(payload.get("artifact_name") or config_path.parent.name)
410                 files = payload.get("files") if isinstance(payload.get("files"), dict) else {}
411                 model_file = str(files.get("model") or f"{artifact_name}.zip")
412                 model_stem = Path(model_file).stem or artifact_name
413                 model_path = config_path.parent / model_file
414                 distance = abs(_safe_mtime(model_path if model_path.exists() else config_path) - progress_mtime)
415                 score = 2 if payload_env == env_name else 1
416                 config_candidates.append((score, -distance, model_stem))
417         except Exception:
418             continue
419
420     if not config_candidates:
421         return None
422
423     config_candidates.sort(reverse=True)
424     return config_candidates[0][2]
425
426
427 def _model_name_from_zip_search(
428     run_folder: Path,
429     env_name: str,
430     env_slug: str,
431 ) -> str | None:
432     """Troba un model zip probable quan les metadades no estiguin disponibles."""
433
434     env_text = env_name.lower()
435     zip_candidates: list[tuple[int, float, str]] = []
436     progress_mtime = _safe_mtime(run_folder / "progress.csv")
437
438     for root in _candidate_model_roots(run_folder):
439         try:
440             zip_paths = (
441                 root.rglob("*.zip")
442                 if root.name in {"trainings", "models"}
443                 else root.glob("*.zip")
444             )
445             for zip_path in zip_paths:
446                 stem_lower = zip_path.stem.lower()
447                 stem_slug = _slugify_model_lookup(zip_path.stem)
448                 score = 0
449                 if env_slug and env_slug in stem_slug:
450                     score = 2
451                 elif env_text and env_text in stem_lower:
452                     score = 1
453                 if score <= 0:
454                     continue
455                 distance = abs(_safe_mtime(zip_path) - progress_mtime)

```

```

456         zip_candidates.append((score, -distance, zip_path.stem))
457     except Exception:
458         continue
459
460     if not zip_candidates:
461         return None
462
463     zip_candidates.sort(reverse=True)
464     return zip_candidates[0][2]
465
466
467 def _safe_mtime(path: Path) -> float:
468     """Retorna un fitxer mtime, o 0 quan no estigui disponible."""
469
470     try:
471         return path.stat().st_mtime
472     except OSError:
473         return 0.0
474
475
476 def load_comparison_data(run_path: str) -> DashboardData | None:
477     """Carrega les dades del panell per a una execució de comparació, retornant None quan no és vàlid."""
478
479     artifacts = get_run_artifacts(run_path)
480     if artifacts.error_message:
481         return None
482
483     try:
484         return load_dashboard_data(artifacts.progress_path, artifacts.observations_path)
485     except Exception:
486         return None
487
488
489 def prepare_obs_time_index(obs: pd.DataFrame) -> pd.DataFrame:
490     """Retorna les observacions amb un DatetimeIndex coherent per a visualitzacions de sèries temporals reals."""
491
492     df = obs.copy()
493     if "datetime" in df.columns:
494         dt = pd.to_datetime(df["datetime"], errors="coerce")
495         df.index = dt
496     elif "timestamp" in df.columns:
497         dt = pd.to_datetime(df["timestamp"], errors="coerce")
498         df.index = dt
499     elif not isinstance(df.index, pd.DatetimeIndex):
500         df.index = _derive_observation_datetime_index(df)
501
502     if isinstance(df.index, pd.DatetimeIndex):
503         df = df[~df.index.isna()]
504         df = df.sort_index(kind="mergesort")
505     return df
506
507
508 def build_real_period_options(obs: pd.DataFrame, period_kind: str) -> list[tuple[str, str]]:
509     """Crea opcions de dia, setmana o mes seleccionables per a les visualitzacions de sèries temporals en brut per al flux Studio."""
510
511     df = prepare_obs_time_index(obs)
512     if df.empty or not isinstance(df.index, pd.DatetimeIndex):
513         return []
514
515     if period_kind == "day":
516         days = pd.Index(df.index.floor("D").unique()).sort_values()
517         return [
518             (day.strftime("%Y-%m-%d"), day.strftime("%d/%m/%Y"))
519             for day in days
520         ]
521
522     if period_kind == "week":
523         normalized = df.index.normalize()
524         week_starts = pd.Index(
525             normalized - pd.to_timedelta(normalized.dayofweek, unit="D")
526         ).unique().sort_values()
527         return [
528             (
529                 start.strftime("%Y-%m-%d"),
530                 f"{start.strftime('%d/%m/%Y')} - {(start + pd.Timedelta(days=6)).strftime('%d/%m/%Y')}",
531             )
532             for start in week_starts
533         ]
534
535     months = pd.Index(df.index.to_period("M").unique()).sort_values()
536     return [
537         (month.strftime("%Y-%m"), month.strftime("%m/%Y"))
538         for month in months
539     ]
540
541
542 def slice_obs_for_real_period(
543     obs: pd.DataFrame,
544     period_kind: str,
545     selected_value: str,
546     *,
547     allow_fallback: bool = False,
548 ) -> tuple[pd.DataFrame, str | None]:

```

```

548 """Filtreu les observacions al període de dia, setmana o mes seleccionat."""
549
550 df = prepare_obs_time_index(obs)
551 if df.empty or not selected_value:
552     return df.iloc[0:0].copy(), None
553
554 if period_kind == "day":
555     target = pd.Timestamp(selected_value).normalize()
556     mask = df.index.floor("D") == target
557     if not mask.any() and allow_fallback:
558         # Les runs de comparació poden tenir un any diferent; en aquest cas comparem
559         # el mateix dia i mes, que és el que l'usuari espera visualment.
560         mask = (df.index.month == target.month) & (df.index.day == target.day)
561     result = df.loc[mask].copy().sort_index(kind="mergesort")
562     if result.empty:
563         return result, target.strftime("%d/%m/%Y")
564     actual_day = result.index[0].normalize()
565     return result, actual_day.strftime("%d/%m/%Y")
566
567 if period_kind == "week":
568     target = pd.Timestamp(selected_value).normalize()
569     week_start = df.index.normalize() - pd.to_timedelta(df.index.dayofweek, unit="D")
570     mask = week_start == target
571     if not mask.any() and allow_fallback:
572         iso_target = target.isocalendar()
573         iso_current = df.index.isocalendar()
574         mask = iso_current.week.astype(int) == int(iso_target.week)
575     result = df.loc[mask].copy().sort_index(kind="mergesort")
576     if result.empty:
577         end = target + pd.Timedelta(days=6)
578         return result, f"{target.strftime('%d/%m/%Y')} - {end.strftime('%d/%m/%Y')}"
579     actual_start = result.index.min().normalize()
580     actual_start = actual_start - pd.Timedelta(days=int(actual_start.dayofweek))
581     actual_end = actual_start + pd.Timedelta(days=6)
582     return result, f"{actual_start.strftime('%d/%m/%Y')} - {actual_end.strftime('%d/%m/%Y')}"
583
584 target = pd.Period(selected_value, freq="M")
585 mask = df.index.to_period("M") == target
586 if not mask.any() and allow_fallback:
587     mask = df.index.month == int(target.month)
588 result = df.loc[mask].copy().sort_index(kind="mergesort")
589 if result.empty:
590     return result, target.strftime("%m/%Y")
591 actual_month = result.index[0].to_period("M")
592 return result, actual_month.strftime("%m/%Y")
593
594
595 def apply_real_period_axis(
596     fig: go.Figure,
597     obs: pd.DataFrame,
598     period_kind: str,
599 ) -> go.Figure:
600     """Mapeja els rastres de figures de sèries temporals en brut en un eix de temps relatiu compacte."""
601
602     if not has_figure_data(fig):
603         return fig
604
605     df = prepare_obs_time_index(obs)
606     if df.empty or not isinstance(df.index, pd.DatetimeIndex):
607         return fig
608
609     start = df.index.min()
610     elapsed = pd.Series(
611         (df.index - start).total_seconds(),
612         index=df.index,
613         dtype=float,
614     )
615
616     if period_kind == "day":
617         relative_x = elapsed / 3600.0
618         markers = pd.Series(df.index.floor("h"), index=df.index)
619         start_flags = markers.ne(markers.shift())
620         tickvals = relative_x[start_flags].tolist()
621         ticktext = [stamp.strftime("%H:%M") for stamp in markers[start_flags].tolist()]
622         axis_title = "Hores del dia"
623     elif period_kind == "week":
624         relative_x = elapsed / 86400.0
625         markers = pd.Series(df.index.floor("D"), index=df.index)
626         start_flags = markers.ne(markers.shift())
627         tickvals = relative_x[start_flags].tolist()
628         ticktext = [f"Dia {idx}" for idx in range(1, len(tickvals) + 1)]
629         axis_title = "Dies del període"
630     else:
631         relative_x = elapsed / 86400.0
632         markers = pd.Series(df.index.floor("D"), index=df.index)
633         start_flags = markers.ne(markers.shift())
634         tickvals = relative_x[start_flags].tolist()
635         ticktext = [stamp.day for stamp in markers[start_flags].tolist()]
636         axis_title = "Dies del mes"
637
638     if not tickvals:
639         return fig

```

```

640
641 relative_x_values = relative_x.tolist()
642 period_start_x_values = relative_x[start_flags].tolist()
643 hour_markers = pd.Series(df.index.floor("h"), index=df.index)
644 hour_start_flags = hour_markers.ne(hour_markers.shift())
645 hour_start_x_values = relative_x[hour_start_flags].tolist()
646 for trace in fig.data:
647     try:
648         # Cada figura pot venir amb granularitats diferents: punts raw, agregats per hora
649         # o agregats pel període. Triem l'eix relatiu mirant quants punts té la traça.
650         x_values = getattr(trace, "x", None)
651         if x_values is None:
652             continue
653         if len(x_values) == len(relative_x_values):
654             trace.x = relative_x_values
655         elif len(x_values) == len(hour_start_x_values):
656             trace.x = hour_start_x_values
657         elif len(x_values) == len(period_start_x_values):
658             trace.x = period_start_x_values
659     except Exception:
660         continue
661
662 fig.update_xaxes(
663     title=axis_title,
664     tickmode="array",
665     tickvals=tickvals,
666     ticktext=ticktext,
667     range=[float(relative_x.iloc[0]), float(relative_x.iloc[-1])],
668 )
669 return fig
670
671
672 def flatten_trace_values(values: Any) -> list[Any]:
673     """Retorna valors escalars de les matrius de traça Plotly possiblement imbricades."""
674
675     # Plotly pot guardar dades com list, Series, ndarray o fins i tot DataFrame.
676     # Ho aplanem tot per poder decidir si una figura té valors numèrics reals.
677     if values is None:
678         return []
679
680     if isinstance(values, (str, bytes)):
681         return [values]
682
683     if isinstance(values, pd.DataFrame):
684         return values.to_numpy().ravel().tolist()
685
686     if isinstance(values, pd.Series):
687         return values.tolist()
688
689     if hasattr(values, "ravel"):
690         try:
691             return values.ravel().tolist()
692         except Exception:
693             pass
694
695     if hasattr(values, "to_numpy"):
696         try:
697             return values.to_numpy().ravel().tolist()
698         except Exception:
699             pass
700
701     try:
702         iterator = list(values)
703     except TypeError:
704         return [values]
705
706     flattened: list[Any] = []
707     for item in iterator:
708         # Les traces de heatmap solen venir imbricades; fem recursió només amb llistes
709         # i tuples per no tractar strings com a seqüències de caràcters.
710         if isinstance(item, (str, bytes)):
711             flattened.append(item)
712             continue
713         if isinstance(item, pd.Series):
714             flattened.extend(item.tolist())
715             continue
716         if isinstance(item, pd.DataFrame):
717             flattened.extend(item.to_numpy().ravel().tolist())
718             continue
719         if hasattr(item, "ravel"):
720             try:
721                 flattened.extend(item.ravel().tolist())
722             except Exception:
723                 pass
724         if isinstance(item, (list, tuple)):
725             flattened.extend(flatten_trace_values(item))
726             continue
727         flattened.append(item)
728
729     return flattened
730
731

```

```

732
733 def trace_has_numeric_values(trace: go.BaseTraceType) -> bool:
734     """Retorna si una traça Plotly conté almenys un valor numèric."""
735
736     for attribute in ("y", "z", "r", "values"):
737         values = flatten_trace_values(getattr(trace, attribute, None))
738         if not values:
739             continue
740         numeric_values = pd.to_numeric(pd.Series(values), errors="coerce").dropna()
741         if not numeric_values.empty:
742             return True
743     return False
744
745
746 def has_figure_data(fig: go.Figure) -> bool:
747     """Retorna si una figura Plotly té almenys un rastre amb valors traçables."""
748
749     if fig is None:
750         return False
751
752     traces = getattr(fig, "data", [])
753     if not traces:
754         return False
755
756     return any(trace_has_numeric_values(trace) for trace in traces)
757
758
759 def has_action_figure_data(fig: go.Figure) -> bool:
760     """Retorna si una figura d'accions d'agent conté una acció real diferent de zero."""
761
762     if not has_figure_data(fig):
763         return False
764
765     for trace in getattr(fig, "data", []):
766         values = flatten_trace_values(getattr(trace, "y", None))
767         if not values:
768             continue
769         numeric_values = pd.to_numeric(pd.Series(values), errors="coerce").dropna()
770         if not numeric_values.empty and bool((numeric_values.abs() > 0).any()):
771             return True
772
773     return False
774
775
776 def add_comfort_percentage_kpi(
777     kpi_dict: dict,
778     df: pd.DataFrame,
779     comfort_config: Any = None,
780 ) -> None:
781     """Afegeix un percentatge d'hores de confort KPI al diccionari KPI proporcionat al seu lloc."""
782
783     violation_column = next(
784         (column for column in df.columns if re.search(r"(?i)temp[_]?violation", column)),
785         None,
786     )
787     if violation_column is not None:
788         series = pd.to_numeric(df[violation_column], errors="coerce").dropna()
789         if not series.empty:
790             kpi_dict["% Hores en Confort"] = round(float((series <= 1e-9).mean() * 100), 1)
791             return
792
793     try:
794         from backend.grafics.figures_common import _ensure_temp_violation_column
795
796         with_violation = _ensure_temp_violation_column(df, comfort_config)
797         violation = pd.to_numeric(with_violation["temp_violation"], errors="coerce").dropna()
798         if not violation.empty:
799             kpi_dict["% Hores en Confort"] = round(float((violation <= 1e-9).mean() * 100), 1)
800             return
801     except Exception:
802         pass
803
804     average_violation = kpi_dict.get("Avg Temp Violation (All Hours, C)")
805     if average_violation is not None:
806         kpi_dict["% Hores en Confort"] = (
807             100.0 if float(average_violation) == 0.0 else None
808         )
809
810
811 def overlay_comparison_traces(
812     main_fig: go.Figure,
813     comp_fig: go.Figure,
814     label_suffix: str = " (ref)",
815 ) -> go.Figure:
816     """Superposeu traces de referència en una figura principal quan el tipus de gràfic ho admet."""
817
818     if not has_figure_data(comp_fig):
819         return main_fig
820
821     main_barmode = (main_fig.layout.barmode or "").lower()
822     if main_barmode in ("stack", "relative"):
823         return main_fig

```

```

824
825 if any(getattr(trace, "type", "") == "pie" for trace in main_fig.data) or any(
826     getattr(trace, "type", "") == "pie" for trace in comp_fig.data
827 ):
828     return main_fig
829 if any(getattr(trace, "type", "") == "heatmap" for trace in main_fig.data) or any(
830     getattr(trace, "type", "") == "heatmap" for trace in comp_fig.data
831 ):
832     return main_fig
833
834 result = go.Figure(main_fig)
835 main_bar_index = 0
836 for trace in result.data:
837     if isinstance(trace, go.Bar):
838         # Les barres de la run principal i la referència han d'anar en grups separats,
839         # no apilades, perquè el delta sigui llegible d'un cop d'ull.
840         trace.update(
841             offsetgroup=f"main-{main_bar_index}",
842             alignmentgroup="comparison-bars",
843         )
844         main_bar_index += 1
845     elif isinstance(trace, (go.Scatter, go.Scattergl)) and getattr(trace, "stackgroup", None):
846         trace.update(stackgroup=f"{trace.stackgroup}-main")
847
848 has_bar_traces = any(isinstance(trace, go.Bar) for trace in comp_fig.data)
849 comp_bar_index = 0
850
851 for trace in comp_fig.data:
852     trace_copy = copy.deepcopy(trace)
853     original_name = getattr(trace_copy, "name", None) or ""
854     trace_copy.name = original_name + label_suffix
855
856     if isinstance(trace_copy, (go.Scatter, go.Scattergl)):
857         # Les línies de referència van puntejades per no confondre-les amb la run actual.
858         if getattr(trace_copy, "stackgroup", None):
859             trace_copy.update(stackgroup=f"{trace_copy.stackgroup}-ref")
860             trace_copy.update(line=dict(dash="dot", width=3.0), opacity=0.92)
861         elif isinstance(trace_copy, go.Bar):
862             trace_copy.update(
863                 marker_line_width=1,
864                 offsetgroup=f"ref-{comp_bar_index}",
865                 alignmentgroup="comparison-bars",
866                 opacity=0.58,
867             )
868             comp_bar_index += 1
869         else:
870             trace_copy.update(opacity=0.55)
871
872     result.add_trace(trace_copy)
873
874 if has_bar_traces:
875     result.update_layout(barmode="group", bargap=0.18, bargroupgap=0.08)
876
877 return result
878
879
880 def _ensure_observation_time_columns(obs: pd.DataFrame) -> pd.DataFrame:
881     """Assegureu-vos que les observacions incloguin camps de mes i hora quan hi hagi marques de temps."""
882
883     if "month" in obs.columns and "hour" in obs.columns:
884         return obs
885
886     if "timestamp" in obs.columns:
887         timestamps = pd.to_datetime(obs["timestamp"], errors="coerce")
888     elif "datetime" in obs.columns:
889         timestamps = pd.to_datetime(obs["datetime"], errors="coerce")
890     else:
891         return obs
892
893     result = obs.copy()
894     result["month"] = timestamps.dt.month
895     result["hour"] = timestamps.dt.hour
896     result.index = timestamps
897     return result
898
899
900 def _derive_observation_datetime_index(df: pd.DataFrame) -> pd.DatetimeIndex:
901     """Obteniu un DatetimeIndex a partir de columnes temporals semblants a EPW o una alternativa de 15 minuts."""
902
903     if not all(column in df.columns for column in ("month", "day_of_month", "hour")):
904         return pd.date_range("2001-01-01", periods=len(df), freq="15min")
905
906     month, day, hour = _coerce_temporal_parts(df)
907
908     if "time_elapsed(hours)" in df.columns:
909         elapsed = pd.to_numeric(df["time_elapsed(hours)"], errors="coerce")
910         valid = elapsed.notna() & month.notna() & day.notna() & hour.notna()
911         if valid.any():
912             first_idx = valid[valid].index[0]
913             first_pos = df.index.get_loc(first_idx)
914             first_elapsed = float(elapsed.iloc[first_pos])
915             base_minutes = int(round((first_elapsed % 1.0) * 60.0)) % 60

```

```

916     base_ts = pd.Timestamp(
917         year=2001,
918         month=int(month.iloc[first_pos]),
919         day=int(day.iloc[first_pos]),
920         hour=int(hour.iloc[first_pos]),
921         minute=base_minutes,
922     )
923     offset_minutes = ((elapsed - first_elapsed) * 60.0).round()
924     try:
925         return pd.DatetimeIndex(base_ts + pd.to_timedelta(offset_minutes, unit="m"))
926     except (OverflowError, ValueError):
927         pass
928
929     hour_blocks = (
930         pd.DataFrame({"month": month, "day": day, "hour": hour})
931         .ne(pd.DataFrame({"month": month, "day": day, "hour": hour}).shift())
932         .any(axis=1)
933         .cumsum()
934     )
935     minute_slot = hour_blocks.groupby(hour_blocks).cumcount() * 15
936     return pd.DatetimeIndex(
937         pd.to_datetime(
938             {
939                 "year": 2001,
940                 "month": month.astype("float64"),
941                 "day": day.astype("float64"),
942                 "hour": hour.astype("float64"),
943                 "minute": minute_slot.astype("float64"),
944             },
945             errors="coerce",
946         )
947     )

```

E.27 backend/resultats_report_backend.py

Listing 48: backend/resultats_report_backend.py

```

1  """Generació d'informes HTML i PDF per a les execucions de Studio completes.
2
3  El backend de l'informe comparteix els carregadors del panell de resultats i la figura Plotly
4  builders, després organitza els gràfics disponibles en un document estructurat. Figures
5  sense dades utilitzables es salten, de manera que els informes exportats reflecteixen els artefactes
6  produït realment per una tirada.
7  """
8
9  from __future__ import annotations
10
11  import base64
12  import math
13  from dataclasses import dataclass
14  from datetime import datetime
15  from html import escape
16  from numbers import Number
17
18  import plotly.graph_objects as go
19
20  from backend.grafics.comfort_scope import comfort_scope_label
21  from backend.grafics.figures_common import (
22      _fix_colors_for_visibility,
23      make_agent_actions_plot,
24      make_battery_charge_with_price_plot,
25      make_battery_power_plot,
26      make_battery_soc_plot,
27      make_battery_vs_grid_plot,
28      make_comfort_compliance,
29      make_energy_price_plot,
30      make_episode_metrics,
31      make_heatmap,
32      make_hvac_consumption_plot,
33      make_hvac_meter_breakdown_plot,
34      make_indoor_humidity_plot,
35      make_indoor_temperature_plot,
36      make_radiant_control_plot,
37      make_setpoints_plot,
38      make_setpoints_vs_indoor_plot,
39      make_violation_bars,
40      make_violation_heatmap_percent,
41      make_zone_temperature_heatmaps,
42      style_figure_semantics,
43  )
44  from backend.grafics.figures_zones import filter_obs_by_zone
45  from backend.grafics.kpis import compute_kpis
46  from backend.resultats_backend import (
47      DashboardData,
48      add_comfort_percentage_kpi,
49      has_action_figure_data,
50      has_figure_data,
51      load_dashboard_data,

```



```

52     select_actions_for_obs,
53     select_radiant_action_data,
54 )
55
56
57 @dataclass(frozen=True)
58 class ReportFigure:
59     """Plotly gràfic més la secció d'informe on hauria d'aparèixer."""
60
61     section: str
62     title: str
63     figure: go.Figure
64
65
66 def generate_report_bytes(
67     progress_path: str,
68     observations_path: str,
69     agg_mode: str,
70     season: str,
71     comfort_scope: str,
72     zone_str: str,
73 ) -> tuple[bytes, str]:
74     """Crea un informe que es pot descarregar per a una execució completada.
75
76     La funció retorna PDF bytes quan ``pdfkit`` i ``wkhtmltopdf`` són
77     disponible. Si la representació estàtica PDF no està disponible, retorna un informe HTML
78     amb el mateix contingut perquè UI encara pugui oferir una exportació completa.
79     """
80
81     data = load_dashboard_data(progress_path, observations_path)
82     selected_zone = None if zone_str in ("ALL", "") else zone_str
83     zone_obs = filter_obs_by_zone(data.obs, selected_zone, data.yaml_cfg)
84     action_data = select_actions_for_obs(data.actions, zone_obs)
85     action_data = select_radiant_action_data(action_data, data)
86
87     kpis = compute_kpis(
88         obs=zone_obs,
89         cost_hourly=data.metrics_dict.get("cost_hourly"),
90         selected_zone=selected_zone,
91         comfort_scope=comfort_scope,
92         comfort_config=data.yaml_cfg,
93     )
94     kpis.pop("Avg Energy Cost (EUR/h)", None)
95     add_comfort_percentage_kpi(kpis, zone_obs, data.yaml_cfg)
96
97     figures = _build_report_figures(
98         data=data,
99         zone_obs=zone_obs,
100         action_data=action_data,
101         agg_mode=agg_mode,
102         season=season,
103         comfort_scope=comfort_scope,
104     )
105     return _render_report_document(
106         data=data,
107         kpis=kpis,
108         figures=figures,
109         agg_mode=agg_mode,
110         season=season,
111         selected_zone=selected_zone,
112         comfort_scope=comfort_scope,
113     )
114
115
116 def _build_report_figures(
117     data: DashboardData,
118     zone_obs,
119     action_data,
120     agg_mode: str,
121     season: str,
122     comfort_scope: str,
123 ) -> tuple[ReportFigure, ...]:
124     """Crea xifres gràfics incloses en un informe de resultats exportat per al flux Studio."""
125
126     # L'informe reutilitza les mateixes factories del dashboard per evitar que PDF i UI
127     # expliquin històries diferents.
128     comfort_mode = "raw" if agg_mode == "day" else agg_mode
129     pivot = data.metrics_dict.get("pivot_consumption")
130     action_fig = make_agent_actions_plot(action_data, agg_mode, season)
131     if not has_action_figure_data(action_fig):
132         action_fig = go.Figure()
133
134     return (
135         ReportFigure(
136             "Confort i clima interior",
137             "Confort (%)",
138             make_comfort_compliance(
139                 zone_obs,
140                 comfort_mode,
141                 season,
142                 comfort_scope=comfort_scope,
143                 comfort_config=data.yaml_cfg,

```

```

144     ),
145 ),
146 ReportFigure(
147     "Confort i clima interior",
148     "Temperatura interior vs exterior",
149     make_indoor_temperature_plot(zone_obs, agg_mode, season),
150 ),
151 ReportFigure(
152     "Confort i clima interior",
153     "Humitat interior vs exterior",
154     make_indoor_humidity_plot(zone_obs, agg_mode, season),
155 ),
156 ReportFigure(
157     "Confort i clima interior",
158     "Infraccions de confort",
159     _fix_colors_for_visibility(
160         make_violation_bars(
161             zone_obs,
162             agg_mode,
163             season,
164             comfort_scope=comfort_scope,
165             comfort_config=data.yaml_cfg,
166         ),
167         kind="violation",
168         mode=agg_mode,
169     ),
170 ),
171 ReportFigure(
172     "Confort i clima interior",
173     "Mapa de calor d'infraccions de confort (%)",
174     make_violation_heatmap_percent(
175         zone_obs,
176         season=season,
177         comfort_config=data.yaml_cfg,
178     ),
179 ),
180 ReportFigure(
181     "Confort i clima interior",
182     "Temperatura per zona (mes x hora)",
183     make_zone_temperature_heatmaps(
184         zone_obs,
185         season=season,
186         agg="mean",
187         max_cols=3,
188     ),
189 ),
190 ReportFigure(
191     "Consum i energia",
192     "Consum HVAC (kWh)",
193     _fix_colors_for_visibility(
194         make_hvac_consumption_plot(zone_obs, agg_mode, season),
195         kind="hvac",
196         mode=agg_mode,
197     ),
198 ),
199 ReportFigure(
200     "Consum i energia",
201     "Desglossament HVAC per meter (kWh)",
202     make_hvac_meter_breakdown_plot(zone_obs, agg_mode, season),
203 ),
204 ReportFigure(
205     "Consum i energia",
206     "Preu energia (EUR/kWh)",
207     make_energy_price_plot(zone_obs, agg_mode, season),
208 ),
209 ReportFigure(
210     "Consum i energia",
211     "Mapa de calor global (consum)",
212     (
213         make_heatmap(pivot, "Total HVAC Consumption (kWh)")
214         if pivot is not None and not pivot.empty
215         else go.Figure()
216     ),
217 ),
218 ReportFigure(
219     "Control HVAC",
220     "Actuació tèrmica (setpoints)",
221     make_setpoints_plot(zone_obs, agg_mode, season),
222 ),
223 ReportFigure(
224     "Control HVAC",
225     "Setpoints vs temperatura interior",
226     make_setpoints_vs_indoor_plot(zone_obs, agg_mode, season),
227 ),
228 ReportFigure(
229     "Control HVAC",
230     "Control radiant",
231     make_radiant_control_plot(zone_obs, agg_mode, season),
232 ),
233 ReportFigure("Control HVAC", "Accions de l'agent", action_fig),
234 ReportFigure(
235     "Bateria i xarxa",

```

```

236         "Càrrega/descàrrega bateria",
237         make_battery_power_plot(zone_obs, agg_mode, season),
238     ),
239     ReportFigure(
240         "Bateria i xarxa",
241         "SOC bateria",
242         make_battery_soc_plot(zone_obs, agg_mode, season),
243     ),
244     ReportFigure(
245         "Bateria i xarxa",
246         "Energia bateria vs xarxa",
247         make_battery_vs_grid_plot(zone_obs, agg_mode, season),
248     ),
249     ReportFigure(
250         "Bateria i xarxa",
251         "Bateria vs preu energia",
252         make_battery_charge_with_price_plot(zone_obs, agg_mode, season),
253     ),
254     ReportFigure(
255         "Episodi",
256         "Mètriques d'episodi",
257         make_episode_metrics(data.progress),
258     ),
259 )
260
261 def _render_report_document(
262     data: DashboardData,
263     kpis: dict,
264     figures: tuple[ReportFigure, ...],
265     agg_mode: str,
266     season: str,
267     selected_zone: str | None,
268     comfort_scope: str,
269 ) -> tuple[bytes, str]:
270     """Renderitza l'informe HTML i converteix-lo a PDF quan el pdfkit estigui disponible."""
271
272     # Muntem primer HTML complet: és fàcil de revisar al navegador i després pdfkit el pot
273     # convertir sense haver de mantenir una plantilla paral·lela.
274     zone_label = selected_zone if selected_zone else "Totes"
275     scope_label = comfort_scope_label(comfort_scope)
276     rendered_figures, chart_count, static_charts = _render_report_figures(figures)
277     generated_at = datetime.now().strftime("%d/%m/%Y %H:%M")
278     timesteps_label = f"{data.timesteps_num:,"}.replace(",", ".")
279
280     html_parts: list[str] = [f'<!DOCTYPE html>']
281
282     <html>
283     <head>
284         <meta charset="utf-8">
285         <title>Sinergym Dashboard Report</title>
286         <style>
287             @page {{ size: A4; margin: 15mm 13mm; }}
288             * {{ box-sizing: border-box; }}
289             body {{
290                 margin: 0;
291                 background: #eef3f8;
292                 color: #17233c;
293                 font-family: "Segoe UI", Arial, sans-serif;
294                 line-height: 1.45;
295             }}
296             .report-shell {{
297                 max-width: 1120px;
298                 margin: 0 auto;
299                 padding: 22px;
300                 background: #ffffff;
301             }}
302             .cover {{
303                 padding: 30px 34px;
304                 border-radius: 14px;
305                 color: #ffffff;
306                 background: linear-gradient(135deg, #0f766e 0%, #1d4ed8 100%);
307                 page-break-inside: avoid;
308             }}
309             .eyebrow {{
310                 margin: 0 0 10px;
311                 font-size: 12px;
312                 font-weight: 700;
313                 letter-spacing: 0.08em;
314                 text-transform: uppercase;
315                 opacity: 0.86;
316             }}
317             h1 {{
318                 margin: 0;
319                 font-size: 32px;
320                 line-height: 1.15;
321                 letter-spacing: 0;
322             }}
323             .subtitle {{
324                 max-width: 780px;
325                 margin: 12px 0 0;
326                 color: rgba(255, 255, 255, 0.88);
327                 font-size: 14px;

```

```

328 }}
329 .summary-grid,
330 .filter-strip,
331 .kpi-row {{
332     display: flex;
333     flex-wrap: wrap;
334 }}
335 .summary-grid {{
336     margin: 22px -5px -5px 0;
337 }}
338 .summary-item {{
339     width: 24%;
340     min-height: 72px;
341     margin: 0 5px 5px 0;
342     padding: 12px 14px;
343     border: 1px solid rgba(255, 255, 255, 0.24);
344     border-radius: 8px;
345     background: rgba(255, 255, 255, 0.12);
346 }}
347 .summary-label {{
348     font-size: 11px;
349     text-transform: uppercase;
350     color: rgba(255, 255, 255, 0.72);
351 }}
352 .summary-value {{
353     margin-top: 4px;
354     font-size: 15px;
355     font-weight: 700;
356     overflow-wrap: anywhere;
357 }}
358 .filter-strip {{
359     margin: 18px -8px 8px 0;
360 }}
361 .filter-item {{
362     width: 24%;
363     min-height: 70px;
364     margin: 0 8px 8px 0;
365     padding: 12px 14px;
366     border: 1px solid #d7deeb;
367     border-radius: 8px;
368     background: #f8fbff;
369 }}
370 .filter-label {{
371     color: #64748b;
372     font-size: 11px;
373     font-weight: 700;
374     text-transform: uppercase;
375 }}
376 .filter-value {{
377     margin-top: 3px;
378     font-size: 14px;
379     font-weight: 700;
380     color: #17233c;
381 }}
382 .block-title {{
383     margin: 26px 0 12px;
384     color: #17233c;
385     font-size: 20px;
386     line-height: 1.2;
387 }}
388 .kpi-row {{
389     margin: 0 -12px 18px 0;
390 }}
391 .kpi {{
392     width: 24%;
393     min-height: 92px;
394     margin: 0 12px 12px 0;
395     border: 1px solid #d7deeb;
396     border-radius: 8px;
397     padding: 14px 16px;
398     background: #ffffff;
399     page-break-inside: avoid;
400 }}
401 .kpi-val {{
402     color: #0f766e;
403     font-size: 24px;
404     font-weight: 800;
405     line-height: 1.15;
406 }}
407 .kpi-lbl {{
408     margin-top: 7px;
409     color: #64748b;
410     font-size: 11px;
411     font-weight: 700;
412     letter-spacing: 0.02em;
413     text-transform: uppercase;
414 }}
415 .report-section {{
416     margin-top: 28px;
417 }}
418 .section-heading {{
419     margin: 0 0 12px;

```

```

420 padding-left: 12px;
421 border-left: 5px solid #0f766e;
422 color: #17233c;
423 font-size: 18px;
424 font-weight: 800;
425 line-height: 1.25;
426 page-break-after: avoid;
427 }}
428 .chart {{
429 margin: 0 0 20px;
430 padding: 14px;
431 border: 1px solid #d7deeb;
432 border-radius: 8px;
433 background: #ffffff;
434 page-break-inside: avoid;
435 }}
436 .chart h3 {{
437 margin: 0 0 10px;
438 color: #17233c;
439 font-size: 15px;
440 line-height: 1.25;
441 }}
442 .chart img {{
443 display: block;
444 width: 100%;
445 max-width: 100%;
446 border: 0;
447 }}
448 .chart .plotly-graph-div {{
449 width: 100% !important;
450 }}
451 .empty-note {{
452 margin: 20px 0 0;
453 padding: 14px 16px;
454 border: 1px solid #d7deeb;
455 border-radius: 8px;
456 color: #64748b;
457 background: #f8fbff;
458 }}
459 @media print {{
460 body {{ background: #ffffff; }}
461 .report-shell {{ padding: 0; }}
462 .cover {{ border-radius: 0; }}
463 }}
464 </style>
465 </head>
466 <body>
467 <main class="report-shell">
468 <section class="cover">
469 <p class="eyebrow">BEMS-RL Studio · Informe de resultats</p>
470 <h1>Resum {escape(data.run_mode)} - Sinergym</h1>
471 <p class="subtitle">
472 Informe generat automàticament amb els indicadors principals i tots els gràfics
473 disponibles per a l'execució seleccionada.
474 </p>
475 <div class="summary-grid">
476 <div class="summary-item">
477 <div class="summary-label">Entorn</div>
478 <div class="summary-value">{escape(data.env_name)}</div>
479 </div>
480 <div class="summary-item">
481 <div class="summary-label">Timesteps</div>
482 <div class="summary-value">{timesteps_label}</div>
483 </div>
484 <div class="summary-item">
485 <div class="summary-label">Model</div>
486 <div class="summary-value">{escape(data.model_name)}</div>
487 </div>
488 <div class="summary-item">
489 <div class="summary-label">Gràfics inclosos</div>
490 <div class="summary-value">{chart_count}</div>
491 </div>
492 </div>
493 </section>
494
495 <section class="filter-strip">
496 <div class="filter-item">
497 <div class="filter-label">Generat</div>
498 <div class="filter-value">{generated_at}</div>
499 </div>
500 <div class="filter-item">
501 <div class="filter-label">Agregació</div>
502 <div class="filter-value">{escape(agg_mode.upper())}</div>
503 </div>
504 <div class="filter-item">
505 <div class="filter-label">Estació</div>
506 <div class="filter-value">{escape(season)}</div>
507 </div>
508 <div class="filter-item">
509 <div class="filter-label">Zona / confort</div>
510 <div class="filter-value">{escape(zone_label)} · {escape(scope_label)}</div>
511 </div>

```

```

512 </section>
513
514 <h2 class="block-title">Indicadors clau</h2>
515 <div class="kpi-row">"""]
516
517 for key, value in kpis.items():
518     value_string = _format_kpi_value(key, value)
519     html_parts.append(
520         f'<div class="kpi"><div class="kpi-val">{escape(value_string)}</div>'
521         f'<div class="kpi-lbl">{escape(key)}</div></div>'
522     )
523
524 html_parts.append("</div>")
525 if rendered_figures:
526     html_parts.extend(rendered_figures)
527 else:
528     html_parts.append(
529         '<div class="empty-note">No hi ha gràfics amb dades disponibles per als filtres seleccionats.</div>'
530     )
531 html_parts.append("</main></body></html>")
532 html = "\n".join(html_parts)
533
534 if static_charts:
535     try:
536         import pdfkit
537
538         options = {
539             "page-size": "A4",
540             "margin-top": "0.45in",
541             "margin-right": "0.45in",
542             "margin-bottom": "0.45in",
543             "margin-left": "0.45in",
544             "encoding": "UTF-8",
545             "enable-local-file-access": None,
546             "disable-smart-shrinking": None,
547             "quiet": "",
548         }
549         return pdfkit.from_string(html, False, options=options), "pdf"
550     except Exception:
551         pass
552
553 return html.encode("utf-8"), "html"
554
555
556 def _format_kpi_value(key: str, value) -> str:
557     """Formata els valors KPI de manera coherent per a l'informe exportat."""
558
559     if value is None:
560         return "N/D"
561     if isinstance(value, Number):
562         numeric = float(value)
563         if not math.isfinite(numeric):
564             return "N/D"
565         if key.strip().startswith("%"):
566             return f"{numeric:.1f}%"
567         return f"{numeric:.2f}"
568     return str(value)
569
570
571 def _render_report_figures(
572     figures: tuple[ReportFigure, ...],
573 ) -> tuple[list[str], int, bool]:
574     """Retorna fragments seccionats de HTML per a cada figura d'informe amb dades visibles."""
575
576     rendered: list[str] = []
577     current_section: str | None = None
578     chart_count = 0
579     static_charts = True
580     plotlyjs_included = False
581
582     for item in figures:
583         if not has_figure_data(item.figure):
584             continue
585
586         chart_html, is_static, plotlyjs_used = _render_single_report_figure(
587             item,
588             include_plotlyjs=not plotlyjs_included,
589         )
590         if chart_html is None:
591             continue
592
593         if item.section != current_section:
594             if current_section is not None:
595                 rendered.append("</section>")
596             current_section = item.section
597             rendered.append(
598                 f'<section class="report-section"><h2 class="section-heading">'
599                 f'{escape(item.section)}</h2>'
600             )
601
602         rendered.append(chart_html)
603         chart_count += 1

```

```

604     static_charts = static_charts and is_static
605     plotlyjs_included = plotlyjs_included or plotlyjs_used
606
607     if current_section is not None:
608         rendered.append("</section>")
609
610     return rendered, chart_count, static_charts
611
612
613 def _render_single_report_figure(
614     item: ReportFigure,
615     *,
616     include_plotlyjs: bool,
617 ) -> tuple[str | None, bool, bool]:
618     """Renderitza una figura com a imatge estàtica, recorrent a HTML interactiu."""
619
620     export_fig = _prepare_export_figure(item.figure)
621     try:
622         image_bytes = export_fig.to_image(format="png", engine="kaleido", scale=2)
623         image_b64 = base64.b64encode(image_bytes).decode()
624         return (
625             f'<div class="chart"><h3>{escape(item.title)}</h3>'
626             f'</div>',
627             True,
628             False,
629         )
630     except Exception:
631         try:
632             chart_html = export_fig.to_html(
633                 full_html=False,
634                 include_plotlyjs=True if include_plotlyjs else False,
635                 config={"displayModeBar": False, "responsive": True},
636             )
637             return (
638                 f'<div class="chart"><h3>{escape(item.title)}</h3>{chart_html}</div>',
639                 False,
640                 include_plotlyjs,
641             )
642         except Exception:
643             return None, False, False
644
645
646 def _prepare_export_figure(fig: go.Figure) -> go.Figure:
647     """Copia i normalitza una figura Plotly per a la representació d'informes."""
648
649     export_fig = style_figure_semantics(go.Figure(fig))
650     current_height = export_fig.layout.height
651     try:
652         height = int(current_height) if current_height else 520
653     except (TypeError, ValueError):
654         height = 520
655     height = max(460, min(height, 980))
656     export_fig.update_layout(
657         width=1100,
658         height=height,
659         margin=dict(l=66, r=36, t=78, b=58),
660         font=dict(size=13, color="#17233c"),
661         title=dict(
662             text=export_fig.layout.title.text,
663             x=0.01,
664             xanchor="left",
665             font=dict(size=18, color="#17233c"),
666         ),
667         legend=dict(
668             orientation="h",
669             yanchor="bottom",
670             y=1.02,
671             xanchor="right",
672             x=1,
673             bgcolor="rgba(255,255,255,0.92)",
674             bordercolor="#d7deeb",
675             borderwidth=1,
676         ),
677     )
678     return export_fig

```

E.28 backend/sb3_utils.py

Listing 49: backend/sb3_utils.py

```

1  """Descoberta, càrrega i format d'accions per a models Stable-Baselines3.
2
3  Els fluxos de treball d'avaluació i control en directe utilitzen aquest mòdul per localitzar polítiques desades,
4  carregar fitxers zip SB3, adjuntar estadístiques VecNormalize, deduir Sinergym compatibles
5  entorns i convertir les accions en representacions amigables amb UI.
6  """
7
8  from __future__ import annotations

```

```

9
10 import io
11 import re
12 from pathlib import Path
13 from typing import Any, Callable, Dict, List, Optional, Tuple
14
15 import gymnasium as gym
16 import numpy as np
17 import pandas as pd
18 from gymnasium.spaces import Box, Discrete, MultiBinary, MultiDiscrete
19 from stable_baselines3 import A2C, DDPG, DQN, PPO, SAC, TD3
20 from stable_baselines3.common.base_class import BaseAlgorithm
21 from stable_baselines3.common.save_util import load_from_zip_file
22 from stable_baselines3.common.vec_env import DummyVecEnv, VecEnv, VecNormalize
23
24
25 ALGOS: Dict[str, type] = {"PPO": PPO, "A2C": A2C, "DQN": DQN, "SAC": SAC, "TD3": TD3, "DDPG": DDPG}
26 VECNORM_SUFFIXES: List[str] = ["vecnormalize.pkl", "vecnorm.pkl", "vec_norm.pkl", "normalize.pkl"]
27
28
29 def scan_model_zips(dir_list_text: str) -> pd.DataFrame:
30     """Escaneja directoris separats per comes per trobar models Stable-Baselines3 ``.zip``."""
31     rows: List[Dict[str, Any]] = []
32     for chunk in dir_list_text.split(","):
33         root = Path(chunk.strip())
34         if not root.exists():
35             continue
36         for path in root.rglob("*.zip"):
37             rows.append(
38                 {
39                     "name": path.name,
40                     "stem": path.stem,
41                     "path": str(path.resolve()),
42                     "dir": str(path.parent.resolve()),
43                 }
44             )
45     if not rows:
46         return pd.DataFrame(columns=["name", "stem", "path", "dir"])
47     return pd.DataFrame(rows).sort_values(by=["stem", "name"]).reset_index(drop=True)
48
49
50 def candidate_vecnorm(model_path: Path) -> Optional[Path]:
51     """Retorna el fitxer d'estadístiques VecNormalize més probable al costat d'un zip de model."""
52     folder = model_path.parent
53     stem = model_path.stem.lower()
54     for suffix in VECNORM_SUFFIXES:
55         for candidate in [folder / f"{model_path.stem}_{suffix}", folder / f"{stem}_{suffix}"]:
56             if candidate.exists():
57                 return candidate
58     for file_path in folder.glob("*.pkl"):
59         if any(file_path.name.lower().endswith(suffix) for suffix in VECNORM_SUFFIXES):
60             return file_path
61     return None
62
63
64 def _algo_cls_from_meta(meta: Dict[str, Any]) -> Optional[type]:
65     """Resol la classe d'algorisme SB3 declarada a les metadades del model."""
66     algo = (meta or {}).get("algo", None)
67     if isinstance(algo, str) and algo.upper() in ALGOS:
68         return ALGOS[algo.upper()]
69     return None
70
71
72 def load_sb3_model_bytes(model_bytes: bytes, device: str = "cpu") -> Tuple[BaseAlgorithm, Dict[str, Any]]:
73     """Carrega un model Stable-Baselines3 des de bytes ZIP i retorna les metadades del model."""
74     buffer = io.BytesIO(model_bytes)
75     meta: Dict[str, Any] = {}
76     try:
77         meta, _, _ = load_from_zip_file(buffer, device=device, print_system_info=False)
78     except Exception:
79         meta = {}
80
81     algo_cls = _algo_cls_from_meta(meta)
82     buffer.seek(0)
83     model: Optional[BaseAlgorithm] = None
84
85     if algo_cls is not None:
86         model = algo_cls.load(buffer, device=device)
87     else:
88         for algo_type in ALGOS.values():
89             try:
90                 buffer.seek(0)
91                 model = algo_type.load(buffer, device=device)
92                 break
93             except Exception:
94                 continue
95
96     if model is None:
97         raise ValueError("No s'ha pogut carregar el model SB3.")
98
99     return model, meta
100

```



```

101
102 def env_id_from_meta_or_name(meta: Dict[str, Any], zip_stem: str) -> Optional[str]:
103     """Dedueix un identificador d'entorn Sinergym registrat a partir de les metadades del model o el nom del fitxer."""
104     for key in ("env_id", "env", "gym_id", "environment"):
105         value = meta.get(key)
106         if isinstance(value, str) and value in gym.envs.registry.keys():
107             return value
108
109     match = re.search(r"(Eplus-[A-Za-z0-9_-]+-v\d+)", zip_stem)
110     if match:
111         return match.group(1)
112
113     tokens = [token for token in re.split(r"[_-]+", zip_stem.lower()) if len(token) >= 3]
114     candidates = [env_id for env_id in gym.envs.registry.keys() if env_id.startswith("Eplus-")]
115     best, best_score = None, 0
116     for env_id in candidates:
117         score = sum(token in env_id.lower() for token in tokens)
118         if score > best_score:
119             best, best_score = env_id, score
120     return best
121
122
123 def load_vecnormalize(vec_path: str, env_factory: Callable[[], Any]) -> VecNormalize:
124     """Carrega les estadístiques de VecNormalize sobre un entorn vectoritzat acabat de crear."""
125     env = DummyVecEnv([env_factory])
126     try:
127         env = VecNormalize.load(vec_path, env)
128     except AssertionError as exc:
129         msg = str(exc)
130         if "spaces must have the same shape" in msg:
131             import re as _re
132             m = _re.search(r"((\d+),\).*?((\d+),\)", msg)
133             detail = f"({m.group(1)} vs {m.group(2)})" if m else ""
134             raise ValueError(
135                 f"L'espai d'observació del VecNormalize no coincideix amb l'entorn seleccionat{detail}. "
136                 "Assegura't d'avaluar el model amb el mateix entorn amb que va ser entrenat."
137             ) from exc
138         raise
139     env.training = False
140     env.norm_reward = False
141     return env
142
143
144 def _unwrap_action_space(vecenv: VecEnv):
145     """Unwrap action space."""
146     return vecenv.action_space
147
148
149 def _space_shape_tuple(space: Any) -> Optional[Tuple[int, ...]]:
150     """Space shape tuple."""
151     shape = getattr(space, "shape", None)
152     if shape is None:
153         return None
154     try:
155         return tuple(int(value) for value in shape)
156     except Exception:
157         return None
158
159
160 def is_normalized_box_space(space: Any) -> bool:
161     """Is normalized box space."""
162     if not isinstance(space, Box):
163         return False
164     low = np.asarray(space.low, dtype=np.float32)
165     high = np.asarray(space.high, dtype=np.float32)
166     return bool(np.allclose(low, -1.0) and np.allclose(high, 1.0))
167
168
169 def action_spaces_compatible(expected_space: Any, actual_space: Any) -> bool:
170     """Espais d'acció compatibles."""
171     if expected_space is None or actual_space is None:
172         return True
173     # Comparar només el tipus no és suficient: en Box importen forma i límits; en
174     # MultiDiscrete importen els nvec. Això evita acceptar models quasi compatibles.
175     if isinstance(expected_space, Box) and isinstance(actual_space, Box):
176         if _space_shape_tuple(expected_space) != _space_shape_tuple(actual_space):
177             return False
178         return bool(
179             np.allclose(expected_space.low, actual_space.low)
180             and np.allclose(expected_space.high, actual_space.high)
181         )
182     if isinstance(expected_space, Discrete) and isinstance(actual_space, Discrete):
183         return int(expected_space.n) == int(actual_space.n)
184     if isinstance(expected_space, MultiDiscrete) and isinstance(actual_space, MultiDiscrete):
185         return bool(np.array_equal(expected_space.nvec, actual_space.nvec))
186     if isinstance(expected_space, MultiBinary) and isinstance(actual_space, MultiBinary):
187         return _space_shape_tuple(expected_space) == _space_shape_tuple(actual_space)
188     return type(expected_space) is type(actual_space)
189
190
191 def should_apply_normalize_action(model_action_space: Any, env_action_space: Any) -> bool:
192     """S'ha d'aplicar l'acció de normalització."""

```

```

193 if not isinstance(model_action_space, Box) or not isinstance(env_action_space, Box):
194     return False
195 if _space_shape_tuple(model_action_space) != _space_shape_tuple(env_action_space):
196     return False
197 return is_normalized_box_space(model_action_space) and not is_normalized_box_space(env_action_space)
198
199
200 def describe_action_space(space: Any) -> str:
201     """Describe action space."""
202     if isinstance(space, Box):
203         return f"Box(shape={space.shape}, low={np.asarray(space.low).tolist()}, high={np.asarray(space.high).tolist()})"
204     if isinstance(space, Discrete):
205         return f"Discrete(n={space.n})"
206     if isinstance(space, MultiDiscrete):
207         return f"MultiDiscrete(nvec={np.asarray(space.nvec).tolist()})"
208     if isinstance(space, MultiBinary):
209         return f"MultiBinary(shape={space.shape})"
210     return type(space).__name__
211
212
213 def _as_action_array(action: Any) -> np.ndarray:
214     """Converteix una acció SB3 en ndarray sense assumir encara la forma final."""
215     if isinstance(action, np.ndarray):
216         return action
217     if isinstance(action, (list, tuple)):
218         return np.asarray(action)
219     return np.asarray([action])
220
221
222 def _reshape_action_batch(
223     action_values: np.ndarray,
224     action_shape: Tuple[int, ...],
225     n_envs: int,
226     dtype: Any,
227 ) -> np.ndarray:
228     """Porta una acció plana a la forma ``(n_envs, *action_shape)`` esperada per VecEnv."""
229     action_dimension = int(np.prod(action_shape))
230     flat_action = action_values.astype(dtype).reshape(-1)
231
232     # SB3 pot retornar una sola acció, una acció per entorn o una forma intermèdia.
233     # Aquí fem que totes acabin amb la mateixa estructura abans d'aplicar límits.
234     if flat_action.size == 1 and action_dimension > 1:
235         flat_action = np.repeat(flat_action.item(), action_dimension)
236     elif flat_action.size > action_dimension and flat_action.size % action_dimension == 0:
237         pass
238     elif flat_action.size > action_dimension:
239         flat_action = flat_action[:action_dimension]
240     elif flat_action.size < action_dimension:
241         fill_value = flat_action[-1] if flat_action.size else 0
242         flat_action = np.concatenate(
243             [flat_action, np.repeat(fill_value, action_dimension - flat_action.size)]
244         )
245
246     action_batch = flat_action.reshape(-1, *action_shape)
247     if action_batch.shape[0] == 1 and n_envs > 1:
248         action_batch = np.repeat(action_batch, n_envs, axis=0)
249     elif action_batch.shape[0] > n_envs:
250         action_batch = action_batch[:n_envs]
251     elif action_batch.shape[0] < n_envs:
252         action_batch = np.concatenate(
253             [action_batch] + [action_batch[-1:] * (n_envs - action_batch.shape[0]),
254                             axis=0,
255         )
256     return action_batch.astype(dtype)
257
258
259 def format_action_for_env(action: Any, vecenv: VecEnv) -> np.ndarray:
260     """
261     Converteix l'acció que retorna el model al format esperat pel VecEnv:
262     - Discrete: array shape (n_envs,), dtype=int64
263     - Box: array shape (n_envs, act_dim), dtype=float32 (clip a [low, high])
264     """
265     action_space = _unwrap_action_space(vecenv)
266     n_envs = getattr(vecenv, "num_envs", 1)
267
268     # SB3 pot retornar escalar, llista o ndarray segons l'algorisme; primer ho normalitzem
269     # per no tenir quatre camins diferents fent el mateix.
270     normalized_action = _as_action_array(action)
271
272     if isinstance(action_space, Discrete):
273         # En Discrete cada entorn paral·lel espera un sol enter; si falta algun valor,
274         # repetim l'últim perquè VecEnv no falli per forma incorrecta.
275         normalized_action = normalized_action.reshape(-1)
276         if normalized_action.size == 1 and n_envs > 1:
277             normalized_action = np.repeat(int(normalized_action.item()), n_envs)
278         if normalized_action.size > n_envs:
279             normalized_action = normalized_action[:n_envs]
280         if normalized_action.size < n_envs:
281             normalized_action = np.pad(normalized_action, (0, n_envs - normalized_action.size), mode="edge")
282         normalized_action = normalized_action.astype(np.int64)
283         normalized_action = np.clip(normalized_action, 0, action_space.n - 1)
284         return normalized_action

```

```

285
286 if isinstance(action_space, MultiDiscrete):
287     # MultiDiscrete és el cas més fàcil de trencar: l'acció pot venir plana o ja
288     # agrupada per entorns. La portem a la forma (n_envs, *shape) abans de retallar.
289     nvec = np.asarray(action_space.nvec, dtype=np.int64)
290     action_shape = tuple(int(value) for value in nvec.shape)
291     normalized_action = _reshape_action_batch(normalized_action, action_shape, n_envs, np.int64)
292     return np.clip(normalized_action, 0, nvec - 1).astype(np.int64)
293
294 if isinstance(action_space, MultiBinary):
295     # MultiBinary comparteix la mateixa idea que MultiDiscrete, però només accepta 0/1.
296     # El clip final evita que un model carregat amb petites diferències retorni valors fora de rang.
297     action_shape = tuple(int(value) for value in action_space.shape)
298     normalized_action = _reshape_action_batch(normalized_action, action_shape, n_envs, np.int64)
299     return np.clip(normalized_action, 0, 1).astype(np.int8)
300
301 if isinstance(action_space, Box):
302     # En espais continus mantenim float32 i fem clip als límits reals de l'entorn.
303     # Això és especialment important quan hi ha wrappers que canvien l'escala de l'acció.
304     action_shape = tuple(int(value) for value in action_space.shape)
305     normalized_action = _reshape_action_batch(normalized_action, action_shape, n_envs, np.float32)
306     low = np.array(action_space.low, dtype=np.float32).reshape(1, *action_shape)
307     high = np.array(action_space.high, dtype=np.float32).reshape(1, *action_shape)
308     normalized_action = np.clip(normalized_action, low, high).astype(np.float32)
309     return normalized_action
310
311 # Fallback conservador per a espais poc habituals: es prioritza que el VecEnv rebi
312 # una mida coherent, encara que després el mateix entorn acabi rebutjant l'acció.
313 normalized_action = normalized_action.reshape(-1)
314 if normalized_action.size == 1 and n_envs > 1:
315     normalized_action = np.repeat(normalized_action.item(), n_envs)
316 elif normalized_action.size > n_envs:
317     normalized_action = normalized_action[:n_envs]
318 elif normalized_action.size < n_envs:
319     normalized_action = np.pad(normalized_action, (0, n_envs - normalized_action.size), mode="edge")
320 return normalized_action.astype(np.int64)

```

E.29 backend/simular_entorn_backend.py

Listing 50: backend/simular_entorn_backend.py

```

1  """Simulacions de referència per a execucions no controlades o basades en regles.
2
3  La pàgina de simulació utilitza aquest backend per inspeccionar entorns registrats, executar
4  episodis de referència, capturen el progrés i persisteixen la mateixa forma d'artefacte esperada
5  pel panell de resultats. Això ofereix als usuaris una prova de referència abans de comparar
6  agents formats.
7  """
8
9  from __future__ import annotations
10
11  import csv
12  import glob
13  import os
14  import time
15  from dataclasses import dataclass
16  from pathlib import Path
17  from typing import Callable
18
19  import gymnasium as gym
20  import numpy as np
21  from stable_baselines3.common.monitor import Monitor
22
23  import sinergym
24  from sinergym.utils.rewards import BaseReward
25  from sinergym.utils.wrappers import CSVLogger, LoggerWrapper
26
27  NO_CONTROL_ACTION = np.empty((0,), dtype=np.float32)
28  BASELINE_TIMESTEPS_PER_HOUR = 4
29  BASELINE_STEP_MINUTES = 60.0 / BASELINE_TIMESTEPS_PER_HOUR
30  DETAILED_HVAC_METERS = {
31      "natural_gas_hvac": "NaturalGas:HVAC",
32      "heating_electricity": "Heating:Electricity",
33      "cooling_electricity": "Cooling:Electricity",
34      "fans_electricity": "Fans:Electricity",
35      "pumps_electricity": "Pumps:Electricity",
36      "heat_rejection_electricity": "HeatRejection:Electricity",
37      "humidifier_electricity": "Humidifier:Electricity",
38      "heat_recovery_electricity": "HeatRecovery:Electricity",
39  }
40  JOULES_PER_KWH = 3_600_000.0
41
42  @dataclass(frozen=True)
43  class EnvironmentSummary:
44      """Descripció compacta d'un entorn Sinergym registrat."""
45      env_id: str
46      env_name: str

```

```

48     building_file: str
49     weather_count: int
50     variables_count: int
51     meters_count: int
52     actuators_count: int
53     reward_name: str
54     step_minutes: float | None
55
56
57 @dataclass(frozen=True)
58 class BaselineSimulationResult:
59     """Sortides persistents i metadades d'estat per a una simulació de referència."""
60     env_id: str
61     run_path: str
62     progress_path: str
63     observations_path: str
64     elapsed_seconds: float
65     completed_steps: int
66     cancelled: bool
67
68
69 class ScheduleBaselineReward(BaseReward):
70     """Reward zero que conserva les mètriques físiques per comparar contra agents entrenats."""
71
72     def __init__(
73         self,
74         temperature_variables: list[str] | None = None,
75         energy_variables: list[str] | None = None,
76         range_comfort_winter: tuple[float, float] = (20.0, 23.5),
77         range_comfort_summer: tuple[float, float] = (23.0, 26.0),
78         summer_start: tuple[int, int] = (6, 1),
79         summer_final: tuple[int, int] = (9, 30),
80     ) -> None:
81         """Inicialitza la instància."""
82         super().__init__()
83         self.temperature_variables = list(temperature_variables or [])
84         self.energy_variables = list(energy_variables or [])
85         self.range_comfort_winter = tuple(range_comfort_winter)
86         self.range_comfort_summer = tuple(range_comfort_summer)
87         self.summer_start = tuple(summer_start)
88         self.summer_final = tuple(summer_final)
89
90     def __call__(self, obs_dict: dict[str, float]) -> tuple[float, dict[str, float]]:
91         """Executa l'objecte cridable."""
92         total_power_demand = self._compute_power_demand(obs_dict)
93         total_temperature_violation = self._compute_temperature_violation(obs_dict)
94         reward_terms = {
95             "energy_term": 0.0,
96             "comfort_term": 0.0,
97             "energy_penalty": -total_power_demand,
98             "comfort_penalty": -total_temperature_violation,
99             "total_power_demand": total_power_demand,
100             "total_temperature_violation": total_temperature_violation,
101             "reward_weight": 0.0,
102         }
103         return 0.0, reward_terms
104
105     def _compute_power_demand(self, obs_dict: dict[str, float]) -> float:
106         """Calcula la demanda de potència."""
107         total = 0.0
108         for variable in self.energy_variables:
109             value = obs_dict.get(variable)
110             if value is None:
111                 continue
112             try:
113                 total += float(value)
114             except (TypeError, ValueError):
115                 continue
116         return total
117
118     def _compute_temperature_violation(self, obs_dict: dict[str, float]) -> float:
119         """Calcula la violació de temperatura."""
120         month = _safe_int(obs_dict.get("month"), default=1)
121         day = _safe_int(obs_dict.get("day_of_month"), default=1)
122         comfort_range = (
123             self.range_comfort_summer
124             if _is_within_summer((month, day), self.summer_start, self.summer_final)
125             else self.range_comfort_winter
126         )
127
128         violation = 0.0
129         for variable in self.temperature_variables:
130             value = obs_dict.get(variable)
131             if value is None:
132                 continue
133             try:
134                 temperature = float(value)
135             except (TypeError, ValueError):
136                 continue
137             lower, upper = comfort_range
138             violation += max(lower - temperature, 0.0) + max(temperature - upper, 0.0)
139         return violation

```

```

140
141
142 def list_environment_ids() -> list[str]:
143     """Retorna els entorns Sinergym que tenen sentit per a una línia de base de planificació."""
144
145     return sorted(
146         env_id
147         for env_id in sinergym.ids()
148         if "discrete" not in env_id.lower()
149     )
150
151
152 def describe_environment(env_id: str) -> EnvironmentSummary:
153     """Crea un resum compacte utilitzat per la interfície."""
154
155     spec = gym.spec(env_id)
156     kwargs = spec.kwargs or {}
157     meters = _with_detailed_hvac_meters(kwargs.get("meters") or {})
158     weather_files = kwargs.get("weather_files") or []
159     if isinstance(weather_files, str):
160         weather_count = 1
161     else:
162         weather_count = len(weather_files)
163
164     reward_cls = kwargs.get("reward")
165     reward_name = getattr(reward_cls, "__name__", str(reward_cls))
166     return EnvironmentSummary(
167         env_id=env_id,
168         env_name=str(kwargs.get("env_name") or env_id),
169         building_file=str(kwargs.get("building_file") or ""),
170         weather_count=weather_count,
171         variables_count=len(kwargs.get("variables") or {}),
172         meters_count=len(meters),
173         actuators_count=len(kwargs.get("actuators") or {}),
174         reward_name=reward_name,
175         step_minutes=BASELINE_STEP_MINUTES,
176     )
177
178
179 def run_baseline_simulation(
180     env_id: str,
181     steps_target: int,
182     seed: int | None,
183     should_stop: Callable[[], bool],
184     on_progress: Callable[[int, int], None],
185 ) -> BaselineSimulationResult:
186     """Executa una línia de base sense control utilitzant els planificacions per defecte d'EnergyPlus."""
187
188     env = _make_baseline_env(env_id)
189     workspace_path = ""
190     completed_steps = 0
191     cancelled = False
192     start_time = time.time()
193
194     try:
195         env.reset(seed=seed)
196         workspace_path = str(env.get_wrapper_attr("workspace_path"))
197
198         for step_number in range(steps_target):
199             if should_stop():
200                 cancelled = True
201                 break
202
203             _, _, terminated, truncated, _ = env.step(NO_CONTROL_ACTION)
204             completed_steps = step_number + 1
205
206             if terminated or truncated:
207                 env.reset()
208
209             if completed_steps % 200 == 0 or completed_steps == steps_target:
210                 on_progress(completed_steps, steps_target)
211         finally:
212             try:
213                 env.close()
214             except Exception:
215                 pass
216
217     progress_path, observations_path = _resolve_run_artifacts(workspace_path)
218     _append_detailed_meter_kwh_columns(observations_path)
219     return BaselineSimulationResult(
220         env_id=env_id,
221         run_path=workspace_path,
222         progress_path=progress_path,
223         observations_path=observations_path,
224         elapsed_seconds=time.time() - start_time,
225         completed_steps=completed_steps,
226         cancelled=canceled,
227     )
228
229
230 def _make_baseline_env(env_id: str):
231     """Crea l'entorn de baseline."""

```

```

232 env_summary = describe_environment(env_id)
233 spec = gym.spec(env_id)
234 spec_kwargs = dict(spec.kwargs or {})
235 baseline_reward_kwargs = _build_baseline_reward_kwargs(spec_kwargs)
236 baseline_name = f"{env_summary.env_name}-baseline"
237 building_config = dict(spec_kwargs.get("building_config") or {})
238 building_config["timesteps_per_hour"] = BASELINE_Timesteps_PER_HOUR
239 meters = _with_detailed_hvac_meters(spec_kwargs.get("meters") or {})
240 env = gym.make(
241     env_id,
242     action_space=gym.spaces.Box(low=0.0, high=0.0, shape=(0,), dtype=np.float32),
243     actuators={},
244     building_config=building_config,
245     env_name=baseline_name,
246     meters=meters,
247     reward=ScheduleBaselineReward,
248     reward_kwargs=baseline_reward_kwargs,
249 )
250 env = LoggerWrapper(env)
251 env = CSVLogger(env)
252 env = Monitor(env)
253 return env
254
255
256 def _resolve_run_artifacts(run_path: str) -> tuple[str, str]:
257     """Retorna les sortides principals CSV de l'execució que acaba d'acabar."""
258
259     progress_path = str(Path(run_path) / "progress.csv")
260     observation_files = sorted(
261         glob.glob(os.path.join(run_path, "**", "observations.csv"), recursive=True),
262         key=os.path.getmtime,
263         reverse=True,
264     )
265     observations_path = observation_files[0] if observation_files else ""
266     return progress_path, observations_path
267
268
269 def _with_detailed_hvac_meters(meters: dict) -> dict[str, str]:
270     """Retorna només els comptadors declarats per la configuració de l'entorn."""
271     return dict(meters or {})
272
273
274 def _append_detailed_meter_kwh_columns(observations_path: str) -> None:
275     """Afegeix columnes detallades del comptador de kWh."""
276     if not observations_path or not os.path.exists(observations_path):
277         return
278
279     with open(observations_path, "r", newline="", encoding="utf-8") as file_handle:
280         reader = csv.DictReader(file_handle)
281         fieldnames = list(reader.fieldnames or [])
282         rows = list(reader)
283
284     if not fieldnames or not rows:
285         return
286
287     # Els meters detallats de Sinergym poden quedar en joules al CSV; afegim *_kwh una
288     # sola vegada perquè el dashboard posterior no faci la conversió repetida.
289     derived_columns = {
290         alias: f"{alias}_kwh"
291         for alias in DETAILED_HVAC_METERS
292         if alias in fieldnames and f"{alias}_kwh" not in fieldnames
293     }
294     if not derived_columns:
295         return
296
297     for row in rows:
298         for source_column, target_column in derived_columns.items():
299             try:
300                 row[target_column] = str(float(row.get(source_column, "")) / JOULES_PER_KWH)
301             except (TypeError, ValueError):
302                 row[target_column] = ""
303
304     output_fields = fieldnames + list(derived_columns.values())
305     with open(observations_path, "w", newline="", encoding="utf-8") as file_handle:
306         writer = csv.DictWriter(file_handle, fieldnames=output_fields)
307         writer.writeheader()
308         writer.writerows(rows)
309
310
311 def _build_baseline_reward_kwargs(env_kwargs: dict) -> dict[str, object]:
312     """Crea kwargs de recompensa de referència."""
313     reward_kwargs = dict(env_kwargs.get("reward_kwargs") or {})
314     variables = env_kwargs.get("variables") or {}
315     meters = env_kwargs.get("meters") or {}
316
317     # La baseline no optimitza res, però igualment necessita variables de confort i energia
318     # perquè els gràfics comparatius surtin amb les mateixes mètriques que un agent.
319     temperature_variables = reward_kwargs.get("temperature_variables")
320     if not temperature_variables:
321         temperature_variables = [
322             name for name in variables.keys()
323             if "air_temperature" in name.lower()

```

```

324     ]
325
326     energy_variables = reward_kwargs.get("energy_variables")
327     if not energy_variables:
328         energy_candidates = list(variables.keys()) + list(meters.keys())
329         energy_variables = [
330             name for name in energy_candidates
331             if any(token in name.lower() for token in ("hvac", "electric", "power", "demand", "energy"))
332         ]
333
334     return {
335         "temperature_variables": list(temperature_variables or []),
336         "energy_variables": list(energy_variables or []),
337         "range_comfort_winter": tuple(reward_kwargs.get("range_comfort_winter") or (20.0, 23.5)),
338         "range_comfort_summer": tuple(reward_kwargs.get("range_comfort_summer") or (23.0, 26.0)),
339         "summer_start": tuple(reward_kwargs.get("summer_start") or (6, 1)),
340         "summer_final": tuple(reward_kwargs.get("summer_final") or (9, 30)),
341     }
342
343
344 def _safe_int(value: object, *, default: int) -> int:
345     """Safe int."""
346     try:
347         return int(float(value))
348     except (TypeError, ValueError):
349         return default
350
351
352 def _is_within_summer(current: tuple[int, int], start: tuple[int, int], end: tuple[int, int]) -> bool:
353     """Is within summer."""
354     return start <= current <= end

```

E.30 backend/training_scripts.py

Listing 51: backend/training_scripts.py

```

1  """Genereu scripts d'ajuda d'entrenament reproduïbles per a les execucions BEMS-RL desades."""
2
3  from __future__ import annotations
4
5  from pprint import pformat
6  from typing import Any, Dict
7
8
9  def build_training_eval_script(training_config: Dict[str, Any]) -> str:
10     """Crea un petit script d'ajuda d'avaluació desat al costat d'una sessió d'entrenament per al flux Studio."""
11     lines = [
12         "# Script d'avaluació autogenerat (no s'executa automàticament)",
13         "# Entorn i recompensa amb la mateixa configuració que l'entrenament.",
14         "",
15         f"env_id = {training_config['env_id']!r}",
16         f"meters = {pformat(training_config.get('meters', {}), sort_dicts=False)}",
17         f"reward_name = {training_config['reward_name']!r}",
18         f"reward_kwargs = {pformat(training_config['reward_kwargs'], sort_dicts=False)}",
19         f"wrapper_configs = {pformat(training_config.get('wrapper_configs', []), sort_dicts=False)}",
20         "",
21     ]
22     return "\n".join(lines)
23
24
25 def build_training_repro_script(
26     artifact_name: str,
27     training_config: Dict[str, Any],
28 ) -> str:
29     """Crea un script autònom que pugui reproduir una configuració d'entrenament desada."""
30     config_literal = pformat(training_config, sort_dicts=False, width=100)
31     # Retornem el fitxer complet com a string perquè l'artefacte guardat sigui executable
32     # fora de Streamlit, amb la mateixa configuració que s'ha fet servir a la UI.
33     return f'"""Script d'entrenament reproducible autogenerat per BEMS-RL-STUDIO."""'
34
35 import csv
36 from pathlib import Path
37
38 import gymnasium as gym
39 import numpy as np
40 from gymnasium import Wrapper
41 from gymnasium.spaces import Box
42 from stable_baselines3 import A2C, DDPG, DQN, PPO, SAC, TD3
43 from stable_baselines3.common.monitor import Monitor
44 from stable_baselines3.common.vec_env import DummyVecEnv, VecNormalize
45 import sinergym.utils.rewards as reward_module
46 import sinergym.utils.wrappers as wrapper_module
47
48
49 TRAINING_NAME = {artifact_name!r}
50 TRAINING_CONFIG = {config_literal}
51 ALGOS = {"PPO": PPO, "A2C": A2C, "DQN": DQN, "SAC": SAC, "TD3": TD3, "DDPG": DDPG}
52 DETAILED_HVAC_METERS = {{

```



```

53     "natural_gas_hvac": "NaturalGas:HVAC",
54     "heating_electricity": "Heating:Electricity",
55     "cooling_electricity": "Cooling:Electricity",
56     "fans_electricity": "Fans:Electricity",
57     "pumps_electricity": "Pumps:Electricity",
58     "heat_rejection_electricity": "HeatRejection:Electricity",
59     "humidifier_electricity": "Humidifier:Electricity",
60     "heat_recovery_electricity": "HeatRecovery:Electricity",
61 }}
62 JOULES_PER_KWH = 3_600_000.0
63
64
65 class EnergyCostFileLogger(Wrapper):
66     def __init__(self, env, filename="energy_cost_log.csv"):
67         super().__init__(env)
68         self.csvfile = open(filename, mode="w", newline="")
69         self.writer = csv.writer(self.csvfile)
70         self.writer.writerow(["step", "electricity_cost_eur_per_kwh", "energy_cost_eur"])
71         self.step_count = 0
72
73     def step(self, action):
74         obs, reward, terminated, truncated, info = self.env.step(action)
75         self.step_count += 1
76         cost_per_kwh = info.get("electricity_cost", 0.0)
77         energy_rate = info.get("HVAC_electricity_demand_rate", 0.0)
78         self.writer.writerow([self.step_count, cost_per_kwh, cost_per_kwh * energy_rate])
79         return obs, reward, terminated, truncated, info
80
81     def reset(self, **kwargs):
82         self.step_count = 0
83         return self.env.reset(**kwargs)
84
85     def close(self):
86         self.csvfile.close()
87         if hasattr(self.env, "close"):
88             self.env.close()
89
90
91 def get_reward_class(name):
92     reward_cls = getattr(reward_module, name, None)
93     if reward_cls is None:
94         raise ImportError(
95             f"La reward '{{name}}' no existeix a sinergym.utils.rewards en aquesta instal·lacio."
96         )
97     return reward_cls
98
99
100 def get_wrapper_class(name):
101     if name == "EnergyCostFileLogger":
102         return EnergyCostFileLogger
103     wrapper_cls = getattr(wrapper_module, name, None)
104     if wrapper_cls is None:
105         raise ImportError(
106             f"El wrapper '{{name}}' no existeix a sinergym.utils.wrappers en aquesta instal·lacio."
107         )
108     return wrapper_cls
109
110
111 def apply_training_wrappers(env, wrapper_configs):
112     for wrapper_config in wrapper_configs:
113         name = wrapper_config["name"]
114         params = wrapper_config.get("params", {})
115         wrapper_cls = get_wrapper_class(name)
116         if name == "PreviousObservationWrapper":
117             env = wrapper_cls(env, **params)
118         elif name == "DatetimeWrapper":
119             env = wrapper_cls(env)
120         elif name == "WeatherForecastingWrapper":
121             env = wrapper_cls(env, **params)
122         elif name == "EnergyCostWrapper":
123             env = wrapper_cls(env, **params)
124         elif name == "DeltaTempWrapper":
125             env = wrapper_cls(env, **params)
126         elif name == "ReduceObservationWrapper":
127             env = wrapper_cls(env, **params)
128         elif name == "MultiObsWrapper":
129             env = wrapper_cls(env, **params)
130         elif name == "NormalizeObservation":
131             env = wrapper_cls(env, **params)
132         elif name == "VariabilityContextWrapper":
133             context_space = Box(
134                 low=np.array(params["context_low"], dtype=np.float32),
135                 high=np.array(params["context_high"], dtype=np.float32),
136                 dtype=np.float32,
137             )
138             env = wrapper_cls(
139                 env,
140                 context_space=context_space,
141                 delta_value=params["delta_value"],
142                 step_frequency_range=tuple(params["step_frequency_range"]),
143             )
144         elif name == "OfficeGridStorageSmoothingActionConstraintsWrapper":

```



```

145         env = wrapper_cls(env)
146     elif name == "IncrementalWrapper":
147         env = wrapper_cls(env, **params)
148     elif name == "DiscreteIncrementalWrapper":
149         env = wrapper_cls(env, **params)
150     elif name == "NormalizeAction" and isinstance(env.action_space, Box):
151         env = wrapper_cls(env, **params)
152     elif name == "EnergyCostFileLogger":
153         env = wrapper_cls(env, **params)
154     return env
155
156
157 def with_detailed_hvac_meters(meters):
158     return dict(meters or {})
159
160
161 def get_training_meters():
162     configured_meters = TRAINING_CONFIG.get("meters")
163     if configured_meters is not None:
164         return with_detailed_hvac_meters(configured_meters)
165
166     try:
167         spec = gym.spec(TRAINING_CONFIG["env_id"])
168         meters = (spec.kwarg or {}).get("meters", {}) or {}
169     except Exception:
170         meters = {}
171     return with_detailed_hvac_meters(meters)
172
173
174 def make_env():
175     reward_cls = get_reward_class(TRAINING_CONFIG["reward_name"])
176     meters = get_training_meters()
177     env = gym.make(
178         TRAINING_CONFIG["env_id"],
179         meters=meters,
180         reward=reward_cls,
181         reward_kwarg=TRAINING_CONFIG["reward_kwarg"],
182     )
183     env = apply_training_wrappers(env, TRAINING_CONFIG.get("wrapper_configs", []))
184     env = get_wrapper_class("LoggerWrapper")(env)
185     env = get_wrapper_class("CSVLogger")(env)
186     env = Monitor(env)
187     return env
188
189
190 def get_workspace_path(vec):
191     try:
192         paths = vec.get_attr("workspace_path")
193         if paths and paths[0]:
194             return str(paths[0])
195     except Exception:
196         pass
197     return None
198
199
200 def append_detailed_meter_kwh_columns(observations_path):
201     observations_file = Path(observations_path) if observations_path else None
202     if observations_file is None or not observations_file.is_file():
203         return
204
205     with open(observations_file, "r", newline="", encoding="utf-8") as file_handle:
206         reader = csv.DictReader(file_handle)
207         fieldnames = list(reader.fieldnames or [])
208         rows = list(reader)
209
210         if not fieldnames or not rows:
211             return
212
213         source_columns = {}
214         for alias, meter_name in DETAILED_HVAC_METERS.items():
215             target_column = alias + "_kwh"
216             if target_column in fieldnames:
217                 continue
218             if alias in fieldnames:
219                 source_columns[alias] = alias
220             elif meter_name in fieldnames:
221                 source_columns[alias] = meter_name
222
223         if not source_columns:
224             return
225
226         derived_columns = {source_alias: source_alias + "_kwh" for source_alias in source_columns}
227         for row in rows:
228             for source_alias, source_column in source_columns.items():
229                 target_column = derived_columns[source_alias]
230                 try:
231                     row[target_column] = str(float(row.get(source_column, "")) / JOULES_PER_KWH)
232                 except (TypeError, ValueError):
233                     row[target_column] = ""
234
235         output_fields = fieldnames + list(derived_columns.values())
236         with open(observations_file, "w", newline="", encoding="utf-8") as file_handle:

```

```

237     writer = csv.DictWriter(file_handle, fieldnames=output_fields)
238     writer.writeheader()
239     writer.writerows(rows)
240
241
242 def append_detailed_meter_kwh_columns_in_workspace(workspace_path):
243     if not workspace_path:
244         return
245
246     workspace = Path(workspace_path)
247     if not workspace.is_dir():
248         return
249
250     for observations_file in workspace.rglob("observations.csv"):
251         try:
252             append_detailed_meter_kwh_columns(observations_file)
253         except Exception:
254             continue
255
256
257 def main() -> None:
258     artifact_dir = Path(__file__).resolve().parent
259     vec = DummyVecEnv([make_env])
260     vec = VecNormalize(vec, norm_obs=True, norm_reward=True)
261     workspace_path = None
262
263     try:
264         algo_cls = ALGOS[TRAINING_CONFIG["algo_name"]]
265         algo_kwargs = dict(TRAINING_CONFIG.get("algo_kwargs") or {})
266         algo_kwargs.setdefault("learning_rate", TRAINING_CONFIG["learning_rate"])
267         if TRAINING_CONFIG["algo_name"] in {"PPO", "A2C"} and TRAINING_CONFIG.get("n_steps") is not None:
268             algo_kwargs.setdefault("n_steps", TRAINING_CONFIG["n_steps"])
269         algo_kwargs["verbose"] = 1
270
271         model = algo_cls(TRAINING_CONFIG["policy_name"], vec, **algo_kwargs)
272         model.learn(total_timesteps=TRAINING_CONFIG["timesteps_per_year"])
273         model.save(str(artifact_dir / f"{{{TRAINING_NAME}}}.zip"))
274         vec.save(str(artifact_dir / f"{{{TRAINING_NAME}}}_vecnorm.pkl"))
275         workspace_path = get_workspace_path(vec)
276     finally:
277         if workspace_path is None:
278             workspace_path = get_workspace_path(vec)
279         vec.close()
280         append_detailed_meter_kwh_columns_in_workspace(workspace_path)
281
282
283 if __name__ == "__main__":
284     main()
285

```

E.31 backend/viewer_3d_backend.py

Listing 52: backend/viewer_3d_backend.py

```

1  """Backend del visor d'edificis 3D: processament de geometria i generació de figures Plotly."""
2
3  from __future__ import annotations
4
5  import math
6  from typing import Any, Dict, List, Tuple
7
8  import plotly.graph_objects as go
9  import streamlit as st
10
11  SURFACE_STYLES: Dict[str, Dict[str, Any]] = {
12      'Wall': {'color': 'orange', 'opacity': 0.45},
13      'Roof': {'color': 'green', 'opacity': 0.45},
14      'Floor': {'color': 'blue', 'opacity': 0.35},
15      'Ceiling': {'color': 'purple', 'opacity': 0.35},
16      'Window': {'color': 'skyblue', 'opacity': 0.60},
17      'Door': {'color': '#8B4513', 'opacity': 0.90},
18      'Glassdoor': {'color': 'cyan', 'opacity': 0.60},
19      'Unknown': {'color': 'darkgray', 'opacity': 0.50},
20  }
21  PV_STYLE_COLOR = "#222222"
22  PV_STYLE_OPACITY = 0.90
23
24
25  def extract_vertices_general(surface: dict) -> List[Tuple[float, float, float]]:
26      """Extreu vèrtexs d'un dictat de superfície.
27
28      Admet tant una llista de "vèrtexs" com camps de coordenades indexats separats
29      (vertex_N_x_coordinate / vertex_N_y_coordinate / vertex_N_z_coordinate).
30      """
31      vertices = []
32      if "vertices" in surface:
33          for vertex in surface["vertices"]:
34              x = vertex.get("vertex_x_coordinate")

```

```

35     y = vertex.get("vertex_y_coordinate")
36     z = vertex.get("vertex_z_coordinate")
37     if None not in (x, y, z):
38         vertices.append((x, y, z))
39
40     else:
41         index = 1
42         while f"vertex_{index}_x_coordinate" in surface:
43             x = surface.get(f"vertex_{index}_x_coordinate")
44             y = surface.get(f"vertex_{index}_y_coordinate")
45             z = surface.get(f"vertex_{index}_z_coordinate")
46             if None not in (x, y, z):
47                 vertices.append((x, y, z))
48             index += 1
49
50     return vertices
51
52 def _classify_type(surf_type_raw: str, kind: str) -> str:
53     """Normalise a raw surface type string to a canonical type label."""
54     surface_type = (surf_type_raw or "Unknown").strip().lower()
55     if kind == "sub":
56         if "door" in surface_type and "glass" in surface_type:
57             return "Glassdoor"
58         if "door" in surface_type:
59             return "Door"
60         if "window" in surface_type:
61             return "Window"
62         if "wall" in surface_type:
63             return "Wall"
64         if "roof" in surface_type:
65             return "Roof"
66         if "ceiling" in surface_type:
67             return "Ceiling"
68         if "floor" in surface_type:
69             return "Floor"
70         return "Unknown"
71
72 def _triangulate_fan(n: int) -> List[Tuple[int, int, int]]:
73     """Retorna l'índex de triangulació en ventall triplicat per a un polígon convex amb n vèrtexs."""
74     return [(0, i, i + 1) for i in range(1, n - 1)] if n >= 3 else []
75
76 def _polygon_normal_newell(vertices: List[Tuple[float, float, float]]) -> Tuple[float, float, float]:
77     """Calcula la normal del polígon amb el mètode de Newell."""
78     nx = ny = nz = 0.0
79     n = len(vertices)
80     # Newell acumula el gir de totes les arestes i és més estable que triar només tres punts
81     # quan les superfícies EnergyPlus arriben lleugerament tortes.
82     for index in range(n):
83         x1, y1, z1 = vertices[index]
84         x2, y2, z2 = vertices[(index + 1) % n]
85         nx += (y1 - y2) * (z1 + z2)
86         ny += (z1 - z2) * (x1 + x2)
87         nz += (x1 - x2) * (y1 + y2)
88     return nx, ny, nz
89
90
91 def _vector_norm(vector: Tuple[float, float, float]) -> float:
92     """Retorna la norma euclidiana d'un vector de 3 components."""
93     return math.sqrt(vector[0] * vector[0] + vector[1] * vector[1] + vector[2] * vector[2])
94
95
96 def _tilt_azimuth_from_normal(
97     nx: float, ny: float, nz: float
98 ) -> Tuple[float, float, str]:
99     """Calculeu l'etiqueta d'inclinació, azimuth i orientació cardinal a partir d'una normal de superfície.
100
101     Retorna:
102         inclinació: Angle des de l'horitzontal en graus (0 = pla).
103         azimuth: en sentit horari des del nord en graus (0=N, 90=E, 180=S, 270=W).
104         orientació: etiqueta cardinal ('N', 'NE', ..., 'H' per a gairebé horitzontal).
105     """
106     norm = _vector_norm((nx, ny, nz)) or 1e-9
107     tilt = math.degrees(math.acos(abs(nz) / norm))
108     azimuth = (math.degrees(math.atan2(nx, ny)) + 360.0) % 360.0
109     directions = ['N', 'NE', 'E', 'SE', 'S', 'SO', 'O', 'NO']
110     index = int((azimuth + 22.5) % 360) // 45
111     orientation = directions[index]
112     if tilt < 15:
113         orientation = 'H'
114     return tilt, azimuth, orientation
115
116
117 def _triangle_area(
118     a: Tuple[float, float, float],
119     b: Tuple[float, float, float],
120     c: Tuple[float, float, float],
121 ) -> float:
122     """Retorna l'àrea d'un triangle definit per tres punts 3D."""
123     ax, ay, az = a
124     bx, by, bz = b
125     cx, cy, cz = c
126     ux, uy, uz = (bx - ax, by - ay, bz - az)

```

```

127 vx, vy, vz = (cx - ax, cy - ay, cz - az)
128 cross = (uy * vz - uz * vy, uz * vx - ux * vz, ux * vy - uy * vx)
129 return 0.5 * _vector_norm(cross)
130
131
132 def _polygon_area(vertices: List[Tuple[float, float, float]]) -> float:
133     """Retorna l'àrea total d'un polígon mitjançant la triangulació en ventall."""
134     triangles = _triangulate_fan(len(vertices))
135     area = 0.0
136     for i, j, k in triangles:
137         area += _triangle_area(vertices[i], vertices[j], vertices[k])
138     return area
139
140
141 def _surface_hover_text(record: Dict[str, Any]) -> str:
142     """Crea una etiqueta de flotació llegible pels humans per a un registre de superfície."""
143     geometry_line = f"Area: {record['area']:.2f} m2"
144     if record['orientation'] == 'H':
145         orientation_line = f"Tilt: {record['tilt']:.1f} deg | Orientation: horizontal"
146     else:
147         orientation_line = (
148             f"Tilt: {record['tilt']:.1f} deg | "
149             f"Azimuth: {record['azimuth']:.1f} deg ({record['orientation']})"
150         )
151     return (
152         f"Surface: {record['name']}<br>"
153         f"Zone: {record['zone']}<br>"
154         f"Type: {record['type']} ({record['kind']})<br>"
155         f"{geometry_line}<br>"
156         f"{orientation_line}"
157     )
158
159
160 def _scaled_offset_polygon(
161     vertices: List[Tuple[float, float, float]],
162     offset: float = 0.05,
163     shrink: float = 0.97,
164 ) -> List[Tuple[float, float, float]]:
165     """Retorna una còpia reduïda i amb desplaçament normal d'un polígon.
166
167     Paràmetres:
168         vèrtexs: vèrtexs de polígons originals.
169         desplaçament: desplaçament al llarg de la unitat normal per evitar la lluita en z.
170         contracció: factor d'escala [0..1] aplicat en relació al baricentre.
171     """
172     if not vertices:
173         return vertices
174     cx = sum(v[0] for v in vertices) / len(vertices)
175     cy = sum(v[1] for v in vertices) / len(vertices)
176     cz = sum(v[2] for v in vertices) / len(vertices)
177     nx, ny, nz = _polygon_normal_newell(vertices)
178     normal_norm = _vector_norm((nx, ny, nz)) or 1e-9
179     nx, ny, nz = nx / normal_norm, ny / normal_norm, nz / normal_norm
180     output = []
181     for x, y, z in vertices:
182         output.append((
183             cx + shrink * (x - cx) + nx * offset,
184             cy + shrink * (y - cy) + ny * offset,
185             cz + shrink * (z - cz) + nz * offset,
186         ))
187     return output
188
189
190 def _collect_surfaces_with_names(epjson: dict) -> List[Dict[str, Any]]:
191     """Recull totes les superfícies d'un epJSON dict.
192
193     Retorna una llista de dictats amb claus: nom, zona, type_raw, tipus ('principal'/'sub'), vèrtexs.
194     """
195     output = []
196     for name, surface in epjson.get("BuildingSurface:Detailed", {}).items():
197         vertices = extract_vertices_general(surface)
198         if len(vertices) >= 3:
199             output.append({
200                 'name': name,
201                 'zone': surface.get("zone_name", "Unknown"),
202                 'type_raw': surface.get("surface_type", "Unknown"),
203                 'kind': 'main',
204                 'vertices': vertices,
205             })
206     for name, surface in epjson.get("FenestrationSurface:Detailed", {}).items():
207         vertices = extract_vertices_general(surface)
208         if len(vertices) >= 3:
209             output.append({
210                 'name': name,
211                 'zone': surface.get("building_surface_name", "Unknown"),
212                 'type_raw': surface.get("surface_type", "Unknown"),
213                 'kind': 'sub',
214                 'vertices': vertices,
215             })
216     return output
217
218

```

```

219 def _bounds(
220     vertices_list: List[List[Tuple[float, float, float]]],
221 ) -> Tuple[float, float, float, float, float, float]:
222     """Retorna (xmin, xmax, ymin, ymax, zmin, zmax) el quadre delimitador de tots els vèrtexs proporcionats."""
223     xs, ys, zs = [], [], []
224     for vertices in vertices_list:
225         for x, y, z in vertices:
226             xs.append(x)
227             ys.append(y)
228             zs.append(z)
229     if not xs:
230         return (0.0, 0.0, 0.0, 0.0, 0.0, 0.0)
231     return min(xs), max(xs), min(ys), max(ys), min(zs), max(zs)
232
233
234 def _color_for(record: Dict[str, Any]) -> str:
235     """Retorna el color de visualització d'un registre de superfície en funció del seu tipus canònic."""
236     return SURFACE_STYLES.get(record['type'], SURFACE_STYLES['Unknown'])['color']
237
238
239 @st.cache_data(show_spinner=False)
240 def build_surface_records(epjson: dict) -> Dict[str, Any]:
241     """Crea registres de superfície enriquets a partir d'un dic epJSON (emmagatzemat en memòria cau).
242
243     Cada registre conté: id, nom, zona, tipus (normalitzat), tipus (principal/sub),
244     vèrtexs originals, vèrtexs centrats, àrea, inclinació, azimuth, orientació,
245     i la mitjana z en coordenades centrades.
246
247     Retorna un dictat amb claus: registres, zones, tipus, centre, z_clip_range, name_index.
248     El name_index mapes el nom de la superfície per registrar l'index només per a les superfícies principals.
249     """
250     items = _collect_surfaces_with_names(epjson)
251     vertices_list = [item['vertices'] for item in items]
252     xmn, xmx, ymn, ymx, zmn, zmx = _bounds(vertices_list)
253     cx = (xmn + xmx) / 2.0
254     cy = (ymn + ymx) / 2.0
255     cz = (zmn + zmx) / 2.0
256
257     records: List[Dict[str, Any]] = []
258     zones: set = set()
259     types: set = set()
260     name_index: Dict[str, int] = {}
261
262     for index, item in enumerate(items):
263         normalized_type = _classify_type(item['type_raw'], item['kind'])
264         zones.add(item['zone'])
265         types.add(normalized_type)
266
267         nx, ny, nz = _polygon_normal_newell(item['vertices'])
268         tilt, azimuth, orientation = _tilt_azimuth_from_normal(nx, ny, nz)
269         area = _polygon_area(item['vertices'])
270         vertices_c = [(x - cx, y - cy, z - cz) for (x, y, z) in item['vertices']]
271
272         record: Dict[str, Any] = {
273             'id': index,
274             'name': item['name'],
275             'zone': item['zone'],
276             'type': normalized_type,
277             'kind': item['kind'],
278             'vertices': item['vertices'],
279             'vertices_c': vertices_c,
280             'area': area,
281             'tilt': tilt,
282             'azimuth': azimuth,
283             'orientation': orientation,
284             'z_mean_c': sum(v[2] for v in vertices_c) / len(vertices_c),
285         }
286         records.append(record)
287         if item['kind'] == 'main':
288             name_index[item['name']] = index
289
290     z_values = [r['z_mean_c'] for r in records] or [0.0]
291     return {
292         'records': records,
293         'zones': sorted(zones),
294         'types': sorted(types),
295         'center': (cx, cy, cz),
296         'z_clip_range': (float(min(z_values)), float(max(z_values))),
297         'name_index': name_index,
298     }
299
300
301 def plot_3d_model(
302     records: List[Dict[str, Any]],
303     visible_types: Dict[str, bool],
304     visible_zones: Dict[str, bool],
305     z_clip: Tuple[float, float],
306     pv_list: List[Dict[str, Any]],
307     name_index: Dict[str, int],
308 ) -> go.Figure:
309     """Construeix una figura 3D Plotly de la geometria de l'edifici.
310

```

```

311 Les superfícies es filtren per tipus, zona i rang de clip d'alçada.
312 Els panells PV es representen com una capa addicional superposada a les seves superfícies host.
313 """
314 fig = go.Figure()
315 zmin, zmax = z_clip
316
317 for record in records:
318     # El filtre es fa abans de crear traces perquè Plotly es torna lent si li passem
319     # superfícies ocultes i després només les amaguem a la llegenda.
320     if not visible_types.get(record['type'], False):
321         continue
322     if not visible_zones.get(record['zone'], False):
323         continue
324     if not (zmin <= record['z_mean_c'] <= zmax):
325         continue
326
327     vertices = record['vertices_c']
328     x, y, z = zip(*vertices)
329     # EnergyPlus dona polígons; Mesh3d necessita triangles. El fan és suficient per a
330     # superfícies habituals d'edifici, que normalment són convexes o gairebé planes.
331     triangles = _triangulate_fan(len(vertices))
332     if not triangles:
333         continue
334     i_idx, j_idx, k_idx = zip(*triangles)
335     color = _color_for(record)
336     opacity = SURFACE_STYLES.get(record['type'], SURFACE_STYLES['Unknown'])['opacity']
337
338     fig.add_trace(go.Mesh3d(
339         x=x, y=y, z=z,
340         i=i_idx, j=j_idx, k=k_idx,
341         color=color, opacity=opacity,
342         name=f"{record['zone']} - {record['type']}",
343         hovertext=_surface_hover_text(record),
344         hoverinfo='text',
345         showscale=False,
346     ))
347
348     if record['kind'] == 'sub':
349         fig.add_trace(go.Scatter3d(
350             x=list(x) + [x[0]],
351             y=list(y) + [y[0]],
352             z=list(z) + [z[0]],
353             mode='lines',
354             line=dict(color='black', width=2),
355             name="Subsurface edge",
356             showlegend=False,
357             hoverinfo='skip',
358         ))
359
360 if pv_list and visible_types.get('PV', False):
361     for pv in pv_list:
362         surface_name = pv.get('surface_name')
363         if surface_name not in name_index:
364             continue
365         record = records[name_index[surface_name]]
366         if not (zmin <= record['z_mean_c'] <= zmax):
367             continue
368         # El panell PV es dibuixa una mica separat i encongint perquè no faci z-fighting
369         # amb la superfície de la coberta o façana que el suporta.
370         pv_polygon = _scaled_offset_polygon(record['vertices_c'], offset=0.03, shrink=0.96)
371         px, py, pz = zip(*pv_polygon)
372         triangles = _triangulate_fan(len(pv_polygon))
373         if not triangles:
374             continue
375         i_idx, j_idx, k_idx = zip(*triangles)
376         fig.add_trace(go.Mesh3d(
377             x=px, y=py, z=pz,
378             i=i_idx, j=j_idx, k=k_idx,
379             color=PV_STYLE_COLOR, opacity=PV_STYLE_OPACITY,
380             name=f"PV - {pv.get('name')}",
381             hovertext=(
382                 f"Panel: {pv.get('name')}<br>"
383                 f"Surface: {surface_name}<br>"
384                 f"DC capacity: {pv.get('capacity_dc', '-')}")
385             ),
386             hoverinfo='text',
387             showscale=False,
388         ))
389         fig.add_trace(go.Scatter3d(
390             x=list(px) + [px[0]],
391             y=list(py) + [py[0]],
392             z=list(pz) + [pz[0]],
393             mode='lines',
394             line=dict(color='black', width=2),
395             name="PV edge",
396             showlegend=False,
397             hoverinfo='skip',
398         ))
399
400 fig.update_layout(
401     scene=dict(xaxis_title="X", yaxis_title="Y", zaxis_title="Z", aspectmode='data'),
402     margin=dict(l=0, r=0, b=0, t=0),

```

```

403     height=550,
404     showlegend=True,
405 )
406 return fig

```

E.32 backend/visor_epw_backend.py

Listing 53: backend/visor_epw_backend.py

```

1  """Utilitats de preparació de dades per al visor climàtic EPW.
2
3  El backend del visor EPW localitza fitxers meteorològics d'EnergyPlus,
4  normalitza els camps EPW en brut, afegeix atributs de calendari i produeix
5  taules de resum per a la UI de Streamlit. Les sortides es mantenen com a
6  DataFrames perquè els gràfics i les descàrregues es puguin construir sense
7  tornar a llegir el fitxer meteorològic.
8  """
9
10 from __future__ import annotations
11
12 from pathlib import Path
13
14 import numpy as np
15 import pandas as pd
16 import streamlit as st
17
18
19 ROOT_PATH = Path.cwd().resolve()
20 WEATHERS_DIR = ROOT_PATH / "sinergym" / "data" / "weather"
21 WEATHER_ROOT = WEATHERS_DIR if WEATHERS_DIR.exists() else ROOT_PATH
22 DEG = "\N{DEGREE SIGN}"
23 DEG_C = f"{DEG}C"
24 SQUARE_2 = "\N{SUPERSCRITPT TWO}"
25 WH_PER_M2 = f"Wh/m{SQUARE_2}"
26 KWH_PER_M2 = f"kWh/m{SQUARE_2}"
27 MID_DOT = "\N{MIDDLE DOT}"
28
29 MONTH_LABELS = {
30     1: "Gen",
31     2: "Feb",
32     3: "Mar",
33     4: "Abr",
34     5: "Mai",
35     6: "Jun",
36     7: "Jul",
37     8: "Ago",
38     9: "Set",
39     10: "Oct",
40     11: "Nov",
41     12: "Des",
42 }
43 MONTH_ORDER = list(MONTH_LABELS.values())
44 WIND_DIRECTION_LABELS = (
45     "N",
46     "NNE",
47     "NE",
48     "ENE",
49     "E",
50     "ESE",
51     "SE",
52     "SSE",
53     "S",
54     "SSW",
55     "SW",
56     "WSW",
57     "W",
58     "WNW",
59     "NW",
60     "NNW",
61 )
62 TIME_AGGREGATION_RULES = {
63     "Hora": None,
64     "Dia": "D",
65     "Setmana": "W-MON",
66     "Mes": "M",
67 }
68 CATALOG_SEPARATOR = f" {MID_DOT} "
69
70 EPW_COLUMNS = [
71     "year",
72     "month",
73     "day",
74     "hour",
75     "minute",
76     "data_source",
77     "dry_bulb_c",
78     "dew_point_c",
79     "relative_humidity_pct",

```

```

80     "atmospheric_pressure_pa",
81     "extraterrestrial_horizontal_radiation_wh_m2",
82     "extraterrestrial_direct_normal_radiation_wh_m2",
83     "horizontal_infrared_radiation_wh_m2",
84     "global_horizontal_radiation_wh_m2",
85     "direct_normal_radiation_wh_m2",
86     "diffuse_horizontal_radiation_wh_m2",
87     "global_horizontal_illuminance_lux",
88     "direct_normal_illuminance_lux",
89     "diffuse_horizontal_illuminance_lux",
90     "zenith_luminance_cd_m2",
91     "wind_direction_deg",
92     "wind_speed_m_s",
93     "total_sky_cover_tenths",
94     "opaque_sky_cover_tenths",
95     "visibility_km",
96     "ceiling_height_m",
97     "present_weather_observation",
98     "present_weather_codes",
99     "precipitable_water_mm",
100    "aerosol_optical_depth_thousandths",
101    "snow_depth_cm",
102    "days_since_last_snowfall",
103    "albedo",
104    "liquid_precipitation_depth_mm",
105    "liquid_precipitation_quantity_hr",
106 ]
107 NUMERIC_EPW_COLUMNS = tuple(column for column in EPW_COLUMNS if column != "data_source")
108
109 EPW_VARIABLES = {
110     "dry_bulb_c": {
111         "label": "Temperatura seca",
112         "unit": "DEG_C",
113         "aggregate": "mean",
114         "color": "#f26b5b",
115     },
116     "relative_humidity_pct": {
117         "label": "Humitat relativa",
118         "unit": "%",
119         "aggregate": "mean",
120         "color": "#4f8ef7",
121     },
122     "wind_speed_m_s": {
123         "label": "Velocitat del vent",
124         "unit": "m/s",
125         "aggregate": "mean",
126         "color": "#2ca58d",
127     },
128     "direct_normal_radiation_wh_m2": {
129         "label": "Radiació directa normal",
130         "unit": "WH_PER_M2",
131         "aggregate": "sum",
132         "color": "#ef7d57",
133         "scale_to_kilo": True,
134         "display_unit_aggregated": "KWH_PER_M2",
135     },
136 }
137
138 MISSING_SENTINELS = {
139     "dry_bulb_c": {99.9},
140     "dew_point_c": {99.9},
141     "relative_humidity_pct": {999.0},
142     "atmospheric_pressure_pa": {999999.0},
143     "extraterrestrial_horizontal_radiation_wh_m2": {9999.0},
144     "extraterrestrial_direct_normal_radiation_wh_m2": {9999.0},
145     "horizontal_infrared_radiation_wh_m2": {9999.0},
146     "global_horizontal_radiation_wh_m2": {9999.0},
147     "direct_normal_radiation_wh_m2": {9999.0},
148     "diffuse_horizontal_radiation_wh_m2": {9999.0},
149     "global_horizontal_illuminance_lux": {999999.0},
150     "direct_normal_illuminance_lux": {999999.0},
151     "diffuse_horizontal_illuminance_lux": {999999.0},
152     "zenith_luminance_cd_m2": {999999.0, 9999.0},
153     "wind_direction_deg": {999.0},
154     "wind_speed_m_s": {999.0},
155     "total_sky_cover_tenths": {99.0},
156     "opaque_sky_cover_tenths": {99.0},
157     "visibility_km": {9999.0},
158     "ceiling_height_m": {99999.0},
159     "precipitable_water_mm": {999.0},
160     "snow_depth_cm": {999.0},
161     "days_since_last_snowfall": {99.0},
162     "albedo": {999.0},
163     "liquid_precipitation_depth_mm": {999.0},
164     "liquid_precipitation_quantity_hr": {99.0},
165 }
166
167 COMFORT_RADIATION_SOURCE_COLUMNS = (
168     "dry_bulb_c",
169     "direct_normal_radiation_wh_m2",
170     "diffuse_horizontal_radiation_wh_m2",
171     "day_of_year",

```



```

172     "hour_start",
173 )
174 COMFORT_RADIATION_COLUMNS = (
175     "direction",
176     "theta_deg",
177     "altitude_band",
178     "radius_start",
179     "radius_size",
180     "comfort_radiation_kwh_m2",
181     "incident_radiation_kwh_m2",
182 )
183
184
185 def _parse_optional_float(value: str | None) -> float:
186     """Retorna un camp numèric EPW com a flotant, o NaN quan el valor està en blanc o no és vàlid."""
187
188     if value is None or value == "":
189         return np.nan
190     try:
191         return float(value)
192     except ValueError:
193         return np.nan
194
195
196 def _header_payload(header_lines: list[str], line_index: int) -> str:
197     """Retorna la càrrega útil després de la primera coma en una línia de capçalera EPW."""
198
199     if len(header_lines) <= line_index or "," not in header_lines[line_index]:
200         return "-"
201     return header_lines[line_index].split(",", 1)[1].strip() or "-"
202
203
204 def _classify_climate(annual_mean_temp: float) -> str:
205     """Classifica la família climàtica a partir de la temperatura mitjana anual de bulb sec."""
206
207     if annual_mean_temp > 22:
208         return "Càlid"
209     if annual_mean_temp < 12:
210         return "Fred"
211     return "Mixt"
212
213
214 def _metadata_coordinates(metadata: dict[str, object]) -> tuple[float, float, float] | None:
215     """Retorna valors vàlids de latitud, longitud i zona horària de les metadades EPW."""
216
217     try:
218         coordinates = (
219             float(metadata.get("latitude")),
220             float(metadata.get("longitude")),
221             float(metadata.get("timezone")),
222         )
223     except (TypeError, ValueError):
224         return None
225     if any(pd.isna(value) for value in coordinates):
226         return None
227     return coordinates
228
229
230 def variable_label(variable_key: str, *, aggregated: bool = False) -> str:
231     """Retorna l'etiqueta UI i la unitat de visualització per a una clau variable EPW."""
232
233     spec = EPW_VARIABLES[variable_key]
234     if aggregated and spec.get("display_unit_aggregated"):
235         unit = spec["display_unit_aggregated"]
236     else:
237         unit = spec["unit"]
238     return f"{spec['label']} ({unit})" if unit else spec["label"]
239
240
241 def _parse_location_line(line: str) -> dict[str, object]:
242     """Analitza la línia de capçalera EPW LOCATION als camps de metadades del lloc."""
243
244     parts = [part.strip() for part in line.strip().split(",")]
245     return {
246         "city": parts[1] if len(parts) > 1 else "-",
247         "state": parts[2] if len(parts) > 2 else "-",
248         "country": parts[3] if len(parts) > 3 else "-",
249         "source": parts[4] if len(parts) > 4 else "-",
250         "wmo": parts[5] if len(parts) > 5 else "-",
251         "latitude": _parse_optional_float(parts[6] if len(parts) > 6 else None),
252         "longitude": _parse_optional_float(parts[7] if len(parts) > 7 else None),
253         "timezone": _parse_optional_float(parts[8] if len(parts) > 8 else None),
254         "elevation_m": _parse_optional_float(parts[9] if len(parts) > 9 else None),
255     }
256
257
258 def _read_epw_header(epw_path: Path) -> list[str]:
259     """Llegeix i torna les vuit línies de capçalera estàndard EPW."""
260
261     with epw_path.open("r", encoding="utf-8", errors="ignore") as file_handle:
262         return [file_handle.readline().rstrip("\n") for _ in range(8)]
263

```

```

264
265 def _build_epw_metadata(epw_path: Path, header_lines: list[str]) -> dict[str, object]:
266     """Crea metadades de fitxer i ubicació a partir de les línies de capçalera EPW."""
267
268     metadata = _parse_location_line(header_lines[0] if header_lines else "")
269     metadata.update(
270         {
271             "file_name": epw_path.name,
272             "stem": epw_path.stem,
273             "comments_1": _header_payload(header_lines, 5),
274             "comments_2": _header_payload(header_lines, 6),
275             "data_periods": header_lines[7] if len(header_lines) > 7 else "-",
276             "header_lines": header_lines,
277         }
278     )
279     return metadata
280
281
282 def _read_epw_data(epw_path: Path) -> pd.DataFrame:
283     """Llegeix les files de dades per hora EPW en un DataFrame sense processar."""
284
285     raw_data_frame = pd.read_csv(
286         epw_path,
287         skiprows=8,
288         header=None,
289         names=EPW_COLUMNS,
290         encoding="utf-8",
291         encoding_errors="ignore",
292     )
293     return raw_data_frame
294
295
296 def _clean_epw_data(data_frame: pd.DataFrame) -> pd.DataFrame:
297     """Converteix EPW columnes numèriques i substituir EPW els sentinelles de valors que falten per NaN."""
298
299     for column in NUMERIC_EPW_COLUMNS:
300         data_frame[column] = pd.to_numeric(data_frame[column], errors="coerce")
301
302     for column, sentinels in MISSING_SENTINELS.items():
303         if column in data_frame.columns:
304             data_frame[column] = data_frame[column].replace(list(sentinels), np.nan)
305
306     return data_frame
307
308
309 def _add_time_features(data_frame: pd.DataFrame) -> pd.DataFrame:
310     """Afegeix segells de temps normalitzats i atributs de calendari utilitzades pels gràfics de visualització."""
311
312     data_frame["hour_start"] = data_frame["hour"].fillna(1).clip(1, 24).astype(int) - 1
313     data_frame["source_year"] = data_frame["year"].fillna(2001).astype("Int64")
314     data_frame["timestamp"] = pd.to_datetime(
315         {
316             "year": 2001,
317             "month": data_frame["month"].fillna(1).astype(int),
318             "day": data_frame["day"].fillna(1).astype(int),
319             "hour": data_frame["hour_start"],
320             "minute": 0,
321         },
322         errors="coerce",
323     )
324     data_frame["date"] = data_frame["timestamp"].dt.normalize()
325     data_frame["month_label"] = pd.Categorical(
326         data_frame["month"].map(MONTH_LABELS),
327         categories=MONTH_ORDER,
328         ordered=True,
329     )
330     data_frame["day_of_year"] = data_frame["timestamp"].dt.dayofyear
331     return data_frame
332
333
334 def _add_climate_metadata(metadata: dict[str, object], data_frame: pd.DataFrame) -> dict[str, object]:
335     """Afegeix mètriques climàtiques derivades a les metadades EPW carregades."""
336
337     annual_mean_temp = float(data_frame["dry_bulb_c"].mean())
338     metadata["annual_mean_temp"] = annual_mean_temp
339     metadata["climate_family"] = _classify_climate(annual_mean_temp)
340     return metadata
341
342
343 @st.cache_data(show_spinner=False)
344 def list_epw_catalog(root_path_str: str) -> tuple[dict[str, str], ...]:
345     """Escaneja un directori arrel i retorna entrades de catàleg cercables per a tots els fitxers EPW."""
346
347     root_path = Path(root_path_str)
348     rows: list[dict[str, str]] = []
349
350     for epw_path in sorted(root_path.rglob("*.epw")):
351         city = "-"
352         country = "-"
353         try:
354             with epw_path.open("r", encoding="utf-8", errors="ignore") as file_handle:
355                 first_line = file_handle.readline()

```

```

356         location = _parse_location_line(first_line)
357         city = str(location["city"])
358         country = str(location["country"])
359     except OSError:
360         pass
361
362     relative_path = epw_path.relative_to(root_path).as_posix()
363     display_name = f"{{city}}, {{country}}{CATALOG_SEPARATOR}{{relative_path}}" if city != "-" else relative_path
364     search_blob = " ".join([display_name, epw_path.stem, relative_path]).lower()
365     rows.append(
366         {
367             "path": str(epw_path),
368             "relative_path": relative_path,
369             "display_name": display_name,
370             "search_blob": search_blob,
371         }
372     )
373
374     return tuple(rows)
375
376
377 @st.cache_data(show_spinner=False)
378 def load_epw_bundle(epw_path_str: str) -> dict[str, object]:
379     """Carrega un fitxer EPW i retorna'n les metadades més un clima enriquit DataFrame."""
380
381     epw_path = Path(epw_path_str)
382     header_lines = _read_epw_header(epw_path)
383     data_frame = _read_epw_data(epw_path)
384     data_frame = _clean_epw_data(data_frame)
385     data_frame = _add_time_features(data_frame)
386     metadata = _build_epw_metadata(epw_path, header_lines)
387     metadata = _add_climate_metadata(metadata, data_frame)
388
389     return {
390         "metadata": metadata,
391         "data": data_frame,
392     }
393
394
395 def summarize_epw_metrics(data_frame: pd.DataFrame) -> dict[str, float]:
396     """Retorna les mètriques del clima, el vent, la humitat, la pressió i la radiació."""
397
398     has_rows = not data_frame.empty
399     ghi_total_kwh_m2 = np.nan
400     dni_total_kwh_m2 = np.nan
401     dhi_total_kwh_m2 = np.nan
402     if has_rows:
403         ghi_total_kwh_m2 = float(data_frame["global_horizontal_radiation_wh_m2"].sum() / 1000.0)
404         dni_total_kwh_m2 = float(data_frame["direct_normal_radiation_wh_m2"].sum() / 1000.0)
405         dhi_total_kwh_m2 = float(data_frame["diffuse_horizontal_radiation_wh_m2"].sum() / 1000.0)
406
407     return {
408         "records": float(len(data_frame)),
409         "mean_temp": float(data_frame["dry_bulb_c"].mean()) if has_rows else np.nan,
410         "min_temp": float(data_frame["dry_bulb_c"].min()) if has_rows else np.nan,
411         "max_temp": float(data_frame["dry_bulb_c"].max()) if has_rows else np.nan,
412         "mean_rh": float(data_frame["relative_humidity_pct"].mean()) if has_rows else np.nan,
413         "mean_wind": float(data_frame["wind_speed_m_s"].mean()) if has_rows else np.nan,
414         "max_wind": float(data_frame["wind_speed_m_s"].max()) if has_rows else np.nan,
415         "ghi_total_kwh_m2": ghi_total_kwh_m2,
416         "dni_total_kwh_m2": dni_total_kwh_m2,
417         "dhi_total_kwh_m2": dhi_total_kwh_m2,
418     }
419
420
421 def build_monthly_summary(data_frame: pd.DataFrame) -> pd.DataFrame:
422     """Agrupar les principals variables tèrmiques, d'humitat, de vent i solars per mes."""
423
424     monthly = (
425         data_frame.groupby(["month", "month_label"], observed=True)
426         .agg(
427             dry_bulb_mean=("dry_bulb_c", "mean"),
428             dry_bulb_min=("dry_bulb_c", "min"),
429             dry_bulb_max=("dry_bulb_c", "max"),
430             relative_humidity_mean=("relative_humidity_pct", "mean"),
431             global_horizontal_radiation_kwh_m2=("global_horizontal_radiation_wh_m2", "sum"),
432             direct_normal_radiation_kwh_m2=("direct_normal_radiation_wh_m2", "sum"),
433             diffuse_horizontal_radiation_kwh_m2=("diffuse_horizontal_radiation_wh_m2", "sum"),
434         )
435         .reset_index()
436         .sort_values("month")
437     )
438     radiation_columns = [
439         "global_horizontal_radiation_kwh_m2",
440         "direct_normal_radiation_kwh_m2",
441         "diffuse_horizontal_radiation_kwh_m2",
442     ]
443     monthly[radiation_columns] = monthly[radiation_columns] / 1000.0
444     return monthly
445
446
447 def aggregate_epw_timeseries(

```

```

448     data_frame: pd.DataFrame,
449     variable_key: str,
450     aggregation_label: str,
451 ) -> pd.DataFrame:
452     """Agrega una variable EPW a la sèrie horària, diària, setmanal o mensual sol·licitada."""
453
454     series_frame = data_frame[["timestamp", variable_key]].dropna().sort_values("timestamp")
455     if series_frame.empty:
456         return series_frame
457
458     rule = TIME_AGGREGATION_RULES.get(aggregation_label)
459     if not rule:
460         return series_frame.reset_index(drop=True)
461
462     variable_spec = EPW_VARIABLES[variable_key]
463     aggregate_mode = variable_spec["aggregate"]
464     if aggregate_mode == "sum":
465         aggregated = series_frame.set_index("timestamp").resample(rule).sum(numeric_only=True).reset_index()
466     else:
467         aggregated = series_frame.set_index("timestamp").resample(rule).mean(numeric_only=True).reset_index()
468
469     if variable_spec.get("scale_to_kilo"):
470         aggregated[variable_key] = aggregated[variable_key] / 1000.0
471
472     return aggregated.reset_index(drop=True)
473
474
475 def build_daily_temperature_profile(data_frame: pd.DataFrame) -> pd.DataFrame:
476     """Retorna els valors diaris mínims, mitjans i màxims de temperatura de bulb sec."""
477
478     daily = (
479         data_frame.dropna(subset=["date"])
480         .groupby("date", observed=False)
481         .agg(
482             dry_bulb_min=("dry_bulb_c", "min"),
483             dry_bulb_mean=("dry_bulb_c", "mean"),
484             dry_bulb_max=("dry_bulb_c", "max"),
485         )
486         .reset_index()
487     )
488     return daily
489
490
491 def build_hourly_profile(data_frame: pd.DataFrame) -> pd.DataFrame:
492     """Retorna perfils horaris mitjans per a les principals variables meteorològiques."""
493
494     hourly = (
495         data_frame.groupby("hour_start", observed=False)
496         .agg(
497             dry_bulb_mean=("dry_bulb_c", "mean"),
498             dew_point_mean=("dew_point_c", "mean"),
499             relative_humidity_mean=("relative_humidity_pct", "mean"),
500         )
501         .reset_index()
502     )
503     return hourly
504
505
506 def build_heatmap_table(data_frame: pd.DataFrame, variable_key: str) -> pd.DataFrame:
507     """Crea una taula dinàmica mes a hora per a la variable EPW seleccionada."""
508
509     heatmap = (
510         data_frame.pivot_table(
511             index="month_label",
512             columns="hour_start",
513             values=variable_key,
514             aggfunc="mean",
515             observed=False,
516         )
517         .reindex(MONTH_ORDER)
518     )
519     return heatmap
520
521
522 def build_annual_hourly_heatmap_table(data_frame: pd.DataFrame, variable_key: str) -> pd.DataFrame:
523     """Crea una taula dinàmica hora a dia de l'any per a la variable EPW seleccionada."""
524
525     heatmap = (
526         data_frame.pivot_table(
527             index="hour_start",
528             columns="day_of_year",
529             values=variable_key,
530             aggfunc="mean",
531             observed=False,
532         )
533         .sort_index()
534         .reindex(index=list(range(24)))
535     )
536     return heatmap
537
538
539 def _solar_position_frame(

```

```

540 data_frame: pd.DataFrame,
541 latitude_deg: float,
542 longitude_deg: float,
543 timezone_hours: float,
544 ) -> pd.DataFrame:
545     """Estima l'altitud i l'azimut solar per a cada fila horària EPW."""
546
547     solar_frame = data_frame[["day_of_year", "hour_start"]].copy()
548     solar_frame = solar_frame.dropna()
549     if solar_frame.empty:
550         return pd.DataFrame(columns=["altitude_deg", "azimuth_deg"])
551
552     day_of_year = solar_frame["day_of_year"].to_numpy(dtype=float)
553     local_hour = solar_frame["hour_start"].to_numpy(dtype=float) + 0.5
554
555     gamma = 2.0 * np.pi / 365.0 * (day_of_year - 1.0 + (local_hour - 12.0) / 24.0)
556     equation_of_time = 229.18 * (
557         0.000075
558         + 0.001868 * np.cos(gamma)
559         - 0.032077 * np.sin(gamma)
560         - 0.014615 * np.cos(2.0 * gamma)
561         - 0.040849 * np.sin(2.0 * gamma)
562     )
563     declination = (
564         0.006918
565         - 0.399912 * np.cos(gamma)
566         + 0.070257 * np.sin(gamma)
567         - 0.006758 * np.cos(2.0 * gamma)
568         + 0.000907 * np.sin(2.0 * gamma)
569         - 0.002697 * np.cos(3.0 * gamma)
570         + 0.00148 * np.sin(3.0 * gamma)
571     )
572
573     true_solar_minutes = (local_hour * 60.0 + equation_of_time + 4.0 * longitude_deg - 60.0 * timezone_hours) % 1440.0
574     hour_angle_deg = true_solar_minutes / 4.0 - 180.0
575     hour_angle_rad = np.deg2rad(hour_angle_deg)
576     latitude_rad = np.deg2rad(latitude_deg)
577
578     cos_zenith = (
579         np.sin(latitude_rad) * np.sin(declination)
580         + np.cos(latitude_rad) * np.cos(declination) * np.cos(hour_angle_rad)
581     )
582     cos_zenith = np.clip(cos_zenith, -1.0, 1.0)
583     zenith_rad = np.arccos(cos_zenith)
584     altitude_deg = np.rad2deg(np.pi / 2.0 - zenith_rad)
585
586     azimuth_deg = (
587         np.rad2deg(
588             np.arctan2(
589                 np.sin(hour_angle_rad),
590                 np.cos(hour_angle_rad) * np.sin(latitude_rad) - np.tan(declination) * np.cos(latitude_rad),
591             )
592         )
593         + 180.0
594     ) % 360.0
595
596     return pd.DataFrame(
597         {
598             "altitude_deg": altitude_deg,
599             "azimuth_deg": azimuth_deg,
600         },
601         index=solar_frame.index,
602     )
603
604
605 def build_comfort_radiation_table(data_frame: pd.DataFrame, metadata: dict[str, object]) -> pd.DataFrame:
606     """Estima la radiació incident per orientació, ponderada pel benefici de calefacció o refrigeració."""
607
608     empty_result = pd.DataFrame(columns=COMFORT_RADIATION_COLUMNS)
609     coordinates = _metadata_coordinates(metadata)
610     if coordinates is None:
611         return empty_result.copy()
612     latitude, longitude, timezone = coordinates
613
614     radiation_frame = data_frame[list(COMFORT_RADIATION_SOURCE_COLUMNS)].dropna().copy()
615     if radiation_frame.empty:
616         return empty_result.copy()
617
618     solar_frame = _solar_position_frame(radiation_frame, latitude, longitude, timezone)
619     if solar_frame.empty:
620         return empty_result.copy()
621
622     radiation_frame = radiation_frame.join(solar_frame)
623     radiation_frame = radiation_frame[radiation_frame["altitude_deg"] > 0.0].copy()
624     if radiation_frame.empty:
625         return empty_result.copy()
626
627     altitude_edges = np.array([0.0, 15.0, 30.0, 45.0, 60.0, 75.0, 90.0])
628     altitude_labels = tuple(
629         f"{int(start)}-{int(end)}{DEG}" for start, end in zip(altitude_edges[:-1], altitude_edges[1:])
630     )
631     altitude_indices = np.digitize(radiation_frame["altitude_deg"].to_numpy(), altitude_edges, right=False) - 1

```

```

632 altitude_indices = np.clip(altitude_indices, 0, len(altitude_labels) - 1)
633
634 temperature = radiation_frame["dry_bulb_c"].to_numpy(dtype=float)
635 direct_normal = radiation_frame["direct_normal_radiation_wh_m2"].to_numpy(dtype=float)
636 diffuse_horizontal = radiation_frame["diffuse_horizontal_radiation_wh_m2"].to_numpy(dtype=float)
637 altitude_rad = np.deg2rad(radiation_frame["altitude_deg"].to_numpy(dtype=float))
638 azimuth_deg = radiation_frame["azimuth_deg"].to_numpy(dtype=float)
639
640 benefit_weight = np.clip((18.0 - temperature) / 8.0, 0.0, 1.0)
641 harm_weight = np.clip((temperature - 24.0) / 8.0, 0.0, 1.0)
642 comfort_weight = benefit_weight - harm_weight
643
644 rows: list[dict[str, float | str]] = []
645 orientation_centers = np.arange(0.0, 360.0, 22.5)
646
647 for direction_label, theta_deg in zip(WIND_DIRECTION_LABELS, orientation_centers):
648     azimuth_delta_rad = np.deg2rad(((azimuth_deg - theta_deg + 180.0) % 360.0) - 180.0)
649     incident_direct = direct_normal * np.maximum(0.0, np.cos(altitude_rad) * np.cos(azimuth_delta_rad))
650     incident_vertical = incident_direct + diffuse_horizontal * 0.5
651     signed_incident = incident_vertical * comfort_weight
652
653     orientation_frame = pd.DataFrame(
654         {
655             "altitude_band": [altitude_labels[int(index)] for index in altitude_indices],
656             "comfort_radiation_kwh_m2": signed_incident / 1000.0,
657             "incident_radiation_kwh_m2": incident_vertical / 1000.0,
658         }
659     )
660     summary = (
661         orientation_frame.groupby("altitude_band", observed=False)
662         .agg(
663             comfort_radiation_kwh_m2=("comfort_radiation_kwh_m2", "sum"),
664             incident_radiation_kwh_m2=("incident_radiation_kwh_m2", "sum"),
665         )
666         .reindex(altitude_labels, fill_value=0.0)
667         .reset_index()
668     )
669
670     for band_index, band_row in summary.iterrows():
671         rows.append(
672             {
673                 "direction": direction_label,
674                 "theta_deg": float(theta_deg),
675                 "altitude_band": str(band_row["altitude_band"]),
676                 "radius_start": float(altitude_edges[band_index]),
677                 "radius_size": float(altitude_edges[band_index + 1] - altitude_edges[band_index]),
678                 "comfort_radiation_kwh_m2": float(band_row["comfort_radiation_kwh_m2"]),
679                 "incident_radiation_kwh_m2": float(band_row["incident_radiation_kwh_m2"]),
680             }
681         )
682
683     return pd.DataFrame(rows)
684
685
686 def _build_wind_rose_table(data_frame: pd.DataFrame) -> pd.DataFrame:
687     """Frequència del vent de retorn i velocitat mitjana agrupades en sectors de brúixola."""
688
689     wind_frame = data_frame[["wind_direction_deg", "wind_speed_m_s"]].dropna()
690     if wind_frame.empty:
691         return pd.DataFrame(columns=["direction", "frequency_pct", "mean_speed"])
692
693     adjusted = (wind_frame["wind_direction_deg"] + 11.25) % 360
694     sector_indices = ((adjusted / 22.5).astype(int) % len(WIND_DIRECTION_LABELS)).to_numpy()
695     sectors = [WIND_DIRECTION_LABELS[int(index)] for index in sector_indices]
696     rose = (
697         pd.DataFrame(
698             {
699                 "direction": sectors,
700                 "wind_speed_m_s": wind_frame["wind_speed_m_s"].to_numpy(),
701             }
702         )
703         .groupby("direction", as_index=False)
704         .agg(
705             count=("wind_speed_m_s", "size"),
706             mean_speed=("wind_speed_m_s", "mean"),
707         )
708         .set_index("direction")
709         .reindex(WIND_DIRECTION_LABELS, fill_value=0.0)
710         .reset_index()
711     )
712     total = rose["count"].sum()
713     rose["frequency_pct"] = np.where(total > 0, rose["count"] / total * 100.0, 0.0)
714     return rose
715
716
717 def build_monthly_wind_rose_tables(data_frame: pd.DataFrame) -> dict[str, pd.DataFrame]:
718     """Construeix una taula resum de la rosa dels vents per a cada mes natural."""
719
720     month_tables: dict[str, pd.DataFrame] = {}
721     for month_number, month_label in MONTH_LABELS.items():
722         month_frame = data_frame[data_frame["month"] == month_number]
723         rose = _build_wind_rose_table(month_frame)

```

```

724         if not rose.empty:
725             month_tables[month_label] = rose
726         return month_tables
727
728
729 def build_download_frame(data_frame: pd.DataFrame) -> pd.DataFrame:
730     """Retorna les columnes EPW netes que s'exporten com a CSV des de UI."""
731
732     export_columns = [
733         "timestamp",
734         "source_year",
735         "month",
736         "day",
737         "hour",
738         "dry_bulb_c",
739         "dew_point_c",
740         "relative_humidity_pct",
741         "wind_speed_m_s",
742         "wind_direction_deg",
743         "atmospheric_pressure_pa",
744         "global_horizontal_radiation_wh_m2",
745         "direct_normal_radiation_wh_m2",
746         "diffuse_horizontal_radiation_wh_m2",
747         "total_sky_cover_tenths",
748         "opaque_sky_cover_tenths",
749     ]
750     export_frame = data_frame[export_columns].copy()
751     export_frame["timestamp"] = export_frame["timestamp"].dt.strftime("%Y-%m-%d %H:%M")
752     return export_frame

```

E.33 backend/weather_profiles.py

Listing 54: backend/weather_profiles.py

```

1  """Anàlisi de perfils meteorològics i vista prèvia per crear entorns."""
2
3  from __future__ import annotations
4
5  import csv
6  import re
7  from dataclasses import dataclass
8  from pathlib import Path
9  from typing import Dict, List, Optional, Sequence, Tuple
10
11  import numpy as np
12  import plotly.graph_objects as go
13
14
15  EPW_HEADER_ROW_COUNT = 8
16  EPW_DRY_BULB_COLUMN_INDEX = 6
17  WEATHER_PREVIEW_RANDOM_SEED = 42
18  MONTH_START_DAYS = (0, 31, 59, 90, 120, 151, 181, 212, 243, 273, 304, 334)
19
20
21  def _sanitize_weather_identififer(raw_value: str, fallback: str) -> str:
22     """Retorna un identificador estable en minúscules per a les claus del perfil meteorològic."""
23
24     slug = re.sub(r"[^a-zA-Z0-9]+", "_", raw_value.strip().lower()).strip("_")
25     return slug or fallback
26
27
28  def _unique_weather_identififer(base: str, used: set[str]) -> str:
29     """Retorna un identificador meteorològic únic dins del conjunt proporcionat."""
30
31     candidate = base
32     suffix = 2
33     while candidate in used:
34         candidate = f"{base}_{suffix}"
35         suffix += 1
36     used.add(candidate)
37     return candidate
38
39
40  @dataclass(frozen=True)
41  class WeatherProfileSuggestion:
42     """Descriu un suggeriment de fitxer meteorològic per a la creació de l'entorn UI."""
43
44     file_name: str
45     path: str
46     suggested_key: str
47     climate_label: str
48     mean_dry_bulb: Optional[float]
49
50
51  def summarize_weather_file(weather_path: Path) -> Tuple[Optional[float], str]:
52     """Analitza un fitxer EnergyPlus Weather (EPW) per suggerir una classificació climàtica.
53
54     Paràmetres:

```

```

55     weather_path (Path): camí cap al fitxer EPW.
56
57     Retorna:
58         Tuple[Optional[float], str]: una tupla que conté la temperatura mitjana de bulb sec (si s'analitza correctament),
59         i una cadena de category ('calent', 'cool', 'mixt' o 'personalitzat').
60     """
61     try:
62         with open(weather_path, "r", encoding="utf-8", errors="ignore") as file_handle:
63             lines = file_handle.readlines()[8:]
64     except OSError:
65         return None, "custom"
66
67     temperatures: List[float] = []
68     for line in lines:
69         parts = line.strip().split(",")
70         if len(parts) <= 6:
71             continue
72         try:
73             temperatures.append(float(parts[6]))
74         except ValueError:
75             continue
76
77     if not temperatures:
78         return None, "custom"
79
80     mean_dry_bulb = sum(temperatures) / len(temperatures)
81     if mean_dry_bulb >= 18.0:
82         return mean_dry_bulb, "hot"
83     if mean_dry_bulb <= 12.0:
84         return mean_dry_bulb, "cool"
85     return mean_dry_bulb, "mixed"
86
87
88 def _parse_epw_int(value: str) -> Optional[int]:
89     """Analitza un nombre enter d'una cel·la EPW tot tolerant les cadenes decimals."""
90
91     try:
92         return int(float(value))
93     except (TypeError, ValueError):
94         return None
95
96
97 def _parse_epw_float(value: str) -> Optional[float]:
98     """Analitza un nombre flotant d'una cel·la EPW i retorna None quan la cel·la no sigui vàlida."""
99
100     try:
101         return float(value)
102     except (TypeError, ValueError):
103         return None
104
105
106 def _calculate_epw_day_of_year(row: Sequence[str], fallback_index: int) -> int:
107     """Retorna el dia de l'any basat en 1 representat per una fila de dades EPW."""
108
109     fallback_day = fallback_index // 24 + 1
110     if len(row) < 3:
111         return fallback_day
112
113     month = _parse_epw_int(row[1])
114     day = _parse_epw_int(row[2])
115     if month is None or day is None or month < 1 or month > 12 or day < 1:
116         return fallback_day
117
118     return MONTH_START_DAYS[month - 1] + day
119
120
121 def load_epw_dry_bulb_series(weather_path: Path) -> List[Dict[str, float]]:
122     """Carrega els registres de temperatura de bulb sec cada hora des d'un fitxer meteorològic EPW.
123
124     Paràmetres:
125         weather_path (Path): camí cap al fitxer meteorològic EPW.
126
127     Retorna:
128         List[Dict[str, float]]: registres horàries amb dia de l'any, index d'hores,
129         i temperatura de bulb sec en graus centígrads. S'ometen files no vàlides.
130     """
131
132     records: List[Dict[str, float]] = []
133     try:
134         with open(weather_path, "r", encoding="utf-8", errors="ignore", newline="") as file_handle:
135             reader = csv.reader(file_handle)
136             for _ in range(EPW_HEADER_ROW_COUNT):
137                 next(reader, None)
138
139             for row in reader:
140                 if len(row) <= EPW_DRY_BULB_COLUMN_INDEX:
141                     continue
142                 temperature = _parse_epw_float(row[EPW_DRY_BULB_COLUMN_INDEX])
143                 if temperature is None:
144                     continue
145
146                 record_index = len(records)

```



```

147         records.append(
148             {
149                 "annual_hour": float(record_index + 1),
150                 "day_of_year": float(_calculate_epw_day_of_year(row, record_index)),
151                 "dry_bulb_temperature_c": temperature,
152             }
153         )
154     except OSError:
155         return []
156
157     return records
158
159
160 def _build_ornstein_uhlenbeck_noise(
161     row_count: int,
162     sigma: float,
163     mu: float,
164     tau: float,
165     *,
166     seed: int = WEATHER_PREVIEW_RANDOM_SEED,
167 ) -> np.ndarray:
168     """Construeix un soroll determinista Ornstein-Uhlenbeck per al gràfic de vista prèvia del clima."""
169
170     safe_tau = max(float(tau), 1e-9)
171     sigma_bis = float(sigma) * np.sqrt(2.0 / safe_tau)
172     rng = np.random.default_rng(seed)
173     noise = np.zeros(row_count)
174
175     for index in range(row_count - 1):
176         noise[index + 1] = (
177             noise[index]
178             + (-(noise[index] - float(mu)) / safe_tau)
179             + sigma_bis * rng.normal()
180         )
181
182     return noise
183
184
185 def build_weather_temperature_preview(
186     weather_path: Path,
187     sigma: float,
188     mu: float,
189     tau: float,
190 ) -> List[Dict[str, float]]:
191     """Crea dades de vista prèvia de la temperatura diària per a la variació estocàstica del clima.
192
193     La vista prèvia reflecteix la pertorbació meteorològica d'Ornstein-Uhlenbeck amb Sinergym
194     una llavor aleatòria fixa, de manera que UI s'actualitza de manera previsible quan l'usuari canvia el
195     controls sigma, mu o tau.
196
197     Paràmetres:
198         weather_path (Path): fitxer EPW utilitzat com a font climàtica base.
199         sigma (float): paràmetre de desviació estàndard per al procés d'OU.
200         mu (float): paràmetre mitjà per al procés d'OU.
201         tau (float): paràmetre constant de temps per al procés d'OU, en hores.
202
203     Retorna:
204         List[Dict[str, float]]: temperatures mitjanes diàries per al EPW original,
205         la vista prèvia modificada determinista i una banda sigma visual.
206     """
207
208     hourly_records = load_epw_dry_bulb_series(weather_path)
209     if not hourly_records:
210         return []
211
212     noise = _build_ornstein_uhlenbeck_noise(
213         len(hourly_records),
214         sigma=float(sigma),
215         mu=float(mu),
216         tau=float(tau),
217     )
218     daily_accumulators: Dict[int, Dict[str, float]] = {}
219
220     for index, record in enumerate(hourly_records):
221         day = int(record["day_of_year"])
222         base_temperature = record["dry_bulb_temperature_c"]
223         expected_temperature = base_temperature + float(mu)
224         modified_temperature = base_temperature + float(noise[index])
225         accumulator = daily_accumulators.setdefault(
226             day,
227             {
228                 "count": 0.0,
229                 "base_temperature_c": 0.0,
230                 "expected_temperature_c": 0.0,
231                 "modified_temperature_c": 0.0,
232                 "lower_temperature_c": 0.0,
233                 "upper_temperature_c": 0.0,
234             },
235         )
236         accumulator["count"] += 1.0
237         accumulator["base_temperature_c"] += base_temperature
238         accumulator["expected_temperature_c"] += expected_temperature

```

```

239     accumulator["modified_temperature_c"] += modified_temperature
240     accumulator["lower_temperature_c"] += expected_temperature - float(sigma)
241     accumulator["upper_temperature_c"] += expected_temperature + float(sigma)
242
243     preview_records: List[Dict[str, float]] = []
244     for day, accumulator in sorted(daily_accumulators.items()):
245         count = max(accumulator.pop("count"), 1.0)
246         preview_records.append(
247             {
248                 "day_of_year": float(day),
249                 "base_temperature_c": accumulator["base_temperature_c"] / count,
250                 "expected_temperature_c": accumulator["expected_temperature_c"] / count,
251                 "modified_temperature_c": accumulator["modified_temperature_c"] / count,
252                 "lower_temperature_c": accumulator["lower_temperature_c"] / count,
253                 "upper_temperature_c": accumulator["upper_temperature_c"] / count,
254             }
255         )
256
257     return preview_records
258
259
260 def build_weather_temperature_preview_figure(preview_records: Sequence[Dict[str, float]]) -> go.Figure:
261     """Crea la figura de previsualització anual de la temperatura per a la pàgina Afegeix un entorn.
262
263     Paràmetres:
264         preview_records (Sequence[Dict[str, float]]): dades de vista prèvia de la temperatura diària.
265
266     Retorna:
267         go.Figure: Plotly figura que compara la temperatura de bulb sec original i modificada.
268     """
269
270     day_values = [record["day_of_year"] for record in preview_records]
271     max_day = max(day_values) if day_values else 365.0
272     fig = go.Figure()
273     fig.add_trace(
274         go.Scatter(
275             x=day_values,
276             y=[record["upper_temperature_c"] for record in preview_records],
277             mode="lines",
278             line=dict(width=0),
279             showlegend=False,
280             hoverinfo="skip",
281         )
282     )
283     fig.add_trace(
284         go.Scatter(
285             x=day_values,
286             y=[record["lower_temperature_c"] for record in preview_records],
287             mode="lines",
288             fill="tonexty",
289             fillcolor="rgba(95, 84, 249, 0.13)",
290             line=dict(width=0),
291             name="Rang sigma",
292             hoverinfo="skip",
293         )
294     )
295     fig.add_trace(
296         go.Scatter(
297             x=day_values,
298             y=[record["base_temperature_c"] for record in preview_records],
299             mode="lines",
300             name="EPW original",
301             line=dict(color="#3b82f6", width=2),
302             hovertemplate="Dia {x:.0f}<br>Original {y:.1f} °C<extra></extra>",
303         )
304     )
305     fig.add_trace(
306         go.Scatter(
307             x=day_values,
308             y=[record["modified_temperature_c"] for record in preview_records],
309             mode="lines",
310             name="Variant simulada",
311             line=dict(color="#ef4444", width=2.2),
312             hovertemplate="Dia {x:.0f}<br>Simulada {y:.1f} °C<extra></extra>",
313         )
314     )
315     fig.update_layout(
316         height=420,
317         margin=dict(l=42, r=24, t=34, b=46),
318         template="plotly_white",
319         paper_bgcolor="#ffffff",
320         plot_bgcolor="#f7f9fe",
321         font=dict(color="#17233c", family="Bahnschrift, Aptos, Segoe UI"),
322         legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1),
323         xaxis=dict(
324             title="Dia de l'any",
325             range=[1, max(365.0, float(max_day))],
326             gridcolor="#dce3f2",
327             zeroline=False,
328         ),
329         yaxis=dict(
330             title="Temperatura seca (°C)",

```

```

331         gridcolor="#dce3f2",
332         zeroline=False,
333     ),
334 )
335 return fig
336
337
338 def build_weather_profile_suggestions(weather_paths: Sequence[Path]) -> List[WeatherProfileSuggestion]:
339     """Crea configuracions de perfils per a fitxers meteorològics amb noms i categorització fàcils d'utilitzar.
340
341     Paràmetres:
342         weather_paths (Sequence[Path]): una col·lecció de camins que apunten a fitxers EPW.
343
344     Retorna:
345         List[WeatherProfileSuggestion]: Configuracions recomanades i resums climàtics per als fitxers.
346     """
347     used_keys: set[str] = set()
348     suggestions: List[WeatherProfileSuggestion] = []
349
350     for path in weather_paths:
351         mean_dry_bulb, suggested_family = summarize_weather_file(path)
352         if mean_dry_bulb is None:
353             climate_label = "Sense resum climàtic automàtic"
354             base_key = _sanitize_weather_identifier(path.stem, "weather")
355         else:
356             if suggested_family == "hot":
357                 climate_label = f"Clima càlid ({mean_dry_bulb:.1f} °C de bulb sec mitjà)"
358             elif suggested_family == "cool":
359                 climate_label = f"Clima fred ({mean_dry_bulb:.1f} °C de bulb sec mitjà)"
360             else:
361                 climate_label = f"Clima mixt ({mean_dry_bulb:.1f} °C de bulb sec mitjà)"
362             base_key = suggested_family
363
364         unique_key = _unique_weather_identifier(base_key, used_keys)
365         suggestions.append(
366             WeatherProfileSuggestion(
367                 file_name=path.name,
368                 path=str(path),
369                 suggested_key=unique_key,
370                 climate_label=climate_label,
371                 mean_dry_bulb=mean_dry_bulb,
372             )
373         )
374
375     return suggestions

```

F Codi: Gràfics i mètriques

F.1 backend/grafics/__init__.py

Listing 55: backend/grafics/__init__.py

```

1  """Gràfics Plotly i mètriques compartides per a resultats i informes.
2
3  Els mòduls d'aquest paquet converteixen les dades d'execució normalitzades en xifres reutilitzables:
4  compliment de comoditat, consum HVAC, preus de l'energia, comportament de la bateria, acció
5  senyals i mapes de calor temporals. Per tant, haurien de romandre independents de Streamlit
6  el panell i el generador d'informes poden compartir la mateixa lògica visual.
7  """

```

F.2 backend/grafics/battery.py

Listing 56: backend/grafics/battery.py

```

1  """Dades del quadre de comandament de la bateria, la xarxa i les tarifes."""
2
3  from __future__ import annotations
4
5  import pandas as pd
6  import plotly.graph_objects as go
7
8  from backend.grafics.column_utils import (
9      _BATTERY_CHARGE_POWER_COLUMNS,
10     _BATTERY_DISCHARGE_ENERGY_COLUMNS,
11     _BATTERY_DISCHARGE_POWER_COLUMNS,
12     _GRID_ENERGY_COLUMNS,
13     _GRID_KWH_COLUMNS,
14     _GRID_POWER_COLUMNS,
15     _find_battery_state_column,
16     _find_energy_price_column,
17     _find_first_present_column,
18 )

```

```

19 from backend.grafics.energy_units import (
20     _battery_state_display_series,
21     _energy_price_series_eur_per_kwh,
22     _energy_to_kwh,
23 )
24 from backend.grafics.style import STUDIO_CHART_COLORS, _apply_figure_theme
25 from backend.grafics.time_utils import (
26     _auto_scale_series,
27     _canon_day_series,
28     _ensure_datetime_index,
29     _hour_series,
30     _month_series,
31     _raw_axis_data,
32     _seasonal_marker_colors,
33     filter_by_season,
34 )
35 from backend.grafics.hvac import _canon_day_total_series, _infer_timestep_hours
36
37
38 def _add_battery_power_trace(
39     fig: go.Figure,
40     base: pd.DataFrame,
41     series: str | None,
42     label: str,
43     sign: int,
44     mode: str,
45     season: str,
46     units: str,
47 ) -> None:
48     """Afegeix un rastre de càrrega o descàrrega a la figura de la bateria."""
49     if series is None or series not in base.columns:
50         return
51     if mode == "hour":
52         df = filter_by_season(base, season)
53         grouped = df.groupby(df.index.hour)[series].mean() * sign
54         x_values = list(range(24))
55         y_values = [grouped.get(hour, float("nan")) for hour in x_values]
56         fig.add_trace(go.Scatter(x=x_values, y=y_values, mode="lines+markers", name=label))
57     elif mode == "day":
58         df = filter_by_season(base, season)
59         x_values, y_values, hover = _canon_day_series(df, series)
60         y_values = [value * sign for value in y_values]
61         fig.add_trace(
62             go.Scatter(
63                 x=x_values,
64                 y=y_values,
65                 mode="lines",
66                 name=label,
67                 hovertext=hover,
68                 hovertemplate="%{hovertext}<br>{%y:.2f} " + units + "<extra></extra>",
69             )
70         )
71     elif mode == "month":
72         grouped = base.groupby(base.index.month)[series].mean() * sign
73         x_values = list(range(1, 13))
74         y_values = [grouped.get(month, float("nan")) for month in x_values]
75         fig.add_trace(go.Scatter(x=x_values, y=y_values, mode="lines+markers", name=label))
76     else:
77         df = filter_by_season(base, season)
78         x_values, y_values, hover = _raw_axis_data(df, series)
79         y_values = [value * sign for value in y_values]
80         fig.add_trace(
81             go.Scatter(
82                 x=x_values,
83                 y=y_values,
84                 mode="lines",
85                 name=label,
86                 hovertext=hover,
87                 hovertemplate="%{hovertext}<br>{%y:.2f} " + units + "<extra></extra>",
88             )
89         )
90
91
92 def _add_battery_soc_line(fig: go.Figure, x_values: list, y_values: list, label: str, mode: str) -> None:
93     """Afegeix una traça de línia d'estat de càrrega."""
94     marker_mode = "lines+markers" if mode in ("hour", "month") else "lines"
95     fig.add_trace(go.Scatter(x=x_values, y=y_values, mode=marker_mode, name=label))
96
97
98 def _add_battery_grid_bar_trace(
99     fig: go.Figure,
100     base: pd.DataFrame,
101     column: str,
102     label: str,
103     color: str,
104     mode: str,
105     season: str,
106 ) -> None:
107     """Afegeix una traça de la barra de bateria contra la xarxa per al mode d'agregació seleccionat."""
108     if column not in base.columns:
109         return
110     # Manté la mateixa convenció temporal que la resta de gràfics:

```

```

111 # hora típica, dia anual, mes mitjà o sèrie raw.
112 if mode == "hour":
113     df = filter_by_season(base, season)
114     if df.empty:
115         return
116     day_keys = pd.Index(df.index.floor("D"), name="day")
117     hour_keys = pd.Index(_hour_series(df), name="hour")
118     slots = df.groupby([day_keys, hour_keys])[column].sum()
119     grouped = slots.groupby(level="hour").mean()
120     x_values = list(range(24))
121     y_values = [grouped.get(hour, float("nan")) for hour in x_values]
122     fig.add_trace(
123         go.Bar(
124             x=x_values,
125             y=y_values,
126             name=label,
127             marker=dict(color=color, line=dict(color=color, width=0.5)),
128         )
129     )
130 elif mode == "day":
131     df = filter_by_season(base, season)
132     if df.empty:
133         return
134     x_values, y_values, hover = _canon_day_total_series(df, column)
135     fig.add_trace(
136         go.Bar(
137             x=x_values,
138             y=y_values,
139             name=label,
140             marker=dict(color=color, line=dict(color=color, width=0.5)),
141             hovertext=hover,
142             hovertemplate="%{hovertext}<br>{%y:.2f} kWh<extra></extra>",
143         )
144     )
145 elif mode == "month":
146     df = base if season == "All" else filter_by_season(base, season)
147     if df.empty:
148         return
149     if isinstance(df.index, pd.DatetimeIndex):
150         monthly = df.groupby([df.index.year, df.index.month])[column].sum()
151         grouped = monthly.groupby(level=1).mean()
152     else:
153         grouped = df.groupby(_month_series(df))[column].sum()
154     x_values = list(range(1, 13))
155     y_values = [grouped.get(month, float("nan")) for month in x_values]
156     marker_colors = _seasonal_marker_colors(x_values, season, active_color=color)
157     fig.add_trace(go.Bar(x=x_values, y=y_values, name=label, marker=dict(color=marker_colors)))
158 elif mode == "raw":
159     df = filter_by_season(base, season)
160     if df.empty:
161         return
162     x_values, y_values, hover = _raw_axis_data(df, column)
163     fig.add_trace(
164         go.Bar(
165             x=x_values,
166             y=y_values,
167             name=label,
168             marker=dict(color=color, line=dict(color=color, width=0.5)),
169             hovertext=hover,
170             hovertemplate="%{hovertext}<br>{%y:.4f} kWh<extra></extra>",
171         )
172     )
173
174
175 def _aggregate_battery_series(
176     base: pd.DataFrame,
177     column: str | None,
178     sign: int,
179     mode: str,
180     season: str,
181 ) -> tuple[list | None, list | None, list | None]:
182     """Retorna les dades de traça de la bateria agregades per al mode seleccionat."""
183     if column is None or column not in base.columns:
184         return None, None, None
185     # sign permet reutilitzar el càlcul per càrrega i descàrrega sense duplicar totes
186     # les branques d'agregació.
187     if mode == "hour":
188         df = filter_by_season(base, season)
189         if df.empty:
190             return None, None, None
191         grouped = df.groupby(_hour_series(df))[column].mean() * sign
192         x_values = list(range(24))
193         y_values = [grouped.get(hour, float("nan")) for hour in x_values]
194         return x_values, y_values, None
195     if mode == "day":
196         df = filter_by_season(base, season)
197         if df.empty:
198             return None, None, None
199         x_values, y_values, hover = _canon_day_series(df, column)
200         return x_values, [value * sign for value in y_values], hover
201     if mode == "month":
202         df = base if season == "All" else filter_by_season(base, season)

```

```

203     if df.empty:
204         return None, None, None
205     grouped = df.groupby(_month_series(df))[column].mean() * sign
206     x_values = list(range(1, 13))
207     y_values = [grouped.get(month, float("nan")) for month in x_values]
208     return x_values, y_values, None
209
210 df = filter_by_season(base, season)
211 if df.empty:
212     return None, None, None
213 x_values, y_values, hover = _raw_axis_data(df, column)
214 return x_values, [value * sign for value in y_values], hover
215
216
217 def _aggregate_price_series(
218     base: pd.DataFrame,
219     price_col: str,
220     has_price: bool,
221     mode: str,
222     season: str,
223 ) -> tuple[list | None, list | None, list | None]:
224     """Retorna les dades de traça de preus agregades per al mode seleccionat."""
225     if not has_price:
226         return None, None, None
227     if mode == "hour":
228         df = filter_by_season(base, season)
229         if df.empty:
230             return None, None, None
231         grouped = df.groupby(_hour_series(df))[price_col].mean()
232         x_values = list(range(24))
233         y_values = [grouped.get(hour, float("nan")) for hour in x_values]
234         return x_values, y_values, None
235     if mode == "day":
236         df = filter_by_season(base, season)
237         if df.empty:
238             return None, None, None
239         return _canon_day_series(df, price_col)
240     if mode == "month":
241         df = base if season == "All" else filter_by_season(base, season)
242         if df.empty:
243             return None, None, None
244         grouped = df.groupby(_month_series(df))[price_col].mean()
245         x_values = list(range(1, 13))
246         y_values = [grouped.get(month, float("nan")) for month in x_values]
247         return x_values, y_values, None
248
249 df = filter_by_season(base, season)
250 if df.empty:
251     return None, None, None
252 return _raw_axis_data(df, price_col)
253
254
255 def make_battery_power_plot(obs: pd.DataFrame, mode: str, season: str) -> go.Figure:
256     """Crea una figura Plotly per a la bateria."""
257     base = _ensure_datetime_index(obs).copy()
258     fig = go.Figure()
259
260     charge_power_col = _find_first_present_column(base.columns, _BATTERY_CHARGE_POWER_COLUMNS)
261     discharge_power_col = _find_first_present_column(base.columns, _BATTERY_DISCHARGE_POWER_COLUMNS)
262     state_col = _find_battery_state_column(base.columns)
263     has_charge_p = charge_power_col is not None
264     has_discharge_p = discharge_power_col is not None
265     has_state = state_col is not None
266
267     if not (has_charge_p or has_discharge_p or has_state):
268         return go.Figure()
269
270     scale_suffix = ""
271     # Reserva: només tenim battery_charge_state → fem servir la diferència per pas
272     if not (has_charge_p or has_discharge_p):
273         s = pd.to_numeric(base[state_col], errors="coerce")
274         d = s.diff().fillna(0.0) # +: carrega, -: descarrega
275         d, scale_suffix = _auto_scale_series(d) # millora llegibilitat
276         base["__charge__"] = d.clip(lower=0.0) # 0
277         base["__discharge__"] = -d.clip(upper=0.0) # 0
278         charge_col, discharge_col = "__charge__", "__discharge__"
279         units = f"Δstate/step {scale_suffix}" if scale_suffix else "Δstate/step"
280     else:
281         charge_col = charge_power_col
282         discharge_col = discharge_power_col
283         units = "W"
284
285     # Convenció del gràfic: càrrega positiva, descàrrega negativa
286     _add_battery_power_trace(fig, base, charge_col, "Charge (+)", +1, mode, season, units)
287     _add_battery_power_trace(fig, base, discharge_col, "Discharge -()", -1, mode, season, units)
288
289     # Eixos i línia y=0
290     fig.update_yaxes(title=f"Battery Power ({units})")
291     fig.add_hline(y=0, line_dash="dash", line_color="black")
292     fig.update_layout(title=f"Battery Charge/Discharge - {mode.capitalize()} ({season})")
293     return fig
294

```

```

295
296
297 def make_battery_soc_plot(obs: pd.DataFrame, mode: str, season: str) -> go.Figure:
298     """Crea una figura Plotly per a la bateria."""
299     base = _ensure_datetime_index(obs).copy()
300     fig = go.Figure()
301
302     state_col = _find_battery_state_column(base.columns)
303     if state_col is None:
304         return go.Figure()
305
306     s = pd.to_numeric(base[state_col], errors="coerce")
307     if s.dropna().empty:
308         return go.Figure()
309
310     s, units, state_label = _battery_state_display_series(base, state_col)
311     base["__soc__"] = s
312
313     if mode == "hour":
314         df = filter_by_season(base, season)
315         g = df.groupby(df.index.hour)["__soc__"].mean()
316         xs = list(range(24)); ys = [g.get(h, float("nan")) for h in xs]
317         _add_battery_soc_line(fig, xs, ys, state_label, mode)
318         fig.update_xaxes(title="Hour of Day", tickmode="linear", dtick=1)
319
320     elif mode == "day":
321         df = filter_by_season(base, season)
322         x, y, hover = _canon_day_series(df, "__soc__")
323         fig.add_trace(go.Scatter(x=x, y=y, mode="lines", name=state_label,
324                                 hovertext=hover,
325                                 hovertemplate="%{hovertext}<br>{%y:.2f} "
326                                                 + units
327                                                 + "<extra></extra>"))
328         fig.update_xaxes(title="Day (1..365)", tickmode="linear", tick0=1, dtick=10)
329
330     elif mode == "month":
331         g = base.groupby(base.index.month)["__soc__"].mean()
332         xs = list(range(1, 13)); ys = [g.get(m, float("nan")) for m in xs]
333         _add_battery_soc_line(fig, xs, ys, state_label, mode)
334         fig.update_xaxes(title="Month", tickmode="linear", dtick=1)
335
336     else: # mode cru
337         df = filter_by_season(base, season)
338         if df.empty:
339             return go.Figure()
340         x, y, hover = _raw_axis_data(df, "__soc__")
341         fig.add_trace(
342             go.Scatter(
343                 x=x,
344                 y=y,
345                 mode="lines",
346                 name=state_label,
347                 line=dict(color=STUDIO_CHART_COLORS["battery"], width=2.0),
348                 hovertext=hover,
349                 hovertemplate="%{hovertext}<br>{%y:.4f} " + units + "<extra></extra>",
350             )
351         )
352         fig.update_xaxes(title="Time (step)")
353
354     fig.update_yaxes(title=f"{state_label} ({units})")
355     fig.update_layout(title=f"{state_label} - {mode.capitalize()} ({'All' if mode=='month' else season})")
356     return fig
357
358 def _grid_source_unit_kind(column_name: str) -> str:
359     """Tipus d'unitat font de la xarxa."""
360     label = column_name.lower()
361     if "kwh" in label:
362         return "kwh"
363     if "energy" in label or "joule" in label:
364         return "joule"
365     if "rate" in label or "power" in label:
366         return "watt"
367     return "watt"
368
369
370 def make_battery_vs_grid_plot(obs: pd.DataFrame, mode: str, season: str) -> go.Figure:
371     """Crea una figura Plotly per a la bateria i la xarxa."""
372     base = _ensure_datetime_index(obs).copy()
373
374     discharge_power_col = _find_first_present_column(base.columns, _BATTERY_DISCHARGE_POWER_COLUMNS)
375     discharge_energy_col = _find_first_present_column(base.columns, _BATTERY_DISCHARGE_ENERGY_COLUMNS)
376     grid_col = _find_first_present_column(
377         base.columns,
378         _GRID_POWER_COLUMNS + _GRID_ENERGY_COLUMNS + _GRID_KWH_COLUMNS,
379     )
380
381     if discharge_power_col is None and discharge_energy_col is None and grid_col is None:
382         return go.Figure()
383
384     ts_hours = _infer_timestep_hours(base)
385
386     if discharge_power_col is not None:

```

```

387     bat_w = pd.to_numeric(base[discharge_power_col], errors="coerce").clip(lower=0.0)
388     base["__bat_kwh__"] = _energy_to_kwh(bat_w, "watt", ts_hours)
389 elif discharge_energy_col is not None:
390     base["__bat_kwh__"] = _energy_to_kwh(
391         pd.to_numeric(base[discharge_energy_col], errors="coerce").clip(lower=0.0),
392         "joule",
393     )
394 if grid_col is not None:
395     grid_kind = _grid_source_unit_kind(grid_col)
396     grid_values = pd.to_numeric(base[grid_col], errors="coerce").clip(lower=0.0)
397     base["__grid_kwh__"] = _energy_to_kwh(grid_values, grid_kind, ts_hours)
398
399 bat_color = STUDIO_CHART_COLORS["battery_charge"]
400 grid_color = "#eab308"
401 fig = go.Figure()
402
403 _add_battery_grid_bar_trace(fig, base, "__bat_kwh__", "Energia Bateria", bat_color, mode, season)
404 _add_battery_grid_bar_trace(fig, base, "__grid_kwh__", "Compra Xarxa", grid_color, mode, season)
405
406 if mode == "hour":
407     fig.update_xaxes(title="Hour of Day", tickmode="linear", dtick=1)
408     y_title = "Mean Hourly Consumption (kWh)"
409 elif mode == "day":
410     fig.update_xaxes(title="Day (1..365)", tickmode="linear", tick0=1, dtick=10)
411     y_title = "Daily Consumption (kWh)"
412 elif mode == "month":
413     fig.update_xaxes(title="Month", tickmode="linear", dtick=1)
414     y_title = "Monthly Consumption (kWh)"
415 else:
416     fig.update_xaxes(title="Time (step)")
417     y_title = "Consumption per Timestep (kWh)"
418
419 fig.update_layout(barmode="group")
420
421 fig.update_yaxes(title=y_title)
422 fig.update_layout(title=f"Energia Bateria vs Xarxa - {mode.capitalize()} ({season})")
423 return fig
424
425
426 # Gràfic de càrrega/descarrega de bateria amb preu d'energia.
427
428 def make_battery_charge_with_price_plot(obs: pd.DataFrame, mode: str, season: str) -> go.Figure:
429     """Crea una figura Plotly per a la càrrega de la bateria amb el preu."""
430     from plotly.subplots import make_subplots as _make_subplots
431     from backend.grafics.observation_columns import normalize_observation_columns
432
433     base = _ensure_datetime_index(normalize_observation_columns(obs)).copy()
434
435     # Detectem les columnes de bateria que existeixen en aquest CSV.
436     charge_col = _find_first_present_column(base.columns, _BATTERY_CHARGE_POWER_COLUMNS)
437     discharge_col = _find_first_present_column(base.columns, _BATTERY_DISCHARGE_POWER_COLUMNS)
438     state_col = _find_battery_state_column(base.columns)
439     has_bat = charge_col is not None or discharge_col is not None or state_col is not None
440
441     # El preu pot venir amb noms diferents segons el wrapper utilitzat.
442     price_col_raw = _find_energy_price_column(base.columns)
443     has_price = price_col_raw is not None
444
445     if not has_bat and not has_price:
446         return go.Figure()
447
448     # Si només tenim l'estat de càrrega, estimem càrrega/descarrega amb la diferència.
449     units = "W"
450     if has_bat:
451         if charge_col is None and discharge_col is None:
452             s = pd.to_numeric(base[state_col], errors="coerce")
453             d = s.diff().fillna(0.0)
454             d, _sfx = _auto_scale_series(d)
455             base["__charge__"] = d.clip(lower=0.0)
456             base["__discharge__"] = -d.clip(upper=0.0)
457             charge_col, discharge_col = "__charge__", "__discharge__"
458             units = f"Δstate/step {sfx}".strip() if _sfx else "Δstate/step"
459
460     # Normalitzem el preu a EUR/kWh perquè l'eix sigui comparable entre runs.
461     price_col = "__price__"
462     if has_price:
463         base[price_col] = _energy_price_series_eur_per_kwh(base, price_col_raw)
464         if base[price_col].dropna().empty:
465             has_price = False
466
467     # Posem la bateria a l'eix principal i el preu a l'eix secundari.
468     fig = _make_subplots(specs=[[{"secondary_y": True}]])
469
470     bat_charge_color = STUDIO_CHART_COLORS["battery_charge"]
471     bat_discharge_color = STUDIO_CHART_COLORS["battery_discharge"]
472     price_color = STUDIO_CHART_COLORS["price"]
473
474     marker_mode = "lines+markers" if mode in ("hour", "month") else "lines"
475
476     # Carrega (eix primari)
477     if has_bat:
478         cx, cy, chover = _aggregate_battery_series(base, charge_col, +1, mode, season)

```



```

479     if cx is not None:
480         ht = ("%{hovertext}<br>Carrega: %{y:.2f} " + units + "<extra></extra>") if chover else ("Carrega: %{y:.2f} " + units + "<extra></extra>")
481         fig.add_trace(
482             go.Scatter(
483                 x=cx, y=cy, mode=marker_mode, name=f"Carrega (+) [{units}]",
484                 line=dict(color=bat_charge_color, width=2.6),
485                 marker=dict(color=bat_charge_color, size=6),
486                 hovertext=chover,
487                 hovertemplate=ht,
488             ),
489             secondary_y=False,
490         )
491
492         # Descarrega (eix primari, valors negatius)
493         dx, dy, dhover = _aggregate_battery_series(base, discharge_col, -1, mode, season)
494         if dx is not None:
495             ht = ("%{hovertext}<br>Descarrega: %{y:.2f} " + units + "<extra></extra>") if dhover else ("Descarrega: %{y:.2f} " +
496 units + "<extra></extra>")
497             fig.add_trace(
498                 go.Scatter(
499                     x=dx, y=dy, mode=marker_mode, name=f"Descarrega (-) [{units}]",
500                     line=dict(color=bat_discharge_color, width=2.6),
501                     marker=dict(color=bat_discharge_color, size=6),
502                     hovertext=dhover,
503                     hovertemplate=ht,
504                     fill="tozero",
505                     fillcolor="rgba(234,88,12,0.10)", # battery_discharge amb alpha
506                 ),
507                 secondary_y=False,
508             )
509
510             fig.add_hline(y=0, line_dash="dash", line_color=STUDIO_CHART_COLORS["neutral_soft"],
511 line_width=1.2)
512
513         # Preu (eix secundari)
514         px_vals, py_vals, phover = _aggregate_price_series(base, price_col, has_price, mode, season)
515         if px_vals is not None:
516             ht = ("%{hovertext}<br>Preu: %{y:.4f} EUR/kWh<extra></extra>") if phover else ("Preu: %{y:.4f} EUR/kWh<extra></extra>")
517             fig.add_trace(
518                 go.Scatter(
519                     x=px_vals, y=py_vals, mode=marker_mode, name="Preu Energia (EUR/kWh)",
520                     line=dict(color=price_color, width=2.2),
521                     marker=dict(color=price_color, size=5),
522                     hovertext=phover,
523                     hovertemplate=ht,
524                 ),
525                 secondary_y=True,
526             )
527
528         # Ajustem els eixos segons l'agregació escollida.
529         x_titles = {"hour": "Hour of Day", "day": "Day (1..365)", "month": "Month", "raw": "Time (step)"}
530         x_dticks = {"hour": 1, "month": 1}
531         fig.update_xaxes(title_text=x_titles.get(mode, ""), tickmode="linear" if mode in x_dticks else None,
532 dtick=x_dticks.get(mode))
533         fig.update_yaxes(title_text=f"Potencia Bateria ({units})", secondary_y=False,
534 zeroline=True, zerolinecolor=STUDIO_CHART_COLORS["neutral_soft"])
535         fig.update_yaxes(title_text="Preu Energia (EUR/kWh)", secondary_y=True,
536 showgrid=False)
537
538         fig.update_layout(
539             title=f"Carrega/Descarrega Bateria vs Preu Energia - {mode.capitalize()} ({season})",
540             legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1),
541         )
542     return _apply_figure_theme(fig)

```

F.3 backend/grafics/column_utils.py

Listing 57: backend/grafics/column_utils.py

```

1  """Detecció de columnes disponibles per construir les figures del panell."""
2
3  from __future__ import annotations
4
5  import pandas as pd
6
7  def _ensure_air_temperature(df: pd.DataFrame) -> pd.DataFrame:
8      """Assegura 'air_temperature'; si no hi és, fa la mitjana de columnes *_air_temperature."""
9      dfc = df.copy()
10     if "air_temperature" not in dfc.columns:
11         zone_cols = [c for c in dfc.columns if c.endswith("_air_temperature")]
12         if zone_cols:
13             dfc["air_temperature"] = dfc[zone_cols].mean(axis=1)
14     return dfc
15
16
17 def _find_first_present_column(columns, candidates) -> str | None:
18     """Troba la primera columna disponible."""

```

```

19     available = set(columns)
20     for candidate in candidates:
21         if candidate in available:
22             return candidate
23
24     lower_lookup = {str(column).lower(): column for column in columns}
25     for candidate in candidates:
26         actual = lower_lookup.get(str(candidate).lower())
27         if actual is not None:
28             return actual
29     return None
30
31
32 _BATTERY_STATE_COLUMNS = (
33     "battery_charge_state",
34     "storage_charge_state",
35     "electric_storage_charge_state",
36     "storage_battery_charge_state",
37     "storage_battery_charge_fraction",
38     "battery_charge_fraction",
39     "battery_state_of_charge",
40     "storage_battery_soc",
41     "battery_soc",
42     "state_of_charge",
43 )
44
45 _BATTERY_CHARGE_POWER_COLUMNS = (
46     "storage_charge_power",
47     "battery_charge_power",
48     "battery_charge_rate",
49 )
50
51 _BATTERY_DISCHARGE_POWER_COLUMNS = (
52     "storage_discharge_power",
53     "battery_discharge_power",
54     "battery_discharge_rate",
55 )
56
57 _BATTERY_DISCHARGE_ENERGY_COLUMNS = (
58     "storage_discharge_energy",
59     "battery_discharge_energy",
60 )
61
62 _GRID_POWER_COLUMNS = (
63     "facility_net_purchased_electricity_rate",
64     "facility_total_purchased_electricity_rate",
65     "net_electricity_purchased",
66     "grid_electricity_rate",
67 )
68
69 _GRID_ENERGY_COLUMNS = (
70     "facility_net_purchased_electricity_energy",
71     "facility_total_purchased_electricity_energy",
72     "net_purchased_electricity_energy",
73     "total_purchased_electricity_energy",
74 )
75
76 _GRID_KWH_COLUMNS = (
77     "facility_net_purchased_electricity_kwh",
78     "facility_total_purchased_electricity_kwh",
79     "net_purchased_electricity_kwh",
80     "total_purchased_electricity_kwh",
81 )
82
83 _ENERGY_PRICE_COLUMNS = (
84     "energy_cost",
85     "electricity_cost",
86     "electricity_cost",
87     "price_eur_per_kwh",
88     "energy_price_kwh",
89     "energy_cost_eur_per_kwh",
90     "electricity_cost_eur_per_kwh",
91 )
92
93
94 def _find_battery_state_column(columns) -> str | None:
95     """Cerca la columna d'estat de la bateria."""
96     return _find_first_present_column(columns, _BATTERY_STATE_COLUMNS)
97
98
99 def _find_energy_price_column(columns) -> str | None:
100     """Cerqueu la columna del preu de l'energia."""
101     return _find_first_present_column(columns, _ENERGY_PRICE_COLUMNS)

```

F.4 backend/grafics/comfort.py

Listing 58: backend/grafics/comfort.py

```
1  """Compliment de la comoditat i xifres d'incompliment de la temperatura."""
2
3  from __future__ import annotations
4
5  import numpy as np
6  import pandas as pd
7  import plotly.express as px
8  import plotly.graph_objects as go
9
10 from backend.grafics.column_utils import _ensure_air_temperature
11 from backend.grafics.comfort_scope import comfort_scope_label, filter_by_comfort_scope
12 from backend.grafics.style import STUDIO_CHART_COLORS, _apply_figure_theme
13 from backend.grafics.time_utils import (
14     _canon_day_series,
15     _ensure_datetime_index,
16     _hour_series,
17     _month_series,
18     _raw_axis_data,
19     _season_months,
20     _seasonal_marker_colors,
21     filter_by_season,
22 )
23
24 DEFAULT_WINTER_COMFORT_RANGE = (20.0, 23.5)
25 DEFAULT_SUMMER_COMFORT_RANGE = (23.0, 26.0)
26 DEFAULT_SUMMER_START = (6, 1)
27 DEFAULT_SUMMER_FINAL = (9, 30)
28
29 def _extract_reward_kwargs(config) -> dict:
30     """Extreu kwargs de recompensa."""
31     if not isinstance(config, dict):
32         return {}
33     if any(
34         key in config
35         for key in (
36             "range_comfort_winter",
37             "range_comfort_summer",
38             "summer_start",
39             "summer_final",
40         )
41     ):
42         return config
43     reward_kwargs = config.get("reward_kwargs")
44     if isinstance(reward_kwargs, dict):
45         return reward_kwargs
46     training_config = config.get("training_config")
47     if isinstance(training_config, dict):
48         reward_kwargs = training_config.get("reward_kwargs")
49         if isinstance(reward_kwargs, dict):
50             return reward_kwargs
51     for value in config.values():
52         if isinstance(value, dict):
53             nested = _extract_reward_kwargs(value)
54             if nested:
55                 return nested
56     return {}
57
58
59 def _coerce_pair(value, default_pair):
60     """Converteix una parella de valors."""
61     if isinstance(value, (list, tuple)) and len(value) >= 2:
62         try:
63             return float(value[0]), float(value[1])
64         except (TypeError, ValueError):
65             pass
66     return default_pair
67
68
69 def _coerce_month_day(value, default_pair):
70     """Converteix una parella de mes i dia."""
71     if isinstance(value, (list, tuple)) and len(value) >= 2:
72         try:
73             return int(value[0]), int(value[1])
74         except (TypeError, ValueError):
75             pass
76     return default_pair
77
78
79 def _comfort_bounds_from_reward_kwargs(df: pd.DataFrame, comfort_config=None):
80     """Retorna els límits de comoditat de reward_kwargs. Els punts de consigna del termòstat són controls,
81     no són criteris de confort, de manera que aquí no s'utilitzen intencionadament.
82     """
83     reward_kwargs = _extract_reward_kwargs(comfort_config)
84     winter_lower, winter_upper = _coerce_pair(
85         reward_kwargs.get("range_comfort_winter"),
86         DEFAULT_WINTER_COMFORT_RANGE,
87     )
```

```

88     summer_lower, summer_upper = _coerce_pair(
89         reward_kwargs.get("range_comfort_summer"),
90         DEFAULT_SUMMER_COMFORT_RANGE,
91     )
92     summer_start = _coerce_month_day(
93         reward_kwargs.get("summer_start"),
94         DEFAULT_SUMMER_START,
95     )
96     summer_final = _coerce_month_day(
97         reward_kwargs.get("summer_final"),
98         DEFAULT_SUMMER_FINAL,
99     )
100
101     months = _month_series(df).astype(int)
102     if "day_of_month" in df.columns:
103         days = pd.to_numeric(df["day_of_month"], errors="coerce").fillna(1).astype(int)
104     elif isinstance(df.index, pd.DatetimeIndex):
105         days = pd.Series(df.index.day, index=df.index).astype(int)
106     else:
107         days = pd.Series(1, index=df.index)
108
109     current = months * 100 + days
110     start_code = summer_start[0] * 100 + summer_start[1]
111     final_code = summer_final[0] * 100 + summer_final[1]
112     if start_code <= final_code:
113         is_summer = (current >= start_code) & (current <= final_code)
114     else:
115         is_summer = (current >= start_code) | (current <= final_code)
116
117     lower = np.where(is_summer, summer_lower, winter_lower)
118     upper = np.where(is_summer, summer_upper, winter_upper)
119     return pd.Series(lower, index=df.index), pd.Series(upper, index=df.index)
120
121
122 def _categorize_comfort(df: pd.DataFrame, comfort_config=None) -> pd.DataFrame:
123     """
124     Afegeix:
125     - air_temperature (si cal)
126     - month/hour (si cal)
127     - comfort_cat: 'Massa fred' | 'En confort' | 'Massa calor'
128     """
129     base = _ensure_datetime_index(df).copy()
130     if "month" not in base.columns:
131         base["month"] = base.index.month
132     if "hour" not in base.columns:
133         base["hour"] = base.index.hour
134
135     base = _ensure_air_temperature(base) # crea 'air_temperature' si no existeix
136     lower, upper = _comfort_bounds_from_reward_kwargs(base, comfort_config)
137
138     if "air_temperature" not in base.columns:
139         if "temp_violation" in base.columns:
140             violation = pd.to_numeric(base["temp_violation"], errors="coerce")
141             valid = violation.notna()
142             in_comfort = valid & (violation <= 1e-9)
143             cat = pd.Series(pd.NA, index=base.index, dtype="object")
144             cat.loc[in_comfort] = "En confort"
145             cat.loc[valid & ~in_comfort] = "Massa calor"
146             base["comfort_cat"] = pd.Categorical(
147                 cat,
148                 categories=["Massa fred", "En confort", "Massa calor"],
149                 ordered=True,
150             )
151             return base
152         base["comfort_cat"] = pd.Categorical(
153             [np.nan] * len(base),
154             categories=["Massa fred", "En confort", "Massa calor"],
155             ordered=True,
156         )
157         return base
158
159     t_in = pd.to_numeric(base["air_temperature"], errors="coerce")
160     lower = pd.to_numeric(lower, errors="coerce")
161     upper = pd.to_numeric(upper, errors="coerce")
162     if "temp_violation" in base.columns:
163         violation = pd.to_numeric(base["temp_violation"], errors="coerce")
164     else:
165         violation = np.maximum(lower - t_in, 0) + np.maximum(t_in - upper, 0)
166         base["temp_violation"] = violation
167
168     valid = violation.notna()
169     in_comfort = valid & (violation <= 1e-9)
170     cold = valid & (~in_comfort) & (t_in < lower)
171     cat = pd.Series(pd.NA, index=base.index, dtype="object")
172     cat.loc[in_comfort] = "En confort"
173     cat.loc[cold] = "Massa fred"
174     cat.loc[valid & ~in_comfort & ~cold] = "Massa calor"
175     base["comfort_cat"] = pd.Categorical(
176         cat, categories=["Massa fred", "En confort", "Massa calor"], ordered=True
177     )
178     return base
179

```

```

180
181 def _ensure_temp_violation_column(df: pd.DataFrame, comfort_config=None) -> pd.DataFrame:
182     """Assegura una columna `temp_violation` per pas de temps.
183
184     Utilitza el valor del monitor quan existeix. En cas contrari, deriva la violació de
185     reward_kwards rangs de comoditat, tornant als rangs predeterminats de Sinergym.
186     Els punts de consigna del termostàt són controls i no s'utilitzen intencionadament aquí.
187     """
188     base = _ensure_datetime_index(df).copy()
189     if "month" not in base.columns:
190         base["month"] = base.index.month
191     if "hour" not in base.columns:
192         base["hour"] = base.index.hour
193     base = _ensure_air_temperature(base)
194
195     if "temp_violation" in base.columns:
196         base["temp_violation"] = pd.to_numeric(base["temp_violation"], errors="coerce")
197         return base
198
199     if "air_temperature" not in base.columns:
200         base["temp_violation"] = np.nan
201         return base
202
203     lower, upper = _comfort_bounds_from_reward_kwards(base, comfort_config)
204     t_in = pd.to_numeric(base["air_temperature"], errors="coerce")
205     lower = pd.to_numeric(lower, errors="coerce")
206     upper = pd.to_numeric(upper, errors="coerce")
207     base["temp_violation"] = np.maximum(lower - t_in, 0) + np.maximum(t_in - upper, 0)
208     return base
209
210
211 # Gràfic del percentatge d'hores en confort.
212 def make_comfort_compliance(
213     obs: pd.DataFrame,
214     mode: str,
215     season: str,
216     comfort_scope: str = "all",
217     comfort_config=None,
218 ) -> go.Figure:
219     """Crea un compliment còmode per al flux Studio."""
220     scoped_obs = filter_by_comfort_scope(obs, comfort_scope)
221     scope_label = comfort_scope_label(comfort_scope)
222     df = _categorize_comfort(scoped_obs, comfort_config)
223
224     # Només apliquem filtre 'destació quan té sentit
225     if mode in ("raw", "hour", "day"):
226         df = filter_by_season(df, season)
227
228     categories = ["Massa fred", "En confort", "Massa calor"]
229     colors_map = {
230         "Massa fred": STUDIO_CHART_COLORS["comfort_cold"],
231         "En confort": STUDIO_CHART_COLORS["comfort_ok"],
232         "Massa calor": STUDIO_CHART_COLORS["comfort_hot"],
233     }
234
235     if df.empty:
236         return go.Figure()
237
238     if mode == "raw":
239         counts = df["comfort_cat"].value_counts().reindex(categories).fillna(0)
240         fig = px.pie(values=counts.values, names=counts.index, hole=0.45,
241             title=f"% d'hores en confort - Donut ({season} | {scope_label})",
242             color=counts.index, color_discrete_map=colors_map)
243         fig.update_traces(textinfo="percent+label", marker=dict(line=dict(color="#ffffff", width=2)))
244         fig.update_layout(legend_title_text="")
245         return _apply_figure_theme(fig)
246
247     # Agreguem en barres apilades per veure el repartiment de confort.
248     if mode == "hour":
249         idx = df["hour"].astype(int); x_title="Hora del dia"; x_order=list(range(24))
250     elif mode == "day":
251         df = df.copy(); df["__day__"] = df.index.floor("D").astype(str)
252         idx = df["__day__"]; x_title="Dia"; x_order=None
253     elif mode == "month":
254         # IMPORTANT: en month NO fem filtre per estació
255         df = _categorize_comfort(scoped_obs, comfort_config).copy()
256         df["__month__"] = df.index.month
257         idx = df["__month__"].astype(int); x_title="Mes"; x_order=list(range(1,13))
258     else:
259         return make_comfort_compliance(
260             scoped_obs,
261             "raw",
262             season,
263             comfort_scope=comfort_scope,
264             comfort_config=comfort_config,
265         )
266
267     tbl = pd.crosstab(idx, df["comfort_cat"])
268     tbl = (tbl.div(tbl.sum(axis=1).replace(0, np.nan), axis=0) * 100).fillna(0.0)
269     tbl = tbl.reindex(columns=categories, fill_value=0.0)
270     if x_order is not None:
271         tbl = tbl.reindex(index=x_order, fill_value=0.0)

```

```

272 fig = go.Figure()
273 for cat in categories:
274     fig.add_bar(x=tbl.index, y=tbl[cat].values, name=cat, marker_color=colors_map[cat])
275
276 fig.update_layout(
277     barmode="stack",
278     title=f"% 'dhores en confort - {mode.capitalize()} ({'All' if mode=='month' else season} | {scope_label})",
279     legend_title_text="",
280 )
281
282 fig.update_yaxes(title="% 'dhores", range=[0, 100])
283 fig.update_xaxes(title=x_title, tickmode="linear", dtick=1 if mode in ("hour", "month") else None)
284 return _apply_figure_theme(fig)
285
286
287 # Gràfic de violació de confort en barres.
288 def make_violation_bars(
289     obs: pd.DataFrame,
290     mode: str,
291     season: str,
292     comfort_scope: str = "all",
293     comfort_config=None,
294 ) -> go.Figure:
295     """Crea barres d'infracció per al flux Studio."""
296     scoped_obs = filter_by_comfort_scope(obs, comfort_scope)
297     scope_label = comfort_scope_label(comfort_scope)
298     base = _ensure_temp_violation_column(scoped_obs, comfort_config)
299
300     fig = go.Figure()
301
302     if mode == "hour":
303         df = filter_by_season(base, season)
304         g = df.groupby(df.index.hour)["temp_violation"].mean()
305         x_vals = list(range(24))
306         y_vals = [g.get(h, float("nan")) for h in x_vals]
307         fig.add_bar(
308             x=x_vals, y=y_vals, name="Mitjana horària",
309             marker=dict(color=STUDIO_CHART_COLORS["violation"], line=dict(color=STUDIO_CHART_COLORS["violation_edge"], width=0.8)),
310             meta="keep_color"
311         )
312         fig.update_xaxes(title="Hora del dia", tickmode="linear", dtick=1)
313
314     elif mode == "day":
315         df = filter_by_season(base, season)
316         g = df.groupby(df.index.floor("D"))["temp_violation"].mean()
317         fig.add_scatter(
318             x=g.index.astype(str), y=g.values, name="Mitjana diària",
319             mode="lines",
320             line=dict(color=STUDIO_CHART_COLORS["violation"], width=2),
321             meta="keep_color"
322         )
323         fig.update_xaxes(title="Dia")
324
325     elif mode == "month":
326         df = base.copy() # ressaltem amb estació via colors, no filtrem
327         g = df.groupby(df.index.month)["temp_violation"].mean()
328         x_vals = list(range(1, 12 + 1))
329         y_vals = [g.get(m, float("nan")) for m in x_vals]
330         months_sel = _season_months(season)
331         c_viol = STUDIO_CHART_COLORS["violation"]
332         marker_colors = _seasonal_marker_colors(x_vals, season, active_color=c_viol)
333         fig.add_bar(
334             x=x_vals, y=y_vals, name="Mitjana mensual",
335             marker=dict(color=marker_colors, line=dict(color=STUDIO_CHART_COLORS["violation_edge"], width=0.8)),
336             meta="keep_color"
337         )
338         fig.update_xaxes(title="Mes", tickmode="linear", dtick=1)
339
340     elif mode == "raw":
341         df = filter_by_season(base, season)
342         fig.add_bar(
343             x=np.arange(len(df)), y=df["temp_violation"].values, name="Registres",
344             marker=dict(color=STUDIO_CHART_COLORS["violation"], line=dict(color=STUDIO_CHART_COLORS["violation_edge"], width=0.8)),
345             meta="keep_color"
346         )
347         fig.update_xaxes(title="Time (step)")
348
349     fig.update_yaxes(title="Temperature Violation (°C)")
350     fig.update_layout(
351         title=f"Temperature Violation (Bars) - {mode.capitalize()} ({season} | {scope_label})"
352     )
353     return fig
354
355
356 def _fix_colors_for_visibility(fig, *, kind: str, mode: str):
357     """
358     Aplica colors/contorns més marcats a barres en 'daily' i 'raw'.
359     kind: 'violation' | 'hvac' | qualsevol altre (ignora).
360     """
361     if mode not in ("day", "raw"):
362         return fig
363 
```

```

364 try:
365     for tr in getattro(fig, "data", []):
366         if getattro(tr, "type", "") != "bar":
367             continue
368         default_color = "#1f2937" if kind == "violation" else "#264653"
369         color = getattro(getattro(tr, "marker", None), "color", None)
370         if color is None or (isinstance(color, str) and color.lower() in {
371             "lightgray", "#d3d3d3", "rgb(211,211,211)", "rgba(211,211,211,1.0)"
372         }):
373             tr.marker.color = default_color
374             tr.marker.line = dict(color="rgba(0,0,0,0.25)", width=0.4)
375             tr.opacity = 0.95
376 except Exception:
377     pass
378 return fig
379
380
381 # Línia única de violació de temperatura amb els filtres actius.
382 def make_violation_single_line(
383     obs: pd.DataFrame,
384     mode: str,
385     season: str,
386     comfort_config=None,
387 ) -> go.Figure:
388     """Crea una línia única d'infracció per al flux Studio."""
389     base = _ensure_temp_violation_column(obs, comfort_config)
390
391     fig = go.Figure()
392     violation_color = STUDIO_CHART_COLORS["violation"]
393
394     if mode == "hour":
395         # En hora mostrem la violació mitjana del dia típic; ajuda a veure franges
396         # problemàtiques de confort.
397         df = filter_by_season(base, season)
398         h = _hour_series(df)
399         g = df.groupby(h)["temp_violation"].mean()
400         x_vals = list(range(24))
401         y_vals = [g.get(hh, float("nan")) for hh in x_vals]
402         fig.add_trace(go.Scatter(
403             x=x_vals,
404             y=y_vals,
405             mode="lines+markers",
406             name="Hourly Mean",
407             line=dict(color=violation_color, width=2.6),
408             marker=dict(color=violation_color, size=6),
409             meta="keep_color",
410         ))
411         fig.update_xaxes(title="Hour of Day", tickmode="linear", dtick=1)
412
413     elif mode == "day":
414         df = filter_by_season(base, season)
415         x_vals, y_vals, hover = _canon_day_series(df, "temp_violation")
416         fig.add_trace(go.Scatter(x=x_vals, y=y_vals, mode="lines+markers", name="Daily Mean",
417             line=dict(color=violation_color, width=2.6),
418             marker=dict(color=violation_color, size=6),
419             meta="keep_color",
420             hovertext=hover, hovertemplate="%{hovertext}<br>{%y:.2f}°C<extra></extra>"))
421         fig.update_xaxes(title="Day (1..365)", tickmode="linear", tick0=1, dtick=10)
422
423     elif mode == "month":
424         df = base if season == "All" else filter_by_season(base, season)
425         m = _month_series(df)
426         g_full = df.groupby(m)["temp_violation"].mean()
427         x_vals = list(range(1, 13))
428         y_vals = [g_full.get(mm, float("nan")) for mm in x_vals]
429         months_sel = _season_months(season)
430         marker_colors = [(violation_color if months_sel and mth in months_sel else "#7f7f7f") for mth in x_vals]
431         fig.add_trace(go.Scatter(x=x_vals, y=y_vals, mode="lines+markers", name="Monthly Mean",
432             line=dict(color=violation_color, width=2.6),
433             marker=dict(color=marker_colors, size=6),
434             meta="keep_color"))
435         fig.update_xaxes(title="Month", tickmode="linear", dtick=1)
436
437     elif mode == "raw":
438         # En raw dibuixem primer tot l'any i, si hi ha filtre de temporada, el ressaltem
439         # a sobre per no perdre el context.
440         df_all = filter_by_season(base, "All")
441         x_all, y_all, hover_all = _raw_axis_data(df_all, "temp_violation")
442         fig.add_trace(go.Scatter(x=x_all, y=y_all, mode="lines", name="All Records",
443             line=dict(color=violation_color, width=1.8), meta="keep_color", opacity=0.92,
444             hovertext=hover_all, hovertemplate="%{hovertext}<br>{%y:.2f}°C<extra></extra>"))
445
446         if season != "All":
447             df_season = filter_by_season(base, season)
448             if not df_season.empty:
449                 x_s, y_s, hover_s = _raw_axis_data(df_season, "temp_violation")
450                 fig.add_trace(go.Scatter(x=x_s, y=y_s, mode="lines",
451                     name=f"{season} Records", line=dict(color=violation_color, width=2.4), meta="keep_color",
452                     hovertext=hover_s, hovertemplate="%{hovertext}<br>{%y:.2f}°C<extra></extra>"))
453             fig.update_xaxes(title="Time (step)")
454
455     fig.update_yaxes(title="Temperature Violation (°C)")
456     fig.update_layout(title=f"Temperature Violation - {mode.capitalize()} ({season})")

```

```
456 return fig
```

F.5 backend/grafics/comfort_scope.py

Listing 59: backend/grafics/comfort_scope.py

```
1 """Filtres d'abast de confort compartits per KPI, gràfics i informes.
2
3 Aquest fitxer decideix si una mètrica de confort s'ha de calcular sobre totes les hores o només sobre
4 les hores amb ocupació, i manté el mateix criteri al dashboard i als informes descarregables.
5 """
6
7 from __future__ import annotations
8
9 from typing import Optional
10
11 import pandas as pd
12
13
14 def derive_occupied_mask(obs: pd.DataFrame) -> Optional[pd.Series]:
15     """Inferir una màscara booleana per als registres ocupats quan les dades ho proporcionen."""
16     if obs is None or obs.empty:
17         return None
18
19     explicit_cols = (
20         "occupied_hour",
21         "people_occupant",
22         "people_occupants",
23         "occupancy",
24         "occupants",
25         "is_occupied",
26     )
27     for col in explicit_cols:
28         if col in obs.columns:
29             values = pd.to_numeric(obs[col], errors="coerce").fillna(0.0)
30             return pd.Series(values > 0.0, index=obs.index)
31
32     fuzzy_cols = [
33         c for c in obs.columns
34         if any(token in c.lower() for token in ("occupant", "occupancy", "people"))
35     ]
36     if fuzzy_cols:
37         values = (
38             obs[fuzzy_cols]
39             .apply(pd.to_numeric, errors="coerce")
40             .fillna(0.0)
41             .sum(axis=1)
42         )
43         return pd.Series(values > 0.0, index=obs.index)
44
45     return None
46
47
48 def has_occupied_data(obs: pd.DataFrame) -> bool:
49     """Retorna si les observacions contenen prou dades d'ocupació per filtrar per àmbit."""
50     return derive_occupied_mask(obs) is not None
51
52
53 def filter_by_comfort_scope(obs: pd.DataFrame, comfort_scope: str) -> pd.DataFrame:
54     """Retorna totes les files o només les files ocupades en funció de l'àmbit seleccionat."""
55     if comfort_scope != "occupied":
56         return obs.copy()
57
58     mask = derive_occupied_mask(obs)
59     if mask is None:
60         return obs.copy()
61     return obs.loc[mask].copy()
62
63
64 def comfort_scope_label(comfort_scope: str) -> str:
65     """Retorna l'etiqueta de visualització associada a un identificador d'abast de confort."""
66     return "Occupied Hours" if comfort_scope == "occupied" else "All Hours"
```

F.6 backend/grafics/control.py

Listing 60: backend/grafics/control.py

```
1 """Figures de control i acciÃ³ per a HVAC i pistes de terra radiant."""
2
3 from __future__ import annotations
4
5 import pandas as pd
6 import plotly.graph_objects as go
7 from plotly.subplots import make_subplots
```



```

8
9 from backend.grafics.style import pick_semantic_trace_color
10 from backend.grafics.time_utils import (
11     _canon_day_series,
12     _ensure_datetime_index,
13     _hour_series,
14     _month_series,
15     _raw_axis_data,
16     filter_by_season,
17 )
18
19
20 def _mean_series(base: pd.DataFrame, columns: list[str]) -> pd.Series:
21     """Retorna la mitjana num rica per files per a les columnes seleccionades."""
22     numeric = base[columns].apply(pd.to_numeric, errors="coerce")
23     return numeric.mean(axis=1)
24
25
26 def _radiant_trace_data(
27     base: pd.DataFrame,
28     series_name: str,
29     mode: str,
30     season: str,
31 ) -> tuple[list, list, list | None, str] | None:
32     """Retorna les dades x/y/hover/mode per a una s rie de control radiant."""
33     if series_name not in base.columns:
34         return None
35     if mode == "hour":
36         df = filter_by_season(base, season)
37         hour_values = _hour_series(df)
38         grouped = df.groupby(hour_values)[series_name].mean()
39         x_values = list(range(24))
40         y_values = [grouped.get(hour, float("nan")) for hour in x_values]
41         return x_values, y_values, None, "lines+markers"
42     if mode == "day":
43         df = filter_by_season(base, season)
44         x_values, y_values, hover = _canon_day_series(df, series_name)
45         return x_values, y_values, hover, "lines"
46     if mode == "month":
47         df = base if season == "All" else filter_by_season(base, season)
48         month_values = _month_series(df)
49         grouped = df.groupby(month_values)[series_name].mean()
50         x_values = list(range(1, 13))
51         y_values = [grouped.get(month, float("nan")) for month in x_values]
52         return x_values, y_values, None, "lines+markers"
53     if mode == "raw":
54         df = filter_by_season(base, season)
55         x_values, y_values, hover = _raw_axis_data(df, series_name)
56         return x_values, y_values, hover, "lines"
57     return None
58
59
60 def _add_radiant_control_trace(
61     fig: go.Figure,
62     base: pd.DataFrame,
63     series_name: str,
64     label: str,
65     secondary_y: bool,
66     suffix: str,
67     mode: str,
68     season: str,
69 ) -> None:
70     """Afegeix un tra  de control radiant a la figura."""
71     data = _radiant_trace_data(base, series_name, mode, season)
72     if data is None:
73         return
74     x_values, y_values, hover, trace_mode = data
75     color = pick_semantic_trace_color(label)
76     scatter_kwargs = dict(
77         x=x_values,
78         y=y_values,
79         mode=trace_mode,
80         name=label,
81         line=dict(color=color, width=2.8, shape="hv" if mode in ("hour", "raw") else "linear"),
82     )
83     if "markers" in trace_mode:
84         scatter_kwargs["marker"] = dict(color=color, size=6)
85     if hover is not None:
86         scatter_kwargs["hovertext"] = hover
87         scatter_kwargs["hovertemplate"] = f"%{{hovertext}}<br>{{y:.2f}} {{suffix}}<extra>/<extra>"
88     else:
89         scatter_kwargs["hovertemplate"] = f"%{{x}}<br>{{y:.2f}} {{suffix}}<extra>/<extra>"
90     fig.add_trace(go.Scatter(**scatter_kwargs), secondary_y=secondary_y)
91
92
93 def _pretty_action_name(column_name: str) -> str:
94     """Retorna una etiqueta de visualitzaci  per a una columna d'acci s."""
95     return column_name.replace("_", " ").replace("radiant ", "radiant: ").title()
96
97
98 def _agent_action_series_data(
99     base: pd.DataFrame,

```

```

100     series_name: str,
101     mode: str,
102     season: str,
103 ) -> tuple[list, list, list | None, str] | None:
104     """Retorna les dades x/y/hover/mode per a una s rie d'acci  d'agent."""
105     if mode == "hour":
106         df = filter_by_season(base, season)
107         if df.empty:
108             return None
109         hour_values = _hour_series(df)
110         grouped = df.groupby(hour_values)[series_name].mean()
111         x_values = list(range(24))
112         y_values = [grouped.get(hour, float("nan")) for hour in x_values]
113         return x_values, y_values, None, "lines+markers"
114     if mode == "day":
115         df = filter_by_season(base, season)
116         if df.empty:
117             return None
118         x_values, y_values, hover = _canon_day_series(df, series_name)
119         return x_values, y_values, hover, "lines"
120     if mode == "month":
121         df = base if season == "All" else filter_by_season(base, season)
122         if df.empty:
123             return None
124         month_values = _month_series(df)
125         grouped = df.groupby(month_values)[series_name].mean()
126         x_values = list(range(1, 13))
127         y_values = [grouped.get(month, float("nan")) for month in x_values]
128         return x_values, y_values, None, "lines+markers"
129     if mode == "raw":
130         df = filter_by_season(base, season)
131         if df.empty:
132             return None
133         x_values, y_values, hover = _raw_axis_data(df, series_name)
134         return x_values, y_values, hover, "lines"
135     return None
136
137
138 def _add_agent_action_trace(
139     fig: go.Figure,
140     base: pd.DataFrame,
141     column: str,
142     row: int,
143     color: str,
144     temp_columns: list[str],
145     mode: str,
146     season: str,
147 ) -> None:
148     """Afegeix un rastre d'acci  a la figura d'acci  de diverses files."""
149     data = _agent_action_series_data(base, column, mode, season)
150     if data is None:
151         return
152     x_values, y_values, hover, trace_mode = data
153     is_temp = column in temp_columns
154     scatter_kwargs = dict(
155         x=x_values,
156         y=y_values,
157         mode=trace_mode,
158         name=_pretty_action_name(column),
159         legendgroup="agent_actions",
160         showlegend=False,
161         line=dict(color=color, width=2.5, shape="hv" if mode in ("hour", "raw") else "linear"),
162     )
163     if "markers" in trace_mode:
164         scatter_kwargs["marker"] = dict(color=color, size=6)
165     suffix = " C" if is_temp else ""
166     if hover is not None:
167         scatter_kwargs["hovertext"] = hover
168         scatter_kwargs["hovertemplate"] = f"%{{hovertext}}<br>%{{y:.2f}}{suffix}<extra></extra>"
169     else:
170         scatter_kwargs["hovertemplate"] = f"%{{x}}<br>%{{y:.2f}}{suffix}<extra></extra>"
171     fig.add_trace(go.Scatter(**scatter_kwargs), row=row, col=1)
172
173
174 def make_radiant_control_plot(obs: pd.DataFrame, mode: str, season: str) -> go.Figure:
175     """Crea una figura Plotly per al control radiant."""
176     base = _ensure_datetime_index(obs).copy()
177
178     # Els noms dels sensors radiants varien molt entre models. Fem una detecc  flexible
179     # i despr s els redu m a tres s ries llegibles: disponibilitat, entrada i sortida.
180     lower_cols = {col: col.lower() for col in base.columns}
181     availability_cols = [
182         col
183         for col, label in lower_cols.items()
184         if "radiant" in label
185         and ("availavility" in label or "availability" in label or "operation_mode" in label or "operation mode" in label)
186     ]
187     inlet_cols = [
188         col
189         for col, label in lower_cols.items()
190         if "radiant" in label and "inlet" in label and "temperature" in label
191     ]

```

```

192 outlet_cols = [
193     col
194     for col, label in lower_cols.items()
195     if "radiant" in label and "outlet" in label and "temperature" in label
196 ]
197
198 if not availability_cols and not inlet_cols and not outlet_cols:
199     return go.Figure()
200
201 if availability_cols:
202     # Si hi ha més d'una zona radiant, la mitjana dona una lectura global del mode actiu.
203     base["__radiant_availability_pct__"] = _mean_series(base, availability_cols) * 100.0
204 if inlet_cols:
205     base["__radiant_inlet_temp__"] = _mean_series(base, inlet_cols)
206 if outlet_cols:
207     base["__radiant_outlet_temp__"] = _mean_series(base, outlet_cols)
208
209 fig = make_subplots(specs=[[{"secondary_y": True}]]))
210
211 _add_radiant_control_trace(fig, base, "__radiant_outlet_temp__", "Radiant Outlet Temp", False, "C", mode, season)
212 _add_radiant_control_trace(fig, base, "__radiant_inlet_temp__", "Radiant Inlet Temp", False, "C", mode, season)
213 _add_radiant_control_trace(fig, base, "__radiant_availability_pct__", "Radiant Operation Mode", True, "%", mode, season)
214
215 if mode == "hour":
216     fig.update_xaxes(title="Hour of Day", tickmode="linear", dtick=1)
217 elif mode == "month":
218     fig.update_xaxes(title="Month", tickmode="linear", dtick=1)
219 elif mode == "day":
220     fig.update_xaxes(title="Day (1..365)", tickmode="linear", tick0=1, dtick=10)
221 else:
222     fig.update_xaxes(title="Time (step)")
223
224 fig.update_yaxes(title="Temperature (C)", secondary_y=False)
225 fig.update_yaxes(title="Operation Mode (%)", range=[-5, 105], secondary_y=True)
226 fig.update_layout(title=f"Radiant Control - {mode.capitalize()} ({season})")
227 return fig
228
229
230 def make_agent_actions_plot(actions: pd.DataFrame, mode: str, season: str) -> go.Figure:
231     """Crea una figura Plotly per a les accions de l'agent."""
232     if actions is None or actions.empty:
233         return go.Figure()
234
235     time_columns = {"datetime", "timestamp", "month", "day_of_month", "hour", "time_elapsed(hours)"}
236     base = _ensure_datetime_index(actions).copy().sort_index(kind="mergesort")
237     action_columns = [col for col in base.columns if col not in time_columns]
238     if not action_columns:
239         return go.Figure()
240
241     for col in action_columns:
242         base[col] = pd.to_numeric(base[col], errors="coerce")
243
244     temp_cols = [
245         col
246         for col in action_columns
247         if "temperature" in col.lower() or "temp" in col.lower()
248     ]
249     binary_cols = [col for col in action_columns if col not in temp_cols]
250     ordered_cols = temp_cols + binary_cols
251     if not ordered_cols:
252         return go.Figure()
253
254     for col in temp_cols:
255         valid = base[col].dropna()
256         if not valid.empty and valid.min() >= 0 and valid.max() <= 20:
257             # Alguns wrappers guarden deltes o consignes normalitzades; si totes cauen en
258             # 0..20 les desplacem per representar-les com a consignes de temperatura llegibles.
259             base[col] = base[col] + 25.0
260
261     rows = len(ordered_cols)
262     # Les accions de temperatura necessiten més alçada que les binàries, on només volem
263     # veure si l'actuador està actiu o no.
264     row_heights = [1.7 if col in temp_cols else 0.55 for col in ordered_cols]
265     fig = make_subplots(
266         rows=rows,
267         cols=1,
268         shared_xaxes=True,
269         vertical_spacing=0.018 if rows > 1 else 0.0,
270         row_heights=row_heights,
271         subplot_titles=[_pretty_action_name(col) for col in ordered_cols],
272     )
273
274     palette = ["#0f766e", "#7c3aed", "#2563eb", "#ea580c", "#16a34a", "#dc2626", "#0891b2", "#ca8a04"]
275     for row, col in enumerate(ordered_cols, start=1):
276         _add_agent_action_trace(fig, base, col, row, palette[(row - 1) % len(palette)], temp_cols, mode, season)
277
278     x_axis_title = "Time (step)"
279     x_axis_kwargs = {}
280     if mode == "hour":
281         x_axis_title = "Hour of Day"
282         x_axis_kwargs = {"tickmode": "linear", "dtick": 1}
283     elif mode == "month":

```

```

284     x_axis_title = "Month"
285     x_axis_kwargs = {"tickmode": "linear", "dtick": 1}
286 elif mode == "day":
287     x_axis_title = "Day (1..365)"
288     x_axis_kwargs = {"tickmode": "linear", "tick0": 1, "dtick": 10}
289
290 for row, col in enumerate(ordered_cols, start=1):
291     fig.update_xaxes(**x_axis_kwargs, row=row, col=1)
292     if col in temp_cols:
293         fig.update_yaxes(title="C", row=row, col=1)
294     else:
295         if mode == "raw":
296             fig.update_yaxes(
297                 title="ON/OFF",
298                 range=[-0.08, 1.08],
299                 tickmode="array",
300                 tickvals=[0, 1],
301                 ticktext=["OFF", "ON"],
302                 row=row,
303                 col=1,
304             )
305         else:
306             fig.update_yaxes(
307                 title="ON fraction",
308                 range=[-0.05, 1.05],
309                 tickmode="array",
310                 tickvals=[0, 0.5, 1],
311                 row=row,
312                 col=1,
313             )
314 fig.update_xaxes(title=x_axis_title, row=rows, col=1)
315 figure_height = 260 + 120 * len(temp_cols) + 58 * len(binary_cols)
316 fig.update_layout(
317     title=f"Agent Actions - {mode.capitalize()} ({season})",
318     height=max(360, figure_height),
319     showlegend=False,
320 )
321 return fig

```

F.7 backend/grafics/data_loader.py

Listing 61: backend/grafics/data_loader.py

```

1  """Carrega i normalitza els fitxers CSV de resultats per a panells i informes.
2
3  La pàgina de resultats i el backend de l'informe passen per aquí per llegir el progrés de Sinergym,
4  arreglar buits de mètriques habituals, estandarditzar noms de columnes i retornar DataFrames preparats
5  per generar KPIs i figures Plotly.
6  """
7
8  from functools import lru_cache
9  from pathlib import Path
10
11  import pandas as pd
12
13  from backend.grafics.observation_columns import (
14      add_meter_kwh_columns,
15      normalize_observation_columns,
16  )
17  from backend.grafics.time_utils import sanitize_observation_time_columns
18
19  _DEFAULT_PROGRESS_HEADERS = [
20      "episode_num",
21      "mean_reward",
22      "std_reward",
23      "mean_reward_comfort_term",
24      "std_reward_comfort_term",
25      "mean_reward_energy_term",
26      "std_reward_energy_term",
27      "mean_comfort_penalty",
28      "std_comfort_penalty",
29      "mean_energy_penalty",
30      "std_energy_penalty",
31      "mean_temperature_violation",
32      "std_temperature_violation",
33      "mean_power_demand",
34      "std_power_demand",
35      "cumulative_power_demand",
36      "comfort_violation_time(%)",
37      "length(timesteps)",
38      "time_elapsed(hours)",
39      "terminated",
40      "truncated",
41  ]
42
43  _NUMERIC_PROGRESS_COLUMNS = tuple(
44      column
45      for column in _DEFAULT_PROGRESS_HEADERS

```

```

46     if column not in {"terminated", "truncated"}
47 )
48
49
50 def _csv_signature(path: str | Path) -> tuple[str, int, int]:
51     """Retorna una clau de memòria cau que canvia quan canvia el fitxer CSV."""
52
53     csv_path = Path(path)
54     stat = csv_path.stat()
55     return str(csv_path), int(stat.st_mtime_ns), int(stat.st_size)
56
57
58 @lru_cache(maxsize=32)
59 def _read_csv_cached(
60     path: str,
61     mtime_ns: int,
62     size: int,
63     mode: str,
64     names: tuple[str, ...],
65 ) -> pd.DataFrame:
66     """Llegeix un fitxer CSV mitjançant una memòria cau en procés conscient de mtime."""
67
68     _ = (mtime_ns, size)
69     if mode == "progress_no_header":
70         return pd.read_csv(path, header=None, names=list(names))
71     if mode == "low_memory_false":
72         return pd.read_csv(path, low_memory=False)
73     return pd.read_csv(path)
74
75
76 def _read_csv(path: str | Path, *, mode: str = "default", names: tuple[str, ...] = ()) -> pd.DataFrame:
77     """Llegeix un fitxer CSV i torneu-ne una còpia perquè les crides que la rebin la puguin mutar amb seguretat."""
78
79     csv_path, mtime_ns, size = _csv_signature(path)
80     return _read_csv_cached(csv_path, mtime_ns, size, mode, names).copy()
81
82
83 def _fill_missing_or_zero(target: pd.Series, replacement: pd.Series) -> pd.Series:
84     """Ompliu els valors objectiu que falten o zero amb una sèrie de substitució numèrica."""
85
86     target = pd.to_numeric(target, errors="coerce")
87     replacement = pd.to_numeric(replacement, errors="coerce")
88     replace_mask = target.isna() | (
89         (target == 0) & replacement.notna() & (replacement != 0)
90     )
91     result = target.copy()
92     result.loc[replace_mask] = replacement.loc[replace_mask]
93     return result
94
95
96 def _repair_power_metrics_from_progress(progress: pd.DataFrame) -> pd.DataFrame:
97     """Repareu les mètriques de potència de progrés quan només hi hagi valors acumulatius o mitjans."""
98
99     repaired = progress.copy()
100
101     if (
102         "cumulative_power_demand" in repaired.columns
103         and "length(timesteps)" in repaired.columns
104     ):
105         derived_mean_from_cumulative = (
106             pd.to_numeric(repaired["cumulative_power_demand"], errors="coerce")
107             / pd.to_numeric(repaired["length(timesteps)"], errors="coerce").replace(0, pd.NA)
108         )
109         if "mean_power_demand" in repaired.columns:
110             repaired["mean_power_demand"] = _fill_missing_or_zero(
111                 repaired["mean_power_demand"], derived_mean_from_cumulative
112             )
113         else:
114             repaired["mean_power_demand"] = derived_mean_from_cumulative
115
116     if (
117         "mean_power_demand" in repaired.columns
118         and "length(timesteps)" in repaired.columns
119     ):
120         derived_cumulative = (
121             pd.to_numeric(repaired["mean_power_demand"], errors="coerce")
122             * pd.to_numeric(repaired["length(timesteps)"], errors="coerce")
123         )
124         if "cumulative_power_demand" in repaired.columns:
125             repaired["cumulative_power_demand"] = _fill_missing_or_zero(
126                 repaired["cumulative_power_demand"], derived_cumulative
127             )
128         else:
129             repaired["cumulative_power_demand"] = derived_cumulative
130
131     return repaired
132
133
134 def _drop_repeated_header_rows(data: pd.DataFrame) -> pd.DataFrame:
135     """Elimineu les files de capçalera CSV repetides accidentalment dels fitxers de registre adjunts."""
136
137     if data.empty:

```

```

138     return data
139
140     header_mask = pd.Series(False, index=data.index)
141     for column in data.columns:
142         header_mask |= data[column].astype(str).str.strip().eq(str(column))
143
144     if not header_mask.any():
145         return data
146
147     return data.loc[~header_mask].reset_index(drop=True)
148
149
150 def _coerce_progress_numeric_columns(progress: pd.DataFrame) -> pd.DataFrame:
151     """Converteix les mètriques de progrés conegudes en tipus d numèrics després de carregar CSV."""
152
153     coerced = progress.copy()
154     for column in _NUMERIC_PROGRESS_COLUMNS:
155         if column in coerced.columns:
156             coerced[column] = pd.to_numeric(coerced[column], errors="coerce")
157     return coerced
158
159
160 def load_data(progress_path, obs_path):
161     """Carrega els fitxers de progrés i observació CSV utilitzats pel panell de resultats."""
162
163     progress = _read_csv(progress_path)
164
165     # Comprova si el progress.csv té capçaleres que falten (comú quan s'afegeixen registres)
166     if "mean_reward" not in progress.columns and "episode_num" not in progress.columns:
167         if len(progress.columns) == len(_DEFAULT_PROGRESS_HEADERS):
168             progress = _read_csv(
169                 progress_path,
170                 mode="progress_no_header",
171                 names=tuple(_DEFAULT_PROGRESS_HEADERS),
172             )
173
174     progress = _drop_repeated_header_rows(progress)
175     progress = _coerce_progress_numeric_columns(progress)
176     progress = _repair_power_metrics_from_progress(progress)
177
178     obs = _read_csv(obs_path)
179
180     infos_path = Path(obs_path).with_name("infos.csv")
181     if infos_path.exists():
182         try:
183             infos = _read_csv(infos_path, mode="low_memory_false")
184             if len(infos) == len(obs):
185                 for col in infos.columns:
186                     if col not in obs.columns:
187                         obs[col] = infos[col]
188         except Exception:
189             pass
190
191     obs = sanitize_observation_time_columns(obs)
192     obs = normalize_observation_columns(obs)
193     obs = add_meter_kwh_columns(obs)
194
195     return progress, obs

```

F.8 backend/grafics/energy_price.py

Listing 62: backend/grafics/energy_price.py

```

1  """Dades del quadre de comandament del preu de l'energia."""
2
3  from __future__ import annotations
4
5  import pandas as pd
6  import plotly.graph_objects as go
7
8  from backend.grafics.column_utils import _find_energy_price_column
9  from backend.grafics.energy_units import _energy_price_series_eur_per_kwh
10 from backend.grafics.style import STUDIO_CHART_COLORS
11 from backend.grafics.time_utils import (
12     _canon_day_series,
13     _ensure_datetime_index,
14     _hour_series,
15     _month_series,
16     _raw_axis_data,
17     _seasonal_marker_colors,
18     filter_by_season,
19 )
20
21 def make_energy_price_plot(obs: pd.DataFrame, mode: str, season: str) -> go.Figure:
22     """Crea una figura de Plotly per al preu de l'energia."""
23     from backend.grafics.observation_columns import normalize_observation_columns
24
25     base = _ensure_datetime_index(normalize_observation_columns(obs)).copy()

```

```

26 price_col = _find_energy_price_column(base.columns)
27 if price_col is None:
28     return go.Figure()
29
30 # El preu pot venir en EUR/J o EUR/kWh segons el wrapper. El convertim sempre a
31 # EUR/kWh perquè les etiquetes i comparacions siguin directes.
32 base["__energy_price_eur_per_kwh__"] = _energy_price_series_eur_per_kwh(base, price_col)
33 if base["__energy_price_eur_per_kwh__"].dropna().empty:
34     return go.Figure()
35
36 fig = go.Figure()
37 price_col = "__energy_price_eur_per_kwh__"
38 color = STUDIO_CHART_COLORS["price"]
39
40 if mode == "hour":
41     # Hora típica: mitjana del preu per hora del dia dins la temporada triada.
42     df = filter_by_season(base, season)
43     if df.empty:
44         return go.Figure()
45     h = _hour_series(df)
46     grouped = df.groupby(h)[price_col].mean()
47     x_vals = list(range(24))
48     y_vals = [grouped.get(hh, float("nan")) for hh in x_vals]
49     fig.add_trace(
50         go.Scatter(
51             x=x_vals,
52             y=y_vals,
53             mode="lines+markers",
54             name="Energy Price",
55             line=dict(color=color, width=2.8),
56             marker=dict(color=color, size=6),
57         )
58     )
59     fig.update_xaxes(title="Hour of Day", tickmode="linear", dtick=1)
60
61 elif mode == "day":
62     df = filter_by_season(base, season)
63     if df.empty:
64         return go.Figure()
65     x_vals, y_vals, hover = _canon_day_series(df, price_col)
66     fig.add_trace(
67         go.Scatter(
68             x=x_vals,
69             y=y_vals,
70             mode="lines",
71             name="Energy Price",
72             line=dict(color=color, width=2.8),
73             hovertext=hover,
74             hovertemplate="%{hovertext}<br>{%{y:.4f}} EUR/kWh<extra></extra>",
75         )
76     )
77     fig.update_xaxes(title="Day (1..365)", tickmode="linear", tick0=1, dtick=10)
78
79 elif mode == "month":
80     # El mes mostra mitjana de preu, no suma; sumar preus no tindria interpretació.
81     df = base if season == "All" else filter_by_season(base, season)
82     if df.empty:
83         return go.Figure()
84     m = _month_series(df)
85     grouped = df.groupby(m)[price_col].mean()
86     x_vals = list(range(1, 13))
87     y_vals = [grouped.get(mm, float("nan")) for mm in x_vals]
88     marker_colors = _seasonal_marker_colors(x_vals, season, active_color=color)
89     fig.add_trace(
90         go.Scatter(
91             x=x_vals,
92             y=y_vals,
93             mode="lines+markers",
94             name="Energy Price",
95             line=dict(color=color, width=2.8),
96             marker=dict(color=marker_colors, size=6),
97         )
98     )
99     fig.update_xaxes(title="Month", tickmode="linear", dtick=1)
100
101 elif mode == "raw":
102     df = filter_by_season(base, season)
103     if df.empty:
104         return go.Figure()
105     x_vals, y_vals, hover = _raw_axis_data(df, price_col)
106     fig.add_trace(
107         go.Scatter(
108             x=x_vals,
109             y=y_vals,
110             mode="lines",
111             name="Energy Price",
112             line=dict(color=color, width=2.4),
113             hovertext=hover,
114             hovertemplate="%{hovertext}<br>{%{y:.4f}} EUR/kWh<extra></extra>",
115         )
116     )
117     fig.update_xaxes(title="Time (step)")

```

```

118 fig.update_yaxes(title="Price (EUR/kWh)")
119 fig.update_layout(title=f"Energy Price - {mode.capitalize()} ({season})")
120
121 return fig

```

F.9 backend/grafics/energy_units.py

Listing 63: backend/grafics/energy_units.py

```

1  """Conversions d'energia, preu i unitats de bateria per als gràfics."""
2
3  from __future__ import annotations
4
5  import pandas as pd
6
7  from backend.grafics.time_utils import _auto_scale_series
8
9  def _energy_price_series_eur_per_kwh(df: pd.DataFrame, col: str) -> pd.Series:
10     """Preu de l'energia sèrie eur per kwh."""
11     price = pd.to_numeric(df[col], errors="coerce")
12     label = col.lower()
13     if "eur_per_kwh" in label or "price_eur_per_kwh" in label or "energy_price_kwh" in label:
14         return price
15
16     median_abs = price.abs().dropna().median()
17     if pd.notna(median_abs) and median_abs > 5:
18         return price / 1000.0
19     return price
20
21
22 def _energy_to_kwh(values: pd.Series, unit_kind: str, timestep_hours: float | None = None) -> pd.Series:
23     """Energy to kwh."""
24     numeric = pd.to_numeric(values, errors="coerce")
25     if unit_kind == "kwh":
26         return numeric
27     if unit_kind == "joule":
28         return numeric / 3_600_000.0
29     if unit_kind == "watt":
30         hours = 0.0 if timestep_hours is None else float(timestep_hours)
31         return (numeric / 1000.0) * hours
32     return numeric
33
34
35 def _battery_state_display_series(
36     base: pd.DataFrame,
37     state_col: str,
38 ) -> tuple[pd.Series, str, str]:
39     """Retorna la sèrie de visualització, la unitat i l'etiqueta per a les sortides de l'estat de la bateria."""
40     raw = pd.to_numeric(base[state_col], errors="coerce")
41     label = state_col.lower()
42
43     if label in {"storage_battery_charge_state"}:
44         # EnergyPlus ElectricLoadCenter: Emmagatzematge: Informes de l'estat de càrrega de la bateria en Ah.
45         return raw, "Ah", "Battery Charge State"
46
47     if (
48         "fraction" in label
49         or "soc" in label
50         or label in {"battery_state_of_charge", "state_of_charge"}
51     ):
52         finite = raw.dropna()
53         if not finite.empty and finite.abs().max() <= 1.5:
54             return raw * 100.0, "%", "Battery State of Charge"
55         return raw, "%", "Battery State of Charge"
56
57     if label in {"battery_charge_state", "storage_charge_state", "electric_storage_charge_state"}:
58         # EnergyPlus ElectricLoadCenter: Emmagatzematge: estat de càrrega dels informes simples en J.
59         return raw / 3_600_000.0, "kWh", "Battery Stored Energy"
60
61     finite_abs = raw.abs().dropna()
62     if not finite_abs.empty and finite_abs.median() > 1e5:
63         return raw / 3_600_000.0, "kWh", "Battery Stored Energy"
64
65     scaled, suffix = _auto_scale_series(raw)
66     unit = f"raw {suffix}".strip() if suffix else "raw"
67     return scaled, unit, "Battery State"

```


F.10 backend/grafics/episode.py

Listing 64: backend/grafics/episode.py

```
1  """Xifres del quadre de comandament a nivell d'episodi."""
2
3  from __future__ import annotations
4
5  import numpy as np
6  import pandas as pd
7  import plotly.graph_objects as go
8  from plotly.subplots import make_subplots
9
10 from backend.grafics.style import STUDIO_CHART_COLORS, _apply_figure_theme
11
12 def _resolve_episode_mean_power(progress: pd.DataFrame) -> pd.Series | None:
13     """Resol la potència mitjana de l'episodi."""
14     mean_power = None
15     if "mean_power_demand" in progress.columns:
16         mean_power = pd.to_numeric(progress["mean_power_demand"], errors="coerce")
17
18     if mean_power is None:
19         mean_power = pd.Series(np.nan, index=progress.index, dtype=float)
20
21     derived_mean = None
22     if (
23         "cumulative_power_demand" in progress.columns
24         and "length(timesteps)" in progress.columns
25     ):
26         derived_mean = (
27             pd.to_numeric(progress["cumulative_power_demand"], errors="coerce")
28             / pd.to_numeric(progress["length(timesteps)"], errors="coerce").replace(0, np.nan)
29         )
30
31     if derived_mean is not None:
32         replace_mask = mean_power.isna() | (
33             (mean_power == 0) & derived_mean.notna() & (derived_mean != 0)
34         )
35         mean_power = mean_power.copy()
36         mean_power.loc[replace_mask] = derived_mean.loc[replace_mask]
37
38     if mean_power.isna().all():
39         return None
40
41     return mean_power
42
43 # Mètriques d'episodi.
44 def make_episode_metrics(progress: pd.DataFrame) -> go.Figure:
45     """Mètriques d'episodi amb 3 subtrames:
46     1) Recompensa
47     2) Violació de la temperatura
48     3) Demanda mitjana de potència
49     """
50     x = progress["episode_num"] if "episode_num" in progress.columns else (progress.index + 1)
51     episode_vals = x.tolist()
52     episode_ticks = [str(v) for v in episode_vals]
53
54     fig = make_subplots(
55         rows=3, cols=1,
56         subplot_titles=[
57             "Reward per Episode",
58             "Average Temperature Violation (°C)",
59             "Mean Power Demand per Episode (W)",
60         ],
61     )
62
63     if "mean_reward" in progress.columns:
64         fig.add_trace(
65             go.Scatter(
66                 x=x,
67                 y=progress["mean_reward"],
68                 mode="lines+markers",
69                 name="Reward",
70                 line=dict(color=STUDIO_CHART_COLORS["reward"], width=3),
71                 marker=dict(color=STUDIO_CHART_COLORS["reward"], size=7),
72             ),
73             row=1,
74             col=1,
75         )
76
77     if "mean_temperature_violation" in progress.columns:
78         fig.add_trace(
79             go.Scatter(
80                 x=x,
81                 y=progress["mean_temperature_violation"],
82                 mode="lines+markers",
83                 name="Violation",
84                 line=dict(color=STUDIO_CHART_COLORS["violation"], width=3),
85                 marker=dict(color=STUDIO_CHART_COLORS["violation"], size=7),
86             ),
87             row=2, col=1,
```

```

88     )
89
90     power_series = _resolve_episode_mean_power(progress)
91     if power_series is not None:
92         fig.add_trace(
93             go.Scatter(
94                 x=x,
95                 y=power_series,
96                 mode="lines+markers",
97                 name="Mean Power",
98                 line=dict(color=STUDIO_CHART_COLORS["consumption"], width=3),
99                 marker=dict(color=STUDIO_CHART_COLORS["consumption"], size=7),
100             ),
101             row=3, col=1,
102         )
103
104     for r in range(1, 4):
105         fig.update_xaxes(
106             row=r, col=1,
107             title_text="Episode",
108             tickmode="array",
109             tickvals=episode_vals,
110             ticktext=episode_ticks,
111         )
112
113     fig.update_layout(height=850, title_text="Episode Metrics", showlegend=False)
114     return _apply_figure_theme(fig)

```

F.11 backend/grafics/figures_common.py

Listing 65: backend/grafics/figures_common.py

```

1  """Paçana de compatibilitat per a tots els creadors de figures del panell de resultats.
2
3  La implementació es divideix per domini del gràfic en aquest paquet. Aquest mòdul
4  manté estable la ruta d'importació anterior per a les pàgines Streamlit i el backend de l'informe.
5  """
6
7  from __future__ import annotations
8
9  from backend.grafics.battery import (
10     _grid_source_unit_kind,
11     make_battery_charge_with_price_plot,
12     make_battery_power_plot,
13     make_battery_soc_plot,
14     make_battery_vs_grid_plot,
15 )
16 from backend.grafics.column_utils import (
17     _BATTERY_CHARGE_POWER_COLUMNS,
18     _BATTERY_DISCHARGE_ENERGY_COLUMNS,
19     _BATTERY_DISCHARGE_POWER_COLUMNS,
20     _BATTERY_STATE_COLUMNS,
21     _ENERGY_PRICE_COLUMNS,
22     _ensure_air_temperature,
23     _find_battery_state_column,
24     _find_energy_price_column,
25     _find_first_present_column,
26 )
27 from backend.grafics.comfort import (
28     DEFAULT_SUMMER_COMFORT_RANGE,
29     DEFAULT_SUMMER_FINAL,
30     DEFAULT_SUMMER_START,
31     DEFAULT_WINTER_COMFORT_RANGE,
32     _categorize_comfort,
33     _coerce_month_day,
34     _coerce_pair,
35     _comfort_bounds_from_reward_kwargs,
36     _ensure_temp_violation_column,
37     _extract_reward_kwargs,
38     _fix_colors_for_visibility,
39     make_comfort_compliance,
40     make_violationBars,
41     make_violation_single_line,
42 )
43 from backend.grafics.control import make_agent_actions_plot, make_radiant_control_plot
44 from backend.grafics.energy_price import make_energy_price_plot
45 from backend.grafics.energy_units import (
46     _battery_state_display_series,
47     _energy_price_series_eur_per_kwh,
48     _energy_to_kwh,
49 )
50 from backend.grafics.episode import _resolve_episode_mean_power, make_episode_metrics
51 from backend.grafics.heatmaps import (
52     _ensure_month_hour,
53     _zone_temperature_columns,
54     compute_zone_temperature_pivots,
55     make_heatmap,
56     make_violation_heatmap_percent,

```

```

57     make_zone_temperature_heatmaps,
58 )
59 from backend.grafics.hvac import (
60     _canon_day_total_series,
61     _infer_timestep_hours,
62     _raw_hourly_total_series,
63     _with_hvac_consumption_kwh,
64     make_hvac_consumption_plot,
65     make_hvac_meter_breakdown_plot,
66 )
67 from backend.grafics.style import (
68     STUDIO_CHART_COLORS,
69     STUDIO_HEATMAP_SCALES,
70     _alpha_color,
71     _apply_figure_theme,
72     pick_semantic_trace_color,
73     style_figure_semantics,
74 )
75 from backend.grafics.thermal import (
76     make_indoor_humidity_plot,
77     make_indoor_temperature_plot,
78     make_setpoints_plot,
79     make_setpoints_vs_indoor_plot,
80 )
81 from backend.grafics.time_utils import (
82     _auto_scale_series,
83     _canon_day_series,
84     _ensure_datetime_index,
85     _get_outdoor_temperature_series,
86     _has_mdh,
87     _hour_series,
88     _month_series,
89     _raw_axis_data,
90     _season_months,
91     _seasonal_marker_colors,
92     filter_by_season,
93 )
94
95 __all__ = [name for name in globals() if not name.startswith("__")]

```

F.12 backend/grafics/figures_zones.py

Listing 66: backend/grafics/figures_zones.py

```

1  # -*- coding: utf-8 -*-
2  """Detecció i filtratge de zones per als panells de resultats.
3
4  Aquest mòdul detecta zones tèrmiques des de la configuració de YAML o de la columna d'observació
5  patrons, i mapes les columnes de la zona seleccionada als camps genèrics que fan servir
6  les figures comunes.
7  """
8
9  from __future__ import annotations
10
11  from typing import Dict, List, Optional
12  import re
13
14  import pandas as pd
15
16  # Patrons flexibles per detectar variables de zona sense dependre d'un únic prefix.
17  VAR_PATTERNS: Dict[str, re.Pattern] = {
18      # temperatura interior de la zona
19      "air_temperature": re.compile(r"(?i)(?:^|_)air[_]?temperature(?:$|_)"),
20      # consignes
21      "htg_setpoint": re.compile(r"(?i)(?:^|_)htg[_]?setpoint(?:$|_|(?:^|_)heating[_]?setpoint(?:$|_))"),
22      "clg_setpoint": re.compile(r"(?i)(?:^|_)clg[_]?setpoint(?:$|_|(?:^|_)cooling[_]?setpoint(?:$|_))"),
23      # violació (si la registres per zona)
24      "temp_violation": re.compile(r"(?i)(?:^|_)temp[_]?violation(?:$|_)"),
25      # opcional (no es fan servir a les figures, però ajuda a detectar zones)
26      "air_humidity": re.compile(r"(?i)(?:^|_)air[_]?humidity(?:$|_)"),
27      "people": re.compile(r"(?i)(?:^|_)people(?:$|_|(?:^|_)occupant(?:$|_))"),
28  }
29
30  # Quines variables mapejarem a genèriques per a les figures
31  GENERIC_MAP_KEYS = ("air_temperature", "htg_setpoint", "clg_setpoint", "temp_violation")
32
33
34  # Detecció de zones a partir del YAML o dels noms de columna.
35  def _zones_from_yaml(yaml_cfg: Optional[dict]) -> List[str]:
36      """Zones definides a YAML sota →variablesair_temperaturekeys."""
37      if not yaml_cfg or not isinstance(yaml_cfg.get("variables"), dict):
38          return []
39      for var_name, var_def in yaml_cfg["variables"].items():
40          if str(var_def.get("variable_names")).lower() == "air_temperature":
41              keys = var_def.get("keys")
42              if isinstance(keys, list) and keys:
43                  return [str(k).strip() for k in keys if str(k).strip()]
44              if isinstance(keys, str) and keys.strip():

```

```

45         return [keys.strip()]
46     return []
47
48
49 def _try_extract_zone_name(col: str, pattern: re.Pattern) -> Optional[str]:
50     """
51     Donada una columna i un patró de variable (p.ex. air_temperature),
52     retorna el 'nom de zona' eliminant aquesta part.
53     Exemple: 'zone1_office_air_temperature' -> 'zone1_office'
54     """
55     m = pattern.search(col)
56     if not m:
57         return None
58     start, end = m.span()
59     # Retirem el separador '_' veí si cal
60     prefix = (col[:start].rstrip("_"))
61     suffix = (col[end:].lstrip("_"))
62     # Preferim el prefix (estil 'zone1_office_air_temperature').
63     zone = prefix if prefix else suffix
64     zone = zone.strip("_")
65     return zone or None
66
67
68 def _zones_from_columns(obs: pd.DataFrame) -> List[str]:
69     """Infereix zones a partir de columnes que contenen variables de zona conegudes."""
70     candidate_sets: List[set] = []
71     for var, pat in VAR_PATTERNS.items():
72         cols = [c for c in obs.columns if pat.search(c)]
73         zones = set()
74         for c in cols:
75             z = _try_extract_zone_name(c, pat)
76             if z:
77                 zones.add(z)
78         if len(zones) >= 2:
79             candidate_sets.append(zones)
80
81     if not candidate_sets:
82         # Últim recurs: busca *_air_temperature estrictament al final
83         zones = set()
84         for c in obs.columns:
85             if c.endswith("_air_temperature"):
86                 zones.add(c[:-len("_air_temperature")])
87         return sorted(zones)
88
89     # Tria el conjunt amb més cobertura (més zones detectades)
90     zones = max(candidate_sets, key=len)
91     return sorted(zones)
92
93
94 def detect_zones(obs: pd.DataFrame, yaml_cfg: Optional[dict] = None) -> List[str]:
95     """API pública de detecció de zones."""
96     zones = _zones_from_yaml(yaml_cfg)
97     if zones:
98         return zones
99     return _zones_from_columns(obs)
100
101
102 # Mapatge de columnes originals cap a noms genèrics per zona.
103 def _zone_columns(obs: pd.DataFrame, zone: str) -> Dict[str, str]:
104     """
105     Per una zona concreta, retorna un mapatge {var_key -> nom_columna} segons VAR_PATTERNS.
106     Exemple: {'air_temperature': 'zone1_office_air_temperature', ...}
107     """
108     mapping: Dict[str, str] = {}
109     for key, pat in VAR_PATTERNS.items():
110         for c in obs.columns:
111             if pat.search(c):
112                 z = _try_extract_zone_name(c, pat)
113                 if z and z.lower() == zone.lower():
114                     # si hi ha duplicats, manté el primer (ordre del CSV)
115                     mapping.setdefault(key, c)
116     return mapping
117
118
119 def filter_obs_by_zone(obs: pd.DataFrame, zone: Optional[str], yaml_cfg: Optional[dict] = None) -> pd.DataFrame:
120     """Retorna les observacions assignades a la zona seleccionada.
121
122     Es conserven les columnes globals, s'eliminen les columnes que pertanyen a altres zones,
123     i els valors de la zona seleccionada s'exposen mitjançant els noms genèrics utilitzats per
124     Constructors de figures comuns: ``air_temperature``, ``htg_setpoint``,
125     ``clg_setpoint`` i ``temp_violation``.
126     """
127     zones = detect_zones(obs, yaml_cfg)
128     if not zone or len(zones) <= 1 or zone not in zones:
129         return obs
130
131     # Construïm llistat de columnes de zona vs globals. No podem fiar-nos només del prefix
132     # perquè alguns CSV barregen noms de zona, alias i variables globals al mateix fitxer.
133     zone_cols_by_zone: Dict[str, set] = {z: set() for z in zones}
134     for key, pat in VAR_PATTERNS.items():
135         for c in obs.columns:
136             if pat.search(c):

```

```

137         z = _try_extract_zone_name(c, pat)
138         if z in zone_cols_by_zone:
139             zone_cols_by_zone[z].add(c)
140
141     keep_cols: List[str] = []
142     for c in obs.columns:
143         # Si pertany a alguna zona
144         matched_zone = None
145         for z, cols in zone_cols_by_zone.items():
146             if c in cols:
147                 matched_zone = z
148                 break
149         if matched_zone is None:
150             # global
151             keep_cols.append(c)
152         elif matched_zone == zone:
153             keep_cols.append(c)
154         # si és d'una altra zona, s'omet
155
156     out = obs[keep_cols].copy()
157
158     # Mapem la zona triada a noms genèrics perquè les figures comunes no hagin de saber
159     # com es deia originalment cada columna al CSV.
160     mapping = _zone_columns(out, zone)
161     for key in GENERIC_MAP_KEYS:
162         col = mapping.get(key)
163         if col and col in out.columns:
164             out[key] = out[col]
165
166     # Evita que _ensure_air_temperature faci mitjanes entre zones
167     # Si tenim la genèrica 'air_temperature', podem eliminar altres *_air_temperature
168     if "air_temperature" in out.columns:
169         candidates = [c for c in out.columns if VAR_PATTERNS["air_temperature"].search(c)]
170         for c in candidates:
171             if c != mapping.get("air_temperature"):
172                 # Drop amb errors='ignore' per si ja no hi és
173                 out.drop(columns=[c], inplace=True, errors="ignore")
174
175     return out
176
177
178 # Opcions de zona per als selectors de la interfície.
179 def get_zone_options(obs: pd.DataFrame, yaml_cfg: Optional[dict] = None) -> List[Dict[str, str]]:
180     """Retorna les opcions de zona."""
181     zones = detect_zones(obs, yaml_cfg)
182     return [{"label": z, "value": z} for z in zones]

```

F.13 backend/grafics/heatmaps.py

Listing 67: backend/grafics/heatmaps.py

```

1  """Xifres de mapes de calor per als resums del panell global i de zona."""
2
3  from __future__ import annotations
4
5  import numpy as np
6  import pandas as pd
7  import plotly.express as px
8  import plotly.graph_objects as go
9  from plotly.subplots import make_subplots
10
11  from backend.grafics.column_utils import _ensure_air_temperature
12  from backend.grafics.comfort import _comfort_bounds_from_reward_kwargs
13  from backend.grafics.time_utils import _ensure_datetime_index, filter_by_season
14
15  def make_heatmap(pivot_df: pd.DataFrame, label: str) -> go.Figure:
16      """Mapa de calor genèric donat un dataframe pivotat (Mes x Hora)."""
17      if pivot_df is None or pivot_df.empty:
18          return go.Figure()
19
20      fig = px.imshow(
21          pivot_df, origin="lower",
22          labels={"x": "Hora", "y": "Mes", "color": label},
23          aspect="auto", color_continuous_scale="Viridis",
24      )
25      fig.update_layout(title=f"Mapa de calor {label} (Mes x Hora)")
26      return fig
27
28
29  def make_violation_heatmap_percent(
30      obs: pd.DataFrame,
31      season: str = "All",
32      comfort_config=None,
33  ) -> go.Figure:
34      """
35      Mapa de calor (Mes x Hora) amb el % de timesteps en violació de confort.
36      - Usa temp_violation si existeix; si no, reward_kwargs o defaults.
37      - Respecta filtre d'estació via months (Winter/Spring/Summer/Autumn/All).

```

```

38     - Escala de color 0..100 (%).
39     """
40     # Preparem les columnes temporals que necessita el mapa.
41     base = _ensure_datetime_index(obs).copy()
42     if "month" not in base.columns:
43         base["month"] = base.index.month
44     if "hour" not in base.columns:
45         base["hour"] = base.index.hour
46     base = _ensure_air_temperature(base)
47
48     # Marquem cada timestep com a violació o confort.
49     if "temp_violation" in base.columns:
50         viol_raw = pd.to_numeric(base["temp_violation"], errors="coerce")
51         viol = (viol_raw > 0.0).where(viol_raw.notna(), np.nan)
52     else:
53         lower, upper = _comfort_bounds_from_reward_kwargs(base, comfort_config)
54         tin = pd.to_numeric(base["air_temperature"], errors="coerce")
55         viol = ((tin < lower) | (tin > upper)).where(tin.notna(), np.nan)
56
57     base = base.assign(__viol__=viol.astype(float))
58
59     # El filtre d'estació s'aplica abans de construir la matriu hora-mes.
60     base = filter_by_season(base, season)
61
62     # Cada cel·la guarda el percentatge de violació per mes i hora.
63     if base.empty:
64         pivot = pd.DataFrame(index=range(1, 13), columns=range(24), dtype=float)
65     else:
66         pivot = (
67             base.groupby(["month", "hour"])["__viol__"]
68                 .mean()
69                 .mul(100.0)
70                 .unstack()
71                 .reindex(index=range(1, 13))
72                 .reindex(columns=range(24))
73         )
74
75     # Pintem el percentatge resultant en una escala comuna de 0 a 100.
76     fig = px.imshow(
77         pivot,
78         origin="lower",
79         labels={"x": "Hora", "y": "Mes", "color": "Violacions (%)"},
80         aspect="auto",
81         color_continuous_scale="Viridis",
82     )
83     fig.update_coloraxes(cmin=0, cmax=100)
84     fig.update_layout(title=f"Violacions de confort (%) - Hora x Mes ({season})")
85     return fig
86
87 # Heatmaps de temperatura per zona.
88
89
90 def _pretty_zone_temperature_name(name: str) -> str:
91     """Retorna una etiqueta de zona llegible des d'un nom de columna de temperatura de l'aire."""
92     base = name[: -len("_air_temperature")]
93     base = base.replace("_", " ").strip()
94     return base if base else name
95
96
97 def _zone_temperature_columns(obs: pd.DataFrame) -> list[tuple[str, str]]:
98     """
99     Detecta columnes de temperatura d'interior per zona.
100     Retorna llista [(nom_zona, "colname"), ...].
101     Si no hi ha columnes *_air_temperature però existeix 'air_temperature',
102     retorna [('All', 'air_temperature')].
103     """
104     cols = [c for c in obs.columns if c.endswith("_air_temperature")]
105     if cols:
106         return [(_pretty_zone_temperature_name(c), c) for c in cols]
107     if "air_temperature" in obs.columns:
108         return [("All", "air_temperature")]
109     return []
110
111
112 def _ensure_month_hour(df: pd.DataFrame) -> pd.DataFrame:
113     """Assegura columnes 'month' i 'hour' a partir de l'index temporal."""
114     base = _ensure_datetime_index(df).copy()
115     if "month" not in base.columns:
116         base["month"] = base.index.month
117     if "hour" not in base.columns:
118         base["hour"] = base.index.hour
119     return base
120
121
122 def _zone_heatmap_layout(nrows: int, ncols: int) -> tuple[float, float, int]:
123     """Retorna l'espai i l'alçada segurs de la subparcel·la per a edificis grans de diverses zones."""
124     horizontal_spacing = 0.06 if ncols > 1 else 0.0
125     if nrows <= 1:
126         vertical_spacing = 0.0
127     else:
128         vertical_spacing = min(0.045, 0.9 / max(nrows - 1, 1))
129     height = max(560, 250 * nrows + 120)

```

```

130     return horizontal_spacing, vertical_spacing, height
131
132
133 def compute_zone_temperature_pivots(
134     obs: pd.DataFrame, season: str = "All", agg: str = "mean"
135 ) -> dict[str, pd.DataFrame]:
136     """
137     Construeix, per a cada zona, un pivot Mes×Hora amb la temperatura d'interior.
138     - Respecta el filtre d'estació (via filter_by_season).
139     - Reindexa sempre a mesos 1..12 i hores 0..23 (cel·les inexistentes -> NaN).
140     - 'agg' pot ser 'mean' o 'median'.
141
142     Retorna: {nom_zona: DataFrame (index=1..12 mesos, columns=0..23 hores)}
143     """
144     base = _ensure_month_hour(_ensure_air_temperature(obs))
145     base = filter_by_season(base, season)
146
147     zones = _zone_temperature_columns(base)
148     if not zones:
149         return {}
150
151     aggfun = "median" if str(agg).lower() == "median" else "mean"
152
153     pivots: dict[str, pd.DataFrame] = {}
154     for zname, col in zones:
155         s = pd.to_numeric(base[col], errors="coerce")
156         g = (
157             s.groupby([base["month"].astype(int), base["hour"].astype(int)])
158             .agg(aggfun)
159             .unstack() # columnes = hour
160         )
161         # assegura el grid complet Mes(1..12) × Hora(0..23)
162         g = g.reindex(index=range(1, 13)).reindex(columns=range(24))
163         pivots[zname] = g
164
165     return pivots
166
167
168 def make_zone_temperature_heatmaps(
169     obs: pd.DataFrame,
170     season: str = "All",
171     agg: str = "mean",
172     max_cols: int = 3,
173 ) -> go.Figure:
174     """Crea mapes de calor de temperatura interior mensuals per hores per a cada zona.
175
176     La figura generada utilitza una escala de colors compartida entre zones, canònica
177     eixos mes/hora, el filtre de temporada seleccionat i la mitjana o la mediana
178     agregació. El creador de pivot omet les dades de zones buides o incompletes.
179
180     Paràmetres:
181         obs: DataFrame d'observació.
182         season: filtre de temporada: ``All``, ``Winter``, ``Spring``, ``Summer`` o
183             ``Autumn``.
184         agg: mètode d'agregació, ja sigui ``mean`` o ``median``.
185         max_cols: nombre màxim de columnes de subgràfic.
186
187     Retorna:
188         Una figura Plotly amb un mapa de calor per zona, o un únic mapa de calor quan el
189         run només conté una zona.
190     """
191     pivots = compute_zone_temperature_pivots(obs, season=season, agg=agg)
192     if not pivots:
193         return go.Figure()
194
195     # Escala comuna: min/max sobre totes les zones (ignorant NaN)
196     all_vals = np.concatenate([p.values.flatten() for p in pivots.values()])
197     finite_vals = all_vals[np.isfinite(all_vals)]
198     if finite_vals.size:
199         cmin = float(np.nanmin(finite_vals))
200         cmax = float(np.nanmax(finite_vals))
201     else:
202         cmin, cmax = np.nan, np.nan
203     if not np.isfinite(cmin) or not np.isfinite(cmax) or cmin == cmax:
204         # reserva segura
205         cmin, cmax = 15.0, 30.0
206
207     znames = list(pivots.keys())
208     n = len(znames)
209     ncols = min(max_cols, n)
210     nrows = (n + ncols - 1) // ncols
211     horizontal_spacing, vertical_spacing, height = _zone_heatmap_layout(nrows, ncols)
212
213     fig = make_subplots(
214         rows=nrows,
215         cols=ncols,
216         subplot_titles=znames,
217         horizontal_spacing=horizontal_spacing,
218         vertical_spacing=vertical_spacing,
219     )
220
221     # Afegeix un heatmap per zona, compartint escala; una sola colorbar (últim subplot)

```

```

222 for i, zname in enumerate(znames):
223     r = i // ncols + 1
224     c = i % ncols + 1
225     pivot = pivots[zname]
226     showscale = (i == n - 1) # només a l'últim
227     heat = go.Heatmap(
228         z=pivot.values,
229         x=list(pivot.columns), # 0..23
230         y=list(pivot.index), # 1..12
231         colorscale="Viridis",
232         zmin=cmin,
233         zmax=cmax,
234         colorbar=dict(title="°C"),
235         showscale=showscale,
236     )
237     fig.add_trace(heat, row=r, col=c)
238     fig.update_xaxes(title_text="Hora", row=r, col=c, tickmode="linear", dtick=2)
239     fig.update_yaxes(title_text="Mes", row=r, col=c, autorange="reversed") # 1..12 de dalt a baix
240
241 fig.update_layout(
242     title=f"Temperatura interior per zona - Mes × Hora ({season}, {agg})",
243     margin=dict(l=60, r=20, t=60, b=40),
244     height=height,
245 )
246 return fig

```

F.14 backend/grafics/hvac.py

Listing 68: backend/grafics/hvac.py

```

1  """HVAC xifres de consum i avaria del comptador."""
2
3  from __future__ import annotations
4
5  import numpy as np
6  import pandas as pd
7  import plotly.graph_objects as go
8
9  from backend.grafics.style import STUDIO_CHART_COLORS, _alpha_color, _apply_figure_theme
10 from backend.grafics.time_utils import (
11     _ensure_datetime_index,
12     _hour_series,
13     _month_series,
14     _raw_axis_data,
15     _season_months,
16     _seasonal_marker_colors,
17     filter_by_season,
18 )
19
20 def _infer_timestep_hours(df: pd.DataFrame) -> float:
21     """Infereix la durada del timestep en hores a partir del temps disponible."""
22     if "time_elapsed(hours)" in df.columns:
23         elapsed = pd.to_numeric(df["time_elapsed(hours)"], errors="coerce")
24         elapsed_diffs = elapsed.diff().dropna()
25         elapsed_diffs = elapsed_diffs[(elapsed_diffs > 0) & np.isfinite(elapsed_diffs)]
26         if not elapsed_diffs.empty:
27             return float(elapsed_diffs.median())
28
29     if isinstance(df.index, pd.DatetimeIndex) and len(df.index) > 1:
30         diffs = pd.Series(df.index, index=df.index).diff().dropna()
31         diffs_h = diffs.dt.total_seconds() / 3600.0
32         diffs_h = diffs_h[(diffs_h > 0) & np.isfinite(diffs_h)]
33         if not diffs_h.empty:
34             return float(diffs_h.median())
35
36     return 0.25
37
38
39 def _with_hvac_consumption_kwh(df: pd.DataFrame, hvac_col: str, unit_kind: str) -> pd.DataFrame:
40     """Afegeix una columna temporal amb el consum HVAC per timestep en kWh."""
41     from backend.grafics.observation_columns import convert_hvac_source_to_kwh
42
43     base = df.copy()
44     base["__hvac_consumption_kwh__"] = convert_hvac_source_to_kwh(
45         base[hvac_col],
46         unit_kind,
47         _infer_timestep_hours(base),
48     )
49     return base
50
51
52 def _canon_day_total_series(df: pd.DataFrame, col: str):
53     """
54     Retorna (x, y, hover) per a consum diari total.
55
56     Si hi ha diversos anys, en fa la mitjana per dia canònic (MM-DD).
57     """
58     if isinstance(df.index, pd.DatetimeIndex):

```



```

59     daily = df[col].resample("D").sum(min_count=1)
60     g = daily.groupby([daily.index.month, daily.index.day]).mean()
61     months = np.array([idx[0] for idx in g.index], dtype=int)
62     days = np.array([idx[1] for idx in g.index], dtype=int)
63     order = pd.to_datetime(
64         {"year": 2001, "month": months, "day": days},
65         errors="coerce",
66     )
67     ord_idx = np.argsort(order.values)
68     x = order.dt.dayofyear.to_numpy()[ord_idx].tolist()
69     y = g.to_numpy()[ord_idx].tolist()
70     hover = [
71         f"{int(month):02d}-{int(day):02d}"
72         for month, day in zip(months[ord_idx], days[ord_idx])
73     ]
74     return x, y, hover
75
76     daily_ids = np.arange(len(df)) // 24 + 1
77     g = pd.Series(df[col].to_numpy()).groupby(daily_ids).sum()
78     x = g.index.tolist()
79     y = g.values.tolist()
80     hover = [f"Day {i}" for i in x]
81     return x, y, hover
82
83
84 def _raw_hourly_total_series(df: pd.DataFrame, col: str):
85     """Retorna un kWh total per hora per als gràfics de consum del període brut."""
86     values = pd.to_numeric(df[col], errors="coerce")
87
88     if isinstance(df.index, pd.DatetimeIndex):
89         hourly = values.resample("h").sum(min_count=1).dropna()
90         x = list(range(1, len(hourly) + 1))
91         y = hourly.to_list()
92         hover = [stamp.strftime("%m-%d %H:00") for stamp in hourly.index]
93         return x, y, hover
94
95     timestep_hours = _infer_timestep_hours(df)
96     if timestep_hours > 0:
97         steps_per_hour = max(1, int(round(1.0 / timestep_hours)))
98     else:
99         steps_per_hour = 4
100
101     groups = np.arange(len(values)) // steps_per_hour
102     hourly = values.groupby(groups).sum(min_count=1).dropna()
103     x = (hourly.index.to_numpy(dtype=int) + 1).tolist()
104     y = hourly.to_list()
105     hover = [f"Hour {hour}" for hour in x]
106     return x, y, hover
107
108
109 def make_hvac_consumption_plot(
110     obs: pd.DataFrame,
111     mode: str,
112     season: str,
113     *,
114     raw_as_rate: bool = False,
115     raw_as_line: bool = False,
116 ) -> go.Figure:
117     """Crea una figura Plotly per al consum de climatització."""
118     from backend.grafics.observation_columns import (
119         HVAC_POWER_COLUMN,
120         find_hvac_consumption_source,
121         normalize_observation_columns,
122     )
123
124     base = _ensure_datetime_index(normalize_observation_columns(obs))
125
126     fig = go.Figure()
127     hvac_source = find_hvac_consumption_source(base.columns)
128     if hvac_source is None:
129         return go.Figure()
130     hvac_col, hvac_unit = hvac_source
131
132     if raw_as_rate and mode == "raw" and hvac_unit == "watt":
133         # En mode interactiu a vegades volem veure potència instantània, no energia per pas.
134         # Si la font és W, podem mostrar-la directament com kW.
135         df = filter_by_season(base, season)
136         if df.empty:
137             return go.Figure()
138         x_vals, y_vals, hover = _raw_axis_data(df, HVAC_POWER_COLUMN)
139         y_kw = (pd.Series(y_vals, dtype=float) / 1000.0).tolist()
140         fig.add_trace(
141             go.Scatter(
142                 x=x_vals,
143                 y=y_kw,
144                 mode="lines",
145                 name="HVAC Demand Rate",
146                 line=dict(color=STUDIO_CHART_COLORS["consumption"], width=2.0),
147                 hovertext=hover,
148                 hovertemplate="%{hovertext}<br>{%y:.2f} kW<extra></extra>",
149             )
150         )

```

```

151 fig.update_xaxes(title="Time (step)")
152 fig.update_yaxes(title="Demand Rate (kW)")
153 fig.update_layout(title=f"HVAC Demand Rate - {mode.capitalize()} ({season})")
154 return fig
155
156 base = _with_hvac_consumption_kwh(base, hvac_col, hvac_unit)
157 energy_col = "_hvac_consumption_kwh_"
158
159 if mode == "hour":
160     # Mitjana per hora típica: sumem cada hora dins de cada dia i després fem la mitjana.
161     df = filter_by_season(base, season)
162     if df.empty:
163         return go.Figure()
164     day_keys = pd.Index(df.index.floor("D"), name="day")
165     hour_keys = pd.Index(_hour_series(df), name="hour")
166     hourly_slots = df.groupby([day_keys, hour_keys])[energy_col].sum()
167     grouped = hourly_slots.groupby(level="hour").mean()
168     x_vals = list(range(24))
169     y_vals = [grouped.get(hh, float("nan")) for hh in x_vals]
170     fig.add_trace(go.Bar(x=x_vals, y=y_vals, name="HVAC Consumption"))
171     fig.update_xaxes(title="Hour of Day", tickmode="linear", dtick=1)
172     y_axis_title = "Mean Hourly Consumption (kW)"
173
174 elif mode == "day":
175     df = filter_by_season(base, season)
176     if df.empty:
177         return go.Figure()
178     x_vals, y_vals, hover = _canon_day_total_series(df, energy_col)
179     fig.add_trace(
180         go.Scatter(
181             x=x_vals,
182             y=y_vals,
183             mode="lines",
184             name="HVAC Consumption",
185             line=dict(color=STUDIO_CHART_COLORS["consumption"], width=2.4),
186             hovertext=hover,
187             hovertemplate="%{hovertext}<br>{%y:.2f} kWh<extra></extra>",
188         )
189     )
190     fig.update_xaxes(title="Day (1..365)", range=[1, 365], tickmode="linear", tick0=1, dtick=10)
191     y_axis_title = "Daily Consumption (kWh)"
192
193 elif mode == "month":
194     # Si hi ha més d'un any, agrupem per any/mes abans de mitjanar per mes.
195     df = base if season == "All" else filter_by_season(base, season)
196     if df.empty:
197         return go.Figure()
198     if isinstance(df.index, pd.DatetimeIndex):
199         monthly = df.groupby([df.index.year, df.index.month])[energy_col].sum()
200         grouped = monthly.groupby(level=1).mean()
201     else:
202         m = _month_series(df)
203         grouped = df.groupby(m)[energy_col].sum()
204     x_vals = list(range(1, 13))
205     y_vals = [grouped.get(mm, float("nan")) for mm in x_vals]
206     months_sel = _season_months(season)
207     marker_colors = _seasonal_marker_colors(
208         x_vals, season, active_color=STUDIO_CHART_COLORS["consumption"]
209     )
210     fig.add_trace(
211         go.Bar(
212             x=x_vals,
213             y=y_vals,
214             name="HVAC Consumption",
215             marker_color=marker_colors,
216         )
217     )
218     fig.update_xaxes(title="Month", tickmode="linear", dtick=1)
219     y_axis_title = "Monthly Consumption (kWh)"
220
221 elif mode == "raw":
222     # La vista raw es reagrupa a hores perquè els CSV de simulació poden tenir pas de
223     # 15 minuts i una barra per timestep seria massa sorollosa.
224     df = filter_by_season(base, season)
225     if df.empty:
226         return go.Figure()
227     x_vals, y_vals, hover = _raw_hourly_total_series(df, energy_col)
228     if not y_vals:
229         return go.Figure()
230
231     if raw_as_line:
232         fig.add_trace(
233             go.Scatter(
234                 x=x_vals,
235                 y=y_vals,
236                 mode="lines",
237                 name="HVAC Consumption",
238                 line=dict(color=STUDIO_CHART_COLORS["consumption"], width=2.2),
239                 meta="keep_color",
240                 hovertext=hover,
241                 hovertemplate="%{hovertext}<br>{%y:.2f} kWh<extra></extra>",
242             )

```

```

243     )
244     else:
245         fig.add_trace(go.Bar(
246             x=x_vals,
247             y=y_vals,
248             name="HVAC Consumption",
249             marker_color=STUDIO_CHART_COLORS["consumption"],
250             hovertext=hover,
251             hovertemplate="%{hovertext}<br>{%y:.2f} kWh<extra></extra>",
252         ))
253         fig.update_xaxes(title="Time (hour)")
254         y_axis_title = "Hourly Consumption (kWh)"
255     else:
256         y_axis_title = "Consumption (kWh)"
257
258     fig.update_yaxes(title=y_axis_title)
259     fig.update_layout(title=f"HVAC Consumption - {mode.capitalize()} ({season})")
260     return fig
261
262
263 # Desglossament del consum HVAC per meters disponibles.
264
265 def make_hvac_meter_breakdown_plot(
266     obs: pd.DataFrame,
267     mode: str,
268     season: str,
269 ) -> go.Figure:
270     """Crea una figura Plotly per a l'avaria del comptador de climatització."""
271     from backend.grafics.observation_columns import (
272         DETAILED_HVAC_METER_KWH_COLUMNS,
273         DETAILED_HVAC_METER_LABELS,
274         ELECTRIC_HVAC_METER_ALIASES,
275         HVAC_ENERGY_METER_COLUMNS,
276         HVAC_POWER_COLUMN,
277         add_meter_kwh_columns,
278         convert_hvac_source_to_kwh,
279         find_hvac_consumption_source,
280         normalize_observation_columns,
281     )
282
283     base = _ensure_datetime_index(add_meter_kwh_columns(normalize_observation_columns(obs)))
284     meter_series = []
285     # Alguns IDF només tenen el total HVAC i altres exposen meters separats per fans,
286     # pumps, heating, cooling, etc. Afegim només les sèries que realment tenen energia.
287     for meter_alias, kwh_column in DETAILED_HVAC_METER_KWH_COLUMNS.items():
288         if kwh_column not in base.columns:
289             continue
290         numeric_values = pd.to_numeric(base[kwh_column], errors="coerce")
291         if not numeric_values.dropna().abs().gt(1e-12).any():
292             continue
293         meter_series.append(
294             (
295                 kwh_column,
296                 DETAILED_HVAC_METER_LABELS.get(meter_alias, meter_alias),
297             )
298         )
299     fig = go.Figure()
300     colors = {
301         "Natural Gas HVAC": "#92400e",
302         "Heating Electricity": STUDIO_CHART_COLORS["heating"],
303         "Cooling Electricity": STUDIO_CHART_COLORS["cooling"],
304         "Fans Electricity": STUDIO_CHART_COLORS["neutral"],
305         "Pumps Electricity": "#0891b2",
306         "Heat Rejection Electricity": "#f97316",
307         "Humidifier Electricity": "#7c3aed",
308         "Heat Recovery Electricity": "#16a34a",
309         "Other / Unmetered HVAC Electricity": STUDIO_CHART_COLORS["consumption_muted"],
310     }
311     electric_detail_columns = [
312         DETAILED_HVAC_METER_KWH_COLUMNS[meter_alias]
313         for meter_alias in ELECTRIC_HVAC_METER_ALIASES
314         if DETAILED_HVAC_METER_KWH_COLUMNS.get(meter_alias) in base.columns
315     ]
316
317     hvac_source = find_hvac_consumption_source(base.columns)
318     if hvac_source is not None:
319         hvac_col, hvac_unit = hvac_source
320         hvac_col_is_total_electric = (
321             hvac_col in HVAC_ENERGY_METER_COLUMNS
322             or hvac_col.lower() in {column.lower() for column in HVAC_ENERGY_METER_COLUMNS}
323             or hvac_col == HVAC_POWER_COLUMN
324         )
325         if hvac_col_is_total_electric:
326             # Quan tenim total elèctric i també submeters, el "Other" és el residu.
327             # Això ajuda a detectar consum HVAC que EnergyPlus no ha separat per categoria.
328             base["_hvac_total_electricity_kwh_"] = convert_hvac_source_to_kwh(
329                 base[hvac_col],
330                 hvac_unit,
331                 _infer_timestep_hours(base),
332             )
333             if electric_detail_columns:
334                 electric_detail_sum = pd.DataFrame(

```

```

335         {
336             column: pd.to_numeric(base[column], errors="coerce")
337             for column in electric_detail_columns
338         },
339         index=base.index,
340     ).sum(axis=1, min_count=1).fillna(0.0)
341 else:
342     electric_detail_sum = pd.Series(0.0, index=base.index)
343 base["__other_hvac_electricity_kwh__"] = (
344     pd.to_numeric(base["__hvac_total_electricity_kwh__"], errors="coerce")
345     - electric_detail_sum
346 ).clip(lower=0.0)
347 if (
348     base["__other_hvac_electricity_kwh__"]
349     .dropna()
350     .abs()
351     .gt(1e-9)
352     .any()
353 ):
354     meter_series.append(
355         (
356             "__other_hvac_electricity_kwh__",
357             "Other / Unmetered HVAC Electricity",
358         )
359     )
360
361 if not meter_series:
362     return go.Figure()
363
364 if mode == "hour":
365     # Per hora fem mitjana entre dies: és millor per veure patrons típics que no pas
366     # sumar tot l'any en una sola barra enorme.
367     df = filter_by_season(base, season)
368     if df.empty:
369         return go.Figure()
370     day_keys = pd.Index(df.index.floor("D"), name="day")
371     hour_keys = pd.Index(_hour_series(df), name="hour")
372     x_vals = list(range(24))
373     for kwh_column, label in meter_series:
374         hourly_slots = df.groupby([day_keys, hour_keys])[kwh_column].sum()
375         grouped = hourly_slots.groupby(level="hour").mean()
376         y_vals = [grouped.get(hh, float("nan")) for hh in x_vals]
377         fig.add_trace(go.Bar(x=x_vals, y=y_vals, name=label, marker_color=colors.get(label)))
378     fig.update_xaxes(title="Hour of Day", tickmode="linear", dtick=1)
379     y_axis_title = "Mean Hourly Consumption (kWh)"
380
381 elif mode == "day":
382     # En vista diària interessa més la proporció entre meters que la sèrie completa,
383     # per això fem un donut amb el consum diari mitjà.
384     df = filter_by_season(base, season)
385     if df.empty:
386         return go.Figure()
387     labels = []
388     values = []
389     marker_colors = []
390     for kwh_column, label in meter_series:
391         _, y_vals, _ = _canon_day_total_series(df, kwh_column)
392         daily_mean = pd.to_numeric(pd.Series(y_vals), errors="coerce").dropna().mean()
393         if pd.isna(daily_mean) or daily_mean <= 1e-12:
394             continue
395         labels.append(label)
396         values.append(float(daily_mean))
397         marker_colors.append(colors.get(label, STUDIO_CHART_COLORS["neutral"]))
398
399     if not values:
400         return go.Figure()
401
402     fig.add_trace(
403         go.Pie(
404             labels=labels,
405             values=values,
406             hole=0.42,
407             marker=dict(colors=marker_colors, line=dict(color="#ffffff", width=2)),
408             textinfo="percent+label",
409             hovertemplate="%{label}<br>Mean daily consumption %{value:.2f} kWh<extra></extra>",
410         )
411     )
412     fig.update_layout(
413         title=f"HVAC Meter Breakdown - Mean Daily Share ({season})",
414         showlegend=True,
415     )
416     return _apply_figure_theme(fig)
417
418 elif mode == "month":
419     # En mesos agrupem per any i mes abans de fer la mitjana, així dos anys simulats
420     # no queden barrejats com si fossin un únic mes gegant.
421     df = base if season == "All" else filter_by_season(base, season)
422     if df.empty:
423         return go.Figure()
424     x_vals = list(range(1, 13))
425     for kwh_column, label in meter_series:
426         if isinstance(df.index, pd.DatetimeIndex):

```

```

427     monthly = df.groupby([df.index.year, df.index.month])[kwh_column].sum()
428     grouped = monthly.groupby(level=1).mean()
429     else:
430         grouped = df.groupby(_month_series(df))[kwh_column].sum()
431         y_vals = [grouped.get(mm, float("nan")) for mm in x_vals]
432         fig.add_trace(go.Bar(x=x_vals, y=y_vals, name=label, marker_color=colors.get(label)))
433         fig.update_xaxes(title="Month", tickmode="linear", dtick=1)
434         y_axis_title = "Monthly Consumption (kWh)"
435
436     elif mode == "raw":
437         df = filter_by_season(base, season)
438         if df.empty:
439             return go.Figure()
440         for kwh_column, label in meter_series:
441             x_vals, y_vals, hover = _raw_axis_data(df, kwh_column)
442             color = colors.get(label, STUDIO_CHART_COLORS["neutral"])
443             fig.add_trace(
444                 go.Scatter(
445                     x=x_vals,
446                     y=y_vals,
447                     mode="lines",
448                     name=label,
449                     line=dict(color=color, width=1.8),
450                     fillcolor=_alpha_color(color, 0.42),
451                     stackgroup="hvac_meter_breakdown",
452                     hovertext=hover,
453                     hovertemplate="%{hovertext}<br>{%y:.4f} kWh<extra></extra>",
454                 )
455             )
456             fig.update_xaxes(title="Time (step)")
457             y_axis_title = "Consumption per Timestep (kWh)"
458         else:
459             y_axis_title = "Consumption (kWh)"
460
461         fig.update_yaxes(title=y_axis_title)
462         fig.update_layout(
463             title=f"HVAC Meter Breakdown - {mode.capitalize()} ({season})",
464             bargroup="group",
465             bargap=0.18,
466             bargroupgap=0.08,
467         )
468         return _apply_figure_theme(fig)

```

F.15 backend/grafics/kpis.py

Listing 69: backend/grafics/kpis.py

```

1  """KPI càlculs per a BEMS-RL Studio resums de resultats.
2
3  Aquest mòdul deriva mètriques operatives compactes a partir de l'observació normalitzada
4  dades, incloses les infraccions de confort, HVAC consum d'energia, cost energètic i seleccionats
5  resums de temperatura de la zona.
6  """
7
8  import re
9
10 import pandas as pd
11
12 from backend.grafics.comfort_scope import comfort_scope_label, derive_occupied_mask
13 from backend.grafics.observation_columns import (
14     HVAC_POWER_COLUMN,
15     normalize_observation_columns,
16 )
17
18
19 def _normalize_name(s: str) -> str:
20     """Normalitza els noms de les zones perquè coincideixin amb les columnes de manera més tolerant."""
21     s = (s or "").lower()
22     s = s.replace(" ", "_").replace("-", "_")
23     return re.sub(r"[^a-z0-9_]+", "", s)
24
25
26 def _pick_zone_temperature_series(
27     obs: pd.DataFrame,
28     selected_zone: str | None,
29 ) -> pd.Series | None:
30     """Retorna la sèrie de temperatura interior utilitzada per les targetes KPI."""
31     zone_temp_cols = [c for c in obs.columns if c.endswith("_air_temperature")]
32     if zone_temp_cols:
33         if not selected_zone or selected_zone == "All":
34             return obs[zone_temp_cols].mean(axis=1)
35
36     target = _normalize_name(selected_zone)
37     cand_map = {
38         _normalize_name(c[:-len("_air_temperature")]): c for c in zone_temp_cols
39     }
40     col = cand_map.get(target)
41     if col is not None:

```

```

42         return pd.to_numeric(obs[col], errors="coerce")
43     for key, col in cand_map.items():
44         if target in key or key in target:
45             return pd.to_numeric(obs[col], errors="coerce")
46     return obs[zone_temp_cols].mean(axis=1)
47
48 if "air_temperature" in obs.columns:
49     return pd.to_numeric(obs["air_temperature"], errors="coerce")
50
51 return None
52
53
54 def _temperature_violation_frame(obs: pd.DataFrame, comfort_config=None) -> pd.DataFrame | None:
55     """Retorna les observacions amb una infracció de confort alineada per registre."""
56     if "temp_violation" in obs.columns:
57         result = obs.copy()
58         result["temp_violation"] = pd.to_numeric(result["temp_violation"], errors="coerce")
59         return result
60
61     try:
62         from backend.grafics.figures_common import _ensure_temp_violation_column
63
64         with_violation = _ensure_temp_violation_column(obs, comfort_config)
65     except Exception:
66         return None
67
68     if "temp_violation" not in with_violation.columns:
69         return None
70     with_violation = with_violation.copy()
71     with_violation["temp_violation"] = pd.to_numeric(
72         with_violation["temp_violation"],
73         errors="coerce",
74     )
75     return with_violation
76
77
78 def compute_kpis(
79     obs: pd.DataFrame,
80     cost_hourly: pd.DataFrame | None,
81     selected_zone: str | None = None,
82     comfort_scope: str = "all",
83     comfort_config=None,
84 ):
85     """Calculeu les targetes KPI mostrades al panell i exportades PDF.
86
87     Els KPI de temperatura respecten la zona seleccionada. HVAC i els KPI de costos segueixen sent globals.
88     La infracció de comoditat es mostra amb una etiqueta d'abast explícita per a hores ocupades
89     les recompenses no es comparen amb una mètrica de totes les hores per accident.
90     """
91     obs = normalize_observation_columns(obs)
92     kpis: dict[str, float | None] = {}
93
94     tin_series = _pick_zone_temperature_series(obs, selected_zone)
95     if tin_series is not None:
96         # Si l'usuari ha triat una zona, els KPI de temperatura han de parlar només
97         # d'aquella zona, no de la mitjana global de l'edifici.
98         kpis["Avg Temp Interior (C)"] = round(tin_series.mean(), 2)
99         kpis["Max Temp Interior (C)"] = round(tin_series.max(), 2)
100        kpis["Min Temp Interior (C)"] = round(tin_series.min(), 2)
101    else:
102        if "air_temperature" not in obs.columns:
103            zone_temp_cols = [c for c in obs.columns if c.endswith("_air_temperature")]
104            tmp = obs[zone_temp_cols].mean(axis=1) if zone_temp_cols else None
105        else:
106            tmp = obs["air_temperature"]
107        if tmp is not None:
108            tmp = pd.to_numeric(tmp, errors="coerce")
109            kpis["Avg Temp Interior (C)"] = round(tmp.mean(), 2)
110            kpis["Max Temp Interior (C)"] = round(tmp.max(), 2)
111            kpis["Min Temp Interior (C)"] = round(tmp.min(), 2)
112        else:
113            kpis["Avg Temp Interior (C)"] = None
114            kpis["Max Temp Interior (C)"] = None
115            kpis["Min Temp Interior (C)"] = None
116
117    if HVAC_POWER_COLUMN in obs.columns:
118        hvac = pd.to_numeric(obs[HVAC_POWER_COLUMN], errors="coerce")
119        kpis["Avg HVAC Power (kW)"] = round(hvac.mean() / 1000, 2)
120    else:
121        kpis["Avg HVAC Power (kW)"] = None
122
123    if cost_hourly is not None and "energy_cost_eur" in cost_hourly.columns:
124        energy_cost = pd.to_numeric(cost_hourly["energy_cost_eur"], errors="coerce")
125        kpis["Avg Energy Cost (EUR/h)"] = round(energy_cost.mean(), 2)
126    else:
127        kpis["Avg Energy Cost (EUR/h)"] = None
128
129    violation_frame = _temperature_violation_frame(obs, comfort_config)
130    primary_label = comfort_scope_label(comfort_scope)
131
132    if violation_frame is None:
133        kpis[f"Avg Temp Violation ({primary_label}, C)"] = None

```

```

134     return kpis
135
136     violation = violation_frame["temp_violation"]
137     occupied_mask = derive_occupied_mask(violation_frame)
138
139     # En rewards amb ocupació, comparar totes les hores amb hores ocupades pot portar a
140     # conclusions falses. Per això calculem l'abast principal i deixem l'altre com a context.
141     scoped_values = violation
142     if comfort_scope == "occupied" and occupied_mask is not None:
143         scoped_values = violation.loc[occupied_mask]
144     scoped_values = scoped_values.dropna()
145     scoped_mean = scoped_values.mean() if not scoped_values.empty else None
146     kpis[f"Avg Temp Violation ({primary_label}, C)"] = (
147         round(scoped_mean, 2) if scoped_mean is not None else None
148     )
149
150     if occupied_mask is not None:
151         all_values = violation.dropna()
152         occupied_values = violation.loc[occupied_mask].dropna()
153         all_mean = all_values.mean() if not all_values.empty else None
154         occupied_mean = occupied_values.mean() if not occupied_values.empty else None
155         kpis["Occupied Samples (%)"] = round(float(occupied_mask.mean()) * 100.0, 2)
156
157     if comfort_scope == "occupied":
158         kpis["Avg Temp Violation (All Hours, C)"] = (
159             round(all_mean, 2) if all_mean is not None else None
160         )
161     else:
162         kpis["Avg Temp Violation (Occupied Hours, C)"] = (
163             round(occupied_mean, 2) if occupied_mean is not None else None
164         )
165
166     return kpis

```

F.16 backend/grafics/metrics.py

Listing 70: backend/grafics/metrics.py

```

1  """Agrega les dades d'observació i progrés en mètriques preparades per a gràfics.
2
3  Les funcions aquí converteixen la sortida en brut Sinergym en mensual, per hora i
4  DataFrames orientats a la comoditat consumits pel panell de resultats, Plotly builders
5  i informes exportats.
6  """
7
8  import numpy as np
9  import pandas as pd
10 from typing import Optional
11
12 from backend.grafics.observation_columns import (
13     ENERGY_COST_COLUMN,
14     HVAC_POWER_COLUMN,
15     convert_hvac_source_to_kwh,
16     find_hvac_consumption_source,
17     normalize_observation_columns,
18 )
19 from backend.grafics.time_utils import sanitize_observation_time_columns
20
21
22 def _price_to_eur_per_kwh(values: pd.Series) -> pd.Series:
23     """Normalitza els valors dels preus de l'electricitat a EUR/kWh quan semblen ser EUR/MWh."""
24     price = pd.to_numeric(values, errors="coerce")
25     median_abs = price.abs().dropna().median()
26     if pd.notna(median_abs) and median_abs > 5:
27         return price / 1000.0
28     return price
29
30
31 def _infer_timestep_hours(obs: pd.DataFrame) -> float:
32     """Infereix la durada del timestep en hores per convertir potència a consum."""
33     if "time_elapsed(hours)" in obs.columns:
34         elapsed = pd.to_numeric(obs["time_elapsed(hours)"], errors="coerce")
35         elapsed_diffs = elapsed.diff().dropna()
36         elapsed_diffs = elapsed_diffs[(elapsed_diffs > 0) & np.isfinite(elapsed_diffs)]
37         if not elapsed_diffs.empty:
38             return float(elapsed_diffs.median())
39
40     time_col = None
41     if "timestamp" in obs.columns:
42         time_col = pd.to_datetime(obs["timestamp"], errors="coerce")
43     elif "datetime" in obs.columns:
44         time_col = pd.to_datetime(obs["datetime"], errors="coerce")
45
46     if time_col is not None:
47         time_diffs = time_col.diff().dropna().dt.total_seconds() / 3600.0
48         time_diffs = time_diffs[(time_diffs > 0) & np.isfinite(time_diffs)]
49         if not time_diffs.empty:
50             return float(time_diffs.median())

```



```

51
52 if all(col in obs.columns for col in ("month", "day_of_month", "hour")):
53     mdh = obs[["month", "day_of_month", "hour"]].apply(
54         pd.to_numeric,
55         errors="coerce",
56     )
57     valid = mdh.dropna()
58     if not valid.empty:
59         hour_blocks = valid.ne(valid.shift()).any(axis=1).cumsum()
60         block_sizes = hour_blocks.value_counts().sort_index()
61         block_sizes = block_sizes[block_sizes > 0]
62         if not block_sizes.empty:
63             return 1.0 / float(block_sizes.median())
64
65 return 0.25
66
67
68 def compute_metrics(progress: pd.DataFrame, obs: pd.DataFrame, yml_cfg: Optional[dict] = None) -> dict:
69     """
70     Calcula tots els DataFrames agregats per a figures,
71     validant contra les dades disponibles i opcionalment el fitxer YAML.
72     """
73     obs = sanitize_observation_time_columns(normalize_observation_columns(obs.copy()))
74
75     # Els CSV poden venir amb timestamp/datetime o amb columnes ja separades. Normalitzem
76     # month/hour aquí perquè totes les figures de resultats puguin reutilitzar el mateix output.
77     if 'month' not in obs.columns:
78         if 'timestamp' in obs.columns:
79             obs['timestamp'] = pd.to_datetime(obs['timestamp'])
80             obs['month'] = obs['timestamp'].dt.month
81         elif 'datetime' in obs.columns:
82             obs['datetime'] = pd.to_datetime(obs['datetime'])
83             obs['month'] = obs['datetime'].dt.month
84         else:
85             raise ValueError("La columna 'month' no existeix ni es pot derivar.")
86     if 'hour' not in obs.columns:
87         if 'timestamp' in obs.columns:
88             obs['timestamp'] = pd.to_datetime(obs['timestamp'])
89             obs['hour'] = obs['timestamp'].dt.hour
90         elif 'datetime' in obs.columns:
91             obs['datetime'] = pd.to_datetime(obs['datetime'])
92             obs['hour'] = obs['datetime'].dt.hour
93         else:
94             raise ValueError("La columna 'hour' no existeix ni es pot derivar.")
95
96     available = set(obs.columns)
97
98     # Si el CSV és multizona i no porta air_temperature global, fem una mitjana simple
99     # només com a fallback per a gràfics globals.
100     if 'air_temperature' not in available:
101         zone_temp_cols = [c for c in available if c.endswith('_air_temperature')]
102         if zone_temp_cols:
103             obs['air_temperature'] = obs[zone_temp_cols].mean(axis=1)
104             available.add('air_temperature')
105
106     if HVAC_POWER_COLUMN not in available:
107         for cand in ('hvac_power', 'HVAC_power', 'hvac_load'):
108             if cand in available:
109                 obs[HVAC_POWER_COLUMN] = obs[cand]
110                 available.add(HVAC_POWER_COLUMN)
111                 break
112
113     if 'temp_violation' not in available and 'air_temperature' in available:
114         from backend.graphics.figures.common import _ensure_temp_violation_column
115
116         obs = _ensure_temp_violation_column(obs, yml_cfg)
117         available.add('temp_violation')
118
119     bins = 12
120     if len(progress) > 0 and 'mean_reward' in progress.columns:
121         # progress.csv no sempre té calendari real; repartim els punts en 12 blocs per
122         # tenir una lectura mensual aproximada sense inventar dates.
123         progress = progress.copy()
124         progress['mean_reward'] = pd.to_numeric(progress['mean_reward'], errors='coerce')
125         progress['month_bin'] = (np.arange(len(progress)) * bins // len(progress)) + 1
126         monthly_reward = (
127             progress.groupby('month_bin')['mean_reward']
128             .mean()
129             .reset_index(name='mean_reward')
130         )
131         monthly_reward['month'] = monthly_reward['month_bin'].astype(int)
132     else:
133         monthly_reward = pd.DataFrame({'month': range(1, bins + 1), 'mean_reward': [0] * bins})
134
135     hourly_all = obs.groupby('hour').mean(numeric_only=True).reset_index()
136     winter = obs[obs['month'].isin([12, 1, 2])].groupby('hour').mean(numeric_only=True).reset_index()
137     summer = obs[obs['month'].isin([6, 7, 8])].groupby('hour').mean(numeric_only=True).reset_index()
138     monthly_obs = (
139         obs.groupby('month').mean(numeric_only=True)
140         .reindex(index=range(1, 13), fill_value=0)
141         .reset_index()
142     )

```



```

143
144 step_hours = _infer_timestep_hours(obs)
145 hvac_source = find_hvac_consumption_source(obs.columns)
146 if hvac_source is not None:
147     # Aquí unifiquem W, J o meters ja en kWh. La resta del dashboard treballa sempre
148     # amb aquesta columna normalitzada.
149     hvac_col, hvac_unit = hvac_source
150     obs["hvac_consumption_kwh"] = convert_hvac_source_to_kwh(
151         obs[hvac_col],
152         hvac_unit,
153         step_hours,
154     )
155     available.add("hvac_consumption_kwh")
156
157 cost_hourly = cost_monthly = None
158 if ENERGY_COST_COLUMN in available and 'hvac_consumption_kwh' in available:
159     obs['energy_price_kwh'] = _price_to_eur_per_kwh(obs[ENERGY_COST_COLUMN])
160     obs['energy_cost_eur'] = obs['hvac_consumption_kwh'] * obs['energy_price_kwh']
161     cost_hourly = obs.groupby('hour')['energy_cost_eur'].sum().reset_index()
162     cost_monthly = (
163         obs.groupby('month')['energy_cost_eur']
164         .sum()
165         .reindex(index=range(1, 13), fill_value=0)
166         .reset_index()
167     )
168
169 violation_hourly = eps_winter = eps_summer = season_line = None
170 if 'temp_violation' in available:
171     # Separem hivern/estiu perquè les consignes de confort poden ser diferents i la
172     # mitjana anual amagaria bastant el problema.
173     violation_hourly = obs.groupby('hour')['temp_violation'].mean().reset_index(name='mean_violation')
174     eps_winter = obs[obs['month'].isin([12, 1, 2])].groupby('hour')['temp_violation'].mean().reset_index(name='winter_violation')
175     eps_summer = obs[obs['month'].isin([6, 7, 8])].groupby('hour')['temp_violation'].mean().reset_index(name='summer_violation')
176     season_line = pd.merge(eps_winter, eps_summer, on='hour', how='outer').fillna(0)
177
178 pivot_violation = None
179 if 'temp_violation' in available:
180     pivot_violation = (
181         obs.groupby(['month', 'hour'])['temp_violation']
182         .mean()
183         .unstack(fill_value=0)
184         .reindex(index=range(1, 13), fill_value=0)
185     )
186
187 pivot_consumption = None
188 if 'hvac_consumption_kwh' in available:
189     pivot_consumption = (
190         obs.groupby(['month', 'hour'])['hvac_consumption_kwh']
191         .sum()
192         .unstack(fill_value=0)
193         .reindex(index=range(1, 13), fill_value=0)
194     )
195
196 return {
197     'monthly_reward': monthly_reward,
198     'hourly_all': hourly_all,
199     'winter': winter,
200     'summer': summer,
201     'monthly_obs': monthly_obs,
202     'cost_hourly': cost_hourly,
203     'cost_monthly': cost_monthly,
204     'violation_hourly': violation_hourly,
205     'eps_winter': eps_winter,
206     'eps_summer': eps_summer,
207     'season_line': season_line,
208     'pivot_violation': pivot_violation,
209     'pivot_consumption': pivot_consumption
210 }

```

F.17 backend/grafics/observation_columns.py

Listing 71: backend/grafics/observation_columns.py

```

1  """Utilitats de normalització de columnes per a fitxers d'observació Sinergym.
2
3  Els entorns Sinergym i les configuracions personalitzades poden emetre comptadors equivalents
4  sota diferents noms o unitats. Aquest mòdul centralitza els àlies, HVAC energia
5  conversió i creació de columnes normalitzades perquè tots els gràfics llegeixin igual
6  camps canònics.
7  """
8
9  from __future__ import annotations
10
11  from typing import Iterable
12
13  import pandas as pd
14
15  HVAC_POWER_COLUMN = "HVAC_electricity_demand_rate"

```

```

16 ENERGY_COST_COLUMN = "energy_cost"
17 JOULES_PER_KWH = 3_600_000.0
18
19 HVAC_ENERGY_METER_COLUMNS = (
20     "total_electricity_hvac",
21     "total_electricity_HVAC",
22     "Electricity:HVAC",
23     "electricity_hvac",
24 )
25
26 DETAILED_HVAC_METER_COLUMNS = {
27     "natural_gas_hvac": (
28         "NaturalGas:HVAC",
29         "natural_gas_hvac",
30         "naturalgas_hvac",
31     ),
32     "heating_electricity": (
33         "Heating:Electricity",
34         "heating_electricity",
35     ),
36     "cooling_electricity": (
37         "Cooling:Electricity",
38         "cooling_electricity",
39     ),
40     "fans_electricity": (
41         "Fans:Electricity",
42         "fans_electricity",
43     ),
44     "pumps_electricity": (
45         "Pumps:Electricity",
46         "pumps_electricity",
47     ),
48     "heat_rejection_electricity": (
49         "HeatRejection:Electricity",
50         "heat_rejection_electricity",
51     ),
52     "humidifier_electricity": (
53         "Humidifier:Electricity",
54         "Humidification:Electricity",
55         "humidifier_electricity",
56         "humidification_electricity",
57     ),
58     "heat_recovery_electricity": (
59         "HeatRecovery:Electricity",
60         "heat_recovery_electricity",
61     ),
62 }
63
64 DETAILED_HVAC_METER_LABELS = {
65     "natural_gas_hvac": "Natural Gas HVAC",
66     "heating_electricity": "Heating Electricity",
67     "cooling_electricity": "Cooling Electricity",
68     "fans_electricity": "Fans Electricity",
69     "pumps_electricity": "Pumps Electricity",
70     "heat_rejection_electricity": "Heat Rejection Electricity",
71     "humidifier_electricity": "Humidifier Electricity",
72     "heat_recovery_electricity": "Heat Recovery Electricity",
73 }
74
75 ELECTRIC_HVAC_METER_ALIASES = (
76     "heating_electricity",
77     "cooling_electricity",
78     "fans_electricity",
79     "pumps_electricity",
80     "heat_rejection_electricity",
81     "humidifier_electricity",
82     "heat_recovery_electricity",
83 )
84
85 DETAILED_HVAC_METER_KWH_COLUMNS = {
86     alias: f"{alias}_kwh"
87     for alias in DETAILED_HVAC_METER_COLUMNS
88 }
89
90 FALLBACK_ELECTRICITY_METER_COLUMNS = (
91     "total_electricity_facility",
92     "total_electricity_building",
93 )
94
95 _COLUMN_ALIASES = {
96     HVAC_POWER_COLUMN: (
97         HVAC_POWER_COLUMN,
98         "HVAC_elèctricity_demand_rate",
99         "hvac_power",
100        "HVAC_power",
101        "hvac_load",
102    ),
103    ENERGY_COST_COLUMN: (
104        ENERGY_COST_COLUMN,
105        "electricity_cost",
106        "elèctricity_cost",
107    ),

```

```

108 }
109
110 _COLUMN_ALIASES.update(
111     {
112         canonical: (canonical, *aliases)
113         for canonical, aliases in DETAILED_HVAC_METER_COLUMNS.items()
114     }
115 )
116
117
118 def find_first_available_column(
119     columns: Iterable[str], candidates: Iterable[str]
120 ) -> str | None:
121     """Cerqueu la primera columna disponible."""
122     available = set(columns)
123     for candidate in candidates:
124         if candidate in available:
125             return candidate
126     return None
127
128
129 def normalize_observation_columns(obs: pd.DataFrame) -> pd.DataFrame:
130     """Normalitza les columnes d'observació."""
131     normalized = obs.copy()
132     for canonical, aliases in _COLUMN_ALIASES.items():
133         if canonical in normalized.columns:
134             continue
135         alias = find_first_available_column(normalized.columns, aliases)
136         if alias and alias != canonical:
137             normalized[canonical] = normalized[alias]
138     return normalized
139
140
141 def find_hvac_consumption_source(columns: Iterable[str]) -> tuple[str, str] | None:
142     """Retorna la millor font de consum HVAC i el seu tipus d'unitat.
143
144     EnergyPlus els comptadors informen l'energia per pas de temps en Joules, mentre que el
145     La variable de sortida de la taxa de demanda informa de la potència en watts.
146     """
147     available = set(columns)
148     lower_lookup = {column.lower(): column for column in columns}
149     for column in HVAC_ENERGY_METER_COLUMNS:
150         if column in available:
151             return column, "joule"
152         actual_column = lower_lookup.get(column.lower())
153         if actual_column is not None:
154             return actual_column, "joule"
155     if HVAC_POWER_COLUMN in available:
156         return HVAC_POWER_COLUMN, "watt"
157     actual_power_column = lower_lookup.get(HVAC_POWER_COLUMN.lower())
158     if actual_power_column is not None:
159         return actual_power_column, "watt"
160     for column in FALLBACK_ELECTRICITY_METER_COLUMNS:
161         if column in available:
162             return column, "joule"
163         actual_column = lower_lookup.get(column.lower())
164         if actual_column is not None:
165             return actual_column, "joule"
166     return None
167
168
169 def convert_hvac_source_to_kwh(
170     values: pd.Series,
171     unit_kind: str,
172     timestep_hours: float | None = None,
173 ) -> pd.Series:
174     """Convertir la font d'HVAC a kwh."""
175     numeric = pd.to_numeric(values, errors="coerce")
176     if unit_kind == "joule":
177         return numeric / JOULES_PER_KWH
178     if unit_kind == "watt":
179         hours = 0.0 if timestep_hours is None else float(timestep_hours)
180         return (numeric / 1000.0) * hours
181     return numeric
182
183
184 def add_meter_kwh_columns(obs: pd.DataFrame) -> pd.DataFrame:
185     """Afegeix columnes derivades de kWh per a les sortides conegudes del comptador EnergyPlus en Joules."""
186     enriched = obs.copy()
187     for meter_alias, kwh_column in DETAILED_HVAC_METER_KWH_COLUMNS.items():
188         if meter_alias not in enriched.columns or kwh_column in enriched.columns:
189             continue
190         enriched[kwh_column] = convert_hvac_source_to_kwh(
191             enriched[meter_alias],
192             "joule",
193         )
194     return enriched

```

F.18 backend/grafics/style.py

Listing 72: backend/grafics/style.py

```
1  """Estil Plotly compartit per a les figures del panell BEMS-RL Studio."""
2
3  from __future__ import annotations
4
5  import numpy as np
6  import plotly.graph_objects as go
7
8
9  # Paleta i format comú de les figures.
10 STUDIO_CHART_COLORS = {
11     "heating": "#c81e1e",
12     "heating_ref": "#ef4444",
13     "cooling": "#1d4ed8",
14     "cooling_ref": "#38bdf8",
15     "consumption": "#f59e0b",
16     "consumption_muted": "#fbbf24",
17     "consumption_edge": "#b45309",
18     "violation": "#dc2626",
19     "violation_muted": "#f87171",
20     "violation_edge": "#991b1b",
21     "indoor": "#0f766e",
22     "outdoor": "#64748b",
23     "comfort_ok": "#16a34a",
24     "comfort_cold": "#2563eb",
25     "comfort_hot": "#dc2626",
26     "battery": "#7c3aed",
27     "battery_charge": "#15803d",
28     "battery_discharge": "#ea580c",
29     "grid_purchase": "#ca8a04",
30     "price": "#0891b2",
31     "reward": "#6d28d9",
32     "neutral": "#475569",
33     "neutral_soft": "#cbd5e1",
34     "grid": "#d7deeb",
35     "paper": "#ffffff",
36     "plot": "#f8fbff",
37     "text": "#17233c",
38 }
39
40 STUDIO_HEATMAP_SCALES = {
41     "consumption": "YlOrBr",
42     "violation": "Reds",
43     "temperature": "RdYlBu_r",
44 }
45
46
47 def pick_semantic_trace_color(
48     trace_name: str | None,
49     *,
50     reference: bool = False,
51     fallback: str | None = None,
52 ) -> str:
53     """Retorna un color semàntic segons el nom de la traça."""
54     label = (trace_name or "").lower()
55
56     # Les figures arriben de mòduls diferents i no sempre comparteixen palette. Fem servir
57     # paraules del nom de la traça per mantenir colors coherents sense acoblar els gràfics.
58     if "heating" in label or "htg" in label:
59         return STUDIO_CHART_COLORS["heating_ref" if reference else "heating"]
60     if "cooling" in label or "clg" in label:
61         return STUDIO_CHART_COLORS["cooling_ref" if reference else "cooling"]
62     if "viol" in label:
63         return STUDIO_CHART_COLORS["violation_muted" if reference else "violation"]
64     if "massa fred" in label or "cold" in label:
65         return STUDIO_CHART_COLORS["comfort_cold"]
66     if "massa calor" in label or "hot" in label:
67         return STUDIO_CHART_COLORS["comfort_hot"]
68     if "comfort" in label or "comfort" in label:
69         return STUDIO_CHART_COLORS["comfort_ok"]
70     if "hvac" in label or "consum" in label or "demand rate" in label or "mean power" in label:
71         return STUDIO_CHART_COLORS["consumption_muted" if reference else "consumption"]
72     if "price" in label or "preu" in label or "eur/kwh" in label:
73         return STUDIO_CHART_COLORS["price"]
74     if "reward" in label:
75         return STUDIO_CHART_COLORS["reward"]
76     if "outdoor" in label:
77         return STUDIO_CHART_COLORS["outdoor"]
78     if "radiant operation" in label or "radiant availability" in label:
79         return STUDIO_CHART_COLORS["battery"]
80     if "radiant inlet" in label:
81         return STUDIO_CHART_COLORS["price"]
82     if "radiant outlet" in label or "radiant water" in label:
83         return STUDIO_CHART_COLORS["battery_discharge"]
84     if "discharge" in label:
85         return STUDIO_CHART_COLORS["battery_discharge"]
86     if "charge" in label:
87         return STUDIO_CHART_COLORS["battery_charge"]
```

```

88     if "battery" in label or "stored energy" in label or "soc" in label or "bateria" in label:
89         return STUDIO_CHART_COLORS["battery_charge"]
90     if "xarxa" in label or "compra" in label or "grid purchase" in label:
91         return STUDIO_CHART_COLORS["grid_purchase"]
92     if "indoor" in label:
93         return STUDIO_CHART_COLORS["indoor"]
94
95     return fallback or STUDIO_CHART_COLORS["neutral"]
96
97
98 def _alpha_color(hex_color: str, alpha: float = 0.30) -> str:
99     """Converteix un color hex a rgba amb transparència."""
100     h = hex_color.lstrip("#")
101     r, g, b = int(h[0:2], 16), int(h[2:4], 16), int(h[4:6], 16)
102     return f"rgba({r},{g},{b},{alpha})"
103
104 def _apply_figure_theme(fig: go.Figure, *, heatmap: bool = False) -> go.Figure:
105     """Aplica un estil més contrastat i homogeni a les figures."""
106     fig.update_layout(
107         template="plotly_white",
108         paper_bgcolor=STUDIO_CHART_COLORS["paper"],
109         plot_bgcolor=STUDIO_CHART_COLORS["plot"],
110         font=dict(color=STUDIO_CHART_COLORS["text"], size=14),
111         title=dict(
112             x=0.02,
113             xanchor="left",
114             font=dict(size=20, color=STUDIO_CHART_COLORS["text"]),
115         ),
116         legend=dict(
117             bgcolor="rgba(255,255,255,0.9)",
118             bordercolor=STUDIO_CHART_COLORS["grid"],
119             borderwidth=1,
120         ),
121         hoverlabel=dict(
122             bgcolor="ffffff",
123             font_color=STUDIO_CHART_COLORS["text"],
124         ),
125         margin=dict(l=60, r=28, t=72, b=56),
126     )
127     fig.update_xaxes(
128         showgrid=not heatmap,
129         gridcolor=STUDIO_CHART_COLORS["grid"],
130         linecolor=STUDIO_CHART_COLORS["grid"],
131         ticks="outside",
132         zeroline=False,
133     )
134     fig.update_yaxes(
135         showgrid=not heatmap,
136         gridcolor=STUDIO_CHART_COLORS["grid"],
137         linecolor=STUDIO_CHART_COLORS["grid"],
138         ticks="outside",
139         zeroline=False,
140     )
141     return fig
142
143
144 def style_figure_semantics(fig: go.Figure) -> go.Figure:
145     """Recolora traces segons la seva semàntica i aplica el tema comú."""
146     heatmap_like = False
147
148     for trace in getattr(fig, "data", []):
149         trace_type = getattr(trace, "type", "")
150         name = getattr(trace, "name", None) or ""
151         base_name = name.replace(" (ref)", "")
152         is_reference = "(ref)" in name.lower()
153         if getattr(trace, "legendgroup", None) == "agent_actions":
154             continue
155         if getattr(trace, "meta", None) == "keep_color":
156             continue
157         # Algunes traces ja porten colors especials (per exemple heatmaps o accions).
158         # Les altres es recoloren aquí perquè el dashboard sembli una sola peça.
159         color = pick_semantic_trace_color(base_name, reference=is_reference)
160
161         if trace_type in ("scatter", "scattergl"):
162             line = getattr(trace, "line", None)
163             width = max(float(getattr(line, "width", 0) or 0), 2.8 if is_reference else 2.6)
164             marker = getattr(trace, "marker", None)
165             marker_color = getattr(marker, "color", None)
166             trace.line = dict(
167                 color=color,
168                 width=width,
169             )
170             if "markers" in str(getattr(trace, "mode", "")):
171                 trace.marker = dict(
172                     color=marker_color if isinstance(marker_color, (list, tuple, np.ndarray)) and not is_reference else color,
173                     size=max(int(getattr(marker, "size", 0) or 0), 6),
174                 )
175             if is_reference:
176                 trace.opacity = max(float(getattr(trace, "opacity", 1.0) or 1.0), 0.92)
177
178         elif trace_type == "bar":
179             marker = getattr(trace, "marker", None)

```

```

180     marker_color = getattr(marker, "color", None)
181     edge_color = (
182         STUDIO_CHART_COLORS["violation_edge"]
183         if color in (STUDIO_CHART_COLORS["violation"], STUDIO_CHART_COLORS["violation_muted"])
184         else STUDIO_CHART_COLORS["consumption_edge"]
185     )
186     trace.marker = dict(
187         color=marker_color if isinstance(marker_color, (list, tuple, np.ndarray)) and not is_reference else color,
188         line=dict(color=edge_color, width=0.8),
189     )
190     if is_reference:
191         trace.opacity = min(float(getattr(trace, "opacity", 1.0) or 1.0), 0.58)
192
193     elif trace_type == "heatmap":
194         heatmap_like = True
195         title_text = str(getattr(getattr(fig.layout, "title", None), "text", "") or "").lower()
196         if "viol" in title_text:
197             trace.colorscalescale = STUDIO_HEATMAP_SCALES["violation"]
198         elif "consum" in title_text or "hvac" in title_text:
199             trace.colorscalescale = STUDIO_HEATMAP_SCALES["consumption"]
200         else:
201             trace.colorscalescale = STUDIO_HEATMAP_SCALES["temperature"]
202
203     return _apply_figure_theme(fig, heatmap=heatmap_like)

```

F.19 backend/grafics/thermal.py

Listing 73: backend/grafics/thermal.py

```

1  """Dades d'entrada de confort tèrmic: temperatura, humitat i consignes."""
2
3  from __future__ import annotations
4
5  import pandas as pd
6  import plotly.graph_objects as go
7
8  from backend.grafics.column_utils import _ensure_air_temperature
9  from backend.grafics.style import pick_semantic_trace_color
10 from backend.grafics.time_utils import (
11     _canon_day_series,
12     _ensure_datetime_index,
13     _get_outdoor_temperature_series,
14     _hour_series,
15     _month_series,
16     _raw_axis_data,
17     filter_by_season,
18 )
19
20
21 def make_indoor_temperature_plot(obs: pd.DataFrame, mode: str, season: str) -> go.Figure:
22     """Renderitza la temperatura interior com a barres (amb una banda d'interval min-max) i la temperatura exterior com una línia amb
23     marcadors."""
24     base = _ensure_datetime_index(_ensure_air_temperature(obs))
25     out_series = _get_outdoor_temperature_series(base)
26
27     _C_BAR = "rgba(79, 142, 247, 0.65)" # blau EPW (igual que Humitat)
28     _C_OUT = "#f26b5b" # salmó EPW (igual que Temp. seca)
29     _C_OUT_EDGE = "#f26b5b"
30
31     fig = go.Figure()
32     if "air_temperature" not in base.columns:
33         return go.Figure()
34
35     if mode in ("hour", "month"):
36         # Hora i mes són agregats; per això fem barres de mitjana interior i línia exterior
37         # amb el mateix eix temporal.
38         if mode == "hour":
39             df = filter_by_season(base, season)
40             grp = _hour_series(df)
41             x_vals = list(range(24))
42             x_title, tick_cfg = "Hour of Day", dict(tickmode="linear", dtick=1)
43         else:
44             df = base if season == "All" else filter_by_season(base, season)
45             grp = _month_series(df)
46             x_vals = list(range(1, 13))
47             x_title, tick_cfg = "Month", dict(tickmode="linear", dtick=1)
48
49     g_mean = df.groupby(grp)["air_temperature"].mean()
50     y_in = [g_mean.get(v, float("nan")) for v in x_vals]
51
52     fig.add_trace(go.Bar(
53         x=x_vals, y=y_in, name="Indoor Temp",
54         marker_color=_C_BAR,
55         meta="keep_color",
56         hovertemplate="%{x}<br>Mitjana: %{y:.1f}°C<extra></extra>",
57     ))
58     if out_series is not None:
59         g_out = df.groupby(grp)[out_series.name].mean()

```

```

59     y_out = [g_out.get(v, float("nan")) for v in x_vals]
60     fig.add_trace(go.Scatter(
61         x=x_vals, y=y_out, mode="lines+markers", name="Outdoor Temp",
62         line=dict(color=_C_OUT, width=2.5),
63         marker=dict(color=_C_OUT, size=7, symbol="circle",
64             line=dict(color="white", width=1.5)),
65         meta="keep_color",
66         hovertemplate="%{x}<br>{%y:.1f}°C<extra></extra>",
67     ))
68     fig.update_xaxes(title=x_title, **tick_cfg)
69     fig.update_layout(bargap=0.22)
70
71 elif mode == "day":
72     # La vista diària conserva la forma anual però redueix el soroll de cada timestep.
73     df = filter_by_season(base, season)
74     x_in, y_in, hover_in = _canon_day_series(df, "air_temperature")
75     fig.add_trace(go.Scatter(
76         x=x_in, y=y_in, mode="lines", name="Indoor Temp",
77         line=dict(color="rgba(79, 142, 247, 0.9)", width=2),
78         meta="keep_color",
79         hovertext=hover_in,
80         hovertemplate="%{hovertext}<br>{%y:.1f}°C<extra></extra>",
81     ))
82     if out_series is not None:
83         x_out, y_out, hover_out = _canon_day_series(
84             df.rename(columns={out_series.name: "_out"}), "_out"
85         )
86         fig.add_trace(go.Scatter(
87             x=x_out, y=y_out, mode="lines", name="Outdoor Temp",
88             line=dict(color=_C_OUT, width=2),
89             meta="keep_color",
90             hovertext=hover_out,
91             hovertemplate="%{hovertext}<br>{%y:.1f}°C<extra></extra>",
92         ))
93     fig.update_xaxes(title="Day (1..365)", tickmode="linear", tick0=1, dtick=10)
94
95 elif mode == "raw":
96     # Raw manté el pas original de simulació, útil per revisar pics puntuals.
97     df = filter_by_season(base, season)
98     x_vals, y_in, hover_in = _raw_axis_data(df, "air_temperature")
99     fig.add_trace(go.Scatter(
100         x=x_vals, y=y_in, mode="lines", name="Indoor Temp",
101         line=dict(color="rgba(79, 142, 247, 0.9)", width=2.2, shape="spline"),
102         meta="keep_color",
103         hovertext=hover_in,
104         hovertemplate="%{hovertext}<br>{%y:.1f}°C<extra></extra>",
105     ))
106     if out_series is not None:
107         x_v2, y_out, hover_out = _raw_axis_data(
108             df.rename(columns={out_series.name: "_out"}), "_out"
109         )
110         fig.add_trace(go.Scatter(
111             x=x_v2, y=y_out, mode="lines", name="Outdoor Temp",
112             line=dict(color=_C_OUT, width=1.8, shape="spline"),
113             meta="keep_color",
114             hovertext=hover_out,
115             hovertemplate="%{hovertext}<br>{%y:.1f}°C<extra></extra>",
116         ))
117     fig.update_xaxes(title="Time (step)")
118
119 fig.update_yaxes(title="Temperature (°C)")
120 fig.update_layout(title=f"Indoor vs Outdoor Temperature - {mode.capitalize()} ({season})")
121 return fig
122
123
124 # Humitat interior amb humitat exterior de referència.
125 def make_indoor_humidity_plot(obs: pd.DataFrame, mode: str, season: str) -> go.Figure:
126     """Renderitza la humitat interior com a barres/linies i la humitat exterior com una línia amb marcadors."""
127     base = _ensure_datetime_index(obs.copy())
128     out_series = base["outdoor_humidity"] if "outdoor_humidity" in base.columns else None
129
130     _C_IN_LINE = "rgba(0, 200, 255, 1.0)"
131     _C_OUT_BAR = "rgba(15, 45, 120, 0.75)"
132
133     fig = go.Figure()
134     if "air_humidity" not in base.columns:
135         return go.Figure()
136
137     if mode in ("hour", "month"):
138         # Igual que amb temperatura: agregats per hora/mes, amb humitat interior com a
139         # línia principal i exterior com a referència.
140         if mode == "hour":
141             df = filter_by_season(base, season)
142             grp = _hour_series(df)
143             x_vals = list(range(24))
144             x_title, tick_cfg = "Hour of Day", dict(tickmode="linear", dtick=1)
145         else:
146             df = base if season == "All" else filter_by_season(base, season)
147             grp = _month_series(df)
148             x_vals = list(range(1, 13))
149             x_title, tick_cfg = "Month", dict(tickmode="linear", dtick=1)

```

```

151 g_mean = df.groupby(grp)["air_humidity"].mean()
152 y_in = [g_mean.get(v, float("nan")) for v in x_vals]
153
154 fig.add_trace(go.Scatter(
155     x=x_vals, y=y_in, mode="lines+markers", name="Indoor Humidity",
156     line=dict(color=_C_IN_LINE, width=2.5),
157     marker=dict(color=_C_IN_LINE, size=7, symbol="circle",
158         line=dict(color="white", width=1.5)),
159     meta="keep_color",
160     hovertemplate="%{x}<br>Mitjana: %{y:.1f}%<extra></extra>",
161 ))
162 if out_series is not None:
163     g_out = df.groupby(grp)[out_series.name].mean()
164     y_out = [g_out.get(v, float("nan")) for v in x_vals]
165     fig.add_trace(go.Bar(
166         x=x_vals, y=y_out, name="Outdoor Humidity",
167         marker_color=_C_OUT_BAR,
168         meta="keep_color",
169         hovertemplate="%{x}<br>%{y:.1f}%<extra></extra>",
170     ))
171 fig.update_xaxes(title=x_title, **tick_cfg)
172 fig.update_layout(bargap=0.22)
173
174 elif mode == "day":
175     df = filter_by_season(base, season)
176     x_in, y_in, hover_in = _canon_day_series(df, "air_humidity")
177     fig.add_trace(go.Scatter(
178         x=x_in, y=y_in, mode="lines", name="Indoor Humidity",
179         line=dict(color=_C_IN_LINE, width=2),
180         meta="keep_color",
181         hovertext=hover_in,
182         hovertemplate="%{hovertext}<br>%{y:.1f}%<extra></extra>",
183     ))
184     if out_series is not None:
185         x_out, y_out, hover_out = _canon_day_series(
186             df.rename(columns={out_series.name: "_out"}), "_out"
187         )
188         fig.add_trace(go.Scatter(
189             x=x_out, y=y_out, mode="lines", name="Outdoor Humidity",
190             line=dict(color=_C_OUT_BAR, width=2),
191             meta="keep_color",
192             hovertext=hover_out,
193             hovertemplate="%{hovertext}<br>%{y:.1f}%<extra></extra>",
194         ))
195     fig.update_xaxes(title="Day (1..365)", tickmode="linear", tick0=1, dtick=10)
196
197 elif mode == "raw":
198     # Raw és útil per detectar pics d'humitat que quedarien amagats en la mitjana.
199     df = filter_by_season(base, season)
200     x_vals, y_in, hover_in = _raw_axis_data(df, "air_humidity")
201     fig.add_trace(go.Scatter(
202         x=x_vals, y=y_in, mode="lines", name="Indoor Humidity",
203         line=dict(color=_C_IN_LINE, width=2.2, shape="spline"),
204         meta="keep_color",
205         hovertext=hover_in,
206         hovertemplate="%{hovertext}<br>%{y:.1f}%<extra></extra>",
207     ))
208     if out_series is not None:
209         x_v2, y_out, hover_out = _raw_axis_data(
210             df.rename(columns={out_series.name: "_out"}), "_out"
211         )
212         fig.add_trace(go.Scatter(
213             x=x_v2, y=y_out, mode="lines", name="Outdoor Humidity",
214             line=dict(color=_C_OUT_BAR, width=1.8, shape="spline"),
215             meta="keep_color",
216             hovertext=hover_out,
217             hovertemplate="%{hovertext}<br>%{y:.1f}%<extra></extra>",
218         ))
219     fig.update_xaxes(title="Time (step)")
220
221 fig.update_yaxes(title="Humidity (%)")
222 fig.update_layout(title=f"Indoor vs Outdoor Humidity - {mode.capitalize()} ({season})")
223 return fig
224
225
226 # Temperatures de consigna de calefacció i refrigeració.
227 def _build_setpoint_trace(
228     base: pd.DataFrame,
229     series_name: str,
230     label: str,
231     line_style: dict,
232     mode: str,
233     season: str,
234 ) -> go.Scatter | None:
235     """Crea una traça relacionada amb el punt de consigna per al mode d'agregació seleccionat."""
236     if series_name not in base.columns:
237         return None
238     if mode == "hour":
239         df = filter_by_season(base, season)
240         hour_values = _hour_series(df)
241         grouped = df.groupby(hour_values)[series_name].mean()
242         x_values = list(range(24))

```



```

243     y_values = [grouped.get(hour, float("nan")) for hour in x_values]
244     return go.Scatter(
245         x=x_values,
246         y=y_values,
247         mode="lines+markers",
248         name=label,
249         line=line_style,
250         marker=dict(color=line_style["color"], size=6),
251     )
252 if mode == "day":
253     df = filter_by_season(base, season)
254     x_values, y_values, hover = _canon_day_series(df, series_name)
255     return go.Scatter(
256         x=x_values,
257         y=y_values,
258         mode="lines",
259         name=label,
260         line=line_style,
261         hovertext=hover,
262         hovertemplate="%{hovertext}<br>{%y:.2f}°C<extra></extra>",
263     )
264 if mode == "month":
265     df = base if season == "All" else filter_by_season(base, season)
266     month_values = _month_series(df)
267     grouped = df.groupby(month_values)[series_name].mean()
268     x_values = list(range(1, 13))
269     y_values = [grouped.get(month, float("nan")) for month in x_values]
270     return go.Scatter(
271         x=x_values,
272         y=y_values,
273         mode="lines+markers",
274         name=label,
275         line=line_style,
276         marker=dict(color=line_style["color"], size=6),
277     )
278 if mode == "raw":
279     df = filter_by_season(base, season)
280     x_values, y_values, hover = _raw_axis_data(df, series_name)
281     return go.Scatter(
282         x=x_values,
283         y=y_values,
284         mode="lines",
285         name=label,
286         line=line_style,
287         hovertext=hover,
288         hovertemplate="%{hovertext}<br>{%y:.2f}°C<extra></extra>",
289     )
290 return None
291
292
293 def make_setpoints_plot(obs: pd.DataFrame, mode: str, season: str) -> go.Figure:
294     """Crea una figura Plotly per als punts de consigna."""
295     base = _ensure_datetime_index(obs).copy()
296
297     # En multizona podem tenir diverses columnes de consigna. Per al gràfic global fem
298     # una mitjana simple i mantenim les figures per zona per al detall.
299     htg_cols = [c for c in base.columns if "htg_setpoint" in c.lower()]
300     clg_cols = [c for c in base.columns if "clg_setpoint" in c.lower()]
301
302     if not htg_cols and not clg_cols:
303         return go.Figure()
304
305     if htg_cols and "htg_setpoint" not in base.columns:
306         base["htg_setpoint"] = base[htg_cols].mean(axis=1)
307     if clg_cols and "clg_setpoint" not in base.columns:
308         base["clg_setpoint"] = base[clg_cols].mean(axis=1)
309
310     fig = go.Figure()
311
312     htg_color = pick_semantic_trace_color("Heating Setpoint")
313     clg_color = pick_semantic_trace_color("Cooling Setpoint")
314     htg_style = dict(color=htg_color, width=3.2, shape="hv" if mode in ("hour", "raw") else "linear")
315     clg_style = dict(color=clg_color, width=3.2, shape="hv" if mode in ("hour", "raw") else "linear")
316     tr1 = _build_setpoint_trace(base, "htg_setpoint", "Heating Setpoint", htg_style, mode, season)
317     tr2 = _build_setpoint_trace(base, "clg_setpoint", "Cooling Setpoint", clg_style, mode, season)
318     if tr1: fig.add_trace(tr1)
319     if tr2: fig.add_trace(tr2)
320
321     if mode == "hour":
322         fig.update_xaxes(title="Hour of Day", tickmode="linear", dtick=1)
323     elif mode == "month":
324         fig.update_xaxes(title="Month", tickmode="linear", dtick=1)
325     elif mode == "day":
326         fig.update_xaxes(title="Day (1..365)", tickmode="linear", tick0=1, dtick=10)
327     else:
328         fig.update_xaxes(title="Time (step)")
329
330     fig.update_yaxes(title="Temperature (°C)")
331     fig.update_layout(title=f"Setpoints - {mode.capitalize()} ({season})")
332     return fig
333
334

```

```

335 # Comparació entre consignes i temperatura interior.
336 def make_setpoints_vs_indoor_plot(obs: pd.DataFrame, mode: str, season: str) -> go.Figure:
337     """Crea una figura Plotly per a punts de consigna vs interior."""
338     base = _ensure_datetime_index(obs).copy()
339
340     # Reutilitzem la mateixa normalització que el gràfic de setpoints perquè la comparació
341     # amb temperatura interior no canviï segons el nombre de zones.
342     htg_cols = [c for c in base.columns if "htg_setpoint" in c.lower()]
343     clg_cols = [c for c in base.columns if "clg_setpoint" in c.lower()]
344
345     if not htg_cols and not clg_cols and "air_temperature" not in base.columns:
346         return go.Figure()
347
348     if htg_cols and "htg_setpoint" not in base.columns:
349         base["htg_setpoint"] = base[htg_cols].mean(axis=1)
350     if clg_cols and "clg_setpoint" not in base.columns:
351         base["clg_setpoint"] = base[clg_cols].mean(axis=1)
352
353     fig = go.Figure()
354
355     htg_color = pick_semantic_trace_color("Heating Setpoint")
356     clg_color = pick_semantic_trace_color("Cooling Setpoint")
357     ind_color = pick_semantic_trace_color("indoor")
358
359     htg_style = dict(color=htg_color, width=3.2, shape="hv" if mode in ("hour", "raw") else "linear")
360     clg_style = dict(color=clg_color, width=3.2, shape="hv" if mode in ("hour", "raw") else "linear")
361     ind_style = dict(color=ind_color, width=2.0, shape="spline" if mode == "raw" else "linear")
362
363     tr1 = _build_setpoint_trace(base, "htg_setpoint", "Heating Setpoint", htg_style, mode, season)
364     tr2 = _build_setpoint_trace(base, "clg_setpoint", "Cooling Setpoint", clg_style, mode, season)
365     tr3 = _build_setpoint_trace(base, "air_temperature", "Indoor Temperature", ind_style, mode, season)
366
367     if tr1: fig.add_trace(tr1)
368     if tr2: fig.add_trace(tr2)
369     if tr3: fig.add_trace(tr3)
370
371     if mode == "hour":
372         fig.update_xaxes(title="Hour of Day", tickmode="linear", dtick=1)
373     elif mode == "month":
374         fig.update_xaxes(title="Month", tickmode="linear", dtick=1)
375     elif mode == "day":
376         fig.update_xaxes(title="Day (1..365)", tickmode="linear", tick0=1, dtick=10)
377     else:
378         fig.update_xaxes(title="Time (step)")
379
380     fig.update_yaxes(title="Temperature (°C)")
381     fig.update_layout(title=f"Setpoints vs Indoor Temperature - {mode.capitalize()} ({season})")
382     return fig

```

F.20 backend/grafics/time_utils.py

Listing 74: backend/grafics/time_utils.py

```

1  """Filtrat temporal i ajuda d'eix per a figures del panell."""
2
3  from __future__ import annotations
4
5  import numpy as np
6  import pandas as pd
7
8  from backend.grafics.style import _alpha_color
9
10
11 def _bounded_int_series(
12     df: pd.DataFrame,
13     column: str,
14     lower: int,
15     upper: int,
16 ) -> pd.Series:
17     """Retorna una sèrie entera anul·lable amb valors temporals impossibles eliminats."""
18
19     values = pd.to_numeric(df[column], errors="coerce").round()
20     values = values.where(values.between(lower, upper))
21     return values.astype("Int64")
22
23
24 def _coerce_temporal_parts(df: pd.DataFrame) -> tuple[pd.Series, pd.Series, pd.Series]:
25     """Retorna les columnes mes/dia/hora netejades per a la reconstrucció de data i hora."""
26
27     month = _bounded_int_series(df, "month", 1, 12)
28     day = _bounded_int_series(df, "day_of_month", 1, 31)
29     hour = _bounded_int_series(df, "hour", 0, 23)
30     return month, day, hour
31
32
33 def sanitize_observation_time_columns(df: pd.DataFrame) -> pd.DataFrame:
34     """Converteix les columnes d'observació temporal i emmascarar valors físicament impossibles."""
35

```

```

36 result = df.copy()
37 bounds = {
38     "month": (1, 12),
39     "day_of_month": (1, 31),
40     "hour": (0, 23),
41 }
42 for column, (lower, upper) in bounds.items():
43     if column in result.columns:
44         result[column] = _bounded_int_series(result, column, lower, upper)
45 return result
46
47
48 def _seasonal_marker_colors(
49     x_vals: list[int],
50     season: str,
51     *,
52     active_color: str,
53     muted_color: str | None = None,
54 ) -> list[str]:
55     """Seasonal marker colors."""
56     months_sel = _season_months(season)
57     if not months_sel:
58         return [active_color] * len(x_vals)
59     dim = muted_color if muted_color is not None else _alpha_color(active_color, 0.30)
60     return [active_color if month in months_sel else dim for month in x_vals]
61
62 def _season_months(name: str):
63     """Season months."""
64     seasons = {
65         "Winter": [12, 1, 2],
66         "Spring": [3, 4, 5],
67         "Summer": [6, 7, 8],
68         "Autumn": [9, 10, 11],
69     }
70     return seasons.get(name, None) # None -> Tot
71
72
73 def _ensure_datetime_index(df: pd.DataFrame) -> pd.DataFrame:
74     """
75     Garanteix un DatetimeIndex per reagrupar per hores/dies/mesos.
76     Si no hi ha 'datetime' ni 'timestamp', reconstrueix un índex sintètic
77     coherent amb els timesteps de 15 minuts del projecte.
78     """
79     dfc = df.copy()
80     if "datetime" in dfc.columns:
81         dfc["datetime"] = pd.to_datetime(dfc["datetime"], errors="coerce")
82         dfc = dfc.set_index("datetime").sort_index()
83         dfc = dfc[-dfc.index.isna()]
84         return dfc
85     if "timestamp" in dfc.columns:
86         dfc["datetime"] = pd.to_datetime(dfc["timestamp"], errors="coerce")
87         dfc = dfc.set_index("datetime").sort_index()
88         dfc = dfc[-dfc.index.isna()]
89         return dfc
90
91     if all(col in dfc.columns for col in ("month", "day_of_month", "hour")):
92         month, day, hour = _coerce_temporal_parts(dfc)
93
94         # Ancorem la sèrie en el primer registre complet: així respectem simulacions que no
95         # comencen exactament a les 00:00.
96         if "time_elapsed(hours)" in dfc.columns:
97             elapsed = pd.to_numeric(dfc["time_elapsed(hours)"], errors="coerce")
98             valid = elapsed.notna() & month.notna() & day.notna() & hour.notna()
99             if valid.any():
100                 first_idx = valid[valid].index[0]
101                 first_pos = dfc.index.get_loc(first_idx)
102                 first_elapsed = float(elapsed.iloc[first_pos])
103                 base_minutes = int(round((first_elapsed % 1.0) * 60.0)) % 60
104                 base_ts = pd.Timestamp(
105                     year=2001,
106                     month=int(month.iloc[first_pos]),
107                     day=int(day.iloc[first_pos]),
108                     hour=int(hour.iloc[first_pos]),
109                     minute=base_minutes,
110                 )
111                 offset_minutes = ((elapsed - first_elapsed) * 60.0).round()
112                 try:
113                     dt = base_ts + pd.to_timedelta(offset_minutes, unit="m")
114                     dfc.index = pd.DatetimeIndex(dt)
115                     dfc = dfc[-dfc.index.isna()]
116                     return dfc.sort_index(kind="mergesort")
117                 except (OverflowError, ValueError):
118                     pass
119
120         hour_blocks = (
121             pd.DataFrame({"month": month, "day": day, "hour": hour})
122             .ne(pd.DataFrame({"month": month, "day": day, "hour": hour}).shift())
123             .any(axis=1)
124             .cumsum()
125         )
126         # Quan no hi ha temps acumulat, repartim cada bloc d'una mateixa hora en quarts
127         # d'hora consecutius, que és el pas habitual dels CSV de Sinergym.

```

```

128     minute_slot = hour_blocks.groupby(hour_blocks).cumcount() * 15
129     dt = pd.to_datetime(
130         {
131             "year": 2001,
132             "month": month.astype("float64"),
133             "day": day.astype("float64"),
134             "hour": hour.astype("float64"),
135             "minute": minute_slot.astype("float64"),
136         },
137         errors="coerce",
138     )
139     dfc.index = pd.DatetimeIndex(dt)
140     dfc = dfc[~dfc.index.isna()]
141     return dfc.sort_index(kind="mergesort")
142
143     # Reserva defensiva: mantenim 15 minuts per pas.
144     dfc.index = pd.date_range("2001-01-01", periods=len(dfc), freq="15min")
145     return dfc.sort_index(kind="mergesort")
146
147 def _month_series(df: pd.DataFrame) -> pd.Series:
148     """
149     Sèrie 'mes' canònica 1..12:
150     - Prioritza columna 'month' si existeix (normalitza possibles 0/13).
151     - Si no, usa el DatetimeIndex.
152     """
153     if "month" in df.columns:
154         s = _bounded_int_series(df, "month", 1, 12)
155         if s.notna().any():
156             return pd.Series(s, index=df.index)
157     if isinstance(df.index, pd.DatetimeIndex):
158         return pd.Series(df.index.month, index=df.index)
159     # amb index sintètic ja és DateTimeIndex
160     return pd.Series(pd.to_datetime(df.index).month, index=df.index)
161
162 def _hour_series(df: pd.DataFrame) -> pd.Series:
163     """
164     Sèrie 'hora' canònica 0..23:
165     - Prioritza columna 'hour' si existeix (normalitza →240).
166     - Si no, usa el DatetimeIndex.
167     """
168     if "hour" in df.columns:
169         h = _bounded_int_series(df, "hour", 0, 23)
170         if h.notna().any():
171             return pd.Series(h, index=df.index)
172     if isinstance(df.index, pd.DatetimeIndex):
173         return pd.Series(df.index.hour, index=df.index)
174     return pd.Series(pd.to_datetime(df.index).hour, index=df.index)
175
176 def filter_by_season(df: pd.DataFrame, season: str) -> pd.DataFrame:
177     """Filtre per estació amb la sèrie de mesos canònica."""
178     months = _season_months(season)
179     if not months:
180         return df
181     mon = _month_series(df)
182     return df[mon.isin(months)]
183
184 def _get_outdoor_temperature_series(df: pd.DataFrame) -> pd.Series | None:
185     """Retorna la columna de temperatura exterior utilitzada per les observacions generades."""
186     for col in ("outdoor_temperature", "outdoor_air_temperature"):
187         if col in df.columns:
188             return pd.to_numeric(df[col], errors="coerce")
189     return None
190
191 def _has_mdh(df: pd.DataFrame) -> bool:
192     """Té columnes month/day_of_month/hour del CSV?"""
193     return all(c in df.columns for c in ("month", "day_of_month", "hour"))
194
195 def _canon_day_series(df: pd.DataFrame, col: str):
196     """
197     Retorna (x, y, hover) per a una sèrie agregada DIÀRIA sense anys:
198     - Si hi ha columnes 'month' i 'day_of_month', fa mean per (MM,DD).
199     - Si només hi ha DatetimeIndex, resample('D') i mitjana per (MM,DD) (uneix anys).
200     Sempre torna UNA observació per MM-DD i X=1..365.
201     """
202     if _has_mdh(df):
203         month, day, _ = _coerce_temporal_parts(df)
204         working = df.assign(month=month, day_of_month=day)
205         g = working.groupby(["month", "day_of_month"])[col].mean().reset_index()
206         g = g.dropna(subset=["month", "day_of_month"])
207         g["order"] = pd.to_datetime(
208             {"year": 2001, "month": g["month"].astype(int), "day": g["day_of_month"].astype(int)},
209             errors="coerce",
210         )
211         g = g.sort_values("order")
212         x = g["order"].dt.dayofyear.tolist()
213         y = g[col].tolist()
214         hover = [f"{int(m):02d}-{int(d):02d}" for m, d in zip(g["month"], g["day_of_month"])]

```

```

220     return x, y, hover
221
222     if isinstance(df.index, pd.DatetimeIndex):
223         s = df[col].resample("D").mean()
224         g = s.groupby([s.index.month, s.index.day]).mean()
225         months = np.array([i[0] for i in g.index], dtype=int)
226         days = np.array([i[1] for i in g.index], dtype=int)
227         order = pd.to_datetime({"year": 2001, "month": months, "day": days}, errors="coerce")
228         ord_idx = np.argsort(order.values)
229         x = order.dt.dayofyear.to_numpy()[ord_idx].tolist()
230         y = g.to_numpy()[ord_idx].tolist()
231         hover = [f"{int(m):02d}-{int(d):02d}" for m, d in zip(months[ord_idx], days[ord_idx])]
232         return x, y, hover
233
234     # Reserva defensiva (24 registres ~ 1 dia)
235     ids = np.arange(len(df)) // 24 + 1
236     g = pd.Series(df[col].to_numpy()).groupby(ids).mean()
237     x = g.index.tolist()
238     y = g.values.tolist()
239     hover = [f"Day {i}" for i in x]
240     return x, y, hover
241
242
243 def _raw_axis_data(df: pd.DataFrame, col: str):
244     """X=1..N (pas), hover='MM-DD HHh' si es pot."""
245     x = list(range(1, len(df) + 1))
246     if _has_mdh(df):
247         hover = [
248             (
249                 f"{int(m):02d}-{int(d):02d} {int(h):02d}h"
250                 if pd.notna(m) and pd.notna(d) and pd.notna(h)
251                 else f"Step {idx}"
252             )
253             for idx, (m, d, h) in enumerate(
254                 zip(df["month"], df["day_of_month"], df["hour"]),
255                 start=1,
256             )
257         ]
258     elif isinstance(df.index, pd.DatetimeIndex):
259         hover = [f"{ix.month:02d}-{ix.day:02d} {ix.hour:02d}h" for ix in df.index]
260     else:
261         hover = [f"Step {i}" for i in x]
262     y = df[col].to_list()
263     return x, y, hover
264
265 def _auto_scale_series(dfcol: pd.Series):
266     """Retorna sèrie escalada i sufix ('', 'x1e3', 'x1e6', 'x1e9')."""
267     absmax = float(np.nanmax(np.abs(dfcol.to_numpy()))) if len(dfcol) else 0.0
268     if absmax >= 1e9: return dfcol / 1e9, "x1e9"
269     if absmax >= 1e6: return dfcol / 1e6, "x1e6"
270     if absmax >= 1e3: return dfcol / 1e3, "x1e3"
271     return dfcol, ""

```

G Codi: Eines de manteniment

G.1 test_eppy.py

Listing 75: test_eppy.py

```

1  """Sonda manual Eppy per inspeccionar les variables de sortida EnergyPlus.
2
3  Aquest script es manté intencionadament com una utilitat de manteniment executable. Importació
4  no té efectes secundaris; executeu-lo directament quan necessiteu verificar que Eppy
5  pot llegir un fitxer IDF i injectar un objecte ``Output:Variable``.
6  """
7
8  import os
9  import glob
10
11  from eppy.modeleditor import IDF
12
13
14  def main() -> int:
15     """Executa la sonda de fum manual Eppy i imprimeix les variables descobertes."""
16
17     eplus_path = "/usr/local/EnergyPlus-24-1-0"
18     IDF.setidname(os.path.join(eplus_path, "Energy+.idd"))
19     idf_path = glob.glob("/workspaces/sinergym/sinergym/data/**/*.idf", recursive=True)[0]
20     idf = IDF(idf_path)
21
22     out_vars = idf.idfobjects["Output:Variable"].upper()
23     print("Current Output:Variables in IDF:")
24     for variable in out_vars:
25         print(variable)
26
27     print("\nTrying to inject new variable...")

```

```

28     idf.newidfobject(
29         "Output:Variable",
30         Key_Value="*",
31         Variable_Name="Zone Mean Air Temperature",
32         Reporting_Frequency="Hourly",
33     )
34     print(idf.idfobjects["Output:Variable"].upper()[-1])
35     return 0
36
37
38 if __name__ == "__main__":
39     raise SystemExit(main())

```

G.2 test_sql.py

Listing 76: test_sql.py

```

1  """Sonda manual EnergyPlus SQLite.
2
3  Executa aquesta utilitat directament dins d'un contenidor de simulació quan necessiteu inspeccionar-lo
4  la taula ``ReportDataDictionary`` de l'última EnergyPlus SQL temporal
5  sortida. La importació del mòdul és segura i no obre cap fitxer.
6  """
7
8  import sqlite3
9  import glob
10
11  import pandas as pd
12
13
14  def main() -> int:
15      """Imprimiu el diccionari d'informes EnergyPlus des del primer fitxer temporal SQL."""
16
17      sql_path = glob.glob("/tmp/tmp*/eplusout.sql")[0]
18      with sqlite3.connect(sql_path) as conn:
19          print(pd.read_sql_query("SELECT * FROM ReportDataDictionary", conn))
20      return 0
21
22
23 if __name__ == "__main__":
24     raise SystemExit(main())

```